

尹朝庆 尹皓 编著

人工智能与专家系统



中国水利水电出版社
www.waterpub.com.cn

人工智能与专家系统

尹朝庆 尹皓 编著

HH 60/07
01

中国水利水电出版社

内 容 提 要

本书较全面地阐述了人工智能的基本理论、方法和专家系统的构造技术。全书共 8 章,可分为两大部分。第一部分包括第 1 章至第 4 章,主要介绍人工智能的三个基本技术,即知识表示、推理及搜索。第二部分包括第 5 章至第 8 章,其中,第 5 章详细介绍了专家系统的开发方法和技术,包括知识库、推理机、解释器和知识获取,以及人工智能语言 and 专家系统工具;第 6 章阐述了知识的不确定性和不确定推理的有关理论与方法,包括概率推理、可信度推理和模糊推理;第 7 章讨论了机器学习的多种方法及其应用实例;第 8 章介绍了人工神经网络的有关模型、学习算法及其应用实例。

本书具有系统性、新颖性、实用性等特点,可作为计算机、自动化、管理科学与工程等专业的本科生和研究生的教材,也可供有关科技人员参考。

图书在版编目(CIP)数据

人工智能与专家系统 / 尹朝庆, 尹皓编著. —北京: 中国水利水电出版社, 2001

ISBN 7-5084-0890-X

I. 人… II. ①尹… ②尹… III. ①人工智能—高等学校—教材②专家系统—高等学校—教材 IV. TP18

中国版本图书馆 CIP 数据核字 (2001) 第 084641 号

书 名	人工智能与专家系统
作 者	尹朝庆 尹皓 编著
出版、发行	中国水利水电出版社 (北京市三里河路 6 号 100044) 网址: www.waterpub.com.cn E-mail: mchannel@public3.bta.net.cn (万水) sale@waterpub.com.cn 电话: (010) 68359286 (万水) 63202266 (总机) 68331835 (发行部)
经 售	全国各地新华书店
排 版	北京万水电子信息有限公司
印 刷	北京天竺颖华印刷厂印刷
规 格	787×1092 毫米 16 开本 21.5 印张 476 千字
版 次	2002 年 1 月第一版 2002 年 1 月北京第一次印刷
印 数	0001—5000 册
定 价	30.00 元

凡购买我社图书,如有缺页、倒页、脱页的,本社发行部负责调换

版权所有·侵权必究

前 言

人工智能是经过 40 多年发展起来的一门综合性学科,它旨在研究如何利用计算机等现代工具设计模拟人类智能行为的系统。在众多的人工智能应用领域中,专家系统是 30 多年来发展起来的一种最具代表性的智能应用系统,它旨在研究如何设计基于知识的计算机程序系统来模拟人类专家求解专门问题的能力。专家系统是人工智能中最活跃的一个分支,是人工智能发展最主要的推动力。

由于人类对自身的思维规律和智能行为仍在探索中,因此,人工智能与专家系统仍然是一门开放的年轻学科。近几年来,人工智能与专家系统的研究越来越深入,新的思想、新的理论以及新的方法与技术不断涌现,新的研究成果不断充实着这一研究领域,尤其是模糊逻辑与神经网络及其结合的研究已成为当前人工智能或智能模拟的重要研究方向,学术论文数以千计,应用成果迭出。

本书吸收和借鉴了国内外诸多同行的研究成果,并结合作者的研究和教学实践,把人工智能和专家系统有关的理论和方法系统地归纳起来,使之对从事这一领域工作的同行们能起到参考作用。本书可用于高等学校计算机、自动化、管理科学与工程等相关专业的本科生和研究生的教学,本书内容符合全国高校计算机专业教学指导委员会与中国计算机学会教育委员会于 1999 年 9 月发表的《计算机学科教学计划 2000》的 AI 模块的要求。

本书有以下主要特点:

(1) 系统性与新颖性相结合。本书比较系统地介绍了人工智能与专家系统的理论、方法与实现技术,力求反映该领域的学科前沿及最新成果。在介绍经典理论与方法的同时,根据目前国内外人工智能与专家系统发展与应用的情况,对知识的不确定性表示与推理、机器学习与神经网络进行了较多的讨论,介绍了多种方法和实现技术。

(2) 理论性与实用性相结合。本书既注重理论上的论述,又注重结合有关语言与工具的编程介绍实现技术,精选应用实例和习题,理论联系实际。

(3) 对有关概念、原理、方法与技术的阐述力求准确、精练,写作风格上尽量通俗易懂、深入浅出。尽量用实例说明抽象的概念与原理,便于读者理解。

全书共分 8 章,可分为两大部分。第一部分包括第 1 章至第 4 章,主要介绍人工智能的三大基本技术,即知识表示、逻辑推理与搜索技术。第二部分包括第 5 章至第 8 章,讨论专家系统的构造技术。第二部分是第一部分的延伸和深化,涉及到人工智能领域的若干新的理论、方法和技术。第 1 章(绪论)介绍人工智能的发展史、主要学派及研究应用领域;第 2 章(知识表示)介绍了多种知识表示方法;第 3 章(经典逻辑推理)首先介绍各种推理的方式、控制策略、模式匹配等基本概念,然后讨论了归结演绎推理和与/或形演绎推理两种经

典逻辑推理的理论与方法；第4章（搜索策略）在给出问题求解过程的两种形式表示的基础上，分别讨论这两种表示形式的推理过程可用的各种搜索策略；第5章（专家系统）以产生式系统为主，讨论专家系统的知识库、推理机与解释器的实现技术，介绍了专家系统工具CLIPS及其应用，以及知识检测与求精的方法；第6章（知识的不确定性与不确定推理）在讨论知识不确定性的基础上，介绍不确定知识表示和不确定推理的有关理论与推理方法；第7章（机器学习）介绍了机器学习的多种方法及其应用实例；第8章（人工神经网络）介绍了人工神经网络有关模型、学习算法及其应用实例。

本书的第2、4、5章由尹皓编写，第1、3、6、7、8章由尹朝庆编写。全书由尹朝庆统稿。

由于作者水平有限，书中疏漏与错误之处在所难免，恳请广大读者指正。

作者
2001年12月
于武汉理工大学

目 录

前言

第 1 章 绪论	1
1.1 人工智能及其发展	1
1.2 人工智能的研究与应用领域	9
习题一	12
第 2 章 知识表示	13
2.1 一阶谓词逻辑表示法	13
2.1.1 谓词逻辑	13
2.1.2 一阶谓词逻辑表示法的特点	20
2.2 产生式表示法	21
2.2.1 产生式与产生式系统	22
2.2.2 产生式系统的分类及其特点	26
2.3 框架表示法	29
2.3.1 框架与框架网络	29
2.3.2 框架的推理及其特点	35
2.4 语义网络表示法	39
2.4.1 语义网络	39
2.4.2 语义网络的推理及其特点	46
2.5 面向对象表示法	49
2.5.1 面向对象的知识表示	49
2.5.2 面向对象表示法的特点	53
习题二	55
第 3 章 经典逻辑推理	58
3.1 推理的基本概念	58
3.1.1 推理方式及其分类	58
3.1.2 推理的控制策略	60
3.1.3 模式匹配及其变量代换	64
3.2 自然演绎推理	67
3.3 归结演绎推理	68
3.3.1 谓词公式化为子句集的方法	68

3.3.2	海伯伦理论	70
3.3.3	归结原理	72
3.3.4	归结反演	75
3.3.5	基于归结反演的问题求解	76
3.3.6	归结反演策略	79
3.4	与/或形演绎推理	84
3.4.1	与/或形正向演绎推理	84
3.4.2	与/或形逆向演绎推理	87
3.4.3	代换的一致性与剪枝策略	90
习题三		91
第 4 章	搜索策略	95
4.1	问题求解过程的形式表示	95
4.1.1	状态空间表示法	95
4.1.2	与/或树表示法	97
4.2	状态空间的盲目搜索策略	100
4.2.1	宽度优先搜索	100
4.2.2	深度优先搜索	102
4.2.3	有界深度优先搜索	104
4.2.4	代价树的宽度优先搜索	105
4.2.5	代价树的深度优先搜索	107
4.3	状态空间的启发式探索策略	109
4.3.1	估价函数与择优搜索	109
4.3.2	图的有序搜索与 A* 算法	111
4.4	与 / 或树的搜索策略	115
4.4.1	与 / 或树的宽度优先搜索	116
4.4.2	与/或树的有界深度优先搜索	118
4.4.3	与 / 或树的有序搜索	120
4.4.4	博弈树的启发式搜索	124
4.5	搜索性能的量度	127
习题四		128
第 5 章	专家系统	134
5.1	专家系统概述	134
5.1.1	专家系统的类型与特点	134
5.1.2	专家系统的结构与开发方法	137
5.2	LISP 语言	140

5.2.1	LISP 语言的特点与表达式.....	140
5.2.2	LISP 语言的基本函数.....	142
5.2.3	迭代与递归.....	149
5.3	知识库与推理机.....	151
5.3.1	产生式规则与规则库的存储结构.....	151
5.3.2	推理机及其实现.....	155
5.3.3	元知识与元规则.....	167
5.4	解释机制与解释器.....	170
5.4.1	解释的方法.....	170
5.4.2	解释器及其实现.....	174
5.5	知识获取.....	177
5.5.1	知识获取的任务与方式.....	177
5.5.2	知识的检测与求精.....	180
5.5.3	知识的组织与管理.....	187
5.6	专家系统工具.....	189
5.6.1	专家系统工具概述.....	189
5.6.2	专家系统工具 CLIPS 及其应用.....	192
5.7	分布式专家系统与协同式专家系统.....	203
5.7.1	分布式专家系统及其驱动方式.....	203
5.7.2	协同式专家系统及其协同方法.....	205
	习题五.....	209
第 6 章	知识的不确定性与不确定推理.....	211
6.1	知识的不确定性.....	211
6.1.1	证据的不确定性.....	211
6.1.2	规则的不确定性.....	212
6.1.3	推理的不确定性.....	213
6.2	基于概率的不确定推理.....	214
6.2.1	有关概率的基本概念与计算.....	214
6.2.2	基于概率的不确定推理方法.....	216
6.3	基于可信度的不确定推理.....	222
6.3.1	可信度不确定推理方法.....	222
6.3.2	带有阈限的不确定推理.....	226
6.3.3	加权的 uncertain 推理.....	227
6.3.4	可信度 uncertain 推理方法的改进.....	230
6.4	模糊逻辑与模糊推理.....	237

6.4.1	模糊集合的定义与运算	237
6.4.2	模糊知识表示与模糊匹配	247
6.4.3	模糊推理方法	252
6.4.4	带有可信度的模糊推理	264
习题六		266
第7章	机器学习	271
7.1	机器学习的概念与方法分类	271
7.2	归纳学习	272
7.2.1	归纳学习的基本概念	272
7.2.2	基于描述空间的归纳学习方法	274
7.2.3	基于决策树的归纳学习方法	276
7.3	基于解释的学习	282
7.3.1	基于解释的学习框架	283
7.3.2	基于解释的学习过程	284
7.4	遗传算法	286
7.4.1	遗传算法的概念与计算方法	286
7.4.2	遗传算法的应用	291
习题七		294
第8章	人工神经网络	296
8.1	人工神经元与感知器	296
8.1.1	人工神经元模型	296
8.1.2	感知器及其学习算法	298
8.2	人工神经网络模型	300
8.2.1	神经网络的互连结构	300
8.2.2	前向神经网络	301
8.2.3	反馈神经网络	303
8.3	神经网络的学习	306
8.3.1	神经网络的两类学习方法	306
8.3.2	BP 学习算法及其改进方法	307
8.4	人工神经网络的应用	314
8.4.1	神经网络专家系统	314
8.4.2	基于神经网络的模糊分类器	317
8.4.3	神经网络在预测中的应用	324
8.4.4	模糊逻辑与神经网络的结合	328
习题八		331
参考文献		333

第 1 章 绪论

40 多年来, 人工智能 (AI, Artificial Intelligence) 获得很大的发展, 引起众多学科的日益重视, 已成为一门具有广泛应用的交叉学科和前沿学科。

1.1 人工智能及其发展

人工智能是计算机科学、控制论、信息论、神经生理学、语言学等多种学科互相渗透而发展起来的一门学科。人工智能的发展虽然已走过了 40 多年的历程, 但是, 人工智能至今尚无统一的定义。尽管学术界有各种各样的说法和定义, 但就其本质而言, 人工智能是研究、设计和应用智能机器或智能系统, 来模拟人类智能活动的能力、以延伸人类智能的科学。人类智能活动的能力是指人类在认识世界和改造世界的活动中, 经过脑力劳动表现出来的能力。一般地说, 可概括为:

1) 通过视觉、听觉、触觉等感官活动, 接受并理解文字、图像、声音、语言等各种外界信息, 这就是认识和理解外界环境的能力。

2) 通过人脑的生理与心理活动以及有关的信息处理过程, 将感性知识抽象为理性知识, 并能对事物运行的规律进行分析、判断和推理, 这就是提出概念、建立方法, 进行演绎和归纳推理、作出决策的能力。

3) 通过教育、训练和学习过程, 日益丰富自身的知识和技能, 这就是学习的能力。

4) 对不断变化的外界环境条件 (如干扰、刺激等外界作用) 能灵活地作出正确地反应, 这就是自适应能力。

不论从什么角度来研究人工智能, 都是通过计算机等现代工具来实现的。计算机科学与技术飞速发展和计算机应用的日益普及, 为人工智能的研究和应用奠定了良好的物质基础。人工智能的发展使计算机更聪明、更有效, 与人更接近。

1. 人工智能的起源

自古以来, 人类对人工智能就有持久的、狂热的追求, 并凭借当时的认识水平和技术条件, 设法用机器来代替人的部分脑力劳动, 用机器来延伸和扩展人类的某种智能行为。例如, 公元前 900 多年, 我国就有歌舞机器人传说的记载。12 世纪末至 13 世纪初, 西班牙的一位神学家和逻辑学家曾试图制造能解决各种问题的通用逻辑机。17 世纪, 法国物理学家和数学家巴斯卡 (B.Pascal) 制成了世界第一台会演算的机械加法器并获得实际应用。随后, 德国数学家和哲学家莱布尼兹 (G.W.Leibniz) 在这台加法器的基础上发展并制成了进行全部四则运算的计算器, 他还提出了逻辑机的设计思想, 即通过形式逻辑符号化, 对思维进行推理计算。

这种“万能符号”和“基于符号的推理计算”的思想是“智能机器”的萌芽，因而他被誉为数理逻辑的第一个奠基人。进入 20 世纪后，人工智能相继出现若干开创性的工作。1936 年，年仅 24 岁的英国数学家图灵（A.M.Turing）在他的一篇“理想计算机”的论文中，提出了著名的图灵机模型。1945 年，他进一步论述了电子数字计算机的设计思想。1950 年，他又在“计算机能思维吗？”一文中提出了机器能够思维的论述。1938 年，德国工程师苏斯（Zuse）研制成第一台累计数字计算机 Z-1。1946 年，在美国诞生了世界上第一台电子数字计算机 ENIAC。在同一时代，控制论和信息论的创立，生物学家设计的脑模型等，都为人工智能学科的诞生作出了理论和实验工具的巨大贡献。

1956 年的一次历史性聚会被认为是人工智能学科诞生的标志。1956 年夏季，在美国达特莫斯（Dartmouth）大学，由当时的年轻数学助教、现任斯坦福大学教授的麦卡锡（J.McCarthy）联合他的三位朋友：哈佛大学年轻数学和神经学家、现任麻省理工学院教授的明斯基（M.L.Minsky），IBM 公司信息研究中心负责人洛切斯特（N.Lochester）和贝尔实验室信息部数学研究员香农（C.E.Shannon）共同发起，邀请 IBM 公司的莫尔（T.Moore）和塞缪尔（A.L.Samuel）、麻省理工学院的塞尔夫利奇（O.Selfridge）和索罗莫夫（R.Solomonoff）、兰德（RAND）公司和卡内基工科大学的纽厄尔（A.Newell）和西蒙（H.A.Simon）等 10 名年轻学者，举办了为期两个月的学术讨论会，讨论机器智能问题。经麦卡锡提议，在会上正式决定使用“人工智能”（Artificial Intelligence）这一术语，从而开创了人工智能作为一门独立学科的研究方向。麦卡锡因而被称为人工智能之父。从此在美国开始形成了以人工智能为研究目标的几个研究组，如纽厄尔和西蒙的 Carnegie-RAND 协作组，明斯基和麦卡锡的 MIT 研究组，塞缪尔的 IBM 工程研究组等。

1956 年，人工智能的研究取得了两项重大突破。第一项是纽厄尔、肖（J.Shaw）和西蒙的研究组编制了一个逻辑理论程序 LT（The Logic Theory Machine），模拟人们用数理逻辑证明定理的思想，采用分解、代入、替换等规则，证明了怀特赫德（A.N.Whitehead）和罗素（B.A.W.Russell）合著的《数学原理》第二章中的 38 条定理。1963 年，修订的程序在大机器上终于完成了该章中全部 52 条定理的证明。一般认为，这是用计算机模拟人的高级思维活动的一个重大成果，是人工智能研究的真正开端。第二项是 IBM 工程研究组的塞缪尔研制的西洋跳棋程序。这个程序可以像一个优秀棋手那样，向前看几步来下棋。尤其是它具有自学习、自组织、自适应的能力，能在下棋过程中积累经验，不断提高棋艺。它能学习棋谱，在学习了 175000 多个棋局后，可以根据棋局猜测棋谱所有推荐的走步，准确度达 48%，这是机器模拟人类学习过程的一次极有意义的探索。1959 年，这个程序战胜了设计者本人，1962 年，它又击败了美国一个州的跳棋冠军。

1957 年，纽厄尔、肖和西蒙通过心理学实验，发现了人在问题求解时思维过程的一般规律，思维过程大致可分为三个阶段：

- 1) 先思考出大致的解题计划。
- 2) 根据记忆中的公理、定理和推理规则组织解题过程。

3) 进行方法和目的分析, 不断修正解题计划。

基于这一规律, 他们于 1960 年合作编制成功一种不依赖于具体领域的通用问题求解程序 GPS (General Problem Solver), 能求解 11 种不同类型的问题。

1959 年, 麻省理工学院研究组的麦卡锡发表了表处理语言 LISP。由于 LISP 语言可以方便地处理符号, 很快成为人工智能程序设计的主要语言。LISP 语言武装了一代人工智能科学家, 时至今日, 仍然是研究人工智能的重要工具。

一连串的研究成果使醉心于人工智能远景的学者们作出了过于乐观的预言。1958 年, 纽厄尔和西蒙曾充满自信地认为: 在 10 年内, 计算机将成为世界的象棋冠军; 计算机将要发现和证明重要的数学定理; 计算机将能谱写具有优秀作曲家水平的乐曲; 大多数心理学理论将在计算机上形成。有人甚至断言: 20 世纪 80 年代将是全面实现人工智能的年代, 到了 2000 年, 机器的智能可以超过人的智能。

但是, 事情的发展远非如此理想。塞缪尔的下棋程序在获得州的冠军之后, 再也没有当上全国冠军。自然语言的机器翻译是人工智能研究最早并取得实验性成果的研究方向之一。人们以为只要用一部双向互译字典和某些语法知识即可很快地解决自然语言之间的互译问题, 实际上, 由机器翻译出来的文字有时会出现十分荒谬的错误。例如, 英语句子 “The spirit is willing but the flesh is weak” (心有余而力不足), 翻译成俄语后再翻译成英语, 竟然成为 “The wine is good but the meat is spoiled” (酒是好的但肉变质了)。

自从人工智能形成一个学科之后, 许多学者遵循的指导思想是: 研究和总结人类思维的普遍规律, 并用计算机来模拟人类的思维活动。他们认为, 实现这种计算机智能模拟的关键是建立一种通用的符号逻辑运算体系。但是, 由于人类的认知和思维过程是一种非常复杂的行为, 至今仍未能被完全解释, 也由于现实世界的复杂性和问题的多样性, 老一辈人工智能科学家为之奋斗的通用逻辑推理体系至今也没有创造出来。他们早期的代表作通用问题求解程序 GPS 的通用性受到严格的限制, 只能对具有相当小的状态集和良定义的形式规则的问题有效。人工智能的早期研究只能停留在实验室里, 作为研究的实验系统或演示系统, 不能解决实际问题。科学家们开始对人工智能探索人类思维普遍规律的研究战略思想进行反思。

2. 人工智能的发展

20 世纪 60 年代中期以后, 人工智能由追求万能、通用的一般研究转入特定的具体研究, 通用的解题策略与特定领域的专业知识及实际经验结合, 产生了以专家系统 (ES, Expert System) 为代表的基于知识的各种人工智能系统, 使人工智能走向社会, 走向实际应用研究。斯坦福大学当时的年轻教授费根鲍姆 (E.A.Feigenbaum) 重新举起了英国 16 世纪的哲学家和自然科学家培根 (Francis.Bacon) 的旗帜: “知识就是力量”, 于 1965 年开创了基于知识的专家系统这一人工智能研究的新领域。与通用问题求解程序 GPS 的系统不同, 专家系统并不试图发现很强有力的和很通用的问题求解方法, 而是把研究范围缩小在一个特定的相对狭小的专业领域中。人类专家之所以成为专家, 是因为他拥有解决自己专业领域问题的大量专门知识, 包括各种有用的经验。

在费根鲍姆的主持下，第一个专家系统课题 DENDRAL 化学分子结构分析系统于 1965 年在斯坦福大学开始研究，1968 年研制成功。该系统能根据质谱仪数据推断未知有机化合物的分子结构。它是一个启发式系统，把化学专家关于分子结构质谱测定法的知识结合到控制搜索的规则中，从而能迅速消去不可能为真的分子结构，避免了搜索空间以指数级膨胀。通过产生全部可能为真的分子结构，它甚至可以找出那些人类专家可能漏掉的结构。DENDRAL 及附属的 CONGEN 系统商品化后，每天为上百个国际用户提供化学结构的解释。这一研究成果使人们看到，在某个专门领域里，以知识为基础的计算机系统完全可能相当于这个领域里的人类专家的作用。

MACSYMA 系统是麻省理工学院于 1968 年开始研制的大型符号数学专家系统。该系统从应用数学家获得了几百条关于一个表达式与另一等价表达式之间转换的规则，擅长于易引起组合爆炸的符号表达式的化简，能执行微分、积分、解方程、台劳级数展开、矩阵运算、向量代数等 600 多种不同的数学符号运算。1971 年研制成功后，由于它具有很强的与应用分析相结合的符号运算能力，很多数学和物理学研究人员以及各类工程师争相使用 MACSYMA 系统，遍及美国各地的很多用户每天都通过 ARPA 网与它联机工作达数小时。

在 DENDRAL 和 MACSYMA 的影响下，在化学、数学、医学、生物工程、地质探矿、石油勘探、气象预报、地震分析、过程控制、计算机配置、集成电路测试、电子线路分析、情报处理、法律咨询和军事决策等各方面出现了一大批专家系统。著名的 MYCIN 系统就是斯坦福大学人工智能研究所于 1973 年开始研制的一个诊断和治疗细菌感染性血液病的专家咨询系统。该系统可以看成是 DENDRAL 系统的直接后继者，并且具有更广泛的影响。MYCIN 拥有的知识包括约 450 条“前提-结论”型的关于细菌性血液感染的诊疗规则，系统根据提供的数据和主动向医生询问获得的数据，运用相应的规则进行推理，最终给出对患者诊断和治疗方面的咨询性建议。经专家小组对医学专家、实习医生以及 MYCIN 系统的行为进行正式测试评价，认为 MYCIN 的行为超过了临床医生助手的作用，尤其在诊断和治疗菌血症和脑膜炎方面有相当高的准确率。在 MYCIN 系统框架基础上建立的肺功能专家系统 PUFF 曾在旧金山太平洋医疗中心使用过相当长的一段时间。

由拉特格尔斯 (Rutgers) 大学于 20 世纪 70 年代中期开发的 CASNET 是一个诊断和治疗青光眼的专家咨询系统。CASNET 的知识不使用 MYCIN 的静态规则的表示方式，而是按因果关系把疾病表示为与病理生理状态相连接的网络。系统共有 150 个状态、350 个测试、50 个分类表。系统利用因果网络推断疾病类型，预测治疗效果。此外，系统还包括广泛的医学参考资料，能对疾病作出相当完善的解释。经评价，该系统接近专家水平，曾被美国和日本一些学术团体用于疾病研究。

CADUCEUS (原名 INTERNIST) 系统也是 20 世纪 70 年代研制的医疗咨询专家系统，由匹兹堡大学的计算机专家和内科专家合作研制，用于内科疾病诊断。它的知识表示方式与 CASNET 类似，它的知识库是当前专家系统中最大的知识库之一，据称拥有比人类任何个人都多的内科知识。它用疾病树表达疾病分类知识，知识库中包含 500 多种内科疾病以及 10 万

条疾病与症状的相关关系。它还采用了一些复杂的策略来区别和组合多种疾病。因此,它能正确地对人类专家感到棘手的复杂病案作出诊断。CADUCEUS 已用于医学教学和某些由美国国家健康研究机构赞助的医疗服务点。

HEARSAY I 和 II 是卡内基-梅隆大学于 20 世纪 70 年代先后研制的两个语音理解系统。HEARSAY II 通过称为“黑板”的全局数据库来组合专家的知识。它的语音理解能力可以和一个 10 岁的孩子接近,但离专家水平甚远。这是因为语音理解是一个更为困难的研究方向。

PROSPECTOR 是一个著名的地质勘探专家系统,由斯坦福大学人工智能研究所于 1976 年开始研制。它能帮助地质学家解释地质矿藏数据,提供硬岩石矿物勘探方面的咨询,包括勘探评价、区域资源估值、钻井井位选择等。该系统的知识来自地质学家的矿床模型,用似然推理网络表示知识,网络的节点为有关探测证据和重要地质假设的各种断言,有向弧表示所连接的节点之间的推理规则,用贝叶斯(Bayes)概率推理处理不确定的数据和知识。该系统拥有 12 种矿藏知识库,共有 1100 多条规则,另有 400 种岩石和地质术语。它的性能足以与地质学家相比较,并已在实际应用中取得巨大的经济效益,例如,它发现了一个钼矿,据说该系统在这个钼矿勘探与开采中的使用价值可能超过 1 亿美元。

XCON(原名 R1)是由卡内基-梅隆大学从 1978 年开始研制的一个 VAX 计算机配置专家系统,它能根据 DEC 公司的 VAX 机的用户对每一部件的需求订单,以及关于部件组合方面的上千条规则型约束知识,形成一个功能上可以接受的 VAX 系统配置。经评价小组的正式性能测试,认为该系统具有足够的专家水平。后来,系统又进行了多次改进和发展,最后成为一个成熟的系统,在 DEC 公司担负了日常的 VAX 计算机系统配置任务,每年为 DEC 公司可节省 1500 万美元的设计费用。

专家系统的一系列研究成果展示了人工智能应用的广阔前景,社会各界对人工智能的兴趣与应用需求与日俱增。卫生界因某些医学专家系统的成功而受到极大鼓舞;军界期望它能开辟军事决策和指挥自动化的新局面;工商企业界出自对新技术的厚望,对人工智能的热情迅速高涨起来,过去对人工智能研究的态度十分保守的 IBM 公司也很快改变了态度。20 世纪 70 年代后期,随着专家系统技术的逐渐成熟和应用领域的不断开拓,人们从各种不同类型的专家系统和知识处理系统中抽取共性,人工智能又从具体系统的研究逐渐回到一般研究。围绕知识这一核心问题,人们重新对人工智能的原理和方法进行探索,并在知识获取、知识表示和知识推理等方面开始出现新的原理、方法、技术和工具。在这一背景下,在国际人工智能联合会(IJCAI, International Joint Conference on Artificial Intelligence)于 1977 年召开的 IJCAI-77 会议上,费根鲍姆提出了“知识工程”(KE, Knowledge Engineering)的概念。费根鲍姆因而被称为知识工程之父。知识工程综合了科学、技术和方法论三方面的因素,研究专门知识的获取、形式化和计算机实现,为研制以知识为基础的各类人工智能应用系统提供一般方法和基本工具。知识工程的研究有利于缩短专家系统的研制周期,促进了专家系统从单学科专用型向多学科通用型的发展,出现了一批通用程度不等、类型不同的专家系统工具,包括骨架型工具、以及有更大通用性的语言型工具和知识处理系统环境。应该说,知识工程和专家系统是人工智能研究中最有

成就的分支领域之一，为推进人工智能研究起到了重要作用。

20 世纪 90 年代，计算机发展的主要趋势是小型化、并行化、网络化和智能化。人工智能技术逐渐与数据库技术、多媒体技术、网络技术相结合，旨在使计算机系统更聪明、更有效，与人更接近。计算机智能化技术的主要研究方向大体上可分为三个方面：一是并行与分布处理技术，包括大规模并行计算机和机群系统的体系结构、并行操作系统，并行编译系统与并行数据库系统，分布式 Client/Server 计算模型及其处理技术，多专家协同工作与知识共享技术等；二是知识的获取、表示、更新和推理的新方法与新技术，大型知识库的组织与维护，新一代逻辑处理机制等；三是多功能的感知技术，包括对语音、文字、图形和图像等多媒体信息的获取、压缩、识别与转化，以及虚拟现实技术等。

我国从 1978 年才开始设立人工智能的研究课题，主要在定理证明、汉语自然语言理解、机器人及专家系统方面，并取得一批研究成果。在我国的“863”高技术研究开发计划中，智能计算机系统和智能机器人被列入我国高技术的重点发展主题。自该计划实施以来，我国在人工智能与智能计算机领域已取得许多可喜成果。根据智能计算机系统的高技术研究开发计划，我国将研制可扩充大规模并行的、智能化的先进计算机系统，它们主要用于智能化的可视计算、事务处理、信息管理、信息分析与服务，它们将具有高速的和可视化计算的能力，具有对大量复杂信息进行存取与分析的能力，具有有效灵活的推理及机器学习的能力，具有以语音、文字、图形、图像为界面的和谐的人机交互的能力，具有方便有效的智能化的软件开发能力。我们相信，21 世纪初期，这种计算机系统将成为促进我国社会经济发展的重要基础。

3. 人工智能的主要学派

随着人工智能的发展，围绕人工智能的基本理论和方法等问题，诸如人工智能的定义、基础、核心、要素、认识过程、学科体系以及人工智能与人类智能的关系等，由于存在不同的观点而形成不同的学派。

1987 年，在美国波士顿由麻省理工学院人工智能研究所、美国国家科学基金（NSF）和美国人工智能学会（AAAI）联合主办了人工智能基础国际研讨会。许多在人工智能发展历史上作过重要贡献的各种学派的人工智能科学家应邀出席会议，并在会上报告了自己的论文。在报告中，他们阐明了各自的学术思想，讨论作为他们方法基础的各种原理，叙述各自方法取得成功的形式，解释这些方法的效用，探讨这些方法的适用性和局限性。这些报告已在国际杂志《人工智能》第 47 卷（1991）1~3 期合刊上发表。

目前，人工智能的主要学派有下述 3 家：

1) 符号主义（Symbolicism）学派，又称为逻辑主义（Logicism）学派、心理学派（Psychologism）或计算机学派（Computerism）。

2) 联结主义（Connectionism）学派，又称为仿生学派（Bionicsism）或生理学派（Physiologism）。

3) 行为主义（Actionism）学派，又称为进化主义（Evolutionism）学派或控制论学派（Cyberneticsism）。

人工智能各学派的起源以及进行人工智能研究所采用的基本理论和方法各不相同。

(1) 人工智能各学派的起源

符号主义认为人工智能源于数理逻辑。形式逻辑从 19 世纪末开始迅速发展,到 20 世纪 30 年代开始用于描述智能行为。计算机出现后,又在计算机上实现了逻辑演绎系统,其有代表性的成果是 1956 年研制的逻辑理论程序 LT 证明了 38 条数学定理,表明了可以应用计算机研究人的思维过程,模拟人类智能活动。正是这些符号主义者在 1956 年首先采用“人工智能”这个术语。后来又发展了启发式算法、专家系统、知识工程理论与技术,并在 20 世纪 80 年代取得很大的发展。符号主义曾长期一枝独秀,为人工智能发展作出了重要贡献,尤其是专家系统的成功开发与应用,为人工智能走向工程应用具有特别重要意义。在人工智能的其他学派出现之后,符号主义学派仍然是人工智能的主流。这个学派的代表人物有纽厄尔、肖、西蒙、尼尔逊(N.J.Nilsson)等。

联结主义认为人工智能源于仿生学,特别是人脑模型的研究。1943 年,由生理学专家麦卡洛克(McCulloch)和数理逻辑学家皮茨(Pitts)创立的脑模型,即 MP 模型,开创了用电子装置模仿人脑结构和功能的途径。它从神经元开始进而研究神经网络模型和脑模型,开辟了人工智能研究的新途径。20 世纪 60 年代至 70 年代,联结主义以感知器为代表的脑模型的研究曾出现过热潮,由于当时的理论模型、生物原型和技术条件的限制,脑模型研究在 70 年代后期至 80 年代初期落入低潮。1982 年,美国加州工学院物理学家荷伯维尔德(Hopfield)提出了人工神经网络(ANN, Artificial Neural Network)的 HNN 模型,有力地推动了人工神经网络的研究。他引入了“计算能量函数”的概念,给出了神经网络稳定性判据。HNN 模型的电子电路实现为神经计算机的研究奠定了基础,同时开拓了神经网络用于联想记忆和优化计算的新途径。1985 年,赫顿(Hinton)和塞罗斯基(Sejnowski)借用统计物理学的概念和方法,提出了 Boltzman 机模型,首次采用了多层神经网络的学习算法,即在学习过程中用模拟退火技术保证神经网络趋于全局稳定。1986 年,鲁梅尔哈特(Rumelhart)和麦克勒兰德(McClelland)提出多层神经网络的反向传播(BP)学习算法,通过实例的学习改变多层神经网络的邻接权矩阵,从而达到学习的目的。这是至今为止使用最广泛的人工神经网络。荷兰德(Holland)提出的分类系统类似于以规则为基础的专家系统,他提出的发现和改进规则的学习算法是对专家系统的重要发展。至此,人工神经网络的研究再次出现热潮。神经网络的发展为认知科学、计算机科学及人工智能的发展开辟了一条崭新的途径。虽然有不少人对神经网络研究的前景存在疑虑,但最悲观的估计仍然认为它会带来重大的科学研究成果和广泛的应用,而最乐观的估计则称之为一种新主义——联结主义,它是一种能解决知识表示、推理、学习、联想记忆等复杂系统的统一模型。

行为主义是近几年来才被被认为是人工智能的一个新的学派。行为主义源于控制论。在 20 世纪 50 年代,控制论的思想已对早期的人工智能工作者产生影响。早期的研究重点是模拟人在控制过程中的智能行为和作用,如对自寻优、自适应、自组织和自学习等控制系统的研究,并进行“控制论动物”的研制。在 20 世纪 60 年代至 70 年代,上述控制论系统的研究

取得一定的进展,产生了智能控制和智能机器人的萌芽,并在80年代诞生了智能控制和智能机器人系统。行为主义学派的代表作首推布鲁克斯(Brooks)的六足行走机器人(机器虫),它被看作新一代的“控制论动物”,是一个基于“感知-动作”模式的模拟昆虫行为的智能控制系统。

(2) 人工智能学派的基本理论框架

人类的认知过程是非常复杂的行为,至今仍未能被完全解释,人工智能各学派从不同的角度对人的智能行为进行研究,从而形成各学派研究人工智能的不同的基本理论。

符号主义认为可以用一个符号系统在计算机上形式化地描述和模拟人的思维活动过程。符号主义还认为知识是智能的基础,人工智能的核心问题是知识表示与知识推理,知识是用一种符号系统来表示的,也可以用符号的操作进行知识推理。因此,有可能建立起基于知识的人类智能和机器智能的统一理论体系。

联结主义利用人工神经网络模仿人类智能,认为人的智能的基本单元是神经元,由许多人工神经元联接起来的人工神经网络可以具有自学习和自适应功能,能更好地模仿人类智能。人工神经网络并不是人脑的真实描写,只是人脑的某种抽象、简化与模拟。人工神经网络实际上是一种非线性自适应信息处理系统,信息处理由神经元之间的相互作用来实现,知识与信息的存储表现为网络各神经元之间的联系,学习则表现为网络各神经元连接权的动态变化过程。

行为主义认为智能取决于感知和表现为行动,智能行为只能在现实世界中与周围环境交互作用而表现出来,从而提出智能行为的“感知-动作”模式。

由于各学派对人工智能研究的基本理论框架不同,因此,各学派对人工智能的研究方法也不同。

符号主义认为人工智能的研究方法应是功能模拟方法,分析人类认知系统所具备的功能和机理,然后用计算机来模拟这些功能,从而实现人工智能。符号主义力图用数理逻辑方法来建立人工智能的统一理论体系。

联结主义认为人工智能应着重于结构模拟,即模拟人的生理神经网络结构,并认为功能与结构是密切相关的,不同的结构表现出不同的功能和智能行为。至今,已经提出多种人工神经网络结构和多种学习算法。

行为主义认为人工智能的研究方法应采用行为模拟方法,也认为功能、结构和智能行为是不可分开的,不同的行为需要不同的控制结构和表现出不同的功能。行为主义的研究方法受到其他学派的怀疑,认为行为主义最多只能创造出低智能的昆虫行为,无法创造出人的高智能行为。

现在,在人工智能的基本理论、研究方法和技术路线等方面存在几种不同的学派,说明人工智能已经从“一枝独秀”的符号主义发展到多学派“百花争艳”,必将促进人工智能的进一步发展,并肩开创人工智能的新纪元。

人工智能的近期研究目标是建造智能计算机,即使得现有的计算机更聪明更有用。正是

根据这一近期研究目标，我们才把人工智能理解为计算机科学的一个分支。人工智能的远期研究目标是研究人类智能和机器智能的基本原理，用智能机器来模拟人类的思维过程和智能行为。这个远期目标几乎涉及所有的自然科学学科和社会学科。

人类智能是人脑系统的整体效应，有着极为丰富的层次和侧面。符号主义、联结主义和行为主义都只抽象出人脑思维的部分特征来模仿人的智能和功能，人的认知过程和智能行为要比人们想象的复杂得多，因此，人工智能需要多学科长期协作研究，才能使人工智能达到一个更高的水平。

1.2 人工智能的研究与应用领域

目前，人工智能的研究更多的是结合具体应用领域来进行的。这里，介绍几个主要的应用研究领域。

1. 定理证明

数学领域中对臆测的定理寻求一个证明或反证，一直被认为是一项需要智能才能完成的工作。证明定理时，不仅需要具有根据假设进行演绎的能力，而且需要具有某些直觉的技巧。例如，为了求证一个定理时，数学家会熟练地运用他的丰富的专业知识，猜测需要先证明哪几个引理，精确判断出已有的哪些定理会在这个定理的证明过程中起作用，并可以把这个定理证明分解为若干子问题，对子问题可分别独立地进行求解。

定理证明的研究在人工智能方法的发展中曾经产生过重要的影响。例如，采用谓词逻辑语言的演绎过程的形式化，可以帮助人们更清楚地理解演绎推理过程。许多其他领域的问题，如医疗诊断、信息检索等也可以应用定理证明的方法，因此，机器定理证明的研究在人工智能研究中具有普遍意义。

2. 专家系统

一般地说，专家系统是一个具有大量专门知识与经验的程序系统。专家系统存储有某个专门领域中经过事先总结、分析并按某种模式表示的专家知识（组成知识库），以及拥有类似于领域专家解决实际问题的推理机制（构成推理机）。系统能对输入信息进行处理，并运用知识进行推理，做出决策和判断，其解决问题的水平达到或接近专家的水平，因此能起到专家或专家助手的作用。专家系统的开发和研究是人工智能中最活跃的一个应用研究领域，涉及到社会各个方面，可以说，需要有专家工作的场合，就可以开发专家系统。

开发专家系统的关键是表示和运用专家知识，即来自领域专家的已被证明对解决有关领域内的典型问题有用的事实和过程。目前，专家系统主要采用基于规则的知识表示和推理技术。由于领域的知识更多是不精确或不确定的，因此，不确定的知识表示与知识推理是专家系统开发与研究的重要课题。此外，专家系统开发工具的研制发展也很迅速，这对扩大专家系统的应用范围，加快专家系统的开发过程，将起到积极地促进作用。随着计算机科学技术整体水平的提高，分布式专家系统、协同式专家系统等新一代专家系统的研究也发展很快。

在新一代专家系统中，不但采用基于规则的推理方法，而且采用诸如人工神经网络的方法。

3. 机器学习

学习是人类智能的主要标志和获得知识的基本手段，学习能力无疑是人工智能研究的一个最重要的方面。学习是一个有特定目的的知识获取过程，其内部表现为新知识的不断建立和知识的更新，而外部表现为系统的性能得到改善。一个学习过程本质上是学习系统把导师或专家提供的学习实例或信息转换成能被学习系统理解并应用的形式存储在系统中。

传统的机器学习倾向于使用符号表示知识而不是数值表示知识；另一个倾向是使用归纳方法进行学习而不是演绎方法。近几年来，又发展了下述各种学习方法：基于解释的学习方法、基于事例的学习方法、基于人工神经网络的学习方法、基于遗传算法的学习方法等。

4. 自然语言理解

自然语言是人类之间信息交流的主要媒介，由于人类有很强的理解自然语言的能力，因此，人们相互间的信息交流轻松自如。但是，目前计算机系统和人类之间的交互几乎还只能使用严格限制的各种非自然语言，因此，解决计算机系统能理解自然语言的问题就是人工智能研究的一个十分重要的课题。在智能计算机的研究中，自然语言理解就是其中的重点研究课题之一。

一个独立的简单句子是比较容易理解它的含义的，但是，对于一段用自然语言表示的较长的文章或对话，要准确地理解其中每个句子的含义就不仅限于这个句子本身，这个句子的含义可能与它的上下文甚至背景知识有关。因此，自然语言理解涉及对上下文知识结构的表示和根据上下文知识进行推理的方法和技术。目前，在理解有限范围的自然语言对话和小段文章方面的程序系统研制已有一些进展。但是，实现功能较强的自然语言理解系统还是一个比较艰巨的任务。显然，在实现机器翻译时，如果机器系统能准确地理解每一个句子的含义，那么就能翻译出更准确更通顺的译文。

5. 智能检索

数据库系统是存储某学科大量事实的计算机系统，随着应用的发展，存储的信息量愈来愈庞大，研究智能检索系统具有重要的实际意义。

智能信息检索系统应具有下述功能：

- 1) 能理解自然语言，允许用户使用自然语言提出检索要求和询问。
- 2) 具有推理能力，能根据数据库存储的事实，推理产生用户要求和询问的答案。

3) 系统拥有一定的常识性知识，以补充数据库中学科范围的专业知识。系统根据这些常识性知识和专业知识能演绎推理出专业知识中没有包含的答案。例如，某单位的人事档案数据库中有如下事实：“张强是采购部工作人员”、“李明是采购部经理”。如果系统具有“部门经理是该部门工作人员的领导”这一常识性知识，就可以对询问“谁是张强的领导”演绎推理出答案“李明”。

6. 机器人学

随着工业自动化和计算机技术的发展，到 20 世纪 60 年代，机器人开始进入大量生产和

实际应用的阶段。后来,由于自动装配、海洋开发、空间探索等领域的需要,对机器人的智能水平提出了更高的要求。特别是危险环境和恶劣环境更迫切需要机器人代替人来工作,从而推动了智能机器人的研究。

智能机器人的运动规划分为高层规划和低层规划两个层次。高层规划是根据感知的环境信息和要求实现的目标可规划出机器人执行的动作命令序列,然后由低层规划将每一个动作命令转换成驱动机器人各关节运动的驱动电机的角速度或角位移,各关节驱动电机的协调运动将保证实现相应的动作命令。

智能机器人是多学科交叉的综合课题,它涉及精密机械,视觉、触觉、力觉等信息传感技术,自动控制,人工智能的规划方法等。这一课题研究有利于促进各学科的相互结合和渗透,并将大大推动人工智能技术的发展。

7. 自动程序设计

自动程序设计是指:设计一个能自动生成程序的程序系统,这个程序系统只需要对其输入要求生成的程序的实现目标的非常高级的描述,就能自动生成能完成这个目标的程序。

从某种意义上来说,编译程序实际上就是做“自动程序设计”的工作,编译程序接受做某一件工作的源代码(源程序),然后生成目标代码(目标程序)去执行这件工作。这里所说的自动程序设计相当于“超级编译程序”,它要求不是给出完整的源代码来详细说明要做的工作,而只需要对要做的工作给出目标性的高级描述就可以生成完成这个工作的程序。

自动程序设计所涉及的基本问题与定理证明和机器人学涉及的问题有关,要求对高级的目标描述通过规划过程生成所需的程序。

8. 组合调度问题

有许多实际问题是属于最佳调度或最佳组合问题。例如,推销员旅行问题就属于这一类问题。推销员旅行问题是指:推销员从某个城市出发,遍访他所要访问的城市一次,回到他出发的城市,求推销员最短的旅行路线。该问题的一般化为:对若干节点组成的一个图,寻找一条最小耗费的路径,使这条路径对每个节点穿行一次。

在大多数组合调度问题中,随着求解问题规模的增大,求解程序都面临着组合爆炸问题。在推销员旅行问题中,问题规模可用需要穿行的城市数目来表示。随着求解问题规模的增大,问题求解程序的复杂性(用于求解程序运行所需的时间和空间或求解步数)可随问题规模按线性关系、多项式关系或指数关系增长。

组合调度问题中有一类问题称为 NP 完全问题, NP 完全问题是指:用目前知道的最好的方法求解,问题求解需要花费的时间(或称为问题求解的复杂性)是随问题规模增大以指数关系增长。推销员旅行问题就是一个 NP 完全问题。至今还不知道对 NP 完全问题是否有花费时间较少的求解方法。例如可使求解时间随问题规模按多项式关系增长。

组合调度问题的求解方法已经应用于交通运输调度、列车编组、空中交通管制和军事指挥自动化等系统。

9. 模式识别

“模式”(Pattern)一词的本意是指完整无缺的供模仿的标本或标识。模式识别就是识别出给定物体所模仿的标本或标识。计算机模式识别系统使一个计算机系统具有模拟人类通过感官接受外界信息、识别和理解周围环境的感知能力。

模式识别是一个不断发展的学科分支,它的理论基础和研究范围也在不断发展。在二维的文字、图形和图像的识别方面,已取得许多成果。三维景物和活动目标的识别和分析是目前研究的热点。语音的识别和合成技术也有很大的发展。基于人工神经网络的模式识别技术在手写字符的识别、汽车牌照的识别、指纹识别、语音识别等方面已经有许多成功的应用。模式识别技术是智能计算机和智能机器人研究的十分重要的基础。

10. 机器视觉

实验表明,人类接受外界信息的80%以上来自视觉,10%左右来自听觉,其他来自嗅觉与味觉及触觉。在机器视觉方面,给计算机系统装上电视摄像输入装置就可以“看见”周围的东西。但是,视觉是一种感知,机器视觉的感知过程包含一系列的处理过程,例如,一个可见的景物由传感器编码输入,表示成一个灰度数值矩阵;图像的灰度数值由图像检测器进行处理,检测器检测出图像的主要成分,如组成景物的线段、简单曲线和角度等;这些成分又被处理,以便根据景物的表面特征和形状特征来推断有关景物的特征信息;最终目标是利用某个适当的模型来表示该景物。

视觉感知问题的要点是形成一个精练的表示来取代极其庞大的未经加工的输入信息,把庞大的视觉输入信息转化为一种易于处理和有感知意义的描述。

机器视觉可分为低层视觉和高层视觉两个层次。低层视觉主要是对视觉图像执行预处理,例如,边缘检测、运动目标检测、纹理分析等,另外还有立体造型、曲面色彩等,其目的是使看见的对象更突显出来,这时还谈不上对它的理解。高层视觉主要是理解对象,显然,实现高层视觉需要掌握与对象相关的知识。

机器视觉的前沿研究课题包括:实时图像的并行处理,实时图像的压缩、传输与复原,三维景物的建模识别,动态和时变视觉等。

习题一

- 1.1 何谓人工智能?人类智能主要包括哪些能力?
- 1.2 “知识工程”是在什么背景下提出的?知识工程对人工智能的发展有何重要作用?
- 1.3 当前计算机发展的主要趋势是什么?智能计算机的主要研究方向有哪些?
- 1.4 人工智能有哪几个主要学派?各学派的基本理论框架和研究方法有何不同?
- 1.5 人工智能的近期研究目标与远期研究目标分别是什么?
- 1.6 人工智能主要的研究应用领域有哪些?

第2章 知识表示

人类的智能活动过程主要是一个获得知识并运用知识的过程，知识是智能的基础。为了使计算机具有智能，使它能模拟人类的智能行为，就必须使它具有知识。但是，需要把人类拥有的知识采用适当的模式表示出来，才能存储到计算机中去，这就是用知识表示要解决的问题。知识表示是对知识的一种描述，或者说是一组约定，是一种计算机可以接受的、用于描述知识的数据结构，对知识进行表示就是把知识表示成便于计算机存储和利用的某种数据结构。知识表示方法又称为知识表示技术，其表示形式称为知识表示模式。

目前使用较多的知识表示方法主要有：一阶谓词逻辑表示法、产生式表示法、框架表示法、语义网络表示法、面向对象表示法等。本章将对这些基本的知识表示模式进行讨论，关于不确定性知识的表示和基于人工神经网络的知识表示将在后面的有关章节中讨论。

2.1 一阶谓词逻辑表示法

谓词逻辑是一种形式语言，也是目前能够表达人类思维活动的一种最精确的语言，它与人类的自然语言比较接近，又可以方便地存储到计算机中，并被计算机进行精确处理。因此，它成为最早应用于人工智能中表示知识的一种逻辑表示方法。

2.1.1 谓词逻辑

谓词逻辑是在命题逻辑的基础上发展起来的，对于知识的形式化表示，特别是在定理的自动证明中发挥了重要作用，在人工智能发展史中占有重要地位。

1. 命题

命题是具有真假意义的语句。命题代表人们进行思维时的一种判断，或者是肯定，或者是否定，只有这两种情况。若命题的意义为真，则称它的真值为真，记为 T；若命题的意义为假，则称它的真值为假，记为 F。一个命题不能同时既为真又为假，但在一定条件下为真，在另一种条件下为假。没有真假意义的语句（如感叹句、疑问句等）不是命题。例如，“中华人民共和国的首都是北京”，“ $3 < 4$ ”都是真值为 T 的命题；“太阳从西边升起”是真值为 F 的命题；“ $1+1=10$ ”在二进制情况下是真值为 T 的命题，但在十进制情况下是真值为 F 的命题。

命题逻辑的这种表示有较大的局限性，它无法描述客观事物的结构及逻辑特征，也不能把不同事物间的共同特征表述出来。例如，对于“杨青是教师”和“李文是教师”这两个命题，用命题逻辑表示时，无法把两人都是教师这一共同特征表示出来。

2. 谓词

在谓词逻辑中, 命题是用谓词表示的。一个谓词可分为谓词名与个体两个部分, 个体表示某个独立存在的事物或者某个抽象的概念, 谓词名用于刻画个体的性质、状态或个体间的关系。例如, 对于“老张是教师”这个命题, 用谓词可表示为 $\text{Teacher}(\text{Zhang})$, 其中, Teacher 是谓词名, Zhang 是个体, “ Teacher ”刻画了“ Zhang ”是教师的职业特征, 又如, “ $5 > 3$ ”这个命题可用谓词表示为 $\text{Greater}(5, 3)$, “ Greater ”刻画了两个个体“5”与“3”之间的“大于”关系。

谓词的一般形式为:

$$P(x_1, x_2, \dots, x_n)$$

其中, P 是谓词名, $x_1, x_2, \dots, x_n$ 是个体。谓词名通常用大写英文字母表示, 个体通常用小写英文字母表示。谓词中包含的个体数目称为谓词的元数, 例如, $P(x)$ 是一元谓词, $P(x, y)$ 是二元谓词, $P(x_1, x_2, \dots, x_n)$ 是 n 元谓词。

谓词中的个体可以是常量, 也可以是变元, 还可以是一个函数。例如, 对于“小王的父亲是教师”, 可以表示为 $\text{Teacher}(\text{father}(\text{Wang}))$, 其中, $\text{father}(\text{Wang})$ 是一个函数, 表示“小王的父亲”, 它是谓词 Teacher 的个体。又如, 对于“ $x < 5$ ”可以表示为 $\text{Less}(x, 5)$, 其中 x 是变元, 5 是常量。显然, 当谓词中的变元都用特定的个体取代时, 谓词就具有一个确定的真值: T 或 F。

个体变元的取值范围称为个体域。个体域可以是有限的, 也可以是无限的。例如, 若用 $I(x)$ 表示“ x 是整数”, 则变元 x 的个体域是所有整数, 它是无限的。

谓词与函数表面上很相似, 容易混淆, 其实这是两个完全不同的概念。谓词的真值是“真”或“假”, 而函数的值是个体域中的某个个体, 函数只是从一个个体到另一个个体的映射, 函数无真值可言。例如 $\text{father}(\text{Wang})$ 是把个体“小王”映射到“小王的父亲”。为了便于区别函数与谓词, 函数用小写英文字母表示。

个体常量、个体变元、函数统称为“项”。

在谓词 $P(x_1, x_2, \dots, x_n)$ 中, 若 $x_i (i=1, 2, \dots, n)$ 都是个体常量、变元或函数, 则称它为一阶谓词。若某个 x_i 本身又是一个一阶谓词, 则称 P 为二阶谓词。余者类推。本书中用到的都是一阶谓词。

3. 谓词公式

在谓词逻辑中, 可以用连词来连接若干个谓词组成一个谓词公式, 以表示一个比较复杂的含义。在谓词公式中还可以引入量词来刻画谓词与个体间的关系。

(1) 连词

在谓词逻辑中, 有下述连词:

1) 非连词 $\neg$: 其作用是否定位于后面的命题。当命题 P 为真时, $\neg P$ 为假; 当 P 为假时, $\neg P$ 为真。

2) 或连词 $\vee$: 它表示被它连接的两个命题有“或”关系。用 $\vee$ 连接两个命题称为析取。

3) 与连词 $\wedge$ ：它表示被它连接的两个命题有“与”关系。用 $\wedge$ 连接两个命题称为合取。

4) 蕴含连词 $\rightarrow$ ：它表示被它连接的两个命题的“蕴含”关系。 $P \rightarrow Q$ 表示“P 蕴含 Q”，即“如果 P，则 Q”，其中 P 称为前件，Q 称为后件。

以上连词的逻辑真值表由表 2.1 给出。

表 2.1 谓词逻辑真值表

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$
T	T	F	T	T	T
T	F	F	T	F	F
F	T	T	T	F	T
F	F	T	F	F	T

(2) 量词

在谓词逻辑中，有下述两个量词：

1) 全称量词 ($\forall x$)：它表示对个体域中的所有（或任一个）个体 x 。

2) 存在量词 ($\exists x$)：它表示在个体域中存在个体 x 。

例如，若谓词 $P(x)$ 表示 x 是正数， $F(x,y)$ 表示 x 与 y 是朋友，则：

$(\forall x)P(x)$ 表示个体域中的所有个体 x 都是正数。

$(\forall x)(\exists y)F(x,y)$ 表示对于个体域中的任何个体 x ，都存在个体 y ， x 与 y 是朋友。

$(\exists x)(\forall y)F(x,y)$ 表示在个体域中存在个体 x ，他与个体域中的任何个体 y 都是朋友。

$(\exists x)(\exists y)F(x,y)$ 表示在个体域中存在个体 x 和个体 y ， x 与 y 是朋友。

(3) 谓词公式

由下述规则得到的谓词公式称为合式公式：

1) 单个谓词是合式公式，称为原子谓词公式。

2) 若 A 是合式公式，则 $\neg A$ 也是合式公式。

3) 若 A 、 B 都是合式公式，则 $A \vee B$ 、 $A \wedge B$ 、 $A \rightarrow B$ 也都是合式公式。

4) 若 A 是合式公式， x 是任一个体变元，则 $(\forall x)A$ 和 $(\exists x)A$ 也都是合式公式。

在合式公式中，连词的优先级别依序为： $\neg$ ， $\wedge$ ， $\vee$ ， $\rightarrow$ 。

位于量词后面的单个谓词或者用括弧括起来的合式公式称为该量词的辖域，辖域内与量词变元同名的变元称为约束变元，不受约束的变元称为自由变元。例如

$$(\exists x)(P(x,y) \rightarrow Q(x,y)) \vee R(x,y)$$

其中， $(P(x,y) \rightarrow Q(x,y))$ 是量词 $(\exists x)$ 的辖域，辖域内的变元 x 是受 $(\exists x)$ 约束的变元，而 $R(x,y)$ 中的 x 是自由变元，公式中所有的 y 都是自由变元。

在谓词公式中，可以把一个变元的名字换成另一个名字。但必须注意，当对量词辖域内的约束变元更名时，必须把辖域内同名的约束变元都统一改成相同的名字，且不能与辖域内的

的自由变元同名；当对辖域内的自由变元改名时，不能改成与约束变元相同的名字。例如，对于公式 $(\forall x)P(x,y)$ ，可以改名为 $(\forall z)P(z,t)$ ，这里，把约束变元 x 更名为 z ，把自由变元 y 更名为 t 。

4. 谓词公式的解释

在命题逻辑中，对命题公式中各个命题的一次真值指派称为命题公式的一个解释。一旦解释确定后，根据各连词的定义就可求出命题公式的真值（T 或 F）。在谓词逻辑中，由于公式中可能有个体常量、个体变元和函数，因此不能像命题公式那样直接通过真值指派给出解释，必须首先考虑个体常量和函数在个体域中的取值，然后才能针对常量与函数的具体取值为谓词分别指派真值。由于存在多种组合情况，所以一个谓词公式的解释可能有很多个。对于每一个解释，谓词公式都可求出一个真值（T 或 F）。

下面首先给出谓词公式的解释的定义，然后用例子说明如何构造一个解释以及如何根据解释求出谓词公式的真值。

定义 2.1 设 D 为谓词公式 P 的个体域，若对 P 中的个体常量、函数和谓词按如下规定赋值：

①为每个个体常量指派 D 中的一个元素。

②为每个 n 元函数指派一个从 D^n 到 D 的映射，其中

$$D^n = \{(x_1, x_2, \dots, x_n) | x_1, x_2, \dots, x_n \in D\}$$

③为每个 n 元谓词指派一个从 D^n 到 $\{F, T\}$ 的映射。

则称这些指派为公式 P 在 D 上的一个解释。

例 2.1 设个体域 $D=\{1, 2\}$ ，求公式 $A=(\forall x)(\exists y)P(x,y)$ 在 D 上的解释，并指出在每一种解释下公式 A 的真值。

解 由于在公式 A 中没有包括个体常量和函数，所以可直接为谓词指派真值。设对谓词 $P(x,y)$ 在个体域 D 上的真值指派为：

$$P(1,1)=T, P(1,2)=F, P(2,1)=T, P(2,2)=F$$

这就是公式 A 在 D 上的一个解释。在此解释下，因为 $x=1$ 时有 $y=1$ 使 $P(x,y)$ 的真值为 T ， $x=2$ 时也有 $y=1$ 使 $P(x,y)$ 的真值为 T ，即 x 对于 D 中的所有取值，都存在 $y=1$ ，使 $P(x,y)$ 的真值为 T ，所以在此解释下公式 A 的真值为 T 。

还可以为公式 A 中的谓词指派另外一组真值，设为：

$$P(1,1)=T, P(1,2)=T, P(2,1)=F, P(2,2)=F$$

这是对公式 A 的另一个解释。在此解释下，对 D 中的所有 x （即 $x=1$ 与 $x=2$ ）不存在一个 y 使得公式 A 的真值为 T ，所以在此解释下公式 A 的真值为 F 。

公式 A 在 D 上共有 16 种解释，这里不再一一列出。

例 2.2 设个体域 $D=\{1,2\}$ ，求公式 $B=(\forall x)(P(x) \rightarrow Q(f(x),b))$ 在 D 上的某一个解释，并指出公式 B 在此解释下的真值。

解 设对个体常量 b 指派 D 中的一个元素为 $b=1$ ，对函数 $f(x)$ 指派到 D 的映射为： $f(1)=2$ ，

$f(2)=1$ 。对谓词指派的真值为：

$$P(1)=F, P(2)=T, Q(1,1)=T, Q(2,1)=F$$

这里，由于已对个体常量 b 指派 $b=1$ ，所以 $Q(1,2)$ 与 $Q(2,2)$ 不可能出现，故没有给它们指派真值。

上述指派就是对公式 B 的一个解释。在此解释下，由于当 $x=1$ 时，有

$$P(1)=F, Q(f(1),1)=Q(2,1)=F$$

所以 $P(1) \rightarrow Q(f(1),1)$ 的真值为 T 。当 $x=2$ 时，有

$$P(2)=T, Q(f(2),1)=Q(1,1)=T$$

所以 $P(2) \rightarrow Q(f(2),1)$ 的真值也为 T 。即对个体域 D 中的所有 x 都有

$$P(x) \rightarrow Q(f(x),b)$$

的真值为 T 。所以公式 B 在此解释下的真值为 T 。

由上述的例子可见，谓词公式的真值是针对某一个解释而言的，它可能在某一个解释下的真值为 T ，在另一个解释下的真值为 F 。

若公式 P 在解释 I 下其真值为 T ，则称 I 为公式 P 的一个模型。

5. 谓词公式的永真性、可满足性、不可满足性

定义 2.2 如果谓词公式 P 对个体域 D 上的任何一个解释都取得真值 T ，则称公式 P 在 D 上是永真的。如果 P 在每个非空个体域上均永真，则称 P 是永真的。

由此定义可以看出，为了判定某个公式永真，必须对每个个体域上的每一个解释逐一判定公式的真值。当解释的个数为有限时，尽管工作量较大，总还是可以判定的，但当解释的个数为无限时，公式的永真性就难以判定了。

定义 2.3 对于谓词公式 P ，如果至少存在一个解释使得公式 P 在此解释下的真值为 T ，则称公式 P 是可满足的。

谓词公式的可满足性又称为相容性。

定义 2.4 如果谓词公式 P 对于个体域 D 上的任何一个解释都取得真值 F ，则称公式 P 在 D 上是永假的。如果 P 在每个非空个体域上均永假，则称 P 是永假的。

谓词公式的永假性又称为不可满足性或不相容性。

6. 谓词公式的等价性

定义 2.5 设 P 与 Q 是两个谓词公式， D 是它们共同的个体域，若对 D 上的任何一个解释， P 与 Q 都有相同的真值，则称公式 P 和 Q 在 D 上是等价的。如果 D 是任意的个体域，则称 P 和 Q 是等价的。记为 $P \Leftrightarrow Q$ 。

下面列出今后要用到一些主要的等价式。

(1) 交换律

$$P \vee Q \Leftrightarrow Q \vee P$$

$$P \wedge Q \Leftrightarrow Q \wedge P$$

(2) 结合律

$$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$$

$$(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$$

(3) 分配律

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

$$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$

(4) 狄·摩根律

$$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$$

$$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$$

(5) 双重否定律

$$\neg \neg P \Leftrightarrow P$$

(6) 吸收律

$$P \vee (P \wedge Q) \Leftrightarrow P$$

$$P \wedge (P \vee Q) \Leftrightarrow P$$

(7) 补余律

$$P \vee \neg P \Leftrightarrow T$$

$$P \wedge \neg P \Leftrightarrow F$$

(8) 连词化归律

$$P \rightarrow Q \Leftrightarrow \neg P \vee Q$$

(9) 量词转换律

$$\neg (\exists x) P \Leftrightarrow (\forall x) (\neg P)$$

$$\neg (\forall x) P \Leftrightarrow (\exists x) (\neg P)$$

(10) 量词分配律

$$(\forall x)(P \wedge Q) \Leftrightarrow (\forall x)P \wedge (\forall x)Q$$

$$(\exists x)(P \vee Q) \Leftrightarrow (\exists x)P \vee (\exists x)Q$$

7. 谓词公式的永真蕴含

定义 2.6 对于谓词公式 P 和 Q , 如果 $P \rightarrow Q$ 永真, 则称 P 永真蕴含 Q , 且称 Q 为 P 的逻辑结论, 称 P 为 Q 的前提, 记为 $P \Rightarrow Q$ 。

下面列出今后要用到一些主要的永真蕴含式。

(1) 化简式

$$P \wedge Q \Rightarrow P$$

$$P \wedge Q \Rightarrow Q$$

(2) 附加式

$$P \Rightarrow P \vee Q$$

$$Q \Rightarrow P \vee Q$$

(3) 析取三段论

$$\neg P, P \vee Q \Rightarrow Q$$

(4) 假言推理

$$P, P \rightarrow Q \Rightarrow Q$$

(5) 拒取式推理

$$\neg Q, P \rightarrow Q \Rightarrow \neg P$$

(6) 假言三段论

$$P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$$

(7) 二难推论

$$P \vee Q, P \rightarrow R, Q \rightarrow R \Rightarrow R$$

(8) 全称固化

$$(\forall x) P(x) \Rightarrow P(y)$$

其中 y 是个体域中的任一个体。利用此永真蕴含式可消去公式中的全称量词。

(9) 存在固化

$$(\exists x) P(x) \Rightarrow P(y)$$

其中 y 是个体域中某一可使 $P(y)$ 为真的个体。利用此永真蕴含式可消去公式中的存在量词。

8. 推理规则、定理与证明

上面列出的等价式和永真蕴含式是进行演绎推理的重要依据, 应用这些公式可保证推理的有效性, 因此这些公式又称为推理规则。推理规则用来由已知的合式公式推导出新的合式公式。在谓词逻辑中, 导出的合式公式称为定理, 而所使用的推理规则的序列则构成该定理的一个证明。在人工智能中, 可以把某些问题求解的任务表示为某个定理求证的任务, 在该证明中所应用的推理规则序列给出了该问题的一个解答。

除上述推理规则之外, 谓词逻辑中还有如下的一些推理规则。

(1) P 规则

P 规则是指: 在推理的任何步骤上都可引入前提。

(2) T 规则

T 规则是指: 在推理时, 如果前面步骤中有一个或多个公式永真蕴含公式 S , 则可把 S 引入推理过程中。

(3) CP 规则

CP 规则是指: 如果能从 R 和前提集合中推出 S 来, 则可从前提集合推出 $R \rightarrow S$ 。

(4) 反证法规则

反证法规则是指: $P \Rightarrow Q$, 当且仅当 $P \wedge \neg Q \Leftrightarrow F$ 。即: Q 为 P 的逻辑结论, 当且仅当 $P \wedge \neg Q$ 是不可满足的。

把反证法推广到谓词公式集, 可得到以下反证法定理:

定理 2.1 Q 为 $P_1, P_2, \dots, P_n$ 的逻辑结论, 当且仅当

$$(P_1 \wedge P_2 \wedge \cdots \wedge P_n) \wedge \neg Q$$

是不可满足的。

该定理将在归结反演中得到应用，它是归结反演的理论根据。

2.1.2 一阶谓词逻辑表示法的特点

谓词逻辑适合于表示事物的状态、属性、概念等事实性的知识，也可以用来表示事物间确定的因果关系，即规则。事实通常用谓词公式的与/或形表示，所谓与/或形是指用合取符号（ $\wedge$ ）及析取符号（ $\vee$ ）连接起来的公式。规则通常用蕴含式表示。

用谓词公式表示知识时，首先需要定义谓词，给出每个谓词的确切含义，然后再用连词把有关谓词连接起来表示一个更复杂的含义。对谓词公式中的变元，根据知识表示的需要，把需要约束的变元用相应的量词予以约束。

例 2.3 用谓词公式表示下列知识：

王林是计算机系的学生，但他不喜欢编程序。

人人爱劳动。

解 首先定义下列谓词：

COMPUTER (x)	表示 x 是计算机系的学生
LIKE (x, y)	表示 x 喜欢 y
LOVE(x, y)	表示 x 爱 y
MAN (x)	表示 x 是人

可用谓词公式把上述知识表示为：

$$\text{COMPUTER}(\text{Wang Lin}) \wedge \neg \text{LIKE}(\text{Wang Lin}, \text{Programing})$$

$$(\forall x) (\text{MAN}(x) \rightarrow \text{LOVE}(x, \text{Labour}))$$

例 2.4 用谓词公式表示下列知识：

自然数是大于零的整数。

所有整数不是偶数就是奇数。

偶数除以 2 是整数。

解 首先定义下列谓词：

N(x)	表示 x 是自然数
I(x)	表示 x 是整数
E(x)	表示 x 是偶数
O(x)	表示 x 是奇数
GZ(x)	表示 x 大于零

可用谓词公式分别表示上述知识为：

$$(\forall x) (\text{N}(x) \rightarrow \text{GZ}(x) \wedge \text{I}(x))$$

$$(\forall x) (\text{I}(x) \rightarrow \text{E}(x) \vee \text{O}(x))$$

$$(\forall x)(E(x) \rightarrow I(f(x)))$$

其中, 函数 $f(x) = x/2$ 。

一阶谓词逻辑是一种形式语言系统, 它用逻辑方法研究推理的规律, 即条件与结论之间的蕴含关系。一阶谓词逻辑表示知识的方法有如下优点。

(1) 自然性

谓词逻辑是一种接近于自然语言的形式语言, 人们比较容易接受, 用它表示知识比较容易理解。

(2) 精确性

谓词逻辑是二值逻辑, 谓词公式的真值只有“真”与“假”, 因此可用它表示精确知识, 并可保证经演绎推理所得出的结论的精确性。

(3) 严密性

谓词逻辑具有严格的形式定义及推理规则, 利用这些推理规则及有关定理证明的方法和技巧可从已知事实推出新的事实, 或证明提出的假设。

(4) 容易实现

用谓词逻辑表示的知识可以比较容易地转换为计算机的内部形式, 便于实现对知识的增加、删除与修改。用它表示知识所进行的归结演绎推理易于在计算机上实现。

一阶谓词逻辑表示法有如下局限性。

(1) 不能表示不确定性的知识

现实世界中的许多知识并不总是只有“真”与“假”两种状态, 在“真”与“假”之间还存在许多中间状态, 即存在着“真”的程度问题, 知识的这一特性称为知识的不确定性。例如, “王林可能是计算机系的学生”就是一个不确定的事实, “如果头痛且流涕, 则可能患了感冒”是一个不确定的规则。

谓词逻辑只能表示精确性的知识, 不能表示不确定的知识, 但是, 由于人类的知识大多都不同程度地具有不确定性, 这就使得它表示知识的范围受到限制。另外, 谓词逻辑难以表示启发性知识, 所谓启发性知识是与被求解的问题的特性有关的知识。

(2) 组合爆炸

在谓词逻辑的推理过程中, 随着事实数目的逐步增大及盲目地使用推理规则, 有可能形成组合爆炸。目前已在这一方面做了大量的研究工作, 出现了一些比较有效的方法, 例如定义一种控制策略来选取合适的规则等。

(3) 效率低

用谓词逻辑表示知识时, 其推理是根据形式逻辑进行的, 把推理与知识的语义割裂开来, 这就使得推理过程冗长, 降低了系统的效率。

2.2 产生式表示法

产生式表示法又称为产生式规则表示法。“产生式”这一术语是由美国数学家波斯特(E.

Post) 在 1943 年首先提出来的。他根据串替代规则提出了一种称为波斯特机的计算模型, 模型中的每一条规则称为一个产生式。在此之后, 经过不断地修改和充实, 如今已被用到许多领域。例如用产生式来描述形式语言的语法, 表示人类心理活动的认知过程等。1972 年, 纽厄尔和西蒙在研究人类的认识模型中开发了基于规则的产生式系统。目前它已成为人工智能中应用最多的一种知识表示模式, 许多成功的专家系统都是用产生式方法来表示知识的。例如费根鲍姆等人研制的化学分子结构专家系统 DENDRAL、肖特里菲等人研制的诊断和治疗细菌感染性疾病的专家系统 MYCIN 等。

2.2.1 产生式与产生式系统

1. 产生式的基本形式

产生式通常用于表示具有因果关系的知识, 其基本形式是:

$$P \rightarrow Q$$

或者

$$\text{if } P \text{ then } Q$$

其中, P 是产生式的前提, 亦可称为前件、条件、前提条件, 用于指出该产生式是否可用的条件; Q 是产生式的结论或操作, 亦可称为后件, 用于指出当前提 P 所指示的条件被满足时, 应该得出的结论或应该执行的操作。整个产生式的含义是: 如果前提 P 被满足, 则可推出结论 Q 或执行 Q 所规定的操作。

谓词逻辑中的蕴含式与产生式的基本形式有相同之处, 但是, 蕴含式与产生式是有差别的。实际上, 通常可以把蕴含式看成是产生式的一种特殊情况, 这是因为:

1) 蕴含式只能表示精确知识, 其真值或者为真、或者为假; 而产生式不仅可以表示精确知识, 也可以表示不精确知识。例如, 在专家系统 MYCIN 中有这样一条产生式:

```

if      微生物的染色斑是革兰氏阴性
        微生物的形状呈杆状
        病人是中间宿主
then    该微生物是绿脓杆菌, 置信度为 0.6
  
```

它表示当前提中列出的 3 个条件都满足时, 结论“该微生物是绿脓杆菌”可以相信的程度为 0.6。这里用 0.6 指出了这一知识的知识强度, 但对于谓词逻辑中的蕴含式是不可以这样做的。

2) 用产生式表示在知识的系统中, 决定一条知识是否可用的方法是检查当前是否有已知事实可与前提中规定的条件匹配。这种匹配可以是精确的, 也可以是不精确的, 只要按某种算法求出的相似度在某个预先指定的范围内就认为是可匹配的。但对谓词逻辑的蕴含式来说, 要求匹配是精确的。

由于产生式与蕴含式的这些区别, 它们在处理方法及应用等方面都有较大的差别。

为了严格地描述产生式, 下面用巴科斯范式 BNF (Backus Normal Form) 给出产生式的形式描述及语义:

$\langle \text{产生式} \rangle ::= \langle \text{前提} \rangle \rightarrow \langle \text{结论} \rangle$
 $\langle \text{前提} \rangle ::= \langle \text{简单条件} \rangle | \langle \text{复合条件} \rangle$
 $\langle \text{结论} \rangle ::= \langle \text{事实} \rangle | \langle \text{操作} \rangle$
 $\langle \text{复合条件} \rangle ::= \langle \text{简单条件} \rangle \text{AND} \langle \text{简单条件} \rangle [(\text{AND} \langle \text{简单条件} \rangle) \cdots]$
 $\quad | \langle \text{简单条件} \rangle \text{OR} \langle \text{简单条件} \rangle [(\text{OR} \langle \text{简单条件} \rangle) \cdots]$
 $\langle \text{操作} \rangle ::= \langle \text{操作名} \rangle [(\langle \text{变元} \rangle, \cdots)]$

2. 产生式系统

把一组产生式放在一起，并让它们互相配合，协同作用；一个产生式生成的结论可以供另一个产生式作为已知事实使用，以求得问题的解决，这样的系统称为产生式系统。一个产生式系统由以下3个基本部分组成：规则库、综合数据库和控制机构。

(1) 规则库

用于描述相应领域内知识的产生式集合称为规则库。规则库是产生式系统进行问题求解的基础，其中的知识是否完整、一致，表达是否准确，对知识的组织是否合理等，不仅将直接影响到系统的性能，而且还会影响到系统的运行效率，因此对规则库的设计与组织应予以足够的重视。一般来说，在建立规则库时应注意以下问题：

1) 有效地表达领域内的过程性知识。规则库中存放的主要是过程性知识，用于实现对问题的求解。为了使系统具有较强的问题求解能力，除了需要获得足够的知识外，还需要对知识进行有效的表达。为此，需要解决如下一些问题：如何把领域中的知识表达出来，即为了求解领域内的各种问题需要建立哪些产生式规则？对知识中的不确定性如何表示？规则库建成后能否对领域内的不同求解问题分别形成相应的推理链，即规则库中的知识是否具有完整性？对于第一个问题可以从下面给出的一个典型例子中得到启发，其他的问题将在后面的章节中予以讨论。

例 2.5 建立一个动物识别系统的规则库，用以识别虎、豹、斑马、长颈鹿、企鹅、鸵鸟、信天翁等7种动物。

解 为了识别这些动物，可以根据动物识别的特征，建立包含下述规则的规则库：

R_1 : if 动物有毛发 then 动物是哺乳动物

R_2 : if 动物有奶 then 动物是哺乳动物

R_3 : if 动物有羽毛 then 动物是鸟

R_4 : if 动物会飞 and 会生蛋 then 动物是鸟

R_5 : if 动物吃肉 then 动物是食肉动物

R_6 : if 动物有犀利牙齿 and 有爪 and 眼向前方 then 动物是食肉动物

R_7 : if 动物是哺乳动物 and 有蹄 then 动物是有蹄类动物

R_8 : if 动物是哺乳动物 and 反刍 then 动物是有蹄类动物

R_9 : if 动物是哺乳动物 and 是食肉动物 and 有黄褐色 and 有暗斑点 then 动物是豹

R_{10} : if 动物是哺乳动物 and 是食肉动物 and 有黄褐色 and 有黑色条纹 then 动物是虎

R_{11} : if 动物是有蹄类动物 and 有长脖子 and 有长腿 and 有暗斑点 then 动物是长颈鹿

R_{12} : if 动物是有蹄类动物 and 有黑色条纹 then 动物是斑马

R_{13} : if 动物是鸟 and 不会飞 and 有长脖子 and 有长腿 and 有黑白二色 then 动物是鸵鸟

R_{14} : if 动物是鸟 and 不会飞 and 会游泳 and 有黑白二色 then 动物是企鹅

R_{15} : if 动物是鸟 and 善飞 then 动物是信天翁

由上述产生式规则可以看出, 虽然该系统是用来识别 7 种动物的, 但并不是简单地只设计 7 条规则分别直接用于识别 7 种动物。规则设计的基本思想是: 首先把动物划分为若干类, 如“哺乳动物”、“鸟”、“食肉动物”、“有蹄类动物”等, 根据“类”的识别特征建立若干条规则, 如规则 $R_1 \sim R_8$, 然后对属于各类的各个动物根据其个性的识别特征建立各自相应的规则, 如规则 $R_9 \sim R_{15}$ 。这样至少有两个好处, 一是当给出的已知事实不完全时, 虽然不能推出最终结论, 但可能会给出分类结果; 二是当需要增加对其他动物(如牛、马等)的识别要求时, 规则库中只需增加关于这些动物个性方面的知识, 对于规则库中已有的分类知识(如规则 $R_1 \sim R_8$)就可以直接使用。

2) 对知识进行合理的组织与管理。对规则库中的知识进行适当的组织, 采用合理的结构形式, 可使推理避免访问那些与当前问题求解无关的知识, 从而提高问题求解的效率。

例如, 对例 2.5 的规则库, 可根据“哺乳动物”和“鸟”这两类动物的识别规则将 15 条规则分为两个子集:

$\{R_1, R_2, R_5, R_6, R_7, R_8, R_9, R_{10}, R_{11}, R_{12}\}$

$\{R_3, R_4, R_{13}, R_{14}, R_{15}\}$

当待识别动物的识别过程一旦开始使用其中某一个子集的规则时, 就可以只使用该子集中的规则完成推理过程, 而无需在另一个子集中去查找所需要的规则, 从而减少查找规则的时间。当然, 这种划分还可以逐级进行下去, 使得相关的知识构成一个子集或子子集, 组成一个层次型的规则库。

(2) 综合数据库

综合数据库又称全局数据库, 或称事实库、黑板、上下文等。综合数据库用于存放问题求解过程中各种当前信息, 例如问题的初始事实、原始证据、推理中得到的中间结论(如上例中的“动物是哺乳动物”、“动物是鸟”等)以及最终结论(如上例中的“动物是虎”等)。当规则库中某一条产生式的前提可与综合数据库中的某些已知事实匹配时, 该产生式就被激活, 并把用它推出的结论放入综合数据库中, 作为其后推理的已知事实。可见, 综合数据库的内容随着推理的进行是在不断动态变化的。

综合数据库中的已知事实通常用字符串、向量、集合、矩阵、表等数据结构表示。例如, 在专家系统 MYCIN 中, 通常用如下一个四元组表示事实:

(特性 对象 值 可信度因子)

其中, “可信度因子”是指对该事实为真的相信程度。例如对事实“动物 A 可能会飞”, 可以用四元组表示为

(动物飞翔 动物 A 会飞 0.5)

这里用可信度因子 0.5 表示对“动物 A 会飞”的可信程度,反映了由修饰词“可能”表示出来的这一事实的不确定性。

(3) 控制机构

控制机构又称为推理机构或推理机,由一组程序组成,实现对问题的推理求解。推理机主要做以下几项工作:

1) 按某种策略从规则库中选择规则与综合数据库中的已知事实进行匹配。所谓匹配是指把规则的前提条件与综合数据库中的已知事实进行比较,如果两者一致,或者近似一致且满足预先规定的近似程度,则称匹配成功,相应规则可被使用;否则称为匹配不成功,相应规则不可用于当前的推理。

2) 若匹配成功的规则有多条,则称为发生冲突。此时,推理机必须调用执行某种冲突消解策略的程序,从中选出一条匹配成功的规则来执行。

3) 在执行某一条规则时,如果该规则的后件是一个或多个结论,则把这些结论添加到综合数据库中;如果规则的后件是一个或多个操作,则依序执行这些操作。

4) 对于不确定性知识,在执行每一条规则时还要按一定的算法来计算结论的不确定性。

5) 随时检查结束推理机运行的条件,在满足结束条件时停止推理机的运行。

以上各项都有许多工作要做,将在后面有关章节中详细讨论。

产生式系统问题求解的一般步骤是:

1) 初始化综合数据库,把求解问题的初始已知事实送入综合数据库。

2) 若规则库中存在尚未使用过的规则,而且该规则的前件可与综合数据库中的已知事实匹配,则转步骤 3);若综合数据库中不存在匹配所需要的事实,则转步骤 5)。

3) 执行当前选中的规则,并对该规则做上使用标记,把该规则后件的结论送入综合数据库中。若该规则后件是操作,则执行这些操作。

4) 检查综合数据库中是否已包含了问题的解,若已包含,则终止问题的求解过程;否则转步骤 2)。

5) 要求用户提供关于问题的其他已知事实,若提供了新的事实,则转步骤 2);否则终止问题求解过程。

6) 若规则库中不再有未使用过的规则,则终止问题求解过程。

可见,问题的求解过程是一个不断地从规则库中选取可用规则与综合数据库中的已知事实进行匹配的过程,规则的每一次成功匹配与执行都使综合数据库增加了新的事实,并向着问题的求解前进了一步,这一过程称为推理。

最后需要说明的是,问题求解的过程与推理的控制策略有关,上述步骤只是针对正向推理方式给出的一般步骤,许多细节均未涉及,如冲突消解、不确定性的处理等。

2.2.2 产生式系统的分类及其特点

从不同角度对产生式系统进行分类,可得出不同的分类结果。按产生式所表示的知识是否具有确定性,产生式系统可分为确定性产生式系统和不确定性产生式系统;按推理机的推理方向它又可分为正向、反向和双向推理产生式系统。这里,仅讨论按规则库及综合数据库的性质与结构特征进行的分类,产生式系统可分为三类:可交换的产生式系统、可分解的产生式系统和可恢复的产生式系统。

1. 可交换的产生式系统

产生式系统求解问题的推理过程是一个反复从规则库中选择合适规则并执行规则的过程。在求解过程中,采用不同的控制策略将会得到不同的规则执行次序,从而反映出不同的求解效率。如果一个产生式系统对规则的使用次序是可交换的,那么无论先使用哪一条规则,都可达到目的,即规则的使用次序对问题的最终求解是无关紧要的。我们称这样的产生式系统为可交换的产生式系统。

首先来看一个简单的例子。设综合数据库 DB 的初始状态是 {a,b,c}, 并设规则库 RB 中有下述规则:

R_1 : if {a,b,c} then {a,b,c, a×b}

R_2 : if {a,b,c} then {a,b,c, b×c}

R_3 : if {a,b,c} then {a,b,c, a×c}

现在希望通过推理使综合数据库 DB 中的内容变为:

{a,b,c,a×b,b×c,a×c}

显然,这三条规则各被使用一次后就可达到目的,且与规则使用的次序无关。所以由上述 RB 和 DB 构成的产生式系统是一个可交换的产生式系统。

一个产生式系统是可交换的,是指定的 RB 和 DB 都具有如下性质:

1) 设 RS 为可应用于 DB 的状态 DB_i 的规则集合,当使用 RS 中任何一条规则 R 使 DB 的状态由 DB_i 改变为 DB_{i+1} 后,该 RS 仍然是 DB_{i+1} 的可用规则集。

2) 如果 DB_i 满足目标条件,则当应用 RS 中任何一条规则所生成的 DB 的新状态 DB_{i+1} 仍然满足目标条件。

3) 若对当前状态 DB_i 使用某一规则序列 $R_1, R_2, \dots, R_k$ 得到 DB 的一个新的状态 DB_{i+k} , 即

$$DB_i \xrightarrow{R_1} DB_{i+1} \xrightarrow{R_2} \dots \xrightarrow{R_k} DB_{i+k}$$

则当改变规则的使用次序后,仍然可得到 DB_{i+k} 。

由以上性质可以看出,在可交换产生式系统中,综合数据库 DB 中的内容是递增的,即对任意一个规则执行序列 $R_1, R_2, \dots, R_k$ 都有

$$DB_i \subseteq DB_{i+1} \subseteq \dots \subseteq DB_{i+k}$$

由此可见,在可交换产生式系统中,被执行的产生式规则后件总是应包含着 DB 当前状态未包含的新的结论,且不应包含删除 DB 中已有事实的操作。还可看出,用可交换产生式

系统求解问题时, 推理过程不必进行回溯。所谓回溯是指: 当执行一条规则使 DB 的状态由 DB_i 变为 DB_{i+1} 时, 如果发现由 DB_{i+1} 不可能得到问题的解, 就需要立即撤消由刚才执行规则所产生的结果, 使 DB 恢复到先前的 DB_i 状态, 然后选用别的规则继续求解。由于求解问题时只需选用任何一个规则序列, 而不必搜索多个序列, 这就节省了问题求解的时空开销, 提高了求解的效率。

2. 可分解的产生式系统

把一个规模较大且较复杂的问题分解为若干个规模较小且较简单的子问题, 然后对每个子问题分别进行求解, 这是人们求解问题时常用到的方法, 可分解的产生式系统就是基于这一思想提出来的。

一个产生式系统可分解的要求是综合数据库 DB 的当前状态 DB_i 可被分解为若干个独立的部分: $DB_i^1, DB_i^2, \dots, DB_i^m$, 且根据 DB 状态确定的推理过程的终止条件也可被分解为对这些独立部分进行推理的终止条件。例如, 设综合数据库 DB 的初始内容为 $DB_0 = \{D, B, Z\}$, 规则库 RB 中有如下规则:

R_1 : if C then {D,L}

R_2 : if C then {B,M}

R_3 : if B then {M,M}

R_4 : if Z then {B,B,M}

终止条件是生成只包含 M 的综合数据库, 即使综合数据库的内容变为

$\{M, M, \dots, M\}$

由于规则库 RB 中的每条规则的前件都只含有单一条件 C 或 B 或 Z, 因此可把综合数据库 DB 的初始状态 $DB_0 = \{C, B, Z\}$ 分解为独立的三个部分: $DB_0^1 = \{C\}$, $DB_0^2 = \{B\}$, $DB_0^3 = \{Z\}$ 。而且每次对 DB 中的一部分执行一条规则使其状态为 DB_i 后, 若 DB_i 中有 m 个事实, 则把 DB_i 分成 m 个独立部分: $DB_i^1, DB_i^2, \dots, DB_i^m$, 每一部分都只含有单一事实。若含有的单一事实是 M, 则对这一部分状态不再使用规则; 否则继续使用合适规则。其求解过程如图 2.1 所示。

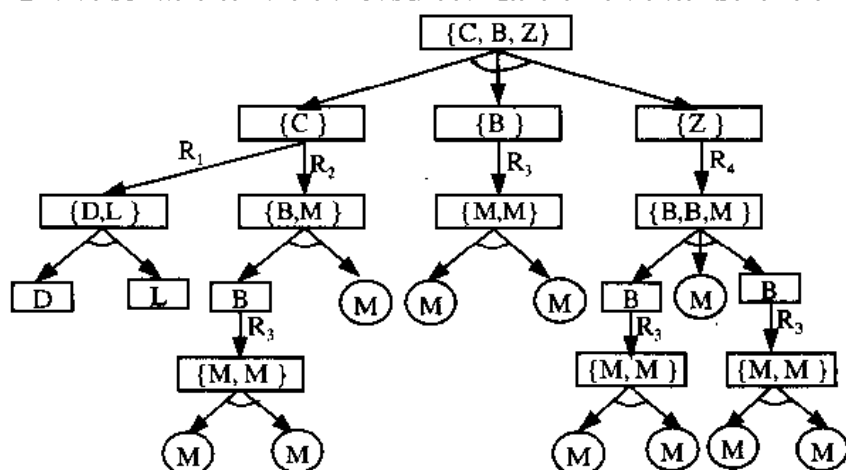


图 2.1 可分解的产生式系统示例

在图 2.1 中,用圆弧连接起来的子节点是“与”关系,不用圆弧连接的子节点是“或”关系。用图表示可分解产生式系统求解问题的过程时,得到的是一棵与/或树。

在可分解产生式系统中,由于综合数据库被分解成若干子库,随着推理过程的进行,有关的子库又可分解成若干子子库,依次类推,从而减小了在规则库搜索规则和与数据库当前状态进行匹配的工作量,可提高问题求解的效率。

3. 可恢复的产生式系统

在可交换产生式系统中,要求每条规则的执行只能为综合数据库增添新的内容,且不能删除和修改综合数据库已有的内容。这一要求是很强的,在许多规则的设计中难以达到。因此就需要产生式系统具有回溯功能,一旦问题求解到某一步发现无法继续下去时,就撤消在此之前得到的某些结果,恢复到先前的某个状态,然后选用别的规则继续求解。在问题求解过程中既可以对综合数据库添加新内容、又可删除或修改老内容的产生式系统称为可恢复的产生式系统。有关可回溯的搜索策略将在后面的章节中详细讨论。

4. 产生式表示法的特点

产生式表示法主要有以下优点。

(1) 自然性

产生式表示法用“如果……则……”的形式表示知识,这是人们常用的一种表达事物因果关系的知识表示形式,既直观、自然,又便于进行推理。正因为这一原因,使得产生式表示法成为人工智能中最重要且应用最多的一种知识表示模式。

(2) 模块化

产生式是规则库中最基本的知识单元,规则库与推理机相对独立,而且每条规则都具有相同的形式,这就便于对规则库进行模块化处理,为知识的增、删、改带来方便。

(3) 有效性

产生式表示法既可表示确定性知识,又可表示不确定性知识;既有利于表示启发式知识,又可方便地表示过程性知识;既可表示领域知识,又可表示元知识。因此,产生式表示法可以把专家系统中需要的多方面的知识用统一的知识表示模式有效地表示出来。这也是目前已建造成功的专家系统大多采用产生式表示法的一个重要原因。

(4) 清晰性

产生式有固定的格式,每一条产生式规则都由前提与结论(操作)两部分组成。产生式规则具有所谓自含性,即一条产生式规则仅仅描述该规则的前提与结论之间的静态的因果关系,且所含的知识量都比较少。这就便于对规则进行设计和保证规则的正确性,同时,也便于在知识获取时对规则库进行知识的一致性和完整性检测。

产生式表示法也有以下不足之处。

(1) 效率不高

在产生式系统问题求解过程中,首先要从规则库中选出可与综合数据库当前状态匹配的可用规则,若可用规则不止一个,就需要按某种冲突消解策略从中选出一条规则来执行。因

此,产生式系统求解问题的过程是一个“匹配——冲突消解——执行”反复进行的过程。由于规则库一般规模较大,拥有的规则条数较多,而规则匹配又是一件十分费时的工作,因此产生式系统的工作效率是不高的。另外,在求解复杂问题时容易引起组合爆炸。

(2) 不能表达具有结构性的知识

产生式适合于表达具有因果关系的过程性知识,但对具有结构关系的知识却无能为力,它不能把具有结构关系的事物之间的结构联系表示出来。因此,产生式除了可以独立作为一种知识表示模式外,还常与其他可表示知识的结构关系的表示法结合起来使用。例如,在专家系统 PROSPECTOR 中把产生式与语义网络相结合,在 Aikins 中把产生式与框架表示相结合等。

2.3 框架表示法

1975 年,美国的人工智能学者明斯基首先提出框架理论,该理论认为人们对现实世界中各种事物的认识都是以一种类似于框架的结构存储在记忆中的,当面临一个新事物时,就从记忆中找出一个合适的框架,并根据实际情况对其细节加以修改、补充,从而形成对当前事物的认识。世界上的各类事物都各自具有不同的属性,而不同事物的属性之间往往具有一定的规律性的联系。这种规律性的知识经过提炼,就可以形成人们认识某一类事物的一种固定的框架。框架表示法就是用来表示这种经验性知识的一种知识表示方法。

2.3.1 框架与框架网络

框架是描述对象(一个事物、一个事件或一个概念)属性的一种数据结构,在框架表示法中,框架被看成是知识表示的基本单位。不同的框架之间可以通过属性之间关系建立联系,从而构成一个框架网络,充分表达相关对象间的各种关系。

1. 框架

一个框架由若干个被称为“槽”的结构组成,每一个槽又可根据实际需要分为若干个“侧面”。一个槽用于描述对象的某一方面的属性,一个侧面用于描述相应属性的一个方面。槽和侧面所具有的属性值分别称为槽值和侧面值。在一个用框架表示知识的系统中都含有多个框架,需要给它们赋予不同的框架名。同样,对一个框架内的不同槽和不同侧面也需要分别赋予不同的槽名和侧面名。

一个框架可以形式地表示如下:

<框架名>

槽名 1: 侧面名 11: 侧面值 11

侧面名 12: 侧面值 12

⋮

侧面名 1n: 侧面值 1n

⋮
槽名 k: 侧面名 k1: 侧面值 k1
 侧面名 k2: 侧面值 k2
 ⋮
 侧面名 km: 侧面值 km

下面给出框架的 BNF 描述:

```
<框架> ::= <框架头> <槽部分> [<约束部分>]
<框架头> ::= 框架名 <框架名的值>
<槽部分> ::= <槽>, [<槽>]
<约束部分> ::= 约束 <约束条件>, [<约束条件>]
<框架名的值> ::= <符号名> | <符号名> (<参数>, [<参数>])
<槽> ::= <槽名> <槽值> | <侧面部分>
<槽名> ::= <系统预定义槽名> | <用户自定义槽名>
<槽值> ::= <静态描述> | <过程> | <谓词> | <框架名的值> | <空>
<侧面部分> ::= <侧面>, [<侧面>]
<侧面> ::= <侧面名> <侧面值>
<侧面名> ::= <系统预定义侧面名> | <用户自定义侧面名>
<侧面值> ::= <静态描述> | <过程> | <谓词> | <框架名的值> | <空>
<静态描述> ::= <数值> | <字符串> | <布尔值> | <其他值>
<过程> ::= <动作> | <动作>, [<动作>]
<参数> ::= <符号名>
```

对框架的 BNF 有如下几点说明:

- 1) 框架名的值允许带有用符号名表示的参数。当一个框架 A 调用另一个带有符号名参数的框架 B 时, 框架 A 需要为框架 B 提供相应参数的实在参数。
- 2) 当槽值或侧面值是一个过程时, 它既可以是一个明确表示出来的<动作>串, 也可以是对程序语言编制的某个过程的调用, 从而可将过程性知识表示出来。
- 3) 当槽值或侧面值是谓词时, 谓词的真值由当时谓词中变元的取值确定。
- 4) 当槽值或侧面值为<空>时, 表示该值还不能确定, 等待以后填入。
- 5) <约束条件>是任选的, 约束条件可以为框架附加上一些说明性的信息, 也可以用于指出什么样的值才能填入到槽或侧面中去。当不指出约束条件时, 表示没有约束。

2. 框架网络

由于框架中的槽值或侧面值都可以是另一个框架的框架名, 这就可在框架之间建立联系, 通过一个框架可以找到另一个框架。共处于某种环境中的若干对象必然会有某些共同的属性, 在对这些对象进行描述时, 可以把它们具有的共同属性抽取出来, 构成一个上层框架, 然后再对各类对象独有的属性分别构成若干个下层框架。为了指明框架之间的这种上下关系,

可在下层框架中设立一个专用的槽（一般称之为“继承”槽），用以指出它的上层框架是哪一个。下层框架可以继承上层框架的属性及值，这样就可以把一组有上下关系的框架组成具有层次结构的框架网络。通过框架网络，可以避免对相关对象的重复描述，节约了时间和空间的开销。

继承性是框架表示法的一个重要特性，它不仅可以在相邻的上、下两层框架之间实现继承，而且可以从最低层追溯到最高层，使高层框架的描述信息逐层向低层框架传递。

我们来看下述的一个例子。

例 2.6 建立一个分层的框架网络，框架网络从最高层框架至最低层框架的框架名依序为：<师生员工>，<教职工>，<教师>，<教师 1>，并为相应框架设置继承槽来避免重复描述。

师生员工框架为：

框架名：<师生员工>

姓名：单位（姓，名）

年龄：单位（岁）

性别：范围（男，女）

缺省：男

健康状况：范围（健康，一般，差）

缺省：一般

住房：<住房>

教职工框架为：

框架名：<教职工>

继承：<师生员工>

工作类别：范围（教师，干部，工人）

缺省：教师

学历：范围（中专，大专，本科，研究生）

缺省：本科

参加工作时间：单位（年，月）

教师框架为：

框架名：<教师>

继承：<教职工>

部门：单位（系，教研室）

语种：范围（英语，法语，德语，日语，俄语）

缺省：英语

职称：范围（教授，副教授，讲师，助教）

缺省：讲师

某个教师的实例框架为：

框架名: <教师 1>

继承: <教师>

姓名: 王林

年龄: 36

健康状况: 健康

参加工作时间: 1982, 9

部门: 计算机系软件教研室

职称: 副教授

由上述框架描述可以看出:

1) 在框架网络中, 既有用“继承”槽指出上、下层框架之间的纵向联系, 也有以框架名作为槽值指出的框架之间的横向联系。例如, 在师生员工框架中, 对槽名为“住房”的槽填入的槽值是一个框架名: <住房>, 从而在<师生员工>框架与<住房>框架之间建立了横向联系的框架体系结构。

2) 实例框架中的每一个槽都应给出槽值, 并可以继承上层框架槽的槽值, 从而获得实例框架中没有直接给出的知识。例如, 在实例框架<教师 1>中, 虽然没有给出“性别”、“工作类别”、“学历”、“语种”槽及其槽值, 但由继承性可得知: 王林的性别是“男”, 工作类别是“教师”, 学历是“本科”, 语种是“英语”。

3) 以框架作为知识表示模式时, 知识是通过事物的属性表示的。为满足问题求解的需要, 就要求框架中有足够的槽把事物有关方面的属性充分表达出来。但是, 一个事物的属性通常都是多方面的, 在选择把哪些属性作为一个框架的槽的描述对象时, 要根据系统的设计目标和属性之间的联系对事物的属性进行筛选和分类, 仅需要对有关的属性设立槽, 不可面面俱到, 以免浪费空间和降低系统的运行效率。

在框架系统中, 事物之间的联系是通过在槽中填入相应的框架名来实现的, 至于它们之间的关系可以由槽名来指明。在框架表示法中, 对于一些常用且可公用的槽名给出了标准槽名及其定义, 称这些槽名为系统预定义槽名。下面给出其中使用较多的系统预定义槽名。

(1) ISA 槽

ISA 槽用于指出事物之间抽象概念上的类属关系, 其含义是“是一个”、“是一种”、“是一只”等。当用它作为某下层框架的槽时, 表示该下层框架所描述的事物是其上层框架的一个特例, 上层框架是比下层框架更一般或更抽象的概念。或者说, 框架可以用一个 ISA 槽指出它的父框架。

例 2.7 以下是一段关于某次地震的报道: 1976 年 3 月 18 日, 一次强度为里氏 8.5 级的强烈地震发生在下斯洛文尼亚 (Low Slabovia) 地区, 造成 25 人死亡和 5 亿美元的财产损失。多年来, 靠近萨迪豪金斯 (Sadie Hawkins) 断层的地区一直是一个地震多发区。

请用分层的框架网络表示上述描述事实。

由于地震可看成是一种灾害事件, 除地震之外, 还有火灾、洪水、飓风等灾害事件, 因

此可组成一个描述灾害事件的框架系统。对于上述的地震报道，可以用如下框架来描述：

框架名：<灾害事件>

地点：字符串

日期：单位（年，月，日）

死亡人数：单位（人）

财产损失：单位（亿美元）

框架名：<地震>

ISA：<灾害事件>

强度：单位（里氏级）

断层：字符串

框架名：<地震 1>

ISA：<地震>

地点：Low Slabovia

日期：1976, 3, 18

死亡人数：25

财产损失：5

强度：8.5

断层：Sadie Howkins

用“ISA”槽所指出的联系都具有继承性，即下层框架可继承其上层框架所描述的属性及其属性值。即<地震>框架可继承其父框架<灾害事件>框架的槽及槽值，<地震 1>框架不仅可继承其父框架<地震>框架的槽及槽值，而且由<地震>框架的继承性，还可继承<灾害事件>框架的属性描述。

（2）AKO 槽

AKO 槽用于具体地指出事物间的类属关系，其含义是“是一种”。当用它作为某下层框架的槽时，就明确地指出该下层框架所描述的事物是其上层父框架所描述事物中的一种，下层子框架可以继承其上层父框架所描述的属性及值。

对上面的例子，可以把框架<地震>和<地震 1>中的槽名“ISA”改为“AKO”。

（3）Subclass 槽

Subclass 槽用于指出子类与类（或子集与超集）之间的类属关系。当用它作为某下层框架的槽时，表示该下层框架是其上层父框架的一个子类（或子集）。对于上例，由于地震灾害是灾害事件中的一个子类，因而可把<地震>框架中的槽名“ISA”改为“Subclass”。同样也可把<地震 1>框架中的槽名“ISA”改为“Subclass”。

（4）Instance 槽

Instance 槽用来建立 AKO 槽的逆关系。当用它作为某上层框架的槽时，用来指出它的下一层子框架是哪一些框架，即 Instance 槽的槽值是它下一层框架的框架名。对于上例，考虑到

需要描述的危害事件除了地震外, 还有其他类型的灾害, 若在框架系统中还给出了火灾、洪水、飓风的描述框架, 则可在<灾害事件>框架中用 Instance 槽来指出它有这些下层子框架, 即:

框架名: <灾害事件>

Instance: <地震>, <火灾>, <洪水>, <飓风>

地点: 字符串

日期: 单位 (年, 月, 日)

死亡人数: 单位 (人)

财产损失: 单位 (亿美元)

由上层框架中的 Instance 槽所建立起来的上、下层框架间的联系具有继承性, 即由 Instance 槽的槽值指出的下层子框架可以继承上层父框架所描述的属性及值。

(5) Part-of 槽

Part-of 槽用于指出“部分”与“全体”的关系。当用它作为某下层框架的槽时, 它指出该下层框架所描述的事物只是其上层框架所描述事物的一部分。例如, 上层框架是对变速箱的描述, 其下层的一个框架是对变速箱齿轮的描述, 齿轮只是变速箱的一个组成部分。

Part-of 槽只是指出下层框架是其上层框架的一个子结构, 两者一般不具有共同的特征, 因此, 下层框架不能继承上层框架所描述的属性及值。例如, <齿轮>是变速箱的一部分, 但变速箱的结构及属性与齿轮的结构及属性完全不同, <齿轮>框架不能继承<变速箱>框架的属性及值。

在前述的例 2.6 的<师生员工>框架中, 对槽名为“住房”填入的槽值是<住房>, 可以用 Part-of 槽来说明<住房>框架是<师生员工>框架的一部分, 即:

框架名: <住房>

Part-of: <师生员工>

地址: 单位 (公寓名, 栋, 号, 单元号)

建筑面积: 单位 (平方米)

户型: 范围 (二房一厅, 三房一厅, 三房二厅, 四房二厅)

缺省: 三房一厅

显然, <住房>框架与<师生员工>框架的结构与属性完全不同, <住房>框架不能承继<师生员工>框架的属性及值。

(6) Infer 槽

Infer 槽用于指出两个框架所描述事物之间的逻辑推理关系, 用它可以表示产生式规则。

例 2.8 用框架表示下述知识: 如果咳嗽、发烧且流涕, 则八成是患了感冒, 需服用感冒清, 一日 3 次, 每次 2 粒, 多喝开水。

对该知识, 可用如下两个框架表示:

框架名: <诊断规则 1>

症状 1: 咳嗽

症状 2: 发烧

症状 3: 流涕

Infer: <结论 1>

可信度: 0.8

框架名: <结论 1>

病名: 感冒

治疗方法: 服用感冒清, 一日 3 次, 每次 2 粒

注意事项: 多喝开水

(7) Possible-Reason 槽

Possible-Reason 槽与 Infer 槽的作用相反, 它用来把某个结论与可能的原因联系起来。例如, 在上述的<结论 1>框架中可增加一个 Possible-Reason 槽, 其槽值是某个框架的框架名, 在该框架中描述产生“感冒”的原因, 如感染了流感病毒等。

除了上述 7 种描述框架层次结构关系、推理关系与解释结论的槽之外, 还有一些描述其他关系的槽, 如占有关系、时间关系、相似关系等, 这里不再一一列出。

2.3.2 框架的推理及其特点

用框架表示知识的系统主要由两大部分组成: 一是由框架网络构成的知识库; 二是由一组程序构成的框架推理机。前者的作用是提供求解问题所需要的知识, 后者的作用是针对用户提的问题, 运用知识库中的知识完成问题求解。

1. 框架推理的基本过程

在用框架表示知识的系统中, 推理主要是通过框架匹配与填槽来实现的。首先把要求解的问题用一个称为问题框架的框架表示出来, 然后把初始问题框架与知识库中已有的框架进行匹配。框架的匹配是把两个框架相应的槽名及槽值逐个进行比较, 如果两个框架的各对应槽没有矛盾或满足预先规定的某些条件, 就认为这两个框架可以匹配。由于框架之间存在继承关系, 一个框架所描述的某些属性及值可能是从它的上层框架继承过来的, 因此两个框架的比较往往要牵涉到它们的上层、上上层框架。

我们来看一个例子。假设例 2.6 提出的关于师生员工的框架网络已建立在知识库中, 当前要求解的问题是从知识库中找出一个满足如下条件的教师: 男性, 年龄在 40 岁以下的副教授, 身体健康, 会英语。把求解问题可表示成如下初始问题框架:

框架名: <教师 x >

姓名:

年龄: <40

性别: 男

健康状况: 健康

职称: 副教授

语种：英语

在初始问题框架中，姓名槽的槽值是空的，反映了需要求解的问题，有待推理过程中填入合适的槽值，从而求出问题的解。

用此问题框架与知识库中的框架匹配，显然<教师 1>框架可以与之匹配，因为“年龄”槽、“健康状况”槽与“职称”槽的槽值都符合要求，<教师 1>框架虽然没有给出“性别”槽与“语种”槽的槽值，但由继承性可知，<教师 1>框架继承有“性别”槽，且槽值是“男”，还继承有“语种”槽，且槽值是“英语”。由框架匹配可知要找的教师可能就是王林。这里之所以说“可能”，是因为知识库中可与问题框架<教师 x>匹配成功的框架可能不止一个，因而目前匹配成功的框架还只能作为预选框架，需要进一步收集信息，以便从中选出一个，或者根据框架中其他槽的内容以及框架间的关系明确下一步查找的方向和线索。

2. 框架的不确定性匹配

如果能找到这样一个框架，使得两个框架的对应槽值完全一致，则称这两个框架是完全匹配，或称为确定性匹配；如果两个框架虽然不能使对应槽值完全一致，但却满足预先指定的条件，则称这两个框架是不完全匹配，或称为不确定性匹配。

一般来说，一个框架所描述的通常是一类事物中典型的或共性的属性，不可能与该类事物中的每一个具体事物都完全一致。另外，一个框架被定义后放入知识库中，它的结构就相对地固定下来，而现实世界中的事物是千变万化的，这也是两者难以完全一致的原因。例如，设有一个<教室>框架，它抽取了各种教室的共同属性后形成关于教室的完整描述，但它不一定与每一个具体的教室的描述都完全一致。事实上，即使在同一个学校内，教室也不一定是完全一致的。再如，一把椅子一般有 4 条腿，但对某个具体的椅子，它与<椅子>框架中其他属性的描述完全一致，但它只有 3 条腿，不能因为它与<椅子>框架不完全一致就认为它们不匹配，从而不承认 3 条腿的椅子是椅子。由此可见，不确定性匹配在框架推理中的重要性。

框架的不确定性匹配的主要方法有以下几种。

(1) 匹配度方法

框架的匹配度是指当前框架所描述的属性与已知框架可匹配的程度。匹配度方法的基本思想是：首先求出两个框架匹配的匹配度，然后再用该匹配度与预先设定的框架匹配阈值进行比较，若能满足阈值条件（如大于等于阈值）就认为两个框架可匹配，否则为不可匹配。这里，关键的问题是如何求匹配度，这个问题与框架系统中如何表示不确定性知识及推理方法有关。假设框架中对于各个属性值都是用模糊集表示的模糊概念、模糊数或模糊语言值描述的，则可通过下述步骤求两个框架的匹配度：

1) 设 $A = \{A_1(\mu_1), A_2(\mu_2), \dots, A_n(\mu_n)\}$ 是当前框架各属性值的模糊集， μ_i 是属性值 A_i 的可信度。 $A' = \{A'_1(\mu'_1), A'_2(\mu'_2), \dots, A'_n(\mu'_n)\}$ 是已知框架各属性值的模糊集， μ'_i 是属性值 A'_i 的可信度。若 A_i 与 A'_i 一致， $i=1, 2, \dots, n$ ，则称 A 与 A' 的匹配距离为：

$$d = \max\{|\mu_1 - \mu'_1|, |\mu_2 - \mu'_2|, \dots, |\mu_n - \mu'_n|\}$$

2) A 与 A' 的匹配度为：

$$t = \min\{1 - d, \mu_i' | i = 1, 2, \dots, n\}$$

3) 设 τ 为两个框架的可匹配阈值, 若 $t \geq \tau$, 则称 A 与 A' 可匹配; 否则不可匹配。

上述方法只是求匹配度的一种方法。在后面的章节中将详细讨论不确定性知识的表示与匹配以及推理方法。

另外, 也有一些系统采用了某些简单有效的匹配度方法来确定两个框架的匹配度。例如, 框架中的各个属性有不同的重要性, 则可根据各个属性的重要程度赋予不同的权值, 每一对匹配上的属性就取得相应的权值, 然后通过某种运算 (如取最大值或取最小值或求平均值, 或求加权平均值等) 求得两个框架的匹配度。若框架匹配度满足框架匹配阈值的要求, 则两个框架是可匹配的。

(2) 充分条件与必要条件方法

为了确定当前框架是否可与已知框架匹配, 可将框架中的某些槽分别设定为“充分条件”槽和“必要条件”槽, 分别指出两个框架可匹配的充分条件和必要条件。如果充分条件可以得到满足, 就认为相应的两个框架可以匹配; 如果充分条件得不到满足, 但满足必要条件, 则需要进一步收集信息后再进行匹配; 如果必要条件不满足, 就认为这两个框架不可匹配。

在框架中设置“充分条件”槽与“必要条件”槽, 不仅较好地解决了框架的匹配问题, 而且在框架推理中可把框架分为确认、激活与沉睡三种状态, 有利于确定推理的方向和对框架的搜索。

(3) 规定属性值变化范围方法

对于某些事物的某些属性, 可通过对其值规定一个变化范围来实现属性的不确定性匹配。例如, 可规定教室的门的数量为 1~3, 规定椅子的腿数为 1~4 等。只要一个具体事物的属性值落在规定的范围内, 就认为这个属性是匹配的。

(4) 功能属性描述方法

对于某些事物除了给出有关外形的属性描述之外, 还可以给出其功能的属性描述, 而且功能属性的重要程度还应高于外形属性, 这将有损于实现框架的不确定性匹配。如果能够在 <椅子> 框架中给出椅子的功能属性, 那么即使对仅有一条腿的椅子, 只要它具有椅子的功能, 仍可把它认为是椅子。

以上讨论了 4 种实现不确定性匹配的具体方法, 可根据实际情况把它们组合起来在同一个系统中应用, 以起到互补的作用。

3. 框架推理

在用框架表示知识的系统中, 通过框架中的 AKO 槽和 Instance 槽把框架连接起来, 构成的框架网络是一个层次结构。框架推理是以此层次结构为基础, 按照一定的搜索策略, 不断寻找可匹配的框架并进行填槽的过程。在此过程中, 有可能找到了合适的框架, 得到了问题的解而成功结束, 也可能因找不到合适框架而被迫终止。这里, 我们按深度优先搜索策略讨论自顶向下进行搜索的框架推理过程。框架推理的步骤如下:

1) 把用户要求解的问题形成一个初始问题框架, 并将已知的知识 (一般为该问题中一

些已知的事实或数据)填入到相应槽中,在此框架中,有些槽是空的,它反映了需要解决的问题,有待推理过程填入合适的值。槽的排列顺序可以与知识库中相应框架中的槽的顺序不相同,但一般要求问题框架中的槽名(特别是关键性的槽名)应包含在知识库的相应框架中。

2) 把框架网络的根框架作为当前框架,即以根框架作为搜索推理的起点。

3) 把问题框架与当前框架进行匹配,即对两个框架除空槽以外的相应槽逐个地进行确定性匹配,或不确定性匹配。如果两个框架能够满足确定性匹配或不确定性匹配的条件,则转步骤4)进行填槽;否则转步骤5)搜索下一个框架。在对两个框架进行匹配的过程中,允许当前框架的某些槽及槽值是从其上层框架继承的。

4) 把当前框架中相应槽的槽值填入到问题框架的对应空槽中,并可以把当前框架从其上层框架继承的槽值或者通过与用户交互得到的槽值填入问题框架的对应空槽中(填槽的过程类似于程序设计语言中对变量进行赋值的过程)。在填槽后,检查问题框架是否已经包含了问题的解,即反映求解问题的空槽是否已填入槽值。若已包含,则转步骤7);否则转步骤5)。

5) 按当前框架的 Instance 槽的槽值找一个尚未进行过匹配操作的子框架。若有这样的子框架,则把该子框架作为当前框架,转步骤3)进行匹配及填槽操作,此时,原已填入到问题框架中的槽值有可能被当前新的填槽操作所修改。若没有这样的子框架,则转步骤6)进行回溯。

6) 由当前框架的 AKO 槽的槽值找到它的父框架。如果该父框架不是根框架,则把该父框架作为当前框架,并转步骤5)。如果该父框架是根框架且有未进行匹配操作的子框架,则把该根框架作为当前框架,并转步骤5);否则,表示由该父框架为根框架的框架网络已不可能求得问题的解,需要另外再选一个根框架重复进行上述推理过程,如果已没有合适的根框架可选,则表示问题无解,结束推理过程。

7) 如果问题的解具有不确定性,则根据采用的不确定性知识表示方法计算解的不确定性度量,成功地结束推理过程。

上述过程实际上是沿 Instance 槽的自顶向下搜索和沿 AKO 槽的回溯组成的深度优先搜索,这只是框架推理的搜索策略之一。有关搜索策略的问题将在后面的章节中详细讨论。

由于槽值可以是满足指定条件时可触发执行的一个动作或过程,因此,在推理过程中,由于执行了某些动作或过程可能会增加或修改框架中已有的知识。

4. 框架表示法的特点

框架表示法有以下主要特点。

(1) 结构性

框架表示法最突出的特点是它善于表示结构性的知识,能够把知识的内部结构关系及知识之间的联系表示出来,因此它是一种经过组织的结构化的知识表示方法。这一特点是产生式表示法所不具备的,产生式系统中的知识的组织单位是产生式规则,这种知识组织单位由于太小而难于处理复杂问题,也不能把知识之间的结构关系显式地表示出来。框架表示法的知识组织单位是框架,而框架由若干槽组成,槽又由若干侧面组成,这就可把知识的内部结

构显式地表示出来。另外,产生式规则只能表示事物间的因果关系,而框架表示法不仅可以
通过 Infer 槽或 Possible-Reason 槽表示事物间的因果关系,而且可以通过其他槽表示出事物
之间更复杂的联系。

(2) 继承性

继承在知识表示方法中支持概念抽象和信息共享等思想,在框架系统中起到极其重要的
作用。继承性不仅减少了框架网络表示知识的冗余,而且较好地保证了知识的一致性。
需要指出的是:由于子框架可以继承父框架的槽值,也可以进行补充和修改,因此多重继
承有可能产生属性描述的多义性。如何解决继承过程中属性的歧义,目前还没有一种统一
的方法。

框架表示法的主要不足之处是不善于表示过程性的知识,因此,可以把框架表示法与产
生式表示法结合起来使用,以取得互补的效果。另外,对于给定的问题领域,如果有关的领
域知识具有比较明显的结构性特点,那么就比较容易建立框架表示的知识库;否则,用框架
网络来形式化领域知识就不是一件简单的事情。

2.4 语义网络表示法

如前所述,产生式表示法主要善于描述事物间的因果关系,框架表示法主要善于描述事
物的内部结构及事物间的类属关系。但是,客观世界中的事物是错综复杂的,相互间除了具
有这些关系外,还存在着其他各种含义的联系。为了描述更复杂的概念、事物及其语义联系,
引入了语义网络表示知识的方法。语义网络是奎廉(J. R. Quillian)于1968年最先提出的。1972
年,西蒙将语义网络用于自然语言理解。语义网络表示法是一种表达能力强而且灵活的知识
表示方法。

2.4.1 语义网络

语义网络是通过概念及其语义关系来表示知识的一种网络图。一个语义网络是一个带标
识的有向图,其中,有向图的节点表示各种事物、概念、属性、动作、状态等,有向弧表示
它所连接的节点间的某种语义联系,每个节点可以带有若干属性,可以用框架或元组来表示
一个节点的若干属性。一个节点还可以是一个语义子网络,从而形成一个多层次嵌套结构的
语义网络。

一个最简单的语义网络是如下一个三元组:

(节点1, 弧, 节点2)

它可以用一个有向图表示,如图2.2所示,称为一个基本网元。其中,A、B分别表示两个节
点, R_{AB} 表示A与B之间的某种语义联系。有向弧 R_{AB} 的方向是有意义的,由节点间的语义
关系确定,例如在表示类属关系时,有向弧箭头所指的节点表示上层概念,箭尾节点表示下
层概念或者一个属于该类的具体事物。如图2.3所示的语义网络就是一个基本网元,其中,

“猎狗”与“狗”之间的语义联系“是一种”具体地指出了“猎狗”与“狗”的语义关系，即“猎狗”是“狗”的一种，两者之间存在类属关系。

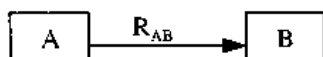


图 2.2 基本网元

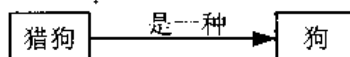


图 2.3 猎狗与狗的语义网络

当把多个基本网元用相应语义联系关联在一起时，就得到一个语义网络。

下面给出语义网络的 BNF 描述：

<语义网络> ::= <基本网元> | Merge (<基本网元>, ...)

<基本网元> ::= <节点> <语义联系> <节点>

<节点> ::= (<属性一值>, ...)

<属性一值> ::= <属性名> : <属性值>

<语义联系> ::= <系统预定义的语义联系> | <用户自定义的语义联系>

其中，Merge (...) 是一个合并过程，它把括号中的所有基本网元关联在一起，即把相同的节点合并为一个节点，从而构成一个语义网络。例如，如图 2.4 所示的三个基本网元，经合并后构成一个语义网络。

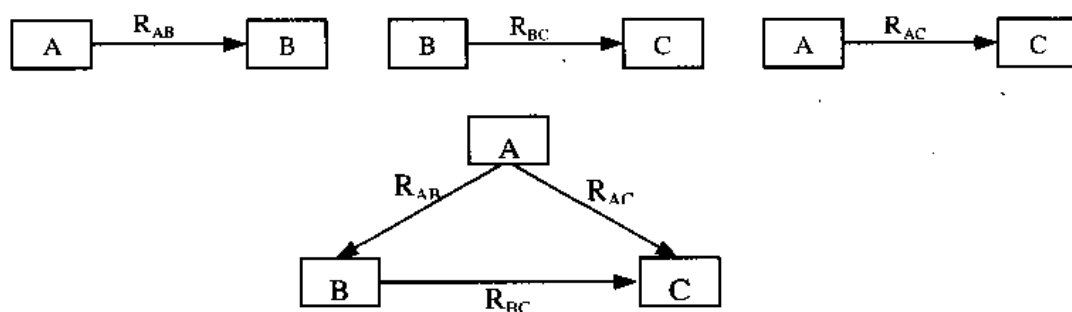


图 2.4 基本网元的合并示例

任何一种知识表示模式都应具有两种功能，一是能表达事实性的知识；二是能表达有关事实之间的联系，从而使得能够从给定的已知事实找到另一些有关的事实。语义网络可以表示事实性的知识，也可表示有关事实性知识之间的复杂联系。

1. 用语义网络表示事实

“猎狗是一种狗”这一简单事实的语义网络如图 2.3 所示。如果还需要进一步指出“狗是一种动物”，并且分别指出“猎狗”、“狗”和“动物”的一些属性，则只需在图 2.3 中增加一个“动物”节点和一条有向弧，并对每个节点附上相应的属性，如图 2.5 所示。图 2.5 中用短线与相应节点相连的部分是该节点所描述的对象属性

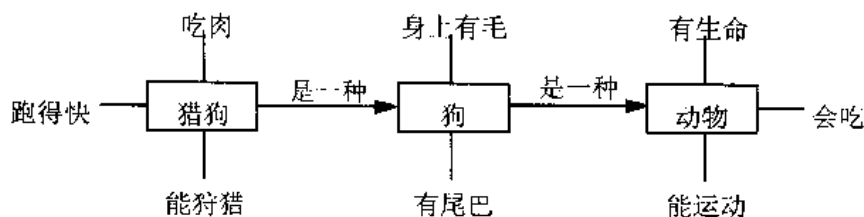


图 2.5 狗的语义网络

与框架表示法一样，语义网络也具有属性的继承性，即下层概念可以继承上层概念的属性，这样就可在下层概念只表示出它独有的属性。在图 2.5 中，虽然没有指出猎狗有尾巴、有毛、有生命、能运动、会吃的特征，但由于在它的上层概念“狗”及“动物”的描述中已指出了这些属性，因此，由继承性可知“猎狗”也具有这些属性。另外，在语义网络中，下层概念还可对其上层概念的属性做进一步的细化、补充、变异，使之能更准确地反映下层概念的特征。例如，在图 2.5 中，“吃肉”与“跑得快”就分别是对“会吃”与“能运动”的细化，而“能狩猎”则是一个新的补充。

在语义网络中，一个节点可以有一组向外的有向弧，用于指出这个节点与多个节点的语义联系，这在表示稍微复杂一点的事实性知识中是常用的，例如，我们来看如下事实：

“小信使”这只鸽子从春天到秋天占有一个窝

用语义网络表示如图 2.6 (a) 所示。在图 2.6 (a) 中设立了一个“占有”节点，之所以要设立这个节点，是由于已知事实中不仅要指出“小信使这只鸽子占用一个窝”，而且还要指出占有的时间是开始于春天和结束于秋天。只有通过设立“占有”节点及相应的 4 条有向弧才能把占有者、占有物和占有的起止时间关联起来。如果把“占有”作为一个语义关系用一条有向弧表示，即如图 2.6 (b) 中所示，则占有的起止时间就无法表示出来。

在一些事实性知识中，经常会用到“并且”和“或者”这样的连接词。用谓词公式表示时，可用合取符号“ $\wedge$ ”和析取符号“ $\vee$ ”分别予以表示。在语义网络中，可通过增设合取节点和析取节点分别予以表示。

2. 用语义网络表示事物间的关系

语义网络可以方便地描述事物之间的多种语义关系，下面给出常用的几种关系。

(1) 分类关系

分类关系是指事物之间的类属关系。前面已给出类属关系的例子。

(2) 聚集关系

如果下层概念是其上层概念的一个方面或者一个部分，则称它们的关系是聚集关系。如图 2.7 所示的语义网络就是一种聚集关系。

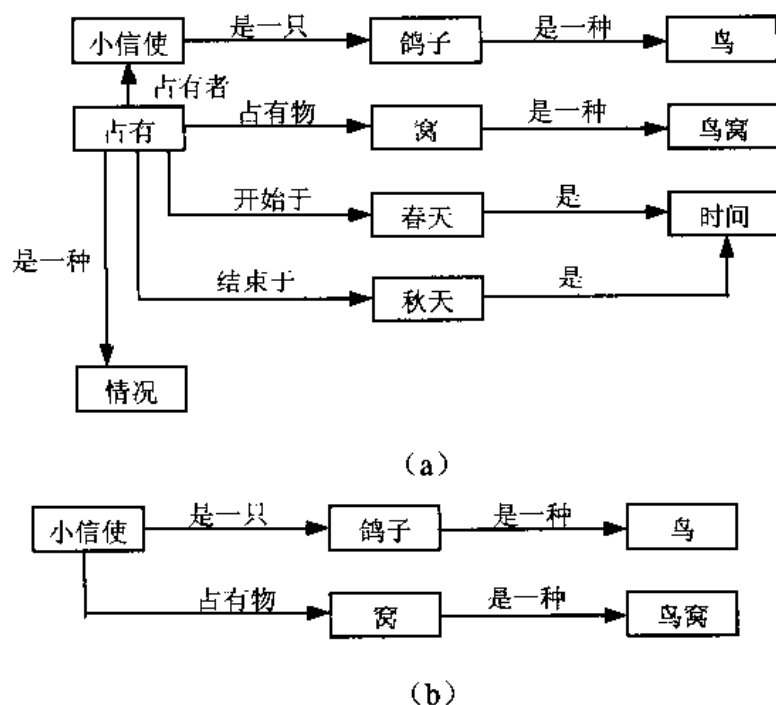


图 2.6 小信使的语义网络

(3) 推论关系

如果一个概念可由另一个概念推出，则称它们之间存在推论关系。如图 2.8 所示的语义网络就是一个简单的推论关系。

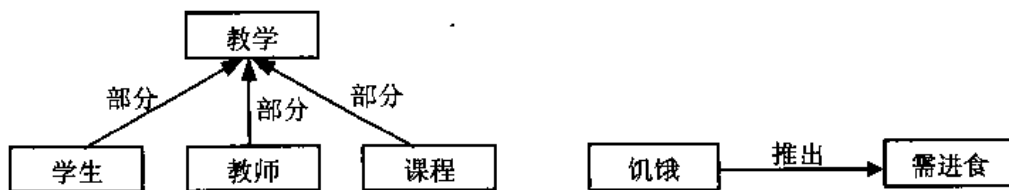


图 2.7 聚集关系示例

图 2.8 推论关系示例

(4) 时间、位置关系

描述一个事物时，常常需要指出它发生的时间、位置等，也可用相应的语义网络表示。

(5) 多元关系

在语义网络中，一条有向弧从一个节点指向另一个节点，表示这两个节点的一种语义关系，称之为一个二元关系表示。但在许多情况下，需要用一种关系把几个节点联系起来。例如在前面的例子中，需要把占有者“小信使”、占有物“窝”和占有起止时间“春天”与“秋天”之间的“占有”关系表示出来，这就是一个多元关系的表示。显然，多元关系无法用一个有向弧来表示，而是用一个节点和多条有向弧来表示，如图 2.6 (a) 所示。

3. 变元与量词在语义网络中的表示

用语义网络表示比较复杂的知识时,往往涉及到对量词及量化变元的表示。对于存在量词可以直接用“是一个”、“是一种”等语义联系来表示,但对全称量词则需要采用网络分区技术来处理。网络分区技术是亨德里克(G. G. Hendrix)在1975年提出的,其基本思想是:把一个表示复杂知识的命题划分为若干子命题,每一个子命题用一个较简单的语义网络表示,称为一个子空间,多个子空间构成一个大空间。每个子空间可以看作是大空间中的一个节点,称为超节点,子空间之间用有向弧连接。空间可以逐层嵌套。

例如,对如下事实:

每个学生都背诵了一首唐诗

可用如图2.9所示的语义网络表示。其中, s 是全称量词变元,表示任一个学生; r 是存在量词变元,表示某一次背诵; p 也是存在量词变元,表示某一首唐诗。 s 、 r 、 p 及其语义联系构成一个子网,即是一个子空间。子空间内的语义联系表示事件 r 的主体对象是 s ,客体对象是 p ,然后,分别通过“是一个”语义联系,分别指明 s 是学生, r 是背诵, p 是唐诗。用节点 g 表示这个子空间,用有向弧 F 指明节点 g 表示的是一个什么样的子空间。有向弧 $\forall$ 指出子空间 g 中的全称量词变元是 s 。若在子空间中有多条全称量词变元,则超节点 g 就有多条 $\forall$ 有向弧指向子空间中的多个变元节点。节点 GS 表示整个空间。

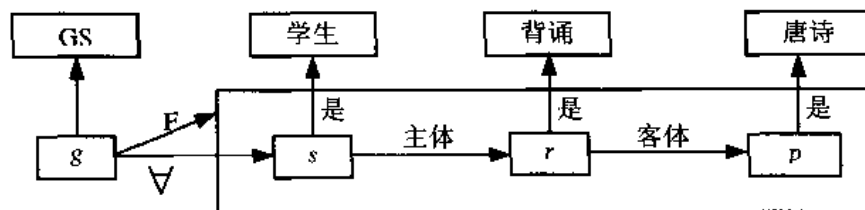


图2.9 具有全称量词的语义网络示例一

需要注意的是,一个子空间中的所有非全称量词变元节点都应是全称量词变元的函数,否则,就放在子空间的外面。

例如,对如下事实:

每个学生都背诵了“静夜思”这首唐诗

由于“静夜思”是一首具体的唐诗,是一个常量,不是全称量词变元 s 的函数,因此,应该把它放在子空间之外,如图2.10所示。

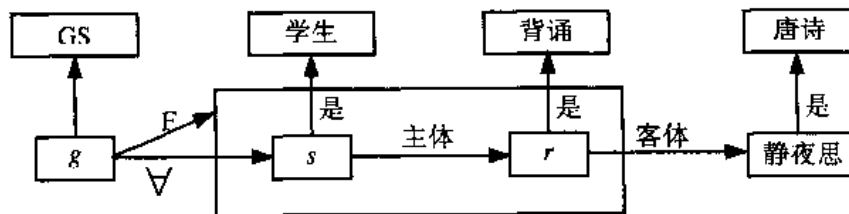


图2.10 具有全称量词的语义网络示例二

4. 常用的语义联系

语义联系反映了节点之间的语义关系，鉴于事物之间语义关系的复杂性，所以可以定义多种多样的语义联系。

在框架表示法中，给出了一些系统预定义槽名用以指明用框架表示的事物之间的联系，这些系统预定义槽名，如 ISA、Subclass、Part-of、Infer 等，同样可作为语义网络知识表示中的系统预定义的语义联系。下面给出另外一些常用的系统预定义语义联系。

(1) A-Member-of 联系

它表示个体与集体（类或集合）之间的关系。由它联系的个体对集体有属性继承性和属性更改权。例如，“李明是学会会员”的语义网络可表示为如图 2.11 所示的语义网络。个体“李明”对集体“学会”的属性有继承性，也可对继承的某些属性有一定的更改，或称为属性继承变异。

(2) Composed-of 联系

它表示“构成”联系，是一种一对多的联系。由它联系的节点间不具有属性继承性。一般需要引入一个“与节点”。如图 2.12 所示的语义网络表示了“整数由正整数，负整数与零组成”。

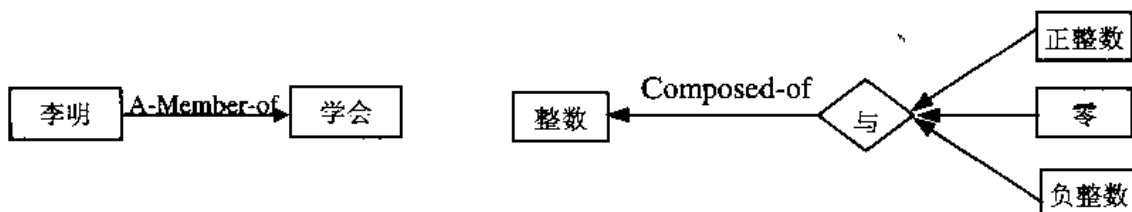


图 2.11 A-Member-of 联系示例

图 2.12 Composed-of 联系示例

(3) Have 联系

它表示事物对属性的“拥有”关系或事物之间的“占有”关系。例如，“李明有计算机”可表示为如图 2.13 所示的语义网络。

(4) Before、After、At 联系

它们用来表示事件发生的时间关系，Before 表示一个事件发生在另一个事件之前，After 表示一个事件发生在另一个事件之后，At 表示事件发生的时间。例如，“9 月份开学”可表示为如图 2.14 所示的语义网络。

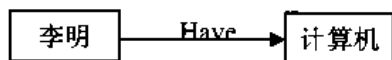


图 2.13 Have 联系示例

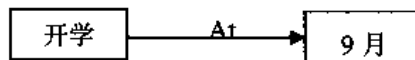


图 2.14 At 联系示例

(5) Located-on (-at, -under, -inside, -outside 等)联系

这些语义联系用来表示事物之间的位置关系。例如，“计算机放在桌子上”可表示为如

图 2.15 所示的语义网络。

(6) Similar-to、Near-to 联系

它们分别用于表示事物之间的相似和接近的关系。例如，“方凳类似于方桌”可表示为如图 2.16 所示的语义网络。

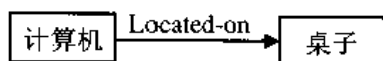


图 2.15 Located-on 联系示例

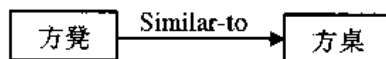


图 2.16 Similar-to 联系示例

例 2.9 由给出的系统预定义语义联系画出下述知识的语义网络：

每位住在金山公寓 3 号楼的人都是金山公司的程序员

解 我们首先把要表示的知识用谓词公式表示出来，然后再用语义网络表示。

上述知识用谓词公式可表示为：

$$(\forall x) ((\text{Person}(x) \wedge \text{Address}(x, \text{金山公寓 3 号楼})) \rightarrow (\text{Occupation}(x, \text{程序员}) \wedge \text{Member}(x, \text{金山公司})))$$

其中，引入了 4 个谓词，分别是：

$\text{Person}(x)$	表示 x 是人
$\text{Address}(x, y)$	表示 x 住在 y
$\text{Occupation}(x, y)$	表示 x 的职业是 y
$\text{Member}(x, y)$	表示 x 是 y 的成员

用语义网络表示上述知识的语义网络图如图 2.17 所示。

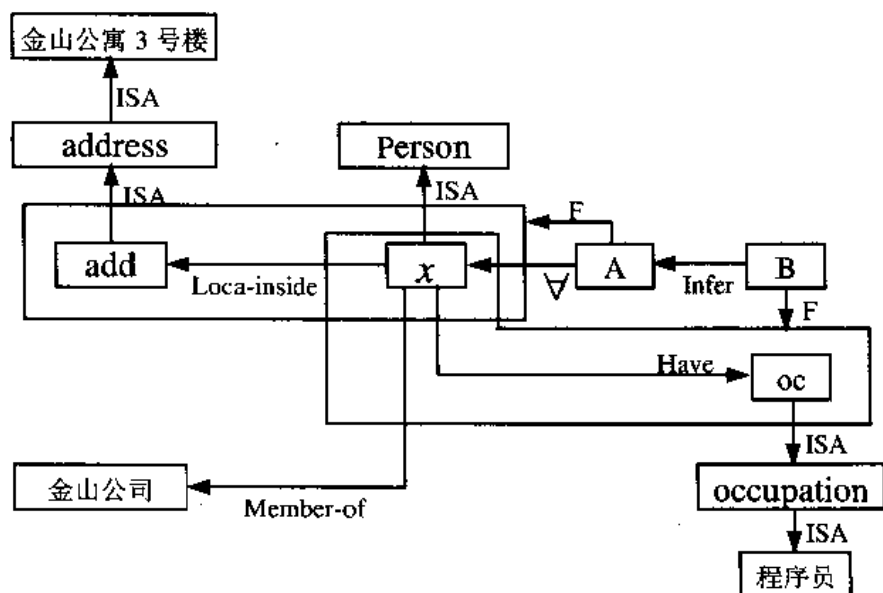


图 2.17 例 2.9 的语义网络表示

在图 2.17 中, 引入了两个超节点 A 和 B 分别代表两个子空间, 超节点 A 对应谓词公式的前件, 超节点 B 对应谓词公式的后件, 用推论关系 Infer 连接 A 到 B。由于全称量词变元 x 的辖域是整个谓词公式, 因此, 在 A 的子空间和 B 的子空间中共用了 x 节点, 并用一个 $\forall$ 有向弧指向 x 节点。在 A 的子空间中, 引入了变元 add 节点, 用语义联系 Located-inside 说明 x 住在 add 中。在 B 的子空间中, 引入了变元 oc 节点, 用语义联系 Have 说明 x 拥有 oc。分别用语义联系 ISA 说明变元 add 是“地址”(address)、变元 x 是“人”(Person)、变元 oc 是“职业”(Occupation), 并进一步把“职业”细化为“程序员”, 把“地址”细化为“金山公寓 3 号楼”。用语义联系 A-Member-of 指明变元 x 与“金山公司”的所属关系。

比较谓词公式与语义网络, 由于谓词既用于表示事物, 也用于表示事物之间的联系, 因此, 在一阶谓词逻辑表示法中, 事物和事物之间的联系都用谓词的形式来表示。在语义网络表示法中, 事物用节点表示, 事物之间的联系用有向弧表示, 因此, 在上述谓词公式的 4 个谓词中, 有的谓词用节点表示, 有的谓词则用有向弧表示。

2.4.2 语义网络的推理及其特点

用语义网络表示知识的问题求解系统称为语义网络系统。该系统主要由两部分组成: 一是由语义网络表示的知识库, 二是利用语义网络求解问题的程序, 称为语义网络推理机。

1. 语义网络系统的推理

大多数语义网络系统采用的推理机制都是以网络结构的匹配为基础的。先根据待求解的问题构造一个问题网络片断, 然后在语义网络知识库中搜寻能与该网络片断匹配的网络。在

用语义网络表示上述事实，并求解问题：赵云主修什么课程？

解 根据给出的事实构造一个语义网络如图 2.18 所示。其中，节点“教育 1”由有向弧 Recipient 连接节点“赵云”，表示赵云所受到的一种教育，实际上，赵云还可能接受其他的教育。

根据求解的问题构造问题网络片断如图 2.19 所示。其中，变元节点“ x ”由 ISA 有向弧说明是一种教育，由 Major 有向弧连接的节点为空，反映了要求解的问题：主修什么课程？

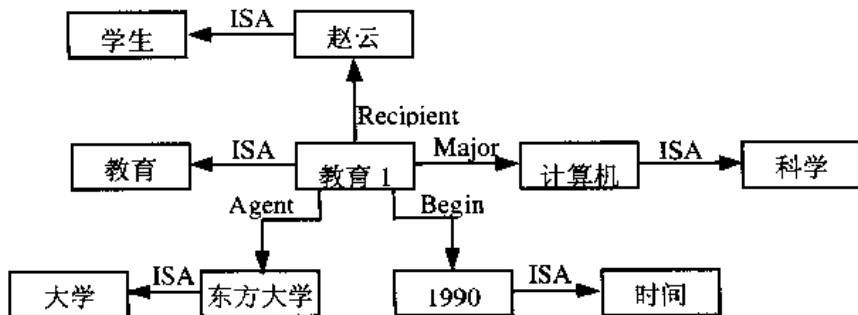


图 2.18 赵云受教育情况的语义网络

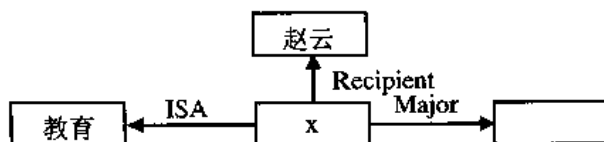


图 2.19 待求解问题的语义网络片断

用图 2.19 所示的问题网络片断与图 2.18 所示的语义网络匹配，对问题片断进行代换(教育 1/ x)，即用常量“教育 1”代换问题网络片断中的变元 x 后，可得出与问题网络片断匹配的语义网络知识库中的网络片断，由 Major 弧指向的节点可得知赵云主修课程是计算机。如果要求解的问题是：赵云在哪个学校主修计算机？那么只需在问题网络片断中增加相应的空节点和有向弧，通过网络匹配就可得出问题的解。

由此例可见，语义网络中的匹配是建立在精确结构基础上的匹配，包含节点和有向弧的匹配。对网络中的变元节点可以通过代换给变元赋值，使之与可匹配网络的相应节点一致。如果同一个事物表示成两个结构不同的语义网络，那么它们将无法匹配。这种情况是会常常发生的，就像表示同一概念的谓词公式会写出多种形式一样。继承推理和网络演绎能帮助解决同一含义而不同结构的语义网络匹配问题。

(2) 继承推理

继承推理是指推理机利用 ISA 和 AKO 等具有继承性的语义联系对网络片断中并不显式存在的结构进行匹配，即网络片断的继承匹配。如果在下层节点的网络片断中没有找到问题

网络片断需要匹配的有向弧和节点,也就是说不能显式匹配。那么,可沿该下层节点的 ISA 有向弧找到其上层节点。只要上层节点的属性描述中有可匹配的有向弧和节点,问题网络片断就完成了继承匹配。

语义网络的继承推理类似框架系统的推理过程,继承推理可以沿分类语义网络一直向上寻找,最终或者得到成功的匹配,或者直到分类语义网络的根节点都不能找到相应的匹配而失败。

(3) 语义网络演绎

带有逻辑推理语义关系的语义网络称为推理网络。在推理网络中由一条逻辑推理语义关系有向弧连接两个节点表示一条推理规则。若有两个节点 A 和 B,由有向弧 Infer 连接 A 至 B,则 A 是规则的前提条件, B 是规则的结论。若由有向弧 Possible-Reason 连接 A 至 B,则 A 是规则的结论, B 是 A 成立的原因。

我们把推理网络与产生式系统的推理进行下述比较,说明两者之间的异同:

1) 推理网络中由一条逻辑推理语义关系有向弧连接的可以是两个超节点,也就是说,一条推理规则的前提和结论都可以分别是语义网络片断。规则的匹配是通过语义网络片断的匹配(含显式匹配和继承匹配)来实现的。

2) 语义网络演绎就是在推理网络上进行的搜索匹配过程。由于推理网络实际上转化为由基本语义网络表示的规则系统,因此,类似于产生式系统的推理过程,语义网络演绎按推理方向划分,也有正向推理、逆向推理和双向推理等推理方式。正向推理是根据给出的已知事实网络片断,从推理网络的最低层开始寻找可匹配的网络片断。若找到可匹配的网络片断,则把该网络片断的 Infer 弧指向的结论网络片断作为当前事实网络片断进行下一次寻找匹配。所以,正向推理是沿着推理网络的 Infer 有向弧所指方向向上搜索匹配。逆向推理是根据给出的假设网络片断,从推理网络的最高层开始寻找可匹配的网络片断。若找到可匹配的网络片断,则把该网络片断的 Possible-Reason 弧指向的原因网络片断与支持初始假设的事实网络片断进行匹配。若可匹配,说明事实网络片断描述的事实支持初始假设成立;否则,把原因网络片断作为当前假设网络片断进行下一次寻找匹配。所以,逆向推理是沿着推理网络的 Possible-Reason 有向弧所指方向(或 Infer 的逆方向)向下搜索匹配。

3) 无论在正向推理还是逆向推理过程中,如果在被匹配的语义网络片断中有两条或两条以上的 Infer 弧或 Possible-Reason 弧,表明存在多于一条的规则可用,那么需要进行冲突的消解。

4) 语义网络演绎按照在推理网络中搜索可匹配的网络片断的搜索策略划分,也有盲目搜索策略和启发式搜索策略等。当语义网络知识库中的信息量很大时,盲目搜索可匹配的网络片断是很费时的。为了加速搜索过程,推理机中可以嵌入一些含启发式知识的选择器函数,由它提供对节点和有向弧的优先匹配次序。

2. 语义网络表示法的特点

语义网络表示法主要有以下特点。

(1) 结构性

语义网络表示法和框架表示法都是结构化的知识表示方法。语义网络表示法把事物的属性和事物之间的联系用语义网络的结构形式显式地表示出来,下层概念的节点可以继承、补充、变异上层概念节点的属性,从而实现信息共享和减少信息冗余。但语义网络与框架又各有特点,框架表示法适合于表示结构固定的概念、事件和行为;语义网络表示法具有更大的灵活性,用其他表示方法表示的知识几乎都可以用语义网络表示出来。

(2) 自然性

语义网络实际上是一个带有标识的有向图,可直观地把事物的属性及事物间的语义联系表示出来,便于理解。自然语言描述的知识比较容易用语义网络来表示。

语义网络表示法的主要局限性有以下几点。

(1) 非严格性

与谓词逻辑相比,语义网络没有公认的形式化表示体系。一个给定的语义网络所表达的含义完全依赖于系统预定义和自定义的语义关系。另外,目前采用的表示量词的网络表示方法在逻辑上是不充分的,不能保证不存在二义性。通过推理网络实现的推理的正确性决定于推理机程序对语义网络的理解与解释。

(2) 复杂性

语义网络表示法具有表示知识的灵活性,但是,也由于它表示知识的非严格性,从而增加了语义网络的复杂程度和对语义网络进行处理的复杂性。由于节点之间的联系可以是线状的、树状的,也可以是网状的,甚至是递归的,这就给知识的存储、修改和检索带来不少困难。因此,语义网络系统的管理和维护以及推理就相对复杂一些,要求对语义网络提供强有力的知识组织原则。

2.5 面向对象表示法

目前,面向对象技术的研究已经深入到计算机软、硬件的多个领域,例如,面向对象程序设计方法学、面向对象数据库、面向对象操作系统、面向对象软件开发环境、面向对象硬件支持等等。在面向对象语言的研制开发方面,自1980年施乐(Xerox)公司推出面向对象语言SMALLTALK-80之后,各种不同风格、不同用途的面向对象语言相继问世,例如,AT&T公司贝尔实验室在1985年研制开发的C++,荷兰阿姆斯特丹大学开发的POOL,施乐公司开发的LOOPS及Common LOOPS等。近几年来,人们开始探讨把面向对象的思想、方法用于智能系统的设计与构造,并在知识表示、知识库的组成与管理、专家系统的设计等方面取得了一定的进展。

2.5.1 面向对象的知识表示

客观世界的问题都是由客观世界的事物及事物之间的相互关系构成的。从面向对象的角度

来看,人们在认识问题和分析问题时,可以把问题分解为一些对象(object)以及对象之间的组合和联系。事实上,任何系统都可以看成为达到某种目的相互作用的一组实体或对象组成。

1. 对象、消息、方法和封装性

从广义上讲,所谓“对象”是指客观世界中的任何事物,即任何事物都可以在一定前提下成为被认识的对象,它既可以是一个具体的简单事物,也可以是由多个简单事物组合而成的复杂事物。从问题求解的角度来讲,对象是与问题领域有关的客观事物。由于客观事物都具有其自然属性及行为,因此,当把与问题有关的属性及行为抽取出来加以研究时,相应客观事物就在这些属性与行为的背景下成为所关心的对象。

在面向对象的知识系统中,一个对象具有的知识组成了该对象的静态属性,一个对象所具有的知识处理方法和各种操作描述了该对象的智能行为。按照面向对象方法学的观点,一个对象的形式定义可以用如下四元组表示:

$$\langle \text{对象} \rangle ::= (\text{ID}, \text{DS}, \text{MS}, \text{MI})$$

也就是说,一个完整的对象由该对象的标识符 ID、数据结构 DS、方法集合 MS 和消息接口 MI 组成。

对象的标识符 ID (Identifier) 又称对象名,用以标识一个特定的对象。

对象的数据结构 DS (Data Structure) 描述了对象当前的内部状态或具有的静态属性,常用一组<属性名: 属性值>表示。

对象的方法集合 MS (Method Set) 用以说明对象具有的内部处理方法或操作。操作分为两类,一类用于对内部数据进行操作,从而改变对象的当前状态;另一类用于产生对外的输出。对一个对象来说,其他对象的操作不能直接操纵该对象私有的数据,只有对象私有的操作可以操纵它,即对象的状态只能由它私有的操作来改变。一个对象的 MS 反映了对象自身的智能行为。

对象的消息接口 MI (Message Interface) 是对象接收外部信息和驱动内部有关操作的惟一对外接口。这里的外部信息称为消息。消息接口以消息模式集的形式给出,每一消息模式有一个消息名,通常还包含必要的参数表。当接收者从它的消息接口受理发送者的某一消息时,首先要判断该消息属于哪一个消息模式,并找出与该消息模式相联系的内部操作,然后执行这个内部操作,进行相应的消息处理或产生向外的输出信息。

在面向对象的系统中,问题求解或程序的执行是依靠对象间传递消息完成的。最初的消息通常来自用户的输入。某一对象在处理相应的消息时,如果需要,又可以通过传递消息去请求其他对象完成某些处理工作或回答某些信息,其他对象在执行所要求的处理时同样可以通过传递消息与别的对象联系,如此继续下去,直至得到问题的解。

可见,在面向对象的系统中,消息流统一了数据流和控制流,它是实现对象之间联系的惟一途径。消息中只包含发送者给出的信息,这些信息往往表示对接受者的某种要求,但仅仅告诉接收者需要做什么,并不指示接收者如何去做。消息完全由接收者解释,接收者可以独立决定以何种方法或何种操作去完成相应的工作。同样的消息可以传递给不同的对象,不

同的对象可以对同样的消息做出不同的反应。同一对象也可以接收多个对象传来的不同消息，对传来的消息可以返回相应的回答信息，也可以不予回答。

消息模式不仅定义了该对象所能受理的消息，也规定了该对象的处理能力。因为每个对象的一种消息模式都与该对象内部的相应方法和操作相联系，方法和操作是对象固有处理能力的具体实现，通常用一个可执行的程序代码段来表示。

如前所述，一个对象的状态只能由它的私有操作来改变，其他对象的操作不能直接改变它的状态。当一个对象需要改变另一个对象的状态时，它只能向该对象发送消息，该对象接受消息后就根据消息的模式找出相联系的操作，并执行操作来改变自己的状态。这里，发送消息与通常所说的过程调用有不同的含义。首先，发送消息只是触发接受消息的对象，同样的消息被对象处理的结果不仅与对象的处理能力有关，而且与对象的当前状态有关。也就是说，同样的输入参数可因对象的状态不同而有不同的结果。对于过程调用，只要输入参数相同必然得到相同的处理结果。其次，在过程调用中，过程是一个独立的实体，显式地为它的使用者所见；而在面向对象中，操作是隶属于对象的，它不是独立存在的实体，只是对象的处理功能的体现。

把一切局部于对象的信息及操作都局限于对象之内，在外面是不可见的，对象之间除了相互传递消息之外，不再有其他联系，这就是对象的封装性。

封装是一种信息隐藏技术，是面向对象的主要特征，面向对象的许多优点都是依靠封装而获得的。对象的封装性使得对象的用户可以不了解对象行为是如何实现的细节，只需要了解对象的外部特征，即对象的所有消息模式及相应于每个消息模式的处理能力，就可以用消息来访问对象，从而可以把精力用于系统一级的设计与构成上。

一个复杂对象由若干简单对象组成，简单对象提供的某些消息可能仅供复杂对象内部使用，复杂对象的这种不向外界公开的消息称为该复杂对象的私有消息；另一方面，复杂对象向外界公开提供的消息称为该复杂对象的公有消息。在面向对象的语言中，对象的外部接口是以对象协议或规格说明的形式提供的。协议是一个对象对外服务的说明，它告知该对象可以为外界做什么。外界对象能够且只能够向该对象发送协议中所包含的消息，也就是说，请求对象进行操作处理的惟一途径是通过该对象协议中所包含的消息进行。从私有消息和公有消息的角度来看，协议是一个对象所能接受的所有公有消息模式的集合。可以说，一个对象就是在它的协议下封装起来的。

2. 类、类层次和继承性

类(Class)在概念上是一种抽象机制，它是对一组相似对象的抽象。具体地说，在一组相似的对象中，会有一些相同的特征（包含部分相同的数据和操作），为了避免相同数据和操作的重复描述及存储，就把共同的部分抽取出来构成一个类。类也是一个对象，只是它的数据和操作是属于该类的各具体对象共同的那部分，或者说类描述了该类对象的共性。经过类的抽象，一个对象除对象名外，形式上只剩下体现该对象个性的内部状态，此时的对象称为所属类的一个实例(Instance)。

其中, Class 是类描述的开始标志, <类名>是该类的名字,它是系统中该类的惟一标识, <超类名>是可选的,当该类有父类时,用它指出父类的名字。<类变量表>是一组变量名构成的序列,该类中所有对象都共享这些变量,对该类对象来说,这些变量是它们的全局变量,当把这些变量实例化为一组具体的值时,就得到了该类中的一个具体对象,即一个实例。Structure 后面的<对象的静态结构描述>用于描述该类对象的数据结构。Method 后面的<对于对象的操作定义>用于定义该类对象可施行的各种操作,它既可以是一组规则,也可以是为实现相应操作所需执行的一段程序,在 C++ 中为成员函数调用。Restraint 后面的<限制条件>指出该类对象所应满足的限制条件,可用包含类变量的谓词或其他形式的条件表达式形式表示。若没有限制条件,则表示没有限制。

2.5.2 面向对象表示法的特点

首先把面向对象表示的基本特点进行小结,然后把面向对象表示法与语义网络表示法和框架表示法进行比较。

1. 面向对象表示的基本特征

(1) 封装性

对象就是被封装的数据和操作。一个对象拥有的方法被触发实施时,通常需要引用自己的内部状态对有关数据进行操作,必要时可以修改自己的内部状态,还往往发送一批消息给其他对象,触发其他对象。每个对象的内部状态不允许其他对象直接引用和修改。对象的内部状态和方法对外界是隐蔽的。一个对象的外部特性只由对象的所有消息模式及相应于每个消息模式的处理功能来定义。面向对象表示的封装性使得用户只需知道对象的外部特性而无需了解其内部细节就可以使用它。

(2) 模块性

显式地把对象的外部定义和对象的内部实现分开是面向对象表示的一大特色,这种对象的封装性就是模块性。公开模块的外部定义和隐蔽模块的内部实现,使面向对象的软件系统便于维护和修改。另外,一个对象是一个可独立存在的实体,其内部状态不直接受到外界的影响,只是通过受理其他对象传来的消息而被触发,这也是软件的模块性所追求的目标。

(3) 继承性

面向对象表示的继承性使子类可继承其父类的数据和操作,从而可以把每个子类拥有的数据和操作分为两部分,一部分是从父类继承过来的共享数据和操作,另一部分是子类自己私有的数据和操作。继承性不仅使面向对象表示具有清晰的层次结构,而且对实现数据共享、数据一致性和减少冗余都十分有利。

(4) 易维护性

对象实现了封装和继承,这就使程序设计中的错误具有局部性,不会传播错误,便于检测和修改程序。

在面向对象的程序设计中,程序设计就是定义对象并建立对象间的通信关系,类是系统

的基本构件。如果开发设计的面向对象系统的功能需求发生变化,通常较少波及到对象类的设计与实现,更多的是影响到它们的组装形式,因此,实现的系统基本构件无需修改或少量修改后仍可使用,从而可采用快速原型法构造系统,以及对系统进行优化和增量开发。

2. 面向对象表示与语义网络及框架表示的比较

语义网络表示法、框架表示法和面向对象表示法都是结构化的知识表示方法。

语义网络的主要优点是表示的灵活性,语义网络中的节点和有向弧可以按规定不加限制地定义。这种灵活性在面向对象的表示中不仅仍然存在,而且对象和对象之间的关系还可以动态建立。语义网络中的节点对应于面向对象表示中的对象,语义网络中由有向弧定义的语义关系对应于面向对象表示中的消息传递,语义网络中具有继承性的语义关系对应于面向对象表示中对象与类之间的继承关系。基本上,面向对象的表示可以看成是一种动态的语义网络。

语义网络的主要缺点是系统的开发和维护比较困难,随着表示对象的节点的数量增加,语义网络的管理变得十分复杂,很难说明某个对象或属性值的修改对整个系统会产生什么影响。面向对象表示的封装性恰好有力地克服了语义网络的这一弱点,它把对象的状态及对状态的修改封装在该对象中。

框架表示的结构与面向对象表示的结构很类似,知识都可以使用类的概念按一定的层次结构来组织。但是,框架知识表示的模块性不能清楚地定义。削弱框架系统结构化的两大因素是:一是框架之间的关系既可能是具有继承关系的子类连接或成员连接,也可能是反映全体和部分关系的组成连接,而且不是惟一关系;二是规则可以通过一个框架连接到另一个框架。在面向对象的知识表示中,两个类之间的连接关系只有具有继承关系的子类连接;规则用作一个类的内部方法,不可能出现跨越两个对象的规则,类的惟一对外接口是消息模式,类之外的代码唯有通过传递有关消息才能与该类的方法打交道。因此,面向对象表示的模块性很好,特别适合于大型知识系统的开发。

综上所述,面向对象程序设计方法以信息隐蔽和抽象数据类型概念为基础,既提供了从一般到特殊的演绎手段(如继承等),又提供了从特殊到一般的归纳形式(如类等),已成为基于知识的人工智能软件的主要开发方法。

目前,面向对象技术不仅在软件设计和知识表示中得到了广泛的应用,而且已拓展到数据库系统和多媒体应用系统等新的应用领域。与此同时,出现了一个比“对象”更富动态性、更具有人工智能含义的概念,即“智能体”(Agent)概念。智能体可被认为是一种可以包含主观信念且可以主动响应外界事件的对象。可以认为,面向智能体技术是面向对象技术在人工智能应用领域中的发展,是人工智能吸纳面向对象技术的结晶。

本章介绍的一阶谓词逻辑表示法、产生式表示法、语义网络表示法、框架表示法与面向对象表示法都是知识的符号表示法。另外还有一类知识表示法称为连接机制表示法,人工神经网络表示法就是一种连接机制表示法。选择哪一种知识表示法表示领域知识,应根据领域知识的特点来选择。例如对于具有因果关系的知识,尽管用框架或语义网络也可表示,但却不如用产生式表示更直观、自然,而且运行效率也不一样。

知识表示还可分为外部表示与内部表示两种形式。外部表示形式是指本章所讨论的各种知识表示模式，如产生式规则、框架、语义网络等。内部表示形式是指某种知识表示模式在编程语言或开发工具中的表示形式，显然，内部表示形式与编程语言或开发工具有关。例如，产生式规则在 LISP 语言、C 语言以及开发工具 CLIPS 中的内部表示形式是不同的。在后面的章节中，我们将介绍有关的内部表示形式。

习题二

2.1 名词解释

命题	谓词
个体	个体域
谓词公式的解释	谓词公式的永真性
谓词公式的可满足性	谓词公式的不可满足性
谓词公式的永真蕴含	谓词公式的等价性

2.2 简述谓词逻辑中的下述推理规则：

- (1) P 规则
- (2) T 规则
- (3) CP 规则
- (4) 反证法规则

2.3 一阶谓词逻辑表示法适合于表示哪种类型的知识？它有哪些主要特点？

2.4 请用相应的谓词公式表示下述语句：

- (1) 有的人喜欢足球，有的人喜欢排球，有的人既喜欢足球又喜欢排球。
- (2) 不是每一个人都喜欢游泳。
- (3) 如果没有利息，那么就没有人去储蓄钱。
- (4) 对于所有的 x 和 y ，如果 x 是 y 的父亲， y 是 z 的父亲，那么 x 是 z 的祖父。
- (5) 对于所有的 x 和 y ，如果 x 是 y 的孩子，那么 y 是 x 的父母。
- (6) 登高望远。
- (7) 响鼓不用重锤。
- (8) 如果 $b > a > 0$ 和 $c > d > 0$ ，则有 $(b \times (a+c)/d) > b$ 。

2.5 产生式与谓词逻辑中的蕴含式有何异同点？

2.6 简述产生式系统问题求解的一般步骤。

2.7 何谓可交换产生式系统？可交换产生式系统的规则库与数据库应具有什么性质？

2.8 何谓可恢复产生式系统？为什么可恢复产生式系统应具有回溯功能？

2.9 简述产生式表示法的主要优缺点。

2.10 有下述猴子摘香蕉问题：在一个房间的地板上有一只猴子和一个箱子，天花板下

挂有一串香蕉，猴子的水平位置为 a ，箱子的水平位置为 b ，香蕉的水平位置为 c ，猴子只有爬到箱子上才能摘到香蕉。猴子被允许有以下 4 种行为：在地板上行走；在地板上推动箱子；爬到箱子上；摘香蕉。为了规划出猴子摘香蕉的行为方案，请把猴子的 4 种行为表示成 4 条产生式规则。

2.11 有下述传教士与野人问题：有 3 名传教士和 3 名野人来到一条河的右岸，欲乘一条船渡河到左岸，该船的最大负载能力为 2 人，传教士或野人皆可撑船。在任何时候，不论是在右岸还是在左岸，如果野人人数超过传教士人数，那么，野人就会吃掉传教士。为了规划出一个渡河方案，把 6 个人都安全地渡过河去，请应用产生式表示法表示求解该问题的所有规则。

2.12 有下述农夫过河问题：农夫、狐狸、山羊和白菜都在一条河的左岸，只有农夫可撑船，要将狐狸、山羊和白菜都渡河到右岸去。但是，农夫每次撑船过河时，船上至多只能载狐狸、或者载山羊、或者载白菜。若农夫不在时，则狐狸会吃掉山羊、山羊会吃掉白菜。为了规划出一个渡河方案，使农夫能把狐狸、山羊和白菜都带过河去，请应用产生式表示法表示求解该问题的所有规则。

2.13 何谓框架知识表示？给出框架的一般表示形式。

2.14 简述框架系统的推理过程。

2.15 简述框架表示法的主要特点。

2.16 试建立一个“学生”框架网络，其中，至少有“学生基本情况”、“学生课程学习情况”和“学生奖惩情况”三个框架描述。

2.17 请用语义网络分别表示下述语句：

(1) 与会者有男、有女，有的年老、有的年轻。

(2) 李明是图灵电脑公司的经理，他住在江滨路 102 号，今年 38 岁。

(3) 大门前的这棵树从春天到秋天都开花。

(4) 计算机系的每个学生都学习“人工智能原理”，它是计算机专业的一门主干课程。

2.18 请把下述事实用一个语义网络表示出来：

(1) 李明是东方大学计算机系的一名学生。

(2) 他住在计算机系的学生宿舍里。

(3) 计算机系的每间学生宿舍都有一台联网的计算机。

(4) 李明喜欢在宿舍的计算机上浏览。

2.19 简述语义网络系统求解问题的基本过程。

2.20 简述语义网络表示法的主要特点与局限性。

2.21 一个对象可以形式地定义为 4 元组：

$\langle \text{对象} \rangle ::= (\text{ID}, \text{DS}, \text{MS}, \text{MI})$

请说明 4 元组中各元的含义。

2.22 何谓对象的封装？为什么说在面向对象的系统中，问题求解是依靠对象之间传递消息来完成的？

2.23 何谓对象的类？为什么说经过类的抽象，一个对象是由它所属的类以及该类的一个实例组成？

2.24 何谓类的继承？给出面向对象方法表示知识的形式描述，由此说明面向对象系统的继承机制。

2.25 简述面向对象表示的主要特点。

第3章 经典逻辑推理

为了使计算机具有智能，仅仅使它拥有知识还是不够的，还必须使它具有思维能力，即能运用知识进行推理来求解问题。

经典逻辑推理是根据经典逻辑（命题逻辑及一阶谓词逻辑）的逻辑规则进行的一种推理，主要推理方法有自然演绎推理、归结演绎推理及与/或形演绎推理等。由于这种推理是基于经典逻辑的，其真值只有“真”和“假”两种，因此它是一种精确推理，或称为确定性推理。

3.1 推理的基本概念

从已知的事实出发，通过运用已掌握的知识，找出其中蕴含的事实，或归纳出新的事实，这一过程通常称为推理。严格地说，所谓推理就是按某种策略由已知判断推出另一判断的思维过程。实现推理的程序称为推理机。

3.1.1 推理方式及其分类

人类的智能活动有多种思维方式，人工智能作为对人类智能的模拟，相应也有多种推理方式，下面分别从不同的角度对它们进行讨论。

1. 演绎推理、归纳推理、默认推理

推理的基本任务是从一种判断推出另一种判断，从新判断推出的途径来分类，推理可分为演绎推理、归纳推理及默认推理。

演绎推理是从全称判断推导出特称判断或单称判断的过程，即由一般性知识推理适合于某一具体情况的结论。这是一种从一般到个别的推理。演绎推理有多种形式，经常用的是三段论式，它包括：

- 1) 大前提，这是已知的一般性知识或假设。
- 2) 小前提，这是关于所研究的具体情况或个别事实的判断。
- 3) 结论，这是由大前提推出的适合于小前提的新判断。

例如：

- 1) 足球运动员的身体都是强壮的。
- 2) 李波是一名足球运动员。
- 3) 所以，李波的身体是强壮的。

这就是一个三段论推理。其中：1) 是大前提；2) 是小前提；3) 是经演绎推出的结论。结论“李波的身体是强壮的”是蕴含于“足球运动员的身体都是强壮的”这一大前提之中的，它

没有超出大前提所断定的范围。在任何情况下,由演绎推理导出的结论都是蕴含在大前提的一般性知识之中的。只要大前提和小前提是正确的,由它们推出的结论也必然是正确的。演绎推理是人工智能中的一种重要推理方式,在目前研制成功的各类智能系统中,大多是用演绎推理实现的。

归纳推理是从足够多的事例中归纳出一般性结论的推理过程,是一种从个别到一般的推理。若从归纳时所选事例的广泛性来划分,归纳推理又可分为完全归纳推理与不完全归纳推理。完全归纳推理是指在进行归纳时考察了相应事物的全部对象,并根据这些对象是否都具有某种属性,从而推出这个事物是否具有这个属性。例如,某厂进行产品质量检查,如果对每一件产品都进行了严格检查,并且都是合格的,则推导出结论“该厂生产的产品是合格的”,这就是一个完全归纳推理。不完全归纳推理是指只考察了相应事物的部分对象,就得出了结论。例如,检查产品质量时,只是随机地抽查了部分产品,只要它们都合格,就得出“该厂生产的产品是合格的”结论,这就是一个不完全归纳推理。不完全归纳推理推出的结论不具有必然性,属于非必然性推理,而完全归纳推理是必然性推理。但由于要考察事物的所有对象通常比较困难,因而大多数归纳推理都是不完全归纳推理。归纳推理是人类思维活动中最基本、最常用的一种推理形式,人们在由个别到一般的思维过程中经常要用到它。

默认推理又称为缺省推理,它是在知识不完全的情况下假设某些条件已经具备所进行的推理。例如,在条件A已成立的情况下,如果没有足够的证据能证明条件B不成立,则就默认B是成立的,并在此默认的前提下进行推理,推导出某个结论。由于这种推理允许默认某些条件是成立的,这就摆脱了需要知道全部有关事实才能进行推理的要求,使得在知识不完全的情况下也能进行推理。在默认推理过程中,如果某时刻发现原先所作的默认不正确,则就要撤消所作的默认以及由此默认推出的结论,重新按新情况进行推理。

2. 确定性推理、不确定性推理

按推理时所用知识的确定性来分类,推理可分为确定性推理与不确定性推理。

确定性推理是指推理时所用的知识都是精确的,推出的结论也是确定的,其真值或者为真,或者为假,没有第三种情况出现。本章将要讨论的经典逻辑推理就属于这一类。

不确定性推理是指推理时所用的知识不都是精确的,推出的结论也不完全是肯定的,其真值位于真与假之间。现实世界中的事物和现象大都是不严格、不精确的,许多概念是模糊的,没有明确的类属界限。在此情况下,若仍用经典逻辑做精确处理,势必要人为地在本来没有明确界限的事物间划定界限,从而舍弃了事物固有的模糊性,失去了真实性。这就是为什么近年来各种非经典逻辑迅速崛起,人工智能亦把不精确知识的表示与处理作为重要研究课题的原因。

3. 单调推理、非单调推理

按推理过程中推出的结论是否单调地增加,或者说推出的结论是否越来越接近最终目标来分类,推理可分为单调推理与非单调推理。

单调推理是指在推理过程中随着推理过程向前推进以及新知识的进入,推出的结论呈单

调增加的趋势，并且越来越接近最终目标，在推理过程中不会出现反复的情况，即不会由于新知识的加入否定了前面推出的结论，从而使推理又退回到前面的某一步。本章将要讨论的基于经典逻辑的演绎推理属于单调性推理。

非单调推理是指在推理过程中由于新知识的加入，不仅没有加强已推出的结论，反而要否定它，使得推理退回到前面的某一步，重新开始。非单调推理多是在知识不完全的情况下发生的。由于知识不完全，为使推理进行下去，就要先做某些假设，并在此假设的基础上进行推理，当以后由于新知识的加入发现原先的假设不正确时，就需要推翻该假设以及以此假设为基础推出的一切结论，再用新知识重新进行推理。显然，前面所说的默认推理是非单调推理。

4. 启发式推理、非启发式推理

按推理中是否运用与问题有关的启发性知识，推理可分为启发式推理与非启发式推理。启发性知识是指与问题有关且能加快推理进程、求得问题最优解的知识。

5. 基于知识的推理、直觉推理

从方法论的角度分类，推理可分为基于知识的推理与直觉推理。

基于知识的推理是根据已掌握的事实，通过运用知识进行推理。我们所讨论的推理都属于这一类。

直觉推理又称为常识性推理，是根据常识进行的推理。例如，当你从某建筑物下面走过时，突然发现有一物体从建筑物上掉落下来，这时你立即会意识到“有危险”，并立即躲开，这就是使用了直觉推理。目前，在计算机上实现直觉推理还是一件很困难的工作，有待进行深入的研究。

除了上述分类方法外，推理还有一些其他分类方法。如根据推理的繁简不同，分为简单推理与复合推理；根据结论是否具有必然性，又可分为必然性推理与或然性推理；在不确定性推理中，推理还可分为似然推理与近似推理（模糊推理），前者是基于概率论的推理，后者是基于模糊逻辑的推理。

3.1.2 推理的控制策略

推理过程是一个求解问题的过程。问题求解的质量与效率依赖于求解问题的策略，即推理的控制策略。

推理的控制策略主要包括推理方向、搜索策略、冲突消解策略、求解策略及限制策略等。这里，我们讨论推理方向、求解策略、限制策略与冲突消解策略。搜索策略（用于确定推理路线）将在下一章进行专门讨论。

1. 推理方向

推理方向用于确定推理的驱动方式，分为正向推理、逆向推理、混合推理及双向推理四种。无论按哪种方向进行推理，一般都要求系统具有一个存放知识的知识库、一个存放初始已知事实及问题状态的数据库和一个用于推理的推理机。

(1) 正向推理

正向推理是以已知事实作为出发点的推理, 又称为数据驱动推理、前向链推理、模式制导推理及前件推理等。

正向推理的基本思想是: 从用户提供的初始已知事实出发, 在知识库 KB 中找出当前可适用的知识, 构成可适用知识集 KS, 然后按某种冲突消解策略从 KS 中选出一条知识进行推理, 并将推出的新事实加入到数据库 DB 中作为下一步推理的已知事实, 在此之后再在知识库中选取可适用知识进行推理, 如此重复进行这一过程, 直到求得了所要求的解或者知识库中再无可适用的知识为止。其推理过程可描述如下:

1) 将用户提供的初始已知事实送入数据库 DB。

2) 检查数据库 DB 中是否已经包含了问题的解, 若有, 则求解结束, 并成功退出; 否则执行下一步。

3) 根据数据库 DB 中的已知事实, 扫描知识库 KB, 检查 KB 中是否有可适用 (即可与 DB 中已知事实匹配) 的知识, 若有, 则转 4); 否则转 6)。

4) 把 KB 中所有的适用知识都选出来, 构成可选用的知识集 KS。

5) 若 KS 不空, 则按某种冲突消解策略从中选出一条知识进行推理, 并将推出的新事实加入 DB 中, 然后转 2); 若 KS 空, 则转 6)。

6) 询问用户是否可进一步补充新的事实, 若可补充, 则将补充的新事实加入 DB 中, 然后转 3); 否则表示求不出解, 失败退出。

上述推理过程在具体实现时还有许多工作要做。例如, 在推理过程中要从知识库 KB 中选出可适用的知识, 这就要用知识库中的知识与数据库中的已知事实进行匹配, 为此就需要确定匹配的方法。而且, 匹配通常难以做到完全一致匹配, 这就需要解决怎样才算是匹配成功的问题。其次, 为了进行匹配, 就要查找知识, 这就牵涉到按什么路线进行查找的问题, 即按什么策略搜索知识库。再如, 如果适用的知识只有一条, 系统立即就可用它进行推理, 并将推出的新事实送入数据库 DB 中; 但是, 如果当前适用的知识有多条, 应该先用哪一条? 这是推理的一个重要问题, 称为冲突消解策略。总之, 为了实现正向推理, 有许多具体问题需要解决, 今后我们将分别对它们进行讨论。

(2) 逆向推理

逆向推理是以某个假设目标作为出发点的推理, 又称为目标驱动推理、逆向链推理、目标制导推理及后件推理等。

逆向推理的基本思想是: 首先选定一个假设目标, 然后寻找支持该假设的证据, 若所需的证据都能找到, 则说明原假设是成立的; 若无论如何都找不到所需要的证据, 则说明原假设不成立, 此时需要另外选定新的假设。其推理过程可描述如下:

1) 提出要求证的目标 (假设)。

2) 检查该目标是否已在数据库 DB 中, 若在, 则该目标成立, 成功退出推理或者对下一个假设目标进行验证; 否则, 转下一步。

3) 判断该目标是否是证据, 即它是否为应由用户证实的原始事实, 若是, 则询问用户; 否则转下一步。

4) 在知识库 KB 中找出所有能导出该目标的知识, 形成适用知识集 KS, 然后转下一步。

5) 从 KS 中选出一条知识, 并将该知识的运用条件作为新的假设目标, 然后转 2)。

与正向推理相比, 逆向推理更复杂一些, 上述过程只是描述了它的大致过程, 许多细节还没有反映出来。

逆向推理的主要优点是不必使用与目标无关的知识, 目的性强。其主要缺点是初始目标的选择有盲目性, 若不符合实际, 就要多次提出假设, 也会影响到系统的效率。

(3) 混合推理

正向推理可能会推出许多与问题求解无关的子目标; 逆向推理中, 若提出的假设目标不符合实际, 也会降低系统的效率。可把正向推理与逆向推理结合起来, 使其各自发挥自己的优势, 取长补短。既有正向又有逆向的推理称为混合推理。在下述几种情况下, 通常需要进行混合推理:

1) 已知的事实不充分。当数据库中的已知事实不够充分时, 若用这些事实与知识的运用条件进行匹配进行正向推理, 可能一条适用知识都选不出来, 这就使推理无法进行下去。此时, 可通过正向推理先把其运用条件不能完全匹配的知识都找出来, 并把这些知识可导出的结论作为假设, 然后分别对这些假设进行逆向推理。由于在逆向推理中可以向用户询问有关证据, 这就可能使推理进行下去。

2) 由正向推理推出的结论可信度不高。用正向推理进行推理时, 虽然推出了结论, 但可信度可能不高, 达不到预定的要求。此时为了得到一个可信度符合要求的结论, 可用这些结论作为假设, 然后进行逆向推理, 通过向用户询问进一步的信息, 有可能会得到可信度较高的结论。

3) 希望得到更多的结论。在逆向推理过程中, 由于可与用户进行对话, 有针对性地向用户提出询问, 这就可能获得一些原来不掌握的有用信息, 这些信息不仅可用于证实要证明的假设, 同时还可能有助于推出一些其他结论。因此, 在用逆向推理证实了某个假设之后, 可以再用正向推理推出另外一些结论。例如在医疗诊断系统中, 先用逆向推理证实了某病人患有某种病, 然后再利用逆向推理过程中获得的信息进行正向推理, 就有可能推出该病人还患有别的什么病。

由以上讨论可以看出, 混合推理分为两种情况: 一种是先进行正向推理, 帮助选择某个目标, 即从已知事实演绎出部分结果, 然后再用逆向推理证实该目标或提高其可信度; 另一种情况是先假设一个目标进行逆向推理, 然后再利用逆向推理中得到的信息进行正向推理, 以推出更多的结论。

(4) 双向推理

双向推理是指正向推理与逆向推理同时进行, 且在推理过程中的某一步骤上“碰头”的一种推理。其基本思想是: 一方面根据已知事实进行正向推理, 但并不推到最终目标; 另一

方面从某假设目标出发进行逆向推理,但并不推至原始事实,而是让它们在中途相遇,即由正向推理所得的中间结论恰好是逆向推理此时所要求的证据,这时推理就可结束,逆向推理的假设就是推理的最终结论。

双向推理的困难在于“碰头”的判断。另外,如何权衡正向推理与逆向推理的比重,即如何确定“碰头”的时机也是一个困难问题。

2. 求解策略

所谓推理的求解策略是指,推理是只求一个解,还是求所有解以及最优解等。例如前述的正向推理只用于求一个解,只要略加修改就可用来求所有解。

3. 限制策略

为了防止无穷的推理过程,以及由于推理过程太长从而增加时间及空间的复杂性,可在控制策略中指定推理的限制条件,以对推理的深度、宽度、时间、空间等进行限制。

4. 冲突消解策略

在推理过程中,系统要不断地用当前已知的事实与知识库中的知识进行匹配,此时可能发生如下三种情况:

1) 已知事实不能与知识库中的任何知识匹配成功。

2) 已知事实恰好与知识库中的一个知识匹配成功。

3) 已知事实可与知识库中的多个知识匹配成功;或者有多个(组)已知事实都可与知识库中某一个知识匹配成功;或者有多个(组)已知事实可与知识库中的多个知识匹配成功。

当第一种情况发生时,由于找不到与当前已知事实匹配成功的知识,就使得推理无法继续进行下去,这或者是由于知识库中缺少某些必要的知识,或者是由于欲求解的问题超出了系统的功能范围等,此时可根据当时的实际情况做相应的处理。对于第二种情况,由于匹配成功的知识只有一个,所以它就是可用知识,可直接把它用于当前的推理。第三种情况不仅有知识匹配成功,而且有多个知识匹配成功,称这种情况为发生了冲突。此时需要按一定策略解决冲突,以便从中挑选一个知识用于当前的推理,称这一解决冲突的过程为冲突消解。解决冲突所用的方法称为冲突消解策略。

例如,在产生式系统中,若出现如下情况就认为发生了冲突:

1) 对正向推理而言,如果有多条产生式规则的前件都和已知事实匹配成功;或者有多组不同的已知事实与同一条产生式规则的前件匹配成功;或者以上两种情况同时出现。

2) 对逆向推理而言,如果有多条产生式规则的后件都和同一个假设匹配成功;或者有多条产生式规则的后件可与多个假设匹配成功。

冲突消解的任务是解决冲突。对正向推理来说,它将决定选择哪一组已知事实来激活哪一条产生式规则,产生其后件指出的结论或执行后件指定的操作。对逆向推理来说,它将决定用哪一个假设与哪一个产生式规则的后件进行匹配,推出相应的前件作为新的假设。

目前已有多种消解冲突的策略,其基本思想都是对知识进行排序,常用的有以下几种。

(1) 按针对性排序

设有如下两条产生式规则:

$$R_1: \text{if } A_1 \text{ and } A_2 \text{ and } \cdots \text{ and } A_n \text{ then } H_1$$

$$R_2: \text{if } B_1 \text{ and } B_2 \text{ and } \cdots \text{ and } B_m \text{ then } H_2$$

如果 R_2 中除了包含 R_1 的全部条件 $A_1, A_2, \dots, A_n$ 外, 还包含其他条件, 则称 R_2 比 R_1 有更大的针对性, R_1 比 R_2 有更大的通用性。

本策略是优先选用针对性较强的产生式规则。因为它要求的条件较多, 其结论一般更接近于目标, 一旦得到满足, 可缩短推理过程。

(2) 按匹配度排序

在不确定性匹配中, 为了确定两个知识模式是否可以匹配, 需要计算这两个模式的相似程度, 当其相似度达到某个预先规定的值时, 就认为它们是可匹配的。相似度又称为匹配度, 它除了可用来确定两个知识模式是否可匹配外, 还可用于冲突消解。若产生式规则 R_1 与 R_2 都可匹配成功, 则可根据它们的匹配度来决定哪一个产生式规则可优先被使用。

(3) 根据领域问题的特点排序

如果事先可知道领域问题的某些特点, 则可根据这些特点事先确定知识库中知识的使用顺序。例如:

1) 当领域问题有固定的求解次序时, 可按该次序对知识库中的知识排序, 排在前面的知识优先被使用。

2) 当已知某些产生式规则被使用后会明显地有利于问题的求解时, 可对这些产生式规则指定较高的优先级, 使这些产生式规则优先被使用。

3.1.3 模式匹配及其变量代换

模式匹配是指两个知识模式 (如两个谓词公式、两个框架片断或两个语义网络片断等) 的比较, 检查这两个知识模式是否完全一致或近似一致。如果两者完全一致, 或者虽不完全一致但其相似程度落在指定的限度内, 就称它们是可匹配的, 否则为不可匹配。

模式匹配是推理中必须进行的一项重要工作, 因为只有经过模式匹配才能从知识库中选出当前适用的知识, 才能进行推理。例如在产生式系统中, 为了由已知的初始事实推出相应的结论, 首先必须从知识库中选出可与已知事实匹配的产生式规则, 然后才能应用这些产生式规则进行推理, 逐步推出结论。框架推理以及语义网络推理与此类似, 也需要先通过匹配选出相应的框架及语义网络片断, 然后再进行推理。

按匹配时两个知识模式的相似程度划分, 模式匹配可分为确定性匹配与不确定性匹配。

确定性匹配是指两个知识模式完全一致, 或者经过变量代换后变得完全一致。例如, 设有如下两个知识模式:

$$P_1: \text{father (李四, 李小四) and man (李小四)}$$

$$P_2: \text{father}(x,y) \text{ and man } (y)$$

若用“李四”代换变量 x , 用“李小四”代换变量 y , 则 P_1 与 P_2 就变得完全一致。若用这

两个知识模式进行匹配,则它们是确定性匹配。确定性匹配又称为完全匹配或精确匹配。

不确定性匹配是指两个知识模式不完全一致,但从总体上看,它们的相似程度又落在规定的限度内。

无论是确定性匹配还是不确定性匹配,在进行匹配时一般都需要进行变量的代换。

定义 3.1 代换是形如

$$\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

的有限集合。其中, $t_1, t_2, \dots, t_n$ 是项; $x_1, x_2, \dots, x_n$ 是互不相同的变元; t_i/x_i 表示用 t_i 代换 x_i , 不允许 t_i 与 x_i 相同, 也不允许变元 x_i 循环地出现在另一个 t_j 中。

例如

$$\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

是一个代换, 但是

$$\{g(y)/x, f(x)/y\}$$

不是一个代换, 因为代换的目的是使某些变元被另外的变元、常量或函数取代, 使之不再在公式中出现, 而 $\{g(y)/x, f(x)/y\}$ 在 x 与 y 之间出现了循环代换的情况, 它既没有消去 x , 也没有消去 y 。如果将它改为

$$\{g(a)/x, f(x)/y\}$$

就可以, 它将把公式中的 x 用 $g(a)$ 代换, y 用 $f(f(a))$ 代换, 从而消去了变元 x 和 y 。

定义 3.2 设

$$\theta = \{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

$$\lambda = \{u_1/y_1, u_2/y_2, \dots, u_m/y_m\}$$

是两个代换, 则此两个代换的复合也是一个代换, 它是从

$$\{t_1 \lambda /x_1, t_2 \lambda /x_2, \dots, t_n \lambda /x_n, u_1/y_1, u_2/y_2, \dots, u_m/y_m\}$$

中删去如下两种元素:

$$\begin{array}{ll} t_i \lambda /x_i & \text{当 } t_i \lambda = x_i \\ u_j/y_j & \text{当 } y_j \in \{x_1, x_2, \dots, x_n\} \end{array}$$

后剩下的元素所构成的集合, 记为 $\theta \circ \lambda$ 。

例如, 设有代换:

$$\theta = \{f(y)/x, z/y\}$$

$$\lambda = \{a/x, b/y, y/z\}$$

则

$$\theta \circ \lambda = \{f(b)/x, y/z\}$$

定义 3.3 设有公式集 $F = \{F_1, F_2, \dots, F_n\}$, 若存在一个代换 λ 使得

$$F_1 \lambda = F_2 \lambda = \dots = F_n \lambda$$

则称 λ 为公式集 F 的一个合一, 且称 $F_1, F_2, \dots, F_n$ 是可合一的。

例如, 设有公式集

$$F=\{P(x, y, f(y)), P(a, g(x), z)\}$$

则下式是它的一个合一:

$$\lambda = \{a/x, g(a)/y, f(g(a))/z\}$$

一个公式集的合一通常是不惟一的。

定义 3.4 设 σ 是公式集 F 的一个合一, 如果对任一个合一 θ 都存在一个代换 λ , 使得

$$\theta = \sigma \circ \lambda$$

则称 σ 是公式集 F 的最一般合一。

最一般合一是惟一的。若用最一般合一去代换那些可合一的谓词公式, 可使它们变成完全一致的谓词公式。由此可知, 为了使两个知识模式匹配, 可用其最一般合一对它们进行代换。

在给出求取最一般合一的算法之前, 先引入差异集的概念。设有如下两个谓词公式:

$$F_1: P(x, y, z)$$

$$F_2: P(x, f(a), h(b))$$

分别从 F_1 与 F_2 的第一个符号开始, 逐个向右比较, 此时发现 F_1 中 y 与 F_2 中的 $f(a)$ 不同, 它们构成了一个差异集:

$$D_1 = \{y, f(a)\}$$

当继续向右边比较时, 又发现 F_1 中的 z 与 F_2 中的 $h(b)$ 不同, 则又得到一个差异集:

$$D_2 = \{z, h(b)\}$$

下面给出求取最一般合一的算法:

1) 令 $k=0, F_k=F, \sigma_k=\epsilon$ 。这里, F 是欲求其最一般合一的公式集, ϵ 是空代换, 它表示不做代换。

2) 若 F_k 只含一个表达式, 则算法停止, σ_k 就是最一般合一。

3) 找出 F_k 的差异集 D_k 。

4) 若 D_k 中存在元素 x_k 和 t_k , 其中 x_k 是变元, t_k 是项, 且 x_k 不在 t_k 中出现, 则置:

$$\sigma_{k+1} = \sigma_k \circ \{t_k/x_k\}$$

$$F_{k+1} = F_k\{t_k/x_k\}$$

$$k=k+1$$

然后转 2)。

5) 算法终止, F 的最一般合一不存在。

例 3.1 设有公式集

$$F=\{P(a, x, f(g(y))), P(z, f(z), f(u))\}$$

求其最一般合一。

1) 令 $\sigma_0 = \epsilon, F_0 = F$, 因 F_0 中含有两个表达式, 所以 σ_0 不是最一般合一。

2) 差异集 $D_0 = \{a, z\}$ 。

3) $\sigma_1 = \sigma_0 \circ \{a/z\} = \{a/z\}$,

$$F_1 = \{P(a, x, f(g(y))), P(a, f(a), f(u))\}。$$

$$4) D_1 = \{x, f(a)\}.$$

$$5) \sigma_2 = \sigma_1 \circ \{f(a)/x\} = \{a/z, f(a)/x\},$$

$$F_2 = F_1\{f(a)/x\} = \{P(a, f(a), f(g(y))), P(a, f(a), f(u))\}.$$

$$6) D_2 = \{g(y), u\}.$$

$$7) \sigma_3 = \sigma_2 \circ \{g(y)/u\} = \{a/z, f(a)/x, g(y)/u\}.$$

$$8) F_3 = F_2\{g(y)/u\} = \{P(a, f(a), f(g(y)))\}.$$

因为 F_3 只含一个表达式, 所以 σ_3 就是最一般合一, 即是最一般合一为:

$$\{a/z, f(a)/x, g(y)/u\}$$

3.2 自然演绎推理

从一组已知为真的事实出发, 直接运用经典逻辑的推理规则推出结论的过程称为自然演绎推理。其中, 基本的推理规则是 P 规则、T 规则、假言推理、拒取式推理等。

假言推理的一般形式是

$$P, P \rightarrow Q \Rightarrow Q$$

它表示: 由 $P \rightarrow Q$ 及 P 为真, 可推出 Q 为真。例如, 由“如果 x 是金属, 则 x 能导电”及“铜是金属”可推出“铜能导电”的结论。

拒取式推理的一般形式是

$$P \rightarrow Q, \neg Q \Rightarrow \neg P$$

它表示: 由 $P \rightarrow Q$ 为真及 Q 为假, 可推出 P 为假。例如, 由“如果下雨, 则地上湿”及“地上不湿”可推出“没有下雨”的结论。

这里, 应注意避免如下两类错误: 一是肯定后件 (Q) 的错误; 二是否定前件 (P) 的错误。所谓肯定后件是指, 当 $P \rightarrow Q$ 为真时, 希望通过肯定后件 Q 为真来推出前件 P 为真, 这是不允许的。所谓否定前件是指, 当 $P \rightarrow Q$ 为真时, 希望通过否定前件 P 来推出后件 Q 为假, 这也是不允许的。

例 3.2 已知如下事实:

1) 凡是容易的课程小王 (Wang) 都喜欢。

2) C 班的课程都是容易的。

3) D_s 是 C 班的一门课程。

求证: 小王喜欢 D_s 这门课程。

证 首先定义谓词:

$EASY(x)$: x 是容易的。

$LIKE(x, y)$: x 喜欢 y 。

$C(x)$: x 是 C 班的一门课程。

把上述已知事实及待求证的问题用谓词公式表示出来:

$EASY(x) \rightarrow LIKE(Wang, x)$	凡是容易的课程小王都喜欢。
$(\forall x)(C(x) \rightarrow EASY(x))$	C 班的课程都是容易的。
$C(Ds)$	Ds 是 C 班的课程。
$LIKE(Wang, Ds)$	小王喜欢 Ds 这门课程, 这是待求证的问题。

应用推理规则进行推理:

$(\forall x)(C(x) \rightarrow EASY(x))$	
$C(y) \rightarrow EASY(y)$	全称固化
$C(Ds), C(y) \rightarrow EASY(y) \Rightarrow EASY(Ds)$	P 规则及假言推理
$EASY(Ds), EASY(x) \rightarrow LIKE(Wang, x) \Rightarrow LIKE(Wang, Ds)$	T 规则及假言推理

即小王喜欢 Ds 这门课程。

自然演绎推理的优点是表达定理证明过程自然, 容易理解, 而且它可以拥有丰富的推理规则, 推理过程灵活, 便于在它的推理规则中嵌入领域启发式知识。其缺点是容易产生组合爆炸, 推理过程中得到的中间结论的数量可能呈指数形式递增, 这对于一个人的复杂问题的推理求解是十分不利的。

3.3 归结演绎推理

自动定理证明是人工智能的一个重要研究领域, 定理证明的实质是对已知前提 P 和待证结论 Q 证明 $P \rightarrow Q$ 的永真性。但是, 正如我们在第 2 章所讨论的那样, 要证明一个谓词公式的永真性是相当困难的, 甚至在某些情况下是不可能的。通过研究发现, 应用反证法的思想可把关于永真性的证明转化为不可满足性的证明, 即欲证明 $P \rightarrow Q$ 永真, 只要证明 $P \wedge \neg Q$ 是不可满足的就可以了。关于不可满足性的证明, 海伯伦 (Herbrand) 及鲁宾逊 (Robinson) 先后进行了卓有成效的研究, 提出了相应的理论和方法。海伯伦的理论和鲁宾逊的归结原理, 都是以子句集为背景开展研究的。

3.3.1 谓词公式化为子句集的方法

在谓词逻辑中, 把原子谓词公式及其否定统称为文字。

定义 3.5 任何文字的析取式称为子句。

例如, $P(x) \vee Q(x), \neg P(x, f(x)) \vee Q(x, g(x))$ 都是子句。

定义 3.6 不包含任何文字的子句称为空子句。

由于空子句不含有文字, 它不能被任何解释满足, 所以空子句是永假的, 不可满足的。

由子句构成的集合称为子句集。在谓词逻辑中, 任何一个谓词公式都可通过应用等价关系及推理规则化成相应的子句集。下面给出把谓词公式化成子句集的步骤:

1) 利用下述等价关系消去谓词公式中的 “ $\rightarrow$ ”

$$P \rightarrow Q \Leftrightarrow \neg P \vee Q$$

例如公式

$$(\forall x)((\forall y)P(x,y)) \rightarrow \neg(\forall y)(Q(x,y) \rightarrow R(x,y))$$

经等价变换后变成

$$(\forall x)(\neg(\forall y)P(x,y) \vee \neg(\forall y)(\neg Q(x,y) \vee R(x,y)))$$

2) 利用下述等价关系把“ $\neg$ ”移到紧靠谓词的位置上:

$$\begin{aligned}\neg(\neg P) &\Leftrightarrow P \\ \neg(P \wedge Q) &\Leftrightarrow \neg P \vee \neg Q \\ \neg(P \vee Q) &\Leftrightarrow \neg P \wedge \neg Q \\ \neg(\forall x)P &\Leftrightarrow (\exists x)\neg P \\ \neg(\exists x)P &\Leftrightarrow (\forall x)\neg P\end{aligned}$$

上式经此等价变换后变为

$$(\forall x)((\exists y)\neg P(x,y) \vee (\exists y)(Q(x,y) \wedge \neg R(x,y)))$$

3) 重新命名变元名, 使不同量词约束的变元有不同的名字。上式经此变换后变为

$$(\forall x)((\exists y)\neg P(x,y) \vee (\exists z)(Q(x,z) \wedge \neg R(x,z)))$$

4) 消去存在量词。若存在量词不在全称量词的辖域内, 则要用一个新的个体常量替换受该存在量词约束的变元就可消去存在量词(因为若原公式为真, 则总能找到一个个体常量, 替换后仍使公式为真); 若存在量词位于一个或多个全称量词的辖域内, 例如

$$(\forall x_1)(\forall x_2)\cdots(\forall x_n)(\exists y)P(x_1, x_2, \cdots, x_n, y)$$

则需要用 Skolem 函数 $f(x_1, x_2, \cdots, x_n)$ 替换受该存在量词约束的变元 y , 然后才能消去存在量词。

在上式中, 存在量词 $(\exists y)$ 及 $(\exists z)$ 都位于 $(\forall x)$ 的辖域内, 所以都需要用 Skolem 函数替换, 设替换 y 和 z 的 Skolem 函数分别是 $f(x)$ 和 $g(x)$, 则替换后得到

$$(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$$

5) 把全称量词全部移到公式的左边。在上式中由于只有一个全称量词, 而且它已位于公式的最左边, 所以这里不需要做任何工作。如果在公式内部有全称量词, 就需要把它们都移到公式的左边。

6) 利用下述等价关系

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

把公式化为 Skolem 标准形。

Skolem 标准形的一般形式是

$$(\forall x_1)(\forall x_2)\cdots(\forall x_n)M$$

其中, M 是子句的合取式, 称为 Skolem 标准形的母式。

把第 5) 步得到的公式化为 Skolem 标准形后得到

$$(\forall x)((\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$$

7) 消去全称量词。由于上式中只有一个全称量词, 所以可直接把它消去, 得到

$$(\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x)))$$

8) 对变元更名, 使不同子句中的变元不同名。上式经更名后得到

$$(\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(y, f(y)) \vee \neg R(y, g(y)))$$

9) 消去合取词。消去合取词后, 上式就变为下述子句集:

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

$$\neg P(y, f(y)) \vee R(y, g(y))$$

如果谓词公式是不可满足的, 则其子句集也一定是不可满足的, 反之亦然。因此, 在不可满足的意义上两者是等价的, 由此给出下述定理。

定理 3.1 设有谓词公式 F , 其标准形的子句集为 S , 则 F 不可满足的充要条件是 S 不可满足。

由此定理可知, 为要证明一个谓词公式是不可满足的, 只要证明相应的子句集是不可满足的就可以了。

3.3.2 海伯伦理论

为了判定子句集的不可满足性, 就需要对子句集中的子句判定其不可满足性。为判断一个子句的不可满足性, 需要对个体域上的一切解释逐个地进行判定, 只有当子句对任何非空个体域上的任何解释都是不可满足的, 才能判定该子句是不可满足的。海伯伦构造了一个特殊的域, 并证明只要对这个特殊域上的一切解释进行判定, 就可得知子句集是否不可满足, 这个特殊的域称为海伯伦域。

定义 3.7 设 S 为子句集, 则按下述方法构造的域 H_∞ 称为海伯伦域, 简记为 H 域:

1) 令 H_0 是 S 中所有个体常量的集合, 若 S 中不包含个体常量, 则令 $H_0 = \{a\}$, 其中 a 为任意指定的一个个体常量。

2) 令 $H_{i+1} = H_i \cup \{S \text{ 中所有 } n \text{ 元函数 } f(x_1, \dots, x_n) | x_j (j=1, \dots, n) \text{ 是 } H_i \text{ 中的元素}\}$, 其中, $i=0, 1, 2, \dots$ 。

下面用例子解释这个定义。

例 3.3 求子句集 $S = \{P(x) \vee Q(x), R(f(y))\}$ 的 H 域。

解 在此例中没有个体常量, 根据 H 域的定义可以任意指定一个常量 a 作为个体常量, 于是得到:

$$H_0 = \{a\}$$

$$H_1 = \{a, f(a)\}$$

$$H_2 = \{a, f(a), f(f(a))\}$$

$$H_3 = \{a, f(a), f(f(a)), f(f(f(a)))\}$$

$\vdots$

$$H_\infty = \{a, f(a), f(f(a)), f(f(f(a))), \dots\}$$

例 3.4 求子句集 $S = \{P(a), Q(b), R(f(x))\}$ 的 H 域。

解 根据 H 域的定义得到:

$$\begin{aligned}
 H_0 &= \{a, b\} \\
 H_1 &= \{a, b, f(a), f(b)\} \\
 H_2 &= \{a, b, f(a), f(b), f(f(a)), f(f(b))\} \\
 &\vdots
 \end{aligned}$$

如果用 H 域中的元素代换子句中的变元, 则所得的子句称为基子句, 其中的谓词称为基原子。子句集中所有基原子构成的集合称为原子集。子句集 S 在 H 域上的解释就是对 S 中出现的常量、函数及谓词取值, 一次取值就是一个解释。

定义 3.8 子句集 S 在 H 域上的一个解释 I 满足下列条件:

- 1) 在解释 I 下, 常量映射到自身。
- 2) S 中的任一个 n 元函数是 $H^n \rightarrow H$ 的映射。即, 设 $h_1, h_2, \dots, h_n \in H$, 则 $f(h_1, h_2, \dots, h_n) \in H$ 。
- 3) S 中的任一个 n 元谓词是 $H^n \rightarrow \{T, F\}$ 的映射。谓词的真值可以指派为 T , 也可以指派为 F 。

例如, 设子句集 $S = \{P(a), Q(f(x))\}$, 它的 H 域为 $\{a, f(a), f(f(a)), \dots\}$, 那么 S 的原子集为 $\{P(a), Q(f(a)), Q(f(f(a))), \dots\}$, 则 S 的解释为:

$$\begin{aligned}
 I_1 &= \{p(a), Q(f(a)), Q(f(f(a))), \dots\} \\
 I_2 &= \{p(a), \neg Q(f(a)), Q(f(f(a))), \dots\} \\
 &\vdots
 \end{aligned}$$

一般来说, 一个子句集的基原子有无限多个, 它在 H 域上的解释也有无限多个。

可以证明, 对给定域 D 上的任一个解释, 总能在 H 域上构造一个解释与它对应, 如果 D 域上的解释能满足子句集 S , 则在 H 域上的相应解释也能满足 S 。由此可推出如下两个定理。

定理 3.2 子句集 S 不可满足的充要条件是 S 对 H 域上的一切解释都为假。

定理 3.3 (海伯伦定理) 子句集 S 不可满足的充要条件是存在一个有限的不可满足的基子句集 S' 。

证明 充分性:

设子句集 S 有一个不可满足的基子句集 S' , 因为它不可满足, 所以一定存在一个解释 I' 使 S' 为假。根据 H 域上的解释与 D 域上的解释的对应关系, 可知在 D 域上一定存在一个解释使 S 不可满足, 即子句集 S 是不可满足的。

必要性:

设子句集 S 不可满足, 由定理 3.2 可知 S 对 H 域上的一切解释都为假, 这样必然存在一个基子句集 S' , 且它是不可满足的。

直接由海伯伦理论证明子句集 S 的不可满足性仍然是很困难的, 因为在证明过程中, 要在 S 的 H 域上生成一系列的基原子集, 并逐个检查这些基原子集的不可满足性。一般情况下, 一个子句集的基原子有无限多个, 因此, 证明过程的时空复杂程度通常无法容忍。为避免这类组合爆炸, 1965 年鲁宾逊提出了归结原理, 又称为消解原理, 才使机器定理证明变为现实。

3.3.3 归结原理

由谓词公式转化子句集的过程可以看出, 在子句集中子句之间是合取关系, 其中只要有一个子句不可满足, 则子句集就不可满足。另外, 已经指出空子句是不可满足的。因此, 若一个子句集中包含空子句, 则这个子句集一定是不可满足的。鲁宾逊归结原理就是基于这一认识提出来的。其基本思想是: 检查子句集 S 中是否包含空子句, 若包含, 则 S 不可满足; 若不包含, 就在子句集中选择合适的子句进行归结, 一旦通过归结能推出空子句, 就说明子句集 S 是不可满足的。

定义 3.9 若 P 是原子谓词公式, 则称 P 与 $\neg P$ 为互补文字。

1. 命题逻辑中的归结原理

定义 3.10 设 C_1 与 C_2 是子句集中的任意两个子句, 如果 C_1 中的文字 L_1 与 C_2 中的文字 L_2 互补, 那么从 C_1 和 C_2 中分别消去 L_1 和 L_2 , 并将二个子句中余下的部分析取, 构成一个新子句 C_{12} , 则称这一过程为归结, 称 C_{12} 为 C_1 和 C_2 的归结式, 称 C_1 和 C_2 为 C_{12} 的亲本子句。

定理 3.4 归结式 C_{12} 是其亲本子句 C_1 与 C_2 的逻辑结论。

证明 设

$$C_1 = L \vee C_1', C_2 = \neg L \vee C_2'$$

通过归结可以得到

$$C_{12} = C_1' \vee C_2'$$

C_1 和 C_2 是 C_{12} 的亲本子句。

$$\therefore C_1' \vee L \Leftrightarrow \neg C_1' \rightarrow L$$

$$\neg L \vee C_2' \Leftrightarrow L \rightarrow C_2'$$

$$\therefore C_1 \wedge C_2 = (\neg C_1' \rightarrow L) \wedge (L \rightarrow C_2')$$

根据假言三段论得到:

$$(\neg C_1' \rightarrow L) \wedge (L \rightarrow C_2') \Rightarrow \neg C_1' \rightarrow C_2'$$

$$\therefore \neg C_1' \rightarrow C_2' \Leftrightarrow C_1' \vee C_2' = C_{12}$$

$$\therefore C_1 \wedge C_2 \Rightarrow C_{12}$$

由第 2 章关于逻辑结论的定义 2.6, 可知 C_{12} 是其亲本子句 C_1 和 C_2 的逻辑结论。

这个定理是归结原理中的一个很重要的定理, 由它可得到如下两个推论。

推论 3.1 设 C_1 与 C_2 是子句集 S 中的两个子句, C_{12} 是它们的归结式, 若用 C_{12} 代替 C_1 和 C_2 后得到新子句集 S_1 , 则由 S_1 的不可满足性可推出原子句集 S 的不可满足性, 即

$$S_1 \text{ 的不可满足性} \Rightarrow S \text{ 的不可满足性}$$

推论 3.2 设 C_1 与 C_2 是子句集 S 中的两个子句, C_{12} 是它们的归结式, 若把 C_{12} 加入 S 中, 得到新子句集 S_2 , 则 S 与 S_2 在不可满足的意义上是等价的, 即

$$S_2 \text{ 的不可满足性} \Leftrightarrow S \text{ 的不可满足性}$$

这两个推论给出了用归结原理证明子句集不可满足性的基本思想: 为了要证明子句集 S

的不可满足性,只要对其中可进行归结的子句进行归结,并把归结式加入子句集 S ,或者用归结式替换它的亲本子句,然后对新子句集 (S_1 或 S_2) 证明不可满足性就可以了。如果经过归结能得到空子句,根据空子句的不可满足性,立即可得出原子句集 S 是不可满足的。

在命题逻辑中,对不可满足的子句集 S ,归结原理是完备的,即:若子句集不可满足,则必然存在一个从 S 到空子句的归结演绎;若存在一个从 S 到空子句的归结演绎,则 S 一定是不可满足的。但是对于可满足的子句集 S ,用归结原理得不到任何结果。

2. 谓词逻辑中的归结原理

在谓词逻辑中,由于子句中含有变元,所有不可直接消去互补文字,而需要先用最一般合一^①对变元进行代换,然后才能进行归结。例如,设有如下两个子句:

$$C_1 = P(x) \vee Q(x)$$

$$C_2 = \neg P(a) \vee R(y)$$

由于 $P(x)$ 与 $P(a)$ 不同,所以 C_1 与 C_2 不能直接进行归结,但若用最一般合一

$$\sigma = \{a/x\}$$

对两个子句分别进行代换:

$$C_1 \sigma = P(a) \vee Q(a) \quad C_2 \sigma = \neg P(a) \vee R(y)$$

就可对它们进行归结,消去 $P(a)$ 与 $\neg P(a)$, 得到如下归结式:

$$Q(a) \vee R(y)$$

下面给出谓词逻辑中关于归结的定义。

定义 3.11 设 C_1 与 C_2 是两个没有相同变元的子句, L_1 和 L_2 分别是 C_1 和 C_2 中的文字,若 σ 是 L_1 和 $\neg L_2$ 的最一般合一,则称

$$C_{12} = (C_1 \sigma - \{L_1 \sigma\}) \cup (C_2 \sigma - \{L_2 \sigma\})$$

为 C_1 和 C_2 的二元归结式, L_1 和 L_2 称为归结式的文字。

例 3.5 设

$$C_1 = P(a) \vee \neg Q(x) \vee R(x)$$

$$C_2 = \neg P(y) \vee Q(b)$$

若选 $L_1 = P(a), L_2 = \neg P(y)$, 则 $\sigma = \{a/y\}$ 是 L_1 与 $\neg L_2$ 的最一般合一,根据定义 3.11, 可得:

$$\begin{aligned} C_{12} &= (C_1 \sigma - \{L_1 \sigma\}) \cup (C_2 \sigma - \{L_2 \sigma\}) \\ &= (\{P(a), \neg Q(x), R(x)\} - \{P(a)\}) \cup (\{\neg P(a), Q(b)\} - \{\neg P(a)\}) \\ &= (\{\neg Q(x), R(x)\}) \cup (\{Q(b)\}) \\ &= \{\neg Q(x), R(x), Q(b)\} \\ &= \neg Q(x) \vee R(x) \vee Q(b) \end{aligned}$$

若选 $L_1 = \neg Q(x), L_2 = Q(b)$, $\sigma = \{b/x\}$, 则可得:

$$\begin{aligned} C_{12} &= (\{P(a), \neg Q(b), R(b)\} - \{\neg Q(b)\}) \cup (\{\neg P(y), Q(b)\} - \{Q(b)\}) \\ &= (\{P(a), R(b)\}) \cup (\{\neg P(y)\}) \\ &= \{P(a), R(b), \neg P(y)\} \end{aligned}$$

$$=P(a) \vee R(b) \vee \neg P(y)$$

例 3.6 设

$$C_1=P(x) \vee Q(a), \quad C_2=\neg P(b) \vee R(x)$$

由于 C_1 与 C_2 有相同的变元, 不符合定义 3.11 的要求。为了进行归结, 需修改 C_2 中变元的名字, 令 $C_2=\neg P(b) \vee R(y)$ 。此时, 对 C_1 和 C_2 有

$$L_1=P(x), \quad L_2=\neg P(b)$$

L_1 与 $\neg L_2$ 的最一般合一 $\sigma=\{b/x\}$, 则

$$\begin{aligned} C_{12} &= (\{P(b), Q(a)\} - \{P(b)\}) \cup (\{\neg P(b), R(y)\} - \{\neg P(b)\}) \\ &= \{Q(a), R(y)\} \\ &= Q(a) \vee R(y) \end{aligned}$$

如果在参加归结的子句内部含有可合一的文字, 则在进行归结之前应对这些文字先进行合一。

例 3.7 设有如下两个子句:

$$C_1=P(x) \vee P(f(a)) \vee Q(x)$$

$$C_2=\neg P(y) \vee R(b)$$

在 C_1 中有可合一的文字 $P(x)$ 与 $P(f(a))$, 若用它们的最一般合一 $\theta=\{f(a)/x\}$ 进行代换, 得到

$$C_1 \theta = P(f(a)) \vee Q(f(a))$$

此时可对 $C_1 \theta$ 和 C_2 进行归结, 从而得到 C_1 与 C_2 的二元归结式。

对 $C_1 \theta$ 和 C_2 分别选 $L_1=P(f(a)), L_2=\neg P(y)$ 。 L_1 和 $\neg L_2$ 的最一般合一是 $\sigma=\{f(a)/y\}$, 则

$$C_{12}=R(b) \vee Q(f(a))$$

3.3.4 归结反演

为了证明 Q 为 $P_1, P_2, \dots, P_n$ 的逻辑结论, 根据定理 2.1 只需证明谓词公式

$$(P_1 \wedge P_2 \wedge \dots \wedge P_n) \wedge \neg Q$$

是不可满足的; 根据定理 3.1, 在不可满足的意义上, 该谓词公式与其子句集是等价的。我们可用归结原理来进行子句集的不可满足性的证明, 从而证明了 Q 是 $P_1, P_2, \dots, P_n$ 的逻辑结论。

应用归结原理证明结论为真的过程称为归结反演。

设 F 为已知前提的公式集, Q 为目标公式 (结论), 用归结反演证明 Q 为真的步骤是:

- 1) 否定 Q , 得到 $\neg Q$ 。
- 2) 把 $\neg Q$ 并入到公式集 F 中, 得到 $\{F, \neg Q\}$ 。
- 3) 把公式集 $\{F, \neg Q\}$ 化为子句集 S 。
- 4) 应用归结原理对子句集 S 中的子句进行归结, 并把每次归结得到的归结式都并入 S 中。如此反复进行, 若出现了空子句, 则停止归结, 此时就证明了 Q 为真。

例 3.8 已知

$$F: (\forall x)(\exists y)(A(x,y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x,y))$$

$$G: \neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x,y) \rightarrow \neg B(y))$$

求证: G 是 F 的逻辑结论。

证 首先把 F 和 $\neg G$ 化为子句集:

$$\left. \begin{array}{l} (1) \neg A(x,y) \vee \neg B(y) \vee C(f(x)) \\ (2) \neg A(x,y) \vee \neg B(y) \vee D(x, f(x)) \end{array} \right\} F$$

$$\left. \begin{array}{l} (3) \neg C(z) \\ (4) A(a,b) \\ (5) B(b) \end{array} \right\} \neg G$$

下面进行归结:

- (6) $\neg A(x,y) \vee \neg B(y)$ 由 (1) 与 (3) 归结, $\{f(x)/z\}$
- (7) $\neg B(b)$ 由 (4) 与 (6) 归结, $\{a/x, b/y\}$
- (8) NIL (空子句) 由 (5) 与 (7) 归结

$\therefore G$ 是 F 的逻辑结论。

上述归结过程可用图 3.1 所示的归结树表示。

例 3.9 在第 2 章的例 2.4 中曾经得到如下公式:

$$F_1: (\forall x)(N(x) \rightarrow GZ(x) \wedge I(x)) \quad \text{自然数都是大于零的整数。}$$

$$F_2: (\forall x)(I(x) \rightarrow E(x) \vee O(x)) \quad \text{所有整数不是偶数就是奇数。}$$

$$F_3: (\forall x)(E(x) \rightarrow I(s(x))) \quad \text{偶数除以 2 是整数。}$$

求证: 所有自然数不是奇数就是其一半为整数的数。

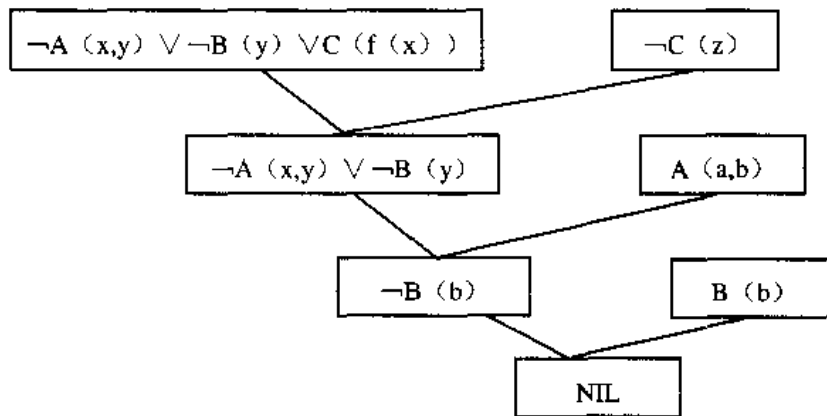


图 3.1 例 3.8 的归结树

证 首先把求证的问题用谓词公式表示出来:

$$G: (\forall x)(N(x) \rightarrow (O(x) \vee I(s(x))))$$

把 F_1 , F_2 , F_3 及 $\neg G$ 化成子句集:

- (1) $\neg N(x) \vee GZ(x)$
- (2) $\neg N(u) \vee I(u)$
- (3) $\neg I(y) \vee E(y) \vee O(y)$
- (4) $\neg E(z) \vee I(s(z))$
- (5) $N(t)$
- (6) $\neg O(t)$
- (7) $\neg I(s(t))$

对上述子句进行归结:

- | | |
|---------------------------|--------------------------|
| (8) $\neg I(y) \vee E(y)$ | (3) 与 (6) 归结, $\{y/t\}$ |
| (9) $\neg E(z)$ | (4) 与 (7) 归结, $\{z/t\}$ |
| (10) $\neg I(y)$ | (8) 与 (9) 归结, $\{y/z\}$ |
| (11) $\neg N(u)$ | (2) 与 (10) 归结, $\{u/y\}$ |
| (12) NIL | (5) 与 (11) 归结, $\{u/t\}$ |

$\therefore$ 所有自然数不是奇数就是其一半为整数的数。

上述归结过程可用图 3.2 所示的归结树表示。

3.3.5 基于归结反演的问题求解

归结反演除了可用于定理证明外, 还可用来求取问题的答案, 问题求解的方法与定理证明类似。问题求解的步骤如下:

- 1) 把已知前提用谓词公式表示出来, 并且化为相应的子句集 S 。

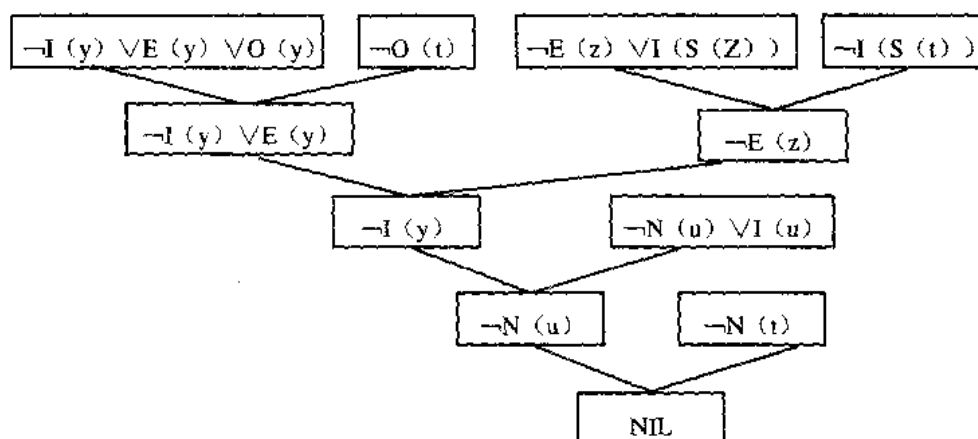


图 3.2 例 3.9 的归结树

2) 把待求解的问题也用谓词公式表示出来, 然后把它的否定式与谓词 ANSWER 构成一个析取式, ANSWER 是一个为了求解问题而专设的谓词, 其变元必须与问题公式的变元完全一致。

3) 把此析取式化为子句集, 并且把该子句集并入到子句集 S 中, 得到子句集 S'。

4) 对 S' 应用归结原理进行归结。

5) 若得到归结式 ANSWER, 则答案就在 ANSWER 中。

例 3.10 已知

F_1 : $T(\text{Wang}, \text{Li})$ 先生是小李 (Li) 的老师。

F_2 : 小李与小张 (Zhang) 是同班同学。

F_3 : 如果 x 与 y 是同班同学, 则 x 的老师也是 y 的老师。

求: 小张的老师是谁?

解 首先定义谓词:

$T(x, y)$: x 是 y 的老师。

$C(x, y)$: x 与 y 是同班同学。

把已知前提及待求解的问题表示成谓词公式:

F_1 : $T(\text{Wang}, \text{Li})$

F_2 : $C(\text{Li}, \text{Zhang})$

F_3 : $(\forall x)(\forall y)(\forall z)(C(x, y) \wedge T(z, x) \rightarrow T(z, y))$

G : $\neg(\exists x)T(x, \text{Zhang}) \vee \text{ANSWER}(x)$

把上述公式化为子句集:

(1) $T(\text{Wang}, \text{Li})$

(2) $C(\text{Li}, \text{Zhang})$

(3) $\neg C(x, y) \vee \neg T(z, x) \vee T(z, y)$

(4) $\neg T(u, Zhang) \vee ANSWER(u)$

应用归结原理进行归结:

(5) $\neg C(Li, y) \vee T(Wang, y)$ (1) 与 (3) 归结

(6) $\neg C(Li, Zhang) \vee ANSWER(Wang)$ (4) 与 (5) 归结

(7) $ANSWER(Wang)$ (2) 与 (6) 归结

由 $ANSWER(Wang)$ 得出小张的老师是王老师。

上述归结过程可用图 3.3 所示的归结树表示。

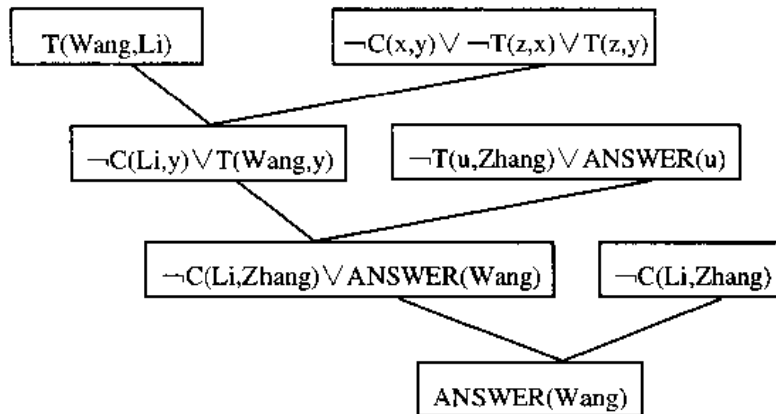


图 3.3 例 3.10 的归结树

例 3.11 设 A, B, C 三人中有人从不说真话, 也有人从不说假话, 某人向这三人分别提出同一个问题: 谁是说谎者? A 答: “B 和 C 都是说谎者”; B 答: “A 和 C 都是说谎者”; C 答: “A 和 B 中至少有一个是说谎者”。求谁是老实人, 谁是说谎者?

解 设用 $T(x)$ 表示 x 说真话。

如果 A 说的是真话, 则有

$$T(A) \rightarrow \neg T(B) \wedge \neg T(C)$$

如果 A 说的是假话, 则有

$$\neg T(A) \rightarrow T(B) \vee T(C)$$

对 B 和 C 说的话作相同的处理, 可得:

$$T(B) \rightarrow \neg T(A) \wedge \neg T(C)$$

$$\neg T(B) \rightarrow T(A) \vee T(C)$$

$$T(C) \rightarrow \neg T(A) \vee \neg T(B)$$

$$\neg T(C) \rightarrow T(A) \wedge T(B)$$

$$(4) \neg T(B) \vee \neg T(C)$$

$$(5) \neg T(C) \vee \neg T(A) \vee \neg T(B)$$

$$(6) T(C) \vee T(A)$$

$$(7) T(C) \vee T(B)$$

下面首先求谁是老实人。把目标公式 $(\exists x)T(x)$ 的否定式 $\neg(\exists x)T(x)$ 与 $ANSWER(x)$ 组成的析取式化为子句并入 S 得到 S_1 , 即 S_1 比 S 多如下一个子句:

$$(8) \neg T(x) \vee ANSWER(x)$$

应用归结原理对 S_1 进行归结:

$$(9) \neg T(A) \vee T(C) \quad (1) \text{ 和 } (7) \text{ 归结}$$

$$(10) T(C) \quad (6) \text{ 和 } (9) \text{ 归结}$$

$$(11) ANSWER(C) \quad (8) \text{ 和 } (10) \text{ 归结}$$

$\therefore C$ 是老实人, 即 C 从不说假话。

除此之外, 无论如何对 S_1 进行归结, 都推不出 $ANSWER(B)$ 和 $ANSWER(A)$ 。

下面来证明 A 和 B 不是老实人。

设 A 不是老实人, 则有 $\neg T(A)$, 把它否定并入到 S 中, 得到子句集 S_2 , 即 S_2 比 S 多如下一个子句:

$$(8) \neg(\neg T(A)) \text{ 即 } T(A)$$

应用归结原理对 S_2 进行归结:

$$(9) \neg T(A) \vee T(C) \quad (1) \text{ 和 } (7) \text{ 归结}$$

$$(10) \neg T(A) \quad (2) \text{ 和 } (9) \text{ 归结}$$

$$(11) NIL \quad (8) \text{ 和 } (10) \text{ 归结}$$

$\therefore A$ 不是老实人。

同理, 可证明 B 也不是老实人。

由上面的例子可以看出, 在归结时并不要求把子句集中的全部子句都用到, 只要在定理证明时能归结出空子句, 在求取问题答案时能归结出 $ANSWER$ 就可以了。另外, 在归结过程中, 一个子句还可以多次被用来进行归结。

3.3.6 归结反演策略

对子句集进行归结反演时, 由于事先不知道哪两个子句可以进行归结, 更不知道通过对哪些子句对的归结可以尽快地得到空子句, 因而必须对子句集中的所有子句逐对地进行比较, 对任何一个可归结的子句对都进行归结, 造成了时空的浪费, 降低了效率。为此, 人们研究出了多种归结反演策略。这些归结反演策略大致可分为两大类: 一类是删除策略; 另一类是限制策略。前一类通过删除某些无用的子句来缩小归结的范围, 后一类通过对参加归结的子句进行种种限制, 尽可能地减小归结的盲目性, 使其尽快地归结出空子句。

1. 归结的一般过程

设有初始子句集

$$S = \{C_1, C_2, C_3, C_4\}$$

其中, C_1, C_2, C_3, C_4 是 S 中的子句。对此子句集进行归结的一般过程是:

1) 从子句 C_1 开始, 逐个与 C_2, C_3, C_4 进行比较, 看哪两个子句可进行归结。若能找到, 就求出归结式。然后用 C_2 与 C_3, C_4 进行比较, 凡可归结的都进行归结, 最后用 C_3 与 C_4 比较, 若能归结也对它们进行归结。经过这一轮的比较及归结后, 就会得到一组归结式, 称为第一级归结式。

2) 再从 C_1 开始, 用 S 中的所有子句分别与第一级归结式中的子句逐个地进行比较、归结, 这样又会得到一组归结式, 称为第二级归结式。

3) 仍然从 C_1 开始, 用 S 中的所有子句及第一级归结式中的所有子句逐个地与第二级归结式中的子句进行比较, 得到第三级归结式。

如此继续, 只要子句集是不可满足的, 上述归结过程一定会归结出空子句而终止。

2. 删除策略

归结过程是一个不断寻找可归结子句的过程, 子句越多, 时空花费就越大。如果在归结时先把子句集中的无用子句删除掉, 就会提高归结的效率。删除策略正是出于这一考虑提出来的, 它有以下几种删除方法。

(1) 纯文字删除法

如果某文字 L 在子句集中不存在可与之互补的文字 $\neg L$, 则称该文字为纯文字。显然, 在归结时纯文字不可能被消去, 因而用包含它的子句进行归结时不可能得到空子句, 即这样的子句对归结是无意义的, 所以可以把它所在的子句从子句集中删去而不会影响子句集的不可满足性。例如, 设有子句集:

$$S = \{P \vee Q \vee R, \neg Q \vee R, Q, \neg R\}$$

其中, P 是纯文字, 因此可将子句 $P \vee Q \vee R$ 从 S 中删去。

(2) 重言式删除法

如果一个子句中同时包含互补文字时, 则称该子句为重言式。例如 $P(x) \vee \neg P(x)$, $P(x) \vee Q(x) \vee \neg P(x)$ 都是重言式。重言式是真值为真的子句。不管 $P(x)$ 为真还是为假, $P(x) \vee \neg P(x)$ 以及 $P(x) \vee Q(x) \vee \neg P(x)$ 都均为真。对于一个子句集来说, 增加或者删去一个真值为真的子句都不会影响它的不可满足性, 因而可从子句集中删去重言式。

(3) 包孕删除法

设有子句 C_1 和 C_2 , 如果存在一个代换 σ , 使得 $C_1 \sigma \subseteq C_2$, 则称 C_1 包孕于 C_2 。例如:

$$P(x) \text{ 包孕于 } P(y) \vee Q(z) \quad \sigma = \{y/x\}$$

$$P(x) \vee Q(a) \text{ 包孕于 } P(f(a)) \vee Q(a) \vee R(y) \quad \sigma = \{f(a)/x\}$$

$$P(x) \vee Q(y) \text{ 包孕于 } P(a) \vee Q(u) \vee R(w) \quad \sigma = \{a/x, u/y\}$$

把子句集中被包孕的子句删去后, 不会影响子句集的不可满足性, 因而可从子句集中删

去被其他子句包孕的子句。

3. 支持集策略

支持集策略对参加归结的子句提出了如下限制：每一次归结时，亲本子句中至少应有一个是由目标公式的否定所得到的子句，或者是它们的后裔。可以证明，支持集策略是完备的，即若子句集是不可满足的，则由支持集策略一定可以归结出空子句。

例 3.12 设有初始子句集：

$$S = \{ \neg I(x) \vee R(x), I(a), \neg R(y) \vee \neg L(y), L(a) \}$$

其中 $\neg I(x) \vee R(x)$ 是目标公式否定得到的子句。

用支持集策略进行归结的过程是：

- S: (1) $\neg I(x) \vee R(x)$
 (2) $I(a)$
 (3) $\neg R(y) \vee \neg L(y)$
 (4) $L(a)$
- S_1 : (5) $R(a)$ (1) 与 (2) 归结
 (6) $\neg I(x) \vee \neg L(x)$ (1) 与 (3) 归结
- S_2 : (7) $\neg L(a)$ (2) 与 (6) 归结
 (8) $\neg L(a)$ (3) 与 (5) 归结
 (9) $\neg I(a)$ (4) 与 (6) 归结
- S_3 : (10) NIL (2) 与 (9) 归结

上述归结过程的归结树如图 3.4 所示。

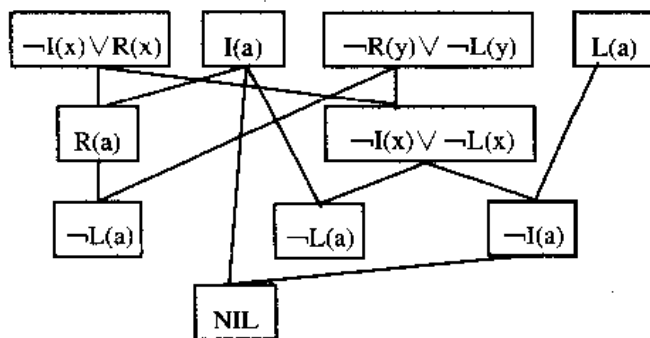


图 3.4 支持集策略归结树

4. 线性输入策略

线性输入策略对参加归结的子句提出了如下限制：参加归结的两个子句中必须至少有一个是初始子句集中的子句。初始子句集是由已知前提与结论的否定组成的子句集。

例 3.13 应用线性输入策略对例 3.12 的初始子句集进行归结。

- S: (1) $\neg I(x) \vee R(x)$

- (2) $I(a)$
 (3) $\neg R(y) \vee \neg L(y)$
 (4) $L(a)$
 S_1 : (5) $R(a)$ (1) 与 (2) 归结
 (6) $\neg I(x) \vee \neg L(x)$ (1) 与 (3) 归结
 (7) $\neg R(a)$ (3) 与 (4) 归结
 S_2 : (8) $\neg I(a)$ (1) 与 (7) 归结
 (9) $\neg L(a)$ (2) 与 (6) 归结
 (10) $\neg L(a)$ (3) 与 (5) 归结
 (11) $\neg I(a)$ (4) 与 (6) 归结
 S_3 : (12) NIL (2) 与 (8) 归结

上述归结过程的归结树如图 3.5 所示。

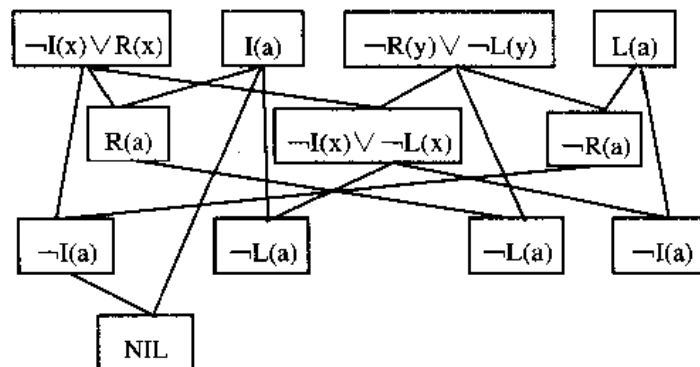


图 3.5 线性输入策略归结树

线性输入策略可限制生成归结式的数量，具有简单、高效的优点，但它是不完备的。也就是说，即使子句集是不可满足的，用线性输入策略进行归结也不一定能归结出空子句。例如，对于子句集

$$S = \{P(x) \vee Q(x), \neg P(y) \vee Q(y), P(u) \vee \neg Q(u), \neg P(t) \vee \neg Q(t)\}$$

可以证明它是不可满足的，但用线性输入策略归结却得不出空子句。

5. 单文字子句策略

如果一个子句只包含一个文字，则称它为单文字子句。单文字子句策略要求参加归结的两个子句中必须有一个是单文字子句。

例 3.14 对例 3.12 给出的初始子句集按单文字子句策略进行归结。

- S : (1) $\neg I(x) \vee R(x)$
 (2) $I(a)$
 (3) $\neg R(y) \vee \neg L(y)$

	(4) $L(a)$	
S_1 :	(5) $R(a)$	(1) 与 (2) 归结
	(6) $\neg R(a)$	(3) 与 (4) 归结
S_2 :	(7) $\neg I(a)$	(1) 与 (6) 归结
	(8) $\neg L(a)$	(3) 与 (5) 归结
S_3 :	NIL	(2) 与 (7) 归结

用单文字子句策略归结时, 归结式将比亲本子句含有较少的文字, 这有利于朝着空子句的方向前进, 因此它有较强的归结效率。但是, 这种归结策略是不完备的。另外, 当初始子句集中不包含单文字子句时, 归结就无法进行。

6. 祖先过滤形策略

祖先过滤形策略与线性输入策略相似, 但放宽了限制。当对两个子句 C_1 和 C_2 进行归结时, 要求它们满足下述两个条件中的任意一个条件:

- 1) C_1 与 C_2 中至少有一个是初始子句集中的子句。
- 2) 如果两个子句都不是初始子句集中的子句, 则一个应是另一个的祖先。所谓一个子句 (例如 C_1) 是另一个子句 (例如 C_2) 的祖先是指 C_2 是由 C_1 与别的子句归结后得到的归结式。

例 3.15 有子句集 $S=\{\neg P(x)\vee Q(x), \neg P(y)\vee \neg Q(y), P(u)\vee Q(u), P(t)\vee \neg Q(t)\}$, 用祖先过滤形策略进行归结时, 归结树如图 3.6 所示。

在此例中, 归结出空子句的两个子句是: $C_1=\neg P(x)$, $C_2=P(u)$, 其中, C_1 是 C_2 的祖先。可以证明, 祖先过滤形策略是完备的。

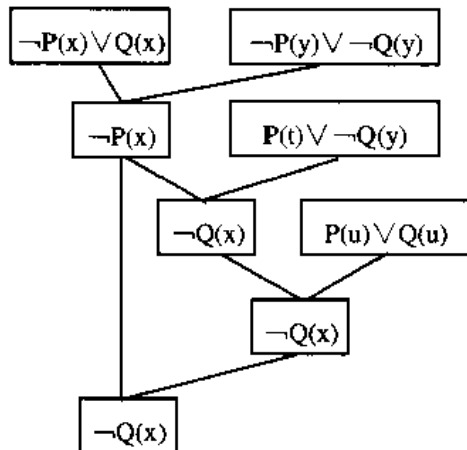


图 3.6 祖先过滤形策略归结树

以上讨论的几种基本的归结策略, 在具体应用时可把几种策略组合在一起使用。归结演绎推理是在自动定理证明领域影响较大的一种推理方法, 它比较简单且又便于在计算机上实现。但由于它要求把逻辑公式转化成子句集, 就可能丢失蕴含式含有的逻辑控制信息。例如下列逻辑公式:

$$\begin{aligned}
 &(\neg A \wedge \neg B) \rightarrow C \\
 &(\neg A \wedge \neg C) \rightarrow B \\
 &(\neg B \wedge \neg C) \rightarrow A \\
 &\neg A \rightarrow (B \vee C) \\
 &\neg B \rightarrow (A \vee C) \\
 &\neg C \rightarrow (A \vee B)
 \end{aligned}$$

它们分别具有不同的逻辑控制信息，但若把它们分别化为子句，则得到的子句却是相同的，即都是： $A \vee B \vee C$

针对归结演绎推理存在的上述问题，人们提出了多种非子句定理证明方法，尼尔逊提出的基于与/或形的演绎推理就是其中的一种。

3.4 与/或形演绎推理

与/或形演绎推理不需要把有关知识化为子句集，而是把领域知识和已知事实分别用蕴含式和与/或形表示，然后运用蕴含式进行演绎推理，从而证明某个目标公式。与/或形演绎推理分为正向演绎与逆向演绎两种基本推理方式。

3.4.1 与/或形正向演绎推理

与/或形正向演绎推理从已知事实出发，正向地使用蕴含式（F 规则）进行演绎推理，直至得到目标公式为止。

1. 事实表达式的与/或形变换及其与/或树表示

与/或形正向演绎推理要求已知事实用不含蕴含符号“ $\rightarrow$ ”的与/或形表示。把表示已知事实的谓词公式化为与/或形的步骤与化子句集类似，只是不必把公式化为了句的合取形式，即不能消去公式中的合取连词。把一个事实表达式变换为与/或形的步骤为：

- 1) 利用 $P \rightarrow Q \Leftrightarrow \neg P \vee Q$ 消去公式中的“ $\rightarrow$ ”。
- 2) 利用德·摩根律及量词转换律把“ $\neg$ ”移到紧靠谓词的位置上。
- 3) 重新命名变元名，使不同量词约束的变元有不同的名字。
- 4) 引入 Skolem 函数消去存在量词。
- 5) 消去全称量词，且使各主要合取式中的变元不同名。

例如对如下事实表达式：

$$(\exists x)(\forall y)\{Q(y, x) \wedge \neg[(R(y) \vee P(y)) \wedge S(x, y)]\}$$

按上述步骤进行转化后得到与/或形表达式：

$$Q(z, a) \wedge \{[\neg R(y) \wedge \neg P(y)] \vee S(a, y)\}$$

事实表达式的与/或形可用一棵与/或树表示出来，上例的与/或树如图 3.7 所示。

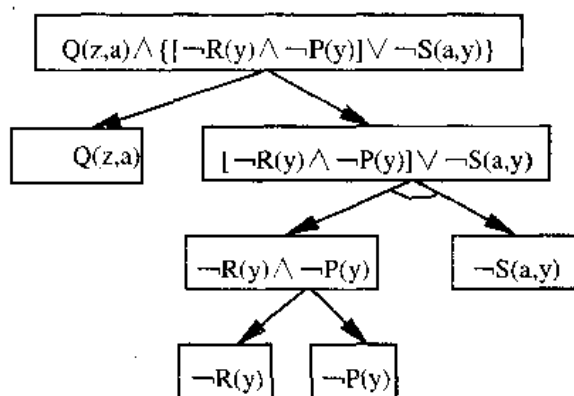


图 3.7 事实表达式的与/或树表示

在图 3.7 中，每个节点表示相应事实表达式的一个子表达式，叶节点为谓词公式中的文字。对于用析取符号“ $\vee$ ”连接而成的表达式，用一个连接符（即图中的半圆弧）把它们连接起来。如果把与/或树中用连接符连接的节点视为具有“与”关系，把不用连接符连接的节点视为具有“或”关系，那么由叶节点所组成的公式：

$$\begin{aligned}
 &Q(z, a) \\
 &\neg R(y) \vee \neg S(a, y) \\
 &\neg P(y) \vee \neg S(a, y)
 \end{aligned}$$

恰好是原表达式化成的子句集。

2. F 规则的表示形式

在与/或形正向演绎推理中，要求 F 规则具有如下形式：

$$L \rightarrow W$$

其中，L 为单文字，W 为与/或形。之所以限制 F 规则的左部为单文字，是因为在进行演绎推理时，要用 F 规则作用于表示事实的与/或树，而该与/或树的叶节点都是单文字，这样就可利用 F 规则的左部与叶节点进行简单匹配（合一）。

如果领域知识的表示形式不是所要求的形式，则需通过变换将它变成规定的形式，变换步骤为：

1) 暂时消去蕴含符号“ $\rightarrow$ ”。例如对公式

$$(\forall x)\{[(\exists y)(\forall z)P(x, y, z)] \rightarrow (\forall u)Q(x, u)\}$$

通过运用等价关系 $P \rightarrow Q \Leftrightarrow \neg P \vee Q$ 可变为：

$$(\forall x)\{\neg[(\exists y)(\forall z)P(x, y, z)] \vee (\forall u)Q(x, u)\}$$

2) 把“ $\neg$ ”移到紧靠谓词的位置上。通过运用德·摩根律及量词转换律可把“ $\neg$ ”移到括弧中，经移动“ $\neg$ ”，上式变为：

$$(\forall x)\{(\forall y)(\exists z)[\neg P(x, y, z)] \vee (\forall u)Q(x, u)\}$$

3) 引入 Skolem 函数消去存在量词。消去存在量词后，上式变为：

$$(\forall x)\{(\forall y)[\neg P(x, y, f(x, y))]\vee(\forall u)Q(x, u)\}$$

4) 消去全称量词。消去全称量词后, 上式变为:

$$\neg P(x, y, f(x, y))\vee Q(x, u)$$

5) 恢复为蕴含式。利用等价关系 $\neg P\vee Q\leftrightarrow P\rightarrow Q$ 将上式变为:

$$P(x, y, f(x, y))\rightarrow Q(x, u)$$

3. 目标公式的表示形式

在与/或形正向演绎推理中, 要求目标公式用子句表示, 否则就需要化成子句形式。

4. 推理过程

应用 F 规则进行推理的目的在于证明某个目标公式。如果从已知事实的与/或树出发, 通过运用 F 规则最终推出了欲证明的目标公式, 则推理就可成功结束。其推理过程为:

- 1) 首先用与/或树把已知事实表示出来。
- 2) 用 F 规则的左部和与/或树的叶节点进行匹配, 将匹配成功的 F 规则加入到与/或树中。
- 3) 重复进行步骤 2), 直到产生一个含有以目标节点作为终止节点的解图为止。

例 3.16 设已知事实为

$$A\vee B$$

F 规则为:

$$r_1: A\rightarrow C\wedge D$$

$$r_2: B\rightarrow E\wedge G$$

欲证明的目标公式为

$$C\vee G$$

其证明过程如图 3.8 所示。其中, 为把这里所说的与/或树与一般意义下的与/或树区别开来, 将它按倒置的形式存放, 双箭头表示匹配。

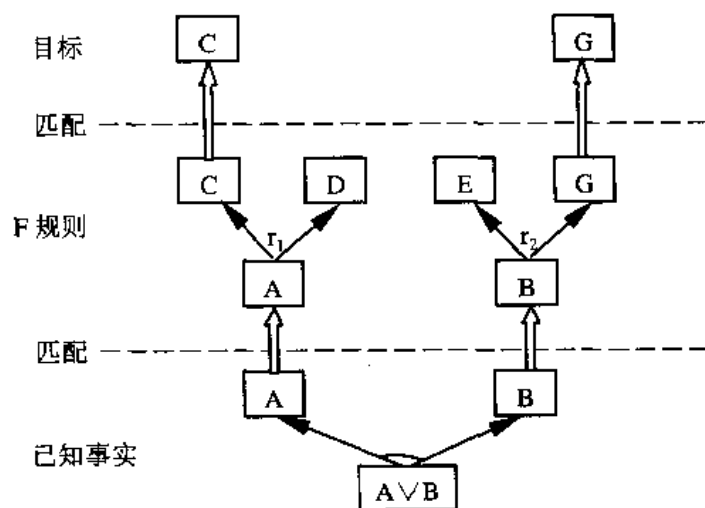


图 3.8 例 3.16 推理过程

对于用谓词公式表示已知事实及 F 规则的情形,推理中需要用最一般合一进行变元的代换,下面用例子说明。

例 3.17 设已知事实为

$$\neg P(a) \vee \{Q(a) \wedge R(a)\}$$

F 规则为:

$$r_1: \quad \neg P(x) \rightarrow \neg S(x)$$

$$r_2: \quad Q(y) \rightarrow N(y)$$

欲证明的目标公式为:

$$\neg S(z) \vee N(z)$$

推理过程如图 3.9 所示。

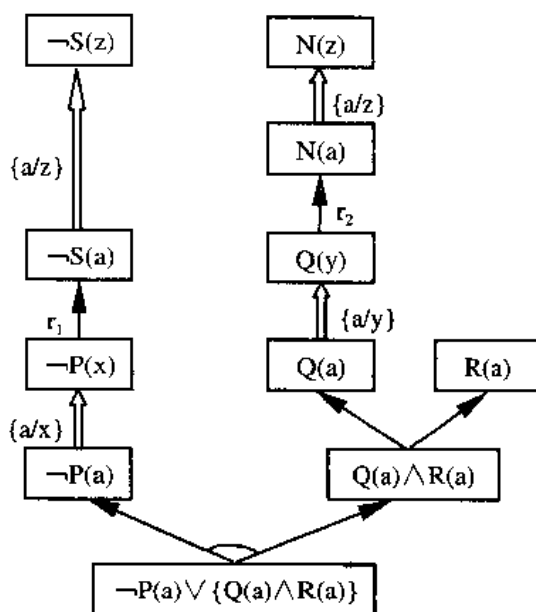


图 3.9 例 3.17 的推理过程

3.4.2 与/或形逆向演绎推理

与/或形逆向演绎推理是从待证明的问题(目标)出发,逆向使用蕴含式(B 规则)进行演绎推理,直至得到包含已知事实的终止条件为止。

1. 目标公式的与/或形变换及其与/或树表示

在与/或形逆向演绎推理中,要求目标公式用与/或形表示,其变换过程与正向演绎推理中对已知事实的变换相似,只是要用存在量词约束的变元的 Skolem 函数替换由全称量词约束的相应变元,并且消去全称量词,然后再消去存在量词,这是与正向演绎推理中对已知事实进行变换的不同之处。例如对如下目标公式:

$$(\exists y)(\forall x)\{P(x) \rightarrow [Q(x,y) \vee \neg(R(x) \wedge S(y))]\}$$

经变换后得到

$$\neg P(f(z)) \vee \{Q(f(y),y) \wedge [\neg R(f(y)) \vee \neg S(y)]\}$$

变换时应注意使各个主要的析取式具有不同的变元名。

目标公式的与/或形可用与/或树表示出来，但其表示方式与正向演绎推理中对已知事实的与/或树表示略有不同，它的连接符用来把具有合取关系的子表达式连接起来，而在正向演绎推理中是把已知事实中具有析取关系的子表达式连接起来。上例的与/或树如图 3.10 所示。

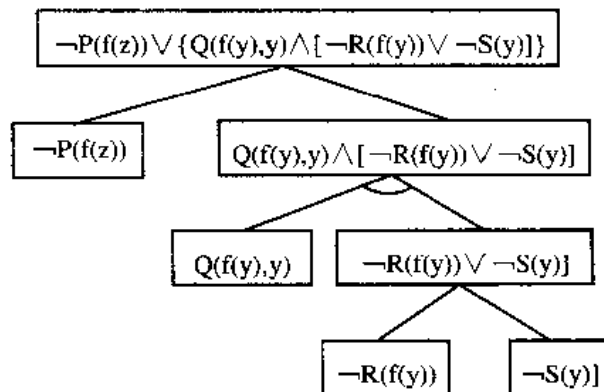


图 3.10 目标公式的与/或树表示

在图 3.10 中，若把叶节点用它们之间的合取及析取关系连接起来，就可得到原目标公式的三个子目标：

$$\begin{aligned} &\neg P(f(z)) \\ &Q(f(y),y) \wedge \neg R(f(y)) \\ &Q(f(y),y) \wedge \neg S(y) \end{aligned}$$

可见子目标是文字的合取式。

2. B 规则的表示形式

B 规则的表示形式为

$$W \rightarrow L$$

其中，W 为任一与/或形公式，L 为文字。之所以限制规则的右部为文字，是因为推理时要用它与目标与/或树中的叶节点进行匹配（合一），而目标与/或树中的叶节点是文字。

如果已知的 B 规则不是所要求的形式，可用与转化 F 规则类似的方法把它化成规定的形式，特别是对于像

$$W \rightarrow (L_1 \wedge L_2)$$

这样的蕴含式可化为两个 B 规则：

$$W \rightarrow L_1, W \rightarrow L_2$$

3. 已知事实的表示形式

在逆向演绎推理中, 要求已知事实是文字的合取式, 即形如

$$F_1 \wedge F_2 \wedge \cdots \wedge F_n$$

在问题求解中, 由于每个 $F_i (i=1, 2, \cdots, n)$ 都可单独起作用, 因此可把上面公式表示为事实的集合:

$$\{F_1, F_2, \cdots, F_n\}$$

4. 推理过程

应用 B 规则进行逆向演绎推理的目的是求解问题, 当从目标公式的与/或树出发, 通过运用 B 规则最终得到了某个终止在事实节点上的一致解图时, 推理就可成功结束。其推理过程为:

- 1) 先用与/或树把目标公式表示出来。
- 2) 用 B 规则的右部和与/或树的叶节点进行匹配, 将匹配成功的 B 规则加入到与/或树中。
- 3) 重复进行步骤 2), 直到产生某个终止在事实节点上的一致解图为止。这里所说的“一致解图”是指在推理过程中所用到的代换应该是一致的。

例 3.18 设有如下事实及规则:

事实:

$f_1: \text{DOG}(\text{Fido})$ Fido 是一只狗

$f_2: \neg \text{BARKS}(\text{Fido})$ Fido 不叫

$f_3: \text{WAGS-TAIL}(\text{Fido})$ Fido 摇尾巴

$f_4: \text{MEOWS}(\text{Myrtle})$ Myrtle 喵喵叫

规则:

$r_1: (\text{WAGS-TAIL}(x_1) \wedge \text{DOG}(x_1)) \rightarrow \text{FRIENDLY}(x_1)$ 狗摇尾巴表示友好

$r_2: (\text{FRIENDLY}(x_2) \wedge \neg \text{BARKS}(x_2)) \rightarrow \neg \text{AFRAID}(y_2, x_2)$ 友好且不叫的狗不可怕

$r_3: \text{DOG}(x_3) \rightarrow \text{ANIMAL}(x_3)$ 狗是动物

$r_4: \text{CAT}(x_4) \rightarrow \text{ANIMAL}(x_4)$ 猫是动物

$r_5: \text{MEOWS}(x_5) \rightarrow \text{CAT}(x_5)$ 喵喵叫的是猫

求解的问题是: 是否有这样的一只猫和一条狗, 而且这只猫不怕这条狗?

该问题的目标公式为

$$(\exists x)(\exists y)[\text{CAT}(x) \wedge \text{DOG}(y) \wedge \neg \text{AFRAID}(x, y)]$$

求解该问题的过程如图 3.11 所示。

该推理过程得到的解图是一致解图。图中有 8 条匹配弧, 每条弧上都有一个代换, 而且这些代换是一致的。终止在事实节点上的代换为 $\{\text{Myrtle}/x\}$ 和 $\{\text{Fido}/y\}$, 把它们应用到目标公式, 就得到该问题的解:

$$\text{CAT}(\text{Myrtle}) \wedge \text{DOG}(\text{Fido}) \wedge \neg \text{AFRAID}(\text{Myrtle}, \text{Fido})$$

它表示: 一只名叫 Myrtle 猫和一条名叫 Fido 的狗, 而且这只猫不怕那条狗。

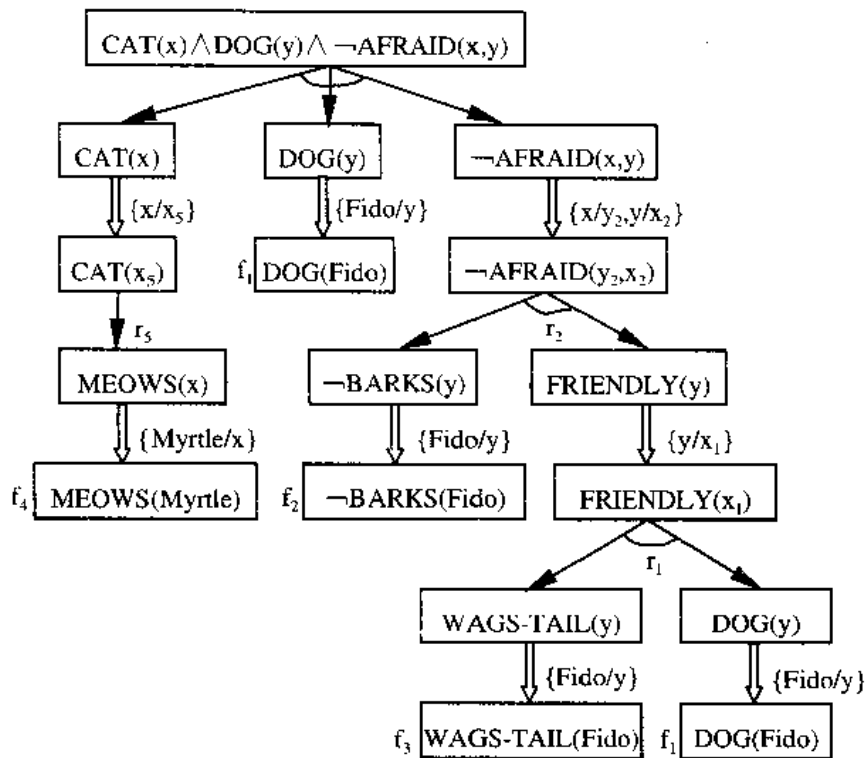


图 3.11 例 3.18 的推理过程

3.4.3 代换的一致性与剪枝策略

1. 代换的一致性

无论对与/或形的正向演绎推理还是逆向演绎推理，都要求推理过程中所用的代换集合具有一致性，下面给出代换集合一致性的定义。

定义 3.13 设代换集合

$$\theta = \{\theta_1, \theta_2, \dots, \theta_n\}$$

中第 i 个代换 $\theta_i (i=1, 2, \dots, n)$ 为

$$\theta_i = \{t_{i1}/x_{i1}, t_{i2}/x_{i2}, \dots, t_{im(i)}/x_{im(i)}\}$$

其中， t_{ij} 为项， x_{ij} 为变元 ($j=1, 2, \dots, m(i)$)，则代换集 θ 是一致的充要条件是如下两个元组：

$$T = \{t_{11}, t_{12}, \dots, t_{1m(1)}, t_{21}, \dots, t_{2m(2)}, \dots, t_{nm(n)}\}$$

$$X = \{x_{11}, x_{12}, \dots, x_{1m(1)}, x_{21}, \dots, x_{2m(2)}, \dots, x_{nm(n)}\}$$

可合一。

例如：

设 $\theta_1 = \{x/y\}$, $\theta_2 = \{y/z\}$, 则 $\theta = \{\theta_1, \theta_2\}$ 是一致的。

设 $\theta_1 = \{f(g(x_1))/x_3, f(x_2)/x_4\}$, $\theta_2 = \{x_4/x_3, g(x_1)/x_2\}$, 则 $\theta = \{\theta_1, \theta_2\}$ 是一致的。

设 $\theta_1 = \{a/x\}$, $\theta_2 = \{b/x\}$, 则 $\theta = \{\theta_1, \theta_2\}$ 是不一致的。

设 $\theta_1 = \{g(y)/x\}$, $\theta_2 = \{f(x)/y\}$, 则 $\theta = \{\theta_1, \theta_2\}$ 是不一致的。

2. 剪枝策略

在与/或形演绎推理中, 要不断地用 F 规则的左部 (正向演绎推理) 或 B 规则的右部 (逆向演绎推理) 和与/或树的叶节点进行匹配 (合一), 在每一步上可匹配的规则都可能不止一条, 为了得到一致解图, 应选哪一条规则? 这里以逆向演绎推理为例讨论一种方法, 即剪枝策略。

剪枝策略的基本思想是: 每当选用一条规则时, 就进行一次一致性检查, 如果当前的部分解图是一致的, 则继续向下扩展, 否则就放弃该规则而选用其他候选规则。由于应用该方法在使用每一条规则时都进行了一致性检查, 所以最终得到的解图必然是一致的。另外, 由于这种方法可尽量地发现并剪除不一致的分枝, 从而避免了大的返工。

例如, 设当前的与/或树如图 3.12 (a) 所示, 可用的 B 规则有:

$$r_1: \quad S(z) \rightarrow P(b)$$

$$r_2: \quad R(y) \rightarrow P(y)$$

若首先选用 r_1 , 则得到代换集合为:

$$\{a/x, b/x\}$$

显然它是不一致的, 因而必须放弃这一选择。接着选用 r_2 , 得到的代换集合为:

$$\{a/x, x/y\}$$

这是一致的, 所以可把 r_2 用于推理, 并把它加入到目标与/或树中, 如图 3.12 (b) 所示。

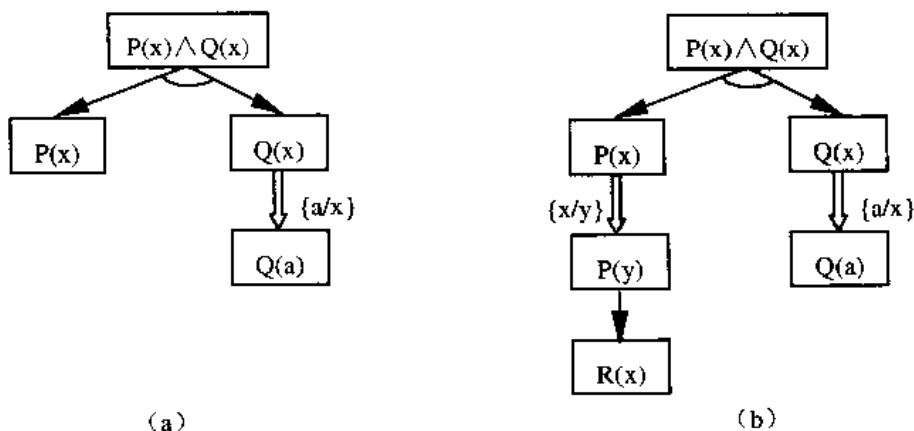


图 3.12 剪枝策略

习题三

3.1 名词解释

演绎推理

归纳推理

确定性推理

不确定性推理

单调推理

非单调推理

基于知识的推理

常识性推理

3.2 何谓正向推理？请根据正向推理的基本过程画出正向推理的流程示意图。

3.3 何谓逆向推理？请根据逆向推理的基本过程画出逆向推理的流程示意图。

3.4 何谓模式匹配？什么是确定性匹配？什么是不确定性匹配？

3.5 何谓冲突消解？在产生式系统中，什么情况下会发生冲突？常用的冲突消解策略有哪些？请分别简要说明。

3.6 公式集的合一是什么含义？何谓最一般合一？

3.7 何谓自然演绎推理？自然演绎推理的基本推理规则有哪些？请分别说明这些基本推理规则的一般形式。

3.8 何谓归结式？为什么说归结式是其亲本子句的逻辑结论？

3.9 归结原理的完备性的含义是什么？有哪些归结策略是完备的？

3.10 下述公式集 F 是否可合一，若可合一，则求出 F 的最一般合一。

$$(1) F = \{P(a, b), P(x, y)\}$$

$$(2) F = \{P(f(x), b), P(y, z)\}$$

$$(3) F = \{P(f(x), y), P(y, f(b))\}$$

$$(4) F = \{P(f(y), y, x), P(x, f(a), f(b))\}$$

3.11 把下列谓词公式分别化为相应的子句集：

$$(1) (\forall x)(\forall y)(P(x, y) \wedge Q(x, y))$$

$$(2) (\forall x)(\forall y)(P(x, y) \rightarrow Q(x, y))$$

$$(3) (\forall x)(\exists y)(P(x, y) \vee (Q(x, y) \rightarrow R(x, y)))$$

$$(4) (\forall x)(\forall y)(\exists z)(P(x, y) \rightarrow Q(x, y) \vee R(x, z))$$

$$(5) (\exists x)(\exists y)(\forall z)(\exists u)(\forall v)(\exists w)(P(x, y, z, u, v, w) \wedge (Q(x, y, z, u, v, w) \vee \neg R(x, z, w)))$$

3.12 判断下列子句集中哪些是不可满足的：

$$(1) S = \{\neg P \vee Q, \neg Q, P, \neg P\}$$

$$(2) S = \{P \vee Q, \neg P \vee Q, P \vee \neg Q, \neg P \vee \neg Q\}$$

$$(3) S = \{P(y) \vee Q(y), \neg P(f(x)) \vee R(a)\}$$

$$(4) S = \{\neg P(x) \vee Q(x), \neg P(y) \vee R(y), P(a), S(a), \neg S(z) \vee R(z)\}$$

$$(5) S = \{\neg P(x) \vee \neg Q(y) \vee \neg L(x, y), P(a), \neg R(z) \vee L(a, z), R(b), Q(b)\}$$

$$(6) S = \{\neg P(x) \vee Q(f(x), a), \neg P(h(y)) \vee Q(f(h(y)), a) \vee \neg P(z)\}$$

$$(7) S = \{P(x) \vee Q(x) \vee R(x), \neg P(y) \vee R(y), \neg Q(a), \neg R(b)\}$$

$$(8) S = \{P(x) \vee Q(x), \neg Q(y) \vee R(y), \neg P(z) \vee Q(z), \neg R(u)\}$$

3.13 对下列各题分别证明 G 是否为前提公式集 F 的逻辑结论：

$$(1) F: (\exists x)(\exists y) P(x, y)$$

$$G: (\forall y)(\exists x) P(x, y)$$

- (2) $F_1: (\forall x)(P(x) \wedge (Q(a) \vee Q(b)))$
 $G: (\exists x)(P(x) \wedge Q(x))$
- (3) $F_1: (\exists x)(\exists y)(P(f(x)) \wedge Q(f(b)))$
 $G: P(f(a)) \wedge P(y) \wedge Q(y)$
- (4) $F_1: (\forall x)(P(x) \rightarrow (\forall y)(Q(y) \rightarrow \neg L(x, y)))$
 $F_2: (\exists x)(P(x) \wedge (\forall y)(R(y) \rightarrow L(x, y)))$
 $G: (\forall x)(R(x) \rightarrow \neg Q(x))$
- (5) $F_1: (\forall x)(P(x) \rightarrow (Q(x) \wedge R(x)))$
 $F_2: (\exists x)(P(x) \wedge S(x))$
 $G: (\exists x)(S(x) \wedge R(x))$
- (6) $F_1: (\forall x)(A(x) \wedge \neg B(x) \rightarrow (\exists y)(D(x, y) \wedge C(y)))$
 $F_2: (\exists x)(E(x) \wedge A(x) \wedge (\forall y)(D(x, y) \rightarrow E(y)))$
 $F_3: (\forall x)(E(x) \rightarrow \neg B(x))$
 $G: (\exists x)(E(x) \wedge C(x))$

3.14 某公司招聘工作人员, A, B, C 三人应试。经面试后, 公司表示如下意见:

- (1) 三人中至少录用一人。
 (2) 如果录用 A 而不录用 B, 则一定录用 C。
 (3) 如果录用 B, 则一定录用 C。

求证: 公司一定录用 C。

3.15 已知前提: 每个储蓄钱的人都获得利息。求证结论: 如果没有利息, 那么就没有人去储蓄钱。

3.16 已知前提:

- (1) 某些病人喜欢所有的医生。
 (2) 没有一个病人喜欢任何一个骗子。

求证结论: 任何一个医生都不是骗子。

3.17 应用归结反演方法证明理发师悖论: 若每个理发师都为不能给自己理发的人理发, 且每个理发师都不为能给自己理发的人理发, 那么不存在任何理发师。

3.18 设已知

- (1) 如果 x 是 y 的父亲, y 是 z 的父亲, 则 x 是 z 的祖父。
 (2) 每个人都有一个父亲。

试用归结演绎推理证明: 对于某人 u, 一定存在一个人 v, 且 v 是 u 的祖父。

3.19 张某家被盗, 公安局派出五个侦察员去调查。研究案情时, 侦察员 A 说“赵与钱中至少有一人作案”; 侦察员 B 说“钱与孙中至少有一人作案”; 侦察员 C 说“孙与李中至少有一人作案”; 侦察员 D 说“赵与孙中至少有一人与此案无关”; 侦察员 E 说“钱与李中至少有一人与此案无关”。如果这五个侦察员的话都是可信的, 试用归结演绎推理求出谁是

盗窃犯。

3.20 已知下述事实：

- (1) 小李只喜欢较容易的课程。
- (2) 工程类课程是较难的。
- (3) PR 系的所有课程都是较容易的。
- (4) PR150 是 PR 系的一门课程。

应用归结演绎推理回答问题：小李喜欢什么课程？

3.21 设已知：

- (1) 能阅读的人是识字的。
- (2) 海豚不识字。
- (3) 有些海豚是很聪明的。

分别用线性输入策略、祖先过滤形策略证明：有些很聪明的人并不识字。

3.22 已知下述事实：

- (1) 小杨、小刘和小林是高山俱乐部成员。
- (2) 高山俱乐部的每个成员是滑雪者或登山者，或者既滑雪又登山。
- (3) 没有一个登山者喜欢下雨。
- (4) 所有滑雪者都喜欢下雪。
- (5) 凡是小杨喜欢的，小刘就不喜欢。
- (6) 凡是小杨不喜欢的，小刘就喜欢。
- (7) 小杨喜欢下雨和下雪。

试证明：俱乐部是否有是登山者而不是滑雪者的成员？如果有，他是谁？

3.23 设已知事实为

$$((P \vee Q) \wedge R) \vee (S \wedge (T \vee U))$$

F 规则为

$$S \rightarrow (X \wedge Y) \vee Z$$

用正向演绎推理推出所有可能的目标子句。

3.24 已知下述事实和规则：

事实：Fido 又叫 (BARKS) 又咬 (BITS)，否则 Fido 就不是狗 (GOD)。

规则：所有的小猎犬 (TERRIES) 都是狗。

任何一种叫声都是噪声 (NOISY)。

应用与/或形正向演绎推理证明：存在某些不是小猎犬而是噪声的东西。

第4章 搜索策略

根据问题的实际情况不断寻找可利用的知识,从而构造一条代价较少的推理路线,使问题得到圆满解决的过程称为搜索。搜索分为盲目搜索和启发式搜索。盲目搜索是按预定的搜索方向进行搜索。由于盲目搜索总是按预先规定的路线进行,没有考虑到问题本身的特性,所以这种搜索效率不高。启发式搜索是在搜索中加入了与问题有关的启发性信息,用以指导搜索朝着最有希望的推理方向前进,加速问题的求解过程并找到最优解。显然,启发式搜索优于盲目搜索。但由于启发式搜索需要具有与问题本身特性有关的信息,并非对每一类问题都可方便地抽取出来,因此盲目搜索仍不失为一种应用较多的搜索策略。

4.1 问题求解过程的形式表示

人工智能虽有多个应用领域,而且每个应用领域又各有自己的规律和特点,但从它们求解具体问题的过程来看,都可抽象为一个“问题求解”的过程。问题求解过程实际上是一个搜索过程。为了进行搜索,首先必须考虑问题及其求解过程的形式表示,其表示是否适当,将直接影响到搜索求解的效率。

4.1.1 状态空间表示法

状态空间表示法是表示问题及其搜索过程的一种形式表示方法。状态空间表示法用“状态”和“算符”来表示问题,其中,“状态”用以描述问题求解过程不同时刻的状态;“算符”表示对状态的操作,算符的每一次使用就使问题由一种状态变换为另一种状态。当到达目标状态时,由初始状态到目标状态所用算符的序列就是问题的一个解。

1. 状态

状态是描述问题求解过程中任一时刻状况的数据结构,可用一组变量的有序集表示:

$$S_k=(S_{k1}, S_{k2}, \dots, S_{kn})$$

当给每一个分量以确定的值时,就得到了一个具体的状态。

2. 算符

引起状态中某些分量发生变化,从而使问题由一个状态变为另一个状态的操作称为算符。在产生式系统中,每一条产生式规则就是一个算符。

3. 状态空间

由问题的全部状态及一切可用算符所构成的集合称为问题的状态空间,一般用一个三元组表示:

$$(S, F, G)$$

其中 S 是问题的所有初始状态构成的集合； F 是算符的集合； G 是目标状态的集合。

状态空间的图示形式称为状态空间图。其中，节点表示状态；有向边（弧）表示算符。

例 4.1 二阶梵塔问题。设有三根柱子，在 1 号柱子上穿有 A、B 两个盘片，盘 A 小于盘 B，盘 A 位于盘 B 的上面。要求把这两个盘片全部移到另一根柱子上，而且规定每次只能移动一片，任何时刻都不能使盘 B 位于盘 A 的上面。

设用 $S_K = (S_{K0}, S_{K1})$ 表示问题的状态， S_{K0} 表示盘 A 所在的柱号， S_{K1} 表示盘 B 所在的柱号。全部可能的状态有以下 9 种：

$$\begin{array}{lll} S_0 = (1,1) & S_1 = (1,2) & S_2 = (1,3) \\ S_3 = (2,1) & S_4 = (2,2) & S_5 = (2,3) \\ S_6 = (3,1) & S_7 = (3,2) & S_8 = (3,3) \end{array}$$

问题的初始状态集合为 $S = \{S_0\}$ ，目标状态集合为 $G = \{S_4, S_8\}$ 。算符分别用 $A(i,j)$ 及 $B(i,j)$ 表示。 $A(i,j)$ 表示把盘 A 从柱 i 号移到 j 上； $B(i,j)$ 表示把盘 B 从柱 i 移到柱 j 上。共有 12 个算符，它们分别是：

$$\begin{array}{l} A(1,2), A(1,3), A(2,1), A(2,3), A(3,1), A(3,2) \\ B(1,2), B(1,3), B(2,1), B(2,3), B(3,1), B(3,2) \end{array}$$

根据 9 种可能的状态和 12 种算符，可构成二阶梵塔问题的状态空间图，如图 4.1 所示。

在图 4.1 所示的状态空间中，从初始节点(1,1)到目标节点(2,2)或者(3,3)的任何一条通路都是问题的一个解，其中最短的路径长度是 3，它由 3 个算符组成，例如 $A(1,3)$, $B(1,2)$, $A(3,2)$ 。

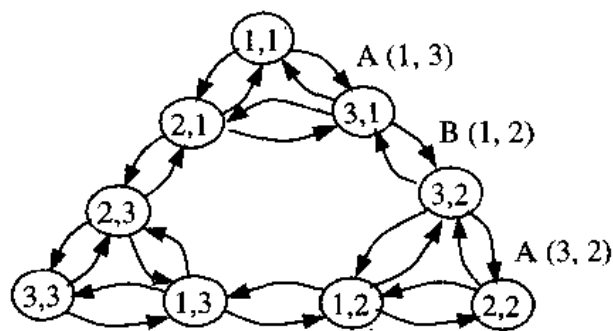


图 4.1 二阶梵塔的状态空间

由此例可以看出：

1) 用状态空间方法表示问题时，首先必须定义状态的描述形式，通过使用这种描述形式可把问题的一切状态都表示出来。其次，还要定义一组算符，通过使用算符可把问题由一种状态转变为另一种状态。

2) 问题的求解过程是一个不断把算符作用于状态的过程。如果在使用某个算符后得到

的新状态是目标状态，就得到了问题的一个解。这个解是从初始状态到目标状态所用算符构成的序列。

3) 算符的一次使用，就使问题由一种状态转变为另一种状态。可能有多个算符序列都可使问题从初始状态变到目标状态，这就得到了多个解。其中有的使用算符较少，有的较多，我们把使用算符最少的解称为最优解。例如在上例中，使用 3 个算符的解是最优解。这只是从解中使用的算符个数来评价解的优劣，更一般地说是使用算符时所付出的代价，只有总代价最小的解才是最优解。

4) 对任何一个状态，可使用的算符可能不止一个，因而由一个状态所生成的后继状态就可能多个。当对这些后继状态使用算符时，首先应对哪一个后继状态进行操作，取决于问题求解采用的搜索策略。搜索策略将影响问题搜索求解过程的效率。

4.1.2 与/或树表示法

与/或树是用于表示问题及其求解过程的又一种形式化方法，也称为问题归约方法。它把初始问题通过一系列变换最终变为一个子问题集合，而这些子问题的解可以直接得到，从而解答了初始问题。

1. 分解

把一个复杂问题分解为若干个较为简单的子问题，每个子问题又可继续分解为若干个更为简单的子问题。重复此过程，直到不需要再分解或者不能再分解为止。然后对每个子问题分别进行求解，最后把各子问题的解复合起来就得到了原问题的解。

问题的分解过程可用一个图表示出来。例如，把问题 P 分解为三个子问题 P_1 、 P_2 、 P_3 ，可用图 4.2 表示。 P_1 、 P_2 、 P_3 是问题 P 的三个子问题，只有当这三个子问题都可解时，问题 P 才可解，称 P_1 、 P_2 、 P_3 之间存在“与”关系；称节点 P 为“与”节点；由 P 、 P_1 、 P_2 、 P_3 所构成的图称为“与”树。在图中，为了标明某个节点是“与”节点，通常用一条弧把各条边连接起来。

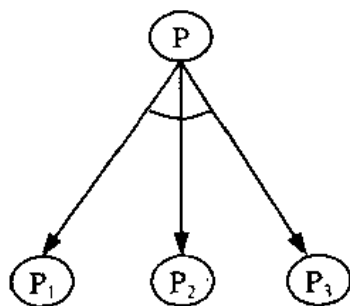


图 4.2 “与”树

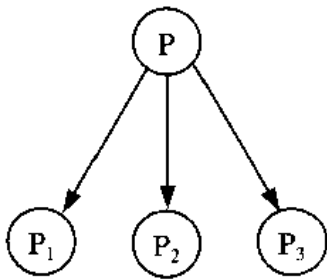


图 4.3 “或”树

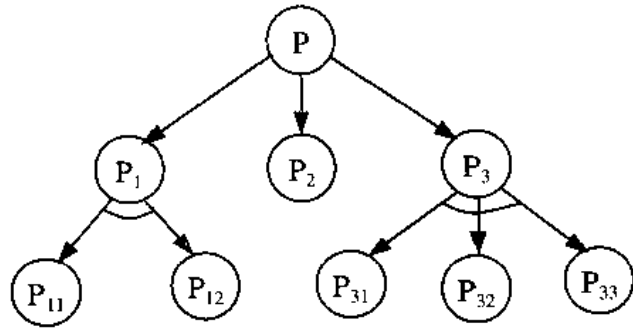


图 4.4 “与/或”树

2. 等价变换

对于一个复杂问题，除了可用“分解”方法进行求解外，还可利用同构或同态的等价变换，把它变换成若干个较容易求解的新问题。若新问题中有一个可求解，则就得到了原问题的解。

问题的等价变换过程也可用一个图表示出来，称为“或”树。例如，问题 P 被等价变换为新问题 P_1 、 P_2 、 P_3 ，可用图 4.3 表示。其中，新问题 P_1 、 P_2 、 P_3 中只要有一个可解，则原问题就可解。我们称 P_1 、 P_2 、 P_3 之间存在“或”关系；节点 P 称为“或”节点；由 P 、 P_1 、 P_2 、 P_3 所构成的图是一个“或”树。

上述两种方法也可结合起来使用，此时的图称为“与/或”树。其中既有“与”节点，也有“或”节点，如图 4.4 所示。

3. 本原问题

不能再分解或变换，而且直接可解的子问题称为本原问题。

4. 端节点与终止节点

在与/或树中，没有子节点的节点称为端节点；本原问题所对应的节点称为终止节点。显然，终止节点一定是端节点，但端节点不一定是终止节点。

5. 可解节点

在与/或树中，满足下列条件之一者，称为可解节点：

- 1) 它是一个终止节点。
- 2) 它是一个“或”节点，且其子节点至少有一个是可解节点。
- 3) 它是一个“与”节点，且其子节点全部是可解节点。

6. 不可解节点

关于可解节点三个条件全部不满足的节点称为不可解节点。

7. 解树

由可解节点所构成的，并且由这些可解节点可推出初始节点（它对应于原始问题）为可解节点的子树称为解树。在解树中一定包含初始节点。

例 4.2 三阶梵塔问题。设有 A、B、C 三个盘片以及三根柱子，三个盘片按从小到大的顺序穿在 1 号柱上，要求把它们全部移到 3 号柱上，而且每次只能移动一个盘片，任何时刻都不能把大的盘片压在小的盘片上面，如图 4.5 所示。

为了求解该问题，可对问题的求解进行下述分析：

1) 为了把三个盘片全部移到 3 号柱上，必须先把盘片 C 移到 3 号柱上。

2) 为了移盘片 C，必须先把盘片 A 及 B 移到 2 号柱上。

3) 当把盘片 C 移到 3 号盘上后，就可把 A、B 从 2 号柱移到 3 号柱上，这样就可完成问题的求解。

由以上分析，可把原问题分解为三个子问题：

1) 把盘片 A 及 B 移到 2 号柱的双盘片问题。

2) 把盘片 C 移到 3 号柱的单盘片问题。

3) 把盘片 A 及 B 移到 3 号柱的双盘片问题。

其中，子问题 1) 与子问题 3) 又分别可分解为三个子问题。



图 4.5 三阶梵塔问题

为了用与/或树把问题的分解过程表示出来，需要先定义问题的状态表示。设用三元组：

(i, j, k)

表示状态，其中 i 表示盘片 C 所在的柱号； j 表示盘片 B 所在的柱号； k 表示盘片 A 所在的柱号。这样初始问题就可表示为：

$(1, 1, 1) \Rightarrow (3, 3, 3)$

可用与/或树把分解过程表示出来，如图 4.6 所示。

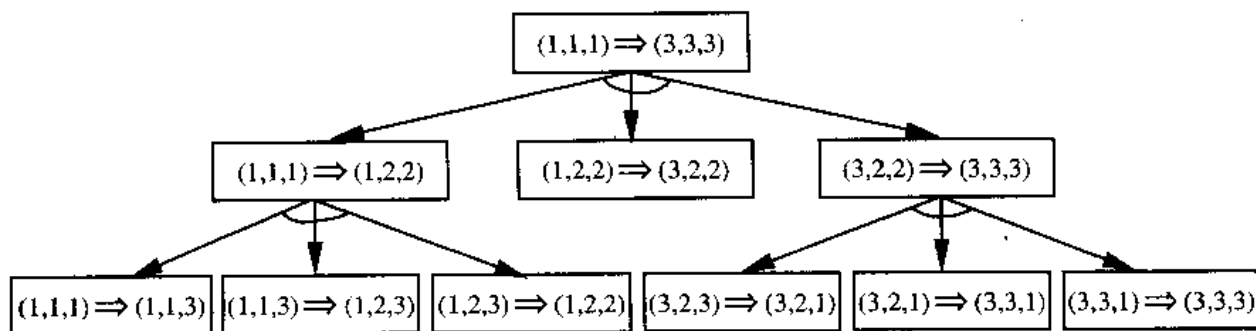


图 4.6 三阶梵塔问题的与/或树

在与/或树中，共有 7 个终止节点，对应于 7 个本原问题，它们是通过“分解”得到的。把这些本原问题的解按从左至右排列，就得到了初始问题的解：

$$\begin{aligned}(1,1,1) &\Rightarrow (1,1,3), (1,1,3) \Rightarrow (1,2,3), (1,2,3) \Rightarrow (1,2,2) \\ (1,2,2) &\Rightarrow (3,2,2), (3,2,2) \Rightarrow (3,2,1), (3,2,1) \Rightarrow (3,3,1) \\ (3,3,1) &\Rightarrow (3,3,3)\end{aligned}$$

4.2 状态空间的盲目搜索策略

状态空间的搜索策略分为盲目搜索和启发式搜索两大类。下面讨论的宽度优先搜索、深度优先搜索、有界深度优先搜索、代价树的宽度优先搜索以及代价树的深度优先搜索都属于盲目搜索策略。其特点是：

- 1) 搜索按规定的路线进行，不使用与问题有关的启发性信息。
- 2) 适用于其状态空间图是树状结构的问题的求解。

一个复杂问题的状态空间一般都是十分庞大的。例如 64 阶梵塔问题（盘片的数目称为梵塔问题的阶）共有 $3^{64} \approx 0.94 \times 10^{30}$ 个不同的状态。若把它们都存储到计算机中去，需占用巨大的存储空间，这是难以实现的。另一方面，把问题的全部状态空间都存储到计算机中也是不必要的，因为对一个确定的具体问题来说，与解题有关的状态空间往往只是整个状态空间的一部分，因此只要能生成并存储这部分状态空间就可求得问题的解。但是，对一个具体问题，如何生成它所需要的部分状态空间从而实现对问题的求解呢？在人工智能中是通过运用搜索技术来解决这一问题的。其基本思想是：首先把问题的初始状态（即初始节点）作为当前状态，选择适用的算符对其进行操作，生成一组子状态（或称后继状态、后继节点、子节点），然后检查目标状态是否在其中出现。若出现，则搜索成功，找到了问题的解；若不出现，则按某种搜索策略从已生成的状态中再选一个状态作为当前状态。重复上述过程，直到目标状态出现或者不再有可供操作的状态及算符时为止。

在搜索过程中，一般都要用两个表，这就是 OPEN 表与 CLOSED 表。OPEN 表用于存放刚生成的节点，对于不同的搜索策略，节点在 OPEN 表中的排列顺序是不同的。例如对宽度优先搜索，节点按生成的顺序排列，先生成的节点排在前面，后生成的节点排在后面。CLOSED 表用于存放将要扩展或者已扩展的节点。所谓对一个节点进行“扩展”是指：用合适的算符对该节点进行操作，生成一组子节点。

4.2.1 宽度优先搜索

宽度优先搜索又称为广度优先搜索。宽度优先搜索的基本思想是：从初始节点 S_0 开始，逐层地对节点进行扩展并考察它是否为目标节点，在第 n 层的节点没有全部扩展并考察之前，不对第 $n+1$ 层的节点进行扩展。OPEN 表中的节点总是按进入的先后顺序排列，先进入的节点排在前面，后进入的节点排在后面。其搜索过程如下：

- 1) 把初始节点 S_0 放入 OPEN 表。

2) 如果 OPEN 表为空, 则问题无解, 退出。

3) 把 OPEN 表的第一个节点 (记为节点 n) 取出放入 CLOSED 表。

4) 考察节点 n 是否为目标节点。若是, 则求得了问题的解, 退出。

5) 若节点 n 不可扩展, 则转第 2) 步。

6) 扩展节点 n , 将其子节点放入 OPEN 表的尾部, 并为每一个子节点都配置指向父节点的指针, 然后转第 2) 步。

该搜索过程可用图 4.7 表示其工作流程。

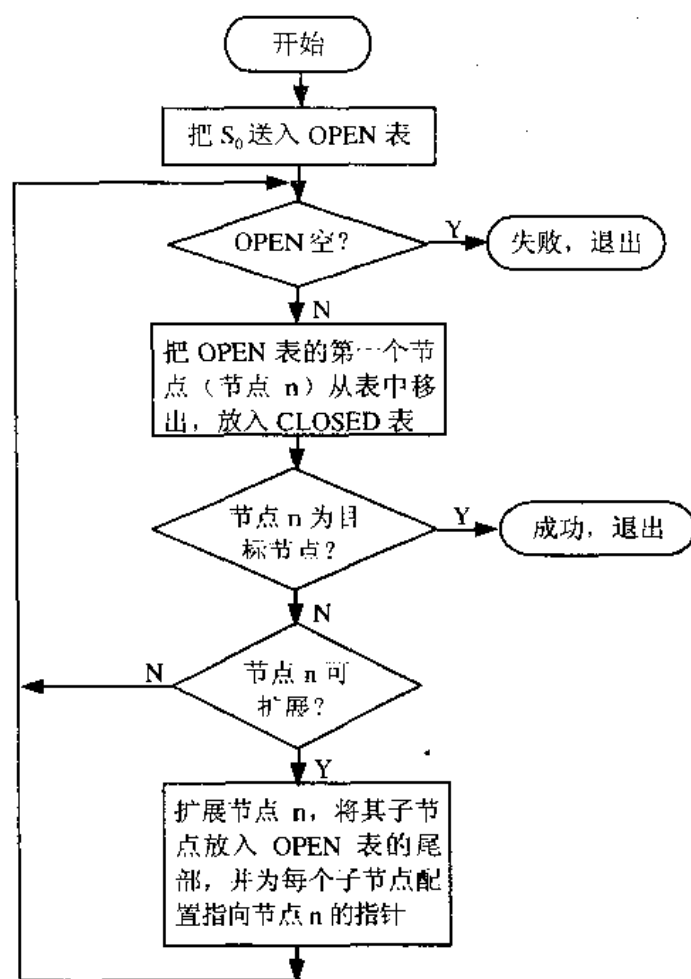


图 4.7 宽度优先搜索流程示意图

例 4.3 重排九宫问题, 在 3×3 的方格棋盘上放置分别标有数字 1、2、3、4、5、6、7、8 共 8 个棋子, 初始状态为 S_0 , 目标状态为 S_g , 如图 4.8 所示。

可使用的算符有:

空格左移, 空格上移, 空格右移, 空格下移

即只允许把位于空格左、上、右、下的邻近棋子移入空格。要求寻找从初始状态到目标状态的路径。



图 4.8 重排九宫问题

应用宽度优先搜索，可得到如图 4.9 所示的搜索树。

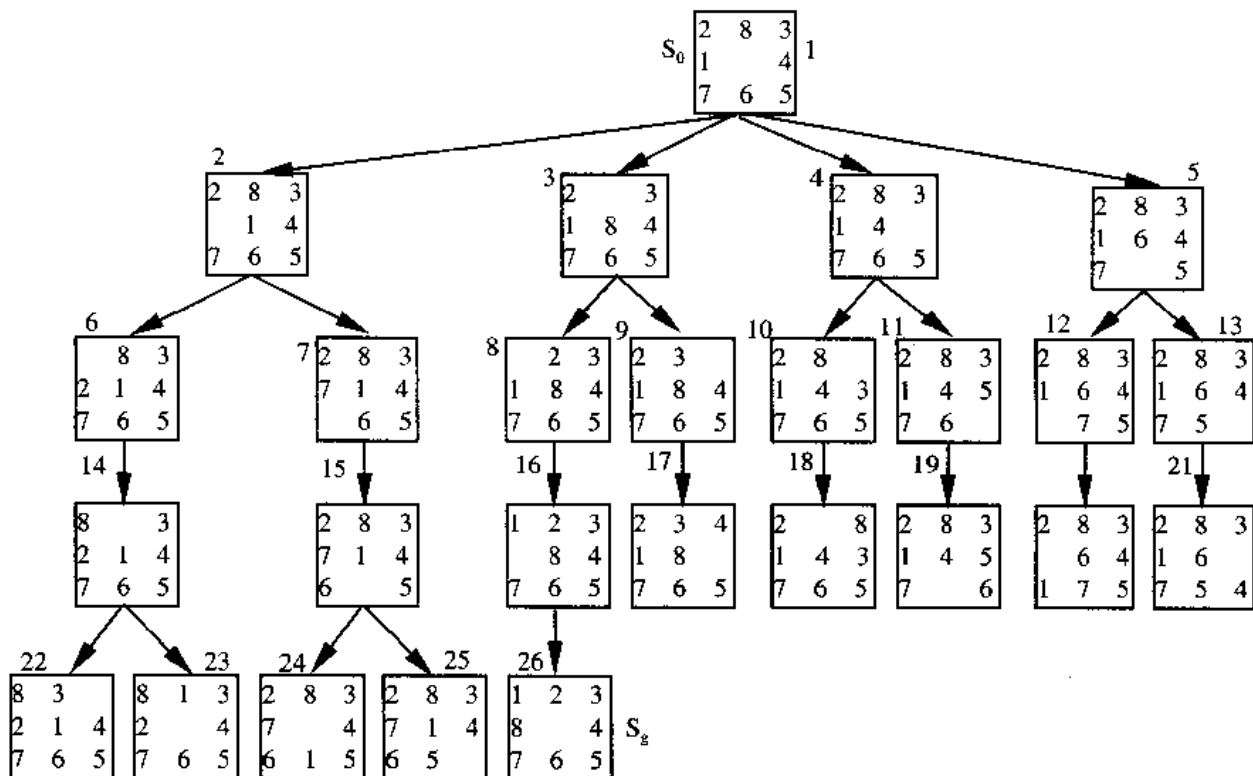


图 4.9 重排九宫的宽度优先搜索

由图 4.9 可以看出，解的路径是

$$S_0 \rightarrow 3 \rightarrow 8 \rightarrow 16 \rightarrow 26$$

解是该路径使用的算符序列，即：空格上移，空格左移，空格下移，空格右移。

宽度优先搜索的盲目性较大，当目标节点距离初始节点较远时将会产生许多无用节点，因此搜索效率低。但是，只要问题有解，用宽度优先搜索总可以得到解，而且得到的是路径最短的解。

4.2.2 深度优先搜索

深度优先搜索的基本思想是：从初始节点 S_0 开始，在其子节点中选择一个节点进行考察，若不是目标节点，则再在该子节点的子节点中选择一个节点进行考察，一直如此向下搜索。

当到达某个子节点，若该子节点既不是目标节点又不能继续扩展，则选择其兄弟节点进行考察。其搜索过程如下：

- 1) 把初始节点 S_0 放入 OPEN 表。
- 2) 如果 OPEN 表为空，则问题无解，退出。
- 3) 把 OPEN 表的第一个节点（记为节点 n ）取出放入 CLOSED 表。
- 4) 考察节点 n 是否为目标节点。若是，则求得了问题的解，退出。
- 5) 若节点 n 不可扩展，则转第 2) 步。

6) 扩展节点 n ，将其子节点放入到 OPEN 表的首部，并为其配置指向父节点的指针，然后转第 2) 步。

该过程与宽度优先搜索的惟一区别是：宽度优先搜索是将节点 n 的子节点放入到 OPEN 表的尾部，而深度优先搜索是把节点 n 的子节点放入到 OPEN 表的首部。仅此区别就使得搜索的路线完全不一样。

例 4.4 对图 4.8 所示的重排九宫问题进行深度优先搜索，可得到图 4.10 所示的搜索树。这只是搜索树的一部分，尚未到达目标节点，仍需继续往下搜索。

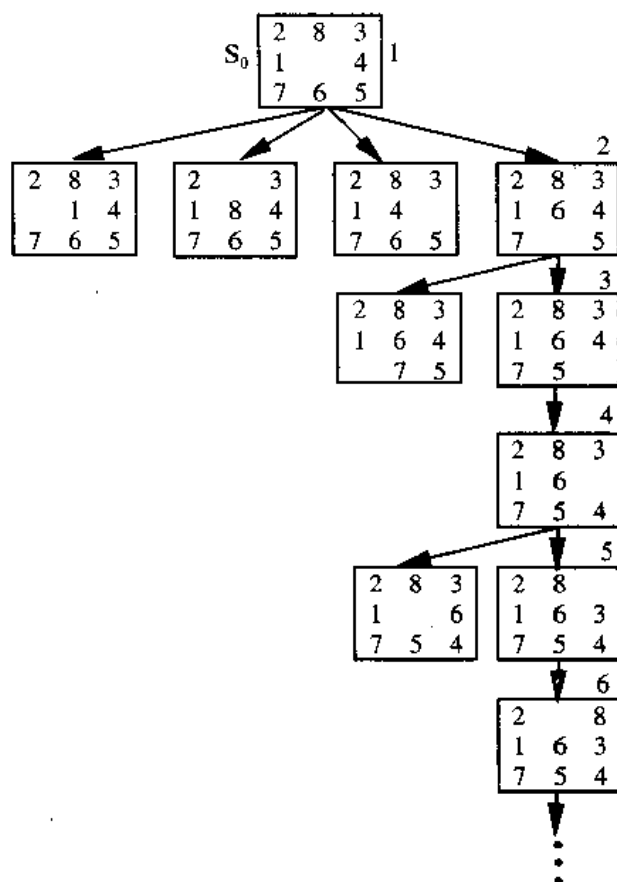


图 4.10 重排九宫的深度优先搜索

在深度优先搜索中，搜索一旦进入某个分支，就将沿着该分支一直向下搜索。如果目标

节点恰好在此分支上,则可较快地得到解。但是,如果目标节点不在此分支上,而该分支又是一个无穷分支,则就不可能得到解。所以深度优先搜索是不完备的,即使问题有解,它也不一定求得解。显然,用深度优先搜索求得的解,也不一定是路径最短的解。

4.2.3 有界深度优先搜索

为了解决深度优先搜索不完备的问题,避免搜索过程陷入无穷分支的死循环,提出了有界深度优先搜索方法。有界深度优先搜索的基本思想是:对深度优先搜索设定搜索深度的界限(设为 d_m),当搜索深度达到了深度界限,而尚未出现目标节点时,就换一个分支进行搜索。

有界深度优先搜索的搜索过程为:

1) 把初始节点 S_0 放入 OPEN 表中,置 S_0 的深度 $d(S_0)=0$ 。

2) 如果 OPEN 表为空,则问题无解,退出。

3) 把 OPEN 表中的第一个节点(记为节点 n)取出放入 CLOSED 表。

4) 考察节点 n 是否为目标节点。若是,则求得了问题的解,退出。

5) 如果节点 n 的深度 $d(\text{节点 } n)=d_m$,则转第 2) 步。

6) 若节点 n 不可扩展,则转第 2) 步。

7) 扩展节点 n ,将其子节点放入 OPEN 表的首部,并为其配置指向父节点的指针。然后转第 2) 步。

如果问题有解,且其路径长度 $\leq d_m$,则上述搜索过程一定能求得解。但是,若解的路径长度 $> d_m$,则上述搜索过程就得不到解。这说明在有界深度优先搜索中,深度界限的选择是很重要的。当 d_m 太大时,搜索时将产生许多无用的子节点,既浪费了计算机的存储空间与时间,又降低了搜索效率。

由于解的路径长度事先难以预料,所以要恰当地给出 d_m 的值是比较困难的。另外,即使能求出解,它也不一定是最优解。为此,可采用下述办法进行改进:先任意给定一个较小的数作为 d_m ,然后进行上述的有界深度优先搜索,当搜索达到了指定的深度界限 d_m 仍未发现目标节点,并且 CLOSED 表中仍有待扩展节点时,就将这些节点送回 OPEN 表,同时增大深度界限 d_m ,继续向下搜索。如此不断地增大 d_m ,只要问题有解,就一定可以找到它。但此时找到的解不一定是最优解。为找到最优解,可增设一个表 R,每找到一个目标节点 S_g 后,就把它放入到 R 表的前面,并令 d_m 等于该目标节点所对应的路径长度,然后继续搜索。由于后求得的解的路径长度不会超过先求得的解的路径长度,所以最后求得的解一定是最优解。

求最优解的有界深度优先搜索过程如图 4.11 所示。其中 S_g^* 是距离 S_0 最近的目标节点。

例 4.5 设深度界限 $d_m=4$,用有界深度优先搜索方法求解图 4.8 所示的重排九宫问题。由有界深度优先搜索方法求解得出的搜索树如图 4.12 所示,解的路径是

$$S_0 \rightarrow 20 \rightarrow 25 \rightarrow 26 \rightarrow 28(S_g)$$

解是该路径使用的算符序列,即:空格上移,空格左移,空格下移,空格右移。

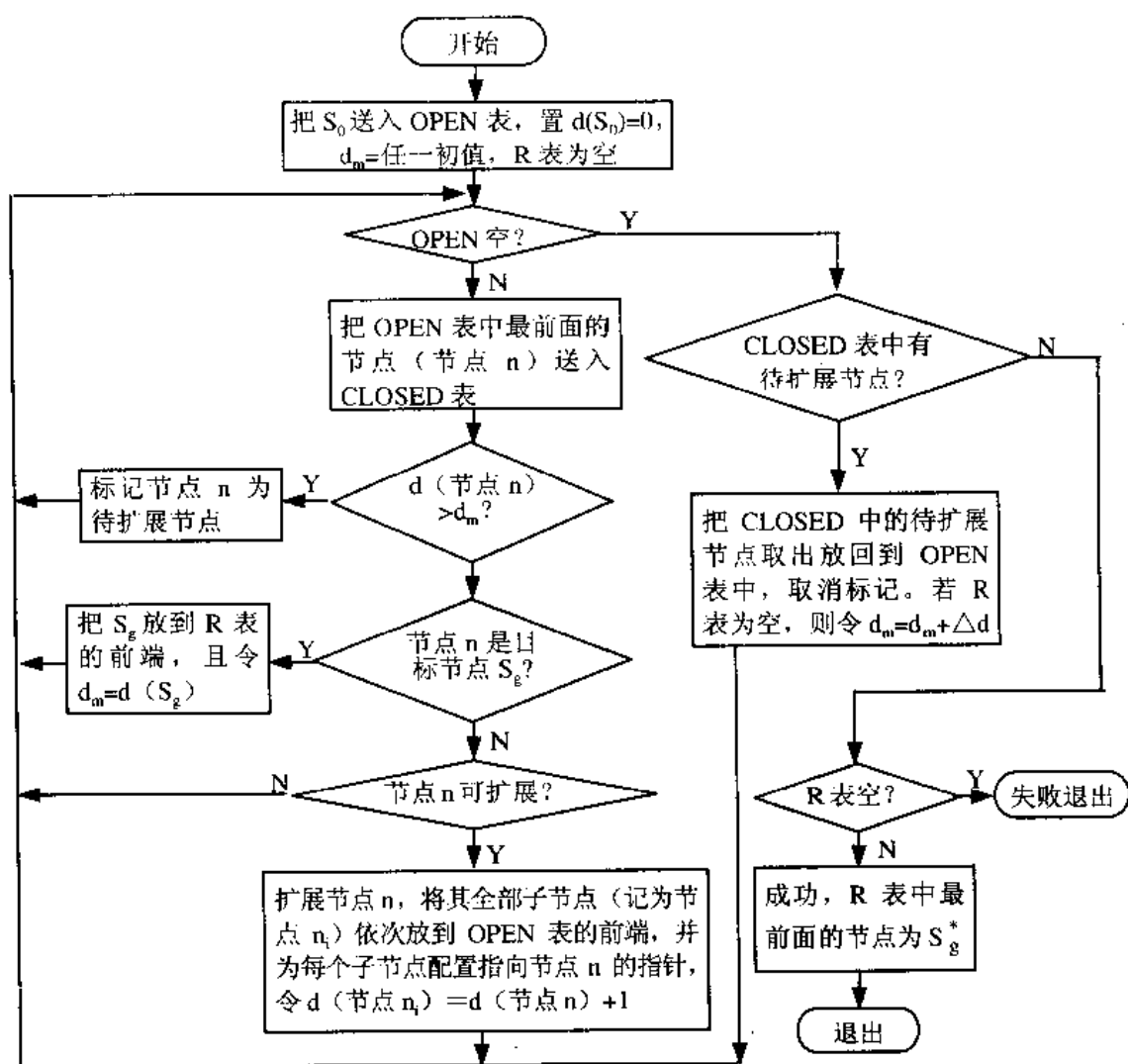


图 4.11 求最优解的有界深度优先搜索流程示意图

4.2.4 代价树的宽度优先搜索

在上面的讨论中, 都没有考虑搜索的代价问题, 可以认为图中各边的代价都相同, 且都为一个单位量, 因此只是用路径长度来代表路径的代价。事实上, 图中各边的代价是可能完全不一样的。

边上标有代价 (或费用) 的树称为代价树。在代价树中, 若用 $g(x)$ 表示从初始节点 S_0 到节点 x 的代价, 用 $c(x_1, x_2)$ 表示从父节点 x_1 到子节点 x_2 的代价, 则有:

$$g(x_2) = g(x_1) + c(x_1, x_2)$$

代价树宽度优先搜索的基本思想是: OPEN 表中的节点在任一时刻都是按其代价从小到大排序的, 代价小的节点排在前面, 代价大的节点排在后面, 每次扩展时总是从 OPEN 表中选取代价最小的节点进行扩展。其搜索过程如下:

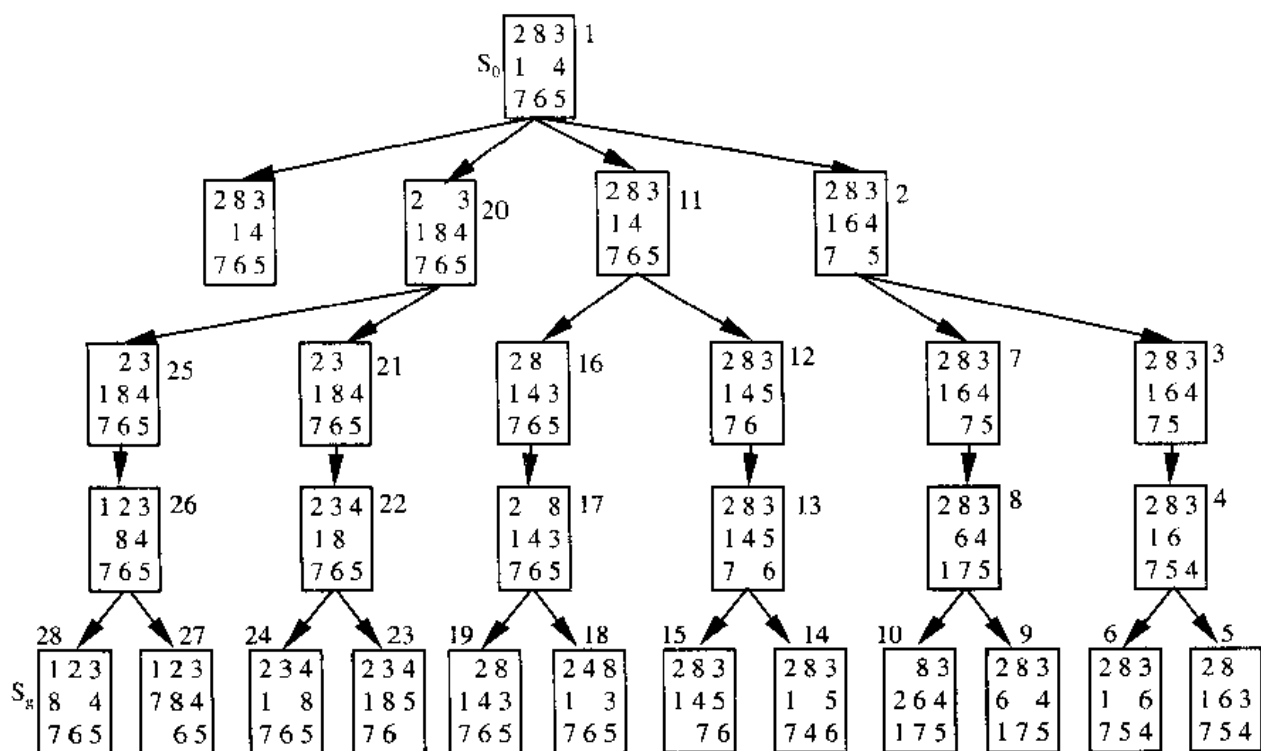


图 4.12 重排九宫的有界深度优先搜索

- 1) 把初始节点 S_0 放入 OPEN 表, 令 $g(S_0)=0$ 。
- 2) 如果 OPEN 表为空, 则问题无解, 退出。
- 3) 把 OPEN 表的第一个节点 (记为节点 n) 取出放入 CLOSED 表。
- 4) 考察节点 n 是否为目标节点。若是, 则求得了问题的解, 退出。
- 5) 若节点 n 不可扩展, 则转第 2) 步。

6) 扩展节点 n , 将其子节点放入 OPEN 表中, 且为其配置指向父节点的指针; 计算各子节点的代价, 并按各节点的代价对 OPEN 表中的全部节点进行排序 (按从小到大的顺序), 然后转第 2) 步。

该搜索过程可用图 4.13 表示其工作流程。

如果问题有解, 该搜索过程一定可以求得解, 并且求出的是最优解。

例 4.6 图 4.14 是五城市间的交通路线图, A 城市是出发地, E 城市是目的地, 两城市间的交通费用 (代价) 如图中数字所示。求从 A 到 E 的最小费用交通路线。

为了应用代价树的宽度优先搜索方法求解此问题, 需先将交通图转换为代价树, 如图 4.15 所示。转换的方法是: 从起始节点 A 开始, 把与它直接相邻的节点作为它的子节点。对其他节点也做相同的处理。但若一个节点已作为某节点的直系先辈节点时, 就不能再作为这个节点的子节点。例如, 与节点 C 相邻的节点有 A 与 D, 但因 A 已作为 C 的父节点在代价树中出现了, 所以它不能再作为 C 的子节点。另外, 图中的节点除起始节点 A 外, 其他节点都可能要在代价树中出现多次, 为区分它的多次出现, 分别用下标 1, 2, ... 标出, 其实它们都是图中的同一节点。例如 E_1, E_2, E_3, E_4 都是图中的节点 E。

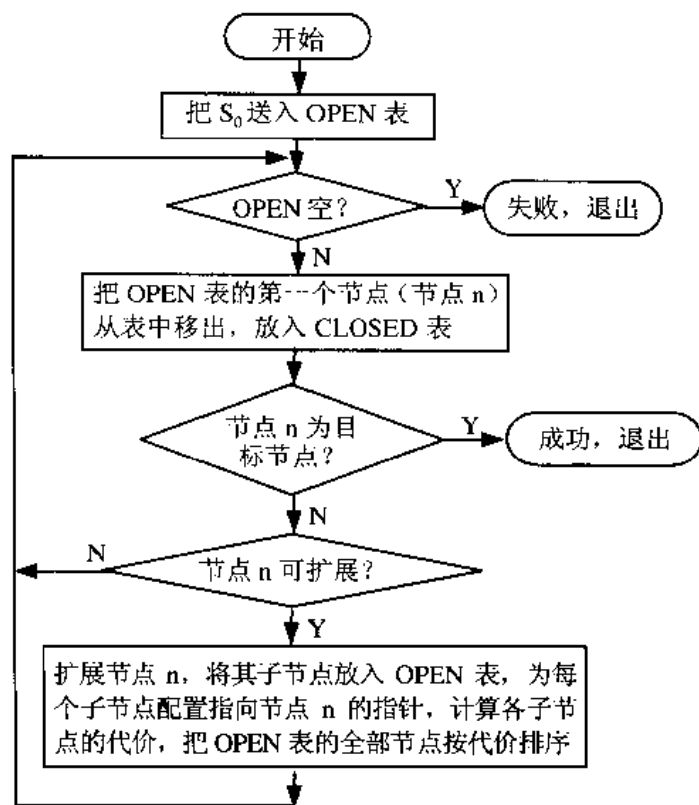


图 4.13 代价树宽度优先搜索流程示意图

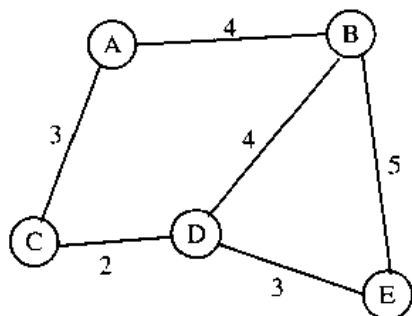


图 4.14 交通图

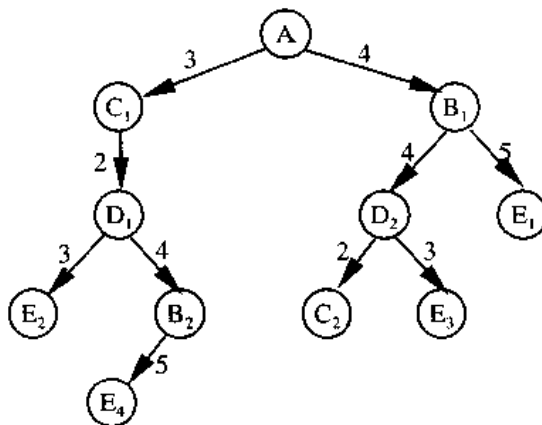


图 4.15 交通图的代价树

对此代价树进行代价树的宽度优先搜索, 可得到最优解路径为

$$A \rightarrow C_1 \rightarrow D_1 \rightarrow E_2$$

代价为 8。由此可知从 A 城市到 E 城市的最小费用路线为

$$A \rightarrow C \rightarrow D \rightarrow E$$

4.2.5 代价树的深度优先搜索

在代价树的宽度优先搜索中, 每次都是从 OPEN 表的全体节点中选择一个代价最小的节

点送入 CLOSED 表进行考察，而代价树的深度优先搜索是从刚扩展出的子节点中选一个代价最小的节点送入 CLOSED 表进行考察。例如，在图 4.15 所示的代价树中，首先对 A 进行扩展，得到 C_1 及 B_1 ，由于 C_1 的代价小于 B_1 的代价，所以首先把 C_1 送入 CLOSED 表进行考察。此时代价树的宽度优先搜索与代价树的深度优先搜索是一致的。但往下继续进行，两者就不一样了。对 C_1 进行扩展得到 D_1 ， D_1 的代价为 5。此时 OPEN 表中有 B_1 与 D_1 ， B_1 的代价为 4。若按代价树的宽度优先搜索方法进行，应选 B_1 送入 CLOSED 表，但按代价树的深度优先搜索方法，则应选 D_1 送入 CLOSED 表。 D_1 扩展后，再选 E_2 ，到达了目标节点。所以，按代价树的深度优先搜索方法，得到的解路径是

$$A \rightarrow C_1 \rightarrow D_1 \rightarrow E_2$$

代价为 8。这与例 4.5 得到的结果相同，但这只是巧合，一般情况下这两种方法得到的结果不一定相同。另外，由于代价树的深度优先搜索有可能进入无穷分支路径，因此它是不完备的。

代价树深度优先搜索的过程为：

- 1) 把初始节点 S_0 放入 OPEN 表中。
- 2) 如果 OPEN 表为空，则问题无解，退出。
- 3) 把 OPEN 表的第一个节点（记为节点 n ）取出放入 CLOSED 表。
- 4) 考察节点 n 是否为目标节点。若是，则求得了问题的解，退出。
- 5) 若节点 n 不可扩展，则转第 2) 步。
- 6) 扩展节点 n ，将其子节点按边代价从小到大的顺序放到 OPEN 表的首部，并为各子节点配置指向父节点的指针，然后转第 2) 步。

该搜索过程可用图 4.16 表示其工作流程。

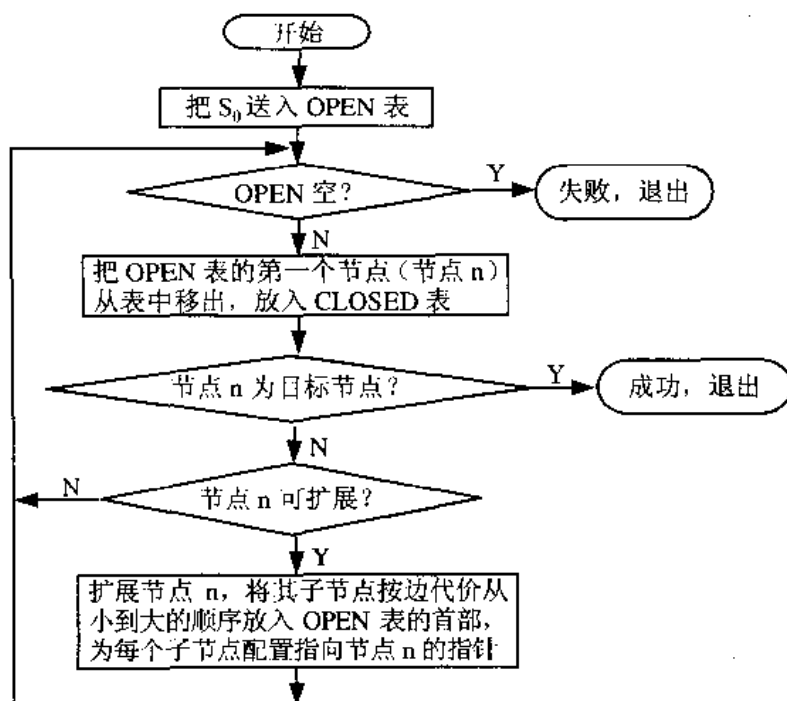


图 4.16 代价树深度优先搜索流程示意图

4.3 状态空间的启发式探索策略

盲目搜索策略或者是按事先规定的路线进行搜索，或者是按已经付出的代价决定下一步要搜索的节点。例如宽度优先搜索是按“层”进行搜索的，先进入 OPEN 表的节点先被考察；深度优先搜索是沿着纵深方向进行搜索的，后进入 OPEN 表的节点先被考察；代价树的宽度优先搜索是根据 OPEN 表中全体节点已付出的代价（即从初始节点到该节点路径上的代价）来决定哪一个节点先被考察；而代价树的深度优先搜索是在当前节点的子节点中选代价最小的节点作为下一个被考察的节点。它们的一个共同特点是都没有利用问题本身的特性信息，在决定被扩展的节点时，都没有考察该节点在解的路径上的可能性有多大，它是否有利于问题求解以及求出的解是否为最优解等。因此这些搜索方法都具有较大的盲目性，产生的无用节点较多，搜索空间较大，效率不高。

启发式搜索要用到问题自身的某些特性信息，以指导搜索朝着最有希望的方向前进。由于这种搜索针对性较强，因而效率较高。

4.3.1 估价函数与择优搜索

在搜索过程中，关键的一步是如何确定下一个要考察的节点，确定的方法不同就形成了不同的搜索策略。如果在确定节点时能充分利用与问题求解有关的特性信息，估计出节点的重要性，就能在搜索时选择重要性较高的节点，以利于求得最优解。像这样可用于指导搜索过程，且与具体问题求解有关的控制性信息称为启发性信息。

用于估价节点重要性的函数称为估价函数。其一般形式为：

$$f(x)=g(x)+h(x)$$

其中 $g(x)$ 为从初始节点 S_0 到节点 x 已经实际付出的代价； $h(x)$ 是从节点 x 到目标节点 S_g 的最优路径的估计代价，它体现了问题的启发性信息，其形式要根据问题的特性确定。例如，它可以是节点 x 到目标节点的距离，也可以是节点 x 处于最优路径上的概率等等。 $h(x)$ 称为启发函数。

1. 局部择优搜索

局部择优搜索是一种启发式搜索方法，是对深度优先搜索方法的一种改进。其基本思想是：当一个节点被扩展以后，按 $f(x)$ 对每一个子节点计算估价值，并选择最小者作为下一个要考察的节点。由于它每次都只是在子节点的范围内选择下一个要考察的节点，范围比较狭窄，所以称为局部择优搜索。局部择优搜索的搜索过程为：

- 1) 初始节点 S_0 放入 OPEN 表，计算 $f(S_0)$ 。
- 2) 如果 OPEN 表为空，则问题无解，退出。
- 3) 把 OPEN 表的第一个节点（记为节点 n ）取出放入 CLOSED 表。
- 4) 考察节点 n 是否为目标节点。若是，则求得了问题的解，退出。

5) 若节点 n 不可扩展, 则转第 2) 步。

6) 扩展节点 n , 用估价函数 $f(x)$ 计算每个子节点的估价值, 并按估价值从小到大的顺序依次放到 OPEN 表的首部, 为每个子节点配置指向父节点的指针。然后转第 2) 步。

深度优先搜索、代价树的深度优先搜索以及局部择优搜索都是以子节点作为考察范围的, 这是它们的共同处。不同的是它们选择节点的标准不一样: 深度优先搜索以子节点的深度作为选择标准, 后生成的子节点先被考察; 代价树深度搜索以各子节点到父节点的代价作为选择标准, 代价小的子节点先被考察; 局部择优搜索以估价函数 f 的值作为选择标准, 哪一个子节点的 f 值最小就优先被选择。实际上, 在局部择优搜索中, 若令 $f(x)=g(x)$, 则局部择优搜索就成为代价树的深度优先搜索; 若令 $f(x)=d(x)$, 这里 $d(x)$ 表示节点 x 的深度, 则局部择优搜索就成为深度优先搜索。所以深度优先搜索和代价树的深度优先搜索可看作局部择优搜索的两个特例。

2. 全局择优搜索

每当要选择一个节点进行考察时, 局部择优搜索只是从刚生成的子节点中进行选择, 选择的范围比较狭窄, 因而又提出了全局择优搜索方法。按这种方法搜索时, 每次总是从 OPEN 表的全体节点中选择一个估价值最小的节点。其搜索过程如下:

1) 把初始节点 S_0 放入 OPEN 表, 计算 $f(S_0)$ 。

2) 如果 OPEN 表为空, 则搜索失败, 退出。

3) 把 OPEN 表中的第一个节点 (记为节点 n) 从表中移出放入 CLOSED 表。

4) 考察节点 n 是否为目标节点。若是, 则求得了问题的解, 退出。

5) 若节点 n 不可扩展, 则转第 2) 步。

6) 扩展节点 n , 用估价函数 $f(x)$ 计算每个子节点的估价值, 并为每个子节点配置指向父节点的指针, 把这些子节点都送入 OPEN 表中, 然后对 OPEN 表中的全部节点按估价值从小至大的顺序进行排序。

7) 转第 2) 步。

比较全局择优搜索与局部择优搜索的搜索过程可以看出, 它们的区别仅在于第 6) 步。

在全局择优搜索中, 如果 $f(x)=g(x)$, 则它就成为代价树的宽度优先搜索; 如果 $f(x)=d(x)$, 则它就成为宽度优先搜索。所以宽度优先搜索与代价树的宽度优先搜索是全局择优搜索的两个特例。

例 4.7 用全局择优搜索求解重排九宫问题, 其初始状态和目标状态仍如图 4.8 所示。

设估价函数为

$$f(x)=d(x)+h(x)$$

其中, $d(x)$ 表示节点 x 的深度, $h(x)$ 表示节点 x 的棋局与目标节点棋局位置不相同的棋子数。搜索树如图 4.17 所示, 图中节点旁标明的数字是该节点的估价函数值。

该问题的解路径为:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_g$$

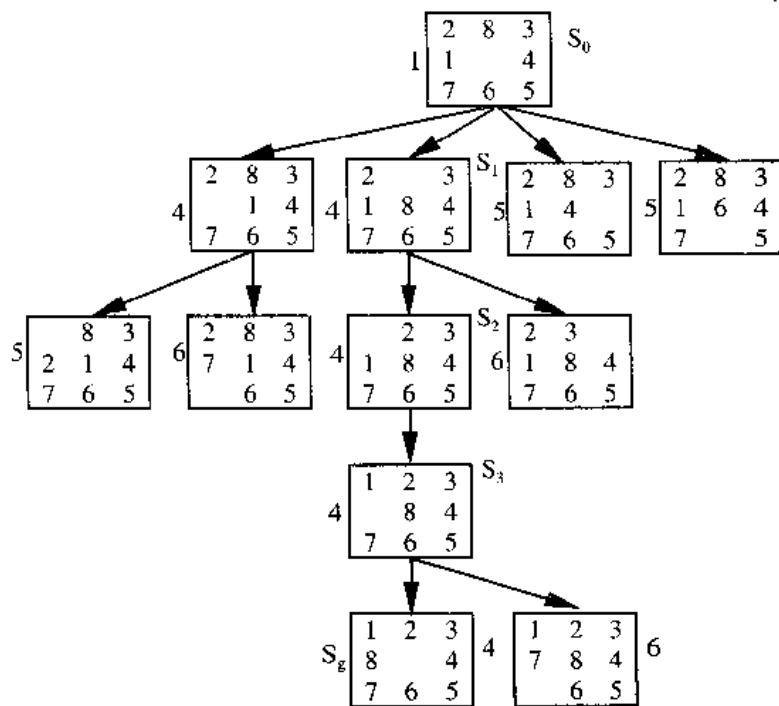


图 4.17 重排九宫问题的全局择优搜索树

由此解路径得到问题的解为：空格上移，空格左移，空格下移，空格右移。

在启发式搜索中，估价函数的定义是十分重要的，如定义不当，则上述搜索算法不一定能找到问题的解，即使找到解，也不一定是最优的。为此，需要对估价函数进行某些限制。下面讨论的 A* 算法就是对估价函数进行了限制的一种搜索算法。

4.3.2 图的有序搜索与 A* 算法

前面讨论的择优搜索算法仅适合于搜索的状态空间是树状结构，树状结构的状态空间中的每个节点都只有惟一的父节点（根节点除外）。因此，对状态空间中的节点扩展生成其子节点时，其子节点只会被生成一次，也就是说，由搜索算法放入到 OPEN 表中的节点的状态是各不相同的，不会有重复的节点。

如果状态空间是一个有向图，也就是说，状态空间中的一个节点可能有多个父节点，那么，在搜索过程中，这个节点可能由不同的搜索路径被多次扩展生成。因此，在 OPEN 表中会出现重复的节点，节点的重复将导致大量冗余的搜索，甚至可能使搜索过程陷入无效的循环而无法找到解。为此，需要对择优搜索进行修正。修正的基本思想是：当搜索过程生成一个节点 i 时，需要把节点 i 的状态与已生成的所有节点的状态进行比较，若节点 i 是一个已生成的节点，则表示找到一条通过节点 i 的新路径。若新路径使节点 i 的估价值更小，则修改节点 i 指向父节点的指针，使之指向新的父节点；否则，不修改节点 i 原有的父节点指针，即保留节点 i 原有的路径。

1. 图的有序搜索算法

尼尔逊在 1971 年提出了有序状态空间搜索算法, 它适用于状态空间是一个无环有向图的搜索求解。其搜索过程如下:

- 1) 初始节点 S_0 放入 OPEN 表, 计算 $f(S_0)$ 。
- 2) 如果 OPEN 表为空, 则搜索失败, 退出。
- 3) 把 OPEN 表中的第一个节点 (记为节点 n) 从表中移出放入 CLOSED 表。
- 4) 考察节点 n 是否为目标节点, 若是, 则求得了问题的解, 退出。
- 5) 若节点 n 不可扩展, 则转第 2) 步。
- 6) 扩展节点 n , 生成其全部子节点。对节点 n 的每个子节点 i , 计算 $f(i)$ 。考察节点 i 是否为已生成过的节点。

①如果节点 i 既不在 OPEN 表中, 又不在 CLOSED 表中, 则节点 i 是一个新节点。为节点 i 配置指向父节点 n 的指针, 把节点 i 放入 OPEN 表中, 然后对 OPEN 表中的全部节点按估价值从小至大的顺序进行排序。

②如果节点 i 已在 OPEN 表中或在 CLOSED 表中, 则节点 i 是一个已生成过的节点。比较节点 i 刚计算的 $f(i)$ 新值与表中记载的 $f(i)$ 旧值, 若新的 $f(i)$ 值较小, 则:

(a) 对表中节点 i 的有关记载进行下述修改: 用 $f(i)$ 的新值代替旧值, 修改节点 i 指向父节点的指针, 使之指向新的父节点 n 。

(b) 若节点 i 在 CLOSED 表中, 则把节点 i 移回 OPEN 表。

7) 转第 2) 步。

由有序搜索算法的搜索过程可见, 如果节点 i 有多个父节点, 则为节点 i 选择一个父节点, 这个父节点使节点 i 具有最小的估价值 $f(i)$ 。因此, 尽管搜索的状态空间是一般的有向图, 但是, 由 OPEN 表和 CLOSED 表共同记载的搜索结果仍是一棵树。

2. A* 算法

如果有序搜索算法的估价函数 f 满足如下限制, 则成为 A* 算法。

节点的估价函数为

$$f(x) = g(x) + h(x)$$

其中:

1) 代价函数 $g(x)$ 是对 $g^*(x)$ 的估计, $g(x) > 0$ 。 $g^*(x)$ 是从初始节点 S_0 到节点 x 的最小代价。

2) 启发函数 $h(x)$ 是 $h^*(x)$ 的下界, 即对所有的 x 均有 $h(x) \leq h^*(x)$ 。 $h^*(x)$ 是从节点 x 到目标节点的最小代价。若有多个目标节点, 则为其中最小的代价。

在 A* 算法中, $g(x)$ 比较容易得到, 它实际上就是从初始节点 S_0 到节点 x 的路径代价, 恒有 $g(x) \geq g^*(x)$, 而且在算法执行过程中, $g(x)$ 的值呈下降的趋势。

$h(x)$ 的确定依赖于具体问题的启发性信息, 其中 $h(x) \leq h^*(x)$ 的限制是十分重要的, 它可保证 A* 算法能找到最优解。

下面我们来讨论 A* 算法的有关特性。

(1) A*算法可纳性

对于可解状态空间图（即从初始节点到目标节点有路径存在）来说，如果一个搜索算法能在有限步内终止，并且能找到最优解，则称该搜索算法是可纳的。

A*算法是可纳的，即它能在有限步内终止并找到最优解。下面分三步证明这一结论。

1) 对于有限图，A*算法一定会有有限步内终止。

证明：对于有限图，其节点个数是有限的。所以 A*算法在经过若干次循环之后只可能出现两种情况：一种是由于搜索到了目标节点在第 4) 步终止；另一种是由于 OPEN 表中的节点被取完而在第 2) 步终止。不管发生哪种情况，A*算法都在有限步内终止。

2) 对于无限图，只要从初始节点到目标节点有路径存在，则 A*算法也必然会终止。

证明：该证明分两步进行。第一步先证明在 A*算法结束之前，OPEN 表中总存在节点 x' ，它是最优路径的一个节点，且满足

$$f(x') \leq f^*(S_0)$$

设最优路径是

$$S_0, x_1, x_2, \dots, x_m, S_g^*$$

由于 A*算法中的 $h(x)$ 满足

$$h(x) \leq h^*(x)$$

所以 $f(S_0), f(x_1), f(x_2), \dots, f(x_m)$ 均不大于 $f(S_g^*), f(S_g^*) = f^*(S_0)$ 。

又因为 A*算法是全局择优的，所以在它结束之前，OPEN 表中一定含有 $S_0, x_1, x_2, \dots, x_m, S_g^*$ 中的一些节点，设 x' 是其中最前面的一个，则它必然满足

$$f(x') \leq f^*(S_0)$$

至此，第一步证明结束。

现在来进行第二步的证明。这一步用反证法，即假设 A*算法不终止，则会得出与上一步矛盾的结论，从而说明 A*算法一定会终止。

假设 A*算法不终止，并设 e 是图中各条边的最小代价， $d^*(x_n)$ 是从 S_0 到节点 x_n 的最短路径长度，则显然有

$$g^*(x_n) \geq d^*(x_n) \times e$$

又因为 $g(x_n) \geq g^*(x_n)$ ，所以有

$$g(x_n) \geq d^*(x_n) \times e$$

因为 $h(x_n) \geq 0, f(x_n) \geq g(x_n)$ ，故得到

$$f(x_n) \geq d^*(x_n) \times e$$

由于 A*算法不终止，随着搜索的进行， $d^*(x_n)$ 会无限增大，从而使 $f(x_n)$ 也无限增大。这就与上一步证明得出的结论矛盾，因为对可解状态空间来说， $f^*(S_0)$ 一定是有限值。

所以，只要从初始节点到目标节点有路径存在，即使对于无限图，A*算法也一定会终止。

3) A*算法一定终止在最优路径上

证明：假设 A*算法不是在最优路径上终止，而是在某个目标节点 t 处终止，即 A*算法未

能找到一条最优路径, 则

$$f(t)=g(t)>f^*(S_0)$$

但由 2) 的证明可知, 在 A* 算法结束之前, OPEN 表中存在节点 x' , 它在最优路径上, 且满足

$$f(x')\leq f^*(S_0)$$

此时, A* 算法一定会选择 x' 来扩展而不会选择 t , 这就与假设矛盾。所以, A* 算法一定终止在最优路径上。

根据可纳性的定义及以上证明可知 A* 算法是可纳的。同时由上面的证明还可得知: A* 算法选择扩展的任何一个节点 x 都满足如下性质:

$$f(x)\leq f^*(S_0)$$

(2) A* 算法的最优性

A* 算法的搜索效率在很大程度上取决于 $h(x)$, 在满足 $h(x)\leq h^*(x)$ 的前提下, $h(x)$ 的值越大越好。 $h(x)$ 的值越大, 表明它携带的启发性信息越多, 搜索时扩展的节点数越少, 搜索的效率越高。

设 $f_1(x)$ 与 $f_2(x)$ 是对同一问题的两个估价函数:

$$f_1(x)=g_1(x)+h_1(x)$$

$$f_2(x)=g_2(x)+h_2(x)$$

A_1^* 与 A_2^* 分别是以 $f_1(x)$ 及 $f_2(x)$ 为估价函数的 A* 算法, 且设对所有非目标节点 x 均有

$$h_1(x)<h_2(x)$$

在此情况下, 我们将证明 A_1^* 扩展的节点数不会比 A_2^* 扩展的节点数少。即 A_2^* 扩展的节点集是 A_1^* 扩展的节点集的子集。用归纳法证明如下:

设 k 表示搜索树的深度。当 $k=0$ 时, 结论显然成立。因为若初始状态就是目标状态, 则 A_1^* 与 A_2^* 都无须扩展任何节点; 若初始状态不是目标状态, 它们都要对初始节点进行扩展, 此时 A_1^* 与 A_2^* 扩展的节点数是相同的。

设当搜索树的深度为 $k-1$ 时结论成立, 即凡 A_2^* 扩展了的前 $k-1$ 代节点, A_1^* 也都扩展了。此时, 只要证明 A_2^* 扩展的第 k 代的任一节点 x_k 也被 A_1^* 扩展就可以了。

由假设可知, A_2^* 扩展的前 $k-1$ 代节点 A_1^* 也都扩展了, 因此在 A_1^* 搜索树中有一条从初始节点 S_0 到 x_k 的路径, 其费用不会比 A_2^* 搜索树中从 S_0 到 x_k 的费用更大, 即

$$g_1(x_k)\leq g_2(x_k)$$

假设 A_1^* 不扩展节点 x_k , 这表示 A_1^* 能找到另一个具有更小估价值的节点进行扩展并找到最优解, 此时有

$$f_1(x_k)\geq f^*(S_0)$$

即

$$g_1(x_k)+h_1(x_k)\geq f^*(S_0)$$

应用关系式 $g_1(x_k)\leq g_2(x_k)$ 到上述不等式中, 得到

$$h_1(x_k)\geq f^*(S_0)-g_2(x_k)$$

由于 $h_2(x_k) = f^*(S_0) - g_2(x_k)$, 所以得到

$$h_1(x_k) \geq h_2(x_k)$$

这与我们最初的假设 $h_1(x) < h_2(x)$ 是矛盾的。

由此可得出“ A^*_1 所扩展的节点数不会比 A^*_2 扩展的节点数少”这一结论是正确的。即：启发函数所携带的启发性信息越多，搜索时扩展的节点数越少，搜索效率越高。

(3) $h(x)$ 的单调性限制

在 A^* 算法中，每当要扩展一个节点时都要先检查其子节点是否已在 OPEN 表或 COLSED 表中，有时还需要调整指向父节点的指针，这就增加了搜索的代价。如果对启发函数 $h(x)$ 加上单调性限制，就可减少检查及调整的工作量，从而减少搜索代价。

所谓单调性限制是指 $h(x)$ 满足如下两个条件：

1) $h(S_g) = 0$ 。

2) 设 x_j 是节点 x_i 的任意子节点，则有

$$h(x_i) - h(x_j) \leq c(x_i, x_j)$$

其中， S_g 是目标节点； $c(x_i, x_j)$ 是节点 x_i 到其子节点 x_j 的边代价。

若把上述不等式改写为如下形式：

$$h(x_i) \leq h(x_j) + c(x_i, x_j)$$

就可以看出，节点 x_i 到目标节点最优费用的估价不会超过从 x_i 到其子节点 x_j 的边代价加上从 x_j 到目标节点最优费用的估价。

可以证明，当 A^* 算法的启发函数 $h(x)$ 满足单调限制时，可得到如下两个结论：

1) 若 A^* 算法选择节点 x_n 进行扩展，则

$$g(x_n) = g^*(x_n)$$

2) 由 A^* 算法所扩展的节点序列其 f 值是非递减的。

这两个结论都是当 $h(x)$ 满足单调限制时才成立。否则，它们不一定成立。例如对第 2) 个结论，当 $h(x)$ 不满足单调限制时，有可能某个要扩展的节点比以前扩展的节点具有较小的 f 值。

4.4 与 / 或树的搜索策略

与 / 或树搜索策略也分为盲目搜索与启发式博弈搜索两大类。下面讨论的宽度优先搜索及深度优先搜索都属于盲目搜索策略，有序搜索及博弈树搜索则属于启发式搜索策略。

与 / 或树上的一个节点是否为可解节点是由它的子节点确定的。对于一个“与”节点，只有当其子节点全部为可解节点时，它才为可解节点；只要子节点中有一个为不可解节点，它就是不可解节点。对于一个“或”节点，只要子节点中有一个是可解节点，它就是可解节点；只有当全部子节点都是不可解节点时，它才是不可解节点。像这样由可解子节点来确定父节点、祖父节点等为可解节点的回溯向上过程称为可解标示过程；由不可解子节点来确定其父节点、祖父节点等为不可解节点的回溯向上过程称为不可解标示过程。在与 / 或树的搜索过程中将反

复使用这两个过程，直到初始节点（即原始问题）被标示为可解或不可解节点为止。

与 / 或树的一般搜索过程为：

- 1) 把原始问题作为初始节点 S_0 ，并把它作为当前节点。
- 2) 应用分解或等价变换算符对当前节点进行扩展。
- 3) 为每个子节点设置指向父节点的指针。
- 4) 选择合适的子节点作为当前节点，反复执行第 2) 步和第 3) 步，在此期间要多次调用可解标示过程和不可解标示过程，直到初始节点被标示为可解节点或不可解节点为止。

由这个搜索过程所形成的节点和指针结构称为搜索树。

与 / 或树搜索的目标是寻找解树，从而求得原始问题的解。如果在搜索的某一时刻，通过可解标示过程可确定初始节点是可解的，则由此初始节点及其下属的可解节点就构成了解树。

可解标示过程和不可解标示过程都是自下而上进行的，即由子节点的可解性确定父节点的可解性。由于与 / 或树搜索的目标是寻找解树，因此，如果已确定某个节点为可解节点，则其不可解的后裔节点就不再有用了，可从搜索树中删去。同样，如果已确定某个节点是不可解节点，则其全部后裔节点都不再有用了，可从搜索树中删去。但当前这个不可解节点还不能删去，因为在判断其先辈节点的可解性时还要用到它。这是与 / 或树搜索的两个特有的性质，可用来提高搜索效率。

4.4.1 与 / 或树的宽度优先搜索

与 / 或树的宽度优先搜索与状态空间的宽度优先搜索类似，也是按照“先产生的节点先扩展”的原则进行搜索，只是在搜索过程中要多次调用可解标示过程和不可解标示过程。其搜索过程如下：

- 1) 把初始节点 S_0 放入 OPEN 表。
- 2) 把 OPEN 表中的第一个节点（记为节点 n ）取出放入 CLOSED 表。
- 3) 如果节点 n 可扩展，则：
 - ① 扩展节点 n ，将其子节点放入 OPEN 表的尾部，并为每个子节点配置指向父节点的指针。
 - ② 考察这些子节点中是否有终止节点。若有，则标示这些终止节点为可解节点，并应用可解标示过程对其父节点、祖父节点等先辈节点中的可解节点进行标示。如果初始节点 S_0 也被标示为可解节点，就得到了解树，搜索成功，退出搜索过程；如果不能确定 S_0 为可解节点，则从 OPEN 表中删去具有可解先辈的节点。
- ③ 转第 2) 步。
- 4) 如果节点 n 不可扩展，则：
 - ① 标示节点 n 为不可解节点。
 - ② 应用不可解标示过程对节点 n 的先辈节点中不可解的节点进行标示。如果初始节点 S_0 也被标示为不可解节点，则搜索失败，表明原始问题无解，退出搜索过程；如果不能确定 S_0 为不可解节点，则从 OPEN 表中删去具有不可解先辈的节点。

③转第2)步。

上述过程可用图 4.18 表示其工作流程。

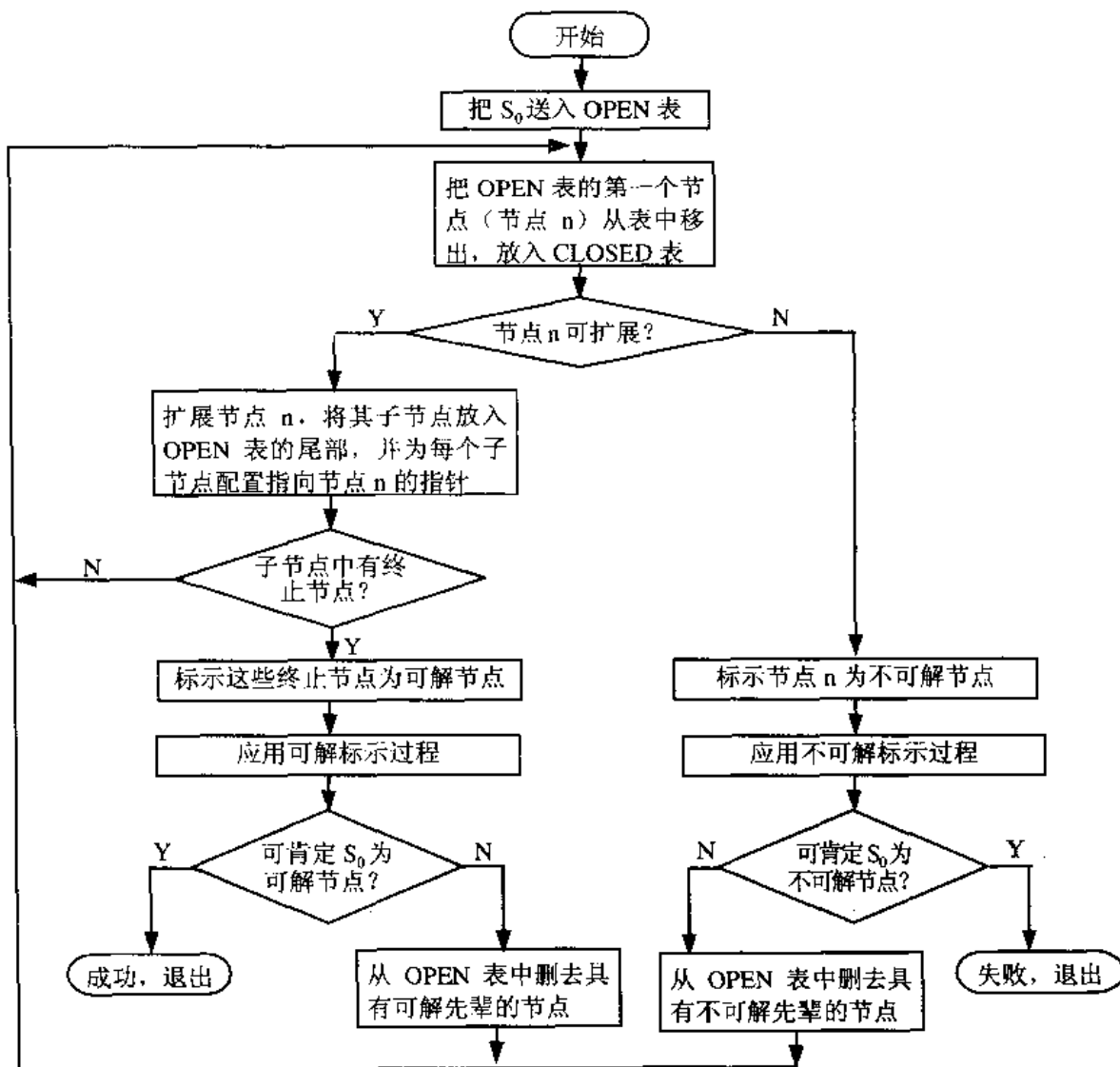


图 4.18 与/或树的宽度优先搜索流程示意图

例 4.8 设有如图 4.19 所示的与/或树, 节点按图中所标注的顺序号进行扩展。其中标有 t_1 、 t_2 、 t_3 、 t_4 的节点均为终止节点, A 和 B 为不可解的端节点。

搜索过程为:

1) 首先扩展节点 1, 得到节点 2 和节点 3, 由于这两个子节点均不是终止节点, 所以接着扩展节点 2。此时 OPEN 表中只剩下节点 3。

2) 扩展节点 2 后, 得到节点 4 和节点 t_1 。此时 OPEN 表中的节点有: 节点 3、节点 4 及节点 t_1 。由于 t_1 是终止节点, 则标示它为可解节点, 并应用可解标示过程, 对其先辈节点中的可解节点进行标示。在此例中, t_1 的父节点 2 是一个“与”节点, 因此仅由 t_1 可解尚不能确

定节点 2 是否为可解节点。所以继续搜索，下一步扩展节点 3。

3) 扩展节点 3 得到节点 5 与节点 B，两者均不是终止节点，所以接着扩展节点 4。

4) 扩展节点 4 后得到节点 A 和 t_2 。由于 t_2 是终止节点，所以标示它为可解节点，并应用可解标示过程标示出节点 4、节点 2 均为可解节点，但节点 1 目前还不能确定是否为可解节点。此时节点 5 是 OPEN 表中的第一个待考察的节点，所以下一步扩展节点 5。

5) 扩展节点 5，得到节点 t_3 和 t_4 。由于 t_3 和 t_4 均为终止节点，所以被标示为可解节点，通过应用可解标示过程可得到节点 5、节点 3 及节点 1 均为可解节点。

6) 搜索成功，得到由节点 1、2、3、4、5 及节点 t_1 、 t_2 、 t_3 、 t_4 构成的解树。如图 4.19 中粗线所示。

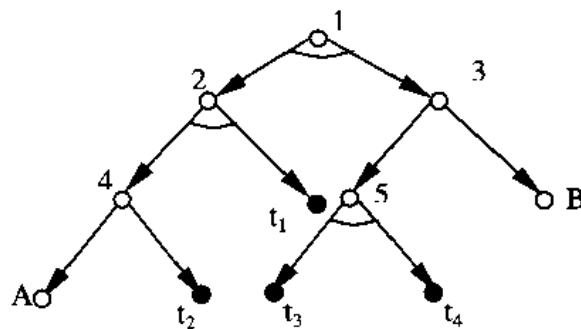


图 4.19 与/或树的宽度优先搜索

4.4.2 与/或树的有界深度优先搜索

与/或树的深度优先搜索过程和与/或树的宽度优先搜索过程基本相同，只是要把第 3) 步的第①点改为“扩展节点 n ，将其子节点放入 OPEN 表的首部，并为每个子节点配置指向父节点的指针”，就可使后产生的节点先被扩展。

可以像状态空间的有界深度优先搜索那样，为与/或树的深度优先搜索规定一个深度界限，使搜索在规定的范围内进行。其搜索过程如下：

1) 把初始节点 S_0 放入 OPEN 表。

2) 把 OPEN 表中的第一个节点（记为节点 n ）取出放入 CLOSED 表。

3) 如果节点 n 的深度大于等于深度界限，则转第 5) 步的第①点。

4) 如果节点 n 可扩展，则：

①扩展节点 n ，将其子节点放入 OPEN 表的首部，并为每个子节点配置指向父节点的指针，以备标示过程使用。

②考察这些子节点中是否有终止节点。若有，则标示这些终止节点为可解节点，并应用可解标示过程对其先辈节点中的可解节点进行标示。如果初始节点 S_0 也被标示为可解节点，则搜索成功，退出搜索过程；如果不能确定 S_0 为可解节点，则从 OPEN 表中删去具有可解先辈的节点。

③转第 2) 步。

5) 如果节点 n 不可扩展, 则:

① 标示节点 n 为不可解节点。

② 应用不可解标示过程对节点 n 的先辈节点中不可解的节点进行标示。如果初始节点 S_0 也被标示为不可解节点, 则搜索失败, 表明原始问题无解, 退出搜索过程; 如果不能确定 S_0 为不可解节点, 则从 OPEN 表中删去具有不可解先辈的节点。

③ 转第 2) 步。

上述过程可用图 4.20 表示其工作流程。

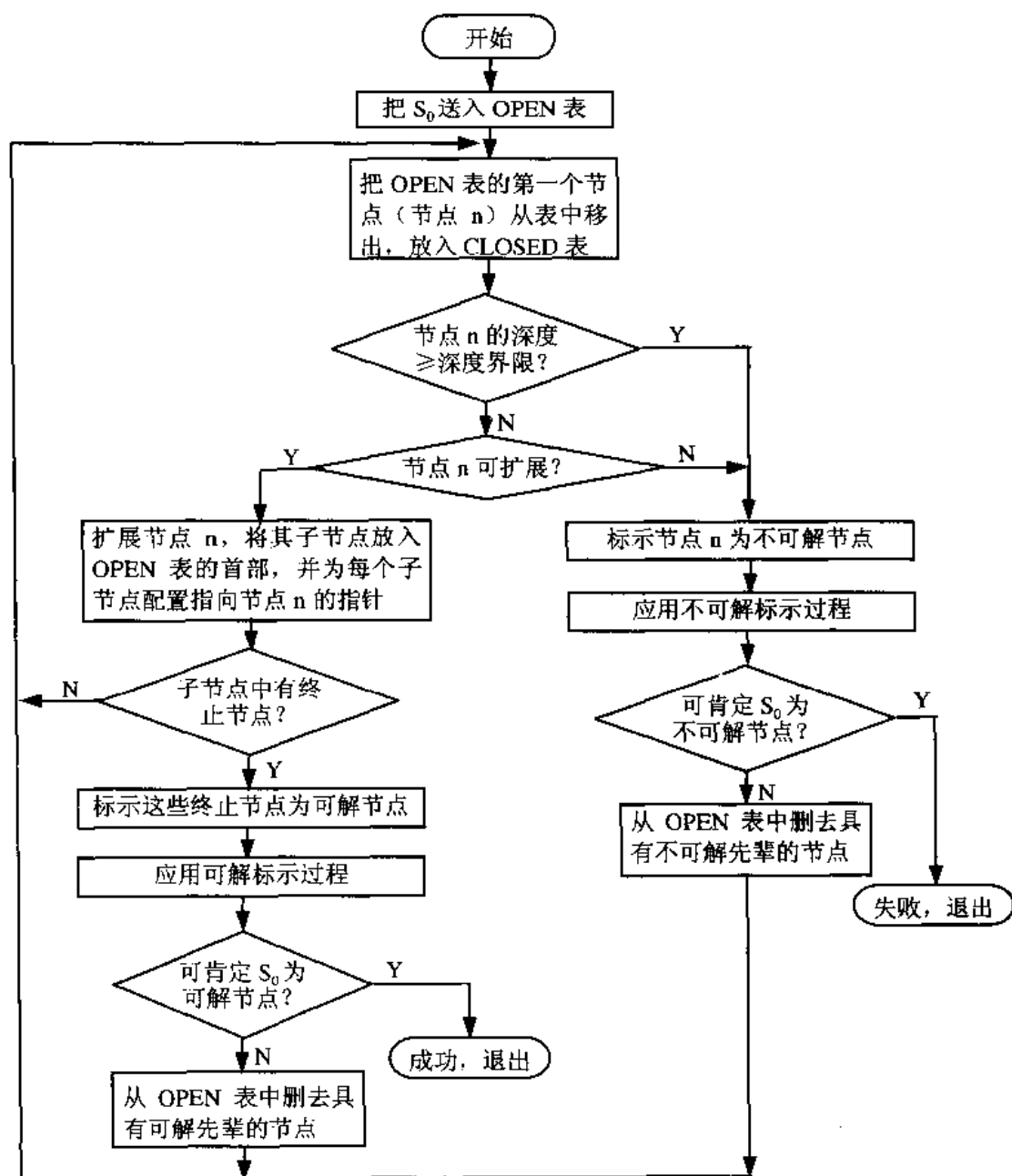


图 4.20 与/或树的有界深度优先搜索流程示意图

若对图 4.19 所示的与/或树进行有界深度优先搜索, 并规定深度界限为 4, 则扩展节点的顺序是: 1, 3, B, 5, 2, 4。其解树仍为图中粗线部分。

4.4.3 与/或树的有序搜索

与/或树的有序搜索可用来求取代价最小的解树, 它是一种启发式搜索策略。

1. 解树的代价

为了进行有序搜索, 需要计算解树的代价。解树的代价可通过计算解树中节点的代价得到。

设 $c(x, y)$ 表示节点 x 到其子节点 y 的代价, 计算节点 x 代价的方法如下:

1) 如果 x 是终止节点, 则定义节点 x 的代价 $h(x)=0$;

2) 如果 x 是“或”节点, $y_1, y_2, \dots, y_n$ 是它的子节点, 则节点 x 的代价为

$$h(x) = \min_{1 \leq i \leq n} \{c(x, y_i) + h(y_i)\}$$

3) 如果 x 是“与”节点, 则节点 x 的代价有两种计算方法: 和代价法与最大代价法。

若按和代价法计算, 则有

$$h(x) = \sum_{i=1}^n (c(x, y_i) + h(y_i))$$

若按最大代价法计算, 则有

$$h(x) = \max_{1 \leq i \leq n} \{c(x, y_i) + h(y_i)\}$$

4) 如果 x 不可扩展, 且又不是终止节点, 则定义 $h(x)=\infty$ 。

由上述计算节点的代价可以看出, 如果问题是可解的, 则由子节点的代价就可推算出父节点代价, 这样逐层上推, 最终就可求出初始节点 S_0 的代价。 S_0 的代价就是解树的代价。

例 4.9 图 4.21 是一棵与/或树, 其中包括两棵解树, 一棵解树由 S_0, A, t_1 和 t_2 组成; 另一棵解树由 S_0, B, D, G, t_4 和 t_5 组成。在此与/或树中, t_1, t_2, t_3, t_4, t_5 为终止节点; E, F 是端节点, 其代价为 ∞ ; 边上的数字是该边的代价。

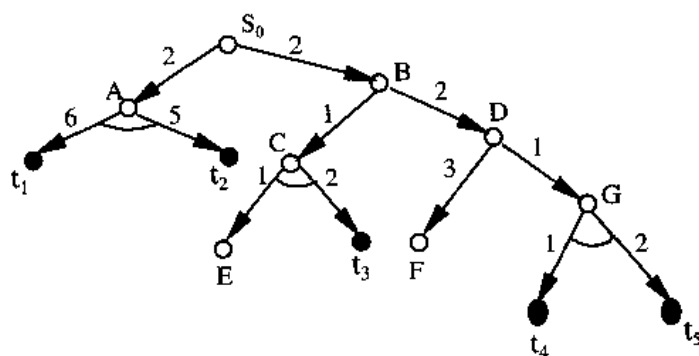


图 4.21 与/或树的代价计算示例

由左边的解树可得:

按和代价: $h(A)=11, h(S_0)=13$

按最大代价: $h(A)=6, h(S_0)=8$

由右边的解树可得:

按和代价: $h(G)=3, h(D)=4, h(B)=6, h(S_0)=8$

按最大代价: $h(G)=2, h(D)=3, h(B)=5, h(S_0)=7$

显然, 若按和代价计算, 右解树是最优解树, 其代价为 8; 若按最大代价计算, 右解树仍然是最优解树, 其代价是 7。有时用不同的计算代价方法得到的最优解树不相同。

2. 希望树

无论是用和代价方法还是最大代价方法, 当要计算任一节点 x 的代价 $h(x)$ 时, 都要求已知其子节点 y_i 的代价 $h(y_i)$ 。但是, 搜索是自上而下进行的, 即先有父节点, 后有子节点, 那么节点 x 的代价 $h(x)$ 如何计算呢? 解决的办法是根据问题本身提供的启发性信息定义一个启发函数, 由此启发函数估算出了节点 y_i 的代价 $h(y_i)$, 然后再算出节点 x 的代价值 $h(x)$ 。有了 $h(x)$, 节点 x 的父节点、祖父节点以及直到初始节点 S_0 的各先辈节点的代价 h 都可自下而上的逐层推算出来。

有序搜索的目的是求出最优解树, 即代价最小的解树。这就要求搜索过程中任一时刻求出的部分解树的代价都应是最低的。为此, 每次选择欲扩展的节点时都应挑选有希望成为最优解树一部分的节点进行扩展。由这些节点及其先辈节点 (包括初始节点 S_0) 所构成的与/或树称为“希望树”。在搜索过程中, 随着新节点的不断生成, 节点的代价值是在不断变化的, 因此希望树也是在不断变化的。

下面给出希望树的定义:

1) 初始节点 S_0 在希望树 T 中。

2) 如果节点 x 在希望树 T 中, 则一定有:

①如果 x 是具有子节点 $y_1, y_2, \dots, y_n$ 的“或”节点, 则具有

$$\min_{1 \leq i \leq n} \{c(x, y_i) + h(y_i)\}$$

值的那个节点 y_i 也应在 T 中。

②如果 x 是“与”节点, 则它的全部子节点都应在 T 中。

3. 与/或树的有序搜索过程

与/或树的有序搜索是一个不断选择、修正希望树的过程。如果问题有解, 则经过有序搜索将找到最优解树。其搜索过程如下:

1) 把初始节点 S_0 放入 OPEN 表中。

2) 求出希望树 T , 即根据当前搜索树中节点的代价 h 求出以 S_0 为根的希望树 T 。

3) 依次把 OPEN 表中 T 的端节点 n 选出放入 CLOSED 表中。

4) 如果节点 n 是终止节点, 则

①标示 n 为可解节点。

②对 T 应用可解标示过程, 把 n 的先辈节点中的可解节点都标示为可解节点。

③若初始节点 S_0 能被标示为可解节点, 则 T 就是最优解树, 成功退出; 否则, 从 OPEN 表中删去具有可解先辈的所有节点。

5) 如果节点 n 不是终止节点, 且它不可扩展, 则:

①标示 n 为不可解节点。

②对 T 应用不可解标示过程, 把 n 的先辈节点中的不可解节点都标示为不可解节点。

③若初始点 S_0 也被标示为不可解节点, 则失败退出; 否则, 从 $OPEN$ 表中删去具有不可解先辈的所有节点。

6) 如果节点 n 不是终止节点, 但它可扩展, 则:

①扩展节点 n , 产生 n 的所有子节点。

②把这些子节点都放入 $OPEN$ 表中, 为每个子节点配置指向父节点 (节点 n) 的指针。

③计算这些子节点的 h 值及其先辈节点的 h 值。

7) 转第 2) 步。

上述搜索过程中可用图 4.22 描述其工作流程。

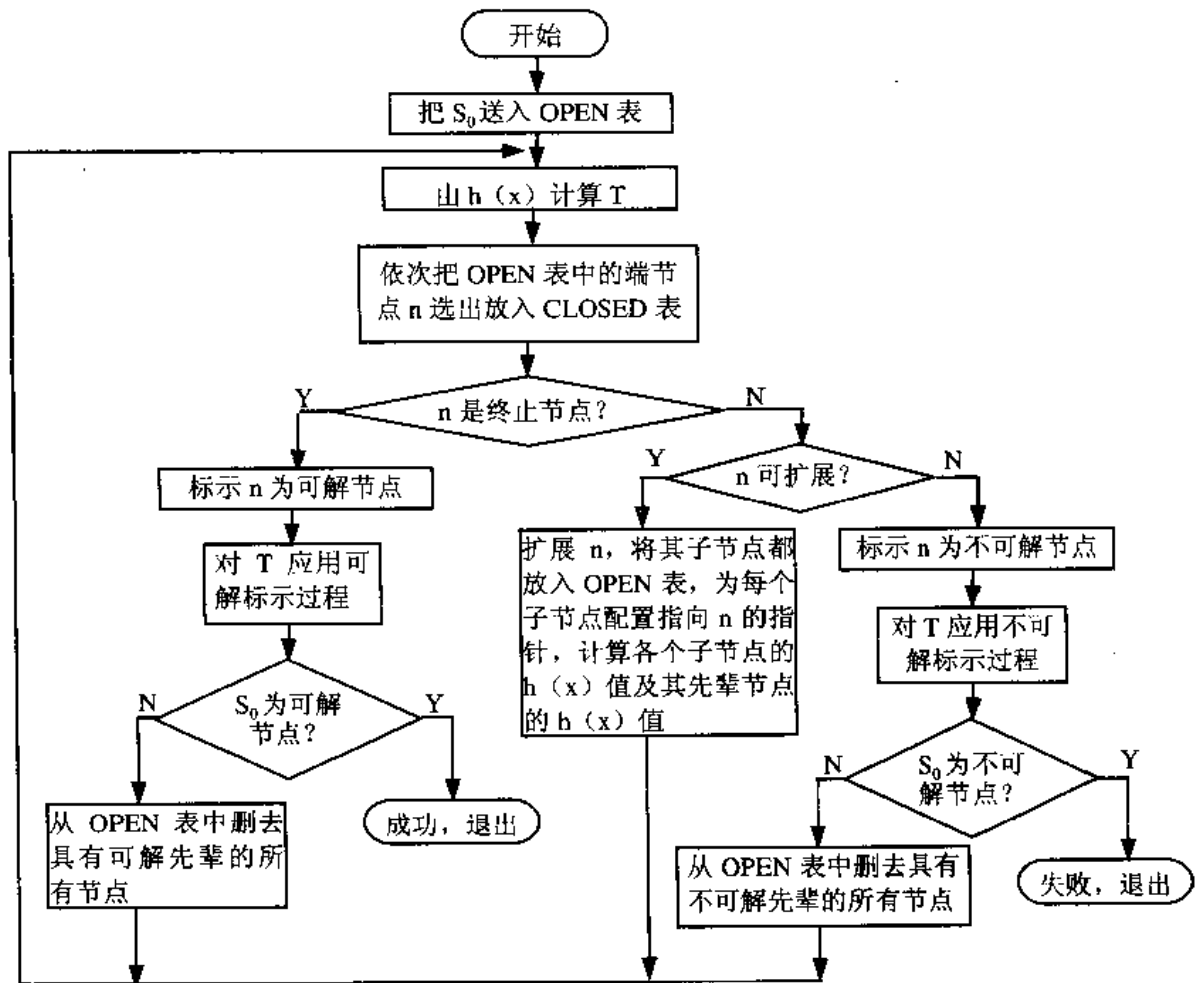


图 4.22 与/或树的有序搜索流程示意图

例 4.10 设初始节点为 S_0 , 每次扩展两层, 且一层是“与”节点, 一层是“或”节点。并设 S_0 经扩展后得到如图 4.23 所示的与/或树。

在图 4.23 中, 子节点 B, C, E, F 用启发函数估算出的 h 值分别是:

$$h(B)=3, h(C)=3, h(E)=3, h(F)=2$$

若按和代价法计算, 则得到:

$$h(A)=8, h(D)=7, h(S_0)=8$$

此时, S_0 的右子树是希望树。

设对节点 E 扩展两层后得到如图 4.24 所示的与/或树, 节点旁的数为用启发函数估算出的 h 值。

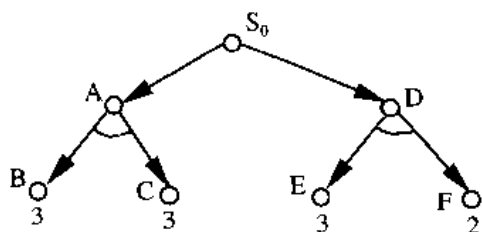


图 4.23 扩展两层后的与/或树

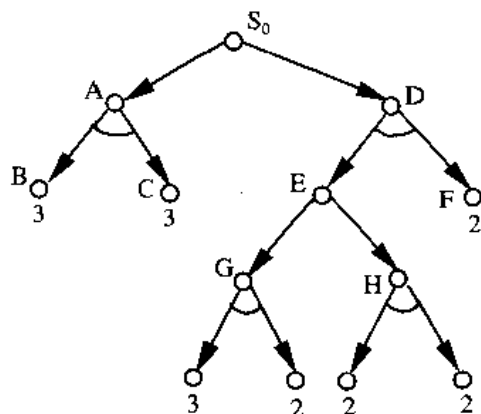


图 4.24 扩展 E 后的与/或树

按和代价法计算, 得到:

$$h(G)=7, h(H)=6, h(E)=7, h(D)=11$$

此时, 由 S_0 的右子树算出的 $h(S_0)=12$ 。但是, 由左子树算出的 $h(S_0)=9$ 。显然, 左子树的代价小, 所以现在改为取左子树作为当前的希望树。

假设对节点 B 扩展两层后得到如图 4.25 所示的与/或树, 节点旁的数字是对相应节点的估算值, 节点 L 的两个子节点是终止节点。

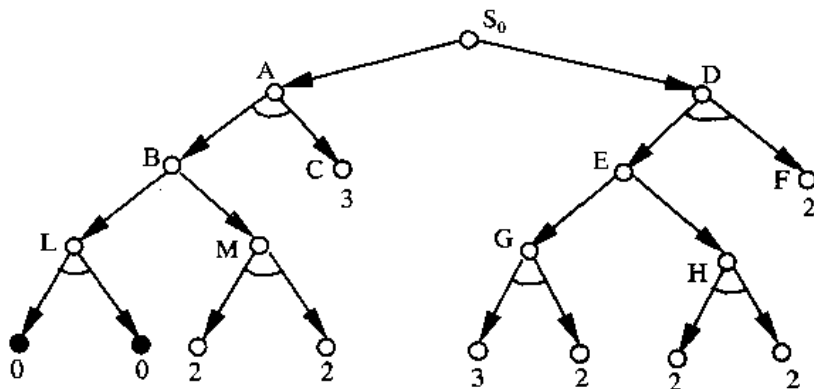


图 4.25 扩展 B 后的与/或树

按和代价法计算, 得到:

$$h(L)=2, h(M)=6, h(B)=3, h(A)=8$$

由此推算出 $h(S_0)=9$ 。另外, 由于 L 的两个子节点都是终止节点, 所以 L、B 都是可解节点。因节点 C 目前还不能肯定是可解节点, 故 A 和 S_0 也还不能确定为可解节点。下面对节点 C 进行扩展。

假设节点 C 扩展两层后得到如图 4.26 所示的与/或树，节点旁的数字是对相应节点的估算值，节点 N 的两个子节点是终止节点。

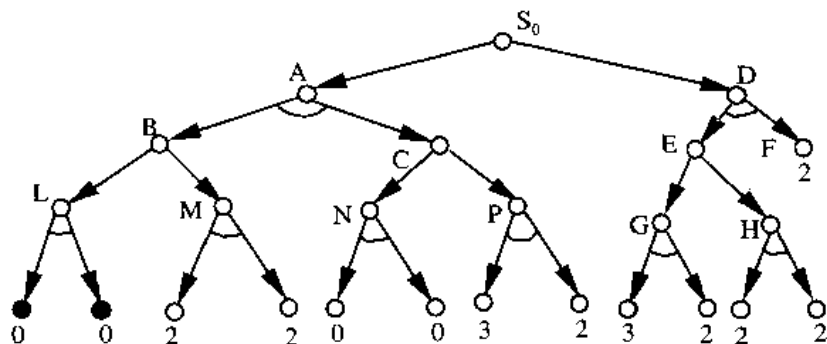


图 4.26 扩展 C 后的与/或树

按和代价法计算，得到：

$$h(N)=2, h(P)=7, h(C)=3, h(A)=8$$

由此可推算出 $h(S_0)=9$ 。另外，由于 N 的两个子节点都是终止节点，所以 N 和 C 都是可解节点。再由前面推出的 B 是可解节点，就可推出 A 和 S_0 都是可解节点。这样就求出了代价最小的解树，即最优解树，如图 4.26 中粗线部分所示。该最优解树是用和代价法求出来的，解树的代价为 9。

4.4.4 博弈树的启发式搜索

1. 博弈树的概念

诸如下棋、打牌、战争等一类竞争性智能活动称为博弈。其中最简单的一种称为双方完备博弈。双方完备博弈是指：

- 1) 对垒的 A、B 双方轮流采取行动，博弈的结果只有三种情况：A 方胜，B 方败；B 方胜，A 方败；双方战成平局。
- 2) 在对垒过程中，任何一方都了解当前的格局及过去的历史。
- 3) 任何一方在采取行动前都要根据当前的实际情况，进行得失分析，选取对自己最为有利而对对方最为不利的对策。

在博弈过程中，任何一方都希望自己取得胜利。因此，在某一方当前有多个行动方案可供选择时，他总是挑选对自己最为有利而对对方最不利的那个行动方案。此时，如果我们站在 A 方的立场上，则可供 A 方选择的若干行动方案之间是“或”关系。但是，若 B 方也有若干个可供选择的行动方案，则对 A 方来说这些行动方案之间是“与”关系，因为这时主动权操在 B 方手里，A 方必须考虑任何一个可能被 B 方选中的行动方案。若把上述博弈过程用图表示出来，得到的是一棵“与/或”树。这里要特别指出，该“与/或”树是始终站在某一方（例如 A 方）的立场上得出的。

称描述博弈过程的与/或树为博弈树，它有如下特点：

- 1) 博弈的初始格局是初始节点。
- 2) 在博弈树中,“或”节点和“与”节点是逐层交替出现的。自己一方扩展的节点之间是“或”关系,对方扩展的节点之间是“与”关系。双方轮流地扩展节点。
- 3) 所有能使自己一方获胜的终局都是本原问题,相应的节点是可解节点,所有使对方获胜的终局都是不可解节点。

2. 极大极小分析法

在二人博弈问题中,为了从众多可供选择的行动方案中选出一个对自己有利的行动方案,就需要对当前情况以及将要发生的情况进行分析,从中选出最优者。最常使用的分析方法是极大极小分析法。其基本思想是:

- 1) 设博弈的双方中一方为 A, 另一方为 B。极大极小分析法是为其中的一方(例如 A)寻找一个最优行动方案的方法。
- 2) 为了找到当前的最优行动方案,需要对各个方案可能产生的后果进行比较。具体地说,就是要考虑每一方案实施后对方可能采取的所有行动,并计算可能的得分。
- 3) 为了计算得分,需要根据问题的特性信息定义一个估价函数,用来估算当前博弈树端节点的得分。此时估算出来的得分称为静态估值。
- 4) 当端节点的估值计算出来后,再推算出父节点的得分。推算的方法是:对“或”节点,选其子节点中一个最大的得分作为父节点的得分,这是为了使自己在可供选择的方案中选一个对自己最有利的方案;对“与”节点,选其子节点中一个最小的得分作为父节点的得分,这是为了立足于最坏的情况。这样计算出的父节点的得分称为倒推值。
- 5) 如果一个行动方案能获得较大的倒推值,则它就是当前最好的行动方案。

图 4.27 给出了计算博弈树倒推值的示例。

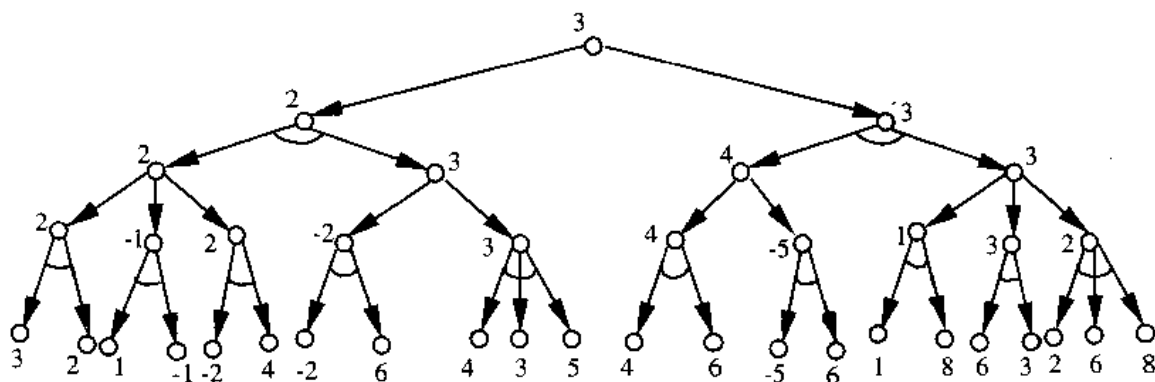


图 4.27 博弈树倒推值的计算

在博弈问题中,每一个格局可供选择的行动方案都有很多,因此会生成十分庞大的博弈树。据统计,西洋跳棋完整的博弈树约有 10^{40} 个节点。试图利用完整的博弈树来进行极大极小分析是困难的。可行的办法是只生成一定深度的博弈树,然后进行极大极小分析,找出当前最好的行动方案。在此之后,再在已经选定的分支上扩展一定深度,再选最好的行动方案。

如此进行下去，直到取得胜败的结果为止。

例 4.11 一字棋游戏。设有如图 4.28 所示的九个空格，由 A, B 二人对弈，轮到谁走棋谁就往空格上放自己的一只棋子，谁先使自己的棋子构成三子成一线，谁就取得了胜利。

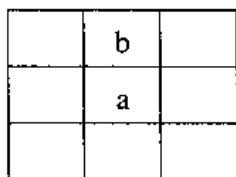


图 4.28 一字棋

设 A 的棋子用 “a” 表示，B 的棋子用 “b” 表示。格局 P 的估价函数 $e(P)$ 定义如下：

- 1) 若 P 是 A 必胜的棋局，则 $e(P) = +\infty$ 。
- 2) 若 P 是 B 必胜的棋局，则 $e(P) = -\infty$ 。
- 3) 若 P 是胜负未定的棋局，则 $e(P) = e(+P) - e(-P)$ 。

其中， $e(+P)$ 表示棋局 P 上有可能由 a 占满的行、列或对角线的数目。 $e(-P)$ 表示棋局 P 上有可能由 b 占满的行、列或对角线的数目。例如对于图 4.28 所示的棋局，则 $e(P) = 6 - 4 = 2$ 。

图 4.29 给出了假定 A 先走棋，且站在 A 的立场上生成的部分博弈树。图中节点旁的数字分别表示相应节点的静态估值或倒推值。由图 4.29 可以看出，对于 A 来说最好的一着棋是 S_3 ，因为 S_3 比 S_1 和 S_2 有较大的倒推值。

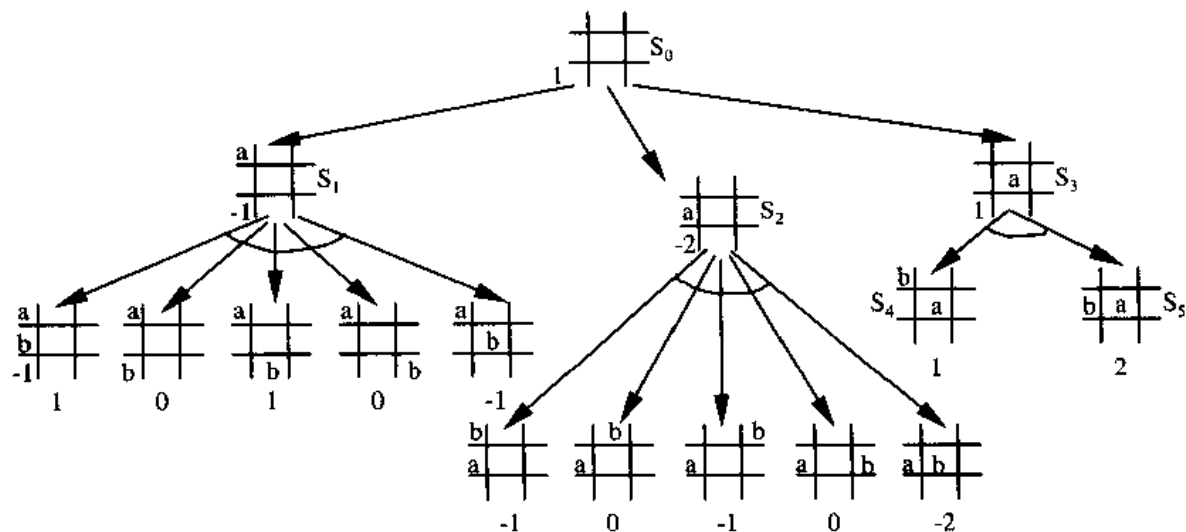


图 4.29 一字棋的极大极小搜索

在 A 走 S_3 这一着棋后，B 的最优选择是 S_4 ，因为这一着棋的静态估值较小，对 A 不利。不管 B 选择 S_4 或 S_5 ，A 都要再次运用极大极小分析法决定下一步应该如何走棋，其过程与上面类似，不再重复。

3. α - β 剪枝技术

在极大极小分析法中，总是先生成一定深度的博弈树，然后对端节点进行估值，再计算出上层节点的倒推值，这样做效率较低。鉴于博弈树具有“与”节点和“或”节点逐层交替出现的特点，如能边生成节点边计算估值及倒推值，就有可能删去一些不必要的节点，从而减少搜索及计算的工作量。例如，设按每次生成两层的原则得到如图 4.30 所示的博弈树。各端节点的估值如图中所示，其中 S_6 尚未计算出其估值。由 S_3 与 S_4 的估值得到 S_1 的倒推值为 3，这表示 S_0 的倒推值最小为 3。另外，由 S_5 的估值得到 S_2 的倒推值最大为 2，因此 S_0 的倒推值为 3。这里，虽然没有计算 S_6 的估值，仍然不影响对上层节点倒推值的推算，这表示这个分枝可以从博弈树中剪去。

像这样通过边生成边计算，从而剪去某些分枝的技术称为 α - β 剪枝技术。

对于一个“与”节点来说，它取当前子节点中的最小倒推值作为它的倒推值的上界，称此值为 β 值。对于一个“或”节点来说，它取当前子节点中的最大倒推值作为它的倒推值的下界，称此值为 α 值。

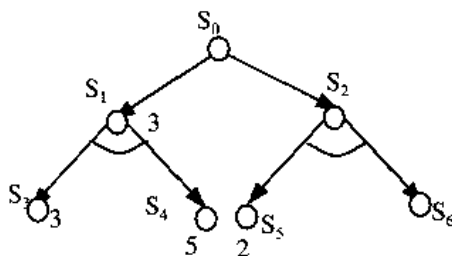


图 4.30 α - β 剪枝示例

由上面的例子可归纳出 α - β 剪枝技术的一般规律：

- 1) 任何“或”节点 x 的 α 值如果不能降低其父节点的 β 值，则对节点 x 以下的分枝可停止搜索，并使 x 的倒推值为 α 。这种剪枝称为 β 剪枝。
- 2) 任何“与”节点 x 的 β 值如果不能升高其父节点的 α 值，则对节点 x 以下的分枝可停止搜索，并使 x 的倒推值为 β 。这种剪枝称为 α 剪枝。

在 α - β 剪枝技术中，一个节点的第一个子节点的倒推值（或估值）是很重要的。对于一个“或”节点，如果估值最高的子节点最先生成，或者对于一个“与”节点，估值最低的子节点最先生成，则被剪除的节点数最多，搜索的效率最高。这称为最优 α - β 剪枝法。

4.5 搜索性能的量度

搜索过程的性能主要是指搜索过程的完备性与搜索效率。

1. 完备性

对于一类可解的问题和一个搜索过程，如果运用该搜索过程一定能求得该类问题的解，

则称该搜索过程为完备的，否则为不完备的。

完备的搜索过程称为“搜索算法”，简称为“算法”。不完备的搜索过程不是算法，称为“过程”。在前面讨论的搜索过程中，宽度优先搜索、代价树的宽度优先搜索、改进后的有界深度优先搜索以及 A* 算法都是完备的搜索过程。

2. 搜索效率

一个搜索过程的搜索效率不仅取决于过程自身的启发能力，而且还与被求解问题的有关属性等多种因素有关。目前虽已有多种定义和计算搜索效率的方法，但都有一定的局限性。下面讨论两种常用的方法，它们适用于比较求解同一问题的不同搜索方法的效率。

(1) 渗透率

渗透率定义为

$$P=L/T$$

其中， L 为从初始节点到目标节点的路径长度； T 为整个搜索过程中所生成的节点总数（包括目标节点，但不包括初始节点）。

渗透率反映了搜索过程中从初始节点向目标节点前进时搜索区域的宽度，它量度出搜索过程所生成的搜索树是“单枝延伸”还是“多枝密叶”的程度。当 $L=T$ 时， $P=1$ ，表示搜索过程中每次只生成一个节点，它恰好是解路径上的节点，搜索效率最高。 P 愈小表示搜索时产生的无用节点愈多，搜索效率愈低。

(2) 有效分枝因数

有效分枝因数 B 定义为

$$B+B^2+\cdots+B^L=T$$

其中， B 是有效分枝因数，它表示在整个搜索过程中每个有效节点平均生成的子节点数目； L 为路径长度； T 为节点总数。

当 $B=1$ 时，有： $1+1^2+\cdots+1^L=L=T$ ，此时所生成的节点数最少，搜索效率最高。

不难证明，有效分枝因数 B 与渗透率 P 之间有如下关系：

$$P=(L \times (B-1))/(B \times B^L-1)$$

$$T=B \times (B^L-1)/(B-1)$$

由此可以看出，当 B 一定时， L 愈大则 P 愈小；当 L 一定时， B 愈大则 P 愈小。另外，对同一个 L 而言， B 愈大则 T 愈大，即对一定的解路径来说，分枝愈多，搜索空间产生的节点也愈多。

习题四

4.1 何谓搜索？盲目搜索与启发式搜索的根本区别是什么？

4.2 用状态空间法表示问题时，什么是问题的解？什么是最优解？问题的最优解是惟一吗？

4.3 在与/或树中,何谓端节点?何谓终止节点?何谓可解节点?何谓解树?

4.4 局部择优搜索与全局择优搜索的区别是什么?在给出的这两种搜索过程中是如何体现这一区别的?

4.5 分别满足什么条件,深度优先搜索和代价树的深度优先搜索可看成是局部择优搜索的特例?

4.6 分别满足什么条件,宽度优先搜索和代价树的宽度优先搜索可看成是全局择优搜索的特例?

4.7 为什么说局部择优搜索与全局择优搜索仅适用于状态空间是代价树的搜索求解,而有序搜索与A*算法可适用于有向图的搜索求解?

4.8 图的有序搜索与A*算法的区别是什么?

4.9 与/或树的解树的代价是什么?如何递推地计算解树的代价?

4.10 与/或树的有序搜索过程中,为什么要引入希望树T?希望树T是如何组成的?

4.11 何谓双方完备博弈?博弈树有何特点?

4.12 应用宽度优先搜索求解重排九宫问题。问题的初始状态 S_0 和目标状态 S_g 分别为:

$$S_0 = \begin{pmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & & 5 \end{pmatrix} \quad S_g = \begin{pmatrix} 1 & 2 & 3 \\ 8 & & 4 \\ 7 & 6 & 5 \end{pmatrix}$$

可使用的算符有:空格左移、空格上移、空格右移、空格下移。画出搜索树,并给出问题的解。

4.13 应用有界深度优先搜索求解重排九宫问题。设搜索深度界限 $d_m=5$, 问题的初始状态 S_0 和目标状态 S_g 分别为:

$$S_0 = \begin{pmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & & 5 \end{pmatrix} \quad S_g = \begin{pmatrix} 1 & 2 & 3 \\ 8 & & 4 \\ 7 & 6 & 5 \end{pmatrix}$$

画出搜索树,并给出问题的解。

4.14 应用全局择优搜索求解重排九宫问题。问题的初始状态 S_0 和目标状态 S_g 分别为:

$$S_0 = \begin{pmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & & 5 \end{pmatrix} \quad S_g = \begin{pmatrix} 1 & 2 & 3 \\ 8 & & 4 \\ 7 & 6 & 5 \end{pmatrix}$$

设估价函数为 $f(x)=d(x)+h(x)$,其中, $d(x)$ 为节点 x 的深度, $h(x)$ 为节点 x 的状态 S_x 与目标节点状态 S_g 的棋局不相同的棋子数。画出搜索树,并给出问题的解。

4.15 应用宽度优先搜索求解下述圆盘旋转问题:设有3个大小不等的圆盘A, B, C套在一根轴上,每个圆盘的盘面上都间隔 90° 标有数字1、2、3、4。每个圆盘都可独立地绕轴逆时针转动,每次转动 90° 。其初始状态 S_0 和目标状态 S_g 如图1所示。画出搜索树,并给出问题的解。

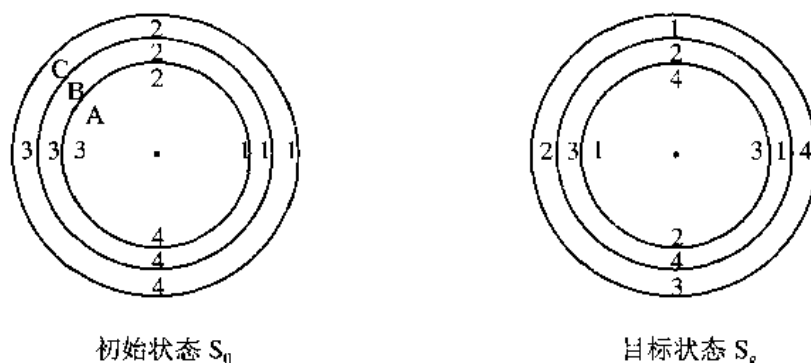


图1 圆盘旋转问题

4.16 设有下述火柴倒置问题：桌面上有 3 根并列的火柴，中间的一根火柴头向上，其余 2 根火柴头向下。规定每次操作为倒置其中一根火柴，最多可连续操作 3 次，使 3 根火柴都头向上或者都头向下。请画出火柴倒置问题的状态空间图，并给出所有可能的解和最优解。

提示：用三元组 $S=(x_1, x_2, x_3)$ 表示状态，其中

$$x_i = \begin{cases} 0 & \text{表示火柴 } i \text{ 头向上} \\ 1 & \text{表示火柴 } i \text{ 头向下} \end{cases}$$

4.17 选择状态空间方法中的一个合适的搜索方法求解如图 4.5 所示的三阶梵塔问题。画出搜索树，并给出问题的解。

4.18 若如图 4.5 所示的三阶梵塔问题中，移动 A 盘一次的代价是 1，移动 B 盘一次的代价是 2，移动 C 盘一次的代价是 3。选择状态空间方法中的一个合适的搜索方法求解，画出搜索树，并给出问题的解。

4.19 设有下述推销员旅行问题：一个推销员从城市 A 出发，访问城市 B、C、D 和 E 一次且一次访问一个城市，最后回到城市 A，求解一条旅行路程最短的路线。城市之间的距离如图 2 所示。选择状态空间方法中一个合适的搜索方法求解，画出搜索树，并给出问题的解。

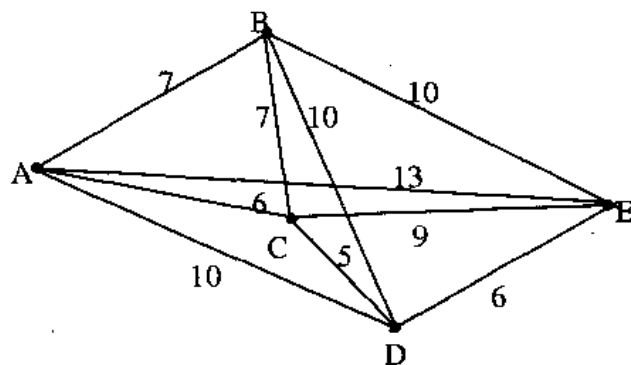


图2 推销员旅行问题

4.20 五个城市之间的交通费用图如图 3 所示，边上的数字是两城市之间的交通费用。若从西安出发，经过每个城市一次且一次，最后到达广州，请找出一条交通费用最少的路线。

选择状态空间方法中的一个适合的搜索方法求解，画出搜索树，并给出问题的解。

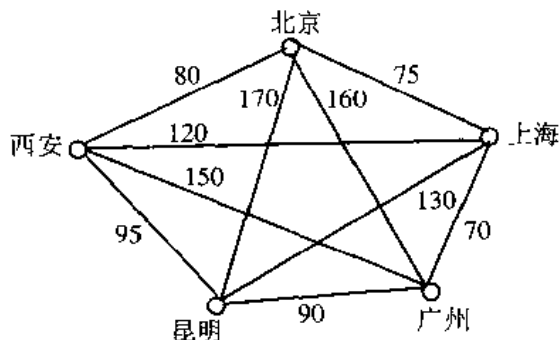


图3 交通费用问题

4.21 设有下述传教士与野人问题：有 3 名传教士和 3 名野人来到一条河的右岸，欲乘一条船渡河到左岸，该船的最大负载能力为 2 人，传教士或野人皆可撑船。在任何时候，不论是在右岸还是在左岸，如果野人人数超过传教士人数，那么，野人就会吃掉传教士。请选择状态空间方法中的一个合适的搜索方法规划出一个渡河方案，把 6 个人都安全地渡过河去。画出搜索树，并给出问题的解。

4.22 设有下述农夫过河问题：农夫、狐狸、山羊和白菜都在一条河的左岸，只有农夫可撑船，要将狐狸、山羊和白菜都渡河到右岸去。但是，农夫每次撑船过河时，船上至多只能载狐狸、或者载山羊、或者载白菜。若农夫不在时，则狐狸会吃掉山羊、山羊会吃掉白菜。请选择状态空间方法中的一个合适的搜索方法规划出一个渡河方案，使农夫能把狐狸、山羊和白菜都带过河去。画出搜索树，并给出问题的解。

提示：用 4 元组 $S=(f, x, g, c)$ 表示状态，其中，变元 f, x, g 和 c 分别表示农夫、狐狸、山羊和白菜在状态 S 下所在的河岸。若其值为 0，则表示在左岸；若其值为 1，则表示在右岸。

4.23 设有如图 4 所示的与/或树，其中， $t_1 \sim t_5$ 是终止节点。请分别应用与/或树的宽度优先搜索和与/或树的有界深度优先搜索求出解树。

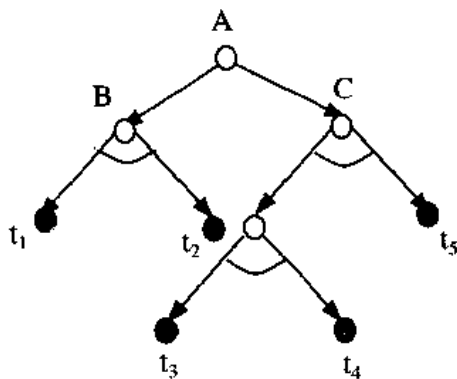


图4 4.23 题与/或树

4.24 设有如图 5 所示的与/或树，其中 $t_1 \sim t_4$ 是终止节点，边上的数字是该边的代价。请分别按和代价法与最大代价法求解树及其解树的代价。

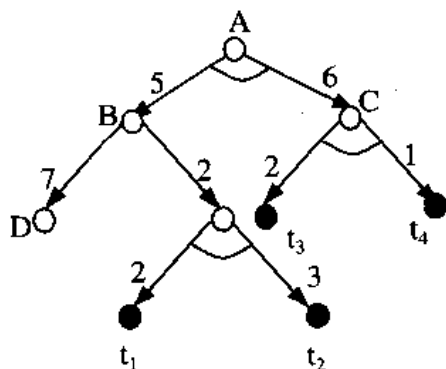


图 5 4.24 题与/或树

4.25 设有如图 6 所示的电路，请用一棵与/或树表示求解该电路的阻抗 Z 的计算问题。已知单独一个电阻 R 、单独一个电感 L 和单独一个电容 C 的阻抗计算问题是本原问题，它们的解分别是： $Z_R=R$ ， $Z_L=j\omega L$ ， $Z_C=1/j\omega C$ 。

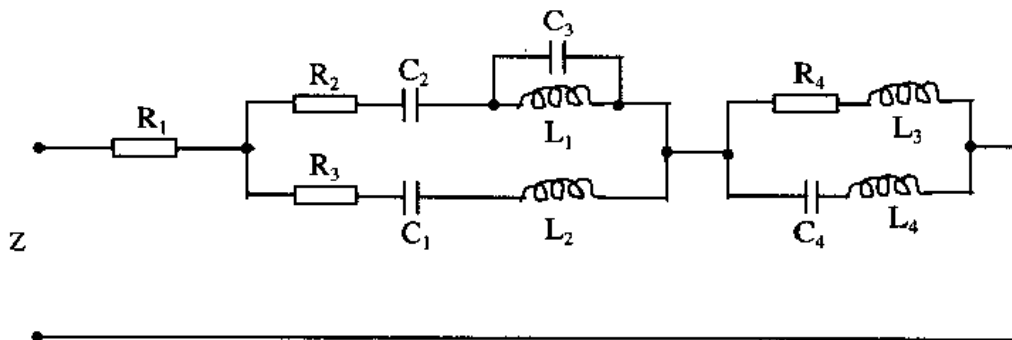


图 6 4.25 题电路图

4.26 请用一棵与 / 或树表示传教士与野人问题的求解。

4.27 设有如图 7 所示的博弈树，其中，给出的数字是相应节点的估值。

- (1) 计算各节点的倒推值。
- (2) 应用 α - β 剪枝技术剪去不必要的分枝。

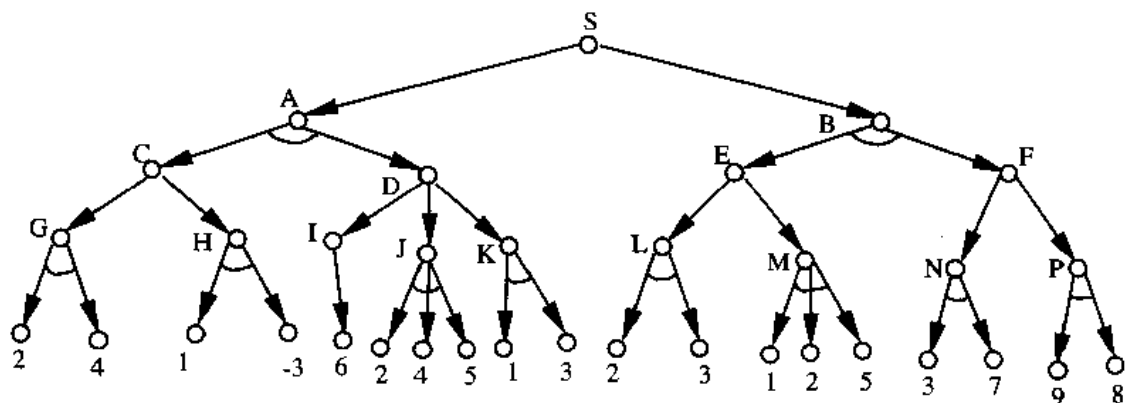


图 7 4.27 题博弈树

4.28 有一堆硬币，开始时有 9 枚。A、B 两人轮流从中取硬币，每次取硬币时，可以取 1 枚或者 2 枚或者 3 枚，拣起最后一枚硬币者为输方。试用博弈树证明：后开始取硬币者总能获胜，或者先开始取硬币者总是会输。

4.29 五子棋棋盘是一个行、列皆有 5 个方格的 5×5 方格棋盘，A、B 两人轮流投子，每次投一子于一个空的方格中，谁先布成五子成一线（行、列、对角线均可），谁就获胜。请定义估价函数，并站在 A 的立场上，找出获胜的最佳走步。

第 5 章 专家系统

专家系统 (ES, Expert System) 是一种模拟人类专家解决领域问题的计算机程序系统。专家系统内部含有大量的某个领域的专家水平的知识与经验, 能够运用人类专家的知识和解决问题的方法进行推理和判断, 模拟人类专家的决策过程, 来解决该领域的复杂问题。

5.1 专家系统概述

专家系统是人工智能应用研究最活跃和最广泛的应用领域之一。自从 1965 年第一个专家系统 DENDRAL 在美国斯坦福大学问世以来, 各种专家系统已遍布各个专业领域, 取得很大的成功。

5.1.1 专家系统的类型与特点

1. 专家系统的类型

(1) 按任务类型的分类

1983 年, Hayes-Roth 等人根据专家系统处理的问题的类型, 把专家系统分为以下 10 种类型。

1) 解释型。解释型专家系统的任务是通过对于已知信息和数据的分析, 解释这些信息和数据的实际含义。典型的解释型专家系统有信号理解和化学结构解释等专家系统。例如, 由质谱仪数据解释化合物分子结构的 DENDRAL 系统、语言理解系统 HEARSAY, 由声纳信号识别舰船的 HASP/SIAP 系统等。都是对于给定数据, 找出与之相一致的、符合客观规律的解释。

2) 诊断型。诊断型专家系统的任务是根据输入信息找出处理对象中存在的故障。主要有医疗、机械和电子等领域中的各种诊断专家系统。例如, 血液凝结疾病诊断系统 CLOT、计算机硬件故障诊断系统 DART、化学处理工厂故障诊断系统 FALCON 等, 都是通过处理对象内部各部件的功能及其相互之间的关系, 来检测和查找可能的故障。

3) 调试型。调试型专家系统的任务是给出已确认故障的排除方案。主要有电子设备和机械设备的计算机辅助调试专家系统。例如, VAX/VMS 计算机系统的辅助调试系统 TIMM/TUNER、石油钻探机械故障的诊断与排除系统 Drilling Advisor 等, 都是根据处理对象和故障的特点, 从多种纠错方案中选择最佳方案。

4) 维修型。维修型专家系统的任务是制定并实施纠正某类故障的规划。例如, 计算机网络的维护专家系统、电话电缆维护专家系统 ACE、诊断和排除内燃机故障的 DELTA 系统等, 都是根据对象系统的故障和纠错方法的特点, 制定出合理的故障维修规划。

5) 教育型。教育型专家系统的任务是根据学生的学习特点,把需要学习的知识以适当的教学方法和教案组织起来,用于对学生进行教学和辅导、诊断和处理学生学习过程中的错误。例如, GUIDON 和 STEAMER 等专家系统、可进行逻辑学与集合论教学的 EXCHECK 教学系统以及一些计算机辅助教学(CAI)系统和聋哑人语言训练系统等。

6) 预测型。预测型专家系统的任务是根据处理对象过去和现在的情况分析及推测未来的演变和发展。典型的应用有天气预报、财政预测、经济发展预测、人口预测、交通预测等。例如,各种产品市场预测专家系统、气象预报系统、军事冲突预测系统 I&W 等,都是进行与时序有关的推理,处理随时间变化的数据和按时间顺序发生的事件。

7) 规划型。规划型专家系统的任务是找出某个能够达到给定目标的动作序列或步骤。规划型专家系统可用于机器人动作规划、交通运输调度、工程项目论证与规划、通信与军事指挥以及生产作业规划等。比较典型的规划型专家系统有 ROPES 机器人运动规划专家系统、制定最佳行车路线的 CARG 系统、安排宇航员在空间站中活动的 KNEECAP 系统、分子遗传学实验设计专家系统 MOLGEN 等,都是在一定的约束条件下,以较小的代价达到给定目标。

8) 设计型。设计型专家系统的任务是根据给定的要求形成所需要的方案或图形描述。典型的应用有电路设计和机械设计。例如, VAX 计算机的总体结构和配置系统 XCON、超大规模集成电路辅助设计系统 KBVLSI、自动程序设计系统 PSI 等,都是在给定要求的限制下,提供最佳或较佳的设计方案。另外,花布图案设计和花布印染专家系统、各种机械零件设计及加工工艺设计专家系统等,都属于设计型专家系统。

9) 监测型。监测型专家系统主要用于完成实时监测任务,对系统、对象或过程的行为进行不间断监测,把监测到的行为与其应当具有的行为进行比较,以发现异常情况,发出警报。典型的应用有空中交通管制监测和核电站安全监测等。例如,航空母舰周围空中交通管制系统 AIRPLAN、核反应堆事故诊断与处理系统 REACTOR、高危病人监护系统 VM 等,都是随时收集有关处理对象的各种数据,并把这些数据与预期的数据比较,一旦发现异常就立即发出报警信号。这类系统通常是诊断型、预测型和调试型的合成。

10) 控制型。控制型专家系统的任务是自适应地管理一个受控对象或客体的全部行为,使之满足预期要求,通常用于实时控制型任务。典型的应用有战场辅助作战指挥系统、汽车变速箱控制系统、生产过程控制和空中交通管制等。例如,维持钻机最佳钻探流特征的 MUD 系统、MVS 操作系统的监督控制系统 YES/MVS 等。这类系统通常是监测型和维修型的合成。

实际上,上述 10 种任务类型之间往往互相关联,有些专家系统通常要求完成具有几种类型的任务。例如, MYCIN 就是一个诊断型和调试型的专家系统。

(2) 分类问题与构造问题的专家系统

1985 年, Clancy 指出,无论专家系统完成什么类型的任务,就领域问题的基本操作来说,专家系统求解的问题可分为分类问题和构造问题两类。求解分类问题的专家系统称为分析型专家系统,广泛用于解释、诊断和调试等类型的任务;求解构造问题的专家系统称为设计型专家系统,广泛用于规划、设计等类型的任务。

1) 分类问题与分析型专家系统。

至今为止,大部分专家系统都是分析型专家系统,求解的问题都是分类问题。对分类问题求解的基本操作称为解释操作。当给出输入数据和相应的输出数据时,要求给出对象系统是否异常及异常的原因。解释操作主要是识别操作,即识别出是哪个对象系统的输入输出。识别操作又可进一步分解为判别对象系统是否异常的监督操作和确定异常原因的诊断操作。当给出输入数据和具体的对象系统时,要求解释什么样的输出是所期望的,解释操作就是预测操作。当给出具体的对象系统及其输出时,求解的问题就是决定所需要的输入,解释操作就是控制操作。

分类问题的一个重要特征是求解的结论都限定在一个预先规定的假设集之中。因此,分析型专家系统进行问题求解的基本特点是:根据获得的各种证据,从预先规定的假设集中选择一个或多个可能的假设作为分类问题的解。因而,分析型专家系统的知识库由数据(证据)集、假设(解)集和将数据与假设联系起来的启发式知识三部分组成,它们的可能组合构成状态空间或问题空间,搜索求解通常就在这一限定空间中进行。

分析型专家系统的主要推理方法是启发式推理分类方法。首先把原始数据或证据经过数据抽象变成形式化的抽象数据,然后经过对抽象数据与抽象解之间的启发式匹配找出可匹配的抽象解集,最后经过解的求精从解集中识别出具体解。

MYCIN 和 PROSPECTOR 都是典型的分析型专家系统。

2) 构造问题与设计型专家系统。

构造问题是在事先给定的设计要求和约束条件下,考虑各种部件的可能组合或各种可能的动作序列,最终求得满足要求的系统设计方案或行动规划。因此,设计型专家系统的解元是各种部件或动作,解是满足一定约束条件的部件组合或动作序列。它的基本操作是合成所需对象系统的构造操作。

由于可能的各种部件组合或动作序列的数目往往十分庞大,事先无法准备好所有候选的可能的解,因此,设计型专家系统比分析型专家系统一般要复杂得多。常用的解决方法是把构造问题分解为多个阶段的分类问题,或者强化问题的约束条件来限定搜索求解的空间规模。

R1/XCON 是一个典型的构造问题专家系统,它是美国 DEC 公司根据用户订货单配置 VAX 计算机系统时采用的专家系统。配置计算机系统是按用户要求将中央处理器、存储器、各种接插件及控制部件连接到输入/输出总线,并合理地布置在底板上,放入合适的机柜中。R1/XCON 首先检查订货单是否安全;然后应用关于部件之间相互关系的知识建立符合要求的计算机配置,决定各部件的空间位置;最后输出的是符合订货单要求的一组表示各部件之间空间位置关系和连接关系的图表。根据这些图表就可以实际装配出满足用户订货单要求的 VAX 计算机系统。

2. 专家系统的一般特点

各种类型的专家系统都有各自的特点,在总体上,专家系统还具有以下一些共同的特点。

(1) 知识的汇集

一个专家系统汇集了某个领域多位专家的经验 and 知识及他们协作解决重大问题的能力。因此,专家系统应表现出更渊博的知识、更丰富的经验和更强的工作能力,而且能够高效率、准确、迅速和不知疲倦地工作。

(2) 启发性推理

专家系统运用专家的经验 and 知识进行启发式推理,对问题作出判断和决策。

(3) 推理和解释的透明性

用户无需了解推理过程,就能从专家系统获得问题的结论,而且推理过程对用户是透明的。专家系统的解释器可以回答用户关于“系统是怎样得出这一结论”和“为什么会提出这样的问题”之类的询问,专家系统是如何实现这些问题的解释对用户也是透明的。

(4) 知识更新

专家系统能够不断地获取知识,增加新的知识,修改原有知识。机器学习就是系统积累知识以改善其性能的重要方法。

5.1.2 专家系统的结构与开发方法

1. 专家系统的结构

专家系统的结构是指专家系统各组成部分的构造方法和组织形式。选择什么结构最为合适,要根据系统的应用环境和所执行任务的特点来确定。系统结构选择恰当与否,直接关系到专家系统的适用性和效率。

专家系统一般的系统结构框图如图 5.1 所示,其组成部分及其主要功能说明如下。

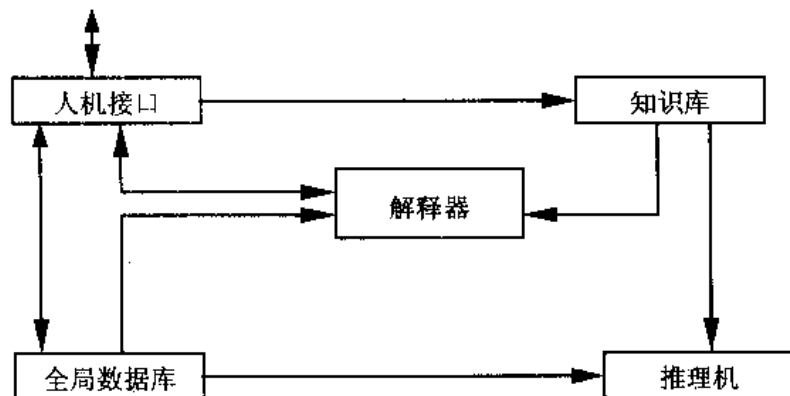


图 5.1 专家系统结构框图

(1) 知识库

知识库 (Knowledge Base) 以某种存储结构存储领域专家的知识,包括事实和可行的操作与规则等。为了建立知识库,首先要解决知识获取与知识表示的问题。知识获取是指知识工程师如何从领域专家那里获得将要纳入知识库的知识。知识表示要解决的问题是如何使用

计算机能够理解的形式来表示和存储知识的问题。

(2) 全局数据库

全局数据库 (Global Database) 亦称为总数据库, 它用于存储求解问题的初始数据和推理过程中得到的中间数据。

(3) 推理机

推理机 (Reasoning Machine) 根据全局数据库的当前内容, 从知识库中选择可匹配的规则, 并通过执行规则来修改数据库中的内容, 再通过不断地推理导出问题的结论。推理机中包含如何从知识库中选择规则的策略和当有多个可用规则时如何消解规则冲突的策略。

(4) 解释器

解释器 (Expositor) 用于向用户解释专家系统的行为, 包括解释“系统是怎样得出这一结论的”、“系统为什么要提出这样的问题来询问用户”等用户需要解释的问题。

(5) 人机接口

人机接口 (Interface) 是系统与用户进行对话的界面。用户通过人机接口输入必要的数
据、提出问题和获得推理结果及系统作出的解释; 系统通过人机接口要求用户回答系统的询问, 回答用户的问题和解释。

由于每个专家系统所需要完成的任务不同, 因此其系统结构也不尽相同。知识库和推理机是专家系统中最基本的模块。知识表示的方法不同, 知识库的结构也就不同。推理机是对知识库中的知识进行操作的, 推理机程序与知识表示的方法及知识库结构是紧密相关的, 不同的知识表示有不同的推理机。

2. 专家系统的开发方法

专家系统的开发是一项综合技术, 一个成功的专家系统的开发需要知识工程师和领域专家的密切配合和坚持不懈地努力。

(1) 建造专家系统的步骤

根据软件工程的生命周期方法, 一个实用专家系统的开发过程可类同一般软件系统开发过程分为认识、概念化、形式化、实现和测试等阶段。

1) 认识阶段。

知识工程师与领域专家合作, 对领域问题进行需求分析。包括认识系统需要处理的问题范围、类型和各种重要特征、预期的效益等, 并确定系统开发所需的资源、人员、经费和进度等。

2) 概念化阶段。

把问题求解所需要的专门知识概念化, 确定概念之间的关系, 并对任务进行划分, 确定求解问题的控制流程和约束条件。

3) 形式化阶段。

把已整理的概念、概念之间的关系和领域专门知识用适合于计算机表示和处理的形式进行描述和表示, 并选择合适的系统结构, 确定数据结构、推理规则和有关控制策略, 建立起

问题求解模型。

4) 实现阶段。

选择适当的程序设计语言或专家系统工具建立可执行的原型系统。

5) 测试阶段

通过运行大量的实例，检测原型系统的正确性及系统性能。通过测试原型系统，对反馈信息进行分析，进而进行必要的修改，包括重新认识问题，建立新的概念或修改概念之间的联系、完善知识表示与组织形式、丰富知识库的内容、改进推理方法等。

专家系统的这一开发过程，类似一般软件系统开发过程的瀑布模型，各阶段目标明确，逐级深化。开发过程的瀑布模型如图 5.2 所示。

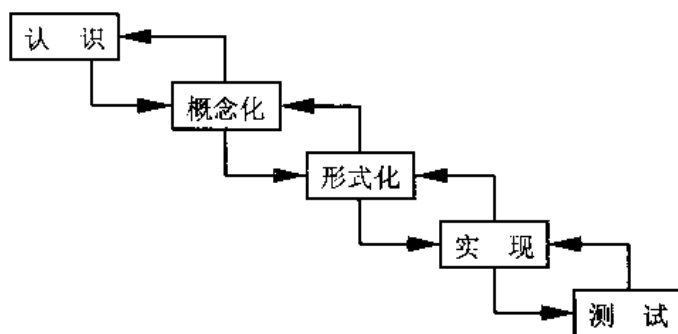


图 5.2 专家系统开发过程的瀑布模型

(2) 原型系统与快速原型法

由于领域专家的知识是长期积累的经验和专门知识，因此，知识工程师不可能在短时间内获得所需要的全部专家知识，并把它们按知识表示方式和知识库的结构要求存入知识库中。也就是说，决定专家系统性能的专门知识是逐步增加和不断完善的。这就需要采用增量式开发方法，即通过对基本功能的逐步扩大来完善系统。专家系统具有需要经常修改和完善的知识库并与相对稳定的推理机相分离的结构特点，从而适应了这种增量式开发方法。增量式开发可能保证对基本功能的有效验证，有利于在整个开发过程中得到一系列功能日趋完善的原型系统。

根据系统的复杂程度和实用性，原型系统一般可分成以下 5 种。

1) 演示原型。

大多数专家系统都开始于一个演示原型，它是仅能解决少量问题的一个演示型系统。演示原型主要有两个作用，一是确信人工智能和专家系统技术能有效地用于所要解决的问题；二是测定问题的定义、范围以及领域知识的表示是否正确。一个典型的基于规则的大型专家系统，其演示原型一般仅有 50~100 条规则，能充分地执行 2~3 个测试实例。

2) 研究原型。

研究原型是能运行多个测试实例的原型系统，这些测试实例能显示领域问题的重要特点。大型专家系统的研究原型一般具有 200~500 条规则。

3) 领域原型。

通过改进研究原型而获得领域原型, 领域原型系统运行可靠, 具有比较流畅和友善的用户接口, 能基本满足用户的需要。大型专家系统的领域原型一般具有 500~1000 条规则, 能很好地执行许多测试实例。

4) 产品原型。

产品原型是已经通过广泛的领域问题测试的原型系统, 并往往用一种效率更高的语言或专家系统工具来实现, 以增加推理的速度和减少存储空间。大型专家系统的产品原型一般具有 500~1500 条规则, 求解领域问题准确快速, 工作可靠。

5) 商品化系统。

商品化系统是已投入商品市场实际销售和运行的系统, 并能适应用户市场的需要。

利用专家系统技术和专家系统的开发工具尽快地建立专家系统的演示原型, 然后进行修改、充实和完善。这也就是专家系统开发的快速原型法。虽然演示原形比较简单, 只能解决少量的领域问题, 也不具备许多辅助功能。但是, 通过演示原型的运行和测试可以实际验证系统方案的可行性和有效性, 检验应用问题的定义范围, 从而在系统设计的最初阶段就能避免较大的原则性错误。而且可以提高领域专家的兴趣和信心, 增强同领域专家的合作。

5.2 LISP 语言

LISP 语言诞生 30 多年来, 最重要的应用领域是人工智能, 尤其是专家系统领域。20 世纪 80 年代, LISP 语言及其应用迅速发展, 如今大约有十多种 LISP 语言。1984 年, 推出的 Common LISP 包括了各种重要的 LISP 语言的优点, 具有丰富的功能, 成为 LISP 语言的事实上的标准语言。在 PC 机上使用的 GCLISP 是 Common LISP 的子集; 在 PC 机上使用的 XLISP 也是 Common LISP 的子集, 并扩展了面向对象的成分; 在 Auto CAD 中的 Auto LISP 还是 Common LISP 的子集。为学习本书提供的产生式系统实例的需要, 本节讲述 Common LISP 基本内容, 涉及的功能和函数在 GCLISP 中也已经实现。

5.2.1 LISP 语言的特点与表达式

LISP 语言与传统的程序设计语言比较, 具有独特的特点。大多数传统的程序设计语言是过程型的, 要求程序员在程序中把问题求解过程的繁琐细节都要描述出来。LISP 语言是一种基于表结构的函数型语言。

1. LISP 语言的特点

LISP 语言具有下述基本特点。

(1) 函数性

函数型语言的基本特点是用函数定义和函数调用构成程序。程序员用函数定义和函数调用组成的表达式来描述求解问题的算法, 表达式的值就是问题的解。用 FORTRAN、PASCAL

和 C 等传统程序设计语言编写的程序是按一定顺序执行的命令序列，执行结果就是问题的解。用这些语言编程时，程序员要规定求解的顺序，即要描述控制流。LISP 语言的编程不考虑程序执行顺序的细节，是更加面向用户的语言。它只确定函数之间的调用，把函数执行的细节交给 LISP 系统来解决，从而减轻了用户编程的负担。

传统的程序设计语言是适应冯·诺依曼型计算机系统结构而发展起来的，LISP 在诺依曼型计算机上运行的效率要低一些。随着计算机系统结构的发展，函数型语言有着广阔的前途。为了适应当前微型机发展水平和程序员使用传统语言编程的习惯，LISP 语言增加了许多非函数型的语言成分，例如 prog、go 等函数，所以，LISP 已不是纯函数型语言，它包含大量的

行快速原型开发方面, LISP 的优势将会得到进一步的发挥。

2. LISP 的符号表达式

LISP 的符号类似其他程序设计语言中的变量, 一个符号名是以字母开头不含规定的特殊字符的字符串, 可以把计算或处理后的结果赋给一个符号。数和符号都称为原子, 它们是 LISP 中不能再分割的对象。若个数或符号用括号括起来就构成一个表, 表中的元素用空格分开。没有元素的表称为空表, 空表可用 () 表示, 也可表示为 nil, 空表也是原子。表和原子是 LISP 的主要对象。

表是可递归定义的, 即可用若干个表来定义另一个表, 或者说, 表中的元素可以是其他的表。表中元素的个数称为表的长度。

原子和表称为 LISP 的符号表达式。LISP 的符号表达式采用前缀表示形式, 即表中第一个元素是函数符号名, 其余的元素是这个函数要求的运算或处理的元素。例如, 符号表达式

(setq y (* 2 3 4))

是一个表, 表中第一个元素是符号名为 setq 的赋值函数名, setq 对 y 赋值 (* 2 3 4)。

(* 2 3 4) 也是一个表, 表中第一个元素是乘法函数名*, 其后的元素是参加乘法运算的数, 因此, (* 2 3 4) 的返回值是 24, (setq y (* 2 3 4)) 的返回值是 y=24。

LISP 中的串是用双引号引起来的字符串, 串中可以包含空格。LISP 中的文件名用串表示。例如, 符号表达式

(load "my.lsp")

也是一个表, 表中第一个元素是装入文件的函数 load, 装入的文件名用一个串表示。

5.2.2 LISP 语言的基本函数

LISP 提供近 400 个基本函数, 我们不能全部介绍, 只能根据需要, 分类说明部分基本函数的功能和使用方法。

1. 数值运算函数

数值运算函数对指定的数进行运算, 函数返回值是数值运算的结果数值。我们对函数功能进行说明时, 用一个箭头 “→” 表示其后是该函数的返回值。

(1) 算术运算函数

算术运算函数有: 加函数+、减函数-、乘函数*、除函数/、加 1 函数 1+、减 1 函数 1-等。+、-、*、/ 等函数可对多个数或已经赋值的符号进行数值运算, 例如

(setq a 10)

(/ a 2 2)

返回值为 2.5。

1+和 1-函数对指定的一个变量加 1 或减 1。

(2) 超越函数

主要的超越函数有:

1) $(\text{exp } n) \rightarrow e^n$

2) $(\text{expt } b \ n) \rightarrow b^n$

3) $(\log \ a \ b) \rightarrow \log_b a$

若缺省 b , 则返回值为 $\log_{10} a$

4) $(\text{sqrt } a) \rightarrow \sqrt{a}$

5) $(\text{abs } n) \rightarrow |n|$

6) $(\text{signum } n) \rightarrow -1, 0, 1$

若 $n < 0$, 则返回值为 -1 ; 若 $n = 0$, 则返回值为 0 ; 若 $n > 0$, 则返回值为 1 。

另外, 还有一组三角函数。

(3) 数的逻辑运算函数

数的逻辑运算函数可把指定的多个十进制整数先转换成二进制数, 然后把这多个二进制数的对应位进行逻辑运算, 把运算结果再转换成十进制整数作为函数返回值。这样的逻辑运算函数有: 逻辑或运算函数 logior 、逻辑异或运算函数 logxor 、逻辑与运算函数 logand 、逻辑非运算函数 lognot 等。例如

$(\text{logior } 25 \ 30) \rightarrow 31$

$(\text{logxor } 25 \ 15) \rightarrow 22$

$(\text{logand } 30 \ 42) \rightarrow 10$

$(\text{lognot } 12) \rightarrow -13$

另外, 还可将二进制数左移或右移若干位后, 再转换成十进制数的移位逻辑运算函数等。

2. 求值与赋值函数

(1) 禁止求值函数

LISP 用符号表达式进行求值。符号表达式是一个表, LISP 把表中第一个元素解释为一个函数, 把其他元素解释这个函数的运算数, 符号表达式的返回值是这个函数的运算结果。但是, 表还有另一种情况, 表中的元素都是数, 即不希望把表中第一个元素解释为一个函数。为此, LISP 提供一个禁止求值函数 quote 对指定的表说明表中元素都是数。例如

$(\text{quote } (a \ b \ c))$

的返回值是表 $(a \ b \ c)$ 。 quote 函数的缩写为 “ $'$ ”, LISP 执行符号表达式 $(\text{setq } x' (a \ b \ c))$ 的结果是对 x 赋值为一个表 $(a \ b \ c)$ 。但是, LISP 对符号表达式 $(\text{setq } x (a \ b \ c))$ 会发出出错信息, 因为 LISP 会把 a 解释为一个函数的符号名, 由于 a 不是一个已定义的函数, 因此, LISP 认为这是一个非法的符号表达式。

(2) 赋值函数

赋值函数 setq 用于对变元赋值, 对一个变元赋的值可以是一个数、一个符号表达式、一个表或者另一个变元。 setq 函数可以对多个变元依序赋值, 例如

$(\text{setq } x' (1 \ 2) \ y \ x)$

setq 先对变元 x 赋值为 $(1 \ 2)$, 再对变元 y 赋值为 x , 使变元 y 的值也为 $(1 \ 2)$ 。

(3) 求值函数

LISP 有多个求值函数。函数 `values` 返回其后各变元的值，函数 `values-list` 返回其后一个表的各元素。例如

```
(values (* 2 3) (+ 4 5)) → 6 9
(values '(a b) '(a b)) → (a b) (c d)
(values-list '(a b c)) → a b c
```

3. 表处理函数

LISP 的基本数据结构是表，LISP 语言最善长于处理的也是表。为此，LISP 提供了若干表处理函数，包括提取表中部分内容的函数、构造表的函数、表结构变换的函数和其他表处理函数。

(1) 取表部分内容的函数

取表部分内容的函数取出一个表中由函数指定的部分内容。

1) `car` 函数取表的第一个元素，例如

```
(car '(a b c)) → a
```

2) `cdr` 函数取表中除掉第一个元素的余下表，例如

```
(cdr '(a b c)) → (b c)
```

3) 函数 `car` 和 `cdr` 可对一个表连续作用，例如

```
(car (cdr (cdr (cdr '(a b c d e f)))))
```

可表示为

```
(caddr '(a b c d e f))
```

`caddr` 也是 LISP 函数，该例的返回值为 `d`。LISP 提供 `car` 和 `cdr` 连续作用函数，但连续作用的次数不超过 4 次，即这一组函数中，任何一个函数名中的 `a` 和 `d` 的个数总共不超过 4 个。

4) `first` 函数等同于 `car` 函数。

5) `second` 函数等同于 `cadr` 函数。

6) `third` 函数等同于 `caddr` 函数。

7) `rest` 函数等同于 `cdr` 函数。

8) `nth` 函数返回表中指定的那一个元素，例如

```
(nth 3 '(a b c d)) → d
```

注意 `a` 为第 0 个元素。

(2) 构造表的函数

1) `cons` 函数把指定的两个元素构造成一个表，如果第 2 个元素是一个表，则把第 1 个元素加到第 2 个元素的表头。例如

```
(cons 'a '(b c)) → (a b c)
(cons '(a b) '(c d)) → ((a b) c d)
```

2) `list` 函数把指定的多个元素按顺序构造成一个表。例如

$(\text{list } 'a \ 'b \ 'c \ 'd) \rightarrow (a \ b \ c \ d)$

$(\text{list } '(a \ b) \ '(c \ d)) \rightarrow ((a \ b) \ (c \ d))$

3) **append** 函数把指定的多个表拼接成一个表。例如

$(\text{append } '(a \ b) \ '(c \ d) \ '(e \ f)) \rightarrow (a \ b \ c \ d \ e \ f)$

(3) 其他表函数

1) **list-length** 函数返回指定的一个表的元素个数。例如

$(\text{list-length } '(a \ (b \ c))) \rightarrow 2$

2) **member** 函数表达式为

$(\text{member item list})$

如果 item 是表 list 中的一个元素, 则 member 返回 list 中从元素 item 开始的余下表; 否则, 返回空表 $()$, 也即是返回 nil。例如

$(\text{member } 'b \ '(a \ b \ c \ d)) \rightarrow (b \ c \ d)$

$(\text{member } '(a \ b) \ '(a \ b \ c \ d)) \rightarrow ()$

$(\text{member } '(b \ c) \ '(a \ (b \ c) \ d)) \rightarrow ((b \ c) \ d)$

4. 逻辑函数

逻辑函数的返回值只能是真 (t) 或假 (nil)。逻辑函数用于判断某种情况是否发生或用于判断函数各变元的逻辑值。

(1) 数据类型判断函数

1) **atom** 函数判断其后的对象是否是原子, 若是一个原子, 则返回 t; 否则, 返回 nil。

例如

$(\text{atom } ()) \rightarrow t$

空表 $()$ 是一个原子。

2) **listp** 函数判断其后的对象是否是一个表, 若是一个表 (包括空表), 则返回 t; 否则, 返回 nil。

3) **null** 函数判断其后的对象是否是一个空表, 若是一个空表, 则返回 t; 否则, 返回 nil。

4) **stringp** 函数判断其后的对象是否是一个串。若是一个串, 则返回 t; 否则, 返回 nil。

5) **characterp** 函数判断其后的对象是否是字符, 若是字符, 则返回 t; 否则, 返回 nil。

6) **symbolp** 函数判断其后的对象是否是符号, 若是符号, 则返回 t; 否则, 返回 nil。例如

$(\text{symbolp } (a \ b)) \rightarrow t$

$(\text{symbolp } '(a \ b)) \rightarrow \text{nil}$

其中, $(a \ b)$ 是一个符号表达式, a 表示一个已定义的函数。' $(a \ b)$ 则是一个表。

7) **numberp** 函数判断其后的对象是否是一个数, 若是一个数, 则返回 t; 否则, 返回 nil。

另外, 还有正数判断函数 **minusp**、负数判断函数 **plusp**、零判断函数 **zerop**、整数判断函数 **integerp**、偶数判断函数 **evenp**、奇数判断函数 **oddp** 等。

(2) 数的比较函数

数的比较函数用于比较两个数的大小，其包含有大于比较函数 $>$ 、小于比较函数 $<$ 、大

cond 函数的表达式为

```
(cond (exp11 exp12 ...)
      (exp21 exp22 ...)
      :
      (expn1 expn2 ...))
```

cond 函数顺序对 n 个表进行处理, 一个表 $(exp_{i1} \ exp_{i2} \ \dots)$ 称为一个 cond 分句。每个分句中的第一个元素 exp_{i1} 是这个分句的测试条件, 其后各元素是符号表达式。

LISP 顺序计算各分句的测试条件 exp_{i1} , 若 exp_{i1} 的值为非 nil, 则计算这个分句各表达式的值, 且以最后一个表达式的值作为 cond 函数的返回值, 不再计算其后的分句; 若所有分句的测试条件均为 nil, 则 cond 函数返回 nil。一种特殊情况是: 若某个 cond 分句中只有测试条件 exp_{i1} 且 exp_{i1} 的值为非 nil, 则 cond 函数返回这个 exp_{i1} 的非 nil 值。

6. 自定义函数与无名函数

(1) 自定义函数 defun

用户可以用自定义的函数 defun 来自行定义所需要的新函数。自定义函数的表达式为

```
(defun name lambda-list {form}*)
```

其中, name 是新函数的函数名, lambda-list 是新函数 name 的变量表, {form}*是由多个表达式组成的新函数 name 的定义体。

例如, 我们用自定义函数 defun 来定义比较函数 compare, 它比较两个数 x 和 y , 若 $x=y$, 则返回串 “numbers are same”; 若 $x>y$, 则返回串 “first is bigger”; 若 $x<y$, 则返回串 “first is smaller”。可如下定义函数 compare:

```
(defun compare (x y)
  (cond ((= x y) "numbers are same")
        ((> x y) "first is bigger")
        (t "first is smaller")))
```

一个新的函数被定义后就可如同 LISP 的基本函数一样被直接调用。例如

```
(compare (3 5)) → "first is smaller"
```

(2) 无名函数 lambda

使用 defun 函数来定义一个函数时, 要给出函数名、变量表和定义体, LISP 要将其保存起来, 以便可用函数名来调用这个函数。若某个新函数定义后只使用一次, 那么, 用 defun 来定义这个新函数就是一种浪费。可以使用无名函数 lambda 来定义并同时执行这个新函数, 定义时无需给出新函数的函数名, 执行后不被保存, 也不能被调用。

lambda 函数的表达式为:

```
(lambda lambda-list {form}* list)
```

其中, 实参表 list 中元素的个数应与变量表 lambda-list 中的形参个数一致。lambda 函数的返回值为用实参表中的实参代替定义体 {form}* 中相应形参后的函数计算值。例如


```
(lambda (x) (* x x x) 3) → 27
(lambda (x y) (+ x y) '(1 3)) → 4
```

7. 高阶函数

高阶函数是以函数为变量而定义的一种函数。若 $g(f(x,y))$ 中 $f(x,y)$ 是函数, 那么, g 称为高阶函数。例如可以定义广义求和函数 $g = \sum_{i=m}^n f(i) = f(m) + f(m+1) + \dots + f(n)$, f 是一个任意函数。在求和函数 g 被定义后, 可对任意指定函数 f 调用求和函数 g , 从而得出对 f 的求和。或者说, 指定一个函数名实参, 就可得到这个函数的高阶函数 g 的值。

例如, 我们定义平方和函数 `sum-squares` 与立方和函数 `sum-cubes`, 它们分别计算从整数 m 到 n 的平方和与立方和。

```
(defun sqare (m)
  (* m m))

(defun cube (m)
  (* m m m))

(defun sum-squares (m n)
  (if (< m n)
      (+ (sqare m) (sum-squares (1+ m) n)))
      0))

(defun sum-cubes (m n)
  (if (< m n)
      (+ (cube m) (sum-cubes (1+ m) n)))
      0))
```

两个求和函数的定义基本相同, 只是 `sum-squares` 调用 `sqare` 函数来计算 m^2 , `sum-cubes` 调用 `cube` 函数来计算 m^3 。因此, 可以采用高阶函数的形式来定义一个广义求和函数 `sum-item`。调用 `sum-item` 时, 若指定其函数名形参为 `sqare` 就得出平方和; 若指定函数名形参为 `cube` 就得出立方和。另外, 我们注意到上述两个求和函数的定义中, 都使用了递归调用的方式, 即一个函数调用这个函数本身。

高阶函数可使用函数 `apply` 来定义。`apply` 函数的表达式为

```
(apply function arg)
```

`apply` 的第一个变量 `function` 是一个函数名或者是一个广义的函数名符号, `apply` 的第二个变量 `arg` 是函数 `function` 的变量。如果 `function` 有多个变量 `{arg}*`, 那么最后一个变量应是一个表。

我们使用 `apply` 函数来定义广义求和函数 `sum-item` 为:

```
(defun sum-item (func m n)
  (if (< m n)
      (+ (apply func (list m))
         (sum-item (func (1+ m) n))))
      0))
```

定义了广义求和函数 `sum-item` 后, 调用 `sum-item` 时, 用已定义的函数的函数名作为实

参替换 sum-item 中广义函数名符号 func, 就可得出求和值。例如

`(sum-item (square 2 3))` $\rightarrow 2^2+3^2=13$

`(sum-item (cube 2 3))` $\rightarrow 2^3+3^3=35$

可以直接使用 apply 函数表达式来计算一个高阶函数的返回值, 在 apply 函数表达式中, function 就是一个已定义的函数名, 且在 function 前要加上 “'”, 表示 apply 不直接对 function 求值, 而 function 只是 apply 的一个变量。例如

`(apply 'cons '(a (b c)))` $\rightarrow (a\ b\ c)$

`(apply '+ '(1 2))` $\rightarrow 3$

5.2.3 迭代与递归

循环程序是一种重要的程序结构, 循环程序可采用迭代和递归两种方法来实现。LISP 提供若干迭代函数来定义迭代方式的函数, 也可以按递归的方法用自定义函数 defun 来定义递归方式的函数。

1. 迭代函数

LISP 提供的迭代函数可分为结构迭代函数、非结构迭代函数和映射函数三类。

(1) 结构迭代函数 do

结构迭代是指迭代过程中没有 goto 之类的无条件转移, 程序的结构性好, 是结构化程序设计方法所提倡的。LISP 的结构迭代函数以函数 do 为代表。

do 函数的表达式为:

```
(do ((var1 init1 upd1)
    (var2 init2 upd2)
    :
    )
    (condition action1 action2 ...)
    {form}*)
```

其中, var1, var2, ... 是变量名, init1, init2, ... 是各变量赋给的初值, 若某个变量的初值没有给出, 则该变量的初值为 nil。upd1, upd2, ... 是每次循环后对相应变量的值进行更新的表达式, 若某个更新表达式没有给出, 则相应变量的值由循环体来更新。(condition action1 action2 ...) 是循环是否结束的判断子句, 若 condition 的值为非 nil, 则循环结束, 并对 action1, action2, ... 依序求值, 把最后一个表达式 action 的值作为 do 函数的返回值; 否则, 执行循环体 {form}*。

例如, 我们可用结构迭代方式来定义计算阶乘 $n!$ 的函数 factorial (n) 为:

```
(defun factorial (n)
  (do ((i 1 (1+ i))
      (result 1)
      )
      ((> i n) result))
```

```
(setq result (* i result)) ) )
```

调用时, 用实参代替函数 factorial 的形参 n。例如

```
(factorial 4) → 24
```

(2) 非结构迭代函数

LISP 提供函数 prog、函数 go 和函数 return 用于定义非结构迭代函数。

1) prog 函数的表达式为

```
(prog ( (var1 init1) (var2 init2) ...)
      exp1 exp2 ...)
```

其中, var1, var2, ... 是变量名, init1, init2, ... 是各变量赋给的初值, 若某个变量的初值没有给出, 则该变量的初值为 nil。在表达式序列 exp1, exp2, ... 中可以有多个表达式含有 go 函数表达式, 至少有一个表达式含有 return 函数表达式。若没有 go 和 return, 则对各表达式顺序求值后退出 prog, 且函数 prog 的返回值为 nil。

2) go 函数的表达式为

```
(go symbol)
```

执行 go 函数时, LISP 在 prog 中找名为 symbol 的表达式 (原子), 若未找到, 则 prog 函数返回出错信息; 若找到, 即 prog 函数转到名为 symbol 的原子之后的表达式继续执行。

3) return 函数的表达式为

```
(return form)
```

执行 return 函数时, 对 form 求值后退出 prog, 且函数 prog 的返回值为 form 的值。

在 prog 函数中, 可以出现多个 go 和 return 的函数表达式, prog 函数的返回值将依赖于具体执行情况。

(3) 映射函数

映射函数是一种隐式的迭代控制方式, LISP 提供一组映射函数的函数名以 map 开头, 它们都以某种方式处理表的每一个元素。

1) mapcar 函数的表达式为

```
(mapcar function {list}*)
```

其中, function 是一个函数名, {list}* 是一个或多个表, 表的个数是函数 function 要求的变量个数。mapcar 把由函数名 function 指定的函数功能映射到表的各个元素上, 映射的结果为一个表, 并作为 mapcar 函数的返回值。在函数名 function 前需要加上 #' 或 ' , 使 LISP 对 function 不直接求值, 只是把函数名 function 作为一个元素送给映射函数 mapcar。例如

```
(mapcar 'car '(a b) '(c d) '(e f)) → (a c e)
(mapcar 'factorial '(1 2 3 4)) → (1 2 6 24)
```

2) maplist 函数的表达式为

```
(maplist function {list}*)
```

maplist 把由函数名 function 指定的函数功能映射到表 {list}*, 产生的映射结果表作为

maplist 返回表的第 1 个元素, 对 {list}* 先用 cdr 作用后, 再映射作为返回表的第 2 个元素, 对 {list}* 先用 cddr 作用后再映射作为返回表的第 3 个元素, …直至 {list}* 为空表为止。例如

```
(maplist 'append '(a b c) '(1 2 3))
→ ((a b c 1 2 3) (b c 2 3) (c 3))
```

2. 递归

递归是一种重要的程序设计方法和技巧, 采用递归方法设计的递归程序给人以简洁优美的感觉。同样, 在 LISP 中可按递归方法定义递归函数。LISP 没有提供特殊的基本函数来支持递归函数的定义和调用。LISP 对递归函数调用和非递归函数的调用过程的处理是相同的, LISP 在内部用栈的数据结构来保存每层调用的实参和中间结果, 并维护着调用的层次关系。由内层到外层返回的过程按栈中各层内容返回进行。正是依靠 LISP 对程序员透明的这种调用和返回机制, 使程序员无需关心调用和返回的过程细节。

递归函数的表现能力要强于迭代函数, 但这并不意味着在任何情况下都使用递归而不使用迭代。一般而言, 递归应用于对上下文无关或块结构语言的句法分析、语义检查及代码生成, 用于求解递归定义的数学问题, 或者更一般地讲, 它应用于自定义嵌套很深的问题的求解。若要重复地对某些事物做同样事情, 则应选用迭代。比如对表中的每个元素都做某种相同的操作。另外, 一般来讲迭代比递归的效率要高一些, 效率包括时间和空间两个方面。如果更看重效率, 那么应尽可能设计一个迭代的解法。

5.3 知识库与推理机

领域专家解决领域问题的能力主要体现在两个方面: 一是专家拥有大量的知识, 二是专家具有选择知识来解决问题的能力。这种选择知识和应用知识求解问题的过程就称为基于知识的推理。知识库与推理机是专家系统中必不可少的组成部分, 是基于知识的推理的基础和核心。

5.3.1 产生式规则与规则库的存储结构

1. 产生式规则的存储结构

一般产生式规则的前件或后件可能是有限个事实或结论的合取式的析取, 例如, 规则 R 为:

$$(F_1 \wedge F_2 \wedge F_3) \vee (F_4 \wedge F_5) \rightarrow H_1 \vee H_2$$

可等价变换为下述 4 条规则:

$$R_{11}: F_1 \wedge F_2 \wedge F_3 \rightarrow H_1$$

$$R_{12}: F_4 \wedge F_5 \rightarrow H_1$$

$$R_{21}: F_1 \wedge F_2 \wedge F_3 \rightarrow H_2$$

$$R_{22}: F_4 \wedge F_5 \rightarrow H_2$$

在规则库中, 允许有前件不同但后件相同的规则, 例如 R_{11} 和 R_{12} 的后件均为 H_1 , R_{21} 和

R_{22} 的后件均为 H_2 。但是产生式规则的一致性要求规则库中的规则之间满足：凡是后件相同的规则，它们的前件没有包含关系，即一条规则的前件不是另一条规则的前件的子集。例如 R_{11} 与 R_{12} 的前件之间没有包含关系， R_{21} 与 R_{22} 的前件之间也没有包含关系。

可以用一个与/或图表示产生式的规则的事实和结论之间的与或关系。例如，规则 R 的与/或图如图 5.3 所示。

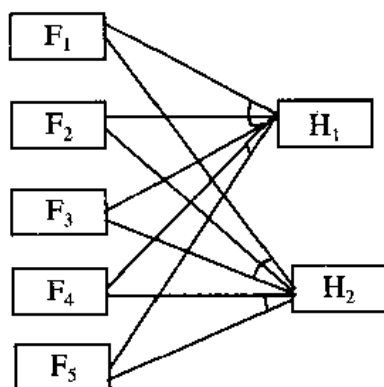


图 5.3 产生式规则与/或图

从产生式规则的形式上看，它与传统程序设计语言中的条件语句很相似，但是，两者之间存在以下根本的区别：

1) 产生式规则具有自含性。所谓自含性有两个方面的含义：一是一条产生式规则仅仅描述了规则前件的条件与后件结论之间的静态关系，因此，规则的正确性可以独立地得到保证；二是一条产生式规则不能直接调用另一条规则，一条规则是否被使用是由综合数据库中的当前内容来决定的。正是由于产生式规则具有自含性，才使得产生式系统的知识表示与知识推理能够分离。

2) 规则之间的控制流不是象传统语言程序中从一条语句向其下一条语句传递，而且满足条件的规则虽然可用但不一定立即执行，这将取决于产生式系统的冲突消解策略。这是传统语言程序与产生式系统行为特征的主要区别。

在 LISP 中，一条产生式规则的存储结构是一个表，通常一条规则存储形式是：

(规则名

(if (条件 1) (条件 2) ... (条件 n))

(then (结论 1) (结论 2) ... (结论 m)))

一条规则的表有 3 个顶层元素，第一个元素是规则名，第二个元素是包括 if 在内的规则前件，第三个元素是包括 then 在内的规则后件。第二个元素和第三个元素又分别是一个表，它们的元素个数分别是 $n+1$ 个和 $m+1$ 个。

2. 规则库的存储结构

在 LISP 中，一个规则库的存储结构是一个分层结构的表。若规则库有 N 条规则，则规

则库表就有 N 个顶层元素, 每个顶层元素是一个规则子表, 每个规则子表有 3 个元素, 分别是规则名、包含 if 在内的规则前件和包含 then 在内的规则后件。一条规则的前件和后件是规则库表的第 3 层子表, 前件子表和后件子表的第 1 个元素分别是 if 和 then, 其余元素分别是规则前件中的多个条件和规则后件中的多个结论。一个规则库的各条规则的条件和结论之间的关系可以用一个与/或图来描述。

把若干条规则写入一个规则库的最简单的方法是使用 setq 函数, 直接把若干条规则组成一个表赋值给一个规则库符号名。我们用例 2.5 中说明的动物识别专家系统为例, 说明如何用 setq 函数直接建立规则库, 并用与/或图来描述这个规则库。

例 5.1 建立动物识别专家系统的规则库, 并用与/或图来描述这个规则库。规则库由 15 条规则组成, 规则名分别是 $rule_1, rule_2, \dots, rule_{15}$, 规则库的符号名为 rules。

用 setq 函数把 15 条规则组成一个表直接赋值给规则库 rules:

```
(setq rules
  (
    (rule1
      (if (animal has hair) )           若动物有毛发 ( $F_1$ )
      (then (animal is mammal) ) )      则动物是哺乳动物 ( $M_1$ )
    (rule2
      (if (animal gives milk) )         若动物有奶 ( $F_2$ )
      (then (animal is mammal) ) )      则动物是哺乳动物 ( $M_1$ )
    (rule3
      (if (animal has feathers) )       若动物有羽毛 ( $F_9$ )
      (then (animal is bird) ) )        则动物是鸟 ( $M_4$ )
    (rule4
      (if (animal flies)                若动物会飞 ( $F_{10}$ )
          (animal lays eggs) )          且生蛋 ( $F_{11}$ )
      (then (animal is bird) ) )        则动物是鸟 ( $M_4$ )
    (rule5
      (if (animal eats meat) )          若动物吃肉 ( $F_3$ )
      (then (animal is carnivore) ) )   则动物是食肉动物 ( $M_2$ )
    (rule6
      (if (animal has pointed teeth)    若动物有犀利牙齿 ( $F_4$ )
          (animal has claws) )          且有爪 ( $F_5$ )
          (animal has forward eyes) )   且眼向前方 ( $F_6$ )
      (then (animal is carnivore) ) )   则动物是食肉动物 ( $M_2$ )
    (rule7
      (if (animal is mammal) )          若动物是哺乳动物 ( $M_1$ )
```

(animal has hoofs))	且有蹄 (F_7)
(then (animal is ungulate)))	则动物是有蹄类动物 (M_3)
(rule8	
(if (animal is mammal)	若动物是哺乳动物 (M_1)
(animal chews cud))	且反刍 (F_8)
(then (animal is ungulate)))	则动物是有蹄类动物 (M_3)
(rule9	
(if (animal is mammal)	若动物是哺乳动物 (M_1)
(animal is carnivore)	且是食肉动物 (M_2)
(animal has tawny color)	且有黄褐色 (F_{12})
(animal has dark spots))	且有暗斑点 (F_{13})
(then (animal is cheetah)))	则动物是豹 (H_1)
(rule10	
(if (animal is mammal)	若动物是哺乳动物 (M_1)
(animal is carnivore)	且是食肉动物 (M_2)
(animal has tawny color)	且有黄褐色 (F_{12})
(animal has black stripes))	且有黑色条纹 (F_{15})
(then (animal is tiger)))	则动物是虎 (H_2)
(rule11	
(if (animal is ungulate)	若动物是有蹄类动物 (M_3)
(animal has long neck)	且有长脖子 (F_{16})
(animal has long legs)	且有长腿 (F_{14})
(animal has dark spots))	且有暗斑点 (F_{13})
(then (animal is giraffe)))	则动物是长颈鹿 (H_3)
(rule12	
(if (animal is ungulate)	若动物是有蹄类动物 (M_3)
(animal has black stripes))	且有黑色条纹 (F_{15})
(then (animal is zebra)))	则动物是斑马 (H_4)
(rule13	
(if (animal is bird)	若动物是鸟 (M_4)
(animal does not fly)	且不会飞 (F_{17})
(animal has long neck)	且有长脖子 (F_{16})
(animal has long legs)	且有长腿 (F_{14})
(animal is black and white))	且有黑白二色 (F_{18})
(then (animal is ostrich)))	则动物是鸵鸟 (H_5)

(rule14

(if (animal is bird)	若动物是鸟 (M_4)
(animal does not fly)	且不会飞 (F_{17})
(animal swims)	且会游泳 (F_{19})
(animal is black and white))	且有黑白二色 (F_{18})
(then (animal is penguin)))	则动物是企鹅 (H_6)

(rule15

(if (animal is bird)	若动物是鸟 (M_4)
(animal flies well))	且善飞 (F_{20})
(then (animal is albatross)))	则动物是信天翁 (H_7)

上述 steq 函数由 LISP 执行后, 把一个表赋给变量 rules, 这个表有 15 个顶层元素, 每一个顶层元素是一条规则, 每条规则都是有 3 个元素的一个表。

在上述规则的说明中, 用 $F_1 \sim F_{20}$ 标记的是初始事实或证据, 用 $M_1 \sim M_4$ 标记的是中间结论, 用 $H_1 \sim H_7$ 标记的是最终结论。用标记表示 15 条规则为:

$R_1: F_1 \rightarrow M_1$
 $R_2: F_2 \rightarrow M_1$
 $R_3: F_9 \rightarrow M_4$
 $R_4: F_{10} \wedge F_{11} \rightarrow M_4$
 $R_5: F_3 \rightarrow M_2$
 $R_6: F_4 \wedge F_5 \wedge F_6 \rightarrow M_2$
 $R_7: F_7 \wedge M_1 \rightarrow M_3$
 $R_8: F_8 \wedge M_1 \rightarrow M_3$
 $R_9: F_{12} \wedge F_{13} \wedge M_1 \wedge M_2 \rightarrow H_1$
 $R_{10}: F_{12} \wedge F_{15} \wedge M_1 \wedge M_2 \rightarrow H_2$
 $R_{11}: F_{13} \wedge F_{14} \wedge F_{16} \wedge M_3 \rightarrow H_3$
 $R_{12}: F_{15} \wedge M_3 \rightarrow H_4$
 $R_{13}: F_{14} \wedge F_{16} \wedge F_{17} \wedge F_{18} \wedge M_4 \rightarrow H_5$
 $R_{14}: F_{17} \wedge F_{18} \wedge F_{19} \wedge M_4 \rightarrow H_6$
 $R_{15}: F_{20} \wedge M_4 \rightarrow H_7$

这个动物识别专家系统的规则库的与/或图如图 5.4 所示。

5.3.2 推理机及其实现

产生式系统有正向推理和反向推理两种基本推理方式。我们首先给出正向推理过程和反向推理过程的基本描述, 然后给出用 LISP 语言编写的一个正向推理机和一个反向推理机。

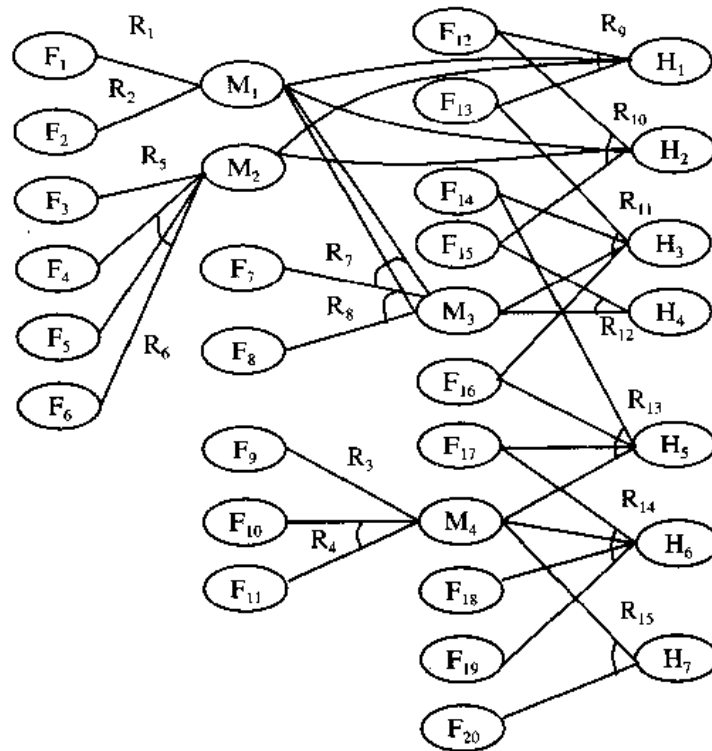


图 5.4 动物识别专家系统规则库与/或图

1. 正向推理过程与反向推理过程

(1) 正向推理过程

正向推理是：根据在综合数据库中给出的已知事实，正向使用规则，即把规则的前件同当前数据库的内容进行匹配来选取可用规则，若有多条规则可用，则按冲突消解策略从中选择一条规则执行，将执行规则的结论添加到综合数据库中，直至问题求解或没有可用规则。

正向推理过程可描述如下：

procedure respond

将规则库中规则的前件同当前数据库内容匹配，若匹配成功，则找到一条可用规则送入可用规则集 S；否则，用下一条规则进行匹配。

while S 非空且问题未求解 **do**

begin

调用 **select-rule (S)**，从 S 中选择一条规则，将该规则的结论添加到数据库中。

调用 **respond**。

end

可见：

1) 正向推理过程 **respond** 是递归的。

2) 若可用规则集 S 中有两条或两条以上的可用规则，则由 **select-rule (S)** 从 S 中选择

一条规则。select-rule (S) 按设计的冲突消解策略来选择规则。

(2) 反向推理过程

反向推理是：根据在综合数据库中给出的假设，反向使用规则，即把规则的后件同当前数据库的内容进行匹配来选取可用规则，若有多条规则可用，则按冲突消解策略从中选择一条规则，将该规则的前件添加到综合数据库中，直至问题求解（假设成立所需要的全部证据和事实在数据库中）或没有可用规则。

反向推理过程可描述如下：

procedure achieve (G)

将规则库中规则的后件同当前数据库内容匹配，若匹配成功。则找到一条可用规则送入可用规则集 S；否则，用下一条规则进行匹配。

if S 为空

then 向用户直接询问假设 G 的真假性，若用户确认 G 为真或假，则问题已求解；否则，把用户回答的有关 G 的证据添加到数据库中。

else

while G 未求解 do

begin

调用 choose-rule (S)，从 S 中选择一条规则 R；

$G' \leftarrow R$ 的前件；

if G' 未求解

then 调用 achieve (G')

if G' 为真

then 把规则 R 的后件添加到数据库中

end

可见：

1) 反向推理过程 achieve (G) 是递归的。

2) choose-rule (S) 按设计的冲突消解策略从可用规则集 S 中选择一条规则。

3) achieve (G) 对目标假设 G 或子目标假设 G' 先由用户直接确认真或假，子目标 G' 是导出目标 G 的规则 R 的前件。即反向推理找出支持假设 G 的条件 G' ，先请用户对 G' 确认来支持 G，若用户不能直接确认 G' 为真，则以 G' 为目标递归调用 achieve (G')。

2. 正向推理机

用 LISP 语言编制的产生式系统中的综合数据库的存储结构也是一个表，我们用 facts 作为综合数据库表的表名。

LISP 语言是函数型语言。因此，用 LISP 语言实现的正向推理机也是一个函数。我们首先给出正向推理机需要调用的几个函数的定义，最后给出实现的正向推理机。

(1) 函数 recall

函数表达式:

```
(recall fact)
```

函数功能: 判断变量 fact 中的一个事实是否在表 facts 中, 若是, recall 返回值是 fact 中的事实; 否则, 返回 nil。

函数定义:

```
(defun (recall fact)
  (cond ((member fact facts) fact)
        (t nil)))
```

(2) 函数 test-if

函数表达式:

```
(test-if rule)
```

函数功能: 判断变量 rule 中的一条规则的前件包含的全部事实是否在表 facts 中, 若是, test-if 返回 t; 否则, 返回 nil。

函数定义:

```
(defun (test-if rule)
  (prog (ifs)
    (setq ifs (cdadr rule))
    loop
    (cond ((null ifs) (return t))
          (recall (car ifs))
          (t (return nil)))
    (setq ifs (cdr ifs))
    (go loop)))
```

(3) 函数 remember

函数表达式:

```
(remember new)
```

函数功能: 判断变量 new 中的一个事实是否在表 facts 中, 若是, remember 返回 nil; 否则, 将 new 中的事实添加到表 facts 的表头, 且 remember 返回 new 中的事实。

函数定义:

```
(defun (remember new)
  (cond ((member new facts) nil)
        (t (setq facts (cons new facts)) new)))
```

(4) 函数 use-then

函数表达式:

(use-then rule)

函数功能：判断变量 rule 中的一条规则的后件包含的全部结论是否在表 facts 中，若全部结论都在 facts 中，则 use-then 返回 nil；否则，将不在 facts 中的结论逐一添加到表 facts 中，且 use-then 返回 t。

函数定义：

```
(defun (use-then rule)
  (prog (thens success)
    (setq thens (caddr rule))
    loop
    (cond ((null thens) (return success))
          ((remember (car thens))
           (print (car rule))
           (print |DEDUCES| (car thens))
           (setq success t))
          (setq thens (cdr thens)))
    (go loop)))
```

在 use-then 的函数定义中 2 次调用了 LISP 输出函数 print，把输出流定向输出到显示终端。第 2 次调用的 print 函数的输出流使用了转义符“|”，两个“|”之间的所有字符都作为普通字符而失去原定义的语法含义。屏幕显示 rule 中规则名，换行显示 DEDUCES 和 thens 中的第一个元素。

(5) 函数 try-rule

函数表达式：

(try-rule rule)

函数功能：判断规则变量 rule 中的一条规则的前件包含的全部事实是否在表 facts 中，若全部事实都在 facts 中，且规则后件有不在 facts 中的结论，则把不在 facts 中的结论逐一添加到表 facts 中，try-rule 返回 t；否则，try-rule 返回 nil。

函数定义：

```
(defun (try-rule rule)
  (and (test-if rule) (use-then rule)))
```

(6) 函数 step-forward

函数表达式：

(step-forward rules)

函数功能：逐次扫描规则库 rules 中的规则，若发现 rules 中有一条可用规则，即该规则的前件包含的全部事实在表 facts 中，则把该规则的后件中不在 facts 中的所有结论添加 facts 中，且 step-forward 返回 t；若 rules 中没有一条可用规则，则 step-forward 返回 nil。

函数定义:

```
(defun (step-forward rules)
  (prog (rule-list)
    (setq rule-list rules)
    loop
    (cond ((null rule-list) (return nil))
          ((try-rule (car rule-list)) (return t)))
    (setq rule-list (cdr rule-list))
    (go loop)))
```

(7) 正向推理机函数 deduce

step-forward 函数只从 rules 中选择一条可用规则, 并使用该规则对 facts 作一次扩充。推理机要完成推理, 通常要对 facts 进行多次扩充, 也就是说, 按扩充了的 facts 继续从 rules 中选择可用规则, 对 facts 再扩充, 直至没有可用规则可选, 不能对 facts 再扩充为止。正向推理机 deduce 可用 step-forward 函数来定义。

函数表达式:

```
(deduce facts)
```

函数功能: 连续不断地从规则库 rules 中选择可用规则, 每选择到一条可用规则, 就把该规则的后件中不在 facts 中的所有结论添加到 facts 中, 对 facts 扩充, 由扩充了的 facts 来选择下一条可用规则对 facts 再次扩充, 直至没有可用规则可选为止。若曾找到一条可用规则对 facts 进行过一次扩充, 则 deduce 返回 t; 否则, deduce 返回 nil。

函数定义:

```
(defun (deduce facts)
  (prog (progress)
    loop
    (cond ((step-forward rules) (setq progress t))
          (t (return progress)))
    (go loop)))
```

例 5.2 对于动物识别专家系统, 若已知的初始事实是 F_{13} 、 F_{12} 、 F_3 和 F_1 , 应用正向推理机 deduce, 说明推理过程和得出的推理结论。

使用 setq 函数把已知的初始事实赋值给事实表 facts:

```
(setq facts
  ((animal has dark spots)
   (animal has tawny color)
   (animal eats meat)
   (animal has hair)))
```

即有 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1)$ 。

调用正向推理机 (`deduce facts`)，使用的规则库 rules 是例 5.1 给出的规则库。推理过程说明如下：

1) 在 rules 中查找规则前件的全部条件在当前 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1)$ 中的可用规则，首先找到规则 R_1 ，则把 R_1 后件中不在 facts 中的结论 M_1 添加到 facts 中，扩充 facts 为 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1 \ M_1)$ 。

实际上，对 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1)$ ，还有另一条可用规则 R_5 ，因为 R_5 的前件 F_3 也在当前 facts 中。但是，由 `step-forward` 函数定义的冲突消解策略是：若有多条可用规则，则按可用规则在规则库表 rules 中的顺序选择第一条可用规则。因此，首先选择 R_1 把 facts 扩充为 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1 \ M_1)$ 。

2) 对当前 facts 在 rules 中查找可用规则，仍然找到规则 R_1 ，但 R_1 的后件结论 M_1 已在 facts 中，因此不会执行规则 R_1 。继续查找可用规则，找到规则 R_5 ，因为 R_5 的后件结论 M_2 不在当前 facts 中，故执行 R_5 ，把 R_5 不在 facts 中的结论 M_2 添加到 facts 中，扩充 facts 为 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1 \ M_1 \ M_2)$ 。

3) 对当前 facts 在 rules 中继续查找可用规则，只有规则 R_9 的前件包含的全部条件 F_{13} 、 F_{12} 、 M_1 和 M_2 在 facts 中，因此 R_9 是可用规则。且 R_9 的后件结论 H_1 不在 facts 中，故而执行 R_9 ，把 R_9 的结论 H_1 扩充到 facts 中，使得 $\text{facts} = (F_{13} \ F_{12} \ F_3 \ F_1 \ M_1 \ M_2 \ H_1)$ 。

4) 对当前 facts ，在 rules 中找不到规则的前件包含的全部条件在 facts 中且后件有不在 facts 中的结论的任何规则，至此，正向推理终止，推理机函数 `deduce` 返回的变量 `progress` 的值为 `t`。

实际上，这个推理过程是按图 5.4 所示的规则库与/或图的一个子图的正向进行推理的。

推理过程终止时，由当前 facts 中的最后一个元素可得出推理的结论是 H_1 ，即 (`animal is cheetah`)。

3. 反向推理机

反向推理是对给定的一个假设 `fact`，判断其是否为真。确定一个假设 `fact` 是否为真的函数为 `verify`，`verify` 函数的定义由下述 4 部分组成：

1) 若 `fact` 已在 facts 中，则 `fact` 为真，函数 `verify` 终止。

2) 若 `fact` 不在 facts 中，则从 rules 中选出所有后件含有 `fact` 的可用规则组成可用规则表 `relevant1`。若 `relevant1` 为空，则直接向用户询问 `fact` 的真假，若用户确认 `fact` 为真，则把 `fact` 添加到 facts 中，函数 `verify` 终止。

3) 若 `relevant1` 不为空，则调用函数 `try-rule` 逐条作用于 `relevant1` 中的可用规则，试看能否直接最终推出 `fact`，若推出 `fact`，则 `fact` 为真，把 `fact` 添加到 facts 中，函数 `verify` 终止。

4) 若不能直接推出 `fact`，则调用函数 `try-rule+` 逐条作用于 `relevant1` 中的可用规则，试看能否间接最终推出 `fact`，若推出 `fact`，则 `fact` 为真，把 `fact` 添加到 facts 中，函数 `verify` 终止。

函数 `try-rule+` 的间接推出与函数 `try-rule` 的直接推出两者不同之处在于：`try-rule` 调用函数 `test-if` 来判断一条规则前件包含的全部事实是否在 facts 中，`test-if` 调用函数 `recall` 来判断

前件中的一个事实是否为真。try-rule+调用函数 test-if+来判断一条规则前件包含的全部事实是否在 facts 中, 而 test-if+是调用函数 verify 来判断前件中的一个事实是否为真。因此, verify 是一个递归函数。

我们首先给出反向推理机需要调用的几个函数的定义, 最后给出实现的反向推理机。

(1) 函数 thenp

函数表达式:

```
(thenp fact rule)
```

函数功能: 判断 fact 是否存在于规则变量 rule 中的一条规则的后件中, 若是, 则函数 thenp 返回规则后件从 fact 开始的余下表; 否则, thenp 返回 nil。

函数定义:

```
(defun (thenp fact rule)
  (member fact (caddr rule)))
```

(2) 函数 inthen

函数表达式:

```
(inthen fact)
```

函数功能: 从规则库表 rules 中选出所有的可用规则组成一个可用规则表, 表中每个元素是一条可用规则。若规则的后件含有 fact, 则这条规则是一条可用规则, 且函数 inthen 返回可用规则表。若没有一条可用规则, 则 inthen 返回空表。

函数定义:

```
(defun (inthen fact)
  (apply 'append
    (mapcar '(lambda (rule)
      (cond ((thenp fact rule) (list rule))
            (t nil)))
      (rules)))))
```

在函数 inthen 的定义中, 调用了映射函数 mapcar, mapcar 把无名函数 lambda 定义的功能映射到规则库表 rules 中的各条规则上去。lambda 函数定义的功能是: 对 rules 中的一条规则 rule 执行 cond 函数定义的功能, 若 fact 在规则 rule 的后件中, 则由 list 函数把规则 rule 形成一个表, 由 lambda 函数返回; 若 fact 不在规则 rule 的后件中, 则 lambda 函数返回一个空表。因此, mapcar 函数从规则库表 rules 中选出多个可用规则, 每个可用规则是一个非空表。函数 inthen 用函数 apply 来定义高阶函数 append。因为 append 的变量是函数 mapcar, 所以定义的函数 append 是一个高阶函数。函数 append 把 mapcar 返回的多个表 (包括返回的空表) 拼接成一个表, 这个表就是可用规则表。当然, 若找不到一条可用规则, 可用规则表就是一个空表。

(3) 函数 test-if+

函数表达式:

```
(test-if+ rule)
```

函数功能: 直接或间接地反向推理确认规则变量 `rule` 中的一条规则的前件包含的所有条件是否都为真, 若是, 则函数 `test-if+` 返回 `t`; 否则, `test-if+` 返回 `nil`。

函数定义:

```
(defun (test-if+ rule)
  (prog (ifs)
    (setq ifs (cdadr rule))
    loop
    (cond ((null ifs) (return t))
          (verify (car ifs))
          (t (return nil))))
    (setq ifs (cdr ifs))
    (go loop)))
```

在函数 `test-if+` 的定义中, 调用了函数 `verify`。函数 `verify` 定义的功能是: 直接或间接地反向推理来确认 `ifs` 中的第一个元素 (规则 `rule` 中的前件包含的一个条件) 是否为真。

(4) 函数 `try-rule+`

函数表达式:

```
(try-rule+ rule)
```

函数功能: 若规则变量 `rule` 中的一条规则的前件包含的所有条件都能直接或间接地反向推理确认为真, 且规则的后件中有不在 `facts` 中的结论, 则把规则后件中不在 `facts` 中的结论添加到 `facts` 中, 函数 `try-rule+` 返回 `t`; 否则, `try-rule+` 返回 `nil`。

函数定义:

```
(defun (try-rule+ rule)
  (and (test-if+ rule) (use-then rule)))
```

(5) 函数 `verify`

函数表达式:

```
(verify fact)
```

函数功能: 直接或间接地反向推理确认变量 `fact` 中的一个假设是否为真; 若为真, 则函数 `verify` 返回 `t`; 否则 `verify` 返回 `nil`。

函数定义:

```
(defun (verify fact)
  (prog (relevant1 relevant2)
    (cond ((recall fact) (return t)))
    (setq relevant1 (inthen fact)))
```



```

(setq relevant2 relevant1 )
( cond  ( (null relevant1)
          ( cond  ( (member fact asked ) (return nil ) )
                  ( (and (print IIS THIS TRUE: fact ) (read ) )
                    (remember fact )
                    ( return t ) )
                  (t (setq asked (cons fact asked ) )
                    ( return nil ) ) ) ) )
loop1
( cond  ( (null relevant1) (go loop2 ) )
        ( (try-rule (car relevant1)) (return t ) ) )
(setq relevant1 (cdr relevant1 ) )
(go loop1 )
loop2
( cond  ( (null relevant2 ) (go exit ) )
        ( (try-rule+ (car relevant2 ) ) (return t ) ) )
(setq relevant2 ( cdr relevant2 ) )
( go loop2 )
exit
( return nil ) ) )

```

在函数 verify 的定义中, 可分为以下 4 部分:

1) 首先调用 recall 函数, 直接确认假设 fact 是否在 facts 中, 若是, 则 fact 为真, 函数 verify 终止并返回 t; 若不是, 则由 (inthen fact) 从 rules 中选出所有的可用规则组成一个表 relevant1, 可用规则是指规则的后件含有 fact, 并生成 relevant1 的一个拷贝 relevant2。

2) 若可用规则表 relevant1 是一个空表, 则由用户直接确认假设 fact 是真或假。用户在屏幕提示 “IS THIS TRUE : fact ” 的询问下, 通过键盘直接确认 fact 是真或假。若用户通过 LISP 输入函数 read 用键盘回答为 t, 则 and 函数返回 t, 执行 (remember fact)。若 fact 不在 facts 中, 则 remember 把 fact 加到 facts 的表头, 且函数 verify 终止并返回 t; 若 fact 在 facts 中, 则 remember 不把 fact 加到 facts 中, 且函数 verify 也终止并返回 t。

若用户通过键盘直接回答为假, 则 and 函数返回 nil, 执行 (setq asked (cons fact asked)), 把 fact 加到表 asked 的表头, 且函数 verify 终止并返回 nil。

3) 若可用规则表 relevant1 不是空表, 则不由用户来直接确认 fact 是真或假, 而由 verify 函数从标号 loop1 开始进行直接反向推理来确认假设 fact 是真或假。

逐条使用 relevant1 中的可用规则, 执行 (try-rule ((car relevant1))), 若规则 (car relevant1) 的前件包含的全部条件在 facts 中, 则把该规则不在 facts 中的后件结论添加到 facts

中, 且函数 `verify` 终止并返回 `t`; 否则, 对 `relevant1` 中的下一条规则执行 `try-rule` 函数功能。只要使用了一条可用规则直接反向推理确认了 `fact` 为真, `verify` 就终止并返回 `t`。

若逐条使用了 `relevant1` 中所有可用规则都不能直接反向推理确认 `fact` 为真, 则转到从标号 `loop2` 开始的间接反向推理。

4) 间接反向推理是通过逐条对 `relevant2` 中所有可用规则调用函数 `try-rule+` 来进行的。如前所述, `try-rule+` 调用函数 `test-if+`, 而 `test-if+` 又调用函数 `verify`。只要使用了一条可用规则间接反向推理确认了假设 `fact` 为真, 函数 `verify` 就终止并返回 `t`; 若逐条使用了所有的可用规则都不能确认 `fact` 为真, 则 `verify` 也终止并返回 `nil`。

(6) 反向推理机函数 `diagnose`

`verify` 函数只能直接和间接地确认一个假设是真或假, 反向推理机需要对用户提出的多个假设确定其真或假。对用户假设表 `hypo` 中设置的多个假设, 反向推理机将使用规则库 `rules` 对多个假设逐一确定其真或假。

函数表达式:

```
(diagnose hypo)
```

函数功能: 逐一确定假设表 `hypo` 中多个假设的真或假, 对确定为真的假设给出屏幕输出说明。

函数定义:

```
(defun (diagnose hypo)
  (prog (poss)
    (setq poss hypo)
    loop
    (cond ((null poss)
      (print "NO HYPOTHESES CAN BE CONFIRMED")
      (return))
      ((verify (car poss))
      (print "HYPOTHESES" (car poss))
      (print "IS TRUE"))
      (setq poss (cdr poss))
      (go loop))))
```

函数 `diagnose` 首先把用户的假设表 `hypo` 生成一个拷贝 `poss`, 若 `poss` 是一个空表, 则屏幕提示: "NO HYPOTHESES CAN BE CONFIRMED", 且 `diagnose` 终止; 若 `poss` 不是空表, 则调用 `verify` 函数逐一确认 `poss` 表中的假设, 对确定为真的假设, 屏幕输出说明: "HYPOTHESES xxxx IS TRUE", 其中, `xxxx` 是这个假设的内容, 直至对 `poss` 中的所有假设都确认完后, `diagnose` 终止。

例 5.3 对于动物识别专家系统, 若用户要求确定的假设是 H_1 , 应用反向推理机 `diagnose`

确定假设 H_1 是否成立, 说明推理过程。

使用 `setq` 函数把假设 H_1 赋给假设表 `hypo` :

```
(setq hypo
      (( animal is cheetah )))
```

即有 `hypo = ( H1 )`。

调用反向推理机 (`diagnose hypo`) , 使用的规则库 `rules` 是例 5.1 给出的规则库。推理过程说明如下:

1) 由 (`diagnose hypo`) 先复制 `poss = (H1)` , 然后调用 (`verify H1`) , 此时, `facts = ( )`。

首先调用 (`recall H1`) , 由于 `fact = H1` , 不在 `facts` 中, 调用 (`inthen H1`) , 从 `rules` 中选出后件含有 H_1 的可用规则只有 R_9 , 故 `relevant1 = relevant2 = ( R9 )`。

由于 `relevant1 = ( R9 ) ≠ ( )` , 调用 (`try-rule R9`) , 因为 R_9 的前件的全部条件不在 `facts` 中, 故对 H_1 进行间接反向推理。

由于 `relevant2 = ( R9 ) ≠ ( )` , 调用 (`try-rule + R9`) 。它首先调用 (`test-if+ R9`) , 把 R_9 的前件包含的全部条件赋给 `ifs` , 即有 `ifs = (F12 F13 M1 M2)` 。在 (`test-if+ R9`) 中, 将顺序调用 (`verify F12`)、(`verify F13`)、(`verify M1`) 和 (`verify M2`)。

2) 调用 (`verify F12`) , 此时, `facts = ( )`。

首先调用 (`recall F12`) , 由于 `fact = F12` 不在 `facts` 中, 调用 (`inthen F12`) , 从 `rules` 中找不到后件含有 F_{12} 的可用规则, 故 `relevant1 = relevant2 = ( )`。

由于 `relevant1 = ( )` , 屏幕出现提示: “IS THIS TRUE: F₁₂”。若用户确认 F_{12} 为真, 则把 F_{12} 加到 `facts` 的表头, 使 `facts = (F12)`。

3) 调用 (`verify F13`) , 此时, `facts = (F12)`。

类似 (`verify F12`) , 若用户在屏幕提示: “IS THIS TRUE: F₁₃” 的询问下, 确认 F_{13} 为真, 则把 F_{13} 加到 `facts` 的表头, 使 `facts = (F13 F12)`。

4) 调用 (`verify M1`) , 此时, `facts = (F13 F12)`。

首先调用 (`recall M1`) , 由于 `fact = M1` 不在 `facts` 中, 调用 (`inthen M1`) , 从 `rules` 中选出后件含有 M_1 的可用规则有 R_1 和 R_2 , 故 `relevant1 = relevant2 = (R1 R2)`。

由于 `relevant1 = (R1 R2)` , 顺序调用 (`try-rule R1`) 和 (`try-rule R2`) , 因为 R_1 的前件条件 F_1 和 R_2 的前件条件 F_2 都不在 `facts` 中, 故对 M_1 进行间接反向推理。

由于 `relevant2 = (R1 R2)` , 顺序调用 (`try-rule+ R1`) 和 (`try-rule+ R2`) 。前者首先调用 (`test-if+ R1`) , 把 R_1 的前件条件赋给 `ifs` , 即有 `ifs = (F1)` 。在 (`test-if+ R1`) 中将调用 (`verify F1`) 。同样, 后者将产生调用 (`verify F2`)。

调用 (`verify F1`) , 类似 (`verify F12`) , 若用户在屏幕提示: “IS THIS TRUE: F₁” 的询问下, 确认 F_1 为真, 则把 F_1 加到 `facts` 的表头, 使 `facts = (F1 F13 F12)`。

调用 (`verify F2`) , 若用户在屏幕提示: “IS THIS TRUE: F₂” 的询问下, 确认 F_2 为真, 则把 F_2 加到 `facts` 的表头, 使 `facts = ( F2 F1 F13 F12 )`。

5) 调用 (verify M_2)，此时， $facts = (F_2 F_1 F_{13} F_{12})$ 。

首先调用 (recall M_2)，由于 $fact = M_2$ 不在 $facts$ 中，调用 (inthen M_2)，从 $rules$ 中选出后件含有 M_2 的可用规则 R_5 和 R_6 ，故 $relevant1 = relevant2 = (R_5 R_6)$ 。

由于 $relevant1 = (R_5 R_6)$ ，顺序调用 (try-rule R_5) 和 (try-rule R_6)，因为 R_5 的前件条件 F_3 与 R_6 的前件包含的全部条件 F_4 、 F_5 和 F_6 都不在 $facts$ 中，故对 M_2 进行间接反向推理。

由于 $relevant2 = (R_5 R_6)$ ，顺序调用 (try-rule+ R_5) 和 (try-rule+ R_6)，前者产生调用 (verify F_3)。后者产生调用 (verify F_4)、(verify F_5) 和 (verify F_6)。在执行这 4 个调用时，若用户在相应屏幕提示下分别确认 F_3 、 F_4 、 F_5 和 F_6 为真，则使 $facts = (F_6 F_5 F_4 F_3 F_2 F_1 F_{13} F_{12})$ 。

至此，调用 (verify H_1) 执行完毕，屏幕输出：“HYPOTHESES H_1 IS TRUE”。由 (setq poss (cdr poss)) 使 poss 为空表，反向推理机 (diagnose hypo) 终止。

实际上，这个推理过程是按图 5.4 所示的规则库与/或图的一个子图的反向进行推理的。

由例 5.3 所见，为了确认用户提出的假设是真或假，反向推理机用规则的后件匹配假设来选择可用规则，然后，对可用规则前件包含的全部条件，逐一向用户询问。对用户不能确认的条件，则以该条件作为中间假设，再选择可用规则。直至支持所有中间假设为真的所有条件都被用户确认为真，才能得出由用户提出的假设是真的判断。反向推理机为用户推断假设是否为真提供了一个以假设为根节点的与/或树，树的所有的叶节点是需要用户确认的初始条件。反向推理机并不能随意确认条件的真或假，它只是引导用户进行反向推理，减少了推理的盲目性。

5.3.3 元知识与元规则

产生式规则是完全自含的，一条规则不能直接调用另一条规则，一条规则是否可用由综合数据库（事实表）的当前内容来决定。这种规则激活机制使产生式系统推理机制具有单纯明确的结构，但是，对于大型产生式系统来说，推理的效率受到严重影响。因此，希望能直接地、显式地表示出激活规则的规则控制知识，用于选择可用规则和激活规则，提高推理效率。这种控制性知识称为元知识，或者更一般地说，元知识是关于知识的知识。

在产生式系统中使用的元知识是由元事实 (Metafact) 和元规则 (Metarule) 构成。元事实用于描述领域知识表示的结构、规则之间的控制约束关系、知识的适用范围等。元规则是用来说明目标规则如何使用的规则，其作用是指导问题求解的推理过程。为了区别这两种不同的推理层次，称使用目标规则进行的推理为目标级推理，它是问题求解实际执行的推理；称使用元规则进行的推理为元级推理，元级推理在目标级推理过程中决定下一步需要执行的规则，从而完成冲突消解，或者改进已有的冲突消解策略，提高目标级推理的效率。

1. 元知识的用途

在专家系统中，知识可以分为以下 4 种主要类型：

- 1) 启发性知识，即经验知识。例如，在动物识别专家系统中，根据经验知识确定的动

物的外形、能力等特征与动物类别（或动物名称）之间的联系规则。

2) 策略性知识, 即控制性知识, 是如何使用知识的知识。例如, 在动物识别专家系统的正向推理机 deduce 中, 从规则库 rules 选择可用规则的顺序是按可用规则在 rules 中的排列次序进行的。这是如何使用目标规则最简单的控制性知识。

3) 结构性知识, 即描述领域对象知识结构的知识。例如, 可以用结构性知识来描述一个领域对象目标规则结构及规则之间关系的结构。

4) 支撑性知识, 即用于论证规则的知识。它给出规则成立的理由、提供者的权威性, 甚至包括一些用于参考的引文。

除第一类知识外, 其他三类知识均属于元知识的范畴。

元知识在产生式系统中有以下主要用途。

(1) 描述事实的完整性约束

事实的完整性约束包括规范约束和语义约束两部分。规范约束元知识用于描述事实的语法约束, 语义约束元知识用于描述事实的一致性。在建立基础事实和增加中间事实时, 利用上述元知识对事实进行检查, 可以维护基础数据的正确性, 保证推理不会因为基础数据的错误导致意外失败。

(2) 描述规则的控制约束

目标规则的自含性使目标规则之间没有调用关系, 规则库与推理机是相互独立的, 由数据库的当前内容和推理机制决定规则执行的次序。有时需要显式地控制规则执行的次序, 就可以利用元知识来描述规则之间的执行次序。我们以医疗诊断专家系统 MYCIN 中的元规则为例来说明。

```
if  1) 感染是骨盆脓肿
    2) 存在前件含有“肠杆菌”的可用规则
    3) 存在前件含有“革兰氏阳性杆菌”的可用规则
then 先使用满足 2) 的规则, 后使用满足 3) 的规则
```

这是一条对目标规则排序使用的元规则。由于肠杆菌感染和骨盆脓肿关系密切, 因此, 当发现患者有骨盆脓肿时, 应首先考虑肠杆菌感染的可能性, 其次考虑革兰氏阳性杆菌感染的可能性。

我们注意到, 如果元规则的形式与目标规则的形式完全相同, 那么元级推理的基本模式也就可以与目标级推理相同。不同的是元规则对目标规则进行推理, 得出目标规则使用的排序关系。目标规则对领域问题进行推理, 得出领域问题的解。

(3) 描述知识的适用范围

对一个专家系统需要使用的知识可以按某些知识特征将其划分为若干适用范围, 利用元知识来描述知识的适用范围, 可以避免知识的盲目使用, 提高推理的效率。例如, 陆地植物生态知识可按海拔高度特征划分若干适用范围。假设有一组知识适用范围是海拔高度 1000~1200 米, 将这一描述知识适用范围的知识用元规则形式描述, 在目标级推理过程中,

用这个元规则检查知识的海拔高度特征，一旦发现推理结果的海拔高度不属于(1000,1200)，就可以中止推理过程。

(4) 描述专家知识的使用环境

在专家系统中，利用元知识来明确地描述知识的使用环境和问题求解的对象，可以避免知识的滥用。

(5) 对规则进行剪枝处理

我们仍以 MYCIN 中的一个元规则为例来说明如何用元规则对目标规则集的剪枝处理。

```
if    1) 培养物不是从无菌部位取得
      2) 目标规则前件含有“当前细菌与以前细菌相同”
then  不使用满足 2) 的目标规则
```

这条元规则剪除了可参加推理的部分目标规则，因为这些目标规则在有关的推理过程中没有有效的作用。

(6) 描述通用问题求解策略

有关通用问题求解策略的元规则可以是域相关的，也可以是域无关的。一条域相关的元规则可以是关于领域的特定控制策略性知识，例如在医疗诊断专家系统中，可以用元规则表示酗酒者或烧伤患者易于受某种特定的细菌感染，以指出在推理中遇到这种情况时首先应考虑的问题。一条相对域无关的元规则可以在产生较大的搜索空间之前，建议系统采用产生相对较小搜索空间的规则。元规则与领域的相关程度与所表示知识的抽象程度有关。例如，下述是一条领域相关的元规则：

```
if    x 是酗酒者
then  先使用与酗酒有关的疾病规则，再使用确定其他疾病的规则
      可以抽象成下述相对域无关的元规则：
if    x 有属性 y, y 是 z 的某种原因，存在与 z 相关的规则
then  先使用与 z 相关的规则，再使用其他可用规则
```

相对域无关的元规则可以应用于较多的领域，域相关的元规则只能在某个特定相关的领域中应用。域相关的元规则可以被看成是相对域无关的元规则的一个特例。

元规则是表示推理控制策略的一种强有力的手段，合理地使用元规则可以大大提高产生式系统的推理速度。但是，对元规则也应该有选择地使用，如果系统需要花太多时间进行元级推理来选择用哪一条目标规则进行目标级推理，反而可能会降低系统推理的效率。

2. 元知识的表示与使用模式

在产生式系统中，元知识一般采用与目标级知识相同的表示形式，并作为一个知识实体与目标级知识共存于知识库中。元规则与目标规则都具有相同产生式规则形式，共存于一个规则库中，这种表示与使用模式的主要优点在于：元级推理与目标级推理可共享一个推理机。

为了保证元规则优先执行，可以为规则增加一个优先数说明模式，在元规则的优先数说明模式中设定较大的优先数，在目标规则的优先数说明模式中设定较小的优先数。这样，当

元规则与目标规则由当前数据库内容确定为可用规则时,将优先执行元规则。在专家系统通用型语言工具 OPS (Official Production System) 和 CLIPS (C Language Integrated Production System) 中,就使元规则与目标规则共存于规则库中,并为规则增加了一个优先数说明模式。

当元级知识与目标级知识明显分离时,即系统分设目标级规则库和元级规则库,系统还将增加一个调度程序。当有多个目标规则可用时,由调度程序根据元级规则与可用目标规则冲突集的匹配情况,从中选择一条可用规则执行。

5.4 解释机制与解释器

在专家系统中,由于知识库与推理机是分离的,系统运行的每一步以及系统激活哪一条规则是由当时的环境信息(综合数据库当前内容、推理控制信息等)和推理机共同决定的,即使采用简单的推理控制策略,动态变化的综合数据库当前内容和多条可用规则也使得程序员难以了解系统的执行行为。因此希望专家系统能够用人们易于理解的方式来解释自身的推理过程。专家系统的解释就是对系统设计者或用户提出的问题给出解释或说明,专家系统的解释器就是专家系统中为完成解释而设置的程序模块。

5.4.1 解释的方法

系统设计者和不同的用户对解释器提出的问题各有侧重,提问题的风格也不相同。一般用户经常询问的是系统得出某个结论的表层原因;一个领域专家更注重系统求解问题的质量;领域的学生更希望了解问题求解的详细过程;知识工程师关心的是问题求解的结论及解释是否与专家的思路吻合,因而更注意审查系统内部执行的流程。

此外,专家系统本身的知识结构与解释机制也有很大的关系,采用适当的知识表示方法可以得到更易表达和更易接受的解释。显然,知识的描述性表示比过程性表示对解释机制实现有效的解释更有利。知识库的层次结构模型不仅有利于提高解释文本的质量,而且便于实现解释的组织和修改。

解释机制可以有以下几种不同的实现方法。

1. 预置文本与路径跟踪法

最简单的解释方法是预置文本,即把问题的解释预先用自然语言或其他易于理解的形式写好,插入程序段或相应的数据库中。在推理过程中或推理之后,一旦用户询问到已有预置解释文本的问题,只需把相应的解释文本填入解释框架,组织成对这个问题的解释提交给用户。对于模糊推理,还需要把模糊语言变量转化为相应的模糊修饰词。

预置文本的解释方法简单直观,知识工程师在编制解释的预置文本时,可以针对不同用户的要求编制出不同的解释文本。其缺点是:对每一个可能的问题都要编制解释预置文本,甚至对一个问题要编制几个解释预置文本,大大增加了系统开发的工作量。大型专家系统的

解释机制采用预置文本法几乎是不能容忍的，当专家系统问题求解的控制知识比较复杂时更是如此。

路径跟踪法是对推理过程进行跟踪，将问题求解所使用的知识自动记录下来。当用户提出需要解释时，解释器向用户显示问题求解路径。路径跟踪法向用户提供 Why 解释和 How 解释，Why 解释用于回答用户关于“系统为什么需要提出这样的问题”的询问，How 解释用于回答用户关于“系统是怎样得出这样的结论”的询问。

实现 Why 解释比较简单，系统从推理过程中确定导致这个问题的规则，将该规则向用户解释即可。实现 How 解释需要从这个结论出发，把导致这个结论的推理链中涉及到的有关规则或知识组织成解释文本，告诉用户是怎样推理得出这个结论的。

医疗诊断专家系统 MYCIN 的解释机制采用预置文本和路径跟踪两种方法实现。MYCIN 对提出的假设反向推理，得出假设成立的一棵证明树。MYCIN 在问题求解的同时，记录了相应的证明树，即记录了问题求解过程中规则的执行轨迹。因此，MYCIN 的解释器只要根据该证明树，并结合预制解释文本，就可实现 Why 解释和 How 解释。但是，MYCIN 的解释机制也存在一些问题，第一，证明树记录了一个问题求解的所有细节，基于证明树的解释将问题求解过程事无巨细向用户全盘托出是不合适的，因为，不同用户的知识水平和关心的问题不同。第二，证明树记录的是问题求解的过程，基于证明树的解释机制便于回答用户是怎样得出这样的结论，但不便于告诉用户为什么需要这样做，即系统缺乏行为合理性的说明。第三，这种解释机制不能说明求解问题的控制信息，因为证明树只记录了目标级知识。

2. 策略解释法

在专家系统许多实际应用中，用户往往不满足于系统简单地告诉他们是怎样一步一步得到问题的结论的，而是要求系统给出求解问题所采用的基本方法和手段。基于认识论和心理学的考虑，为了让用户在更高的层次上把握系统进行问题求解的行为特征，Clancey 等人在他们开发的 NEOMYCIN 系统中提出了一种新的解释方法，这就是策略解释法。

策略解释法向用户解释的是与问题求解策略有关的策略和方法，从策略的抽象表示及其策略使用过程中产生关于问题求解的策略解释。为实现策略解释，NEOMYCIN 在推理控制和知识库的构造上提供了有力的支持。它使用元规则表示策略知识，并与领域知识分离；按分类学的观点对疾病进行分类组织，使得假设的推理空间能够用分类知识显式地表示出来；策略知识、分类知识和把数据与假设联系起来的经验知识被分开编码存储。NEOMYCIN 的问题求解与解释均围绕称为差别表的当前假设集进行，推理控制策略不像 MYCIN 那样采用全局的反向推理方式，而是按照显式表示的元规则所揭示的策略执行。例如，NEOMYCIN 中有下述一条元规则：

Metarule 397:

```
if      There are two items on the differential that differ in some disease process feature
then    Ask a question that differentiates between these two kinds of process
```

这条元规则表示了一种通用的策略，即当差别表中存在两个假设，且存在一个数据可以恰当

地区分它们时，那么向用户寻求相应的数据。

可以说，策略解释法的实质是对表示推理策略的元规则进行的一种路径跟踪法。

3. 自动程序员方法

通过跟踪推理过程中规则的执行而给出的解释仅仅描述了系统的行为过程，没有论证其行为的合理性。行为合理性的解释与行为的背景知识有关，而这些知识一般没有组织到系统中去。行为合理性已不是 Why 解释所能回答的问题，它是在比领域知识和控制知识更高、更抽象的层次上要求系统将设计思想向用户说明的问题。

为了解决这一问题，Swartout 在 XPLAIN 系统的设计中提出了自动程序员解释方法。其基本思想是：在设计一个专家咨询系统中，对领域知识进行描述的同时，把自动程序员程序模块嵌入其中，通过自动程序员把描述性领域知识转化成可执行的程序，并产生有关程序行为的合理性说明，从而向用户提供一个非常有力的解释机制。

XPLAIN 系统解释机制的基本结构如图 5.5 所示。其中，领域模型是一组领域事实的描述，包括各种分类知识和因果关系。领域原理描述了领域问题的求解方法和启发式控制策略。在一般的专家的系统设计中，领域原理往往直接嵌入解释器或者以元知识的形式来存放，而在 XPLAIN 中，领域模型和领域原理都是纯描述性的，不能作为专家系统的知识库直接使用，它们只是为执行程序的生成提供输入源。由领域模型和领域原理到执行程序的转化是由自动程序员（程序生成器）来完成的。求精结构记录了程序生成器生成执行程序的过程中所留下的踪迹，根据求精结构、领域原理和领域模型，解释生成器向用户提供有关目标的结论及解释信息。

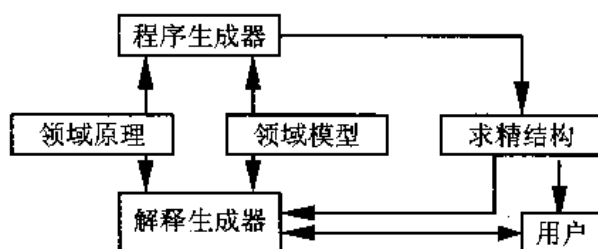


图 5.5 XPLAIN 系统解释机制的基本结构

4. 解释型专家系统

1985 年，Neches 等人在 XPLAIN 系统的研究基础上，提出了一种新的专家系统设计模式，即解释型专家系统（Explainable Expert System）。其基本思想是把专家系统的设计与解释机制的设计进行全盘考虑，从而能给出更合理的解释方案。

解释型专家系统的基本框架如图 5.6 所示。知识库中除包含 XPLAIN 系统已经给出的领域模型和领域原理外，还包含三种其他类型的知识，即优先级控制策略、综合知识和优化知识。另外，在 XPLAIN 系统中，术语定义属于领域模型和领域原理的一部分，在解释型专家系统中，术语定义是分开独立存放在知识库中。其中，优先级控制策略对程序生成器考虑的

一组目标设置优先级；综合知识在程序生成器的运行过程中解决领域原理之间的冲突；优化知识用于改善生成的执行程序的效率。

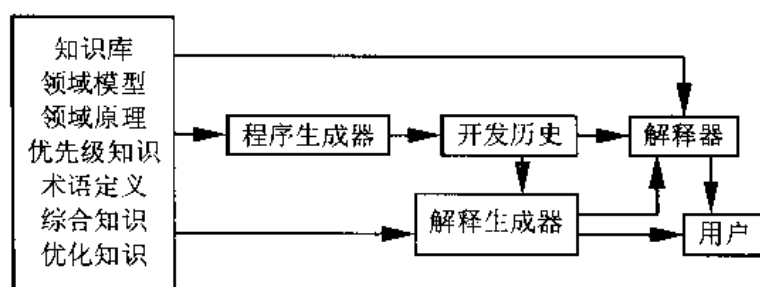


图 5.6 解释型专家系统的基本结构

Neches 设计的解释型专家系统采用语义网络 NIKL 来表示知识，该表示方法是由 KL-one 语言派生出来的。利用 NIKL 可以建立一组概念之间的联系，每一概念由一组槽构成，通过填写相应的槽值来表示概念有关内容（槽值）之间的各种关系。NIKL 还提供了一个分类器（Classifier），利用分类器可以将新的概念组织到语义网络中的适当的位置上。从而逐渐递增地建立起一个大型的领域模型。

基于上述思想，Neches 建立了一个程序增强辅助系统 PEA（Program Enhance Advisor）。PEA 的设计目的是向程序员提供改善其程序可读性的建议。例如，为改善程序可读性，建议将 LISP 语言中的 cond 函数改为 if-then-else 结构，根据概念的组织方法，PEA 建议将

```
(cond ((atom x) x) (t (car x)))
```

改写为更易读的形式：

```
if (atom x) then x else (car x)
```

问题求解知识也可用 NIKL 进行表示，根据求解的目标把有关知识组织成一个层次网络。求解目标与求解的知识是相关的，每一求解知识都有相应的“能力描述”（Capability description）来说明求解知识可能达到的目标。另外，NIKL 还保留术语知识，程序生成器可用术语知识来收集概念的实例或完成其他工作。

同 XPLAIN 系统一样，解释型专家系统的程序生成器也采用自顶向下的工作方式对目标求精。在 PEA 中，也是采用这种方式对目标求精来得到可读性强、效率更高的程序。Neches 将程序生成器的这种工作方式称为“用户驱动的动态求精”（User-directed dynamic refinemet）。如果选择可读性作为目标，这一目标将自动加入目标/子目标求精模式，通过对程序的扫描来寻找相应的程序变换，让用户证实变换的可行性后，就执行程序的变换工作。

解释型专家系统的优点是明显的，它可以给出问题求解与控制的高级策略，经过求精，再把高级策略变换成低级的策略。这一方式可向用户提供关于系统行为合理性的有力说明。在一般专家系统中，高级策略往往隐藏在系统内部，难以向用户说明。此外，采用 NIKL 可以完成新知识的自动分类，从而减轻了专家和知识工程师建立知识库的工作量。总之，解释型专家系统扩充了自动程序员解释方法，为建立有强大解释能力的专家系统提供了一种新思路。

5.4.2 解释器及其实现

我们将给出基于 LISP 语言实现的产生式系统的一个解释器，这个解释器的解释机制采用路径跟踪方法，可向用户提供 Why 解释和 How 解释。路径跟踪法需要把推理过程中使用过的规则自动记录下来，才能向用户提供解释。为此，我们需要重新定义推理机中的有关函数，对其增加记录功能。

1. 有关函数的定义

(1) 正向推理机中函数 try-rule 的重新定义

函数表达式：

```
(try-rule rule)
```

函数功能：判断规则变量 rule 中的一条规则的前件包含的全部事实是否在表 facts 中，若全部事实都在 facts 中，且规则后件有不在 facts 中的结论，则把不在 facts 中的结论逐一添加到表 facts 中，并把 rule 中的这条规则添加到表 rules-used 的表头，try-rule 返回 t；否则，try-rule 返回 nil。

函数定义：

```
(defun (try-rule rule)
  (cond ((and (test-if rule) (use-then rule))
        (setq rules-used (cons rule rules-used))
        t)))
```

(2) 反向推理机中函数 try-rule+ 的重新定义

函数表达式：

```
(try-rule+ rule)
```

函数功能：若规则变量 rule 中的一条规则的前件包含的所有条件都能直接或间接地反向推理确认为真，且规则后件中有不在 facts 中的结论，则把规则后件中不在 facts 中的结论都添加到 facts 中，并把 rule 中的这条规则添加到表 rules-used 的表头，try-rule+ 返回 t；否则，try-rule+ 返回 nil。

函数定义：

```
(defun (try-rule+ rule)
  (cond (and (test-if+ rule) (use-then rule))
        (setq rules-used (cons rule rules-used))
        t)))
```

(3) 函数 usedp

函数表达式：

```
(usedp ruln)
```

函数功能：若推理过程中使用过由规则名变量 ruln 记载的一个规则名指定的规则，则

usedp 返回 t；否则，usedp 返回 nil。

函数定义：

```
(defun (usedp ruln)
  (prog (poss)
    (setq poss rules-used)
    loop
    (cond ((null poss) (return nil))
          ((equat ruln (car poss)) (return t)))
    (setq poss (cdr poss))
    (go loop)))
```

2. How 解释

用 LISP 语言设计的解释器，各种解释被定义为相应的函数，在用户的询问下，调用相应的函数，在屏幕上给出回答。How 解释用于回答用户关于“系统是怎样得出这一结论”的询问。

函数表达式：

```
(how fact)
```

函数功能：告诉用户变量 fact 中结论是怎样得出的，若 fact 中的结论或假设是经推理确认为真，则找到支持这一结论或假设成立的规则，显示该规则前件的所有事实。若 fact 中的结论或假设是已在 facts 中的给定事实，则给出相应的说明。否则，fact 中的结论或假设没有被确认。

函数定义：

```
(defun (how fact)
  (prog (poss success)
    (setq poss rules-used)
    loop
    (cond ((null poss)
      (cond (success (return t))
            ((recall fact) (print fact | WAS GIVEN |) (return t))
            (t (print fact | IS NOT ESTABLISHED |) (return nil))))
      ((thenp fact (car poss))
        (setq success t)
        (print fact | IS DEMONSTRATED BY |)
        (mapcar '(lambda (a) (print a)
                  (cdadr (car poss))))))
    (setq poss (cdr poss)))
```

```
(go loop) ) )
```

若使用过规则表 `rules-used` 不为空, 则调用函数 `thenp` 对规则表 `rules-used` 逐个规则判断 `fact` 是否是规则后件中的一个结论, 若有这样的规则, 则逐一把这些规则的前件包含的所有事实作为“是怎样得出 `fact` 结论”的解释, 屏幕上给出说明:

`fact` 中的结论 “IS DEMONSTRATED BY” 规则前件的所有事实

若使用过规则表 `rules-used` 为空或 `rules-used` 中没有一条规则的后件中有 `fact` 结论, 则调用函数 `recall` 判断 `fact` 是否是 `facts` 中的一个给出的事实, 若是, 则屏幕上给出说明:

`fact` 中事实 “WAS GIVEN”

若不是上述两种情况, 则屏幕上直接给出说明:

`fact` 中的假设 “IS NOT ESTABLISHED”

3. Why 解释

Why 解释用于回答用户关于“为什么需要这一事实”的询问。

函数表达式:

```
(Why fact)
```

函数功能: 告诉用户为什么需要变量 `fact` 中的事实, 若 `fact` 中的事实是在推理过程中作为一条规则的前件事实用过, 则找到这条规则, 向用户说明 `fact` 中的事实用以支持这条规则的后件结论。若 `fact` 中的事实是已在 `facts` 中的给定事实, 则给出相应的说明。否则, `fact` 中的事实没有被确认。

函数定义:

```
(defun (why fact)
  (prog (poss success)
    (setq poss rules-used)
    loop
    (cond ((null poss)
      (cond (success (return t))
            ((recall fact) (print fact | WAS HYPOTHESIS | ) (return t))
            (t (print fact | IS NOT ESTABLISHED | ) (return nil)))))
    ((ifp fact (car poss))
      (setq success t)
      (print fact | NEEDED TO SHOW | )
      (mapcar '(lambda (a) (print a))
              (cdaddr (car poss))))
    (setq poss (cdr poss))
    (go loop) ) )
```

其中函数 `ifp` 用于判定 `fact` 是否在规则的 `if` 部分中。`ifp` 函数定义为:

```
(defun (ifp fact rule) (member fact (cdadr rule)))
```

若 fact 中的事实是在推理过程中作为使用过规则表 rules-used 中某些规则的前件事实使用过, 则逐一把这规则的后件结论作为“为什么需要 fact 事实”的解释, 在屏幕上给出说明:

fact 中的事实“NEEDED TO SHOW”规则后件的结论

若 fact 中的事实没有被规则使用过, 则判断 fact 是否是 facts 中已被确认的事实, 若是, 则屏幕上给出说明:

fact 中事实“WAS HYPOTHESIS”

若不是上述两种情况, 则屏幕上直接给出说明:

fact 中的事实“IS NOT ESTABLISHED”

5.5 知识获取

拥有知识是专家系统有别于其他计算机软件系统的重要标志, 而知识的质量和数量又是决定专家系统性能的关键因素。知识获取就是要解决如何使专家系统获得高质量的知识。

知识获取是一个与领域专家、知识工程师以及专家系统自身都密切相关的复杂问题, 由于各方面的原因, 至今仍然是一件相当困难的工作, 被公认是专家系统建造中的一个“瓶颈”问题。虽然已有许多人工智能学者在开展这方面的研究工作, 希望实现知识的自动获取, 即由建造的专家系统自动完成知识的获取, 并且也取得了一些成果, 但距离知识的完全自动获取还相距甚远, 还需要解决许多理论及技术上的困难问题。目前, 知识获取通常是由知识工程师与专家系统中的知识获取模块共同完成的, 知识工程师负责通过领域专家抽取知识, 并用适当的知识表示方式把知识表示出来。专家系统中的知识获取模块负责把知识转换为计算机可存储的内部形式, 把它们存入知识库。在知识存储的过程中, 要对知识进行一致性、完整性的检测。

5.5.1 知识获取的任务与方式

知识获取的基本任务是为专家系统获取知识, 建立起健全、完善、有效的知识库, 以满足求解领域问题的需要。按知识获取的自动化程度划分, 知识获取可分为非自动知识获取和自动知识获取两种方式。

1. 知识获取的任务

知识获取需要做以下几项工作。

(1) 抽取知识

抽取知识是指把蕴含于知识源(领域专家、书本、相关论文及系统的运行实践等)中的知识经过识别、理解、筛选、归纳等抽取出来, 以用于建立知识库。

知识的主要来源是领域专家及相关的专业技术文献, 但知识库所需知识并不都是以某种现成的形式存在于这些知识源中, 为了从中得到所需的知识需要做大量的工作。领域专家虽然可以自如地处理领域内的各种困难问题, 但由于缺少总结, 不一定能有条理地说出处理问

题的道理和原则。他们可以列举出大量处理过的实例，但不一定能建立起相互间的联系，给出这些实例处理的一般规则，有的经验性知识甚至只可意会而不能言传。另外，领域专家一般都不熟悉专家系统的有关技术，不知道应该提供一些什么知识以及用什么样的形式来表达这些知识。由于不能强求领域专家按专家系统的要求来提供知识，因此，为了从领域专家得到有用的知识，需要反复多次地与领域专家交谈，并且有目的地引导交谈的内容，然后通过分析、综合、去粗取精、去伪存真，归纳出可供建立知识库的知识。

知识的另一来源是系统自身的运行实践，这就需要从实践中学习、总结出新的知识。一般来说，一个系统初步建成后是很难做到完美无缺的，通过运行才会发现知识不够健全，需要补充新的知识。此时除了请领域专家提供进一步的知识外，还可由系统根据运行经验从已有的知识或实例中演绎、归纳出新知识，补充到知识库中去，这就要求系统自身具有一定的“学习”能力，这将对知识获取提出更高的要求。

(2) 知识转换

知识转换是指把知识由一种表示形式转换为另一种表示形式。

人类专家或科技文献中的知识通常是用自然语言、图形、表格等形式表示的，而知识库中的知识是用计算机能够识别、运用的形式表示的，两者之间较大的差别。为了把从专家及有关文献中抽取出来的知识送入知识库供求解问题使用，需要进行知识表示形式的转换。知识转换一般分两步进行：第一步是把从专家及文献资料抽取的知识转换为某种知识表示模式，如产生式规则、框架等；第二步是把该模式表示的知识转换为系统可直接利用的内部形式。前一步工作通常由知识工程师完成，后一步工作一般通过输入及编译实现。

(3) 知识输入

知识输入是指把用适当的知识表示模式表示的知识经过编辑、编译送入知识库的过程。目前，知识的输入一般是通过两种途径实现：一种是利用计算机系统提供的编辑软件；另一种是用专门编制的知识编辑系统，称之为知识编辑器。前一种的优点是简单，可直接拿来使用，减少了编制专门的知识编辑器的工作。后一种的优点是专门的知识编辑器可根据实际需要实现相应的功能，使其具有更强的针对性和适用性，更加符合知识输入的需要。

(4) 知识检测

知识库的建立是通过对知识进行抽取、转换、输入等环节实现的，任何环节上的失误都会造成知识错误，直接影响到专家系统的性能。因此，必须对知识库中的知识进行检测，以便尽早发现并纠正错误。另外，经过抽取转换后的知识可能存在知识的不一致和不完整等问题，也需要通过知识检测来发现是否有知识的不一致和不完整，并采取相应的修正措施，使专家系统的知识具有一致性和完整性。

2. 知识获取方式

(1) 非自动知识获取

在非自动知识获取方式中，知识获取分两步进行，首先由知识工程师从领域专家和有关技术文献获取知识，然后由知识工程师用某种知识编辑软件输入到知识库中。

如前所述,领域专家一般不熟悉知识处理,不能强求他们把自己的知识按专家系统的要求进行知识抽取和转换。另外,专家系统的设计和建造者虽然熟悉专家系统的建造技术,却不掌握专家知识。因此,需要在这两者之间有一个中介专家,他既懂得如何与领域专家打交道,能从领域专家及有关文献中抽取专家系统所需的知识,又熟悉知识处理,能把获得的知识用合适的知识表示模式或语言表示出来,这样的中介专家称为知识工程师。实际上,知识工程师的工作大多是由专家系统的设计与建造者担任。知识工程师的主要任务是:

1) 与领域专家进行交谈,阅读有关文献,获取专家系统所需要的原始知识。这是一件很费力费时的工作,知识工程师往往需要从头学习一门新的专业知识。

2) 对获得的原始知识进行分析、整理、归纳,形成用自然语言表述的知识条款,然后交领域专家审查。知识工程师与领域专家可能需要进行多次交流,直至有关的知识条款能完全确定下来。

3) 把最后确定的知识条款用知识表示语言表示出来,通过知识编辑器进行编辑输入。

知识编辑器是一种用于知识输入的软件,通常是在建造专家系统时根据实际需要编制的。目前,知识编辑器应具有以下主要功能:

1) 把用某种知识表示模式或语言所表示的知识转换成计算机可表示的内部形式,并输入到知识库中。

2) 检测输入知识中的语法错误,并报告错误性质与位置,以便进行修正。

3) 检测知识的一致性,报告非一致性的原因,以便知识工程师征询领域专家意见并进行修正。

非自动方式是使用较普遍的一种知识获取方式。专家系统 MYCIN 就是其中最具代表性的,它对非自动知识获取方法的研究和发展起到了重要作用。MYCIN 用产生式作为表示知识的模式,并用 LISP 语言表示每条规则。其知识获取通过以下几步完成:

1) 知识工程师通过交互方式向系统输入规则的前提条件、结论以及规则强度。

2) 系统把它翻译为 LISP 语言的表示形式,然后再用英语的描述形式显示出来,供知识工程师或领域专家检查它是否正确。

3) 如有错误,则由知识工程师与领域专家协商修改,然后重复 1) 和 2) 的工作,直到被确认正确为止。

4) 对于新规则的输入,则把新规则与知识库中的已有规则进行一致性检查。如发现不一致,就及时报告,由知识工程师和领域专家对有关规则进行修正。

5) 将正确的规则送入知识库中。

至此,一条规则的输入已经完成。如还要输入其他规则,则重复上述过程。

(2) 自动知识获取

自动知识获取是指系统自身具有获取知识的能力,它不仅可以直接与领域专家对话,从专家提供的原始信息中“学习”到专家系统所需的知识,而且还能从系统自身的运行实践中总结、归纳出新的知识,发现知识中可能存在的错误,不断自我完善,建立起性能优良、知

识完善的知识库。为达到这一目的,自动知识获取至少应具备以下能力:

1) 具备识别语音、文字、图像的能力。专家系统中的知识主要来源于领域专家以及有关的多媒体文献资料等。为了实现知识的自动获取,就必须使系统能与领域专家直接对话,能阅读和理解相关的多媒体文献资料,这就要求系统应具有识别语音、文字与图像的能力。只有这样,它才能直接获得专家系统所需要的原始知识。

2) 具有理解、分析、归纳的能力。领域专家提供的知识通常是处理具体问题的实例,不能直接用于知识库。为了把这些实例转变为知识库中的知识,必须对实例集进行分析、归纳、综合,从中抽取专家系统所需的知识送入知识库。在非自动知识获取方式中,这一工作是由知识工程师完成的,而在自动知识获取方式中则由系统自动完成。

3) 具有从运行实践中学习的能力。在知识库初步建成投入使用后,随着应用的发展,知识库的不完备性就会逐渐暴露出来。知识的自动获取系统应能在运行实践中学习,产生新知识,纠正可能存在的错误,不断地对知识库进行更新和完善。

总之,在自动知识获取系统中,原来需要知识工程师做的工作都由系统来完成,并且还应做更多的工作。自动知识获取是一种理想的知识获取方式,它的实现涉及到人工智能的多个研究领域,例如模式识别、自然语言理解、机器学习等,对硬件也有更高的要求。

近几年来,自然语言理解、机器学习等领域的研究已取得较大的进展,特别是关于人工神经网络的研究提出了多种学习算法,这都为知识获取提供了有利条件。在建造知识获取系统时,应充分利用这些成果,向知识的自动获取过渡,提高系统的智能程度。事实上,在近年来建造的专家系统中,都不同程度地进行了尝试与探讨,在非自动知识获取的基础上增加了部分学习功能,使系统能从实例集中归纳出某些知识。这些系统虽然没有达到完全自动知识获取的程度,但可称之为半自动知识获取。在不同的系统中,知识获取的“半自动”程度是有较大区别的。

5.5.2 知识的检测与求精

知识库中知识的一致性、完整性是影响专家系统性能的重要因素。

1. 知识的一致性与完整性

知识库的建立过程是知识经过一系列变换后存入计算机系统的过程,在这个过程中存在各种因素会导致知识不健全。例如:

1) 领域专家提供的知识中存在某些不一致、不完整、甚至错误的知识。由于专家系统是以专家知识为基础的,因而专家知识中的任何不一致、不完整必然影响知识库的知识一致性与完整性。

2) 知识工程师未能准确、全面地理解领域专家的意图,使得所抽取的知识条款隐含着种种错误,影响到知识的一致性和完整性。

3) 对知识库中的知识进行增、删、改时没有充分考虑到可能产生的影响,以致在对知识库进行了这些知识更新操作后使知识库出现了知识不一致或不完整的情况。由于知识之间

存在着复杂的联系,因此,对知识的更新操作可能产生意想不到的后果。

由于这些原因,知识库中可能出现各种问题,主要表现为知识冗余、矛盾、从属、环路、不完整等方面。下面以产生式表示模式为例说明各种问题的表现形式及处理方法。

(1) 知识冗余

知识冗余是指知识库中存在多余的知识或者存在多余的约束条件。知识冗余有以下三种表现形式:

1) 等价规则。当两条产生式规则在相同条件下有相同的结论时,称它们为等价规则。例如,若有如下产生式规则:

$R_1: \text{ if } P \wedge Q \text{ then } R$

$R_2: \text{ if } Q \wedge P \text{ then } R$

则它们是等价的。此时, R_1 与 R_2 中有一条规则是多余的,可从知识库中删去。

2) 冗余规则链。如果两条规则链中第一条规则的条件相同,且最后一条规则的结论等价,则称此两条规则链之间存在冗余。例如,若有如下产生式规则:

$R_1: \text{ if } P \text{ then } Q$

$R_2: \text{ if } Q \text{ then } R$

$R_3: \text{ if } P \text{ then } S$

$R_4: \text{ if } S \text{ then } R$

其中,如果 Q 只是在 R_2 的条件部分出现,且不再在其他规则的条件中出现,则 R_1 和 R_2 都是冗余规则,可从知识库中删去。同理,如果 S 只是在 R_4 的条件部分出现,且不再在其他规则的条件中出现,则 R_3 和 R_4 都是冗余规则,可从知识库中删去。如果除 R_2 与 R_4 外, Q 与 S 同时都不再别的规则的条件中出现,则 $R_1 \sim R_4$ 都是冗余规则,均可从知识库中删去,但应补充如下一条规则:

$\text{ if } P \text{ then } R$

否则将破坏知识库的完整性。

3) 冗余条件。如果两条规则有相同的结论,但一条规则中的某个子条件在另一条规则的条件中被否定,而其他子条件保持一致,则称这两条规则有多余的条件。例如,若有如下两条产生式规则:

$R_1: \text{ if } P \wedge Q \text{ then } R$

$R_2: \text{ if } P (\wedge \neg Q) \text{ then } R$

则子条件 Q 与 $\neg Q$ 都是多余的,此时需要删去这两条规则,并增加如下一条规则:

$\text{ if } P \text{ then } R$

(2) 知识矛盾

如果两条规则或规则链在相同条件下得到的结论是互斥的;或者它们虽有相同的结论,但规则强度不同,则称它们是矛盾的。例如,若有如下两条产生式规则:

$R_1: \text{ if } P \text{ then } Q_1$

R_2 : if P then Q_2

且 $Q_1 = \neg Q_2$, 则 R_1 与 R_2 是矛盾的。

再如有如下产生式规则:

R_1 : if P then Q

R_2 : if Q then R

R_3 : if R then S_1

R_4 : if P then T

R_5 : if T then S_2

其中, R_1 、 R_2 、和 R_3 是一条规则链, R_4 和 R_5 是另一条规则链, 它们有相同的初始条件, 即条件 P。若 $S_1 = \neg S_2$, 则这两条规则链是矛盾的。对于矛盾规则或矛盾规则链, 不能共处于同一个知识库中, 必须从中舍弃一个。至于舍弃哪一个, 需要征询领域专家意见。

又如, 若有如下两条产生式规则:

R_1 : if P then Q (CF_1)

R_2 : if P then Q (CF_2)

R_1 与 R_2 的条件及结论都分别相同, 但有不同的规则强度 ($CF_1 \neq CF_2$), 则称它们是矛盾的。它们也不能共处于同一个知识库中, 需根据领域专家的意见保留其中一个。

(3) 知识从属

如果规则 R_1 与 R_2 有相同的结论, 但 R_1 比 R_2 有更多的条件, 则称 R_1 是 R_2 的从属规则。

例如, 若有如下两条产生式规则:

R_1 : if P $\wedge$ Q then R

R_2 : if P then R

则 R_1 是 R_2 的从属规则。

当出现从属规则时, 需征询领域专家的意见, 分别进行以下几种处理:

1) 若领域专家认为 R_1 比 R_2 能更准确地描述实际情况, 则保留 R_1 , 舍弃 R_2 。

2) 若领域专家认为 R_1 中的 Q 是可有可无的, 则舍弃 R_1 , 保留 R_2 。

3) 若领域专家认为这两条规则分别适用于不同的情况, 则把它们都保留, 但需把适用条件增设到相应规则的条件部分中去。

(4) 知识环路

当一组规则形成一条循环链时, 则称它们构成了一个环路。例如, 若有如下一组产生式规则:

R_1 : if P then Q

R_2 : if Q then R

R_3 : if R then S

R_4 : if S then P

则这 4 条规则构成一个环路。对环路无论先执行哪一条规则, 最终又回到出发点。环路可能

使推理陷入死循环, 需要引起足够的重视。

当知识库中出现环路时, 应征询领域专家的意见, 修改或舍弃其中的一条规则, 破坏形成环路的条件。

(5) 不可达知识

不可达知识是指知识的约束条件永远得不到满足的知识, 这种知识在推理过程中不会被激活, 不会出现在任何推理链中, 应被舍弃。

(6) 知识不完整

知识不完整是指知识库中的知识不完全, 不能满足预先定义的约束条件, 即当存在应该推出某一结论的条件时, 却推不出这一结论, 不能形成产生这一结论的推理链; 或者虽能推出结论, 但却是错误的。

当知识库的知识不完整时, 需要通过知识求精不断改进、完善, 使其能满足问题求解的需要。

2. 知识的检测方法

为了保证知识库的正确性, 需要做好知识的检测。知识检测分为静态检测和动态检测。静态检测是指在知识输入之前由领域专家及知识工程师所做的检查工作。动态检测是指在知识输入过程中以及对知识库进行增、删、改时由系统所进行的检查。在系统运行过程中出现错误时也需要对知识库进行动态检测。这里, 我们讨论动态检测方法。

对知识冗余、矛盾等的检测实际上是通过知识的相应部分进行比较来实现的。例如对两条产生式规则的条件部分进行比较, 看其是否等价, 再对它们的结论部分进行比较, 看其是否一致等。因此, 检测中的基本环节是检查两个逻辑表达式的等价性。

(1) 逻辑表达式等价性的检测

产生式规则的条件部分和结论部分都可以分别表示为一个合取式, 两个合取式的等价性的检测就是逐一比较它们的合取项是否一致。设 A 和 B 分别是两个合取式, $L(A)$ 与 $L(B)$ 分别表示 A 与 B 的长度, 即 A 与 B 分别包含的合取项的个数, A_i 表示 A 的第 i 个合取项, B_j 表示 B 的第 j 个合取项。若 $L(A) \neq L(B)$, 则 A 与 B 不等价。若 $L(A) = L(B)$, 则把 B 的所有的合取项逐一地与 A 的所有合取项进行比较, 每当 B 的某一个合取项 B_j 与 A 的某个合取项 A_i 等价时, 就为 B_j 做一标记, 不再把 B_j 与 A 的其他合取项进行比较。如果 B 的所有合取项都被做上标记, 则 B 与 A 等价; 否则, B 与 A 不等价。

(2) 冗余的检测

以逻辑表达式等价性检测方法为基础, 就可以进行冗余的检测。

1) 等价规则的检测。产生式规则的条件部分和结论部分都是合取式, 只要对两条规则的条件部分及结论部分分别检查等价性就可得知这两条规则是否等价。

2) 冗余规则链的检测。为了发现冗余规则链, 首先应该检查两条规则链的第一条规则的条件是否等价。若有等价条件, 则再检查这两条规则链的最后一项规则的结论是否等价。若又有等价结论, 则可能有冗余规则链。

冗余规则链的检测可分为三步进行:

第一步: 建立两个二维表, 一个表称为 IF-IF 表, 用以存放不同规则条件部分的等价比较结果; 另一表称为 THEN-THEN 表, 用以存放不同规则结论部分的等价比较结果。

第二步: 根据 IF-IF 表, 取出条件部分等价的两条规则, 并分别建立它们的推理链。由这两条推理链的最后一条规则的结论部分查找 THEN-THEN 表, 若两个结论部分是不等价的, 则这两条推理链不是冗余的; 若两个结论部分是等价的, 则这两条推理链可能是冗余推理链, 由第三步进一步确定。

第三步: 把一条推理链中的所有规则的结论部分与其他所有规则的条件部分进行等价比较, 若都不等价, 则这条推理链是一条多余的推理链, 可将这条推理链的所有规则舍弃。若两条推理链都是多余推理链, 则可将这两条推理链的所有规则都舍弃, 但应补充一条新的规则, 新规则的条件部分是推理链第一条规则的条件部分, 新规则的结论部分是推理链最后一条规则的结论部分。

3) 冗余条件的检测。为了发现冗余条件, 首先应检查两条规则的结论是否等价, 这可从 THEN-THEN 表得到。当两条规则的结论等价而条件部分不等价时, 可把其中一条规则条件部分的各个合取项逐一变为否定, 并逐次检测这两条规则条件部分的等价性。若等价, 则刚才被否定的子条件及在另一条规则条件部分中与之对应的那个子条件都是冗余条件。

(3) 矛盾规则及矛盾规则链的检测

首先根据 IF-IF 表找出两个条件部分等价的规则, 然后将其中一条规则的结论部分变为否定后再与另一条规则的结论部分进行等价比较, 若等价, 则这两条规则是矛盾的。

对于由规则强度不同引起的矛盾, 先检查两条规则是否等价, 若等价, 再检查它们的规则强度是否相同, 若不同, 则这两条规则是矛盾的。

矛盾规则链的检测与冗余规则链的检测方法类似。首先根据 IF-IF 表找出两个条件部分等价的规则, 并分别建立它们的推理链, 然后遍历这两条推理链, 若发现这两条推理链有一对规则的结论部分是矛盾的, 则从这一对规则分别沿各自推理链回溯至第一条规则的两条子推理链是矛盾的。

(4) 从属规则的检测

为了发现从属规则, 首先要检查两条规则的结论是否等价, 这可从 THEN-THEN 表得到。若两条规则的结论等价, 再检查一条规则的条件部分是否是另一条规则条件部分的一部分, 若是, 则表明后者比前者要求更多的约束条件, 故而后者是前者的从属规则。

(5) 环路的检测

为了检测知识之间是否存在规则链环路, 需要找到这样的规则, 即该规则的结论与其他规则的条件等价, 这样的规则可能会是一个环路中的一条规则。为此, 可建立一个名为 IF-THEN 的二维表, 表中存放每条规则的条件部分与其他规则的结论部分等价比较的结果。检测时, 首先根据 IF-THEN 表找出一条结论与其他规则的条件等价的规则, 然后从这条规则开始, 根据 IF-THEN 表沿着规则链进行查找, 找出下一条结论与其他规则条件等价的规则, 直

到出现如下两种情况之一时为止:

1) 在规则链中找到了一条规则, 它的结论与规则链中已有的某条规则的条件等价, 这说明这个规则链有一个环路。

2) 沿规则链找不到一条结论与规则链中已有规则的条件等价的规则, 且规则链结束, 这说明这个规则链没有形成环路。

若对每条规则链都进行环路检测, 则可把知识库中的环路都找出来, 并进行相应的处理。

3. 知识求精

知识库中除了可能存在上述的冗余、矛盾等问题外, 还可能存在着知识不完整的问题, 以致在系统运行时产生错判或漏判的错误。错判是指对给定的不应产生某一结论的条件, 经系统运行却得出了这一结论。例如, 对于一个肝病诊断专家系统来说, 根据给定的条件, 把不应诊断为肝炎的病例诊断为肝炎, 这就是错判。漏判是指对给定条件本应推出的结论没有推出来, 例如, 把是肝炎的病例没有诊断为肝炎。

为了找出导致错误的原因, 就需要找出产生这些错误的知识, 予以改进, 以提高知识库的可靠性, 称之为知识求精。实现知识求精的一般方法是: 用一批有已知结论的实例考核知识库, 看有多少实例被系统错判和漏判, 然后对知识进行适当的修正, 以提高知识库的可靠性。

知识求精方法与知识的表示方法有密切关系, 下面讨论知识库管理系统 KBRs 中的知识求精方法。

在 KBRs 的知识表示中, 并没有把规则的前提条件显式地列入规则的条件部分中, 而是单独描述导致每个结论的各种可能前提, 并且把前提条件分为主要条件、次要条件和附加条件。在每条规则的条件部分只显式指明至少必须满足几个主要条件和几个次要条件以及显式表示的附加条件。规则的这种结构便于实现不完全匹配, 即规则只需达到要求匹配的主要条件和次要条件的数目以及能匹配显式表示的附加条件, 并不具体指定要求匹配哪几个主要条件和次要条件。

下面给出一个用 KBRs 表示知识的例子。

*C₁

CLASS: 肠热症

MAJORS: 5

- (1) 体温增高, 持续不退
- (2) 肚腹胀满
- (3) 腹泻
- (4) 昏睡
- (5) 呕吐

MINORS: 3

- (1) 流鼻血
- (2) 头痛

(3) 便秘

*R₁:

IF (1) MAJORS (3, 肠热症)

(2) MINORS (2, 肠热症)

(3) 胸部出现少量的玫瑰红色疹子

THEN CONCLUSION 85% IS 肠热症

类 C₁ 给出了属于肠热症的 5 个主要条件和 3 个次要条件。规则 R₁ 的前提条件要求满足主要条件中的 3 个、次要条件中 2 个, 并且满足显式表明的附加条件“胸部出现少量的玫瑰红色疹子”, 则有 85% 的可能性是患了肠热症。

一条规则 R 判断一个实例 E 属于类 C, 必须满足如下三个要求:

1) R 的结论是 C。

2) E 满足 R 的前提条件 (匹配一致的主要条件和次要条件的数目分别达到要求的数目以及要求的附加条件)。

3) R 的可信度超过规定的阈值。

KBRS 的知识求精是通过使用一批已知结论的实例来检测知识库, 且只考虑如下两类知识缺陷:

1) 错判: 对于不应该是结论 C 的实例, 在给定条件下却推出了 C。

2) 漏判: 对于应该是结论 C 的实例, 在给定条件下却推不出 C。

KBRS 的求精方法是:

1) 首先用一批已知结论的实例检测知识库的总体质量, 即对每个结论分类 C 分别得出下列数据:

实例数: 已知属于类 C 的用于检测的实例个数。

正判数: 已知属于类 C, 判断也是类 C 的实例个数。

错判数: 不属于类 C, 却被判断为类 C 的实例个数。

漏判数: 已知属于类 C, 但判断为不是类 C 的的实例个数。

然后, 可计算出:

错判率 = 错判数 / 实例数

漏判率 = 漏判数 / 实例数

2) 如果类 C 的漏判率大于错判率, 且漏判率大于 20%, 则对以 C 为结论的规则进行泛化处理。

3) 如果类 C 的错判率大于漏判率, 且错判率大于 20%, 则对以 C 为结论的规则进行特化处理。

4) 如果类 C 的错判率等于漏判率, 且大于 20%, 则对以 C 结论的规则进行泛化处理。

泛化处理是指减弱规则的条件, 可行的方法有:

1) 减少规则要求匹配的主要条件或次要条件的数目。

2) 删去规则中某一个或某几个附加条件。

3) 增大规则的可信度。

特化处理是指增强规则的条件,可行的方法有:

1) 增加规则要求匹配的主要条件或次要条件的数目。

2) 增加规则的附加条件。

3) 降低规则的可信度。

在具体进行泛化处理时,还应仔细分析造成漏判的主要原因,即分别统计由于实例的主要条件数不够、或次要条件数不够、或附加条件不满足、或可信度达不到规定的阈值而被漏判的实例数,然后采取相应的方法对规则进行泛化处理。

在具体进行特化处理时,KBRS 不直接分析那些被错判的实例是由于什么原因被错判的,而是采用了间接方法,查看那些被正确判断的实例还有多少条件没用上,并分析这些没用上的条件属于何种条件,然后再统计对每一种条件没用上的实例个数,并以此作为特化处理的依据。例如,设实例 E 除了具有 R_1 所规定的主要条件、次要条件及附加条件外,还具有另外 2 个主要条件,这表示 R_1 所规定的主要条件可能有点少。当对多个正确判断的实例进行统计,得知多数实例都有这一情况(即都多出 2 个主要条件)时,那么就可将 R_1 中关于主要条件的个数增加 2 个。

5.5.3 知识的组织与管理

专家系统的性能一方面取决于知识的质量与数量、推理方法及控制策略,另一方面也取决于知识的组织与管理。

1. 知识的组织

当把知识送入知识库时,面临的问题就是要事先确定知识库的存储结构,以便于建立知识库中知识的逻辑联系,称这一工作为知识的组织。

知识的组织方式一方面依赖于知识的表示模式,另一方面也与计算机系统提供的软件环境有关。原则上可用于数据组织的方法都可用于知识的组织,究竟选用哪种组织方式,要视知识的表示形式和对知识的使用方式而定。一般来说,在确定知识的组织方式时应遵循如下基本原则。

(1) 知识的独立性

知识库与推理机相分离是专家系统的特征之一。选用的知识组织方式应使知识具有相对的独立性,这就不会因为知识的变化而对推理机产生影响。

(2) 便于对知识的搜索

在推理过程中,对知识库进行搜索是经常要进行的工作,而知识组织方式直接与搜索相关,会直接影响到系统工作的效率。因此,在确定知识的组织方式时要充分考虑到采用的搜索策略,使两者能够密切配合,以提高对知识库的搜索速度。

(3) 便于对知识进行维护与管理

知识库建成后对知识库的维护与管理是一项经常性的工作。知识的组织方式应便于检测知识可能存在的冗余、不一致、不完整之类的错误,便于向知识库增加新知识、删除错误知识以及对知识的修改。

(4) 便于在知识库中同时存储用多种模式表示的知识

把多种知识表示模式有机地结合起来是一个专家系统的知识表示的常用方法。例如,把框架和产生式结合起来,既可表示知识的结构性,又可表示过程性知识。知识的组织方式应对这种多模式表示的知识实现存储,而且便于对知识的利用。

(5) 尽量节省存储空间

知识库一般需要占用较大的存储空间,其规模一方面取决于知识的数量,另一方面也与知识的组织方式有关。因此,在确定知识的组织方式时,关于存储空间的利用问题也应作为考虑的一个因素,特别是在确定大型知识库时更应考虑这一问题。

2. 知识的管理

严格地说,知识的维护和知识的组织都属于知识管理的范畴,除上述工作之外,知识的管理还包括以下主要内容。

(1) 知识库的重组

为了提高系统运行效率,建立知识库时总是采用适合领域问题求解的组织形式。但当系统经过一段时间运行后,以及对知识库进行了多次的增、删、改,知识在知识库中的位置可能发生一些变化,使得某些使用频率较高的知识不是处于容易被搜索到的位置上,从而直接影响系统的运行效率。此时需要对知识库中的知识重新进行组织,使那些使用较多的知识处于容易被搜索到的位置上,使那些逻辑上关系比较密切的知识尽量放在一起,等等。

(2) 记录系统运行的实例

问题实例的运行过程是求解问题的过程,也是系统积累经验、发现自身缺陷及错误的过程,因此应对运行的实例做适当的记录。记录的内容没有严格的规定,可根据实际需要确定。例如,记录的项目可以有:实例编号、提交人、提交时间、实例内容、实例运行中出现的问题、推理结论是否正确、运行时间等。为了对系统运行的实例进行记录,需要建立专用的运行实例库。

(3) 记录系统的运行史

专家系统是在使用过程中不断完善的,为了对系统的进一步完善提供依据,除了记录系统的运行实例外,还需要记录系统的运行史,记录的内容与知识的检测及求精方法有关,没有统一标准。一般来说,应当记录:系统运行过程中激活的知识、产生的结论以及产生这些结论的条件、推理步长、专家对推理结论的评价等。这些记录不仅可用来评价系统的性能,而且对知识的维护以及系统向用户的解释都有重要作用。为了记录系统的运行史,需要建立运行史库以存储上述信息。

(4) 记录知识库的发展史

对知识库的增、删、改将使知识库的内容发生变化,如果将其变化情况及知识的使用情

况记录下来,将有利于评价知识的性能、改善知识库的组织结构,达到提高系统效率的目的。为了记录知识库的发展变化情况,需要建立知识库发展史库,记录内容一般包括:

1) 知识库设计者及建造者的姓名、初始建成的时间。

2) 每条知识的编号及它进入知识库的时间,知识提供的专家姓名与编号,以及知识的“权”系数。

3) 如有知识被删除,则记录删除者姓名、被删除的知识及删除的时间。可把被删除知识的上述有关信息记录在后备库中,只在发展史库中登记相应信息在后备库中的地址。这样既可减少发展史库的存储空间,提高查找速度,又可以恢复被错删的知识。

4) 如有知识被修改,则记录修改者姓名、修改前的知识及修改时间。也可把这些信息记录在后备库中,而把它在后备库中的地址登记在发展史库中。

5) 统计并记录各类知识使用次数,如有可能还可对各条知识的性能进行分析。从而可把使用频率较高的知识放在容易搜索到的位置上,提高系统运行效率。还可以对功能较弱的知识进行完善。

(5) 知识库的安全与保密

知识库的安全是指不要使知识库受到破坏。知识库是专家系统赖以生存的基础,必须建立严格的安全保护措施以防止由于操作失误等主观或客观原因使知识库遭到破坏。可行的安全保护措施,既可以像数据库系统那样通过设置口令来验证操作者的身份、对不同操作者设置不同的操作权限、预留备份等,也可以针对知识库的特点采取特殊的有效措施。

知识库的保密是指防止知识的泄漏。知识是领域专家多年实践及研究的结晶,是经过领域专家和知识工程师抽取、检测和求精后的极其宝贵的财富,在未取得有关人员同意的情况下是不能外传的。因此,专家系统应对知识库采取严格的保密措施,严防未经许可就查阅、复制的行为。至于保密的措施,通常用于软件加密的各种手段都可用于知识库的保密。

5.6 专家系统工具

知识工程师建造一个专家系统时,使用专家系统工具可以极大地简化建立专家系统的工作,减少建造的工作量,提高建造的专家系统的性能。专家系统工具的出现,是知识工程的重大进程,它帮助知识工程师完成了许多工作,能大大缩短专家系统的研制周期。对于各领域的专家来说,工具比一般的程序设计语言更易学习和使用,因此,在专家系统的开发中更多地使用专家系统工具来建造一个专家系统。

5.6.1 专家系统工具概述

20 世纪 80 年代以来,专家系统工具的研究和开发受到高度重视,各种性能良好的工具不断涌现,并已显示了工具的巨大作用。例如,利用工具 EMYCIN 成功地开发了 PUFF (肺功能失调诊断专家系统)、SACON (结构分析专家系统)、CLOT (血凝疾病诊断专家系统)、

HEAMED（精神药理学顾问）、LITHO（结石光图形分析专家系统）、DART（计算机故障诊断专家系统）和 BLUEBOX（医院日常监护专家系统）等专家系统。其中，SACON 专家系统的开发过程中只用了大约 4 个月，获取了 170 条规则和 40 个参数。在 PUFF 专家系统的开发过程中，与领域专家合作提取知识只用了 50 个小时，建立基本模型只用了 10 人周的工作量，通过 150 个病例来测试该模型的性能，系统的诊断结果与专家结论一致率达到 90% 以上。另一个典型例子是使用知识工程语言 OPS 构造的 VAX 计算机配置专家系统 R1，该系统具有为获得全部的计算机组件及其关系所需的 3000 多条启发式规则。R1 系统投入使用后，可以为生产 VAX 计算机的 DEC 公司每年节省几千万美元。

专家系统工具按其功能主要分为两类，一类是用于生成专家系统的工具，称为生成工具；另一类用于改善专家系统性能的工具，称为辅助工具。

1. 系统生成工具

系统生成工具主要帮助知识工程师构造专家系统中的推理机和知识库结构。按照生成工具的本身特征又可分为以下 4 类。

（1）程序设计语言

程序设计语言是开发专家系统的最基本的工具。典型的程序设计语言是 LISP 语言和 PROLOG 语言，用这两种人工智能语言能方便地表示知识和设计各种推理机。具有面向对象风格的语言 C++，以及传统语言 C 和 PASCAL 等也是构造专家系统的常用语言。

（2）骨架系统

骨架系统是把一个成功的专家系统删去其特定领域知识而留下的系统框架。例如，系统生成工具 EMYCIN 就是删去医疗诊断专家系统 MYCIN 的医疗诊断知识而获得的骨架系统。骨架系统继承了原专家系统中行之有效的知识表示方式、推理机和知识库结构以及全部辅助工具。因此，利用骨架系统建造专家系统时，只要把特定领域的知识按照该骨架系统的知识表示方式输入到知识库中，就构成了一个特定领域的专家系统。

骨架系统之所以能快速方便地构造一个专家系统，是由于专家系统的推理机与知识库是分离的，只要知识库的知识表示方式和知识库的结构确定后，推理机就随之确定了。

由于骨架系统生成的专家系统完全继承了原系统的知识表示方式和知识库结构以及推理机等，因此，限制了专家系统设计者的设计选择。另一方面，选择某个骨架系统生成一个特定领域的专家系统时，若生成的专家系统与骨架系统的原系统是属于同一类问题领域，那么就更易生成和效果更好。或者说，骨架系统作为生成工具，缺乏通用性和灵活性，每一个骨架系统只适合于某一类特定的问题领域。

（3）知识工程语言

知识工程语言是专门用于构造和调试专家系统的通用程序设计语言，它能够处理不同的问题领域和问题类型，提供各种控制结构。用知识工程语言设计推理机和知识库，比用一般的人工智能程序设计语言（如 LISP 或 PROLOG 等）更为方便。由于知识工程语言并不与特定的结构和方法紧密联系，因此比骨架系统更为灵活和通用。

常见的知识工程语言工具有:

1) OPS (Official Production System) 是著名的通用型语言工具, 从 OPS1 到 OPS5、OPS5+、OPS5e、OPS83, 已经历了许多版本的修改。OPS 是用 LISP 语言实现的, 主要采用基于规则的知识表示和正向推理。用 OPS83 能很快地编写出紧凑的产生式专家系统程序, 其数据类型特征、用户定义的函数及过程均与当前的过程型程序设计语言 PASCAL 和 C 语言类同。OPS 已广泛用于开发计算机设计、模式识别、图像处理、故障诊断、实时控制等方面的专家系统。

2) KEE (Knowledge Engineering Environment) 也是著名的通用语言工具, 是 Intellicorp 公司于 1984 年开发的。它把基于框架、基于规则、面向过程和面向对象的方法结合在一起, 提供了一种多范例程序设计环境, 适于设计范围较大的实用专家系统。KEE 已用来建造了卫星失灵诊断、加热冷却管理、金融保险分析、工厂控制模拟、遗传工程研究、光导纤维制造以及原子能发电站紧急状态报警等方面数以千计规模不等的实用专家系统。

3) ROSIE (Rule-Oriented System for Implementing Expertise) 是第一个专家系统的通用程序设计系统, 主要特点是基于规则和面向英语解释, 具有英语语法。ROSIE 是 Rand 公司于 1981 年开发的, 用 InterLISP 和 C 语言实现的。它已用来开发了法律决策、战况分析、空战规划等专家系统。1982 年, Rand 公司推出了它的另一个专家系统工具 ROSS, 主要特点是基于规则和面向对象。

4) CLIPS (C Language Integrated Production System) 是自 20 世纪 80 年代中期以来使用广泛的通用语言工具, 其特点是具有产生式系统的实用特征, 并集成了 C 语言的基本语言成份。把人工智能与传统语言相结合, 是当代通用型语言工具的重要发展方向。CLIPS 是美国航空航天管理局 (NASA) 于 1985 年推出的, 它可以在许多机器上运行, 已广泛用于开发各类专家系统和知识处理系统。

5) ART (Automatic Reasoning Tool) 是美国 Inference 公司于 1984 年推出的一种通用语言工具, 它是采用 LISP 语言和 C 语言实现的。主要特点是基于规则和基于框架的知识表示、面向过程的正向与反向推理及双向推理。

(4) 专家系统开发环境

专家系统开发环境是以一种或多种工具和方法为核心, 加上与之配套的各种辅助工具和界面环境的完整的集成系统。近几年来, 专家系统的规模越来越大, 出现了知识数量达数千条乃至数万条规则, 知识层次包括元知识、经验性知识、原理性知识和常识性知识等几个层次的专家系统, 因此, 超大规模知识库的组织和管理变得突出起来。不同的知识表示系统之间以及人工智能技术与数据库等传统主流技术之间的系统集成技术引起了人们的高度重视。把数据库、逻辑推理、模块化技术、面向对象程序设计方法、支持智能体通信以及多媒体用户界面等先进技术集成到一个智能系统开发工具中, 已成为专家系统和智能系统开发工具的主要发展方向。

目前, 有些知识工程语言系统已经发展成这样的集成系统, 集成系统中有一组预先定义

的称为组件的程序模块，每个组件实现一种人工智能技术。这种环境可提供多种类型的推理机制和多种知识表示方法，帮助专家系统的建造者选择结构、设计规则语言和使用各种组件，使之成为一个完整的专家系统。

美国 Inference 公司于 1993 年推出的 ART*Enterprise 是一种集成化的智能应用软件开发工具，它具有面向对象、多种数据库管理、基于事例的推理 (Case-Based Reasoning) 和多媒体用户界面 (GUI with Multimedia) 等特点。对于金融业、汽车工业、电子工业、钢铁工业、航空航天部门、通信部门、计算机设计与制造业等的信息咨询与决策、故障诊断、设计规划已有广泛的应用。

2. 系统辅助工具

系统辅助工具主要用于帮助建造高质量的知识库和调试专家系统。

知识获取工具和知识库管理与维护工具是最重要的辅助工具。知识获取工具有自动知识获取工具，知识库编辑工具、面向问题求解方法的知识获取工具、面向特定知识生成技术的知识获取工具、面向特定问题领域的知识获取工具以及基于特定语言的知识获取工具等类型。其中，自动知识获取工具采用机器学习方法来进行知识获取，例如，EXPERT—EASY 通过归纳学习能自动生成问题领域的求解规则。知识库编辑工具能把专家领域知识加工、编辑到知识库中，这样的编辑工具有 TEIRESIAS 编辑器。知识库管理与维护工具能检查输入知识的一些常见错误，自动维护知识库中知识的一致性和完备性。这些工具不仅能帮助知识工程师加快建造专家系统的速度，还能保证和提高知识库的质量，调试和改进专家系统。

5.6.2 专家系统工具 CLIPS 及其应用

CLIPS (C Language Integrated Production System) 是美国航空航天管理局于 1985 年推出的一种专家系统工具，它实际上是一种通用的产生式语言，具有产生式系统的使用特征，并集成了 C 语言的一些基本语言成分。它可以在许多机器上运行，已广泛用于开发各类专家系统和知识处理系统。

1. CLIPS 的知识表示

(1) 域

CLIPS 知识表示的基本单位是域 (field)，域分为常量域和变量域两种。

常量域有以下 3 种类型：

- 1) 字 (word)：以英文字母为首的一串字符，例如，word、father-of 等。
- 2) 字符串 (string)：用双引号括起来的一串字符，其间可以有任意个字符。
- 3) 数 (number)：有整型数和实型数两种。

变量域有以下 2 种形式：

1) 单域变量 ?name。其中，name 是变量名，用一个字表示。? 称为单域通配符，含义是相应变量可以与任何单一常量匹配。变量域中的变量名可以省缺，此时，称变量域 ? 为匿名变量域。例如，?x、?shore、? 等都是单域变量。

2) 多域变量\$?name。其中, name 是变量名, 用一个字表示。\$?称为多域通配符, 表示相应变量可以与任意多个常量域匹配。

对一个变量域可以附加各种约束关系, 把变量的取值范围约束于某一个特定范围, 其中常用的是逻辑约束。逻辑约束有与、或、非 3 种, 它们与单域变量结合使用的形式分别为:

1) ?x & value 表示变量域?x 的值必需与 value 的值相同, value 可以是常量域或变量域。例如, ?fs & ?xs 表示变量域?fs 的值与?xs 的值一致。

2) ?x & value1 | value2 表示变量域?x 的值必需为 value1 或 value2 的值之一, value1 和 value2 可以是常量域或变量域。

3) ?x & ~value 表示变量域?x 的值不能取为 value 的值, value 可以是常量域或变量域。

变量域的取值约束关系也可以用一个函数的返回值来描述, 其形式为:

?name= (function-name arg₁ ... arg_n)

其中, function-name 为函数名, arg 是这个函数的形参 (变量域)。CLIPS 把函数的返回值赋给变量?name。

(2) 事实

事实用一个有序的常量域表来表示, 其表示形式为:

(field₁ ... field_n)

在实际使用中, 事实的第一个域通常是一个字, 用来描述其后各域之间的关系。例如, 可以用一个事实:

(opposite-of shore1 shore2)

表示河的两岸 shore1 和 shore2 处于相对的位置。

初始事实可在程序中直接给出, 定义初始事实形式是:

(deffacts fact-name fact₁ ... fact_n)

其中, fact₁ ... fact_n 是给出的 n 个初始事实, fact-name 是这一组初始事实的名字, 例如, 在程序中定义的一个初始事实为:

```
(deffacts opposites
  ( opposite-of shore1 shore2 )
  (opposite-of shore2 shore1 )
```

它定义了名为 opposites 的一组事实, 一个事实是 shore1 的对岸是 shore2, 另一个事实是 shore2 的对岸是 shore1。

(3) 模式

模式 (pattern) 是组成规则前件的基本单位, 一个模式用一个有序的域表来表示, 域表中的域可以是常量域, 也可以是变量域。

我们用下面的一个例子来说明模式的定义。

例 5.4 农夫过河问题: 农夫、狐狸、山羊和白菜都在一条河的同岸上, 农夫有一条船要将狐狸、山羊和白菜都带到对岸去。但是, 农夫每次撑船过河时, 船上至多只能载狐狸、

或者山羊、或者白菜；若农夫不在时，则狐狸会吃掉山羊，山羊会吃掉白菜。问：农夫如何安排过河，才能达到目的？

农夫每次撑船过河后都会使由农夫所在河岸、狐狸所在河岸、山羊所在河岸和白菜所在河岸组成的状态发生变化，因此，我们可以定义一个状态模式为：

(status ?num ?name ?nn ?fs ?xs ?gs ?cs)

其中，状态模式的第 1 个域是一个字 status，用来说明这是问题的状态模式，后面 4 个变量域分别是农夫所在河岸变量?fs、狐狸所在河岸变量?xs、山羊所在河岸变量?gs 和白菜所在河岸变量?cs。第 2 个域是状态搜索深度变量?num，第 3 个域是状态的惟一标识符变量?name，第 4 个域是其父状态的惟一标识符变量?nn。

若一个状态模式的各域都是常量域，那么，它表示问题的一个状态。例如，在农夫过河问题中，若农夫、狐狸、山羊和白菜开始都在岸 shore1，那么，可在程序中直接定义初始状态事实为：

(deffacts initial-positions

(status 0 initial-setup no-parent shore1 shore1 shore1 shore1))

1) 模式变量。若将一个模式赋给一个变量，则该变量称为模式变量。定义一个模式变量的形式为：

?name ← pattern

其中，pattern 为一个模式的有序域表。

2) 规则优先数说明模式。CLIPS 使用一个特定模式来设定一个规则的优先数。若在推理中有多个可用规则，则按规则优先数来选择可用规则。这个特定模式称为规则优先数说明模式，其表示形式为：

(declare (salience n))

其中，n 为设定的优先数， $-10000 \leq n \leq 10000$ 。若一条规则缺省规则优先数说明模式，则规则优先数缺省值为 0。

(4) 活动

活动 (action) 是组成规则后件的基本单位。一条规则的后件可以有多个活动。一个活动可以用一条语句来表示。下面，介绍 CLIPS 的几个基本活动语句。

1) 插入事实语句 assert

assert 语句把指定的事实 fact 添加到工作存储器中。assert 语句的形式为：

(assert fact)

2) 删除事实语句 retract

retract 语句从工作存储器中删除由模式变量指定的所有事实。retract 语句的形式为：

(retract ?rm₁, ..., ?rm_n)

其中，?rm₁, ..., ?rm_n 都是已被定义的模式变量。

3) 变量约束语句 bind

bind 语句用于建立新变量或修改在规则中已赋值的变量的值。bind 语句可以把一个变量的取值约束于常量、其他已赋值的变量或一个函数表达式的返回值。例如

```
( bind ?nn ( gensym ) )
```

使变量?nn 的值为 CLIPS 的函数 gensym 的返回值, 函数 gensym 生成一个与约束变量?nn 原值不等的字作为返回值赋给变量?nn。

4) 暂停语句 halt

halt 语句用来暂停该规则的执行。若一条规则的后件中有一个 halt 语句, 那么, 这条规则后件中从 halt 语句之后的活动将不再执行。halt 语句的形式为:

```
(halt)
```

5) 返回语句 return

return 语句结束推理程序并返回。return 语句的形式为:

```
(return)
```

6) 条件语句 if

if 语句的形式为:

```
(if predicate-function
    then actions1
    [else actions2] )
```

其中, predicate-function 是谓词函数, 其返回值是真或假。若为真, 则执行活动组 actions1; 否则, 执行活动组 actions2。语句中用方括号括起来的内容可以缺省。

7) 循环语句 while

while 语句的形式为:

```
(while predicate-function [do] actions)
```

若谓词函数 predicate-function 的返回值为真, 就循环执行活动组 actions, 直至谓词函数的返回值为假。

(5) 规则

规则表示的形式为:

```
(defrule rule-name declare-string
    patterns  $\Rightarrow$  actions )
```

其中, rule-name 为定义的规则的规则名; declare-string 为这一规则的说明字符串, 要用双引号括起来, 说明字符串不能缺省, 但可以是空串“”。patterns 是规则前件的一组模式, actions 是规则后件的一组活动。

CLIPS 的规则具有产生式规则的表示形式和使用特征。CLIPS 对程序中定义的规则的使用方式是: 若一条规则前件的每一模式通过模式匹配均能在当前工作存储器中找到可匹配的事实, 那么, 这条规则就是一条可用规则。若有多条可用规则, 则需要通过某种冲突消解策略或算法激发其中一条规则。激发这条规则就是顺序执行这条规则后件中给出的一组活动。

我们通过下面的例子来说明如何使用 CLIPS 的知识表示方法来编写一个问题求解的 CLIPS 程序。

例 5.5 写出求解农夫过河问题的 CLIPS 程序。

```

; ; ; Initial state
( deffacts inintial-positions
  ( status o initial-setup no-parent shore1 shore1 shore1 shore1 ) )
( deffacts opposites
  ( opposite-of shore1 shore2 )
  ( opposite-of shore2 shore1 ) )
; ; ; Rules
( defrule move-alone " "
  ( status ?num ?name ?fs ?xs ?gs ?cs )
  ( opposite-of ?fs ?ns )
  =>
  ( bind ?nn ( gensym ) )
  ( assert ( status (+1 ?num ) ?nn ?name ?ns ?xs ?gs ?cs ) ) )
( defrule move-with-fox " "
  ( status ?num ?name ? ?fs ?fs ?gs ?cs )
  ( opposite-of ?fs ?ns )
  =>
  ( bind ?nn ( gensym ) )
  ( assert ( status (+1 ?num ) ?nn ?name ?ns ?ns ?gs ?cs ) ) )
( defrule move-with-goat " "
  ( status ?num ?name ? ?fs ?xs ?fs ?cs )
  ( opposite-of ?fs ?ns )
  =>
  ( bind ?nn ( gensym ) )
  ( assert ( status (+1 ?num ) ?nn ?name ?ns ?xs ?ns ?cs ) ) )
( defrule move-with-cabbage " "
  ( status ?num ?name ? ?fs ?xs ?gs ?fs )
  ( opposite-of ?fs ?ns )
  =>
  ( bind ?nn ( gensym ) )
  ( assert ( status (+1 ?num ) ?nn ?name ?ns ?xs ?gs ?ns ) ) )
( defrule fox-eats-goat " "

```

```

(declare (salience 8000))
?rm ← (status ? ?name ? ?s1 ?s2 & ~?s1 ?s2 ?)
⇒
(retract ?rm)
(defrule goat-eats-cabbage " "
(declare (salience 8000))
?rm ← (status ? ?name ? ?s1 ? ?s2 & ~?s1 ?s2)
⇒
(retract ?rm))
(defrule circular-path " "
(declare (salience 9000))
(status ?num ? ? ?fs ?xs ?gs ?cs)
?rm ← (status ?num1 & (≤ ?num ?num1) ? ? ?fs ?xs ?gs ?cs)
⇒
(retract ?rm))
(defrule reasoning-end " "
(declare (salience 10000))
(status ? ? ? shore2 shore2 shore2 shore2)
⇒
(return))

```

上述 CLIPS 程序可分为两部分，第一部分定义了两组初始事实，程序运行时，CLIPS 把初始事实自动写入工作存储器中；第二部分定义了 8 条规则 $R_1 \sim R_8$ ，对这 8 条规则的定义分别说明如下。

规则 R_1 是农夫单独过河规则 move-alone。 R_1 前件的第 1 个模式是一个状态模式，第 2 个模式是说明农夫所河岸 ?fs 同变量 ?ns 的值是相对河岸。通过模式匹配，若 R_1 被激发，那么就执行 R_1 后件的两个活动语句，第 1 条语句由系统函数 gensym 把变量 ?nn 的原值修改为另一个字，这个字即是第 2 条语句中表示一个状态事实 status 的状态名。第 2 条语句由 assert 把状态事实 status 添加到工作存储器中。状态事实 status 是一个表，根据程序对状态的定义，第 1 个元素是这个状态的深度，其值为其父节点状态深度变量 ?num 的值加 1；第 2 个元素是这个状态的状态名，其值为变量 ?nn 的值；第 3 个元素是这个状态的父节点状态标识符，即变量 ?name 的值，变量 ?name 即是 R_1 前件状态模式中的状态名变量；第 4 个元素是变量 ?ns 的值，由 R_1 前件的第 2 个模式，可知 ?ns 的值是农夫所在河岸变量 ?fs 值相对的河岸，也就是说，在产生的这个新的状态事实中，农夫已过渡到他原来所在河岸的对岸；第 5、6、7 个元素分别是狐狸所在河岸变量 ?xs 的值、山羊所在河岸变量 ?gs 的值、白菜所在河岸变量 ?cs 的值，这 3 个元素即是 R_1 前件状态模式中相应位置的变量名，也就是说，由执行 R_1 产生的新

状态事实中, 狐狸、山羊和白菜仍留在原来所在河岸。由此可见, 规则 R_1 自含地表示出农夫单独过河导致的状态转移。

规则 R_2 是农夫带狐狸过河规则 `move-with-fox`。 R_2 前件的状态模式中的第 4 个元素是父节点状态名, 这里, 采用了单域通配符?, 因此, 在模式匹配时, 工作存储器中任何一个状态事实的状态名都可作为父节点状态名进行匹配。但是, R_2 前件状态模式中的第 5 个元素和第 6 个元素同为?fs, 即要求可匹配的状态事实中, 农夫所在河岸与狐狸所在河岸有相同的值。通过模式匹配, 若 R_2 被激发, 则执行 R_2 后件的活动, 把一个新状态事实添加到工作存储器中。在新的状态事实中, 农夫和狐狸都到达由变量?ns 的值指定的河岸, 而变量?ns 的值是同农夫和狐狸原所在河岸?fs 值相对的河岸。

规则 R_3 是农夫带山羊过河规则 `move-with-goat`, 类似 R_2 , 增加到工作存储器的新状态事实中, 农夫和山羊都到达?ns 的值指定河岸, 而?ns 的值是同农夫和山羊原所在河岸?fs 值相对的河岸。

规则 R_4 是农夫带白菜过河规则 `move-with-cabbage`, 类似 R_2 , 也增加一个新状态事实到工作存储器中。

规则 R_5 是狐狸吃山羊规则 `fox-eats-goat`。 R_5 前件的第 1 个模式是一个规则优先数说明模式, 为 R_5 设定的优先数为 8000。第 2 个模式是定义一个模式变量?rm 为状态模式, 在这个状态模式中, 农夫所在河岸变量为?s1; 狐狸所在河岸变量为?s2, 且约束?s2 的值与?s1 的值不同; 山羊所在河岸变量也为?s2, 即山羊与狐狸在同一河岸; 白菜所在河岸变量为一个单域通配符?, 即在模式匹配时不论白菜在什么地方都可匹配。通过模式匹配, 若 R_5 被激发, 则执行 R_5 后件的活动语句 (`retract ?rm`), 从工作存储器中删除所有与模式变量?rm 指定的状态模式匹配的状态事实。由于与?rm 指定状态模式可匹配的状态事实中, 狐狸与山羊在同一河岸, 而农夫在对岸, 会发生狐狸吃山羊, 因此, 必需把这样的状态事实从工作存储器中删除。 R_5 的优先数设定为 8000, 因此, 当 R_5 与 $R_1 \sim R_4$ 都有可与当前工作存储器中的事实匹配时, 将优先激发 R_5 , 删除与问题求解相悖的状态事实。

规则 R_6 是山羊吃白菜规则 `goat-eats-cabbage`。类似 R_5 , 由一个规则优先数说明模式设定 R_6 的优先数为 8000。赋给模式变量?rm 的状态模式中, 农夫所在河岸变量为?s1; 狐狸所在河岸变量为单域通配符?; 山羊所在河岸变量为?s2, 且约束?s2 的值与?s1 的值不同; 白菜所在河岸变量为?s2。即山羊与白菜在同一河岸, 而农夫不在该河岸。

规则 R_7 的规则名是 `circular-path`, 它将从工作存储器中删除重复的状态事实。这些重复的状态事实中, 农夫所在河岸变量?fs、狐狸所在河岸变量?xs、山羊所在河岸变量?gs 和白菜所在河岸变量?cs 都分别有相同的值, 对这些状态事实只保留深度值最小的一个状态事实。由于规则 R_7 有较高的优先数, 因此, 一旦在工作存储器中增加了深度值、状态名和父节点状态名不同, 但?fs、?xs、?gs 和?cs 的值相同的状态事实, 就会由规则 R_7 将新的状态事实删除。

规则 R_8 是结束推理过程的规则 `reasoning-end`。规则 R_8 具有最高优先数, 因此, 若工作存储器中, 一旦出现这样的状态事实, 不论状态的深度值、状态名和父节点状态名是什么,

只要农夫、狐狸、山羊和白菜所在河岸都是 shore_2 ，那么，就执行 R_8 后件的 `return` 语句，结束推理过程。

由此例可见，CLIPS 的规则具有产生式规则表示形式和自含性特点。但是，在 CLIPS 规则的前件中，引入了可含变量域的模式概念；在规则的后件中，引入了可表示活动的语句，从而使 CLIPS 具有更强的知识表示能力和推理能力。

2. CLIPS 的推理机制

CLIPS 的推理过程就是不断地通过模式匹配来寻找可用规则，激发可用规则来修改工作存储器中的内容的过程。若规则前件的每一模式均能在当前工作存储器中找到可匹配的事实，那么，这条规则就是一条可用规则。若有多条可用规则，则选择优先数最高的可用规则激发，执行激发规则后件的活动来修改工作存储器的内容，直至推理过程结束。因此，CLIPS 推理机制中最重要的是模式匹配。

规则前件的一个模式是含有变量域的有序域表，而一个事实是只含常量域的有序域表，因此，在模式匹配的操作中，要把模式中的变量用可匹配事实的相应常量来代换。由于一条规则的前件可有多个模式，因此，在选择可用规则的模式匹配中，将确定可与多个模式相匹配而使用的通代 σ ，即对模式之间的同一变量能取得统一的约束值。

若存在一个通代 σ ，使得规则

$$\text{if } P_1, P_2, \dots, P_m \text{ then } A_1, A_2, \dots, A_n$$

的每一个模式 P_i ，在工作存储器中有一个事实 W_i 满足

$$P_i \sigma = W_i \quad i=1,2,\dots,m$$

那么，这个规则是一条可用规则。 σ 作用于某个模式的结果 $P_i \sigma$ 称为这个模式的一个模式实例， σ 作用于整个规则的结果称为这个规则的一个规则实例。

一条可用规则被激发，实际上执行的是这条规则相应的规则实例，即这条规则的后件活动中的变量先被 σ 指定的常量代换成为规则实例后再执行。

我们仍以农夫过河问题为例来说明 CLIPS 的推理机制。

例 5.6 说明例 5.5 给出的求解农夫过河问题的 CLIPS 程序的推理过程。

程序在工作存储器 WS 中设置的初始事实有：

$$WS_0 = (S_0 \ L_1 \ L_2)$$

$$S_0 = (\text{status } 0 \ n_0 \ \text{no-parent} \ \text{shore1} \ \text{shore1} \ \text{shore1} \ \text{shore1})$$

$$L_1 = (\text{opposite-of} \ \text{shore1} \ \text{shore2})$$

$$L_2 = (\text{opposite-of} \ \text{shore2} \ \text{shore1})$$

对工作存储器 WS 中的初始事实的集合 WS_0 通过模式匹配进行推理的过程，可以用图 5.7 所示的一个有向图来表示。图中的节点表示工作存储器中当前事实的集合 WS_i ，有向弧表示由当前工作存储器激发的规则 R_j 。

对图 5.7 所示有向图的生成过程说明如下。

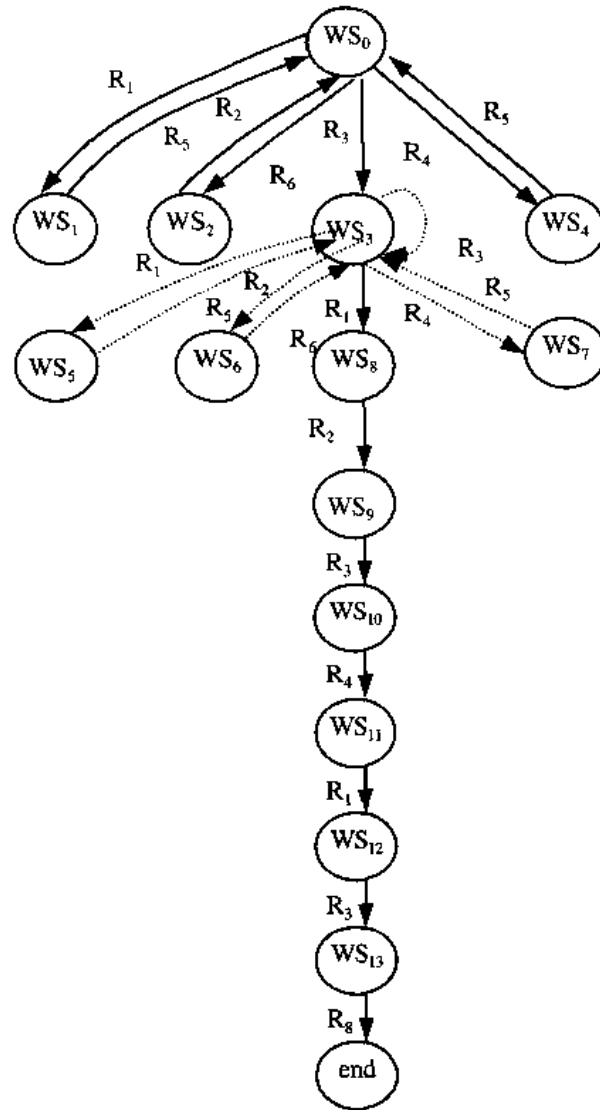


图 5.7 例 5.6 的推理过程有向图

1) 由于 $R_1 \sim R_4$ 都可与 S_0 和 L_1 匹配, 故 $R_1 \sim R_4$ 都是可用规则, 且优先数相同, 因此, 规则的激发有下述 4 种选择:

① R_1 与 S_0 和 L_1 模式匹配, 有通代 $\sigma = (?num/0 \quad ?name/n_0 \quad ?fs/shore1 \quad ?xs/shore1 \quad ?gs/shore1 \quad ?cs/shore1 \quad ?ns/shore2)$, 对 WS 增加状态事实 S_1 , 使

$$WS = WS_1 = (S_0 \quad L_1 \quad L_2 \quad S_1)$$

$$S_1 = (\text{status} \quad 1 \quad n_1 \quad n_0 \quad \text{shore2} \quad \text{shore1} \quad \text{shore1} \quad \text{shore1})$$

② R_2 与 S_0 和 L_1 模式匹配, 有通代 $\sigma = (?num/0 \quad ?name/n_0 \quad ?fs/shore1 \quad ?gs/shore1 \quad ?cs/shore1 \quad ?ns/shore2)$, 对 WS 增加状态事实 S_2 , 使

$$WS = WS_2 = (S_0 \quad L_1 \quad L_2 \quad S_2)$$

$$S_2 = (\text{status} \quad 1 \quad n_2 \quad n_0 \quad \text{shore2} \quad \text{shore2} \quad \text{shore1} \quad \text{shore1})$$

③ R_3 与 S_0 和 L_1 模式匹配,有通代 $\sigma = (?num/0 \quad ?name/n_0 \quad ?fs/shore1 \quad ?xs/shore1 \quad ?cs/shore1 \quad ?ns/shore2)$,对 WS 增加状态事实 S_3 ,使

$$WS = WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$$

$$S_3 = (\text{status} \quad 1 \quad n_3 \quad n_0 \quad \text{shore2} \quad \text{shore1} \quad \text{shore2} \quad \text{shore1})$$

④ R_4 与 S_0 和 L_1 模式匹配,有通代 $\sigma = (?num/0 \quad ?name/n_0 \quad ?fs/shore1 \quad ?xs/shore1 \quad ?gs/shore1 \quad ?ns/shore2)$,对 WS 增加状态事实 S_4 ,使

$$WS = WS_4 = (S_0 \quad L_1 \quad L_2 \quad S_4)$$

$$S_4 = (\text{status} \quad 1 \quad n_4 \quad n_0 \quad \text{shore2} \quad \text{shore1} \quad \text{shore1} \quad \text{shore2})$$

2)对于 $WS_1 \sim WS_4$,由于 $R_1 \sim R_2$ 可再次与 S_0 和 L_1 匹配,而使 $R_1 \sim R_4$ 为可用规则,若激发 $R_1 \sim R_4$,则将对 $WS_1 \sim WS_4$ 分别增加状态事实 S_1' 、 S_2' 、 S_3' 和 S_4' ,且 $S_1' = S_1$, $S_2' = S_2$, $S_3' = S_3$, $S_4' = S_4$ 。但是,对 WS_1 、 WS_2 和 WS_4 有优先数较高的可用规则 R_5 和 R_6 ,故应选择 R_5 和 R_6 。

①对 $WS_1 = (S_0 \quad L_1 \quad L_2 \quad S_1)$ 有规则 R_5 可与 S_1 匹配,有通代 $\sigma = (?name/n_1 \quad ?s1/shore2 \quad ?s2/shore1)$,删除 WS_1 中的状态事实 S_1 ,使

$$WS = WS_0 = (S_0 \quad L_1 \quad L_2)$$

②对 $WS_2 = (S_0 \quad L_1 \quad L_2 \quad S_2)$ 有规则 R_6 可与 S_2 匹配,有通代 $\sigma = (?name/n_2 \quad ?s1/shore2 \quad ?s2/shore1)$,删除 WS_2 中的状态事实 S_2 ,使

$$WS = WS_0 = (S_0 \quad L_1 \quad L_2)$$

③对 $WS_4 = (S_0 \quad L_1 \quad L_2 \quad S_4)$ 有规则 R_5 可与 S_4 匹配,有通代 $\sigma = (?name/n_4 \quad ?s1/shore2 \quad ?s2/shore1)$,删除 WS_4 中的状态事实 S_4 ,使

$$WS = WS_0 = (S_0 \quad L_1 \quad L_2)$$

3)由于 $WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$ 中的 S_3 既不能与 R_5 匹配,也不能与 R_6 匹配,因此不能优先选择 R_5 和 R_6 。

① R_1 可再次与 S_0 和 L_1 匹配,对 WS_3 增加 S_1 ,使 $WS = WS_5 = (S_0 \quad L_1 \quad L_2 \quad S_3 \quad S_1)$ 。对 WS_5 将优先选择 R_5 来删除 S_1 ,使 $WS = WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$ 。

② R_2 可再次与 S_0 和 L_1 匹配,对 WS_3 增加 S_2 ,使 $WS = WS_6 = (S_0 \quad L_1 \quad L_2 \quad S_3 \quad S_2)$ 。对 WS_6 将优先选择 R_6 来删除 S_2 ,使 $WS = WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$ 。

③ R_3 可再次与 S_0 和 L_1 匹配,对 WS_3 增加 S_3' ,且 $S_3' = S_3$,故 $WS = WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$ 。

④ R_4 可再次与 S_0 和 L_1 匹配,对 WS_3 增加 S_4 ,使 $WS = WS_7 = (S_0 \quad L_1 \quad L_2 \quad S_3 \quad S_4)$ 。对 WS_7 将优先选择 R_5 来删除 S_4 ,使 $WS = WS_3 = (S_0 \quad L_1 \quad L_2 \quad S_3)$ 。

由①~④可见,由于允许规则再次与工作存储器中同样的事实匹配来选择可用规则,尽管有优先数较高的规则(R_5 和 R_6)来删除工作存储器中的冗余事实,但是,增加了推理的时空开销,甚至可能导致推理过程陷入无限循环的危险境地,因此,需要禁止目标规则对工作存储器再次匹配相同的事实子集而被激发,不再次产生相同的规则实例。

在图 5.7 所示的有向图中,用虚线有向弧表示①~④的推理。以下的推理将不考虑再次产

生相同规则实例的推理。

⑤ R_1 可与 $WS_3 = (S_0 \ L_1 \ L_2 \ S_3)$ 中的 S_3 和 L_2 匹配, 有通代 $\sigma = (?num/1 \ ?name/n_3 \ ?fs/shore2 \ ?xs/shore1 \ ?gs/shore2 \ ?cs/shore1 \ ?ns/shore1)$, 对 WS 增加状态事实 S_5 , 使

$$WS = WS_8 = (S_0 \ L_1 \ L_2 \ S_3 \ S_5)$$

$$S_5 = (\text{status } 2 \ n_5 \ n_3 \ \text{shore1} \ \text{shore1} \ \text{shore2} \ \text{shore1})$$

4) R_2 可与 $WS_8 = (S_0 \ L_1 \ L_2 \ S_3 \ S_5)$ 中的 S_5 和 L_1 匹配, 有通代 $\sigma = (?num/2 \ ?name/n_5 \ ?fs/shore1 \ ?gs/shore2 \ ?cs/shore1 \ ?ns/shore2)$, 对 WS 增加状态事实 S_6 , 使

$$WS = WS_9 = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6)$$

$$S_6 = (\text{status } 3 \ n_6 \ n_5 \ \text{shore2} \ \text{shore2} \ \text{shore2} \ \text{shore1})$$

5) R_3 可与 $WS_9 = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6)$ 中的 S_6 和 L_2 匹配, 有通代 $\sigma = (?num/3 \ ?name/n_6 \ ?fs/shore2 \ ?xs/shore2 \ ?cs/shore1 \ ?ns/shore1)$, 对 WS 增加状态事实 S_7 , 使

$$WS = WS_{10} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7)$$

$$S_7 = (\text{status } 4 \ n_7 \ n_6 \ \text{shore1} \ \text{shore2} \ \text{shore1} \ \text{shore1})$$

6) R_4 可与 $WS_{10} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7)$ 中的 S_7 和 L_1 匹配, 有通代 $\sigma = (?num/4 \ ?name/n_7 \ ?fs/shore1 \ ?xs/shore2 \ ?gs/shore1 \ ?ns/shore2)$, 对 WS 增加状态事实 S_8 , 使

$$WS = WS_{11} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7 \ S_8)$$

$$S_8 = (\text{status } 5 \ n_8 \ n_7 \ \text{shore2} \ \text{shore2} \ \text{shore1} \ \text{shore2})$$

7) R_1 可与 $WS_{11} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7 \ S_8)$ 中的 S_8 和 L_2 匹配, 有通代 $\sigma = (?num/5 \ ?name/n_8 \ ?fs/shore2 \ ?xs/shore2 \ ?gs/shore1 \ ?cs/shore2 \ ?ns/shore2)$, 对 WS 增加状态事实 S_9 , 使

$$WS = WS_{12} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7 \ S_8 \ S_9)$$

$$S_9 = (\text{status } 6 \ n_9 \ n_8 \ \text{shore1} \ \text{shore2} \ \text{shore1} \ \text{shore2})$$

8) R_3 可与 $WS_{12} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7 \ S_8 \ S_9)$ 中的 S_9 和 L_1 匹配, 有通代 $\sigma = (?num/6 \ ?name/n_9 \ ?fs/shore1 \ ?xs/shore2 \ ?cs/shore2 \ ?ns/shore2)$, 对 WS 增加状态事实 S_{10} , 使

$$WS = WS_{13} = (S_0 \ L_1 \ L_2 \ S_3 \ S_5 \ S_6 \ S_7 \ S_8 \ S_9 \ S_{10})$$

$$S_{10} = (\text{status } 7 \ n_{10} \ n_9 \ \text{shore2} \ \text{shore2} \ \text{shore2} \ \text{shore2})$$

9) 对 WS 的当前内容 WS_{13} , 由于具有最高优先数的规则 R_8 可与 S_{10} 匹配, 故执行 R_8 的后件活动 return 结束推理过程。

工作存储器 WS 中的目标状态事实 S_{10} 的第 3 个域是状态名且值为 n_{10} , 第 4 个域是父状态名且值为 n_9 , 可在 WS 中找到状态名为 n_9 的事实 S_9 ; 再由 S_9 的父状态名 n_8 在 WS 中找到事实 S_8 ; ……直至找到初始状态事实 S_0 , S_0 的第 4 个域的值是 no-parent, 故终止回溯查找,

由此, 得出推理求解的路径为:

$$n_0 \rightarrow n_3 \rightarrow n_5 \rightarrow n_6 \rightarrow n_7 \rightarrow n_8 \rightarrow n_9 \rightarrow n_{10}$$

若在状态事实中增加一个常量域来记录推理使用的规则名, 则可得出由使用的规则名序列表示的问题的解为:

$$R_3, R_1, R_2, R_3, R_4, R_1, R_3, R_8$$

即是:

```
move - with - goat
move - alone
move - with - fox
move - with - goat
move - with - cabbage
move - alone
move - with - goat
reasoning - end
```

5.7 分布式专家系统与协同式专家系统

随着高性能计算机与计算机网络技术的发展, 为分布式专家系统和协同式专家系统的研究与实现提供了开发平台。分布式专家系统和协同式专家系统能提高规模较大的专家系统的运行速度和实现多专家的知识共享与协同工作。

5.7.1 分布式专家系统及其驱动方式

分布式专家系统 (DES, Distributed Expert System) 是具有分布处理特征的专家系统。一般来说, 规模越大的专家系统对问题进行推理求解所需时间也越长, 因此, 可以把一个专家系统的功能分解为若干个任务程序分配到多个处理机上并行工作, 以缩短专家系统问题求解的时间。分布式专家系统的运行环境可以是紧耦合的多处理机系统, 也可以是松耦合的计算机网络系统。

为了设计和实现一个分布式专家系统, 主要需要考虑以下的问题。

1. 功能分布与知识分布

设计分布式专家系统需要把专家系统的功能合理分解为若干个任务, 并把多个任务分配到多个处理机上并行执行, 共同完成问题求解。一个任务在一台处理机上执行的时间开销主要由两部分组成, 一部分是任务用于推理求解的时间 R , 另一部分是任务用于与其他处理机上的任务之间交换信息的通信时间 C , 称比值 R/C 为任务的粒度 (Granularity)。如果可为分布式专家系统提供更多的处理机, 那么, 希望把问题求解的功能分解成更多的任务, 使每个任务的 R 更小; 但是, 这样可能使一个任务需要与更多处理机上的任务进行通信, 从而使

任务的通信时间开销 C 增大, 反而可能使总的问题求解的时间增大。因此, 在功能分解时, 需要根据多处理机系统或计算机网络系统的结构与性能来确定合理的任务粒度 R/C , 并不是说, 分解成的任务越多, 任务粒度越细, 分布式专家系统的效率就越高。

另外, 支持任务程序进行推理的知识也需要经过合理划分, 并分配到多个处理机的本地存储器中。一个处理机上的推理程序访问这个处理机本地存储器中存储的知识所需的时间, 比采用通信方式访问其他处理机的远程存储器中存储的知识所需的时间要小得多, 因此, 应根据功能的分布来设计知识的分布。确定知识的分布时, 一方面要考虑尽量减少知识的冗余, 以避免由于知识更新可能引起的知识不一致性, 或者为了维护冗余知识的一致性而需要增加过大的系统维护开销; 另一方面, 也要考虑需要一定的知识冗余来支持各任务程序推理的需求, 以尽量减少任务对远程存储器访问的通信时间开销。

可见, 为获得分布式专家系统的最佳性能, 需要综合权衡功能的分布与知识的分布。

2. 驱动方式

系统中各任务模块以什么方式来驱动是分布式专家系统设计中需要研究的一个问题。常见的驱动方式有以下几种。

(1) 控制驱动

控制驱动方式是根据问题求解过程的需要, 当需要某个任务模块工作时, 就直接将控制转到它, 或将它作为一个过程直接调用它, 使它立即工作。这种驱动方式最常用, 实现方便。但是, 由于被驱动模块是被动地等待驱动命令, 即使其运行条件已经具备, 若无其他模块来的驱动命令, 它自身不能自动开始工作, 因此, 这种驱动方式的并行性较差, 并行推理的效率较低。

(2) 数据驱动

任务模块都是对指定的输入数据进行处理, 然后给出相应的输出数据。数据驱动方式是指任何一个任务模块只要当它所需的所有输入数据已经具备后即可自行启动工作, 然后, 把输出结果送到各自应该送去的模块。数据驱动方式不需要等待来自其他模块的驱动命令, 因此, 可以发掘多个任务模块之间可能的并行性, 提高并行推理的效率。在这种驱动方式下, 各任务模块之间只有通过相互传送消息来传送数据的联系, 其他操作都局限于模块内进行, 具有面向对象的工作特征。

(3) 要求驱动

要求驱动也称为目标驱动, 当一个任务模块的输出被其他任务模块需要时才被启动执行。要求驱动是一种自顶向下的驱动方式, 从最顶层的目标开始, 为了驱动一个目标工作可能需要先驱动若干子目标, 为了驱动各个子目标, 可能又要分别驱动一些子子目标, 如此层层驱动下去。与此同时, 又按数据驱动的原则让输入数据已经具备的模块进行工作, 输出相应的结果并送到各自应该送去的模块。这样, 把对输出结果的要求和其输入数据的齐备两个条件组合起来作为驱动模块的条件, 既可实现较高的并行性, 又可避免单纯数据驱动方式可能出现不根据需求而盲目产生很多暂时用不上的数据的“数据积压”问题。在数据驱动方式

中, 需要把积压的数据放入一个调度队列中等待调度使用。

(4) 事件驱动

事件驱动是比数据驱动更为广义的一个概念。一个模块的输入数据齐备可认为是一种事件, 还可以有其他各种事件, 例如某些条件得到满足或某个物理事件发生等。采用事件驱动方式时, 各个模块都要规定使它开始工作所必需的一个事件集合。所谓事件驱动方式即是当且仅当模块的相应事件集合中的所有事件都已发生时, 才能驱动该模块开始工作; 只要其中有一个事件尚未发生, 模块就要等待, 即使模块的输入数据已经齐备也不能被启动。由于可以把一个模块的输入数据齐备和该模块的输出被其他模块需要都定义为这个模块的事件, 因此, 事件驱动是把数据驱动和要求驱动均包含在内的一种更广义的驱动方式。

5.7.2 协同式专家系统及其协同方法

一般而言, 一个专家系统是应用于某个专业领域对某一类问题求解, 因此, 单个专家系统的应用有较大的局限性。协同式专家系统 (CES, Cooperative Expert System) 是能综合若干个相近领域或一个领域的多个方面的分专家系统, 互相协作共同解决一个更广领域中的问题的专家系统。协同式专家系统亦称为“群专家系统”。在现实世界中, 依靠多个专家系统的协同工作来求解一个复杂的问题的需求是很多的。协同式专家系统与分布式专家系统有一些共性, 协同式专家系统通常采用计算机网络作为开发平台, 组成协同式专家系统的各个分专家系统是分布的。但是, 协同式专家系统更强调的是各分专家系统之间的协同合作, 而不是更看重处理的分布和知识的分布。协同式专家系统旨在研究分散的、松耦合的一组知识处理实体或处理机节点协同进行问题求解的方法。这种知识处理实体称为智能体 (Agent), 每个智能体具有自主性, 并根据自己具有的知识和周围发生的事件进行推理、规划与通信。智能体之间彼此在逻辑上相互独立, 通过共享知识和协同工作形成问题的解决方案。多个智能体是分散的, 意味着智能体的控制、数据和知识在逻辑上甚至在地域上是分布的, 既没有全局控制, 也没有全局的数据和知识的存储器。智能体之间是松耦合的, 意味着智能体的任务粒度是粗粒度的, 各智能体更多的时间用于计算和推理, 用于通信的时间则相对较少。

为了设计和实现一个协同式专家系统, 主要需要考虑以下的问题。

1. 协同方法

由于单个智能体仅从局部利益出发来求得局部解, 因此需要对多个智能体共同完成全局任务、求得并改进全局解的协同方法研究。目前, 已经提出了下述三种协同方法。

(1) 多智能体规划 (Multi-agents Planning) 方法

多智能体规划方法需要设计一个规划智能体, 由它进行整体规划, 形成一个多智能体问题求解方案。它描述了每个智能体应采取的行动和目标, 然后, 由规划智能体将这一规划分布到各智能体按规划要求的行动执行, 对指定的目标求解。经过事先规划的局部解综合为全局解。

(2) 功能化有效协同 (Functionally Effective Cooperative Approach) 方法

各智能体根据各自问题求解的功能需求与有关智能体交换部分结果,并最终生成一个全局解。

(3) 谈判 (Negotiation) 方法

智能体将自己的局部解公布出去,按照事先设计的一个谈判协议,对相关的局部解经过裁决等综合处理后求得全局解。

为了支持协同工作,智能体的设计通常还应考虑下述问题:

- 1) 让强壮的智能体不受干扰地形成自己的局部解,或者说,智能体之间的协同不是对等的,这样可以增加局部解的多样性,有利于改进全局解。
- 2) 让多个智能体采用不同的推理方法求解同一问题,有利于提高全局解的可信度。
- 3) 把重要的任务分配给几个智能体同时求解,可以避免某些智能体求解失败而导致全局失败。
- 4) 智能体应具有交换信息的选择能力,智能体的协同应使智能体等待的时间尽可能小。

2. 知识的组织与分布

组成一个协同式专家系统的多个智能体拥有的知识有共性的知识,也有个性的知识。为了避免知识冗余带来知识一致性维护的困难,通常需要把共性的知识分离出来存储在一个公共知识库中,供各智能体共享。

由于每个智能体可以有自己特殊的知识表示方法,因此,智能体必须具备将其他智能体的知识和共性知识转化为自身知识表示的能力。当然,如果所有的智能体采取统一的知识表示方法,那么,知识共享的效率会高得多。

知识可以分为静态知识和动态知识两种。静态知识是指系统开始运行时加载到各智能体中的知识;动态知识是指在问题求解过程中在智能体之间进行传送的知识。静态知识是各智能体的个性知识,动态知识是支持多智能体协同工作所需要的知识。

通信是智能体之间实现知识共享、协同求解问题的主要手段,通信需要占用智能体的时间开销。因此,与分布式专家系统中讨论的情况类似,需要对分配到各智能体上的任务进行合理分解,使之具有合理的任务粒度,并由此确定共性知识与个性知识的划分、静态知识与动态知识的划分。一般来说,协同式专家系统的任务粒度是粗粒度的,任务粒度太细,可能会影响智能体的功能独立性和个性知识的模块性,而且可能由于智能体之间相互过多的依赖而需要更多的通信开销,从而影响协同工作的效率,甚至可能得出非期望的结果。

3. 裁决方法

(1) 是非问题的裁决方法

若多个智能体对同一个是非问题产生多个局部解,则可采用下述方法对多个局部解进行裁决处理,得出全局解。

- 1) 表决法,或称为简单多数法,即以多数智能体的解作为全局解。
- 2) 加权平均法。根据各智能体解决该问题的权威程度赋给不同的权值 W_i ,满足

$\sum_i w_i = 1$ 。设 A_i 是智能体 i 对该问题的解, 若 $A_i=1$, 表示问题的解为“是”; 若 $A_i=0$, 表示问题的解为“非”。全局解为加权平均 $\sum_i w_i A_i$, 若 $\sum_i w_i A_i > 1/2$, 则问题的全局解为“是”; 若 $\sum_i w_i A_i < 1/2$, 则问题的全局解为“非”; 若 $\sum_i w_i A_i = 1/2$, 则不能裁决, 需要辅以其他裁决方法。

(2) 评分问题的裁决方法

若多个智能体对同一个问题产生的局部解 A_i 是对这个问题的评分, 则可采用下述方法对多个局部解进行裁决处理。

1) 加权平均法。用加权平均 $\sum_i w_i A_i$ 作为最后的评分。

2) 取中数法。先把各智能体的评分按大小排序, 然后取位于中间的评分 (若有奇数个评分) 或者取中间 2 个评分的平均值 (若有偶数个评分) 作为最后的评分。

(3) 互补问题的裁决方法

若多个智能体的任务是互补的, 即各自解决求解问题的一个部分, 那么, 多个局部解的综合就是全局解。全局解中某个局部解直接来自相应的“主管智能体”, 无需仲裁。

4. 驱动方式

与分布式专家系统讨论的情况类似, 在分布式专家系统中讨论的几种驱动方式都可在协同式专家系统中使用。

5. 系统结构

由多个智能体组成一个协同式专家系统, 可以有以下几种结构形式。

(1) 主从式

整个系统有一个主智能体和若干个从智能体。由主智能体把任务分配给从智能体, 并统

的问题给出各自的局部解，然后由一个综合智能体通过综合所有的局部解给出问题的全局解。

(5) 网络式

根据智能体协同求解问题的“与”“或”关系，把系统表示成以智能体为节点的与/或图，其中有一个节点仅有输出弧，称为开始节点。用户提出的问题总是从开始节点智能体进入系统，然后沿有向弧的走向不断请求驱动各后继节点智能体进行工作。

Smith 曾给出一种协同式专家系统模型，称为契约网模型。契约网模型的基本思想是：获得求解问题的某个智能体发布一个告示，把问题广播到其他智能体，其他智能体根据各自求解问题的能力投标，该智能体检查它们的投标，决定中标者，把任务分配给相应中标者来完成。在问题求解过程中，某一签约方求解失败后，可由管理方再次广播这一任务的告示，与新的中标者签定新的契约，直到问题得到求解。在契约网模型中，智能体在不同的求解问题中分别扮演管理方和签约方。管理方负责监督任务的执行情况，并对执行的结果进行处理，即根据任务分解方式及该任务多个签约方的求解情况，将多个签约方的局部解综合成一个完整的全局解。签约方负责中标任务的执行，给出该任务的局部解。智能体并非先验地被指定是管理方还是签约方，而是根据问题提交和任务分解的不同情况动态地扮演不同的角色。

在契约网模型中，每个知识源反映了某一个特定领域或某个方面的专家知识和经验。静态知识在系统初始化时被加载到各个相应的智能体上，动态知识在问题求解过程中被动态地发布和传送。动态知识的动态分配可以采用下述三种方式：

- 1) 管理方智能体把签约方任务求解要求的知识直接传送给签约方智能体。

- 2) 把动态知识通过告示广播到其他智能体。

- 3) 投标智能体可在其投标中附加一个任务，该附加任务是申请特定知识以用于它将要承担的任务的求解。

消息是契约网中智能体之间进行通信的载体，用消息来表示智能体之间相互联系、协同工作所需的各种信息。契约网提供一种“公共内部节点语言”(Common Internode Language)用于消息的描述。在能实现点对点通信的智能体之间可以直接发送和接收消息；不能实现点对点通信的智能体之间可以通过一个“邮箱”来收发消息，每个智能体的消息收发均通过这个“邮箱”来完成。

契约网中，智能体之间的协同是通过任务的动态分解与分布来完成的。在分配任务时，契约网提供了一种机制专门用于搜索有能力、有时间来解决相应任务的智能体，以确定与哪一个智能体签约。

契约网模型为知识共享、协同工作以及提高全局解的可信度提供了一种良好的分布式问题求解模型。目前，分布式客户机/服务器(Client/Server)计算机系统已成为分布处理系统的主流，为协同式专家系统的研究和开发提供了强有力的支撑平台。随着 Client/Server 计算机系统的发展，DES 和 CES 也随之进入一个新的发展时期。

习题五

- 5.1 何谓专家系统？按照专家系统处理的问题类型，专家系统可分为哪几种类型？
- 5.2 简述专家系统的一般特点。
- 5.3 画出专家系统一般的组成框图，说明各组成部分的主要功能。
- 5.4 根据专家系统开发过程的瀑布模型，说明开发过程各阶段应完成的主要工作内容。
- 5.5 简述 LISP 语言的基本特点。
- 5.6 用 LISP 的非结构迭代方式定义一个计算前 n 个自然数的和的函数 $\text{sumup}(n)$ 。
- 5.7 用 LISP 的结构迭代方式定义一个计算 m^n 的函数 $\text{expt}(m,n)$ ，其中 m 和 n 都是自然数。
- 5.8 用 `prog` 函数定义函数 `our-member`，该函数的功能与函数 `member` 相同。
- 5.9 简述产生式系统的知识库与推理机能够相互独立的原因。
- 5.10 根据 5.3.2 节给出的正向推理机 `deduce` 有关函数的功能定义，用 C 语言改写有关函数，编制出正向推理机的 C 语言程序。
- 5.11 根据 5.3.2 节给出的反向推理机 `diagnose` 有关函数的功能定义，用 C 语言改写有关函数，编制出反向推理机的 C 语言程序。
- 5.12 对 2.10 题给出的猴子摘香蕉问题，把有关知识表示成产生式规则，并写入到规则库 `rules` 中，把初始状态事实写入到事实表 `facts` 中，然后调用正向推理机 `deduce` 进行正向推理。说明推理过程，并给出问题的解。
- 5.13 对 2.11 题给出的传教士与野人问题，把有关知识表示成产生式规则，并写入到规则库 `rules` 中，把初始状态事实写入到事实表 `facts` 中，然后调用正向推理机 `deduce` 进行正向推理。说明推理过程，并给出问题的解。
- 5.14 对 2.12 题给出的农夫过河问题，把有关知识表示成产生式规则，并写入到规则库 `rules` 中，把初始状态事实写入到事实表 `facts` 中，然后调用正向推理机 `deduce` 进行正向推理。说明推理过程，并给出问题的解。
- 5.15 对如图 4.5 所示的三阶梵塔问题，把有关知识表示成产生式规则，并写入到规则库 `rules` 中，把初始状态事实写入到事实表 `facts` 中，然后调用正向推理机 `deduce` 进行正向推理。说明推理过程，并给出问题的解。
- 5.16 对如图 5.4 所示的动物识别专家系统规则库，若最初赋给假设表 `hypo` 的值为 H_4 ，即 `hypo = (H4)`，且调用反向推理机 `diagnose` 进行反向推理。若用户在反向推理过程中只能确认事实 F_{15} 、 F_7 、 F_2 为真，那么，推理机能否推理出 H_4 为真的结论？若能够，请画出 H_4 的与/或解树。若用户只能确认事实 F_{15} 、 F_8 、 F_7 为真，那么，能否推出 H_4 为真的结论？为什么？
- 5.17 何谓元知识？元知识的主要用途有哪些？
- 5.18 什么是目标级推理？什么是元级推理？在什么情况下，元级推理可与目标级推理

共用一个推理机？在此情况下，可采用什么方法保证元规则优先执行？

5.19 简述专家系统解释机制的几种实现方法。

5.20 根据 5.3.2 节和 5.4.2 节给出的正向推理机 deduce 和 how 解释器的有关函数的功能定义，用 C 语言改写有关函数，编制 deduce 和 how 的 C 语言程序，构成一个正向推理的 C 语言骨架系统。在计算机上进行下述问题的求解，由 how 解释器给出问题的解。

- 1) 由 5.12 题的产生式规则库，求解猴子摘香蕉问题。
- 2) 由 5.13 题的产生式规则库，求解传教士与野人问题。
- 3) 由 5.14 题的产生式规则库，求解农夫过河问题。
- 4) 由 5.15 题的产生式规则库，求解三阶梵塔问题。

5.21 简述知识获取一般应完成的工作内容。

5.22 何谓自动知识获取？为实现自动知识获取，系统至少应具备哪些功能？

5.23 在规则库中，何谓冗余规则链？何谓矛盾规则链？何谓循环规则链？试分别举例说明。

5.24 如何检测和处理规则库中的冗余规则链？

5.25 如何检测和处理规则库中的循环规则链？

5.26 何谓知识求精？简述实现知识求精的一般方法。

5.27 何谓知识的组织？在确定知识的组织方式时应遵循哪些原则？

5.28 专家系统的生成工具可分为哪几类？试比较它们的特点。

5.29 写出求解 5.12 题给出的猴子摘香蕉问题的 CLIPS 程序，画出推理过程有向图，并给出问题的解。

5.30 写出求解 5.15 题给出的三阶梵塔问题的 CLIPS 程序，画出推理过程有向图，并给出问题的解。

5.31 何谓分布式专家系统？简述分布式专家系统的几种模块驱动方式。

5.32 何谓协同式专家系统？简述协同式专家系统的几种协同方法。

第6章 知识的不确定性与不确定推理

在各种实际应用领域中，严格精确的和确定的知识并不多见，大量的知识是不精确的和不确定的，需要采用不确定推理或称为不精确推理对不确定知识进行处理。可以说，不确定性（Uncertainty）是智能问题的本质特征，因此，智能系统的能力更主要反映在求解不确定性问题的能力上。

6.1 知识的不确定性

有时，人们难于用精确的概念来描述事物，例如，可以按身高把人分为“高”与“不高”两类，但是难以给出精确的度量来区分“高”与“不高”。用精确的推理去处理不精确的事实，有时会得出荒谬的结论。例如，“秃子”是一个模糊的概念，人们难以用有多少根头发来区分秃子或不是秃子。但是可以有一条精确的推理规则：比秃子多一根头发的人仍是秃子。如果我们反复地用这条精确的规则处理“秃子”这个不精确的概念，就会得出一个荒谬的结论：比秃子多 100 万根头发的人仍是秃子。这就是著名的“秃子悖论”。显然，对于不确定的知识要用不确定推理来处理。

专家系统中的不确定性表现在三个方面，第一是证据或事实的不确定性，第二是规则的不确定性，第三是推理的不确定性。

6.1.1 证据的不确定性

证据或事实的不确定性主要反映在以下方面。

（1）证据的歧义性

证据的歧义性是指证据可具有多种含义明显不同的解释，如果离开证据所在环境和证据的上下文，往往难以确定其真正的含义。例如，“同意总理开会”，一般理解为某人赞同总理去开会，但也可以理解为某人陪同意大利总理去开会。又如“I saw a man with a telescope”，既可理解为“我看见一个带着望远镜的人”，也可理解为“我用望远镜看见一个人”。

歧义的消除是机器翻译和自然语言理解要着重解决的问题。在专家系统中，证据的歧义性要在知识获取阶段中消除掉，通过与专家交谈来了解可能引起歧义的证据的确切含义，并用不会引起歧义的语言来表达证据。

（2）证据的不完全性

证据的不完全性有两方面的含义，一是证据尚未收集完全；二是证据的特征值不完全。任何一个专业领域的知识都是发展变化和不断积累的，因此，领域专家的大部分决策都是在

知识不完全的情况下作出的。例如，在经济预测中，市场信息瞬息万变，要获得全部完整的信息才作出决策几乎是不可能的。

（3）证据的不精确性

证据的不精确性是指证据表示的值与证据的真实值之间存在一定的差异。例如，某人的身高大约是 1.70 米，这里，“大约是 1.70 米”就是不精确的表示。这种证据的不精确性与因测量误差引起的数据不精确性不是同一概念，这种证据的不精确表示是智能问题的本质特征，这正是专家系统不确定推理的一个重要的研究对象。

（4）证据的模糊性

证据的模糊性是指证据的取值范围的边界是模糊的、不明确的。例如，“年轻”就是一个模糊的概念，很难具体指明哪一年龄层次的人为年轻人，其边界是不明确的。证据的模糊性也是智能问题的本质特征。

（5）证据的可信性

证据的可信性是指专家主观上对提出的证据的可靠性的信任程度。如果证据不是完全可信的，那么，在进行决策时，对这样的证据要打一定的折扣并经过综合处理后才能使用。

（6）证据的随机性

证据的随机性是指证据是随机出现的。例如，某张牌分发给某人是随机的。

证据的上述不确定性没有完全包括客观世界中不确定性所具有的丰富内涵，它们相互之间的区分也不是绝对的。

6.1.2 规则的不确定性

专家系统的知识库中包含大量的启发式知识，这些知识来源于领域专家处理问题的知识和经验。既然领域专家的知识 and 经验是不确定的，因而，知识库中的规则也就必然具有不确定性。

产生式规则的不确定性主要有以下几个方面。

（1）构成规则前件的模式的不确定性

例如，有一条产生式规则：“如果患者发高烧且常流清鼻涕，则患者感冒”。这条规则的前件有两个模式，即“发高烧”和“常流清鼻涕”，这两个模式都是模糊的概念，难以明确指明“发高烧”的体温值边界和发烧的时间值边界，也难以明确指明“清鼻涕”的鼻涕浓度值边界的“常流”的时间值边界。

（2）观察证据的不确定性

例如，患者发高烧的体温观察值并不是恒定的。晚上可能是 39°C ，早上可能是 38.5°C 。这种证据的观察值的不精确性被用于推理时，与“发高烧”的模式进行模式匹配的结果也就与模式的真实含义之间存在一定的差异。

（3）规则前件的证据组合的不确定性

例如，有一位患者的体温一直都是 40°C ，流清鼻涕，但并不是常流清鼻涕，那么，这两

个证据组合起来在多大程度上符合规则前件的条件,也包含着不确定性。

(4) 规则本身的不确定性

上述产生式规则是从发高烧且常流清鼻涕引出感冒的结论。实际上,判断一位患者是否患有感冒还会有其他的证据;另外,发高烧和常流清鼻涕也会是患有其他疾病的证据。因此,领域专家对这一条规则持有某种信任程度。也就是说,每一条规则并不都具有 100% 的信任程度,这就是规则的不确定性,或称为规则强度。

(5) 规则结论的不确定性

由于规则的前件包含有各种不确定性因素,运用不确定的规则,导出的结论也就不可避免地是不确定的。由于上述产生式规则引出的结论应是“患者可能感冒”,这是一个不确定的结论,但更符合领域专家的判断。

此外,在规则使用过程中,还有两种典型的使用规则的不确定性。第一种不确定性是:在推理过程中,若有多条规则可用时,则需要通过冲突消解从多条可用规则中选择一条规则激发。冲突消解策略包含有使用规则的不确定性。第二种不确定性是:在反向推理过程中,若有多个假设需要通过推理来验证时,先选择哪一个假设进行反向推理验证同样包含有使用规则的不确定性。使用规则的不确定性与使用规则的控制策略有关,它比一个证据、一条规则和一步推理所包含的不确定性更为复杂。

6.1.3 推理的不确定性

推理的不确定性是指:由于证据的不确定性和规则的不确定性在推理过程中的动态积累和传播从而导致推理结论的不确定性。因此,需要采用某种不确定性的测度,并能在推理过程中来传递和计算这种不确定性的测度,最终得到结论的不确定性测度。

在不确定推理中,不确定性测度的计算有以下三种基本的计算模式。

(1) 证据组合的不确定性测度计算模式

已知证据 $e_1, e_2, \dots, e_n$ 的不确定性测度为 $MU_1, MU_2, \dots, MU_n$, 求出逻辑组合的不确定性测度 MU 。

证据的逻辑组合有三种基本形式。因此,证据逻辑组合的不确定性测度分别为:

1) 证据的合取组合的不确定性测度。

$e_1, e_2, \dots, e_n$ 的合取组合为 $e_1 \wedge e_2 \wedge \dots \wedge e_n$, n 个证据合取组合的不确定性测度为 $MU=f(MU_1, MU_2, \dots, MU_n)$ 。

2) 证据的析取组合的不确定性测度。

$e_1, e_2, \dots, e_n$ 的析取组合为 $e_1 \vee e_2 \vee \dots \vee e_n$, n 个证据析取组合的不确定性测度为 $MU=g(MU_1, MU_2, \dots, MU_n)$ 。

3) 证据的否定的不确定性测度。

证据 e_i 的否定为 $\bar{e}_i$, 证据 e_i 的否定的不确定性测度为 $MU=n(MU_i)$ 。

更复杂的证据逻辑组合都可由上述三种基本组合来表示,其不确定性测度可由上述三种

基本组合的不确定性测度的计算得出。

(2) 并行规则的不确定性测度计算模式

已知有多条规则 if e_i then h 有相同的结论 h , 各条规则的不确定性测度为 $MU_i, i=1, 2, \dots, n$ 。若 n 条规则都被满足, 那么, 结论 h 的不确定性测度为 $MU=p(MU_1, MU_2, \dots, MU_n)$ 。

这一计算模式也称为并行法则。并行法则给出了推理过程中有多条路径导致同一结论的情况下, 由规则的不确定性而导致结论的不确定性测度的计算模式。

(3) 顺序(串行)规则的不确定性测度计算模式

已知两条规则 if e then e' 和 if e' then h 的规则不确定性测度分别为 MU_1 和 MU_2 , 那么, 规则 if e then h 的规则不确定性测度为 $MU=s(MU_1, MU_2)$

这一计算模式也称为顺序法则。顺序法则给出了规则不确定性在推理链中传播的计算模式。

对于一个专家系统, 一旦给定上述三种计算模式的不确定性测度计算方法, 即给出 f 、 g 、 n 、 p 、 s 的具体计算方法, 那么, 就可获得证据不同组合的不确定性测度值, 并根据在推理过程中使用规则的情况, 由并行法则和顺序法则最终得出结论的不确定性测度值。专家系统中的不确定性推理就是上述三种计算模式的组合。但是, 在不同的专家系统中, 不确定性测度的计算方法可以不同。根据不确定性测度计算方法的不同, 不确定推理可以有基于概率理论的不确定推理、基于可信度理论的不确定推理和基于模糊理论的不确定推理等。

6.2 基于概率的不确定推理

概率论是处理不确定性的经典理论。概率论是处理随机性事件发生的理论, 但是, 事件可以认为是论域的一个子集, 所以, 概率论能够处理集合。概率论中事实之间的关系和运算与集合论中集合之间的关系和运算是一致的。在不确定推理中, 我们把能使某个结论为真的证据组成一个集合, 把这样的集合也称为一个事件, 包含在一个事件集合中的元素都能使对应的结论为真, 那么, 就可以用事件发生的概率来描述和计算推理的不确定性测度。

6.2.1 有关概率的基本概念与计算

我们给出有关概率的几个基本概念和计算公式。

定义 6.1 事件 A 的对立事件记为 $\bar{A}$, $\bar{A}$ 是 A 的补集, 称 A 与 $\bar{A}$ 是互逆的事件。事件 A 和事件 B 至少有一个发生, 记为 $A \cup B$, $A \cup B$ 是 A 和 B 的并集。事件 A 和事件 B 同时发生, 记为 $A \cap B$, $A \cap B$ 是 A 和 B 的交集。事件 A 发生而事件 B 不发生, 记为 $A - B$, $A - B$ 是 A 和 B 的差集。

事件 $A_1, A_2, \dots, A_n$ 中至少有一个事件发生, 记为 $A_1 \cup A_2 \cup \dots \cup A_n$, 简记为 $\bigcup_{i=1}^n A_i$ 。事件 $A_1, A_2, \dots, A_n$ 同时发生, 记为 $A_1 \cap A_2 \cap \dots \cap A_n$, 简记为 $\bigcap_{i=1}^n A_i$, 或者记为 $A_1 A_2 \dots A_n$ 。

定义 6.2 若事件 A 和事件 B 没有相同元素, 即 A 和 B 的交集是一个空集, $A \cap B = \Phi$,

则称事件 A 和事件 B 互不相容。

定义 6.3 用 $P(A_1), P(A_2), \dots, P(A_n)$ 分别表示事件 $A_1, A_2, \dots, A_n$ 发生的概率, 若事件 $A_1, A_2, \dots, A_n$ 互不相容, 则事件 $A_1, A_2, \dots, A_n$ 中至少有一个事件发生的概率为:

$$P(A_1 \cup A_2 \cup \dots \cup A_n) = P(A_1) + P(A_2) + \dots + P(A_n) = \sum_{i=1}^n P(A_i) \quad (6-1)$$

(6-1) 式称为概率的有限可加性。

定义 6.4 事件 A 发生的概率为 $P(A)$, 事件 B 发生的概率为 $P(B)$ 。若在发生事件 A 的条件下发生事件 B 的概率为 $P(B|A)$, 则事件 A 和事件 B 同时发生的概率 $P(AB)$ 为:

$$P(AB) = P(A)P(B|A)$$

定义 6.5 若事件 B 有条件独立性, 即事件 B 的发生概率不依赖于事件 A 的发生, 即有 $P(B|A) = P(B)$, 那么, 有 $P(AB) = P(A)P(B)$, 则称事件 A 和 B 相互独立。

事件 $A_1, A_2, \dots, A_n$ 发生的概率分别是 $P(A_1), P(A_2), \dots, P(A_n)$, 若事件 $A_1, A_2, \dots, A_n$ 相互独立, 则事件 $A_1, A_2, \dots, A_n$ 同时发生的概率为:

$$P(A_1 A_2 \dots A_n) = P(A_1)P(A_2) \dots P(A_n) = \prod_{i=1}^n P(A_i) \quad (6-2)$$

定义 6.6 设 S 为随机试验 E 的样本空间, $B_1, B_2, \dots, B_n$ 为 E 的一组事件, 若有:

$$\textcircled{1} B_i B_j = \Phi \quad i \neq j$$

$$\textcircled{2} B_1 \cup B_2 \cup \dots \cup B_n = S$$

则称 $B_1, B_2, \dots, B_n$ 为样本空间 S 的一个划分。

由概率的定义, 可得出下述有关概率的性质和公式

性质 6.1 若 $\bar{A}$ 是事件 A 的对立事件, 则 $\bar{A}$ 的概率为:

$$P(\bar{A}) = 1 - P(A)$$

性质 6.2 (全概率公式) 设随机试验 E 的样本空间为 S, A 是 E 的一个事件, $B_1, B_2, \dots, B_n$ 是 S 的一个划分, 且 $P(B_i) > 0, i=1, 2, \dots, n$, 则事件 A 的概率为:

$$P(A) = P(A|B_1)P(B_1) + P(A|B_2)P(B_2) + \dots + P(A|B_n)P(B_n) = \sum_{i=1}^n P(A|B_i)P(B_i) \quad (6-4)$$

由条件概率的定义, 有:

$$P(B_i|A) = \frac{P(B_i A)}{P(A)} = \frac{P(A|B_i)P(B_i)}{P(A)}$$

又由全概率公式 (6-4) 式, 得:

$$P(B_i|A) = \frac{P(A|B_i)P(B_i)}{\sum_{j=1}^n P(A|B_j)P(B_j)} \quad (6-5)$$

(6-5) 式称为贝叶斯 (Bayes) 公式。

6.2.2 基于概率的不确定推理方法

在基于概率的不确定推理模型中, 证据 e 的不确定性用 e 发生的概率 $P(e)$ 表示, 产生式规则 “if e then h ” 的不确定性表示为在事件 e (证据) 已经发生的条件下、事件 h (结论) 发生的概率 $P(h|e)$ 。

若已知证据 e 发生的概率 $P(e)$ 和结论 h 发生的概率 $P(h)$ 以及结论 h 成立时证据 e 出现的概率 $P(e|h)$, 那么, 由贝叶斯公式 (6-5) 式可得证据 e 支持结论 h 的不确定性测度为:

$$P(h|e) = \frac{P(e|h)P(h)}{P(e)} \quad (6-6)$$

基于概率的不确定推理有两个基本假设:

1) 若一组论据 $e_1, e_2, \dots, e_n$ 同时支持结论 h , 则对于 h 和 $\bar{h}$, 证据之间互相独立。即有:

$$P(e_1 e_2 \dots e_n | h) = P(e_1 | h) P(e_2 | h) \dots P(e_n | h) = \prod_{i=1}^n P(e_i | h) \quad (6-7)$$

$$P(e_1 e_2 \dots e_n | \bar{h}) = P(e_1 | \bar{h}) P(e_2 | \bar{h}) \dots P(e_n | \bar{h}) = \prod_{i=1}^n P(e_i | \bar{h}) \quad (6-8)$$

2) 若一个证据 e 支持多个结论 $h_1, h_2, \dots, h_n$, 则结论 $h_1, h_2, \dots, h_n$ 之间互不相容。即有:

$$P(e | h_1 \cup h_2 \cup \dots \cup h_n) = P(e | h_1) + P(e | h_2) + \dots + P(e | h_n) = \sum_{i=1}^n P(e | h_i) \quad (6-9)$$

1. 证据组合的不确定计算

证据的逻辑组合有三种基本组合形式, 我们分别说明这三种基本形式的基于概率的不确定计算方法。

(1) 证据的否定的不确定计算

定理 6.1 若证据 e 的不确定性为 $P(e)$, 则证据 $\bar{e}$ 的不确定性为:

$$P(\bar{e}) = 1 - P(e) \quad (6-10)$$

(2) 证据合取组合的不确定计算

定理 6.2 若证据 $e_1, e_2, \dots, e_n$ 的不确定性分别为 $P(e_1), P(e_2), \dots, P(e_n)$, 则证据的合取组合 $e_1 \wedge e_2 \wedge \dots \wedge e_n$ 的不确定性为:

$$P(e_1 e_2 \dots e_n) = \prod_{i=1}^n P(e_i) \quad (6-11)$$

(3) 证据析取组合的不确定计算

定理 6.3 若证据 $e_1, e_2, \dots, e_n$ 的不确定性分别为 $P(e_1), P(e_2), \dots, P(e_n)$, 则证据的析取组合 $e_1 \vee e_2 \vee \dots \vee e_n$ 的不确定性为:

$$P(\bigcup_{i=1}^n e_i) = 1 - \prod_{i=1}^n (1 - P(e_i)) \quad (6-12)$$

(6-10) 式和 (6-11) 式是显然成立的。下面给出 (6-12) 式的证明:

$$\begin{aligned}
 P\left(\bigcup_{i=1}^n e_i\right) &= P\left(\bigcap_{i=1}^n \bar{e}_i\right) \\
 &= 1 - P\left(\bigcap_{i=1}^n \bar{e}_i\right) \\
 &= 1 - \prod_{i=1}^n P(\bar{e}_i) \\
 &= 1 - \prod_{i=1}^n (1 - P(e_i))
 \end{aligned}$$

上述证明过程中使用了 (6-10) 式和 (6-11) 式。

2. 并行规则的不确定计算 (并行法则)

定理 6.4 若证据 $e_1, e_2, \dots, e_n$ 同时支持结论 h , 且 $e_1, e_2, \dots, e_n$ 相对于 h 和 $\bar{h}$ 条件独立, 则证据 $e_1, e_2, \dots, e_n$ 同时支持结论 h 的几率为:

$$O(h|e_1 e_2 \dots e_n) = \prod_{i=1}^n \lambda(h, e_i) O(h) \quad (6-13)$$

其中,

$$\lambda(h, e_i) = \frac{P(e_i | h)}{P(e_i | \bar{h})} \quad (6-14)$$

是结论 h 关于证据 e_i 的似然率, $P(e_i|h)$ 是结论 h 成立时 e_i 发生的概率, $P(e_i|\bar{h})$ 是结论 h 不成立时 e_i 发生的概率。

$$O(h) = \frac{P(h)}{P(\bar{h})} = \frac{P(h)}{1 - P(h)} \quad (6-15)$$

是结论 h 的几率, $P(h)$ 是结论 h 成立的概率, $P(\bar{h})$ 是结论 h 不成立的概率。

$$\begin{aligned}
 \text{证 } O(h|e_1, e_2, \dots, e_n) &= \frac{P(h|e_1 e_2 \dots e_n)}{P(\bar{h}|e_1 e_2 \dots e_n)} \\
 &= \frac{\frac{P(e_1 e_2 \dots e_n | h) P(h)}{P(e_1 e_2 \dots e_n)}}{\frac{P(e_1 e_2 \dots e_n | \bar{h}) P(\bar{h})}{P(e_1 e_2 \dots e_n)}} \\
 &= \frac{P(e_1 e_2 \dots e_n | h) P(h)}{P(e_1 e_2 \dots e_n | \bar{h}) P(\bar{h})} \\
 &= \frac{P(e_1 | h) P(e_2 | h) \dots P(e_n | h) P(h)}{P(e_1 | \bar{h}) P(e_2 | \bar{h}) \dots P(e_n | \bar{h}) P(\bar{h})} \\
 &= \lambda(h, e_1) \lambda(h, e_2) \dots \lambda(h, e_n) O(h)
 \end{aligned}$$

$$= \prod_{i=1}^n \lambda(h, e_i) O(h)$$

由定理 6.4 可见, 在证据的独立性条件下, 有

$$\lambda(h, e_1 e_2 \cdots e_n) = \prod_{i=1}^n \lambda(h, e_i) \quad (6-16)$$

λ 可以看作是一个几率修改因子。若 $\lambda > 1$, 则后验几率 $O(h|e_1 e_2 \cdots e_n)$ 大于先验几率 $O(h)$, 表示相应证据支持结论 h ; 若 $\lambda < 1$, 则后验几率小于先验几率, 表示相应证据反对结论 h 。

3. 顺序规则的不确定计算 (顺序法则)

定理 6.5 若证据 e_1 支持中间结论 e_2 , e_2 支持结论 h , 且 e_1 、 h 相对于 e_2 、 $\bar{e}_2$ 条件独立, 即

$$\begin{aligned} P(e_1 h | e_2) &= P(e_1 | e_2) P(h | e_2) \\ P(e_1 h | \bar{e}_2) &= P(e_1 | \bar{e}_2) P(h | \bar{e}_2) \end{aligned}$$

则有:

$$P(h | e_1) = P(h | e_2) P(e_2 | e_1) + P(h | \bar{e}_2) P(\bar{e}_2 | e_1) \quad (6-17)$$

(6-17) 式亦称为不确定证据公式。

证 由全概率公式, 有:

$$P(h | e_1) = P(h | e_2 e_1) P(e_2 | e_1) + P(h | \bar{e}_2 e_1) P(\bar{e}_2 | e_1)$$

由贝叶斯公式和 e_1 、 e_2 相对于 h 的条件独立, 有:

$$P(h | e_2 e_1) = \frac{P(e_1 e_2 | h) P(h)}{P(e_1 e_2)} = \frac{P(e_1 | h) P(e_2 | h) P(h)}{P(e_1) P(e_2)}$$

证据 e_1 的发生概率不依赖于结论 h 的发生概率, 因而有:

$$P(e_1 | h) = P(e_1)$$

对 $P(e_2 | h)$, 由贝叶斯公式, 可改写为:

$$P(e_2 | h) = \frac{P(h | e_2) P(e_2)}{P(h)}$$

故而, 可有:

$$P(h | e_2 e_1) = P(h | e_2)$$

类似, 可有:

$$P(h | \bar{e}_2 e_1) = P(h | \bar{e}_2)$$

代入到全概率公式的表达式中, 可得顺序法则的计算公式为:

$$P(h | e_1) = P(h | e_2) P(e_2 | e_1) + P(h | \bar{e}_2) P(\bar{e}_2 | e_1)$$

(6-17) 式给出顺序使用两条规则:

$$\begin{aligned} &\text{if } e_1 \text{ then } e_2 \\ &\text{if } e_2 \text{ then } h \end{aligned}$$

的结论 h 的不确定计算方法。

我们通过一个例子来进一步说明基于概率的不确定推理中的不确定性测度的计算方法。

例 6.1 现有下述 4 条规则:

$$R_1: a \wedge b \rightarrow c$$

$$R_2: c \rightarrow d$$

$$R_3: d \rightarrow f$$

$$R_4: \bar{e} \rightarrow f$$

若已知证据和结论的不确定性的测度分别为:

$$P(a)=0.4 \quad P(b)=0.25 \quad P(c)=0.1 \quad P(d)=0.08 \quad P(e)=0.15 \quad P(f)=0.06$$

已知各条规则的不确定性测度分别为:

$$R_1: a \text{ 和 } b \text{ 同时支持 } c \text{ 的不确定性测度 } P(ab|c)=0.8$$

$$R_2: c \text{ 支持 } d \text{ 的不确定性测度 } P(c|d)=0.9$$

$$R_3: d \text{ 支持 } f \text{ 的不确定性测度 } P(d|f)=0.7$$

$$R_4: e \text{ 不支持 } f \text{ 的不确定性测度 } P(\bar{e}|f)=0.2$$

求证据 a、b、e 同时出现时, 结论 f 的不确定性测度 $P(flabe)$ 。

解 由给出的 4 条规则及证据与规则的不确定性测度, 可画出推理与/或图如图 6.1 所示。

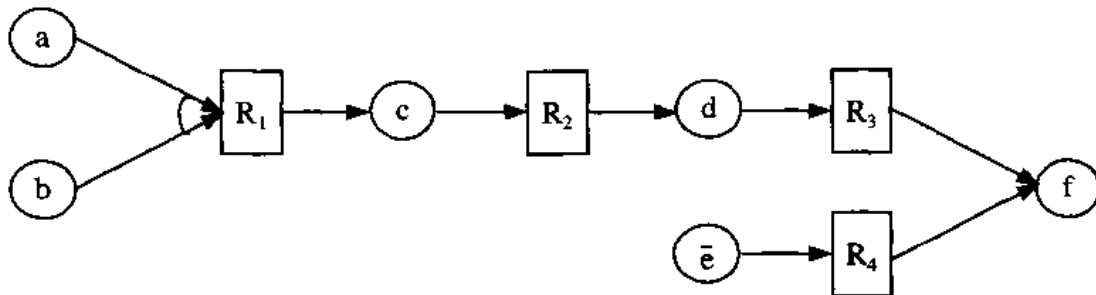


图 6.1 例 6.1 的推理与/或图

由推理与/或图可见, 结论 f 的不确定性测度 $P(flabe)$ 可由两条规则链通过并行法则计算得出, 一条规则链是 $R_1 \rightarrow R_2 \rightarrow R_3$, 另一条规则链是 R_4 。

1) 由顺序法则计算 a、b 同时出现时, 结论 f 的不确定性测度 $P(flabe)$ 。

① 计算 a、b 同时出现时, 中间结论 c 成立的不确定性测度 $P(c|ab)$ 为:

因为:

$$P(ab)=P(a)P(b)=0.4 \times 0.25=0.1$$

$$P(c)=0.1$$

$$P(ab|c)=0.8$$

故而可得:

$$P(c|ab)=\frac{P(ab|c)P(c)}{P(ab)}=\frac{0.8 \times 0.1}{0.1}=0.8$$

a、b 同时出现时, 中间结论 c 不成立的不确定性测度为:

$$P(\bar{c}|ab)=1-P(c|ab)=1-0.8=0.2$$

②计算 c 成立时, 中间结论 d 成立的不确定性测度 $P(d|c)$ 为:

$$P(d|c)=\frac{P(c|d)P(d)}{P(c)}=\frac{0.9 \times 0.08}{0.1}=0.72$$

计算 c 不成立时, 中间结论 d 成立的不确定性测度 $P(d|\bar{c})$ 为:

因为有:

$$P(\bar{c}|d)=1-P(c|d)=1-0.9=0.1$$

$$P(\bar{c})=1-P(c)=1-0.1=0.9$$

$$P(d)=0.08$$

故而可得:

$$P(d|\bar{c})=\frac{P(\bar{c}|d)P(d)}{P(\bar{c})}=\frac{0.1 \times 0.08}{0.9}=0.009$$

③由顺序法则计算 a、b 同时出现时, 中间结论 d 成立的不确定性测度 $P(d|ab)$ 为:

$$P(d|ab)=P(d|c)P(c|ab)+P(d|\bar{c})P(\bar{c}|ab)$$

$$=0.72 \times 0.8 + 0.009 \times 0.2 = 0.578$$

计算 a、b 同时出现时, 中间结论 d 不成立的不确定性测度为:

$$P(\bar{d}|ab)=1-P(d|ab)=1-0.578=0.422$$

④计算 d 成立时, 结论 f 成立的不确定性测度 $P(f|d)$ 为:

$$P(f|d)=\frac{P(d|f)P(f)}{P(d)}=\frac{0.7 \times 0.06}{0.08}=0.525$$

计算 d 不成立时, 结论 f 成立的不确定性测度 $P(f|\bar{d})$ 为:

因为有:

$$P(\bar{d}|f)=1-P(d|f)=1-0.7=0.3$$

$$P(\bar{d})=1-P(d)=1-0.08=0.92$$

$$P(f)=0.06$$

故而可得:

$$P(f|\bar{d})=\frac{P(\bar{d}|f)P(f)}{P(\bar{d})}=\frac{0.3 \times 0.06}{0.92}=0.02$$

⑤由顺序法则计算 a、b 同时出现时, 结论 f 成立的不确定性测度 $P(f|ab)$ 为:

$$P(f|ab)=P(f|d)P(d|ab)+P(f|\bar{d})P(\bar{d}|ab)$$

$$=0.525 \times 0.578 + 0.02 \times 0.422 = 0.312$$

计算 a、b 同时出现时, 结论 f 不成立的不确定性测度为:

$$P(\bar{f}|ab)=1-P(f|ab)=1-0.312=0.688$$

2) 计算 e 出现时, 结论 f 成立的不确定性测度 $P(f|e)$ 为:

因为有:

$$P(e|f)=1-P(\bar{e}|f)=1-0.2=0.8$$

$$P(e)=0.15$$

$$P(f)=0.06$$

故而可得:

$$P(fe)=\frac{P(e|f)P(f)}{P(e)}=\frac{0.8\times 0.06}{0.15}=0.32$$

计算 e 出现时, 结论 f 不成立的不确定性测度为:

$$P(\bar{f}|e)=1-P(fe)=1-0.32=0.68$$

3) 由并行法则计算 a、b、e 同时出现时, 结论 f 成立的不确定性测度 $P(flabe)$ 。

由并行法则(6-13)式, 有:

$$O(flabe)=\lambda(f,ab)\lambda(f,e)O(f)$$

其中, 由规则链 $R_1 \rightarrow R_2 \rightarrow R_3$ 传递不确定性的修正因子 $\lambda(f, ab)$ 为:

$$\begin{aligned}\lambda(f, ab) &= \frac{P(ab|f)}{P(ab|\bar{f})} = \frac{\frac{P(f|ab)P(ab)}{P(f)}}{\frac{P(\bar{f}|ab)P(ab)}{P(\bar{f})}} \\ &= \frac{P(f|ab)P(\bar{f})}{P(\bar{f}|ab)P(f)} = \frac{P(f|ab)(1-P(f))}{P(\bar{f}|ab)P(f)} \\ &= \frac{0.312 \times (1-0.06)}{0.688 \times 0.06} = 7.1\end{aligned}$$

由规则链 R_4 传递不确定性的修正因子 $\lambda(f, e)$ 为:

$$\lambda(f, e) = \frac{P(e|f)}{P(e|\bar{f})} = \frac{P(f|e)P(\bar{f})}{P(\bar{f}|e)P(f)} = \frac{0.32 \times (1-0.06)}{0.68 \times 0.06} = 7.37$$

结论 f 成立的几率 $O(f)$ 为:

$$O(f) = \frac{P(f)}{P(\bar{f})} = \frac{P(f)}{1-P(f)} = \frac{0.06}{1-0.06} = 0.064$$

故而可得 a、b、e 同时出现时, 结论 f 成立的几率 $O(flabe)$ 为:

$$\begin{aligned}O(flabe) &= \lambda(f,ab)\lambda(f,e)O(f) \\ &= 7.1 \times 7.37 \times 0.064 = 3.34\end{aligned}$$

由几率与概率的关系式 $O = \frac{P}{1-P}$, 得 $P = \frac{O}{1+O}$, 由此可得 a,b,e 同时出现时, 结论 f 成立的不确定性测度为:

$$P(flabe) = \frac{O(flabe)}{1+O(flabe)} = \frac{3.34}{1+3.34} = 0.77$$

由于 $\lambda(f,ab)=7.1>1$, 说明证据 a 和 b 同时支持结论 f 成立; 同样, $\lambda(f,e)=7.37>1$, 说

明证据 e 支持结论 f 成立: $\lambda(f,abe) = \lambda(f,ab) \lambda(f,e) = 7.1 \times 7.37 = 5.23 > 1$, 说明证据 a 、 b 、 e 同时支持结论 f 成立。事实上, a 、 b 、 e 同时出现时, 结论 f 成立的不确定性测度 $P(f|abe) = 0.77$, 而没有证据 a 、 b 、 e 支持时, 结论 f 成立的不确定性测度只有 $P(f) = 0.06$, 后者远远小于前者。

6.3 基于可信度的不确定推理

基于概率的不确定推理虽然具有概率论严密的理论依据, 但是, 它要求给出知识的概率, 即使富有经验的领域专家也难以直接给出, 因此使其应用受到限制。人们对一个事物的认识, 往往可根据经验判断这个事物的真伪程度。根据经验对一个事物或一件事情为真的相信程度称为可信度 (Certainty Factor)。

6.3.1 可信度不确定推理方法

基于可信度的不确定推理是肖特里菲 (E.H.Shortliffe) 等人在确定性理论 (Theory of Confirmation) 的基础上提出的一种不确定推理方法, 并首先在医疗诊断专家系统 MYCIN 中得到成功的应用。这种方法直观, 不确定性测度的计算也比较简便, 因而在许多专家系统中得到有效地应用。

1. 信任度与不信任度

定义 6.7 信任度 $MB(h, e)$ 表示证据 e 出现时, 对结论 h 成立的信任程度的增加量。不信任度 $MD(h, e)$ 表示证据 e 出现时, 对结论 h 成立的不信任程度的增加量。 $MB(h, e)$ 和 $MD(h, e)$ 的取值范围为 $[0, 1]$ 。它们形式化地定义为:

$$MB(h, e) = \begin{cases} 1 & P(h) = 1 \\ \frac{\max(P(h|e), P(h)) - P(h)}{1 - P(h)} & P(h) \neq 1 \end{cases} \quad (6-18)$$

$$MD(h, e) = \begin{cases} 1 & P(h) = 0 \\ \frac{\min(P(h|e), P(h)) - P(h)}{-P(h)} & P(h) \neq 0 \end{cases} \quad (6-19)$$

其中, $P(h)$ 为结论 h 成立的先验概率; $P(h|e)$ 为在证据 e 出现的条件下, 结论 h 成立的条件概率。

显然, $MB(h, e)$ 和 $MD(h, e)$ 有下述性质。

性质 6.3 (互斥律) 一个证据 e 不可能既支持又不支持某个结论 h , 因此, 有:

如果 $MB(h, e) > 0$, 那么 $MD(h, e) = 0$

如果 $MD(h, e) > 0$, 那么 $MB(h, e) = 0$

性质 6.4 若 $P(h|e) > P(h)$, 表明证据 e 的出现增加了对结论 h 成立的信任程度, 但是, 不改变对结论 h 成立的不信任程度。

若 $P(h|e) > P(h)$, 由 (6-18) 式可见, 有 $MB(h, e) > 0$; 由 (6-19) 式可见, 有 $MD(h, e) = 0$ 。

性质 6.5 若 $P(h|e) = P(h)$, 表明证据 e 的出现不改变对结论 h 成立的信任程度, 也不改

变对结论 h 成立的不信任程度, 即表明证据 e 与结论 h 之间相互独立。

若 $P(h|e)=P(h)$, 由(6-18)式可见, 有 $MB(h,e)=0$; 由(6-19)式可见, 有 $MD(h,e)=0$ 。即信任程度与不信任程度都不变。

性质 6.6 若 $P(h|e)<P(h)$, 表明证据 e 的出现增加了对结论 h 成立的不信任程度, 但是, 不改变对结论 h 成立的信任程度。

若 $P(h|e)<P(h)$, 由(6-18)式可见, 有 $MB(h,e)=0$; 由(6-19)式可见, 有 $MD(h,e)>0$ 。

2. 可信度

在可信度不确定推理模型中, 把信任度 MB 与不信任度 MD 组合成一个单一的不确定性测度, 这就是可信度。

定义 6.8 可信度形式化地定义为:

$$CF(h,e)=MB(h,e)-MD(h,e) \quad (6-20)$$

由 $CF(h,e)$ 、 $MB(h,e)$ 和 $MD(h,e)$ 的定义 (6-20) 式、(6-18) 式和 (6-19) 式以及 MB 与 MD 的互斥性质, 可得出 $CF(h,e)$ 的计算公式为:

$$CF(h,e)=\begin{cases} 1 & \text{若 } P(h)=1 \\ \frac{P(h|e)-P(h)}{1-P(h)} & \text{若 } P(h|e)>P(h) \\ 0 & \text{若 } P(h|e)=P(h) \\ \frac{P(h|e)-P(h)}{P(h)} & \text{若 } P(h|e)<P(h) \\ -1 & \text{若 } P(h)=0 \end{cases} \quad (6-21)$$

由 (6-21) 式可直观地看出可信度 $CF(h,e)$ 的下述意义:

1) 若 $CF(h,e)>0$, 则 $P(h|e)>P(h)$ 。说明证据 e 的出现增加了结论 h 为真的概率, 即增加了 h 为真的可信度, $CF(h,e)$ 的值越大, 增加 h 为真的可信度就越大。若 $CF(h,e)=1$, 则可推出 $P(h|e)=1$, 即证据 e 的出现使 h 为真。

2) 若 $CF(h,e)<0$, 则 $P(h|e)<P(h)$ 。说明证据 e 的出现减少了结论 h 为真的概率, 即增加了 h 为假的可信度, $CF(h,e)$ 的值越小, 增加 h 为假的可信度就越大。若 $CF(h,e)=-1$, 则可推出 $P(h|e)=0$, 即证据 e 的出现使 h 为假。

3) 若 $CF(h,e)=0$, 则 $P(h|e)=P(h)$, 表示 h 与 e 独立, 即证据 e 的出现对 h 没有影响。

当已知 $P(h)$ 和 $P(h|e)$ 时, 通过上述计算公式就可求出 $CF(h,e)$ 。但是, 在实际应用中, 获得 $P(h)$ 和 $P(h|e)$ 的值是比较困难的, $CF(h,e)$ 的值反而比较容易通过领域专家直接给出。设定 $CF(h,e)$ 的值的原理是: 若相应证据 e 能增加结论 h 为真的可信度, 则使 $CF(h,e)>0$, 证据 e 越是支持 h 为真, 就使 $CF(h,e)$ 的值越大; 反之, 使 $CF(h,e)<0$, 证据 e 越是支持 h 为假, 就使 $CF(h,e)$ 值越小; 若证据 e 与 h 无关, 则使 $CF(h,e)=0$ 。领域专家根据拥有的专业知识和实践经验, 不难对领域知识给出可信度。用可信度表示不确定性知识的方法比较直观、简单, 效果也比较好。

3. 三种基本不确定性测度的计算

CF 模型给出了不确定推理中证据组合的不确定性测度、并行规则的不确定性测度和顺序规则的不确定性测度这三种基本不确定性测度的计算方法。

(1) 证据组合的不确定计算

1) 证据合取组合的不确定计算

定义 6.9 若证据 e_1 的可信度为 $CF(e_1, e')$, 证据 e_2 的可信度为 $CF(e_2, e')$, 则证据 e_1 与证据 e_2 的合取组合 $e_1 \wedge e_2$ 的可信度为:

$$CF(e_1 \wedge e_2, e') = \min (CF(e_1, e'), CF(e_2, e')) \quad (6-22)$$

同样给出信任度与不信任度的计算方法为:

$$MB(e_1 \wedge e_2, e') = \min (MB(e_1, e'), MB(e_2, e'))$$

$$MD(e_1 \wedge e_2, e') = \max (MD(e_1, e'), MD(e_2, e'))$$

2) 证据析取组合的不确定计算

定义 6.10 若证据 e_1 的可信度为 $CF(e_1, e')$, 证据 e_2 的可信度为 $CF(e_2, e')$, 则证据 e_1 与证据 e_2 的析取组合 $e_1 \vee e_2$ 的可信度为:

$$CF(e_1 \vee e_2, e') = \max (CF(e_1, e'), CF(e_2, e')) \quad (6-23)$$

同样给出信任度与不信任度的计算方法为:

$$MB(e_1 \vee e_2, e') = \max (MB(e_1, e'), MB(e_2, e'))$$

$$MD(e_1 \vee e_2, e') = \min (MD(e_1, e'), MD(e_2, e'))$$

(2) 并行规则的不确定计算 (并行法则)

定义 6.11 若证据 e_1 的出现使结论 h 成立的可信度为 $CF(h, e_1)$, 证据 e_2 的出现使结论 h 成立的可信度为 $CF(h, e_2)$, 则证据 e_1 与 e_2 同时出现使结论 h 成立的可信度 $CF(h, e_1 \wedge e_2)$ 为:

$$CF(h, e_1 \wedge e_2) = \begin{cases} CF(h, e_1) + CF(h, e_2) - CF(h, e_1)CF(h, e_2) & \text{若 } CF(h, e_1) \geq 0, CF(h, e_2) \geq 0 \\ \frac{CF(h, e_1) + CF(h, e_2)}{1 - \min(|CF(h, e_1)|, |CF(h, e_2)|)} & \text{若 } CF(h, e_1) \text{ 与 } CF(h, e_2) \text{ 异号} \\ CF(h, e_1) + CF(h, e_2) + CF(h, e_1)CF(h, e_2) & \text{若 } CF(h, e_1) < 0, CF(h, e_2) < 0 \end{cases} \quad (6-24)$$

(3) 顺序规则的不确定计算 (顺序法则)

定义 6.12 若顺序使用两条规则:

if e_1 then e_2

if e_2 then h

且证据 e_1 出现使中间结论 e_2 成立的可信度为 $CF(e_2, e_1)$, 中间结论 e_2 出现使结论 h 成立的可信度为 $CF(h, e_2)$, 则证据 e_1 出现使结论 h 成立的可信度 $CF(h, e_1)$ 为:

$$CF(h, e_1) = CF(h, e_2) \max (0, CF(e_2, e_1)) \quad (6-25)$$

同样给出信任度与不信任度的计算方法为:

$$MB(h, e_1) = MB(h, e_2) \max (0, CF(e_2, e_1))$$

$$MD(h, e_1) = MD(h, e_2) \max (0, CF(e_2, e_1))$$

在定义 6.12 中, CF 模型的顺序推理链的可信度 CF 和信任度 MB 与不信任度 MD 的计算方法中, 都对 $CF(e_2, e_1)$ 取 $\max(0, CF(e_2, e_1))$, 因此, 若 $CF(e_2, e_1) < 0$, 则 $CF(h, e_1) = 0$, $MB(h, e_1) = 0$, $MD(h, e_1) = 0$ 。也就是说, CF 模型在顺序推理链中只传播信任度, 不传播不信任度。

例 6.2 现有下述 5 条规则及相应规则的可信度为:

$$R_1: a \rightarrow c \quad CF(c, a) = 0.8$$

$$R_2: b \rightarrow c \quad CF(c, b) = 0.5$$

$$R_3: c \rightarrow d \quad CF(d, c) = 0.7$$

$$R_4: d \rightarrow f \quad CF(f, d) = 0.9$$

$$R_5: e \rightarrow f \quad CF(f, e) = -0.3$$

计算证据 a、b、e 同时出现时结论 f 成立的可信度 $CF(f, abe)$ 。

解 由给出的 5 条规则, 可画出推理与/或图如图 6.2 所示。

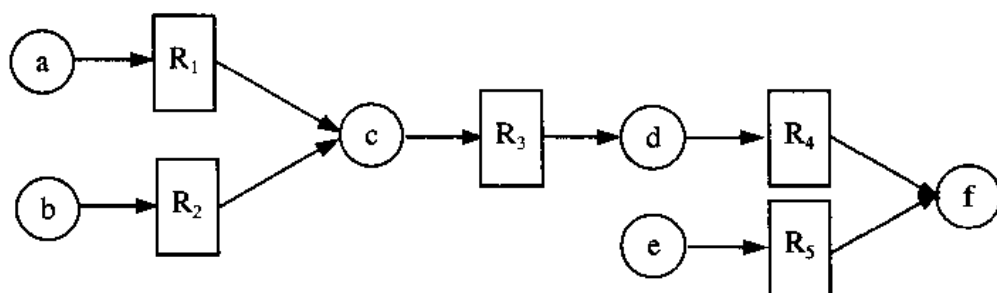


图 6.2 例 6.2 的推理与/或图

由并行法则 (6-24) 式计算证据 a、b 同时出现时 c 成立的可信度 $CF(c, ab)$ 为:

$$\begin{aligned} CF(c, ab) &= CF(c, a) + CF(c, b) - CF(c, a)CF(c, b) \\ &= 0.8 + 0.5 - 0.8 \times 0.5 = 0.9 \end{aligned}$$

再由顺序法则 (6-25) 式计算证据 a、b 同时出现时, d 成立的可信度 $CF(d, ab)$ 为:

$$\begin{aligned} CF(d, ab) &= CF(d, c) \max(0, CF(c, ab)) \\ &= 0.7 \times 0.9 = 0.63 \end{aligned}$$

再由顺序法则 (6-25) 式计算证据 a、b 同时出现时, f 成立的可信度 $CF(f, ab)$ 为:

$$\begin{aligned} CF(f, ab) &= CF(f, d) \max(0, CF(d, ab)) \\ &= 0.9 \times 0.63 = 0.567 \end{aligned}$$

最后, 由并行法则 (6-24) 式计算证据 a、b、e 同时出现时, f 成立的可信度 $CF(f, abe)$ 为:

$$CF(f, abe) = \frac{CF(f, ab) + CF(f, e)}{1 - \min(|CF(f, ab)|, |CF(f, e)|)} = \frac{0.567 - 0.3}{1 - 0.3} = 0.381$$

由于 $CF(f, ab) = 0.567$, $CF(f, abe) = 0.381$, 说明证据 e 的出现, 反而降低了结论 f 成立的可信度。事实上, 由 $CF(f, e) = -0.3 < 0$, 已表示证据 e 的出现使结论 f 成立更不可信。

6.3.2 带有阈限的不确定推理

为了使可信度方法能求解更多的问题，人们在 CF 模型的基础上提出了一些更具有一般性的处理方法。我们首先讨论带有阈限的可信度不确定推理方法。

1. 带有阈限的规则表示

在带有阈值限度的不确定性推理中，规则用下述形式表示：

$$\text{if } e \text{ then } h \quad (CF(h,e), \lambda)$$

其中：

1) e 可以是一个简单证据，也可以是多个证据的合取与析取。

2) $CF(h,e)$ 是规则的可信度，也称为规则强度，它指出规则为真的可信程度，取值范围为 $0 < CF(h,e) \leq 1$ 。

3) λ 为规则的阈值，它对规则的可使用规定了一个限制，只有当规则的相应证据 e 的可信度 $CF(e) \geq \lambda$ 时，该规则才有可能被使用， λ 的取值范围为 $0 < \lambda \leq 1$ 。

2. 三种基本不确定性测度的计算

(1) 证据组合的不确定计算

证据组合的不确定计算与 CF 模型一样，多个证据的合取组合的可信度为多个证据可信度的极小，多个证据的析取组合的可信度为多个证据可信度的极大。

定义 6.13 多个证据的合取组合与析取组合的可信度为：

$$\begin{aligned} CF(e_1 \wedge e_2 \wedge \cdots \wedge e_n) &= \min(CF(e_1), CF(e_2), \cdots, CF(e_n)) \\ CF(e_1 \vee e_2 \vee \cdots \vee e_n) &= \max(CF(e_1), CF(e_2), \cdots, CF(e_n)) \end{aligned} \quad (6-26)$$

(2) 并行规则的不确定计算（并行规则）

定义 6.14 设有多条规则有相同的结论，即

$$\begin{array}{llll} \text{if} & e_1 & \text{then} & h \quad (CF(h,e_1), \lambda_1) \\ \text{if} & e_2 & \text{then} & h \quad (CF(h,e_2), \lambda_2) \\ & \vdots & & \\ \text{if} & e_n & \text{then} & h \quad (CF(h,e_n), \lambda_n) \end{array}$$

若 n 条规则都满足：

$$CF(e_i) \geq \lambda_i \quad i=1,2,\cdots,n$$

且都被使用，则首先分别对每条规则求出结论可信度 $CF_i(h)$ ，即

$$CF_i(h) = CF(h,e_i) \times CF(e_i) \quad i=1,2,\cdots,n$$

然后选择下述方法中的一种方法求出结论 h 的可信度 $CF(h)$ ：

1) 求极大值法。对 n 个 $CF_i(h)$ 取极大，即

$$CF(h) = \max(CF_1(h), CF_2(h), \cdots, CF_n(h)) \quad (6-27)$$

2) 加权求和法。用下式计算 $CF(h)$ ：

$$CF(h) = \frac{1}{\sum_{i=1}^n CF(h, e_i)} \sum_{i=1}^n CF(h, e_i) \times CF(e_i) \quad (6-28)$$

3) 有限和法。对 n 个 $CF_i(h)$ 用下式计算 $CF(h)$:

$$CF(h) = \min\left(\sum_{i=1}^n CF_i(h), 1\right) \quad (6-29)$$

(3) 顺序规则的不确定计算 (顺序法则)

定义 6.15 有以下两条规则:

$$R_1: \quad \text{if } e_1 \quad \text{then } e_2 \quad (CF(e_2, e_1), \lambda_1)$$

$$R_2: \quad \text{if } e_2 \quad \text{then } h \quad (CF(h, e_2), \lambda_2)$$

若 $CF(e_1) \geq \lambda_1$, 且规则 R_1 被使用, 则 e_2 的可信度为:

$$CF(e_2) = CF(e_2, e_1) \times CF(e_1)$$

若 $CF(e_2) \geq \lambda_2$, 且规则 R_2 被使用, 则得出 h 的可信度为:

$$\begin{aligned} CF(h) &= CF(h, e_2) \times CF(e_2) \\ &= CF(h, e_2) \times CF(e_2, e_1) \times CF(e_1) \end{aligned} \quad (6-30)$$

这里, “ $\times$ ” 既可以是“乘”运算, 也可以是“取极小”或其他运算。

6.3.3 加权的不确定推理

在前面讨论的可信度不确定推理的方法中, 若一条规则的前件有多个证据, 则认为各证据都是平等的。在实际问题中并非都是这样。一般来说, 一条规则的多个证据对结论的支持程度不相同, 或者说各个证据对结论具有不同的重要程度。为此, 可在规则中为每个证据引入加权因子, 使不同的证据具有不同的“权”。

1. 加权的规则表示

在加权的不确定性推理中, 规则用下述形式表示:

$$\text{if } e_1(w_1) \wedge e_2(w_2) \wedge \cdots \wedge e_n(w_n) \quad \text{then } h \quad (CF(h, e), \lambda)$$

其中:

1) w_i 称为证据 e_i 的加权因子, 权值的取值范围一般规定为 $0 \leq w_i \leq 1$, $i=1, 2, \dots, n$, 且应满足归一条件, 即

$$\sum_{i=1}^n w_i = 1$$

2) 如果证据 e_i 对结论 h 成立的重要性较高, 则应使 e_i 具有较大的权值; 如果一个证据具有较大的独立性, 而其他证据对它有依赖关系, 则应使这个证据具有较大的权值。证据权值的确定应通过领域专家给出。

2. 三种基本不确定性测度的计算

(1) 证据组合的不确定计算

定义 6.16 对于证据组合

$$e=e_1(w_1)\wedge e_2(w_2)\wedge\cdots\wedge e_n(w_n)$$

若 n 个证据的权值满足归一条件, 则 n 个证据的合取组合的可信度用下式计算:

$$CF(e)=\sum_{i=1}^n w_i \times CF(e_i) \quad (6-31)$$

若 n 个证据的权值不满足归一条件, 则 n 个证据的合取组合的可信度用下式计算:

$$CF(e)=\frac{1}{\sum_{i=1}^n w_i} \sum_{i=1}^n (w_i \times CF(e_i)) \quad (6-32)$$

(2) 并行规则与顺序规则的不确定计算

对于加权规则的并行法则和顺序法则, 根据定义 6.16 计算出各规则的证据组合的可信度 $CF(e)$ 之后, 与带有阈限的并行法则和顺序法则一样, 采用 (6-27) 式、(6-28) 式和 (6-29) 式中的一个计算式计算并行规则的结论可信度 $CF(h)$, 采用 (6-30) 式计算顺序规则的结论可信度 $CF(h)$ 。

3. 加权规则的不确定推理

在规则中引入证据的加权因子, 不仅解决了多个证据对结论支持的重要程度不同和各个证据之间的独立性与依赖性不同的表示问题, 而且还解决了当证据不完整时不确定推理问题。

首先, 我们来看下面一个例子。

例 6.3 在动物识别专家系统的规则库中有下述一条加权规则:

```
if  动物有蹄 (0.3)
    且动物有长颈 (0.2)
    且动物有长腿 (0.2)
    且动物是黄褐色 (0.13)
    且动物体上有暗黑色斑点 (0.13)
    且动物体重在 100kg 以上 (0.04)
then 动物是长颈鹿 (0.95,0.8)
```

若综合数据库中有以下证据:

动物有蹄	可信度 $CF(e_1)=1$
动物有长颈	可信度 $CF(e_2)=1$
动物有长腿	可信度 $CF(e_3)=1$
动物是黄褐色	可信度 $CF(e_4)=0.8$
动物体上有暗黑色斑点	可信度 $CF(e_5)=0.6$

问该动物是什么动物?

由于数据库中提供的证据比规则前提要求的证据少了一个, 即“动物体重在 100kg 以上”, 因此, 若使用不带加权因子的不确定推理, 那么这条规则就不能被使用。但是, 应用加权的

不确定推理, 这条规则就可能被使用, 因为通过匹配后, 并根据(6-31)式计算出该规则不完全匹配的组合证据可信度为:

$$\begin{aligned} CF(e) &= \sum_{i=1}^n w_i \times CF(e_i) \\ &= 0.3 \times 1 + 0.2 \times 1 + 0.2 \times 1 + 0.13 \times 0.8 + 0.13 \times 0.6 \\ &= 0.882 \end{aligned}$$

由于规则的阈值 $\lambda = 0.8$, 故有 $CF(e) > \lambda$, 所以该规则可被使用, 并推出结论“动物是长颈鹿”, 且结论 h 的可信度为:

$$\begin{aligned} CF(h) &= CF(h, e) \times CF(e) \\ &= 0.95 \times 0.882 \\ &= 0.84 \end{aligned}$$

由此例可见, 对规则引入加权因子, 使加权不确定推理可对提供的不完整证据进行推理。在实际情况下, 经常会出现某些不重要的证据没有被提供的情况, 如果因此就不能进行推理, 从而推不出本应推出的结论, 那将是不合情理的。

4. 加权规则的冲突消解

对规则引入加权因子后, 有利于加权规则的冲突消解。我们首先说明加权不确定推理的冲突消解方法。

设有以下两条规则:

$$\begin{aligned} R_1: & \text{ if } e_1(w_1) \wedge e_2(w_2) \quad \text{ then } h_1 (CF_1, \lambda_1) \\ R_2: & \text{ if } e_3(w_3) \wedge e_4(w_4) \wedge e_5(w_5) \quad \text{ then } h_2 (CF_2, \lambda_2) \end{aligned}$$

若与数据库中已有证据经过匹配, 并计算得到:

$$\begin{aligned} CF(e_1 \wedge e_2) &> \lambda_1 \\ CF(e_3 \wedge e_4 \wedge e_5) &> \lambda_2 \end{aligned}$$

则两条规则都可被使用, 而且它们发生了冲突。冲突消解的方法是: 比较发生冲突的规则的组合证据的可信度值, 选择组合证据可信度大的规则优先执行。若 $CF(e_3 \wedge e_4 \wedge e_5) > CF(e_1 \wedge e_2)$, 则优先执行规则 R_2 。

例 6.4 设有以下规则:

$$\begin{aligned} R_1: & \text{ if } e_1(0.6) \wedge e_2(0.4) \quad \text{ then } e_6 (0.8, 0.75) \\ R_2: & \text{ if } e_3(0.5) \wedge e_4(0.3) \wedge e_5(0.2) \quad \text{ then } e_7 (0.7, 0.6) \\ R_3: & \text{ if } e_6(0.7) \wedge e_7(0.3) \quad \text{ then } h (0.75, 0.6) \end{aligned}$$

若数据库中提供的证据有 e_1 、 e_2 、 e_3 、 e_4 和 e_5 , 它们的可信度分别是:

$$CF(e_1)=0.9 \quad CF(e_2)=0.8 \quad CF(e_3)=0.7 \quad CF(e_4)=0.6 \quad CF(e_5)=0.5$$

求结论 h 的可信度 $CF(h)$ 的值。

解 规则 R_1 可匹配, 并计算组合证据可信度为:

$$CF(e_1 \wedge e_2) = w_1 \times CF(e_1) + w_2 \times CF(e_2)$$

$$\begin{aligned}
 &=0.6 \times 0.9 + 0.4 \times 0.8 \\
 &=0.86
 \end{aligned}$$

且因为 $\lambda_1=0.75$, 有 $CF(e_1 \wedge e_2) > \lambda_1$, 故 R_1 可被使用。

规则 R_2 也可匹配, 并计算组合证据可信度为:

$$\begin{aligned}
 CF(e_3 \wedge e_4 \wedge e_5) &= w_3 \times CF(e_3) + w_4 \times CF(e_4) + w_5 \times CF(e_5) \\
 &= 0.5 \times 0.7 + 0.3 \times 0.6 + 0.2 \times 0.5 \\
 &= 0.63
 \end{aligned}$$

且因为 $\lambda_2=0.6$, 有 $CF(e_3 \wedge e_4 \wedge e_5) > \lambda_2$, 故 R_2 也可被使用。

又因为 $CF(e_1 \wedge e_2) > CF(e_3 \wedge e_4 \wedge e_5)$, 故 R_1 被优先使用, 在数据库中增加证据 e_6 , 且 e_6 的可信度为:

$$\begin{aligned}
 CF(e_6) &= CF(e_6, e_1 \wedge e_2) \times CF(e_1 \wedge e_2) \\
 &= 0.8 \times 0.86 \\
 &= 0.69
 \end{aligned}$$

然后执行规则 R_2 , 在数据库中增加证据 e_7 , 且 e_7 的可信度为:

$$\begin{aligned}
 CF(e_7) &= CF(e_7, e_3 \wedge e_4 \wedge e_5) \times CF(e_3 \wedge e_4 \wedge e_5) \\
 &= 0.7 \times 0.63 \\
 &= 0.44
 \end{aligned}$$

最后, 规则 R_3 可匹配, 并计算组合证据可信度为:

$$\begin{aligned}
 CF(e_6 \wedge e_7) &= w_6 \times CF(e_6) + w_7 \times CF(e_7) \\
 &= 0.7 \times 0.69 + 0.3 \times 0.44 \\
 &= 0.615
 \end{aligned}$$

且因为 $\lambda_3=0.6$, 有 $CF(e_6 \wedge e_7) > \lambda_3$, 故 R_3 被使用, 在数据库中增加结论 h , 且 h 的可信度为:

$$\begin{aligned}
 CF(h) &= CF(h, e_6 \wedge e_7) \times CF(e_6 \wedge e_7) \\
 &= 0.75 \times 0.615 \\
 &= 0.46
 \end{aligned}$$

6.3.4 可信度不确定推理方法的改进

可信度不确定推理方法的主要优点是简单易行, 但是, 6.3.1 节给出的 CF 模型是一种直觉化的模型, 缺乏严密理论来保证可信度计算的正确性和完备性。在证据相关的情况下, 前面给出的 CF 模型的并行法则和顺序法则可能导致矛盾的结果。

例如, 对推理链 $e_1 \rightarrow e_2 \rightarrow h$, 若 $CF(h, e_2) < 0$, $CF(e_2, e_1) < 0$, 则由顺序法则有:

$$CF(h, e_1) = CF(h, e_2) \max(0, CF(e_2, e_1)) = 0$$

按可信度 CF 的定义, $CF(h, e_2) < 0$, 表示证据 e_2 使结论 h 成立更不可信, 那么, 证据 $\bar{e}_2$ 应使结论 h 成立更可信, 即应有 $CF(h, \bar{e}_2) > 0$ 。同样, 应有 $CF(\bar{e}_2, e_1) > 0$ 。

若在证据 e_1 与结论 h 之间有另一条推理链 $e_1 \rightarrow \bar{e}_2 \rightarrow h$, 由于有 $CF(h, \bar{e}_2) > 0$, $CF(\bar{e}_2, e_1)$

>0 , 则由顺序法则, 有:

$$CF(h, e_1) = CF(h, \bar{e}_2) \max(0, CF(\bar{e}_2, e_1)) > 0$$

可见, CF 模型的顺序法则得出了两个不一致的结果。其原因在于: 根据证据 e 和结论 h 的逻辑关系, 如果某一证据 e 支持结论 h , 即有 $CF(h, e) > 0$, 那么, 这一证据的否定 $\bar{e}$ 应反对结论 h , 即应有 $CF(h, \bar{e}) < 0$ 。同样, 如果证据 e 支持结论 h , 即有 $CF(h, e) > 0$, 那么, 这一证据 e 应反对结论的否定 $\bar{h}$, 即应有 $CF(\bar{h}, e) < 0$ 。但是, CF 模型采用的经验法则并不反映证据的否定和结论的否定的逻辑关系, 也不传播对结论的否定程度。

为了克服 CF 模型的缺陷, Heckerman 对可信度 CF 提出了新的解释, 以及与这一解释相一致的不确定推理模型的不确定性计算方法。

1. CF 的概率解释

概率具有坚实的理论基础, Heckerman 引入概率来解释可信度 CF, 从而在 CF 模型的基础上建立起新的 CF 不确定性推理模型, 称之为一般 CF 模型。

设 $B(h)$ 表示结论 h 的先验信任度, $B(h|e)$ 表示在证据 e 出现的条件下结论 h 的后验信任度, $CF(h, e)$ 表示在证据 e 出现的条件下结论 h 的信任度的修改量。一般 CF 模型的基本思想是: 利用给出的 $B(h)$ 和 $CF(h, e)$, 计算出结论 h 的后验信任度 $B(h|e)$, 即存在一个函数 f , 使得:

$$B(h|e) = f(CF(h, e), B(h))$$

由基于概率的并行法则(6-13)式, 可以有:

$$O(h|e) = \lambda(h, e) O(h)$$

就可以用来描述 $B(h|e)$ 和 $CF(h, e)$ 与 $B(h)$ 的关系, 由 $\lambda(h, e)$ 部分地充当 $CF(h, e)$ 的角色。但是, $\lambda(h, e)$ 的值域是 $[0, \infty]$, 而 $CF(h, e)$ 的值域是 $[-1, 1]$, 因此, 需要引入一个函数 F :

$$CF(h, e) = F(\lambda(h, e))$$

把 $\lambda(h, e)$ 的值域 $[0, \infty]$ 映射到 $[-1, 1]$ 。函数 F 应该满足下述性质:

- ① $F(x)$ 为递增函数
- ② $F(1/x) = -F(x)$
- ③ $F(+\infty) = 1$

Heckerman 证明了满足上述性质的函数 F 一定能实现从 $\lambda(h, e)$ 到 $CF(h, e)$ 的变换, 且 $CF(h, e)$ 与 $\lambda(h, e)$ 具有同样的概率模型。这样, 就把基于可信度的 CF 模型与基于概率的不确定推理模型统一起来了。

2. 一般 CF 不确定推理模型

首先, 我们给出满足上述三个性质的函数 F 。这样的函数可能很多, 我们给出的 F 函数定义为:

$$F(x) = \begin{cases} (x-1)/x & x \geq 1 \\ x-1 & 0 \leq x < 1 \end{cases}$$

上述 F 函数显然满足上述要求的三个性质, 而且会发现有下列的巧合: 一是由 $F(\lambda(h, e))$

导出的 $CF(h,e)$ 与前述 CF 模型中的 $CF(h,e)$ 的定义非常相似：二是由一般 CF 模型的 $CF(h,e)$ 导出的并行法则与前述 CF 模型的并行法则完全相同。

定义 6.17 由函数 F 的定义，给出一般 CF 模型的 $CF(h,e)$ 的定义为：

$$\begin{aligned} CF(h,e) &= F(\lambda(h,e)) \\ &= \begin{cases} (\lambda(h,e)-1)/\lambda(h,e) & \lambda(h,e) \geq 1 \\ \lambda(h,e)-1 & \lambda(h,e) < 1 \end{cases} \end{aligned} \quad (6-33)$$

由 (6-14) 式和 (6-6) 式，有：

$$\lambda(h,e) = \frac{P(e|h)}{P(e|\bar{h})} = \frac{(1-P(h))P(h|e)}{P(h)(1-P(h|e))}$$

代入 $CF(h,e)$ 的定义式 (6-33) 中，得到 $CF(h,e)$ 基于概率的定义式为：

$$CF(h,e) = \begin{cases} \frac{P(h|e)-P(h)}{(1-P(h))P(h|e)} & P(h|e) \geq P(h) \\ \frac{P(h|e)-P(h)}{P(h)(1-P(h|e))} & P(h|e) < P(h) \end{cases} \quad (6-34)$$

比较 (6-34) 式与前述 CF 模型中关于 $CF(h,e)$ 的定义式 (6-21) 式，(6-34) 式中只在分母中多了一个因子，对于 $P(h|e) \geq P(h)$ ，(6-34) 式的分母中增加乘积因子 $P(h|e)$ ；对于 $P(h|e) < P(h)$ ，(6-34) 式的分母中增加乘积因子 $(1-P(h|e))$ 。

为了导出一般 CF 模型的并行法则和顺序法则，把 $CF(h,e)$ 的定义式 (6-33) 式转换为：

$$\lambda(h,e) = \begin{cases} \frac{1}{1-CF(h,e)} & CF(h,e) \geq 0 \\ 1+CF(h,e) & CF(h,e) < 0 \end{cases} \quad (6-35)$$

为了避免前述 CF 模型的缺陷，在一般 CF 模型中，有下述定义。

定义 6.18 若某一证据 e 支持结论 h ，则这一证据的否定 $\bar{e}$ 反对结论 h ，即：若 $CF(h,e) > 0$ ，则 $CF(h, \bar{e}) < 0$ 。反之亦然，即：若 $CF(h, \bar{e}) > 0$ ，则 $CF(h,e) < 0$ 。

定义 6.19 若某一证据 e 支持结论 h ，则这一证据 e 反对结论的否定 $\bar{h}$ ，即：若 $CF(h,e) > 0$ ，则 $CF(\bar{h}, e) < 0$ 。反之亦然，即：若 $CF(\bar{h}, e) > 0$ ，则 $CF(h,e) < 0$ 。

定义 6.20 若 e_1 、 e_2 相对于 h 、 $\bar{h}$ 条件独立，则 $CF(h, e_1 e_2)$ 的符号与 $CF(h, e_1) + CF(h, e_2)$ 的符号相同。

定义 6.21 对于顺序推理链 $e_1 \rightarrow e_2 \rightarrow h$ ，若 e_1 、 h 相对于 e_2 、 $\bar{e}_2$ 条件独立，则 $CF(h, e_1)$ 的符号与 $CF(h, e_2) CF(e_2, e_1)$ 的符号相同。

(1) 一般 CF 模型的并行法则

上述定义 6.20 说明证据 e_1 和 e_2 同时出现使结论 h 成立的信任程度是增加或是降低，由 $CF(h, e_1) + CF(h, e_2)$ 的符号决定。因此， $CF(h, e_1 e_2)$ 的符号取决于 $CF(h, e_1)$ 和 $CF(h, e_2)$ 的以下 6 种不同的组合：

1) 若 $CF(h, e_1) \geq 0$ ， $CF(h, e_2) \geq 0$ ，则 $CF(h, e_1 e_2) \geq 0$ 。

- 2) 若 $CF(h, e_1) \geq 0, CF(h, e_2) < 0$, 且 $|CF(h, e_1)| \geq |CF(h, e_2)|$, 则 $CF(h, e_1 e_2) \geq 0$ 。
- 3) 若 $CF(h, e_1) \geq 0, CF(h, e_2) < 0$, 且 $|CF(h, e_1)| < |CF(h, e_2)|$, 则 $CF(h, e_1 e_2) < 0$ 。
- 4) 若 $CF(h, e_1) < 0, CF(h, e_2) \geq 0$, 且 $|CF(h, e_1)| \leq |CF(h, e_2)|$, 则 $CF(h, e_1 e_2) \geq 0$ 。
- 5) 若 $CF(h, e_1) < 0, CF(h, e_2) \geq 0$, 且 $|CF(h, e_1)| > |CF(h, e_2)|$, 则 $CF(h, e_1 e_2) < 0$ 。
- 6) 若 $CF(h, e_1) < 0, CF(h, e_2) < 0$, 则 $CF(h, e_1 e_2) < 0$ 。

定理 6.6 一般 CF 模型与 CF 模型的并行法则具有完全相同的不确定计算方式。

证 CF 模型的并行法则需要按上述 6 种不同组合情况来推导。

- 1) 若 $CF(h, e_1) \geq 0, CF(h, e_2) \geq 0$, 由 (6-16) 式, 有:

$$\lambda(h, e_1 e_2) = \lambda(h, e_1) \lambda(h, e_2)$$

因为 $CF(h, e_1) \geq 0, CF(h, e_2) \geq 0$, 则有 $CF(h, e_1 e_2) \geq 0$, 由 (6-35) 式, 有:

$$\lambda(h, e_1) = \frac{1}{1 - CF(h, e_1)}$$

$$\lambda(h, e_2) = \frac{1}{1 - CF(h, e_2)}$$

$$\lambda(h, e_1 e_2) = \frac{1}{1 - CF(h, e_1 e_2)}$$

代入上式, 得:

$$\frac{1}{1 - CF(h, e_1 e_2)} = \frac{1}{1 - CF(h, e_1)} \cdot \frac{1}{1 - CF(h, e_2)}$$

即得:

$$CF(h, e_1 e_2) = CF(h, e_1) + CF(h, e_2) - CF(h, e_1)CF(h, e_2) \quad (6-36)$$

比较 (6-36) 式与 (6-24) 式, 两式完全一致。

- 2) 若 $CF(h, e_1) \geq 0, CF(h, e_2) < 0$, 且 $|CF(h, e_1)| \geq |CF(h, e_2)|$, 则有 $CF(h, e_1 e_2) \geq 0$ 。由 (6-35) 式, 有:

$$\lambda(h, e_1) = \frac{1}{1 - CF(h, e_1)}$$

$$\lambda(h, e_2) = 1 + CF(h, e_2)$$

$$\lambda(h, e_1 e_2) = \frac{1}{1 - CF(h, e_1 e_2)}$$

代入 $\lambda(h, e_1 e_2) = \lambda(h, e_1) \lambda(h, e_2)$ 中, 得:

$$CF(h, e_1 e_2) = \frac{CF(h, e_1) + CF(h, e_2)}{1 + CF(h, e_2)} \quad (6-37)$$

若由 (6-24) 式, 则有:

$$CF(h, e_1 e_2) = \frac{CF(h, e_1) + CF(h, e_2)}{1 - \min(|CF(h, e_1)|, |CF(h, e_2)|)}$$

$$\begin{aligned}
 &= \frac{CF(h, e_1) + CF(h, e_2)}{1 - |CF(h, e_2)|} \\
 &= \frac{CF(h, e_1) + CF(h, e_2)}{1 + CF(h, e_2)}
 \end{aligned}$$

所得结果与 (6-37) 式完全一致。

3) 若 $CF(h, e_1) \geq 0, CF(h, e_2) < 0$, 且 $|CF(h, e_1)| < |CF(h, e_2)|$, 则有 $CF(h, e_1 e_2) < 0$ 。由 (6-35) 式, 有:

$$\begin{aligned}
 \lambda(h, e_1) &= \frac{1}{1 - CF(h, e_1)} \\
 \lambda(h, e_2) &= 1 + CF(h, e_2) \\
 \lambda(h, e_1 e_2) &= 1 + CF(h, e_1 e_2)
 \end{aligned}$$

代入 $\lambda(h, e_1 e_2) = \lambda(h, e_1) \lambda(h, e_2)$ 中, 得:

$$CF(h, e_1 e_2) = \frac{CF(h, e_1) + CF(h, e_2)}{1 - CF(h, e_1)} \quad (6-38)$$

若由 (6-24) 式, 则有:

$$\begin{aligned}
 CF(h, e_1 e_2) &= \frac{CF(h, e_1) + CF(h, e_2)}{1 - \min(|CF(h, e_1)|, |CF(h, e_2)|)} \\
 &= \frac{CF(h, e_1) + CF(h, e_2)}{1 - |CF(h, e_1)|} \\
 &= \frac{CF(h, e_1) + CF(h, e_2)}{1 - CF(h, e_1)}
 \end{aligned}$$

所得结果与 (6-38) 式完全一致。

其他三种情况与上述三种情况是对称的, 得到的结果与 (6-24) 式完全一致, 也就是说, 一般 CF 模型与前述 CF 模型的并行法则完全一致。

(2) 一般 CF 模型的顺序法则

为了推导一般 CF 模型的顺序法则, 我们先给出下述定理。

定理 6.7 对于顺序推理链 $e_1 \rightarrow e_2 \rightarrow h$, 若 e_1, h 相对于 $e_2, \bar{e}_2$ 条件独立, 对给定的 $\lambda(e_2, e_1)$ 、 $\lambda(h, e_2)$ 、 $\lambda(h, \bar{e}_2)$, 则有:

$$\lambda(h, e_1) = \frac{\lambda(e_2, e_1) \lambda(h, e_2) (1 - \lambda(h, \bar{e}_2)) + \lambda(h, \bar{e}_2) (\lambda(h, e_2) - 1)}{\lambda(e_2, e_1) (1 - \lambda(h, \bar{e}_2)) + (\lambda(h, e_2) - 1)} \quad (6-39)$$

证 由 (6-14) 式和 (6-6) 式, 有:

$$\begin{aligned}
 \lambda(h, e_1) &= \frac{P(e_1 | h)}{P(e_1 | \bar{h})} = \frac{\frac{P(h | e_1) P(e_1)}{P(h)}}{\frac{P(h | \bar{e}_1) P(e_1)}{P(h)}} = \frac{P(h | e_1) P(\bar{h})}{P(h | \bar{e}_1) P(h)} \\
 &\quad P(R)
 \end{aligned}$$

对顺序推理链 $e_1 \rightarrow e_2 \rightarrow h$, 且 e_1, h 相对于 $e_2, \bar{e}_2$ 条件独立, 则由 (6-17) 式, 有:

$$P(h|e_1)=P(h|e_2)P(e_2|e_1)+P(h|\bar{e}_2)P(\bar{e}_2|e_1)$$

$$P(\bar{h}|e_1)=P(\bar{h}|e_2)P(e_2|e_1)+P(\bar{h}|\bar{e}_2)P(\bar{e}_2|e_1)$$

代入上式, 得:

$$\begin{aligned}\lambda(h, e_1) &= \frac{(P(h|e_2)P(e_2|e_1)+P(h|\bar{e}_2)P(\bar{e}_2|e_1))P(\bar{h})}{(P(\bar{h}|e_2)P(e_2|e_1)+P(\bar{h}|\bar{e}_2)P(\bar{e}_2|e_1))P(h)} \\ &= \frac{\frac{P(h|e_2)}{P(h)}P(e_2|e_1)+\frac{P(h|\bar{e}_2)}{P(h)}P(\bar{e}_2|e_1)}{\frac{P(\bar{h}|e_2)}{P(\bar{h})}P(e_2|e_1)+\frac{P(\bar{h}|\bar{e}_2)}{P(\bar{h})}P(\bar{e}_2|e_1)} \\ &= \frac{P(e_2|h)\frac{P(e_2|e_1)}{P(e_2)}+P(\bar{e}_2|h)\frac{P(\bar{e}_2|e_1)}{P(\bar{e}_2)}}{P(e_2|\bar{h})\frac{P(e_2|e_1)}{P(e_2)}+P(\bar{e}_2|\bar{h})\frac{P(\bar{e}_2|e_1)}{P(\bar{e}_2)}} \\ &= \frac{P(e_2|h)\frac{P(e_1|e_2)}{P(e_1)}+P(\bar{e}_2|h)\frac{P(e_1|\bar{e}_2)}{P(e_1)}}{P(e_2|\bar{h})\frac{P(e_1|e_2)}{P(e_1)}+P(\bar{e}_2|\bar{h})\frac{P(e_1|\bar{e}_2)}{P(e_1)}} \\ &= \frac{P(e_2|h)P(e_1|e_2)+P(\bar{e}_2|h)P(e_1|\bar{e}_2)}{P(e_2|\bar{h})P(e_1|e_2)+P(\bar{e}_2|\bar{h})P(e_1|\bar{e}_2)}\end{aligned}$$

由 (6-14) 式, 有:

$$\lambda(e_2, e_1) = \frac{P(e_1|e_2)}{P(e_1|\bar{e}_2)}$$

得:

$$P(e_1|e_2) = \lambda(e_2, e_1)P(e_1|\bar{e}_2)$$

代入上式, 得:

$$\begin{aligned}\lambda(h, e_1) &= \frac{P(e_2|h)\lambda(e_2, e_1)+P(\bar{e}_2|h)}{P(e_2|\bar{h})\lambda(e_2, e_1)+P(\bar{e}_2|\bar{h})} \\ &= \frac{P(e_2|h)\lambda(e_2, e_1)+(1-P(e_2|h))}{P(e_2|\bar{h})\lambda(e_2, e_1)+(1-P(e_2|\bar{h}))}\end{aligned}$$

由 (6-14) 式, 还可以有:

$$\begin{cases} \lambda(h, e_2) = \frac{1-P(e_2|h)}{1-P(e_2|\bar{h})} \\ \lambda(h, \bar{e}_2) = \frac{P(\bar{e}_2|h)}{P(\bar{e}_2|\bar{h})} = \frac{1-P(e_2|h)}{1-P(e_2|\bar{h})} \end{cases}$$

对上述方程组, 求解 $P(e_2|h)$ 和 $P(e_2|\bar{h})$, 得:

$$\begin{cases} P(e_2|h) = \lambda(h, e_2) \frac{1 - \lambda(h, \bar{e}_2)}{\lambda(h, e_2) - \lambda(h, \bar{e}_2)} \\ P(e_2|\bar{h}) = \frac{1 - \lambda(h, \bar{e}_2)}{\lambda(h, e_2) - \lambda(h, \bar{e}_2)} \end{cases}$$

将 $P(e_2|h)$ 和 $P(e_2|\bar{h})$ 代入上述 $\lambda(h, e_1)$ 的表达式中, 即得 (6-39) 式:

$$\lambda(h, e_1) = \frac{\lambda(e_2, e_1)\lambda(h, e_2)(1 - \lambda(h, \bar{e}_2)) + \lambda(h, \bar{e}_2)(\lambda(h, e_2) - 1)}{\lambda(e_2, e_1)(1 - \lambda(h, \bar{e}_2)) + (\lambda(h, \bar{e}_2) - 1)}$$

由定义 6.21 说明有顺序推理链 $e_1 \rightarrow e_2 \rightarrow h$ 时, 证据 e_1 使结论 h 成立的信任程度是增加或是降低, 由 $CF(h, e_2)CF(e_2, e_1)$ 的符号决定。因此, $CF(h, e_1)$ 的符号取决于 $CF(e_2, e_1)$ 和 $CF(h, e_2)$ 的以下 4 种不同的组合:

- 1) 若 $CF(e_2, e_1) \geq 0$, $CF(h, e_2) \geq 0$, 则 $CF(h, e_1) \geq 0$ 。
- 2) 若 $CF(e_2, e_1) > 0$, $CF(h, e_2) < 0$, 则 $CF(h, e_1) < 0$ 。
- 3) 若 $CF(e_2, e_1) < 0$, $CF(h, e_2) > 0$, 则 $CF(h, e_1) < 0$ 。
- 4) 若 $CF(e_2, e_1) < 0$, $CF(h, e_2) < 0$, 则 $CF(h, e_1) > 0$ 。

这里, 我们仅以第一种组合情况推导一般 CF 模型的顺序法则。

定理 6.8 若有顺序推理链 $e_1 \rightarrow e_2 \rightarrow h$ 时, 且 e_1 、 h 相对于 e_2 、 $\bar{e}_2$ 条件独立, $CF(e_2, e_1) \geq 0, CF(h, e_2) \geq 0$, 则证据 e_1 支持结论 h 的可信度为:

$$CF(h, e_1) = \frac{CF(e_2, e_1)CF(h, e_2)CF(h, \bar{e}_2)}{CF(e_2, e_1)CF(h, e_2)(1 + CF(h, \bar{e}_2)) - CF(h, \bar{e}_2) + CF(h, e_2) - CF(h, e_2)CF(h, \bar{e}_2)} \quad (6-40)$$

证 因为 $CF(h, e_2) \geq 0$, 故由定义 6.14, 有 $CF(h, \bar{e}_2) < 0$ 。又因为 $CF(h, e_2) \geq 0$, $CF(e_2, e_1) \geq 0$, 故由定义 6.21, 有 $CF(h, e_1) \geq 0$ 。

由 (6-35) 式, 可得:

$$\lambda(e_2, e_1) = \frac{1}{1 - CF(e_2, e_1)}$$

$$\lambda(h, e_2) = \frac{1}{1 - CF(h, e_2)}$$

$$\lambda(h, \bar{e}_2) = 1 + CF(h, \bar{e}_2)$$

$$\lambda(h, e_1) = \frac{1}{1 - CF(h, e_1)}$$

代入 (6-39) 式, 即可得出 (6-40) 式。

其他 3 种组合情况的 $CF(h, e_1)$ 表达式的推导可类似进行, 得出相应的结果。

比较 (6-40) 式与 (6-25) 式, 可见一般 CF 模型的顺序法则的计算比前述 CF 模型的顺序法则的计算要复杂得多。在实际的专家系统中, 采用一般 CF 不确定推理模型会使不确定

性计算的开销太大。但是,一般 CF 模型有坚实的概率论基础,避免了前述 CF 模型中可能出现的推理结论不一致的问题。

6.4 模糊逻辑与模糊推理

自从 1965 年扎德(Zadeh)发表关于模糊集合论的具有开创性的论文以后,模糊理论及应用的研究发展很快,其应用领域几乎包括了自然科学、社会科学和工程技术的每一个分支。1978 年,扎德又提出了模糊逻辑(Fuzzy Logic)和可能性理论(Possibility Theory),立即引起了人工智能领域的广泛关注。经过众多研究者的不断努力,模糊逻辑和可能性理论已经得到进一步发展,并在专家系统不确定推理模型的设计中显示了很强的生命力。人们认为,人工智能的更大突破在很大程度上有赖于人工智能与模糊逻辑的更深层次上的结合。

6.4.1 模糊集合的定义与运算

在经典集合论中,论域是要讨论的问题所涉及到的对象的全体组成的一个普通集合,通常用大写字母 U、V、W 等表示论域。论域 U 的子集 A 在经典集合论中可以有以下两种表示方式:

1) A 为满足某种性质 $p(x)$ 的对象集合,即

$$A = \{x | x \in U, \text{ 且 } x \text{ 满足 } p(x)\}$$

2) 用特征函数表示,即

$$\mu_A(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$$

也就是说,若对象 x 属于集合 A,则 x 的特征函数 $\mu_A(x)=1$;若对象 x 不属于集合 A,则 x 的特征函数 $\mu_A(x)=0$ 。因此,经典集合论只能表示出对象“非此即彼”或“非真即伪”。但是,现实世界中许多事物的性质或特征的取值边界是模糊的,使得对事物的分类存在“亦此亦彼”的现象,不能简单地把某一事物划归某个集合或者排除在这个集合之外。因此,需要把经典集合扩展到模糊集合,并形成完整的模糊集合论和模糊推理的方法。

在经典集合论中,若论域 U 中的子集 A 和 B 的运算用特征函数来表示,则并集 $A \cup B$ 、交集 $A \cap B$ 、补集 $\bar{A}$ 的特征函数分别为:

$$\begin{aligned} \mu_{A \cup B}(x) &= \max\{\mu_A(x), \mu_B(x)\} & \forall x \in U \\ \mu_{A \cap B}(x) &= \min\{\mu_A(x), \mu_B(x)\} & \forall x \in U \\ \mu_{\bar{A}}(x) &= 1 - \mu_A(x) & \forall x \in U \end{aligned}$$

1. 模糊集合的定义与表示

定义 6.22 论域 $U=\{x\}$ 上的集合 A 可由隶属函数 $\mu_A(x)$ 表示, $\mu_A(x)$ 在闭区间 $[0, 1]$ 中的取值称为 x 属于模糊集合 A 的隶属度,若隶属度越接近于 1,则 x 属于 A 的程度就越大,反之就越小。

这就是说, 论域 $U=\{x\}$ 上的模糊集合是指 U 中的元素 x 具有某种特征或性质的元素集合, 论域元素总是分明的, 但元素 x 属于集合 A 的程度是不分明的, 因此, 集合 A 是一个模糊集合。

例如, 论域 U 是 0~120 之间的年龄值, 模糊集合 “年轻” 可以用隶属函数表示为:

$$\mu_{\text{年轻}}(x) = \frac{1}{1 + (\frac{x}{30})^2} \quad x=0,1,2,\dots,120$$

这个 “年轻” 的隶属函数的函数图形如图 6.3 所示。

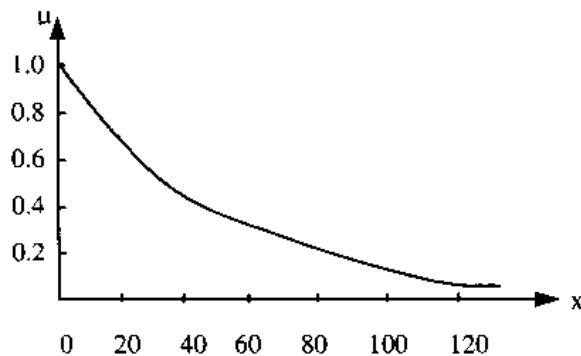


图 6.3 “年轻” 的隶属函数的函数图形

模糊集合还有一种表示形式, 由定义 6.23 和定义 6.24 给出。

定义 6.23 设论域 U 是有限域, 即 $U=\{x_1, x_2, \dots, x_n\}$, U 上的任一模糊集合 A 可表示为:

$$\begin{aligned} A &= \mu_A(x_1)/x_1 + \mu_A(x_2)/x_2 + \dots + \mu_A(x_n)/x_n \\ &= \sum_{i=1}^n \mu_A(x_i)/x_i \end{aligned}$$

其中, $\mu_A(x_i)$ 是 x_i 属于 A 的隶属度。这里, $+$ 与 Σ 不是表示相加与求和, $\mu_A(x_i)/x_i$ 也不是分数, 它们只是为表示模糊集而引用的符号。若 $\mu_A(x_i)=0$, 则模糊集 A 的上述表示中的相应 $\mu_A(x_i)/x_i$ 项可以省略。

例如, 模糊集 “年轻” 可以表示为:

$$\text{年轻} = \sum_{i=0}^{120} (1 + (\frac{x_i}{30})^2)^{-1} / x_i$$

设论域 $U=\{1,2,\dots,9\}$, 若 A 为接近 5 的整数集合, 则 A 可以表示为:

$$A=0.1/1+0.2/2+0.4/3+0.7/4+1/5+0.7/6+0.4/7+0.2/8+0.1/9$$

定义 6.24 设论域 U 是无限域, U 上的任一模糊集 A 可表示为:

$$A = \int_{x \in U} \mu_A(x) / x$$

同样, $\int$ 不是积分符号, 只是表示无限论域上的一个模糊集的符号。

例如, 设论域 U 是实数集 R , A 为小实数的集合, 则 A 可以表示为:

$$A = \int_R (1+x^2)^{-1} / x$$

定义 6.25 当且仅当对于论域 U 中的任意元素 x , 有 $\mu_A(x) = \mu_B(x)$, 则称两个模糊集 A 和 B 相等, 记为 $A=B$ 。

定义 6.26 若 $\forall x \in U$, 有 $\mu_A(x) \leq \mu_B(x)$, 则称模糊集 A 是模糊集 B 的子集, 记为 $A \subseteq B$ 。若 $A \subseteq B$, 且 $\forall x \in U$, 有 $\mu_A(x) < \mu_B(x)$, 则称模糊集 A 是模糊集 B 的真子集, 记为 $A \subset B$ 。

由定义 6.25 和定义 6.26, 模糊集有下述性质。

性质 6.7 设 ϕ 表示隶属函数恒为 0 的模糊集, 即空模糊集, U 表示隶属函数恒为 1 的模糊集, 即论域, 则有:

- 1) 最大、最小模糊集的存在性: $\phi \subseteq A \subseteq U$ 。
- 2) 模糊集的自反性: $A \subseteq A$ 。
- 3) 模糊集的对称性: 若 $A \subseteq B$, $B \subseteq A$, 则 $A=B$ 。
- 4) 模糊集的传递性: 若 $A \subseteq B$, $B \subseteq C$, 则 $A \subseteq C$ 。

2. 模糊集合的运算

模糊集合是经典集合的扩展, 因此, 可类似经典集合定义模糊集合的并集、交集和补集的运算。

定义 6.27 设 A 、 B 是 U 上的模糊集, A 和 B 的并集 $A \cup B$ 、交集 $A \cap B$ 和补集 $\bar{A}$ 的隶属函数定义分别为:

$$\begin{aligned}\mu_{A \cup B}(x) &= \max(\mu_A(x), \mu_B(x)) = \mu_A(x) \vee \mu_B(x) \\ \mu_{A \cap B}(x) &= \min(\mu_A(x), \mu_B(x)) = \mu_A(x) \wedge \mu_B(x) \\ \mu_{\bar{A}}(x) &= 1 - \mu_A(x)\end{aligned}$$

在模糊集合论中, 常用 “ $\vee$ ” 表示 $\max$, 用 “ $\wedge$ ” 表示 $\min$, 分别称为取极大、取极小运算。不要把它们与谓词逻辑中的析取符号 “ $\vee$ ” 与合取符号 “ $\wedge$ ” 混淆起来, 在不同应用场合, 它们的含义是不同的。

定义 6.28 按照论域 U 分别是有限域和无限域, 模糊集 A 和 B 的并、交和补的计算分别为:

- 1) 论域 $U = \{x_1, x_2, \dots, x_n\}$, 且 $A = \sum_{i=1}^n \mu_A(x_i) / x_i$, $B = \sum_{i=1}^n \mu_B(x_i) / x_i$, 则

$$A \cup B = \sum_{i=1}^n (\mu_A(x_i) \vee \mu_B(x_i)) / x_i$$

$$A \cap B = \sum_{i=1}^n (\mu_A(x_i) \wedge \mu_B(x_i)) / x_i$$

$$\bar{A} = \sum_{i=1}^n (1 - \mu_A(x_i)) / x_i$$

- 2) 论域 U 为无限域, 且 $A = \int_{x \in U} \mu_A(x) / x$, $B = \int_{x \in U} \mu_B(x) / x$, 则

$$A \cup B = \int_{x \in U} (\mu_A(x) \vee \mu_B(x)) / x$$

$$A \cap B = \int_{x \in U} (\mu_A(x) \wedge \mu_B(x)) / x$$

$$\bar{A} = \int (1 - \mu_A(x)) / x$$

例 6.5 设论域 $U = \{x_1, x_2, x_3, x_4, x_5\}$, 且有

$$A = 0.2/x_1 + 0.7/x_2 + 1/x_3 + 0.5/x_5$$

$$B = 0.5/x_1 + 0.3/x_2 + 0.1/x_4 + 0.7/x_5$$

计算 $A \cup B$ 、 $A \cap B$ 和 $\bar{A}$ 。

解 由定义 6.28, 可得:

$$\begin{aligned} A \cup B &= (0.2 \vee 0.5)/x_1 + (0.7 \vee 0.3)/x_2 + (1 \vee 0)/x_3 + (0 \vee 0.1)/x_4 + (0.5 \vee 0.7)/x_5 \\ &= 0.5/x_1 + 0.7/x_2 + 1/x_3 + 0.1/x_4 + 0.7/x_5 \end{aligned}$$

$$\begin{aligned} A \cap B &= (0.2 \wedge 0.5)/x_1 + (0.7 \wedge 0.3)/x_2 + (1 \wedge 0)/x_3 + (0 \wedge 0.1)/x_4 + (0.5 \wedge 0.7)/x_5 \\ &= 0.2/x_1 + 0.3/x_2 + 0.5/x_5 \end{aligned}$$

$$\begin{aligned} \bar{A} &= (1 - 0.2)/x_1 + (1 - 0.7)/x_2 + (1 - 1)/x_3 + (1 - 0)/x_4 + (1 - 0.5)/x_5 \\ &= 0.8/x_1 + 0.3/x_2 + 1/x_4 + 0.5/x_5 \end{aligned}$$

性质 6.8 设 A 、 B 、 C 是论域 U 上的模糊集, 模糊集的并、交、补运算有下述性质:

①幂等律 $A \cup A = A$

$$A \cap A = A$$

②吸收律 $A \cup (A \cap B) = A$

$$A \cap (A \cup B) = A$$

③交换律 $A \cup B = B \cup A$

$$A \cap B = B \cap A$$

④分配律 $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

⑤结合律 $(A \cup B) \cup C = A \cup (B \cup C)$

$$(A \cap B) \cap C = A \cap (B \cap C)$$

⑥0—1 律 $A \cup \phi = A$

$$A \cup U = U$$

$$A \cap \phi = \phi$$

$$A \cap U = A$$

⑦复原律 $\bar{\bar{A}} = A$

⑧对偶律 $\overline{A \cup B} = \bar{A} \cap \bar{B}$

$$\overline{A \cap B} = \bar{A} \cup \bar{B}$$

证 可根据模糊集交、并、补运算的定义, 通过简单的计算即可证明上述模糊集运算的性质。现举例说明。

证明分配律的第一式。由定义 6.27, 有:

$$\mu_{A \cup (B \cap C)}(x) = \mu_A(x) \vee \mu_{B \cap C}(x)$$

$$\begin{aligned}
 &= \mu_A(x) \vee (\mu_B(x) \wedge \mu_C(x)) \\
 &= (\mu_A(x) \vee \mu_B(x)) \wedge (\mu_A(x) \vee \mu_C(x)) \\
 &= \mu_{A \cup B}(x) \wedge \mu_{A \cap C}(x) = \mu_{(A \cup B) \cap (A \cap C)}(x)
 \end{aligned}$$

由定义 6.25, 得:

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

证明对偶律的第一式。由定义 6.27, 有:

$$\begin{aligned}
 \mu_{\overline{A \cup B}}(x) &= 1 - \mu_{A \cup B}(x) \\
 &= 1 - (\mu_A(x) \vee \mu_B(x)) \\
 &= (1 - \mu_A(x)) \wedge (1 - \mu_B(x)) \\
 &= \mu_{\overline{A}}(x) \wedge \mu_{\overline{B}}(x) \\
 &= \mu_{\overline{A} \cap \overline{B}}(x)
 \end{aligned}$$

由定义 6.25, 得:

$$\overline{A \cup B} = \overline{A} \cap \overline{B}$$

一般来说, 模糊集的互补律不成立, 即: $A \cup \overline{A} \neq U$, $A \cap \overline{A} \neq \Phi$ 。例如, 若 $\mu_A(x) = 0.2$, 则有 $\mu_{\overline{A}}(x) = 1 - \mu_A(x) = 0.8$, 但是, $\mu_{\overline{A \cup \overline{A}}}(x) = \mu_A(x) \vee \mu_{\overline{A}}(x) = 0.8 \neq 1$, $\mu_{\overline{A \cap \overline{A}}}(x) = \mu_A(x) \wedge \mu_{\overline{A}}(x) = 0.2 \neq 0$ 。这一事实表明, 模糊集不再具有经典集合的“非此即彼”或“非真即伪”的分明性, 这正是模糊集的本质特征。

上述关于模糊集并、交、补运算的定义是由扎德给出的, 因此也称为扎德算子。在模糊集合论的研究中, 人们根据不同的研究方法的需要, 提出了另外一些算子, 它们既可用于模糊集的并、交运算, 也可被认为是在模糊集上另外增加的运算。下面列出其中较多的几种算子。

1) 有界和算子“ $\oplus$ ”与有界积算子“ $\odot$ ”

$$\begin{aligned}
 A \oplus B &: \min\{1, \mu_A(x) + \mu_B(x)\} \\
 A \odot B &: \max\{0, \mu_A(x) + \mu_B(x) - 1\}
 \end{aligned}$$

2) 概率和算子“ $\hat{+}$ ”与实数积算子“ $\cdot$ ”

$$\begin{aligned}
 A \hat{+} B &: \mu_A(x) + \mu_B(x) - \mu_A(x) \times \mu_B(x) \\
 A \cdot B &: \mu_A(x) \times \mu_B(x)
 \end{aligned}$$

3) 爱因斯坦 (Einstein) 和算子“ $\overset{+}{\varepsilon}$ ”与爱因斯坦积算子“ $\overset{\cdot}{\varepsilon}$ ”

$$\begin{aligned}
 A \overset{+}{\varepsilon} B &: \frac{\mu_A(x) + \mu_B(x)}{1 + \mu_A(x) \times \mu_B(x)} \\
 A \overset{\cdot}{\varepsilon} B &: \frac{\mu_A(x) \times \mu_B(x)}{1 + (1 - \mu_A(x)) \times (1 - \mu_B(x))}
 \end{aligned}$$

4) 亚格尔 (Yager) 和算子 “ $\dot{Y}$ ” 与亚格尔积算子 “ $\dot{Y}$ ”

$$A \dot{Y} B: \min \{1, [(\mu_A(x))^P + (\mu_B(x))^P]^{1/P}\}$$

$$A \dot{Y} B: 1 - \min \{1, [(1 - \mu_A(x))^P + (1 - \mu_B(x))^P]^{1/P}\}$$

其中, 常数 P 在 $[1, +\infty]$ 上选取。

上述运算中, 取极大 “ $\vee$ ”、有界和 “ $\oplus$ ”、概率和 “ $\hat{+}$ ”、爱因斯坦和 “ $\dot{+}$ ”、亚格尔和 “ $\dot{Y}$ ” 分别对应于模糊集的 “并” 运算, 统称为 “并型运算” 算子; 取极小 “ $\wedge$ ”、有界积 “ $\odot$ ”、实数积 “ $\cdot$ ”、爱因斯坦积 “ $\dot{\cdot}$ ”、亚格尔积 “ $\dot{Y}$ ” 分别对应于模糊集的 “交” 运算, 统称为 “交型运算” 算子。

模糊集还有一个重要的运算, 这就是笛卡尔乘积运算。

定义 6.29 设 $A_1, A_2, \dots, A_n$ 分别是论域 $U_1, U_2, \dots, U_n$ 上的模糊集, $A_1, A_2, \dots, A_n$ 的笛卡尔乘积记为 $A_1 \times A_2 \times \dots \times A_n$, 它是论域 $U = U_1 \times U_2 \times \dots \times U_n$ 上的一个模糊集, 其隶属函数定义为:

$$\mu_{A_1 \times A_2 \times \dots \times A_n}(x_1, x_2, \dots, x_n) = \mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2) \wedge \dots \wedge \mu_{A_n}(x_n)$$

若论域是有限域, 模糊集的笛卡尔乘积为:

$$A_1 \times A_2 \times \dots \times A_n = \sum (\mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2) \wedge \dots \wedge \mu_{A_n}(x_n)) / (x_1, x_2, \dots, x_n)$$

若论域是无限域, 模糊集的笛卡尔乘积为:

$$A_1 \times A_2 \times \dots \times A_n = \int_U (\mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2) \wedge \dots \wedge \mu_{A_n}(x_n)) / (x_1, x_2, \dots, x_n)$$

例 6.6 设 $U_1 = U_2 = \{3, 5, 7\}$, $A_1 = 0.5/3 + 1/5 + 0.6/7$, $A_2 = 1/3 + 0.6/5$, 计算 $A_1 \times A_2$ 。

$$\begin{aligned} \text{解 } A_1 \times A_2 &= (0.5 \wedge 1)/(3, 3) + (1 \wedge 1)/(5, 3) + (0.6 \wedge 1)/(7, 3) \\ &\quad + (0.5 \wedge 0.6)/(3, 5) + (1 \wedge 0.6)/(5, 5) + (0.6 \wedge 0.6)/(7, 5) \\ &= 0.5/(3, 3) + 1/(5, 3) + 0.6/(7, 3) + 0.5/(3, 5) + 0.6/(5, 5) + 0.6/(7, 5) \end{aligned}$$

例 6.7 设 $U = \{x | 0 \leq x \leq 1\}$, $A = \int_U (2x - 1)^2 / x$, 计算 $\bar{A}$ 、 $A \cap \bar{A}$ 、 $A \cup \bar{A}$ 、 $A \times \bar{A}$ 。

$$\text{解 } \bar{A} = \int_U (1 - \mu_A(x)) / x = \int_U 4x(1 - x) / x$$

$$A \cap \bar{A} = \int_U \min((2x - 1)^2, 4x(1 - x)) / x$$

$$A \cup \bar{A} = \int_U \max((2x - 1)^2, 4x(1 - x)) / x$$

$$A \times \bar{A} = \int_{U \times U} \min((2x - 1)^2, 4y(1 - y)) / (x, y)$$

3. 模糊关系

模糊关系是经典集合的普通关系的扩展。普通关系描述两个集合的元素之间是否关联, 模糊关系描述两个集合的元素之间的关联程度有多大。

定义 6.30 设 U 和 V 分别是论域, 模糊关系 R 是笛卡尔乘积 $U \times V = \{(x, y) | x \in U, y \in V\}$ 中的模糊集, R 的隶属函数表示为 $\mu_R(x, y)$ 。

若 $U = \{x_1, \dots, x_m\}, V = \{y_1, \dots, y_n\}$, 隶属度 $\mu_{ij} = \mu_R(x_i, y_j)$ 表示 U 中的元素 x_i 与 V 中的元素 y_j 的关联程度, 则二元模糊关系 R 可以表示成隶属度矩阵的形式:

$$R = \begin{bmatrix} \mu_{11} & \cdots & \mu_{1n} \\ \vdots & & \\ \mu_{m1} & \cdots & \mu_{mn} \end{bmatrix}$$

一般地, 笛卡尔乘积 $U_1 \times U_2 \times \cdots \times U_n$ 上的 n 元模糊关系 R 的隶属函数表示为:

$$\mu_R(x_1, x_2, \dots, x_n)$$

论域 $U \times U$ 上的模糊关系也称为论域 U 上的模糊关系。

例如, 设 $U = \{x_1, x_2, x_3\}$ 表示三个人的集合, $U \times U$ 上表示“彼此熟悉”的模糊关系 R 可以表示为:

$$R = 1/(x_1, x_1) + 0.7/(x_1, x_2) + 0.5/(x_1, x_3) + 0.9/(x_2, x_1) + 1/(x_2, x_2) \\ + 0.4/(x_2, x_3) + 0.5/(x_3, x_1) + 0.1/(x_3, x_2) + 1/(x_3, x_3)$$

也可以表示为隶属度矩阵的形式:

$$R = \begin{bmatrix} 1 & 0.7 & 0.5 \\ 0.9 & 1 & 0.4 \\ 0.5 & 0.1 & 1 \end{bmatrix}$$

式中, $\mu_{ii} = 1, i = 1, 2, 3$, 表示每个人最熟悉自己; $\mu_{32} = \mu_R(x_3, x_2) = 0.1$, 表示 x_3 对 x_2 稍稍有点熟悉, 而 $\mu_{23} = \mu_R(x_2, x_3) = 0.4$, 表示 x_2 对 x_3 比较熟悉, 显然, x_2 对 x_3 的熟悉程度高于 x_3 对 x_2 的熟悉程度。

由于模糊关系也是模糊集, 因此, 可类似模糊集来定义模糊关系的包含、相等、并、交、补的运算。

定义 6.31 设 R_1, R_2 是 $U \times V$ 上的两个模糊关系, 则有:

1) 包含: 若 $R_1 \subseteq R_2$, 当且仅当 $\mu_{R_1}(x, y) \leq \mu_{R_2}(x, y), \forall x \in U, \forall y \in V$ 。

2) 相等: 若 $R_1 = R_2$, 当且仅当 $\mu_{R_1}(x, y) = \mu_{R_2}(x, y), \forall x \in U, \forall y \in V$ 。

3) 模糊关系 R_1 和 R_2 的并集 $R_1 \cup R_2$ 的隶属函数为:

$$\mu_{R_1 \cup R_2}(x, y) = \max(\mu_{R_1}(x, y), \mu_{R_2}(x, y)) = \mu_{R_1}(x, y) \vee \mu_{R_2}(x, y)$$

4) 模糊关系 R_1 和 R_2 的交集 $R_1 \cap R_2$ 的隶属函数为:

$$\mu_{R_1 \cap R_2}(x, y) = \min(\mu_{R_1}(x, y), \mu_{R_2}(x, y)) = \mu_{R_1}(x, y) \wedge \mu_{R_2}(x, y)$$

5) 模糊关系 R 的补集 $\bar{R}$ 的隶属函数为:

$$\mu_{\bar{R}}(x, y) = 1 - \mu_R(x, y)$$

定义 6.32 设 R 是 $U \times U$ 上的模糊关系, 定义下述几个特殊的模糊关系:

1) 逆模糊关系。 R 的逆模糊关系记为 R^T , R^T 的隶属函数为:

$$\mu_{R^T}(x, y) = \mu_R(y, x) \quad \forall x, y \in U$$

2) 对称模糊关系。若 R 满足:

$$\mu_R(x, y) = \mu_R(y, x) \quad \forall x, y \in U$$

则称 R 是对称模糊关系。

3) 恒等模糊关系。若 R 满足:

$$\mu_R(x, y) = \begin{cases} 1 & \text{若 } x = y \\ 0 & \text{若 } x \neq y \end{cases} \quad \forall x, y \in U$$

则称 R 是恒等模糊关系, 简记为 I 。实际上, I 就是一个普通关系。

4) 全 0 模糊关系。若 R 满足:

$$\mu_R(x, y) = 0 \quad \forall x, y \in U$$

则称 R 是全 0 模糊关系, 简记为 O 。

5) 全 1 模糊关系。若 R 满足:

$$\mu_R(x, y) = 1 \quad \forall x, y \in U$$

则称 R 是全 1 模糊关系, 简记为 E 。

模糊关系有一个独特的运算, 就是模糊关系的合成。

定义 6.33 若 R_1 是论域 $U \times V$ 上的模糊关系, R_2 是论域 $V \times W$ 上的模糊关系, 则 R_1 和 R_2 的合成 $R_1 \circ R_2$ 是 $U \times W$ 上的模糊关系, $R_1 \circ R_2$ 的隶属度定义为:

$$\begin{aligned} \mu_{R_1 \circ R_2}(x, z) &= \max(\min(\mu_{R_1}(x, y), \mu_{R_2}(y, z))) \\ &= \bigvee_{y \in V} (\mu_{R_1}(x, y) \wedge \mu_{R_2}(y, z)) \quad \forall (x, z) \in (U \times W) \end{aligned}$$

由于二元模糊关系可以用一个隶属度矩阵来表示, 因此, 模糊关系的并、交、补和合成运算也有相应的矩阵运算形式。

定义 6.34 若有二元模糊关系 $A = [a_{ij}]_{m \times n}$, $B = [b_{ij}]_{m \times n}$, 矩阵元素 a_{ij} 和 b_{ij} 分别是 A 和 B 中相应的隶属度 μ_{ij} , 则模糊关系 A 和 B 的并、交、补运算分别是:

$$A \cup B = [a_{ij} \vee b_{ij}]_{m \times n}$$

$$A \cap B = [a_{ij} \wedge b_{ij}]_{m \times n}$$

$$\bar{A} = [1 - a_{ij}]_{m \times n}$$

若有二元模糊关系 $A = [a_{ij}]_{m \times q}$, $B = [b_{kj}]_{q \times n}$, 则模糊关系 A 和 B 的合成 $A \circ B$ 为:

$$A \circ B = [r_{ij}]_{m \times n}$$

其中

$$r_{ij} = \max_{k=1}^q (\min(a_{ik}, b_{kj}))$$

记为

$$r_{ij} = \bigvee_{k=1}^q (a_{ik} \wedge b_{kj})$$

例 6.8 若有二元模糊关系 $A = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, $B = \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.7 \end{bmatrix}$, 计算 A 、 $A \cup B$ 、 $A \cap B$ 和 $A \circ B$ 。

解 $A = \begin{bmatrix} 1-0.1 & 1-0.2 \\ 1-0.3 & 1-0.1 \end{bmatrix} = \begin{bmatrix} 0.9 & 0.8 \\ 0.7 & 0.9 \end{bmatrix}$

$$A \cup B = \begin{bmatrix} 0.1 \vee 0.3 & 0.2 \vee 0.2 \\ 0.3 \vee 0.1 & 0.1 \vee 0.7 \end{bmatrix} = \begin{bmatrix} 0.3 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}$$

$$A \cap B = \begin{bmatrix} 0.1 \wedge 0.3 & 0.2 \wedge 0.2 \\ 0.3 \wedge 0.1 & 0.1 \wedge 0.7 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 \\ 0.1 & 0.1 \end{bmatrix}$$

$$A \circ B = [r_{ij}]_{2 \times 2}$$

$$\begin{aligned} r_{11} &= \bigvee (a_{11} \wedge b_{11}, a_{12} \wedge b_{21}) = \bigvee (0.1 \wedge 0.3, 0.2 \wedge 0.1) \\ &= \bigvee (0.1, 0.1) = 0.1 \end{aligned}$$

$$\begin{aligned} r_{12} &= \bigvee (a_{11} \wedge b_{12}, a_{12} \wedge b_{22}) = \bigvee (0.1 \wedge 0.2, 0.2 \wedge 0.7) \\ &= \bigvee (0.1, 0.2) = 0.2 \end{aligned}$$

$$\begin{aligned} r_{21} &= \bigvee (a_{21} \wedge b_{11}, a_{22} \wedge b_{21}) = \bigvee (0.3 \wedge 0.3, 0.1 \wedge 0.1) \\ &= \bigvee (0.3, 0.1) = 0.3 \end{aligned}$$

$$\begin{aligned} r_{22} &= \bigvee (a_{21} \wedge b_{12}, a_{22} \wedge b_{22}) = \bigvee (0.3 \wedge 0.2, 0.1 \wedge 0.7) \\ &= \bigvee (0.2, 0.1) = 0.2 \end{aligned}$$

$$A \circ B = \begin{bmatrix} r_{11} & r_{12} \\ r_{21} & r_{22} \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.2 \end{bmatrix}$$

4. 模糊变换

定义 6.35 设

$$A = \{ \mu_A(u_1), \mu_A(u_2), \dots, \mu_A(u_n) \}$$

是论域 U 上的模糊集, R 是 $U \times V$ 上的模糊关系, 则

$$A \circ R = B$$

称为模糊变换。

例 6.9 设模糊集 A 和模糊关系 R 分别为:

$$A = \{0.2, \quad 0.5, \quad 0.3\}$$

$$R = \begin{bmatrix} 0.2 & 0.7 & 0.1 & 0 \\ 0 & 0.4 & 0.5 & 0.1 \\ 0.2 & 0.3 & 0.4 & 0.1 \end{bmatrix}$$

求模糊集 $B=A \circ R$ 。

解 由模糊变换的合成运算, 可得

$$B=A \circ R=\{0.2, 0.4, 0.5, 0.1\}$$

例 6.10 设对某产品进行评判, 评判的标准是: 质量(u_1)、价格(u_2)、售后服务(u_3), 它们构成了论域 U :

$$U=\{u_1, u_2, u_3\}$$

由若干评委根据评判标准对产品进行模糊评判, 评判的等级是: 好(v_1)、较好(v_2)、一般(v_3)、差(v_4), 它们构成了论域 V :

$$V=\{v_1, v_2, v_3, v_4\}$$

若对“质量”(u_1)评判的结果是: 60%的评委认为是“好”, 20%的评委认为是“较好”, 20%的评委认为是“一般”, 则对该产品的“质量”的评价是: $\{0.6, 0.2, 0.2, 0\}$ 。

若对“价格”(u_2)的评价是: $\{0.8, 0.1, 0.1, 0\}$

若对“售后服务”(u_3)的评价是: $\{0.3, 0.3, 0.3, 0.1\}$

则可得出多名评委对产品的模糊评价 R 为:

$$R = \begin{bmatrix} 0.6 & 0.2 & 0.2 & 0 \\ 0.8 & 0.1 & 0.1 & 0 \\ 0.3 & 0.3 & 0.3 & 0.1 \end{bmatrix}$$

若根据客户的反馈意见, 30%的客户最关心的产品的质量(u_1), 30%的客户最关心产品的价格(u_2), 40%的客户最关心产品的售后服务(u_3), 则它们构成了 U 上的一个模糊向量 A :

$$A=\{0.3, 0.3, 0.4\}$$

由 R 和 A 可得出对该产品的模糊综合评判为 B :

$$B=A \circ R$$

$$= \{0.3, 0.3, 0.4\} \circ \begin{bmatrix} 0.6 & 0.2 & 0.2 & 0 \\ 0.8 & 0.1 & 0.1 & 0 \\ 0.3 & 0.3 & 0.3 & 0.1 \end{bmatrix}$$

$$=\{0.3, 0.3, 0.3, 0.1\}$$

B 是 V 上的模糊集, 由 B 可得出综合评判结果: 30%评委认为该产品是“好”, 30%的评委认为该产品是“较好”, 30%评委认为该产品是“一般”, 10%的评委认为该产品是“差”。当然, 上述结论是用客户反馈意见 A 对评委评判意见 R 进行了模糊变换后的结果。

6.4.2 模糊知识表示与模糊匹配

我们首先说明模糊知识的表示方式, 然后讨论模糊知识的匹配问题。

1. 模糊知识表示

(1) 模糊命题

人们在日常生活中经常会用到一些模糊概念或模糊数据, 例如:

李明是一个年轻人, 他的身高在 1.75m 左右。

这里, “年轻”是一个模糊概念, “1.75m 左右”是一个模糊数据。另外, 人们在表述一个事件时, 通常会对事件发生的可能性或确信程度作出判断, 例如:

今年冬季不会太冷的可能性很大。

这里, 用模糊语言值“很大”描述了模糊事件“不会太冷”出现的可能性或确信程度。

含有模糊概念、模糊数据或带有确信程度的语句称为模糊命题。它的一般表示形式为:

$$x \quad \text{is} \quad A$$

或者

$$x \quad \text{is} \quad A \quad (\text{CF})$$

其中, x 是论域上的变量, 用以代表所论对象的属性; A 是模糊概念或模糊数, 用相应的模糊集及隶属函数刻画; CF 是该模糊命题的确信度或相应事件发生的可能性程度, 它既可以是一个确定的数, 也可以是一个模糊数或模糊语言值。

所谓模糊语言值是指表示大小、长短、轻重、快慢、多少等程度的修饰词。具体应用时可根据实际需要来确定自己的模糊语言值集合, 例如, 表示大小程度的一种模糊语言值集合可以是:

$$V = \{\text{最大, 极大, 很大, 相当大, 比较大, 稍大, 稍小, 比较小, 相当小, 很小, 极小, 最小}\}$$

扎德等人主张对模糊语言值用定义在 $[0, 1]$ 上的模糊集来表示, 例如, 若 $\mu_A(u)$ 表示 u 是“大”的隶属函数, 则 u 是“极大”、“很大”、“相当大”、“比较大”、“稍大”、“略大”的隶属函数分别是:

$$\mu_{\text{极大}}(u) = \mu_A^4(u)$$

$$\mu_{\text{很大}}(u) = \mu_A^2(u)$$

$$\mu_{\text{相当大}}(u) = \mu_A^{1.5}(u)$$

$$\mu_{\text{比较大}}(u) = \mu_A^{0.8}(u)$$

$$\mu_{\text{稍大}}(u) = \mu_A^{0.5}(u)$$

$$\mu_{\text{略大}}(u) = \mu_A^{0.3}(u)$$

(2) 模糊规则

模糊规则的一般形式为:

if E then H (CF, λ)

其中, E 是用模糊命题表示的模糊条件, 它既可以是由单个模糊命题表示的简单条件, 也可以是由多个模糊命题构成的组合条件; H 是用模糊命题表示的模糊结论; CF 是模糊规则的可信度因子, 它既可以是一个确定的数, 也可以是一个模糊数或模糊语言值; λ 是规则的阈值, 用于指出规则可被使用的限制。

2. 模糊匹配

由于模糊规则的模糊条件是模糊命题, 欲之匹配的证据也是模糊命题, 因此需要采用模糊匹配的方法来计算两个模糊命题的相似程度, 即匹配度。例如, 设有下述模糊规则及证据:

if x is 小 then y is 大 (0.6)
 x is 较小

其中, x 是论域 U 上的一个变元, 规则的前提条件 “x is 小” 可表示成模糊集 A, 规则的阈值 $\lambda=0.6$, 证据 “x is 较小” 可表示成模糊集 B:

$$U=\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$A=1/1+0.8/2+0.6/3+0.4/4+0.2/5$$

$$B=1/1+0.89/2+0.77/3+0.63/4+0.45/5$$

为了确定规则的条件是否可与证据模糊匹配, 就需要对两个模糊集 A 和 B 来计算匹配度 $\delta(A, B)$, 若 $\delta(A, B) \geq \lambda$, 就认为 A 与 B 匹配。

(1) 贴近度

贴近度是指两个模糊概念互相贴近的程度, 可用来作为匹配度。

定义 6.36 设 A 与 B 分别是论域 $U=\{u_1, u_2, \dots, u_n\}$ 上的表示两个模糊概念的模糊集, 则它们的贴近度定义为:

$$(A, B) = \frac{1}{2} [A \cdot B + (1 - A \odot B)]$$

其中, $A \cdot B$ 称为 A 与 B 的内积, $A \odot B$ 称为 A 与 B 的外积, 分别为:

$$A \cdot B = \bigvee_U (\mu_A(u_i) \wedge \mu_B(u_i))$$

$$A \odot B = \bigwedge_U (\mu_A(u_i) \vee \mu_B(u_i))$$

当用贴近度作为匹配度时, 贴近度越大表示越匹配。若贴近度大于等于规则指定的阈值时, 就认为规则的模糊条件与证据匹配。

例 6.11 设 $U=\{a, b, c, d, e, f\}$

$$A=0.6/a+0.8/b+1/c+0.8/d+0.6/e+0.4/f$$

$$B=0.4/a+0.6/b+0.8/c+1/d+0.8/e+0.6/f$$

求 A 与 B 的贴近度。

解 由定义 6.36, 可计算得出:

$$A \cdot B = 0.4 \vee 0.6 \vee 0.8 \vee 0.8 \vee 0.6 \vee 0.4 = 0.8$$

$$A \odot B = 0.6 \wedge 0.8 \wedge 1 \wedge 1 \wedge 0.8 \wedge 0.6 = 0.6$$

$$(A, B) = \frac{1}{2} [0.8 + (1 - 0.6)] = 0.6$$

$$\delta(A, B) = (A, B) = 0.6$$

(2) 语义距离

可以通过计算两个模糊集 A 与 B 的语义距离 $d(A, B)$ 来计算 A 与 B 的匹配度。语义距离有多种计算方法。

定义 6.37 设 A 与 B 分别是论域 $U = \{u_1, u_2, \dots, u_n\}$ 上的表示两个模糊概念的模糊集, 则它们之间的海明距离定义为:

$$d(A, B) = \frac{1}{n} \sum_{i=1}^n |\mu_A(u_i) - \mu_B(u_i)|$$

欧几里德距离定义为:

$$d(A, B) = \frac{1}{\sqrt{n}} \times \sqrt{\sum_{i=1}^n (\mu_A(u_i) - \mu_B(u_i))^2}$$

明可夫斯基距离定义为:

$$d(A, B) = \left[\frac{1}{n} \times \sum_{i=1}^n |\mu_A(u_i) - \mu_B(u_i)|^q \right]^{1/q} \quad q \geq 1$$

切比雪夫距离定义为:

$$d(A, B) = \max_{1 \leq i \leq n} |\mu_A(u_i) - \mu_B(u_i)|$$

实际上, 海明距离和欧几里德距离是明可夫斯基距离的特例, 当 $q=1$ 时, 就得到海明距离; 当 $q=2$ 时, 就得到欧几里德距离。

由语义距离 $d(A, B)$, 可得出匹配度为:

$$\delta(A, B) = 1 - d(A, B)$$

例 6.12 设 $U = \{u_1, u_2, u_3\}$

$$A = 0.3/u_1 + 0.5/u_2 + 0.2/u_3$$

$$B = 0.5/u_1 + 0.8/u_2 + 0.4/u_3$$

由定义 6.37, 可计算 A 与 B 的海明距离为:

$$d(A, B) = \frac{1}{3} \times (|0.3 - 0.5| + |0.5 - 0.8| + |0.2 - 0.4|) = 0.233$$

欧几里德距离为:

$$\begin{aligned} d(A, B) &= \frac{1}{\sqrt{3}} \times \sqrt{(0.3 - 0.5)^2 + (0.5 - 0.8)^2 + (0.2 - 0.4)^2} \\ &= 0.237 \end{aligned}$$

切比雪夫距离为:

$$d(A,B)=\max\{|0.3-0.5|,|0.5-0.8|,|0.2-0.4|\}=0.3$$

(3) 相似度

除了贴近度和语义距离可用来确定模糊条件与证据是否模糊匹配外,相似度也可作为匹配度用于模糊匹配。相似度也有多种计算方法。

定义 6.38 设 A 与 B 分别是论域 $U=\{u_1, u_2, \dots, u_n\}$ 上的表示两个模糊概念的模糊集, 则 A 与 B 之间的相似度定义为:

$$r(A,B)=\frac{\sum_{i=1}^n \min\{\mu_A(u_i), \mu_B(u_i)\}}{\sum_{i=1}^n \max\{\mu_A(u_i), \mu_B(u_i)\}}$$

算术平均相似度定义为:

$$r(A,B)=\frac{\sum_{i=1}^n \min\{\mu_A(u_i), \mu_B(u_i)\}}{\frac{1}{2} \times \sum_{i=1}^n (\mu_A(u_i) + \mu_B(u_i))}$$

几何平均相似度定义为:

$$r(A,B)=\frac{\sum_{i=1}^n \min\{\mu_A(u_i), \mu_B(u_i)\}}{\sum_{i=1}^n \sqrt{\mu_A(u_i) \times \mu_B(u_i)}}$$

例 6.13 设 $U=\{a, b, c, d\}$

$$A=0.3/a+0.4/b+0.6/c+0.8/d$$

$$B=0.2/a+0.5/b+0.6/c+0.7/d$$

由定义 6.38, 可计算 A 与 B 的相似度为:

$$r(A,B)=\frac{0.3 \wedge 0.2 + 0.4 \wedge 0.5 + 0.6 \wedge 0.6 + 0.8 \wedge 0.7}{0.3 \vee 0.2 + 0.4 \vee 0.5 + 0.6 \vee 0.6 + 0.8 \vee 0.7} = 0.86$$

算术平均相似度为:

$$r(A,B)=\frac{0.3 \wedge 0.2 + 0.4 \wedge 0.5 + 0.6 \wedge 0.6 + 0.8 \wedge 0.7}{\frac{1}{2} \times (0.3 + 0.2 + 0.4 + 0.5 + 0.6 + 0.6 + 0.8 + 0.7)} = 0.93$$

几何平均相似度为:

$$r(A,B)=\frac{0.3 \wedge 0.2 + 0.4 \wedge 0.5 + 0.6 \wedge 0.6 + 0.8 \wedge 0.7}{\sqrt{0.3 \times 0.2} + \sqrt{0.4 \times 0.5} + \sqrt{0.6 \times 0.6} + \sqrt{0.8 \times 0.7}} = 0.93$$

(4) 精确概念与模糊概念的匹配

两个模糊集的匹配可采用上述计算两个模糊集的匹配度来确定是否匹配。实际上, 规则的前提条件与证据也可能一个是精确概念, 另一个是模糊概念。对任意 $u \in U$, $\mu_A(u)$ 表示 u 属于模糊概念 A 的隶属度, 若精确概念 x 与 u 相匹配, 则 x 与 A 的匹配程度就是 $\mu_A(u)$ 。

例 6.14 设 $U=\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$$\text{小}=1/1+0.8/2+0.6/3+0.4/4+0.2/5$$

$$\text{较小}=1/1+0.89/2+0.77/3+0.63/4+0.45/5$$

$$\text{大}=0.2/4+0.4/5+0.6/6+0.8/7+1/8+1/9+1/10$$

且有如下模糊规则:

$$R_1: \text{if } x \text{ is 小 then } y \text{ is } B_1 \quad (0.15)$$

$$R_2: \text{if } x \text{ is 较小 then } y \text{ is } B_2 \quad (0.25)$$

$$R_3: \text{if } x \text{ is 大 then } y \text{ is } B_3 \quad (0.3)$$

若提供的证据为:

$$e: x \text{ is } 5$$

计算证据 e 与三条模糊规则的匹配度

解 由于精确证据分别与模糊集“小”、“较小”、“大”中的 $u=5$ 匹配, 因此, 匹配度分别是 $\mu_{\text{小}}(5)$ 、 $\mu_{\text{较小}}(5)$ 和 $\mu_{\text{大}}(5)$, 即:

$$\delta(\text{小}, 5) = \mu_{\text{小}}(5) = 0.2$$

$$\delta(\text{较小}, 5) = \mu_{\text{较小}}(5) = 0.45$$

$$\delta(\text{大}, 5) = \mu_{\text{大}}(5) = 0.4$$

由于证据 e 与规则 R_1 、 R_2 和 R_3 的前提条件的匹配度都分别大于规则给定的阈值, 即:

$$\delta(\text{小}, 5) > \lambda_1 = 0.15$$

$$\delta(\text{较小}, 5) > \lambda_2 = 0.25$$

$$\delta(\text{大}, 5) > \lambda_3 = 0.3$$

因此, 这三条规则发生冲突, 需要按一定的冲突消解策略选出一条规则来执行。若冲突消解策略为匹配度大的规则优先, 那么, 将选择规则 R_2 执行。

(5) 模糊规则组合条件与证据的匹配

上述讨论的方法都只考虑模糊规则的前提条件是单一条件与单一证据的模糊匹配问题, 实际上, 模糊规则的前提条件可以是多个条件的组合条件。

组合条件与多个证据的模糊匹配的步骤如下:

1) 选择一种计算单一条件与单一证据匹配度的方法, 分别对规则组合条件中的每一个子条件计算出与相应证据的匹配度。

例如对一条规则的组合条件:

$$E=(x_1 \text{ is } A_1) \wedge (x_2 \text{ is } A_2) \wedge (x_3 \text{ is } A_3)$$

及相应证据 E' :

$$x_1 \text{ is } A_1'$$

$$x_2 \text{ is } A_2'$$

$$x_3 \text{ is } A_3'$$

分别计算出 A_i 与 A_i' 的匹配度 $\delta(A_i, A_i')$, $i=1, 2, 3$ 。

2) 选择一种计算总匹配度的方法, 计算组合条件与证据的总匹配度。对于合取组合条件, 计算总匹配度的方法有“取极小”和“相乘”方法, 即:

$$\delta(E, E') = \min\{\delta(A_i, A'_i) | i=1, 2, \dots, n\}$$

或

$$\delta(E, E') = \prod_{i=1}^n \delta(A_i, A'_i)$$

3) 若总匹配度 $\delta(E, E')$ 大于等于规则的阈值 λ , 则可匹配; 否则, 不可匹配。

6.4.3 模糊推理方法

我们首先给出模糊推理的基本模式, 然后讨论模糊推理的几种方法。

1. 模糊推理的基本模式

模糊推理有三种基本的推理模式, 即: 模糊假言推理、模糊拒取式推理和模糊三段论推理。

(1) 模糊假言推理

设 A 和 B 分别是论域 U 和 V 上的模糊集, 且具有下述规则:

$$\text{if } x \text{ is } A \text{ then } y \text{ is } B$$

若有 U 上的模糊集 A' 可以与 A 模糊匹配, 则可推出 $y \text{ is } B'$, 且 B' 是 V 上的模糊集。称这种模糊推理为模糊假言推理。

模糊假言推理可直观地表示为:

规则: if $x \text{ is } A$ then $y \text{ is } B$

证据: $x \text{ is } A'$

结论: $y \text{ is } B'$

对组合条件的规则, 模糊假言推理可以表示为:

规则: if $(x_1 \text{ is } A_1) \wedge (x_2 \text{ is } A_2) \wedge \dots \wedge (x_n \text{ is } A_n)$ then $y \text{ is } B$

证据: $x_1 \text{ is } A'_1 \quad x_2 \text{ is } A'_2 \quad \dots \quad x_n \text{ is } A'_n$

结论: $y \text{ is } B'$

如果在规则和 (或) 证据中带有可信度因子, 则还需要对结论的可信度因子按某种算法进行计算。

(2) 模糊拒取式推理

设 A 和 B 分别是论域 U 和 V 上的模糊集, 且具有下述规则:

$$\text{if } x \text{ is } A \text{ then } y \text{ is } B$$

若有 V 上的模糊集 B' 可以与 B 模糊匹配, 则可推出 $x \text{ is } A'$, 且 A' 是 U 上的模糊集。称这种模糊推理为模糊拒取式推理。

模糊拒取式推理可直观地表示为:

规则: if $x \text{ is } A$ then $y \text{ is } B$

证据: $y \text{ is } B'$

结论: x is A'

(3) 模糊三段论推理

设 A 、 B 、 C 分别是论域 U 、 V 、 W 上的模糊集, 若由规则链

$$\begin{aligned} \text{if } x \text{ is } A \text{ then } y \text{ is } B \\ \text{if } y \text{ is } B \text{ then } z \text{ is } C \end{aligned}$$

可以推出

$$\text{if } x \text{ is } A \text{ then } z \text{ is } C$$

则称这种模糊推理为模糊三段论推理, 或称为模糊顺序推理法则。

2. 简单模糊推理方法

简单模糊推理是指模糊规则的前提条件是单一条件、且规则和证据都不带可信度因子的模糊推理。

按照扎德等人提出的模糊推理方法, 对于规则:

$$\text{if } x \text{ is } A \text{ then } y \text{ is } B$$

首先要构造出模糊集 A 与 B 之间的模糊关系 R , 然后通过 R 与证据的合成求出结论。

如果已知证据是 x is A' , 且 A 与 A' 可以模糊匹配, 则由模糊假言推理得到结论 y is B' , 且模糊集 B' 由下述合成运算求出:

$$B' = A' \circ R$$

现在的问题是如何构造模糊关系 R , 扎德等人分别提出了多种构造 R 的方法。

(1) 扎德方法

扎德提出了两种构造模糊关系的方法, 一种称为条件命题的极大极小规则来构造模糊关系, 记为 R_m ; 另一种称为条件命题的算术规则来构造模糊关系, 记为 R_a 。

设 A 和 B 分别是论域 U 和 V 上的模糊集, 且

$$A = \int_U \mu_A(u) / u \quad B = \int_V \mu_B(v) / v$$

R_m 与 R_a 的定义分别为:

$$\begin{aligned} R_m &= (A \times B) \cup (\bar{A} \times V) \\ &= \int_{U \times V} (\mu_A(u) \wedge \mu_B(v)) \vee (1 - \mu_A(u)) / (u, v) \\ R_a &= (\bar{A} \times V) \oplus (U \times B) \\ &= \int_{U \times V} 1 \wedge (1 - \mu_A(u) + \mu_B(v)) / (u, v) \end{aligned}$$

1) 对于模糊假言推理, 由 R_m 与 R_a 分别求得 B_m' 与 B_a' 分别为:

$$\begin{aligned} B_m' &= A' \circ R_m = A' \circ [(A \times B) \cup (\bar{A} \times V)] \\ B_a' &= A' \circ R_a = A' \circ [(\bar{A} \times V) \oplus (U \times B)] \end{aligned}$$

它们的隶属函数分别为:

$$\mu_{B'_m}(v) = \bigvee_{u \in U} \{ \mu_{A'}(u) \wedge [(\mu_A(u) \wedge \mu_B(v)) \vee (1 - \mu_A(u))] \}$$

$$\mu_{B'_a}(v) = \bigvee_{u \in U} \{ \mu_{A'}(u) \wedge [1 \wedge (1 - \mu_A(u) + \mu_B(v))] \}$$

2) 对于模糊拒取式推理, 由 R_m 与 R_a 分别求得 A'_m 与 A'_a 分别为:

$$A'_m = R_m \circ B' = [(A \times B) \cup (\bar{A} \times V)] \circ B'$$

$$A'_a = R_a \circ B' = [(\bar{A} \times V) \oplus (U \times B)] \circ B'$$

它们的隶属函数分别为:

$$\mu_{A'_m}(u) = \bigvee_{v \in V} \{ [(\mu_A(u) \wedge \mu_B(v) \vee (1 - \mu_A(u))] \wedge \mu_{B'}(v) \}$$

$$\mu_{A'_a}(u) = \bigvee_{v \in V} \{ [1 \wedge (1 - \mu_A(u) + \mu_B(v))] \wedge \mu_{B'}(v) \}$$

例 6.15 设 $U=V=\{1, 2, 3, 4, 5\}$

$$A=1/1+0.5/2$$

$$B=0.4/3+0.6/4+1/5$$

且模糊规则为:

if x is A then y is B

1) 求模糊集 A 与 B 的模糊关系 R_m 与 R_a .

2) 若有模糊证据 x is A' , 且模糊集 A' 为:

$$A'=1/1+0.4/2+0.2/3$$

应用模糊假言推理求结论的模糊集 B'_m 与 B'_a 。

3) 若有模糊证据 y is B' , 且模糊集 B' 为:

$$B'=0.2/1+0.4/2+0.6/3+0.5/4+0.3/5$$

应用模糊拒取式推理求结论的模糊集 A'_m 与 A'_a 。

解 1) A 与 B 的模糊关系 R_m 与 R_a 的第 i 行第 j 列的元素分别表示为 $R_m(i, j)$ 与 $R_a(i, j)$, 且有:

$$R_m(i, j) = (\mu_A(u_i) \wedge \mu_B(v_j) \vee (1 - \mu_A(u_i)))$$

$$R_a(i, j) = 1 \wedge (1 - \mu_A(u_i) + \mu_B(v_j))$$

从而可计算出 R_m 与 R_a 的各元素的值, 例如:

$$\begin{aligned} R_m(1, 3) &= (\mu_A(u_1) \wedge \mu_B(v_3)) \vee (1 - \mu_A(u_1)) \\ &= (1 \wedge 0.4) \vee (1 - 1) \\ &= 0.4 \end{aligned}$$

$$\begin{aligned} R_a(2, 3) &= 1 \wedge (1 - \mu_A(u_2) + \mu_B(v_3)) \\ &= 1 \wedge (1 - 0.5 + 0.4) \\ &= 0.9 \end{aligned}$$

由此可得出 R_m 与 R_a 分别为:

$$R_m = \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R_a = \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.9 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

2) 由模糊假言推理得到结论:

y is B'

若模糊关系采用 R_m , 则结论的模糊集 B' 为 B'_m ; 若模糊关系采用 R_a , 则结论的模糊集 B' 为 B'_a 。可分别求得 B'_m 与 B'_a 为:

$$\begin{aligned} B'_m &= A' \circ R_m \\ &= \{1, 0.4, 0.2, 0, 0\} \circ \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\ &= \{0.4, 0.4, 0.4, 0.6, 1\} \end{aligned}$$

$$\begin{aligned} B'_a &= A' \circ R_a \\ &= \{1, 0.4, 0.2, 0, 0\} \circ \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.9 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\ &= \{0.4, 0.4, 0.4, 0.6, 1\} \end{aligned}$$

即得出结论中的模糊集 B' 分别为:

$$B'_m = 0.4/1 + 0.4/2 + 0.4/3 + 0.6/4 + 1/5$$

$$B'_a = 0.4/1 + 0.4/2 + 0.4/3 + 0.6/4 + 1/5$$

这里, $B'_m = B'_a$ 只是巧合, 一般来说, 它们不一定相同。

3) 由模糊拒取式推理得到结论:

x is A'

若模糊关系采用 R_m , 则结论的模糊集 A' 为 A'_m ; 若模糊关系采用 R_a , 则结论的模糊集 A' 为

A'_s 。可分别求得 A'_m 与 A'_s 为:

$$\begin{aligned}
 A'_m &= R_m \circ B' \\
 &= \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \circ \begin{bmatrix} 0.2 \\ 0.4 \\ 0.6 \\ 0.5 \\ 0.3 \end{bmatrix} \\
 &= \{0.5, 0.5, 0.6, 0.6, 0.6\} \\
 A'_s &= R_s \circ B' \\
 &= \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0.5 & 0.5 & 0.9 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \circ \begin{bmatrix} 0.2 \\ 0.4 \\ 0.6 \\ 0.5 \\ 0.3 \end{bmatrix} \\
 &= \{0.5, 0.6, 0.6, 0.6, 0.6\}
 \end{aligned}$$

即得出结论中的模糊集 A' 分别为:

$$A'_m = 0.5/1 + 0.5/2 + 0.6/3 + 0.6/4 + 0.6/5$$

$$A'_s = 0.5/1 + 0.6/2 + 0.6/3 + 0.6/4 + 0.6/5$$

(2) 麦姆德尼方法

麦姆德尼 (Mamdani) 提出了称为条件命题的最小运算规则来构造模糊关系, 记为 R_c 。模糊关系 R_c 定义为:

$$\begin{aligned}
 R_c &= A \times B \\
 &= \int_{U \times V} \mu_A(u) \wedge \mu_B(v) / (u, v)
 \end{aligned}$$

1) 对于模糊假言推理, 由 R_c 求得结论的模糊集 B'_c 为:

$$\begin{aligned}
 B'_c &= A' \circ R_c \\
 &= A' \circ (A \times B)
 \end{aligned}$$

它的隶属函数为:

$$\mu_{B'_c}(v) = \bigvee_{u \in U} [\mu_{A'}(u) \wedge (\mu_A(u) \wedge \mu_B(v))]$$

2) 对于模糊拒取式推理, 由 R_c 求得结论的模糊集 A'_c 为:

$$\begin{aligned}
 A'_c &= R_c \circ B' \\
 &= (A \times B) \circ B'
 \end{aligned}$$

它的隶属函数为:

$$\mu_{A_c}(u) = \bigvee_{v \in V} [\mu_A(u) \wedge \mu_B(v) \wedge \mu_{B'}(v)]$$

例 6.16 对例 6.15 给出的数据, 应用麦姆德尼方法求推理结论的模糊集。

解 1) A 与 B 的模糊关系 R_c 的第 i 行第 j 列的元素为:

$$R_c(i,j) = \mu_A(u_i) \wedge \mu_B(v_j)$$

从而可计算出 R_c 各元素的值, 得出 R_c 为:

$$R_c = \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0 & 0 & 0.4 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2) 由模糊假言推理得到结论 y is B' , 若模糊关系采用 R_c , 则结论的模糊集 B' 为 B'_c , 且 B'_c 为:

$$\begin{aligned} B'_c &= A' \circ R_c \\ &= \{1, 0.4, 0.2, 0, 0\} \circ \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0 & 0 & 0.4 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ &= \{0, 0, 0.4, 0.6, 1\} \end{aligned}$$

即得出结论中的模糊集 B' 为:

$$B'_c = 0.4/3 + 0.6/4 + 1/5$$

3) 由模糊拒取式推理得出结论 x is A' , 若模糊关系采用 R_c , 则结论的模糊集 A' 为 A'_c , 且 A'_c 为:

$$\begin{aligned} A'_c &= R_c \circ B' \\ &= \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0 & 0 & 0.4 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \circ \begin{bmatrix} 0.2 \\ 0.4 \\ 0.6 \\ 0.5 \\ 0.3 \end{bmatrix} \\ &= \{0.5, 0.5, 0, 0, 0\} \end{aligned}$$

即得出结论中的模糊集 A' 为:

$$A'_c = 0.5/1 + 0.5/2$$

(3) 米祖莫托方法

米祖莫托 (Mizumoto) 等人根据多值逻辑中计算 $T(A \rightarrow B)$ 的定义, 提出了一组构造模糊关系的方法, 由此构造的模糊关系分别记为 R_s 、 R_g 、 R_{sg} 、 R_{gg} 、 R_{gs} 、 R_{ss} 、 R_b 等。其定义分别为:

$$R_s = \int_{U \times V} [\mu_A(u) \xrightarrow{s} \mu_B(v)] / (u, v)$$

其中:

$$\mu_A(u) \xrightarrow{s} \mu_B(v) = \begin{cases} 1 & \mu_A(u) \leq \mu_B(v) \\ 0 & \mu_A(u) > \mu_B(v) \end{cases}$$

$$R_g = \int_{U \times V} [\mu_A(u) \xrightarrow{g} \mu_B(v)] / (u, v)$$

其中:

$$\mu_A(u) \xrightarrow{g} \mu_B(v) = \begin{cases} 1 & \mu_A(u) \leq \mu_B(v) \\ \mu_B(v) & \mu_A(u) > \mu_B(v) \end{cases}$$

$$R_{sg} = \int_{U \times V} \{ [\mu_A(u) \xrightarrow{s} \mu_B(v)] \wedge [(1 - \mu_A(u)) \xrightarrow{g} (1 - \mu_B(v))] \} / (u, v)$$

$$R_{gg} = \int_{U \times V} \{ [\mu_A(u) \xrightarrow{g} \mu_B(v)] \wedge [(1 - \mu_A(u)) \xrightarrow{g} (1 - \mu_B(v))] \} / (u, v)$$

$$R_{gs} = \int_{U \times V} \{ [\mu_A(u) \xrightarrow{g} \mu_B(v)] \wedge [(1 - \mu_A(u)) \xrightarrow{s} (1 - \mu_B(v))] \} / (u, v)$$

$$R_{ss} = \int_{U \times V} \{ [\mu_A(u) \xrightarrow{s} \mu_B(v)] \wedge [(1 - \mu_A(u)) \xrightarrow{s} (1 - \mu_B(v))] \} / (u, v)$$

$$R_b = \int_{U \times V} [(1 - \mu_A(u)) \vee \mu_B(v)] / (u, v)$$

例 6.17 对例 6.15 给出的数据, 应用米祖莫托方法求模糊关系 R_s 与 R_g , 并分别求出模糊假言推理得出的结论。

解 1) A 与 B 的模糊关系 R_s 的第 i 行第 j 列的元素为:

$$R_s(i, j) = \mu_A(u_i) \xrightarrow{s} \mu_B(v_j) = \begin{cases} 1 & \mu_A(u_i) \leq \mu_B(v_j) \\ 0 & \mu_A(u_i) > \mu_B(v_j) \end{cases}$$

由给出的模糊集 A 和 B, 可得出模糊关系 R_s 为:

$$R_s = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

A 与 B 的模糊关系 R_g 的第 i 行第 j 列的元素为:

$$R_g(i, j) = \mu_A(u_i) \xrightarrow{g} \mu_B(v_j) = \begin{cases} 1 & \mu_A(u_i) \leq \mu_B(v_j) \\ \mu_B(v_j) & \mu_A(u_i) > \mu_B(v_j) \end{cases}$$

由给出的模糊集 A 和 B, 可得出模糊关系 R_g 为:

$$R_g = \begin{bmatrix} 0 & 0 & 0.4 & 0.6 & 1 \\ 0 & 0 & 0.4 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

2) 由模糊假言推理得到结论 y is B' , 若模糊关系采用 R_s , 则结论的模糊集 B' 为 B'_s ; 若模糊关系采用 R_g , 则结论的模糊集 B' 为 B'_g 。且可分别计算出 B'_s 和 B'_g 为:

$$\begin{aligned} B'_s &= A' \circ R_s \\ &= \{0.2, 0.2, 0.2, 0.4, 1\} \\ B'_g &= A' \circ R_g \\ &= \{0.2, 0.2, 0.4, 0.6, 1\} \end{aligned}$$

即可得出结论 y is B' 中的模糊集 B' 分别为:

$$\begin{aligned} B'_s &= 0.2/1 + 0.2/2 + 0.2/3 + 0.4/4 + 1/5 \\ B'_g &= 0.2/1 + 0.2/2 + 0.4/3 + 0.6/4 + 1/5 \end{aligned}$$

由例 6.15~例 6.17 可以看出, 对相同的模糊规则及证据使用不同的模糊关系进行推理时, 得到的结论一般是不同的。这说明不同的模糊关系使模糊推理的性能表现出一定的差异。通过分析, 我们可以得知: 无论是对于模糊假言推理还是对于模糊拒取式推理, R_s 、 R_{sg} 、 R_{ss} 是性能比较好的模糊关系, R_g 、 R_{gg} 、 R_{gs} 、 R_c 次之, R_m 、 R_a 、 R_b 等模糊关系的性能较差。

3. 模糊三段论推理方法

模糊三段论推理又称为模糊顺序推理法则。设有以下模糊规则:

$$\begin{aligned} R_1: & \text{ if } x \text{ is } A \text{ then } y \text{ is } B \\ R_2: & \text{ if } y \text{ is } B \text{ then } z \text{ is } C \end{aligned}$$

其中, A、B、C 分别是论域 U、V、W 上的模糊集。由规则 R_1 给出的模糊集 A 与 B 可构造出定义在 $U \times V$ 上的模糊关系 $R(A, B)$, 由规则 R_2 给出的模糊集 B 与 C 可构造出定义在 $V \times W$ 上的模糊关系 $R(B, C)$ 。若选择的构造模糊关系的方法满足模糊三段论推理, 那么可得出 $U \times W$ 上的模糊关系 $R(A, C)$ 为:

$$R(A, C) = R(A, B) \circ R(B, C)$$

若已知证据为 x is A' , 则可由模糊假言推理得出结论 z is C' , 且模糊集 C' 为:

$$C' = A' \circ R(A, C)$$

若已知证据 z is C' , 则可由模糊拒取式推理得出结论 x is A' , 且模糊集 A' 为:

$$A' = R(A, C) \circ C'$$

在前述的各种模糊关系的构成方法中, 有一些能满足模糊三段论推理, 有一些不能满足模糊三段论推理。我们通过下面一个例子予以说明:

例 6.18 设 $U=V=W=\{1, 2, 3, 4, 5\}$

$$A=1/1+0.6/2+0.2/3$$

$$B=0.3/3+0.7/4+1/5$$

$$C=0.09/3+0.49/4+1/5$$

请分别验证构成的模糊关系 R_m 和 R_g 是否满足模糊三段论。

解 给出的模糊集 A、B、C 分别为：

$$A=\{1, 0.6, 0.2, 0, 0\}$$

$$B=\{0, 0, 0.2, 0.7, 1\}$$

$$C=\{0, 0, 0.09, 0.49, 1\}$$

1) 由模糊关系 R_m 的构成方法, 可分别得出:

$$R_m(A, B) = \begin{bmatrix} 0 & 0 & 0.3 & 0.7 & 1 \\ 0.4 & 0.4 & 0.4 & 0.6 & 0.6 \\ 0.8 & 0.8 & 0.8 & 0.8 & 0.8 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R_m(B, C) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0.7 & 0.7 & 0.7 & 0.7 & 0.7 \\ 0.3 & 0.3 & 0.3 & 0.49 & 0.7 \\ 0 & 0 & 0.09 & 0.49 & 1 \end{bmatrix}$$

$$R_m(A, C) = \begin{bmatrix} 0 & 0 & 0.09 & 0.49 & 1 \\ 0.4 & 0.4 & 0.4 & 0.49 & 0.6 \\ 0.8 & 0.8 & 0.8 & 0.8 & 0.8 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

将 $R_m(A, B)$ 与 $R_m(B, C)$ 合成, 得到:

$$R_m(A, B) \circ R_m(B, C) = \begin{bmatrix} 0.3 & 0.3 & 0.3 & 0.49 & 1 \\ 0.4 & 0.4 & 0.4 & 0.49 & 0.6 \\ 0.8 & 0.8 & 0.8 & 0.8 & 0.8 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

显然, $R_m(A, B) \circ R_m(B, C) \neq R_m(A, C)$ 。由此说明 R_m 不满足模糊三段论。

2) 由模糊关系 R_g 的构成方法, 可分别得出:

$$R_g(A, B) = \begin{bmatrix} 0 & 0 & 0.3 & 0.7 & 1 \\ 0 & 0 & 0.3 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R_g(B, C) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0.09 & 1 & 1 \\ 0 & 0 & 0.09 & 0.49 & 1 \\ 0 & 0 & 0.09 & 0.49 & 1 \end{bmatrix}$$

$$R_g(A, C) = \begin{bmatrix} 0 & 0 & 0.09 & 0.49 & 1 \\ 0 & 0 & 0.09 & 0.49 & 1 \\ 0 & 0 & 0.09 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

将 $R_g(A, B)$ 与 $R_g(B, C)$ 合成, 得到

$$R_g(A, B) \circ R_g(B, C) = \begin{bmatrix} 0 & 0 & 0.09 & 0.49 & 1 \\ 0 & 0 & 0.09 & 0.49 & 1 \\ 0 & 0 & 0.09 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

显然, $R_g(A, B) \circ R_g(B, C) = R_g(A, C)$ 。实际上 R_g 满足模糊三段论。

实际上, 满足模糊三段论的模糊关系有: R_c 、 R_s 、 R_g 、 R_{sg} 、 R_{gg} 、 R_{gs} 、 R_{ss} ; 不满足模糊三段论的模糊关系有: R_m 、 R_a 、 R_b 等。

4. 多维模糊推理

多维模糊推理是指组合条件规则的模糊假言推理, 多维模糊推理的一般模式为:

规则: if $(x_1 \text{ is } A_1) \wedge (x_2 \text{ is } A_2) \wedge \cdots \wedge (x_n \text{ is } A_n)$ then $y \text{ is } B$

证据: $x_1 \text{ is } A'_1 \quad x_2 \text{ is } A'_2 \quad \cdots \quad x_n \text{ is } A'_n$

结论: $y \text{ is } B'$

其中, A_i 与 A'_i 是论域 U_i 上的模糊集, $i=1, 2, \cdots, n$, B 与 B' 是论域 V 上的模糊集。

多维模糊推理求结论的模糊集 B' 的方法主要有以下几种方法。

(1) 扎德方法

扎德方法进行多维模糊推理的步骤如下:

1) 求出 $A_1, A_2, \cdots, A_n$ 的交集, 记为 A , 即

$$A = A_1 \cap A_2 \cap \cdots \cap A_n$$

$$= \int_{U_1 \times U_2 \times \dots \times U_n} \mu_{A_1}(u_1) \wedge \mu_{A_2}(u_2) \wedge \dots \wedge \mu_{A_n}(u_n) / (u_1, u_2, \dots, u_n)$$

2) 用一种构造模糊关系的方法构造出 A 与 B 的模糊关系 $R(A, B)$, 记为 $R(A_1, A_2, \dots, A_n, B)$ 。

3) 求出证据中 $A'_1, A'_2, \dots, A'_n$ 的交集, 记为 A' , 即

$$A' = A'_1 \cap A'_2 \cap \dots \cap A'_n$$

$$= \int_{U_1 \times U_2 \times \dots \times U_n} \mu_{A'_1}(u_1) \wedge \mu_{A'_2}(u_2) \wedge \dots \wedge \mu_{A'_n}(u_n) / (u_1, u_2, \dots, u_n)$$

4) 根据模糊假言推理, 由 A' 与 $R(A, B)$ 的合成求出结论的模糊集 B' , 即

$$B' = A' \circ R(A, B)$$

$$= (A'_1 \cap A'_2 \cap \dots \cap A'_n) \circ R(A_1, A_2, \dots, A_n, B)$$

例 6.19 设论域 $U=V=W=\{1, 2, 3, 4, 5\}$, 规则的组合条件分别在 U 与 V 上有模糊集 $A_1=\{1, 0.6, 0, 0, 0\}$ 与 $A_2=\{0, 1, 0.5, 0, 0\}$, 规则的结论在 W 上有模糊集 $B=\{0, 0, 1, 0.8, 0\}$ 。若已知证据分别在 U 与 V 上有模糊集 $A'_1=\{0.8, 0.5, 0, 0, 0\}$ 与 $A'_2=\{0, 0.9, 0.5, 0, 0\}$, 求推理结论的模糊集 B' 。

解 这是一个二维模糊推理 (即 $n=2$) 问题, 我们分别采用 R_a 、 R_m 和 R_s 来求 B' 。

首先分别求出组合条件及证据的交集, 即

$$A_1 \cap A_2 = \{0, 0.6, 0, 0, 0\}$$

$$A'_1 \cap A'_2 = \{0, 0.5, 0, 0, 0\}$$

1) 若用 R_a 构造模糊关系, 则得到

$$R_a(A_1, A_2, B) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0.4 & 0.4 & 1 & 1 & 0.4 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{aligned} B'_a &= (A'_1 \cap A'_2) \circ R_a(A_1, A_2, B) \\ &= \{0.4, 0.4, 0.5, 0.5, 0.4\} \end{aligned}$$

2) 若用 R_m 构造模糊关系, 则得到

$$R_m(A_1, A_2, B) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0.4 & 0.4 & 0.6 & 0.6 & 0.4 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{aligned} B'_m &= (A'_1 \cap A'_2) \circ R_m(A_1, A_2, B) \\ &= \{0.4, 0.4, 0.5, 0.5, 0.4\} \end{aligned}$$

3) 若用 R_s 构造模糊关系, 则得到

$$R_s(A_1, A_2, B) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\begin{aligned} B'_s &= (A'_1 \cap A'_2) \circ R_s(A_1, A_2, B) \\ &= \{0, 0, 0.5, 0.5, 0\} \end{aligned}$$

(2) 祖卡莫托方法

祖卡莫托 (Tsukamoto) 方法进行多维模糊推理的步骤如下:

1) 用一种构造模糊关系的方法对组合条件的每个单一条件中的 A_i 与 B 构造模糊关系

$$R(A_i, B) \quad i=1, 2, \dots, n$$

2) 对每个单一证据 A'_i 求出简单模糊推理的结论 B'_i , 即

$$B'_i = A'_i \circ R(A_i, B) \quad i=1, 2, \dots, n$$

3) 求出所有 B'_i 的交集, 从而得到 B' , 即

$$B' = B'_1 \cap B'_2 \cap \dots \cap B'_n$$

例 6.20 对例 6.19 给出的数据, 若采用 R_s 构造模糊关系, 试用祖卡莫托方法求推理结论的模糊集 B' 。

解 由给出的 A_1 、 A_2 和 B , 按 R_s 构造模糊关系, 得出

$$R_s(A_1, B) = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R_s(A_2, B) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

分别由给出的证据 A'_1 和 A'_2 , 按简单模糊推理, 得出

$$\begin{aligned} B'_{s1} &= A'_1 \circ R_s(A_1, B) \\ &= \{0, 0, 0.8, 0.5, 0\} \end{aligned}$$

$$\begin{aligned} B'_{s2} &= A'_2 \circ R_s(A_2, B) \\ &= \{0, 0, 0.9, 0.5, 0\} \end{aligned}$$

最后得出二维模糊推理结论的模糊集 B'_s 为:

$$B'_s = B'_{s1} \cap B'_{s2}$$

$$=\{0, 0, 0.8, 0.5, 0\}$$

(3) 苏更诺方法

苏更诺 (Sugeno) 方法是采取逐维递推计算方式来求出多维模糊推理的结论的模糊集, 即:

$$B'_1 = A'_1 \circ R(A_1, B)$$

$$B'_2 = A'_2 \circ R(A_2, B'_1)$$

$$\vdots$$

$$B' = B'_n = A'_n \circ R(A_n, B'_{n-1})$$

例 6.21 对例 6.19 给出的数据, 若采用 R_s 构造模糊关系, 试用苏更诺方法求解推理结论的模糊集 B' 。

解 首先由给出的 A_1 和 B , 按 R_s 构造模糊关系, 并计算 B'_{s1} , 即

$$R_s(A_1, B) = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$B'_{s1} = A'_1 \circ R_s(A_1, B)$$

$$=\{0, 0, 0.8, 0.5, 0\}$$

然后由给出的 A_2 和得到的 B'_{s1} , 按 R_s 构造模糊关系, 即

$$R_s(A_2, B'_{s1}) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

最后由给出的 A'_2 和得到的 $R_s(A_2, B'_{s1})$ 的合成, 得出 B'_s , 即

$$B'_s = B'_{s2} = A'_2 \circ R_s(A_2, B'_{s1})$$

$$=\{0, 0, 0.5, 0.5, 0\}$$

6.4.4 带有可信度的模糊推理

随机性与模糊性是知识的两种主要的不确定性, 我们分别在基于可信度的不确定推理和模糊推理中讨论了这两种不确定性知识的表示方法和处理方法。但是, 现实世界中的许多事物不仅具有模糊性, 也同时具有随机性, 这就要求把这两种方法结合起来, 既能表示和处理模糊性, 又能表示和处理随机性。带有可信度的模糊推理就是用来解决此类问题的一种方法。在这种方法中, 由随机性引起的不确定性用可信度 CF 表示, 由模糊性引起的不确定性仍用模糊集表示。

1. 带有可信度的简单模糊推理

带有可信度的简单模糊推理的推理模式为:

规则: if x is A then y is B CF_1

证据: x is A' CF_2

结论: y is B' CF

其中, CF_1 与 CF_2 分别是给出的规则和证据的可信度, CF 是待求的结论的可信度。

由上述推理模式可以看出, 带有可信度的模糊推理需要解决两个问题: 一是如何推出结论 y is B' , 二是如何求出结论的可信度 CF 。实际上, 解决第一个问题的方法就是不带可信度的模糊推理方法, 所以, 问题在于如何在推理过程中求出结论的可信度 CF 。分为以下两种情况讨论。

(1) 条件模糊集与证据模糊集相等

若条件模糊集 A 与证据模糊集 A' 相等, 即 $A=A'$, 则结论的可信度 CF 可分别用下述三种方法来计算:

$$1) CF=CF_1 \times CF_2$$

$$2) CF=\min\{CF_1, CF_2\}$$

$$3) CF=\max\{0, CF_1+CF_2-1\}$$

(2) 条件模糊集与证据模糊集不等

若条件模糊集 A 与证据模糊集 A' 不等, 即 $A \neq A'$, 但 A 与 A' 可模糊匹配, 即 A 与 A' 的匹配度 $\delta(A, A') \geq \lambda$, λ 是规则的阈值, 则结论的可信度 CF 可分别用下述四种方法来计算:

$$1) CF=\delta(A, A') \times CF_1 \times CF_2$$

$$2) CF=\delta(A, A') \times \min\{CF_1, CF_2\}$$

$$3) CF=\delta(A, A') \times \max\{0, CF_1 + CF_2 - 1\}$$

$$4) CF=\min\{\delta(A, A'), CF_1, CF_2\}$$

2. 带有可信度的多维模糊推理

带有可信度的多维模糊推理的推理模式为:

规则: if $(x_1$ is $A_1) \wedge (x_2$ is $A_2) \wedge \cdots \wedge (x_n$ is $A_n)$ then y is B CF_E

证据: x_1 is A'_1 CF_1

x_2 is A'_2 CF_2

$\vdots$ $\vdots$

x_n is A'_n CF_n

结论: y is B' CF

求多维模糊推理的结论可信度 CF 的步骤如下:

首先按照 6.4.2 节介绍的组合条件与多个证据模糊匹配的方法, 求得组合条件与多个证据的总匹配度 $\delta(E, E')$, 总匹配度的计算方法是:

$$\delta(E, E') = \min\{\delta(A_i, A'_i) \mid i=1, 2, \dots, n\}$$

或者

$$\delta(E, E') = \delta(A_1, A'_1) \times \delta(A_2, A'_2) \times \dots \times \delta(A_n, A'_n)$$

若 $\delta(E, E') \geq \lambda$ ，则规则的前提条件与证据可匹配；否则，不可匹配。

若规则的组合条件与证据是可匹配的，则计算证据的总可信度 CF_E 。证据的总可信度的计算方法是：

$$CF_E = \min\{CF_i \mid i=1, 2, \dots, n\}$$

或者

$$CF_E = CF_1 \times CF_2 \times \dots \times CF_n$$

总匹配度与总可信度求出后，多维模糊推理的结论可信度 CF 的计算方法就可等同于简单模糊推理中的计算方法，可分别用下述四种方法来计算：

- 1) $CF = \delta(E, E') \times CF_E \times CF_{E'}$
- 2) $CF = \delta(E, E') \times \min\{CF_E, CF_{E'}\}$
- 3) $CF = \delta(E, E') \times \max\{0, CF_E + CF_{E'} - 1\}$
- 4) $CF = \min\{\delta(E, E'), CF_E, CF_{E'}\}$

3. 带有可信度的并行模糊推理

设有当前证据经过两条独立的模糊推理链分别推出了如下的两个结论：

$$y \text{ is } B'_1 \quad CF_1$$

$$y \text{ is } B'_2 \quad CF_2$$

则可用下述方法计算出当前证据共同支持的结论 $y \text{ is } B'$ 中的模糊集 B' 及结论的可信度 CF ：

$$B' = B'_1 \cap B'_2$$

$$CF = CF_1 + CF_2 - CF_1 \times CF_2$$

习题六

- 6.1 证据的不确定性主要反映在哪些方面？规则的不确定性主要反映在哪些方面？
- 6.2 何谓推理的不确定性？请简述不确定推理的不确定性测试的三种基本计算模式。
- 6.3 设 h_1 、 h_2 、 h_3 分别是三个结论， e 是支持这些结论的一个证据，且已知：

$$\begin{aligned} P(h_1) &= 0.3 & P(h_2) &= 0.4 & P(h_3) &= 0.5 \\ P(e \mid h_1) &= 0.5 & P(e \mid h_2) &= 0.3 & P(e \mid h_3) &= 0.4 \end{aligned}$$

求证据 e 分别支持结论 h_1 、 h_2 和 h_3 的概率。

- 6.4 已知结论 h_1 、 h_2 、 h_3 发生的概率分别为：

$$P(h_1) = 0.4 \quad P(h_2) = 0.3 \quad P(h_3) = 0.3$$

若结论成立时，证据 e_i 出现的概率分别为：

$$P(e_1 | h_1)=0.5 \quad P(e_1 | h_2)=0.6 \quad P(e_1 | h_3)=0.3$$

结论成立时, 证据 e_2 出现的概率分别为:

$$P(e_2 | h_1)=0.7 \quad P(e_2 | h_2)=0.9 \quad P(e_2 | h_3)=0.1$$

且证据 e_1 与 e_2 相互独立, 求:

- 1) 证据 e_1 与 e_2 的合取组合的概率 $P(e_1 e_2)$ 和析取组合的概率 $P(e_1 \vee e_2)$ 。
- 2) 当只有证据 e_1 出现时, $P(h_1 | e_1)$ 、 $P(h_2 | e_1)$ 、 $P(h_3 | e_1)$ 的值各为多少? 比较它们的值, 说明了什么?
- 3) 当 e_1 和 e_2 同时出现时, $P(h_1 | e_1 e_2)$ 、 $P(h_2 | e_1 e_2)$ 、 $P(h_3 | e_1 e_2)$ 的值各为多少? 与 2) 的结果比较, 说明了什么?

6.5 现有下述 4 条规则及规则的不确定性测度分别为:

$$\begin{array}{ll} R_1: a \wedge b \rightarrow c & P(abc)=8 \\ R_2: c \rightarrow d & P(cld)=0.9 \\ R_3: d \rightarrow f & P(dlf)=0.7 \\ R_4: \bar{e} \rightarrow f & P(\bar{e}lf)=0.2 \end{array}$$

若已知证据和结论的不确定性测度分别为:

$$P(a)=0.5 \quad P(b)=0.4 \quad P(c)=0.2 \quad P(d)=0.15 \quad P(e)=0.25 \quad P(f)=0.15$$

求证据 a 、 b 、 c 同时支持结论 f 的不确定性测度 $P(f | abc)$ 。

6.6 设有下述 5 条规则及其规则的可信度为:

$$\begin{array}{ll} R_1: \text{if } e_1 \text{ then } h & CF_1=0.8 \\ R_2: \text{if } e_2 \text{ then } h & CF_2=0.6 \\ R_3: \text{if } e_3 \text{ then } h & CF_3=-0.5 \\ R_4: \text{if } e_4 \wedge (e_5 \vee e_6) \text{ then } e_1 & CF_4=0.7 \\ R_5: \text{if } e_7 \wedge e_8 \text{ then } e_3 & CF_5=0.9 \end{array}$$

且已知有关证据的可信度分别为:

$$\begin{array}{lll} CF(e_2)=0.8 & CF(e_4)=0.5 & CF(e_5)=0.6 \\ CF(e_6)=0.7 & CF(e_7)=0.6 & CF(e_8)=0.9 \end{array}$$

求结论 h 的可信度 $CF(h)$ 。

6.7 设有下述 3 条带有阈限的推理规则:

$$\begin{array}{ll} R_1: \text{if } e_3 \wedge e_4 \wedge e_5 \text{ then } e_1 & (0.6, 0.5) \\ R_2: \text{if } e_1 \text{ then } h & (0.7, 0.3) \\ R_3: \text{if } e_2 \text{ then } h & (0.8, 0.6) \end{array}$$

且已知有关证据的可信度分别为:

$$CF(e_2)=0.9 \quad CF(e_3)=0.8 \quad CF(e_4)=0.6 \quad CF(e_5)=0.7$$

试分别应用求极大值法、加权求和法、有限和法求结论 h 的可信度 $CF(h)$ 。

6.8 设有下述带有加权因子的 3 条推理规则:

- $R_1: \text{if } e_1(0.7) \wedge e_2(0.3) \text{ then } h_1 \quad (0.9, 0.6)$
 $R_2: \text{if } e_3(0.4) \wedge e_4(0.3) \wedge e_5(0.3) \text{ then } h_2 \quad (0.8, 0.5)$
 $R_3: \text{if } e_6(0.3) \wedge h_1(0.5) \wedge h_2(0.2) \text{ then } h \quad (0.6, 0.2)$

且已知有关证据的可信度分别为:

$$\begin{aligned}
 CF(e_1) &= 0.9 & CF(e_2) &= 0.85 & CF(e_3) &= 0.85 \\
 CF(e_4) &= 0.7 & CF(e_5) &= 0.75 & CF(e_6) &= 0.9
 \end{aligned}$$

请应用加权的 uncertain 推理方法求结论 h 的可信度 $CF(h)$ 。

6.9 设有论域 $U = \{x_1, x_3, x_4, x_5\}$, A 与 B 是 U 上的两个模糊集, 且有:

$$\begin{aligned}
 A &= 0.6/x_1 + 0.4/x_2 + 0.3/x_3 + 0.2/x_4 + 0.1/x_5 \\
 B &= 0.8/x_3 + 0.6/x_4 + 1/x_5
 \end{aligned}$$

请分别计算 $A \cap B$, $A \cup B$, $\bar{A}$, $A \oplus B$, $A \odot B$ 。

6.10 学生域 $U = \{u_1, u_2, u_3\}$ 中的 3 名学生 u_1 、 u_2 和 u_3 分别对球类运动域 $V = \{v_1, v_2, v_3, v_4\}$ 中的 4 种球类运动有不同程度的爱好。若学生 u_i 最爱好某项球类运动 v_j , 则打分为“1”; 若最不爱好的 v_j , 则打分为“0”。请按照学生对球类运动爱好的模糊语言设计一个打分表, 并由此给出 U 与 V 的模糊关系。

6.11 设有如下两个模糊关系:

$$R = \begin{bmatrix} 0.4 & 0.5 & 0.1 \\ 0.2 & 0.6 & 0.2 \\ 0.5 & 0.3 & 0.2 \end{bmatrix} \quad R_2 = \begin{bmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \\ 0.6 & 0.4 \end{bmatrix}$$

求 R_1 与 R_2 的合成 $R_1 \circ R_2$ 。

6.12 若对食品的评价标准是: 色 (u_1)、香 (u_2)、味 (u_3), 它们组成评价标准论域 $U = \{u_1, u_2, u_3\}$ 。由若干名评委对食品按色、香、味标准评判打分, 打分的等级是: 好 (v_1)、较好 (v_2)、一般 (v_3) 和差 (v_4), 它们组成等级论域 $V = \{v_1, v_2, v_3, v_4\}$, v_i 的值为选择该等级的评委百分比 (若 30% 的评委认为是好, 则 $v_1 = 0.3$)。若对某种食品评判的结果如下:

- 对“色”的评价: $\{0.6, 0.2, 0.2, 0\}$
 对“香”的评价: $\{0.8, 0.1, 0.1, 0\}$
 对“味”的评价: $\{0.3, 0.3, 0.3, 0.1\}$

且根据该食品的特点和评委的一致意见, 对该食品的评价标准增设了权重, “色”的权重为 0.3, “香”的权重为 0.3, “味”的权重为 0.4。应用模糊变换, 求出对该食品的综合评价, 并对得出的综合评价予以解释。

6.13 设有下述带阈值的模糊规则与证据:

$$\begin{aligned}
 &\text{if } x \text{ is } A \text{ then } y \text{ is } B \quad (0.7) \\
 &x \text{ is } A'
 \end{aligned}$$

且 A 与 A' 分别是论域 $U = \{u_1, u_2, u_3\}$ 上的模糊集:

$$A=0.3/u_1+0.5/u_2+0.2/u_3$$

$$A'=0.5/u_1+0.8/u_2+0.4/u_3$$

1) 分别求 A 与 A' 的贴近度, 海明距离, 欧几里德距离和切比雪夫距离。

2) 匹配度采用哪种方式时, 规则与证据可匹配?

6.14 由 5 名篮球运动员组成论域 $U=V=\{a, b, c, d, e\}$, 设有下述模糊规则:

if x is 球技较好 then y is 得分较多

$x \in U, y \in V$ 。根据不完全记载, 得出对这 5 名运动员的模糊评价为:

$$\text{球技较好: } A=1/a+0.5/b$$

$$\text{得分较多: } B=0.4/c+0.6/d+1/e$$

1) 分别构造出 A 与 B 的模糊关系 $R_m(A, B)$ 与 $R_s(A, B)$ 。

2) 根据一场球赛的表现, 若得出运动员的球技水平的模糊评价为:

$$A'=1/a+0.4/b+0.2/c$$

应用模糊假言推理, 求对 5 名运动员得分能力的模糊评价。

3) 根据一场球赛的表现, 若得出运动员得分能力的模糊评价为:

$$B'=0.2/a+0.4/b+0.6/c+0.5/d+0.3/e$$

应用模糊拒取式推理, 求对 5 名运动员球技水平的模糊评价。

6.15 对 6.14 题, 应用麦姆德尼方法构造模糊关系 $R_c(A, B)$, 并求出模糊假言推理与模糊拒取式推理的模糊结论。

6.16 对 6.14 题, 应用米祖莫托方法构造模糊关系 $R_s(A, B)$ 与 $R_g(A, B)$, 并分别由 $R_s(A, B)$ 与 $R_g(A, B)$ 求出模糊假言推理与模糊拒取式推理的模糊结论。

6.17 对 6.14 题, 若规则是带有可信度和阈值的模糊规则, 即

if x is 球技较好 then y is 得分较多 $CF_1=0.7 \quad \lambda=0.8$

对论域 $U=V=\{a, b, c, d, e\}$ 中运动员的模糊评价为:

$$\text{球技较好: } A=1/a+0.5/b$$

$$\text{得分较多: } B=0.4/c+0.6/d+1/e$$

且根据一场球赛得出运动员的球技水平的模糊评价为:

$$A'=1/a+0.4/b+0.2/c \quad CF_2=0.75$$

1) 计算 A 与 A' 的贴近度, 并由此确定规则与证据是否可匹配?

2) 若规则与证据可匹配, 则构造模糊关系 $R_m(A, B)$, 应用模糊假言推理求出模糊结论, 并应用 4 种方法分别计算出结论的可信度。

6.18 设有论域:

$$U=V=W=\{1, 2, 3, 4, 5, 6\}$$

且设有下述模糊规则:

R_1 : if x is A then y is B

R_2 : if y is B then z is C

R_3 : if x is A then z is C

其中, A, B, C 的模糊集分别为:

$$A=1/1+0.8/2+0.5/3+0.4/4+0.1/5$$

$$B=0.1/2+0.2/3+0.4/4+0.6/5+0.8/6$$

$$C=0.2/3+0.5/4+0.8/5+1/6$$

请分别采用 R_m 、 R_c 、 R_g 构造模糊关系, 并验证其是否满足模糊三段论。

6.19 设有论域:

$$U=V=W=\{1, 2, 3, 4, 5\}$$

且设有下述模糊规则:

if $(x_1 \text{ is } A_1) \wedge (x_2 \text{ is } A_2)$ then y is B

已知事实为:

$$x_1 \text{ is } A'_1$$

$$x_2 \text{ is } A'_2$$

其中, A_1 、 A_2 、 B 、 A'_1 及 A'_2 的模糊集分别为:

$$A_1=1/1+0.8/2+0.6/3+0.4/4$$

$$A_2=0.2/1+0.4/2+0.6/3+0.8/4+1/5$$

$$B=0.5/3+0.7/4$$

$$A'_1=0.8/1+0.7/2+0.5/3+0.3/4$$

$$A'_2=0.2/1+0.3/2+0.5/3+0.7/4+0.9/5$$

请分别用 R_s 与 R_g 构造模糊关系, 并用扎德方法求出 B'_1 和 B'_2 。

6.20 对 6.19 题, 请用祖卡莫托方法求出 B'_s 。

6.21 对 6.19 题, 请用苏更诺方法求出 B'_s 。

第 7 章 机器学习

机器学习是智能系统不断地积累经验以改善系统性能的过程。20 世纪 80 年代, 机器学习蓬勃发展起来, 逐渐形成成为人工智能研究的一个主流研究领域。主要的学习方法有归纳学习方法和分析学习方法两种。另外, 遗传算法的研究、人工神经网络的兴起, 都给机器学习领域带来了强有力的生命力, 并对人工智能的其他研究与应用领域产生了很大的影响。本章介绍机器学习的有关概念和归纳学习、基于解释的分析学习及遗传算法。人工神经网络将在第 8 章介绍。

7.1 机器学习的概念与方法分类

机器学习的系统模型可以简单地表示为图 7.1 所示形式。其中, 环境向系统提供学习信息; 学习元对这些信息进行整理、分析、归纳或类比, 生成新的知识元或改进知识库的组织结构; 执行元以学习后得到的新知识库为基础, 执行一系列任务, 并将执行结果报告学习元, 以完成对新知识库的评价, 指导进一步的学习工作。

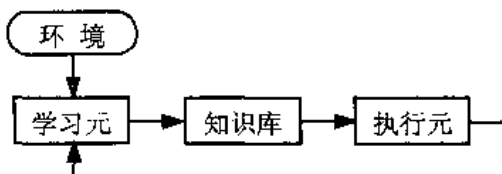


图 7.1 机器学习系统模型的框图

从系统行为的标准来看, 机器学习是使系统具有自适应能力。对于环境的变化, 系统也随之产生适应性的变化, 从而使得系统下一次完成同样或类似的任务比上一次更有效。在系统内部, 机器学习表现为新知识结构的建立和完善以及知识的更新。

一个机器学习系统通常应该具有如下主要特征:

- 1) 目的性。系统的学习行为有高度的目的性, 即系统必须知道学习什么。
 - 2) 结构性。系统必须具备适当的知识存储结构来记忆学到的知识, 能够修改和完善知识表示与知识的组织形式。
 - 3) 有效性。系统学习到的知识应受到实践的检验, 新知识必须对改善系统的行为起到有益的作用。
 - 4) 开放性。系统的能力应在实际使用过程中、在同环境进行信息交互的过程中不断改进。
- 机器学习方法的种类很多, 而且, 机器学习的研究正处在发展的高峰时期, 各种新思想、

新方法和新技术不断涌现, 很难对它们进行系统的分类。这里, 仅列举下述一些比较重要的学习方法。

(1) 归纳学习

归纳学习 (Inductive Learning) 是由环境提供一系列正例和反例, 通过归纳推理, 机器将这些例子进行推广, 产生一个或一组一般的概念描述。

(2) 类比学习

类比是人类求解问题的一种基本方法。通过类比学习 (Learning by Analogy), 可以把两个不同领域中的理论的相似性抽取出来, 用一个领域求解问题的思想来指导另一个领域的问题求解。

(3) 基于解释的学习

基于解释的学习 (Explanation-Based Learning) 是指已知理论和这一理论的一个实例, 通过解释为什么这一实例可以用这一理论来求解, 从而产生关于待学概念的一个解释。在这一过程中, 系统得到启发, 从而建立一套求解类似问题的规范。这种先通过演绎解释, 然后通过归纳来构造一般原则的学习方法称为基于解释的学习方法。

(4) 遗传算法

遗传算法 (Genetic Algorithm) 是借鉴生物遗传机制的一种随机化非线性计算算法。它通过对对象系统第一代群体及其后代群体中的个体不断地选优汰劣与随机性遗传变异来获得对象系统的一个非线性映射模型。这个映射模型就是采用遗传算法对对象系统的第一代群体学习的结果, 也就是对象系统的知识表示。

(5) 人工神经网络

人工神经网络 (Artificial Neural Networks) 由一些类似神经元的单元及单元间带权的连接弧构成, 其中每个单元具有一个状态。通过各类实例 (样本) 的反复训练, 人工神经网络不断调整各连接弧上的权值及神经元的内部状态, 当神经网络达到一定的稳定状态后, 神经网络就能恰当地反映网络输入模式对输出模式的映射关系, 从而达到学习的目的。一个对象系统通过实例训练获得一个稳定权值分布的神经网络, 就是这个对象系统的知识表示。

7.2 归纳学习

归纳学习是从特例推导一般规则的学习方法。人类知识的增长主要得益于归纳学习, 例如, 通过观察“燕子会飞”、“麻雀会飞”等特殊事实, 可以归纳得到“鸟会飞”这样的一般结论。虽然归纳得到的新知识不如演绎推理的结论那样可靠, 但是对于认识的发展与完善具有重要的启发意义。

7.2.1 归纳学习的基本概念

在机器学习领域中, 可把归纳学习形式化地描述为使用训练实例以导出一般规则的搜索

问题。全体可能的实例构成实例空间，全体可能的一般规则构成规则空间，学习的任务就是要完成实例空间和规则空间之间同时协调的搜索。归纳学习的双空间模型可表示为如图 7.2 所示的形式。按照双空间模型建立的归纳学习系统的执行过程可以大致描述为：根据规则空间提供的一般规则，由实验规划过程通过对实例空间的搜索完成实例选择，并将这些选中的活跃实例提交解释过程；解释过程对实例经过适当的转换，将活跃实例变换为规则空间中的特定概念，以引导对规则空间的搜索。

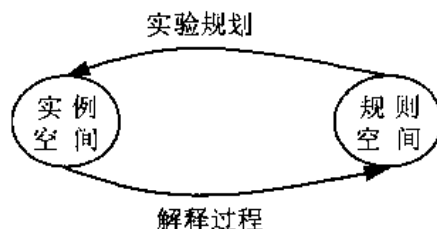
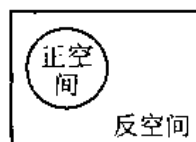
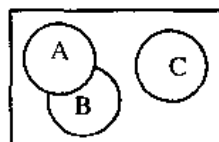


图 7.2 归纳学习的双空间模型

归纳学习方法可以划分为单概念学习和多概念学习两类。这里，概念是指用某种描述语言表示的谓词，当应用于概念的正实例时，谓词为真；应用于反实例时，谓词为假。从而，概念谓词将实例空间划分为正、反两个子集。对于单概念学习，学习的目的是从概念空间（即规则空间）中寻找某个与实例空间一致的概念。对于多概念学习，学习的目的是在概念空间（即规则空间）中找出若干概念描述，对于每个概念描述，实例空间中均有相应的空间与之相应。这两类学习方法对实例空间划分的差异如图 7.3 所示。在图 7.3 (b) 中，实例空间中划分出 A、B、C 三个子空间，每个子空间内的实例又可由相应的概念划分为正、反两个子集。



(a) 单概念学习的实例空间划分



(b) 多概念学习的实例空间划分

图 7.3 单概念学习与多概念学习系统实例空间的划分

单概念学习按行为方式的差异可分为数据驱动、规则驱动和模型驱动等三类方法。其中，数据驱动学习方法有 Mitchell 的描述空间法；规则驱动学习方法有 BACON 程序和 Quinlan 的 ID3 方法；模型驱动学习方法有 Dietterich 和 Michalski 提出的 Induce 算法。

典型的多概念学习方法和机器学习系统有 Michalski 的 AQ11、DENDRAL 和 AM 程序等。多概念学习可以划分成多个单概念学习来完成，例如，AQ11 对每一概念的学习均采用 Induce 算法来实现。多概念学习与单概念学习的差别在于多概念学习方法还必须解决多个概念学习之间的冲突问题。

7.2.2 基于描述空间的归纳学习方法

在 Mitchell 提出的描述空间学习方法中, 根据概念的一般性和特殊性把概念空间表示成偏序集的形式。偏序集的最大元素 G 称为零描述, 零描述是概念空间中恒真的概念描述, 是概念空间中最一般的概念的集合。偏序集的最小元素 S 是所有的训练实例, 是概念空间中最特殊的概念的集合。我们通过下述的 2 个例子来说明基于描述空间的归纳学习方法。

例 7.1 设实例空间是一组实体, 每一个实体用“大小”和“形状”两个特征描述, 若“大小”的属性值有: small、large, “形状”的属性值有: square、circle。对这个实例空间, 请给出其概念空间偏序集的最大元素和最小元素。

解 每个实体有“大小”和“形状”两个属性, 可以用 $(x \ y)$ 表示实体概念空间的最一般概念, x 描述实体的“大小”, y 描述实体的“形状”, $(x \ y)$ 是该实体概念空间中的零描述。偏序集的最大元素 $G=\{(x \ y)\}$ 。

偏序集的最小元素 S 是所有的训练实例, 故有 $S=\{(small \ square), (small \ circle), (large \ square), (large \ circle)\}$ 。

对于给定的训练实例, 利用偏序集可以非常紧凑地表示与训练实例相一致的合理假设集合 (一组概念描述) H , H 由两个子集 G 和 S 所限定, H 是由 G 和 S 以及 G 与 S 之间的元素构成的偏序集, 即

$$H=G \cup S \cup \{K \mid S < K < G\}$$

其中, “ $<$ ” 表示概念空间中的偏序关系。

在例 7.1 的实体空间的概念空间 H 中还有一组与训练实例相一致的概念描述 $K=\{(small \ y), (large \ y), (x \ square), (x \ circle)\}$, G 、 K 、 S 的偏序关系为 $S < K < G$ 。概念空间 $H=G \cup K \cup S$ 的偏序集表示如图 7.4 所示。

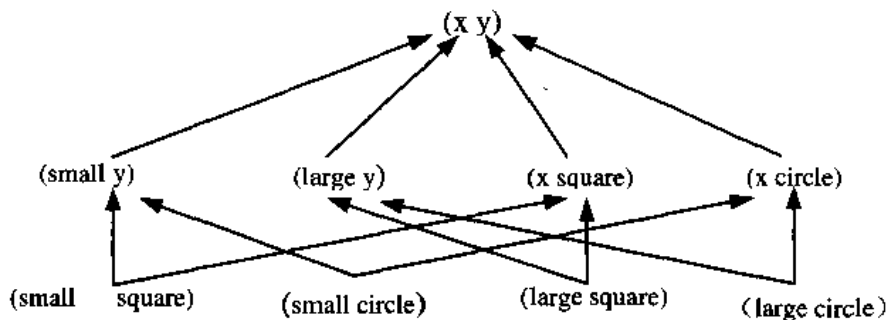


图 7.4 概念空间的偏序集表示

Mitchell 将概念空间中所有合理假设称为描述空间, 一个描述空间 H 是所有当前为止所得到的与训练实例相一致的概念的集合。当提供训练实例后, 通过删去描述空间中与当前实例不相容 (不一致) 的候选假设, 从而对描述空间不断求精。一旦描述空间中只包含惟一的元素时, 就发现了所要学习的概念。根据对描述空间不断求精来获得概念的学习思想, 可提

出下述消除候选者算法。

消除候选者算法

1) 初始化概念空间 $H=\langle G, S \rangle$, 其中, G 为零描述集合, S 为训练集合, 初始化时, S 中只含第一个正例。

2) 接受一个新的训练实例。如果实例为正例, 则从 G 中删去所有与该例不相容的概念, 并更新集合 S , 尽可能小地对 S 进行一般化, 以相容这个新的正实例; 如果实例为反例, 则首先从 S 中删去所有与该例相容的概念, 并更新集合 G , 尽可能小地特殊化 G 中的元素, 以便它们不相容这个反例。

3) 如果 $G \neq S$, 则重复步骤 2); 否则输出 H 。

例 7.2 对例 7.1 给出的概念空间, 应用消除候选者学习算法, 说明学习得到一般概念为“园”的学习步骤和学习结果。

解 由例 7.1 给出的概念空间可得知, “园”的一般概念显然应表示为 $(x \text{ circle})$ 。这一概念可采用消除候选者算法获得, 使用该算法的学习过程如下:

1) 初始化概念空间 $H=\langle G, S \rangle$, 其中:

$$G=\{(x \text{ y})\}$$

$$S=\{(\text{small square}), (\text{small circle}), (\text{large square}), (\text{large circle})\}$$

2) 若首先提供一个正例(small circle), 由消除候选者算法步骤 2) 对 $H=\langle G, S \rangle$ 进行更新:

$G=\{(x \text{ y})\}$ 中的概念 $(x \text{ y})$ 是最一般概念, 与提供的正例 (small circle) 相容, 故仍有 $G=\{(x \text{ y})\}$ 。

删去 S 中与提供的正例不相容的元素(large square), 对 S 中的另外 2 个元素尽可能小地一般化, 即使这 2 个元素分别为 (small y) 和 $(x \text{ circle})$, 从而使它们与提供的正例(small circle) 相容, 故有 $S=\{(\text{small y}), (\text{small circle}), (x \text{ circle})\}$ 。

3) 此时有 $G \neq S$, 若再提供一个反例(large square), 继续对 H 更新:

由于 S 中的元素都与提供的反例不相容, 故 S 不变, 仍有 $S=\{(\text{small y}), (\text{small circle}), (x \text{ circle})\}$ 。

对 G 中的元素 $(x \text{ y})$ 尽可能小地特殊化, 可由 $(x \text{ y})$ 得到与提供的反例(large square) 不相容的 2 个元素(small y) 和 $(x \text{ circle})$, 故有 $G=\{(\text{small y}), (x \text{ circle})\}$ 。

4) 此时有 $G \neq S$, 若再提供一个正例(large circle), 继续对 H 更新:

删去 G 中与提供的正例(large circle) 不相容的元素 (small y), 故有 $G=\{(x \text{ circle})\}$ 。

删去 S 中与提供的正例 (large circle) 不相容的元素 (small y) 和 (small circle), 故有 $S=\{(x \text{ circle})\}$ 。

5) 此时有 $G=S=\{(x \text{ circle})\}$, 故而算法中止, 由概念空间 $H=\langle G, S \rangle$ 得出经过学习后获得的一般概念 $(x \text{ circle})$ 。

由于待求解问题的概念空间通常是很庞大的, 消除候选者学习算法在训练实例指导下对概念空间进行搜索的开销很大, 因此, 基于描述空间的学习方法在实际应用中受到限制。

7.2.3 基于决策树的归纳学习方法

Hunt 提出的概念学习系统 CLS 是一种基于决策树的归纳学习系统。1979 年, J.R.Quinlan 在 CLS 基础上进行了发展, 提出了 ID3 算法, 该算法不仅能方便地表示概念的“属性—值”信息的结构, 而且能从大量实例中有效地生成相应的决策树模型。

1. 决策树及决策树构造算法 CLS

在 CLS 的决策树中, 节点对应于待分类对象的属性, 由某一节点引出的弧对应于这个属性的可能取值, 叶节点对应于分类的结果。

例如, 假设天气预测分类系统确定用于描述天气的分类有 3 种属性: outlook、humidity 和 windy, 这 3 种属性的值域分别为:

Value Type (outlook)={sunny, overcast, rain}

Value Type (humidity)={high, normal}

Value Type (windy)={true, false}

若对天气实例只需给出 2 种分类结果 N 和 P, 并认为 P 是正实例的分类结果, N 是反实例的分类结果, 那么, 天气预测的 CLS 决策树如图 7.5 所示。

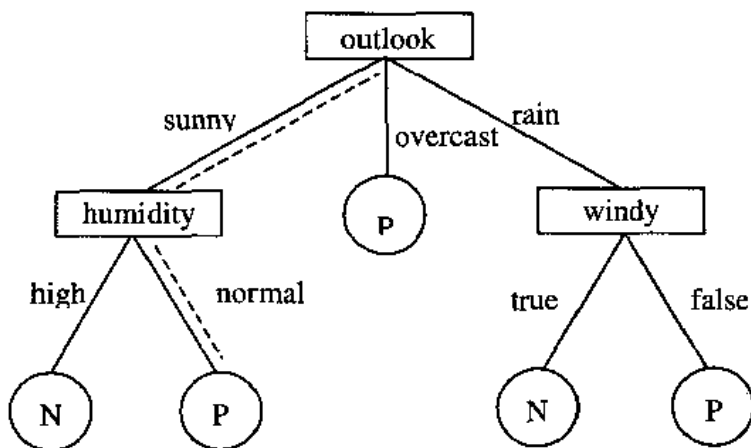


图 7.5 天气预测的 CLS 决策树

由此决策树可对任何的天气实例进行分类。例如, 若有一个天气实例为 (sunny, normal, true), 那么由决策树得出该天气实例的分类结果为 P。在图 7.5 中, 从树根至一个叶节点 P 的虚线给出了这一实例由决策树分类的分类过程。

设分类对象的属性表为 $AttrList=\{A_1, A_2, \dots, A_n\}$, 每一个属性 A_i 的值域记为 $Value\ Type(A_i)$, 值域可以是离散的, 也可以是连续的。把分类的分类结果组成分类结果表 $Class=\{C_1, C_2, \dots, C_m\}$ 。一般应有 $n \geq 1$, $m \geq 2$ 。训练实例集 $T=\{<X, C>\}$ 中的一个元素就是一个训练实例 $<X, C>$, 训练实例 $<X, C>$ 的特征向量为 $X=(a_1, a_2, \dots, a_n)$, 其中 a_i 为这个实例的第 i 个属性 A_i 的取值, C 为这个实例的分类结果, $C \in Class$ 。可递归地描述下述决策树的构造算法 CLS。

决策树构造算法 CLS

- 1) 如果训练实例集 T 中所有实例的分类结果均为 C , 则返回 C 。
- 2) 从属性表 AttrList 中任选一个属性 A_i 作为检测属性。
- 3) 若属性 A_i 的值域 $\text{ValueType}(A_i)$ 中有 s 个不同的取值, 则将 T 分成 s 个子集 $T_1, \dots, T_s$, 每个子集的所有元素的属性 A_i 的取值相同。
- 4) 更新属性表, 从属性表 AttrList 中删去检测属性 A_i 。
- 5) 对每一个子集 T_k , 若子集 T_k 中的所有实例的分类结果均为 C_j , 则生成叶节点 C_j ; 若子集 T_k 中的所有实例的分类结果有两个或两个以上, 则对子集 T_k 和更新后的属性表转至步骤 2), 递归调用 CLS 构造算法, 生成 T_k 的子树。

我们通过下述的例子来说明决策树构造算法 CLS 及其应用。

例 7.3 假设需要根据人员的外貌特征对人员进行分类, 用于人员分类的外貌特征有 3 个, 它们组成人员分类属性表为:

$\text{AttrList}=\{\text{height, hair, eyes}\}$

各属性的值域分别为:

$\text{ValueType}(\text{height})=\{\text{short, tall}\}$

$\text{ValueType}(\text{hair})=\{\text{blond, red, dark}\}$

$\text{ValueType}(\text{eyes})=\{\text{blue, brown}\}$

若对人员进行分类的结果有两种, 分别用 “+” 和 “-” 表示, 组成分类结果表为:

$\text{Class}=\{+, -\}$

若提供学习的训练实例集为:

$T=\{<(\text{short, blond, blue}), +>, <(\text{tall, blond, brown}), ->, \\ <(\text{tall, red, blue}), +>, <(\text{short, dark, blue}), ->, \\ <(\text{tall, dark, blue}), ->, <(\text{tall, blond, blue}), +>, \\ <(\text{tall, dark, brown}), ->, <(\text{short, blond, brown}), ->\}$

应用 CLS 构造算法构造决策树, 说明构造过程, 画出构造的决策树。

解 执行 CLS 算法构造决策树的构造过程如下:

1) 若选取属性表 $\text{AttrList}=\{\text{height, hair, eyes}\}$ 中的属性 height 作为第 1 个检测属性, 则将 T 分为 2 个子集 T_1 和 T_2 , 其中, T_1 是属性 height 取值为 short 的训练实例子集, T_2 是属性 height 取值为 tall 的训练实例子集。故有:

$T_1=\{<(\text{short, blond, blue}), +>, <(\text{short, dark, blue}), ->, \\ <(\text{short, blond, brown}), ->\}$

$T_2=\{<(\text{tall, blond, brown}), ->, <(\text{tall, red, blue}), +>, \\ <(\text{tall, dark, blue}), ->, <(\text{tall, blond, blue}), +>, \\ <(\text{tall, dark, brown}), ->\}$

从属性表中删去检测属性 height , 故新的属性表为:

AttrList={hair, eyes}

至此, 生成的决策树如图 7.6 (a) 所示。

2) 由于 T_1 和 T_2 中的实例的分类结果仍有两种, 故需按新的属性表递归构造 T_1 和 T_2 的子树。

若选取新属性表中的属性 hair 作为第 2 个检测属性, 且由于 hair 的值域 Value-Type(hair)={blond, red, dark} 有 3 个取值, 故将 T_1 分为 3 个子集 T_{11} 、 T_{12} 和 T_{13} , 将 T_2 也分为 3 个子集 T_{21} 、 T_{22} 和 T_{23} , 它们分别为:

$T_{11}=\{ \langle (\text{short, blond, blue}), + \rangle, \langle (\text{tall, blond, brown}), - \rangle \}$

$T_{12}=\{ \quad \}$

$T_{13}=\{ \langle (\text{short, dark, blue}), - \rangle \}$

$T_{21}=\{ \langle (\text{tall, blond, brown}), - \rangle, \langle (\text{tall, blond, blue}), + \rangle \}$

$T_{22}=\{ \langle (\text{tall, red, blue}), + \rangle \}$

$T_{23}=\{ \langle (\text{tall, dark, blue}), - \rangle, \langle (\text{tall, dark, brown}), - \rangle \}$

从属性表中删去检测属性 hair, 故新的属性表为:

AttrList={eyes}

至此, 生成的决策树如图 7.6 (b) 所示。

3) 由于 T_{13} 、 T_{22} 和 T_{23} 中的实例的分类结果只有一种, 故无需对 T_{13} 、 T_{22} 和 T_{23} 中的实例划分子集, 并用 T_{13} 的实例分类结果“-”作为叶节点代替图 7.6 (b) 所示决策树中的节点 T_{13} , 同样, 用 T_{22} 的实例分类结果“+”作为叶节点代替 T_{22} , 用 T_{23} 的实例分类结果“-”作为叶节点代替 T_{23} 。

T_{12} 是一个空集, 故从决策树中删去节点 T_{12} 。

由于 T_{11} 和 T_{21} 中的实例的分类结果仍有两种, 故还需按新的属性表递归构造 T_{11} 和 T_{21} 的子树。

以新属性表中的属性 eyes 作为第 3 个检测属性, 且 eyes 的值域 Value-Type (eyes)={blue, brown} 有 2 个取值, 故将 T_{11} 分为 2 个子集 T_{111} 和 T_{112} , 将 T_{21} 也分为 2 个子集 T_{211} 和 T_{212} , 它们分别为

$T_{111}=\{ \langle (\text{short, blond, blue}), + \rangle \}$

$T_{112}=\{ \langle (\text{short, blond, brown}), - \rangle \}$

$T_{211}=\{ \langle (\text{tall, blond, blue}), + \rangle \}$

$T_{212}=\{ \langle (\text{tall, blond, brown}), - \rangle \}$

至此, 生成的决策树如图 7.6 (c) 所示。

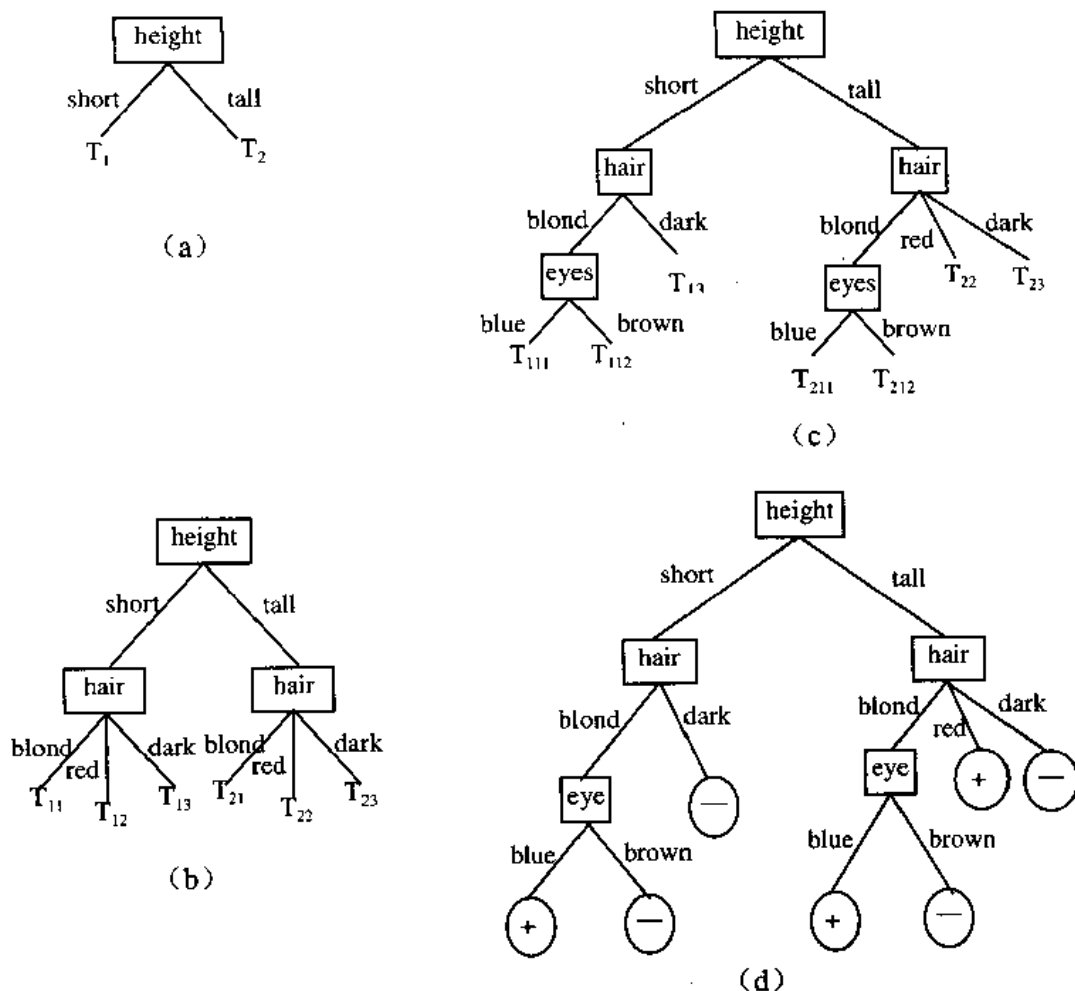


图 7.6 决策树构造算法构造决策树的过程示例

2. ID3 算法

CLS 决策树构造算法每次从属性表 AttrList 中任选一个属性 A_i 作为检测属性来扩展生成决策树，按照训练实例集 T 中所有实例的属性 A_i 的取值来划分实例子集，因此，CLS 算法的开销较大，效率较低。Quinlan 提出了 ID3 算法，对检测属性的选择给出一种启发式规则，这个规则选择平均信息量（熵）最小的属性作为检测属性 A_i ，因此，又称为最小熵原理。

若选择属性表 $\text{AttrList}=\{A_1, A_2, \dots, A_i, \dots, A_n\}$ 中的属性 A_i 为检测属性，则由 A_i 的值域 $\text{ValueType}(A_i)=\{V_1, \dots, V_s\}$ 的 s 个取值把训练实例集 T 分为 s 个子集：

$$T = \bigcup_{k=1}^s T_k^{(i)}$$

子集 $T_k^{(i)}$ 中的所有实例的属性 A_i 的取值为 V_k 。

T 中的实例的分类结果组成了分类结果表 $\text{Class}=\{C_1, C_2, \dots, C_j, \dots, C_m\}$ ，若分类结果为 C_j 的

实例数为 e_j , $1 \leq j \leq m$, 且 $\sum_{j=1}^m e_j = |T|$, $|T|$ 表示训练实例集 T 中的实例总数, 则实例分类结果为 C_j 的概率为 $P_j = e_j / |T|$ 。

定义训练实例集 T 的实例平均信息量为:

$$I(T) = -\sum_{j=1}^m P_j \log_2 P_j = -\sum_{j=1}^m \frac{e_j}{|T|} \log_2 \frac{e_j}{|T|} = -(\sum_{j=1}^m e_j \log_2 \frac{e_j}{|T|}) / |T|$$

类似可定义实例子集 $T_k^{(i)}$ 的实例平均信息量为:

$$I(T_k^{(i)}) = -(\sum_{j=1}^m e_j^{(i)} \log_2 \frac{e_j^{(i)}}{|T_k^{(i)}|}) / |T_k^{(i)}|$$

其中, $e_j^{(i)}$ 为子集 $T_k^{(i)}$ 中分类结果为 C_j 的实例数, $|T_k^{(i)}|$ 为子集 $T_k^{(i)}$ 中的实例总数, $\sum_{j=1}^m e_j^{(i)} = |T_k^{(i)}|$ 。

若选择属性 A_i 作为检测属性将训练实例集 T 分为 s 个子集后, 可以由各实例子集的实例总信息量 $|T_k^{(i)}| \cdot I(T_k^{(i)})$ 之和对实例集 T 的实例总数 $|T|$ 的平均值来表示实例集 T 的实例平均信息量, 即:

$$I(T, A_i) = (\sum_{k=1}^s |T_k^{(i)}| \cdot I(T_k^{(i)})) / |T|$$

选择属性作为检测属性 A_i 来将实例集 T 划分子集的启发式规则为: 选择属性作为检测属性, 应由属性 A_i 的不同取值把实例集划分为若干子集之前与之后的实例平均信息量的差最大, 即使得:

$$GI(T, A_i) = I(T) - I(T, A_i)$$

的值最大。 $GI(T, A_i)$ 可认为是属性 A_i 对训练实例集 T 的信息量, 因此, 启发式规则实际上是选择信息量最大的属性作为检测属性 A_i 来划分实例集。

ID3 算法在 CLS 算法的基础上引入了启发函数 $GI(T, A_i)$ 来生成决策树。

例 7.4 应用 ID3 算法构造例 7.3 要求构造的决策树, 说明构造过程。

解 应用 ID3 算法构造决策树的构造过程如下:

1) 在实例集 T 中, 分类结果为 “+” 的实例数为 3, 分类结果为 “-” 的实例数为 5, 故 T 的实例平均信息量为:

$$I(T) = -(5 \log_2 5/8 + 3 \log_2 3/8) / 8 = 0.954 \text{ (bit)}$$

若选属性表 $AttrList = \{\text{height}, \text{hair}, \text{eyes}\}$ 中的第 1 个属性 height 为检测属性, 其值域 $ValueType(\text{height}) = \{\text{short}, \text{tall}\}$ 有 2 个取值, 则将 T 分为 2 个子集 $T_1^{(1)}$ 和 $T_2^{(1)}$:

$$T_1^{(1)} = \{ \langle (\text{short}, \text{blond}, \text{blue}), + \rangle, \langle (\text{short}, \text{dark}, \text{blue}), - \rangle, \langle (\text{short}, \text{blond}, \text{brown}), - \rangle \}$$

$$T_2^{(1)} = \{ \langle (\text{tall}, \text{blond}, \text{brown}), - \rangle, \langle (\text{tall}, \text{red}, \text{blue}), + \rangle, \langle (\text{tall}, \text{dark}, \text{blue}), - \rangle, \langle (\text{tall}, \text{blond}, \text{blue}), + \rangle, \langle (\text{tall}, \text{dark}, \text{brown}), - \rangle \}$$

$T_1^{(1)}$ 中分类结果为“+”的实例数为 1, 分类结果为“-”的实例数为 2, 可得 $T_1^{(1)}$ 的实例平均信息量为:

$$I(T_1^{(1)}) = -(\log_2 1/3 + 2\log_2 2/3)/3 = 0.918 \quad (\text{bit})$$

$T_2^{(1)}$ 中分类结果为“+”的实例数为 2, 分类结果为“-”的实例数为 3, 可得 $T_2^{(1)}$ 的实例平均信息量为:

$$I(T_2^{(1)}) = -(2\log_2 2/5 + 3\log_2 3/5)/5 = 0.971 \quad (\text{bit})$$

故而, 若以属性 height 为检测属性扩展后的 T 的实例平均信息量为:

$$\begin{aligned} I(T, \text{height}) &= (3 \times I(T_1^{(1)}) + 5 \times I(T_2^{(1)}))/8 \\ &= 3 \times 0.918 + 5 \times 0.971 = 0.951 \quad (\text{bit}) \end{aligned}$$

由此可得属性 height 对实例集 T 的信息量为:

$$\begin{aligned} GI(T, \text{height}) &= I(T) - I(T, \text{height}) \\ &= 0.954 - 0.951 = 0.003 \quad (\text{bit}) \end{aligned}$$

同理可得:

$$GI(T, \text{hair}) = 0.5 \quad (\text{bit})$$

$$GI(T, \text{eyes}) = 0.347 \quad (\text{bit})$$

由于 $GI(T, \text{hair}) > GI(T, \text{eyes}) > GI(T, \text{height})$, 故选取第 2 个属性 hair 作为检测属性, 并从属性表中删去属性 height, 更新属性表为:

$$\text{AttrList} = \{\text{height}, \text{eyes}\}$$

至此, 生成的决策树如图 7.7 (a) 所示, 图中的 3 个子节点 $T_1^{(2)}$ 、 $T_2^{(2)}$ 、 $T_3^{(2)}$ 是由检测属性 hair 的值域 $\text{ValueType}(\text{hair}) = \{\text{blond}, \text{red}, \text{dark}\}$ 的 3 个取值将 T 划分成的 3 个子集, 分别是:

$$\begin{aligned} T_1^{(2)} &= \{ \langle (\text{short}, \text{blond}, \text{blue}), + \rangle, \langle (\text{tall}, \text{blond}, \text{brown}), - \rangle, \\ &\quad \langle (\text{tall}, \text{blond}, \text{blue}), - \rangle, \langle (\text{short}, \text{blond}, \text{brown}), - \rangle \} \end{aligned}$$

$$T_2^{(2)} = \{ \langle (\text{tall}, \text{red}, \text{blue}), + \rangle \}$$

$$\begin{aligned} T_3^{(2)} &= \{ \langle (\text{short}, \text{dark}, \text{blue}), - \rangle, \langle (\text{tall}, \text{dark}, \text{blue}), - \rangle, \\ &\quad \langle (\text{tall}, \text{dark}, \text{brown}), - \rangle \} \end{aligned}$$

2) 由于 $T_2^{(2)}$ 中的实例的分类结果只有“+”, 故用叶节点“+”代替节点 $T_2^{(2)}$; $T_3^{(2)}$ 中的实例的分类结果都是“-”, 故用叶节点“-”代替节点 $T_3^{(2)}$ 。由于 $T_1^{(2)}$ 中的实例分类结果仍有两种, 故还需按新属性表启发式选择检测属性来构造 $T_1^{(2)}$ 子树。

在实例集 $T_1^{(2)}$ 中, 分类结果为“+”的实例数为 2, 分类结果为“-”的实例数也为 2, 可得 $T_1^{(2)}$ 的实例平均信息量为:

$$I(T_1^{(2)}) = -(2\log_2 2/4 + 2\log_2 2/4)/4 = 1 \quad (\text{bit})$$

若选新属性表 $\text{AttrList} = \{\text{height}, \text{eyes}\}$ 中的第 1 个属性 height 为检测属性, 则由其值域 $\text{ValueType}(\text{height}) = \{\text{short}, \text{tall}\}$ 的 2 个取值将 $T_1^{(2)}$ 分为 2 个子集 $T_{11}^{(21)}$ 和 $T_{12}^{(21)}$:

$$T_{11}^{(21)} = \{ \langle (\text{short}, \text{blond}, \text{blue}), + \rangle, \langle (\text{short}, \text{blond}, \text{brown}), - \rangle \}$$

$$T_{12}^{(21)} = \{ \langle (\text{tall}, \text{blond}, \text{brown}), - \rangle, \langle (\text{tall}, \text{blond}, \text{blue}), + \rangle \}$$

$T_{11}^{(2)}$ 和 $T_{12}^{(2)}$ 中的分类结果为“+”的实例数都为 1, 分类结果为“-”的实例数也都为 1, 可得 $T_{11}^{(2)}$ 和 $T_{12}^{(2)}$ 的实例平均信息量分别为:

$$I(T_{11}^{(2)}) = -(\log_2 1/2 + \log_2 1/2)/2 = 1 \quad (\text{bit})$$

$$I(T_{12}^{(2)}) = -(\log_2 1/2 + \log_2 1/2)/2 = 1 \quad (\text{bit})$$

故而, 若以属性 height 为检测属性扩展后的 $T_1^{(2)}$ 的实例平均信息量为:

$$I(T_1^{(2)}, \text{height}) = (2 \times I(T_{11}^{(2)}) + 2 \times I(T_{12}^{(2)}))/4 = 1 \quad (\text{bit})$$

由此可得属性 height 对实例集 $T_1^{(2)}$ 的信息量为:

$$GI(T_1^{(2)}, \text{height}) = I(T_1^{(2)}) - I(T_1^{(2)}, \text{height}) = 0$$

若选新属性表 AttrList={height, eyes} 中的第 2 个属性 eyes 为检测属性, 由其值域 ValueType(eyes)={blue, brown} 的 2 个取值将 $T_1^{(2)}$ 分为 2 个子集 $T_{11}^{(22)}$ 和 $T_{12}^{(22)}$:

$$T_{11}^{(22)} = \{ \langle (\text{short}, \text{blond}, \text{blue}), + \rangle, \langle (\text{tall}, \text{blond}, \text{blue}), + \rangle \}$$

$$T_{12}^{(22)} = \{ \langle (\text{tall}, \text{blond}, \text{brown}), - \rangle, \langle (\text{short}, \text{blond}, \text{brown}), - \rangle \}$$

由于属性 height 对实例集 $T_1^{(2)}$ 的信息量 $GI(T_1^{(2)}, \text{height})=0$, 因此选择属性 eyes 作为检测属性, 由 eyes 的 2 个取值扩展生成 2 个子集节点 $T_{11}^{(22)}$ 和 $T_{12}^{(22)}$ 。至此, 生成的决策树如图 7.7(b) 所示。

3) 由于 $T_{11}^{(22)}$ 中的实例的分类结果都是“+”, 故用叶节点“+”代替节点 $T_{11}^{(22)}$; $T_{12}^{(22)}$ 中的实例的分类结果都是“-”, 故用叶节点“-”代替节点 $T_{12}^{(22)}$ 。至此, 得到最后的完整的决策树如图 7.7(c) 所示。

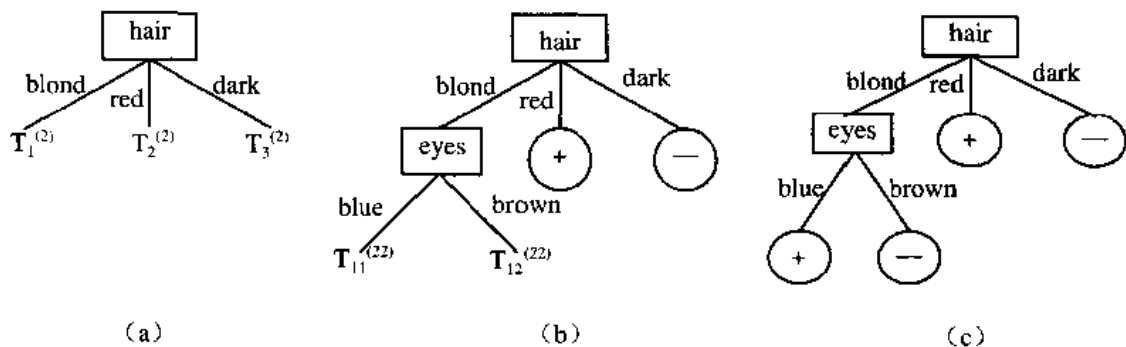


图 7.7 ID3 算法构造决策树的过程示例

比较图 7.6 (d) 和图 7.7 (c) 可见, 由于 ID3 算法使用了训练实例集含有的启发信息来选择检测属性, 因此, 生成的决策树的节点数比 CLS 算法生成的决策树的节点数要少得多。也就是说, 启发式学习算法 ID3 算法的学习效率高于 CLS 算法。

7.3 基于解释的学习

基于解释的学习 (EBL, Explanation-Based Learning) 是一种基于实例分析的学习方法, 通过分析为什么实例是某个目标概念的一个具体例子, 并对分析过程加以推广, 剔除与具体例子有关的成分, 从而产生目标概念的一个描述。也就是说, 通过对一个实例的学习, 使抽

象的目标概念被具体化, 变得更易理解与操作, 从而为类似问题的求解提供有效的经验。

人类的学习行为中也常常使用分析学习方法, 从分析单个实例的求解过程来得到同类问题求解的一般概念或规则。例如, 在学习一元二次方程 $ax^2+bx+c=0$ 的求解方法时, 通过分析一个实例 $x^2+8x+12=0$ 的求解过程, 可以得到一元二次方程的一般求解方法。我们把单个实例的求解过程称为可用于同类问题求解的一般概念的一个解释。

7.3.1 基于解释的学习框架

基于解释的学习的一般框架可以用一个 4 元组表示为:

$$EBL = (DT, TC, E, C)$$

其中, DT 称为领域理论 (Domain Theory), 它包含一组事实和规则, 用于证明训练实例如何满足目标概念; TC 为目标概念 (Target Concept), 即待学概念的一个非操作性描述; E 为训练实例, 即目标概念的一个例子; C 称为操作性准则 (Operationality Criterion), 它是定义在概念描述上的一个二阶谓词, 用以表示学习得到的目标概念可用哪些基本的、可操作的概念表示。

EBL 的目标是找出训练实例的一个一般描述, 该描述中的谓词均是可操作的, 且构成目标概念成立的充分条件。

例 7.5 给出关于“某人是自杀而死”的判断规则的基于解释学习的框架描述。

解 若已知法律知识的若干规则和有关 John 的某些事实 (训练实例), 希望通过基于解释的学习来产生“某人是自杀而死”的判断规则, 为此, 首先需要给出学习的下述框架描述。

领域理论 (DT):

可以由法律领域的有关知识给出下述一组规则。

1) 如果一个人感到沮丧, 他就会恨自己。

$R_1: \text{Depressed}(x) \rightarrow \text{Hate}(x, x)$

2) 如果某人买过某物, 那么他就拥有该物。

$R_2: \text{Buy}(x, y) \rightarrow \text{Posses}(x, y)$

3) 猎枪是武器。

$R_3: \text{Shotgun}(x) \rightarrow \text{Weapon}(x)$

4) 手枪是武器。

$R_4: \text{Pistol}(x) \rightarrow \text{Weapon}(x)$

操作规则 (C):

若领域理论 DT 给出的上述规则中的谓词 $\text{Depressed}(x)$ 、 $\text{Buy}(x, y)$ 、 $\text{Shotgun}(x)$ 、 $\text{Pistol}(x)$ 以及谓词 $\text{Kill}(x, y)$ 是可操作谓词, 即是由领域理论给出明确定义的谓词, 那么把可操作谓词分别表示为下述二阶谓词:

$\text{Operational}(\text{Depresses}(x))$

$\text{Operational}(\text{Buy}(x, y))$

Operational (Shotgun (x))

Operational (Pistol (x))

Operational (Kill (x, y))

目标概念 (TC) :

目标概念是待学概念的一个非操作性描述, 可以用规则的形式表示。所谓“非操作性”是指目标概念规则中有的谓词不是可操作谓词。我们可以给出下述与“自杀”可能有关的一个目标概念规则: 如果 x 恨 y , 并且 x 拥有一件武器, 那么 x 可能杀死 y 。

$\text{Hate}(x, y) \text{ and } \text{Weapon}(x) \text{ and } \text{Posses}(x, z) \rightarrow \text{Kill}(x, y)$

在给出的目标概念规则中, 谓词 Hate 、 weapon 和 Posses 都不是操作准则 C 中定义的可操作谓词, 因此, 这个目标规则有待通过学习使之具有可操作性。

训练实例 (E) :

若已知 John 自杀事件的下述由可操作谓词定义的事实, 则组成学习的一个训练实例。

$\text{Depresses}(\text{John})$

$\text{Shotgun}(\text{Obj})$

$\text{Buy}(\text{John}, \text{Obj})$

$\text{Kill}(\text{John}, \text{John})$

给出上述 EBL 的框架 $\text{EBL} = (\text{DT}, \text{TC}, \text{E}, \text{C})$ 后, 就可以通过学习找出自杀事件的一个一般描述或规则, 这个规则中的谓词都应是可操作谓词。

7.3.2 基于解释的学习过程

在学习的框架表示基础上, 基于解释的学习过程可分下述两个阶段。

(1) 分析阶段

分析阶段对训练实例提供的事实, 使用领域理论的有关规则和目标概念规则生成一棵树, 树的根节点是待学概念的一个事实, 如果能得到树的所有叶节点都是已知的实例事实, 那么称这棵树为这个训练实例的证明树, 它解释了这个训练实例为什么是目标概念的一个实例, 或称其为对目标概念的一个解释。下面, 我们给出证明树生成算法。

证明树生成算法

1) 以目标概念事实作为根节点。

2) 寻找一个匹配规则对当前节点扩展。

从目标概念规则和领域理论规则中寻找一个匹配规则对当前节点扩展。若规则的结论部分的谓词与当前节点中的谓词匹配, 则该规则是当前节点的匹配规则。匹配时可对规则中的谓词变量用当前节点中相应的谓词常量来代换。若有多条规则可匹配, 则需采用消解策略从中选择一条匹配规则。若找不到一条可匹配规则, 则算法失败终止。

由匹配规则的条件部分的与/或关系的若干谓词对当前节点扩展生成其与/或关系的若干子节点。

3) 如果所有的叶节点都已经是已知实例事实, 则成功地获得这个实例的证明树, 算法成功终止; 否则转步骤2)。

(2) 基于解释的泛化阶段

基于解释的泛化 (EBG, Explanation Based Generalization) 是将实例证明树中的谓词常量用相应的谓词变量进行替换, 这一过程也称为解释的泛化, 从而得到一棵基于解释的泛化树 (EBG 树)。由泛化树的根节点和所有叶节点可直接得出目标概念规则, 根节点的目标概念是这个规则的结论部分, 所有叶节点组成这个目标概念的充分条件。由于实例证明树的所有叶节点中的谓词都是可操作谓词, 因此, 由相应泛化树的所有叶节点组成的充分条件的所有谓词也都是可操作谓词, 也就是说, 经过实例学习后得到目标概念是一个可操作规则。

例 7.6 对例 7.5 给出的框架描述, 应用证明树生成算法生成有关“自杀”概念的证明树, 并应用解释的泛化得出相应的泛化树, 最后由泛化树得出“自杀”概念的一般规则。

解 根据例 7.5 中给出的学习框架表示, 自杀概念可表示为 $Kill(x, x)$ 。希望通过基于解释的学习, 得出自杀的一般概念 $Kill(x, x)$ 的充分条件, 并由此组成一个判断自杀的一般规则。

首先由证明树生成算法生成自杀实例的证明树。

从学习框架的训练实例 E 中, 选择自杀概念事实 $Kill(John, John)$ 作为根节点。在学习框架的所有规则中, 只有目标概念规则是根节点的匹配规则, 匹配时, 对这个规则中的谓词变量用相应的常量进行了代换, 即对规则执行了代换 $\sigma = \{John/x, John/y, Obj/z\}$ 。从而生成“与”关系的 3 个子节点: $Hate(John, John)$ 、 $Weapon(Obj)$ 和 $Posses(John, Obj)$ 。

上述 3 个子节点都不是已知的实例事实, 因此都需要为其寻找匹配规则, 继续扩展。

节点 $Hate(John, John)$ 的匹配规则是 R_1 , 匹配时, 对 R_1 执行了代换 $\sigma_1 = \{John/x\}$, 从而扩展生成的子节点为 $Depressed(John)$, 这个子节点是一个已知的实例事实, 故不再需要扩展。

节点 $Weapon(Obj)$ 的匹配规则是 R_3 , 匹配时, 对 R_3 执行了代换 $\sigma_3 = \{Obj/x\}$, 从而扩展生成的子节点为 $Shotgun(Obj)$, 这个子节点也是一个已知的实例事实, 也不再需要扩展。

节点 $Posses(John, Obj)$ 的匹配规则是 R_2 , 匹配时, 对 R_2 执行了代换 $\sigma_2 = \{John/x, Obj/y\}$, 从而扩展生成的子节点为 $Buy(John, Obj)$, 这个子节点也是一个已知的实例事实, 也不再需要扩展。

至此, 证明树生成算法成功终止, 生成的自杀实例证明树如图 7.8 所示。

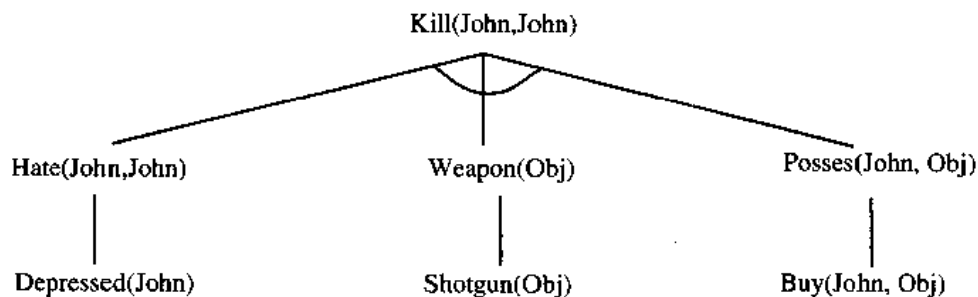


图 7.8 自杀实例的证明树

对图 7.8 所示的证明树进行泛化，泛化过程从叶节点开始，用变量代换谓词常量，直至根节点，即对各节点执行代换 $\sigma = \{x/\text{John}, y/\text{Obj}\}$ ，得到自杀实例的 EBG 树如图 7.9 所示。

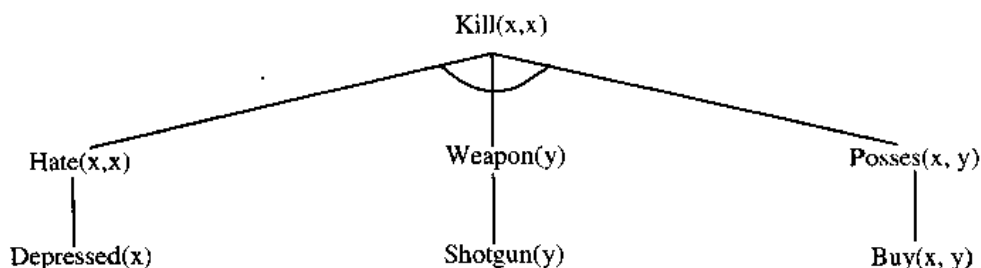


图 7.9 自杀实例的 EBG 树

由于该 EBG 树中每个叶节点对应的概念均是可操作谓词，因此它们可组成自杀概念的一个可操作的充分条件，得出判断自杀的一个可操作规则为：

$$\text{Depressed}(x) \text{ and } \text{Shotgun}(y) \text{ and } \text{Buy}(x, y) \rightarrow \text{Kill}(x, x)$$

7.4 遗传算法

遗传算法 (GA, Genetic Algorithm) 是借鉴生物遗传机制的一种随机化搜索算法，其主要特点是群体搜索和群体中的个体之间的信息交换。遗传算法尤其适用于处理传统方法难以解决的复杂的和非线性的问题，可广泛用于机器学习、组合优化和自适应控制等领域。遗传算法的概念和思想由美国 J.Holland 教授提出后，已在许多领域有广泛的应用，它将成为智能计算的主要技术之一。

7.4.1 遗传算法的概念与计算方法

遗传算法是遗传学和计算机科学相互结合渗透而成的新的计算方法，它使用了遗传学的一些概念和基本术语来描述它的计算方法，因此，了解这些术语对于学习和应用遗传算法是十分必要的。

生物遗传物质的主要载体是染色体，最主要的遗传物质是 DNA。染色体由基因组成，染色体中基因的位置称为基因座，基因所取的值称为等位基因。基因和基因座决定了染色体的特征，也决定了生物个体的性状。

染色体有两种表示模式，即基因型 (genotype) 表示模式和表现型 (phenotype) 表示模式。表现型是指生物个体所表现出来的性状。基因型是指与生物个体表现出来的性状密切相关的基因组成。同一种基因型的生物个体在不同的环境条件下表现出来的性状会有差异，因此，表现型是基因型与环境条件相互作用的结果。

在遗传算法中，与染色体相对应的是数据或数组。在标准的遗传算法中，染色体可用一维的串结构数据来表示，串的各个位置对应染色体的基因座，各位置上所取的值对应等位基

因。遗传算法对染色体进行处理,或者称为对基因型个体(individuals)进行处理。一定数量的个体组成群体(population),群体中的个体数称为群体大小(population size)或群体规模。各个个体对环境的适应程度称为适应度(fitness)。

执行遗传算法时有两个必需的数据转换操作,一个操作是把搜索空间中的参数或解数据转换成遗传空间的染色体或个体,或者说是把染色体数据从表现型表示转换成基因型表示,这个过程称为编码操作;另一个操作称为译码操作,它是同编码操作相反的数据转换操作,译码操作把染色体数据从基因型表示转换成表现型表示。

遗传算法的基本流程如图 7.10 所示。遗传算法以群体中的所有个体为对象,选择(selection)、杂交(crossbreed)和变异(mutation)是遗传算法的 3 个主要操作算子,它们构成了所谓的遗传操作(Genetic Operations),使遗传算法具有了其他传统方法所没有的特征。

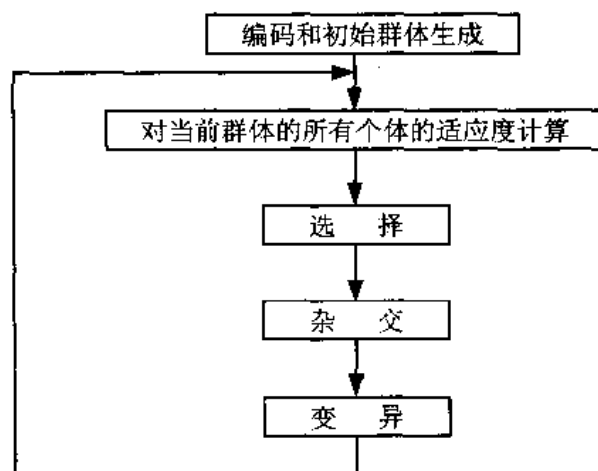


图 7.10 遗传算法的基本流程

应用遗传算法求解问题,需要根据求解的具体问题进行下述 5 个方面的工作:

- 1) 参数编码的格式设定及参数编码。
- 2) 初始群体的设定。
- 3) 适应度函数的设计。
- 4) 遗传操作设计。

5) 控制参数设计,主要是指群体规模和遗传操作中所需使用的有关控制参数的设定和设计。

上述 5 个方面的工作是遗传算法的核心内容,也称为遗传算法的 5 个基本要素。我们以一个简单的函数极值求解为例,来说明遗传算法的计算方法和处理过程。

例 7.7 应用遗传算法求 $f(x)=x^2$ 的最大值的解, $x \in [0, 31]$ 。

解 我们按遗传算法的 5 个基本要素来说明遗传算法对此例的计算方法和处理过程。

(1) 编码

此问题给出的数据 0~31 是表现型表示模式,需要通过编码把它们转换成遗传空间的基

因型表示。基因型表示模式通常是串结构形式，十进制数可以很方便地转换成二进制数串，因此，我们把 0~31 编码为 5 位长的二进制无符号整数表示形式。例如， $x=8$ 的基因型表示为 01000， $x=13$ 的基因型表示为 01101 等。

(2) 初始群体的生成

遗传算法是对一个群体中的所有个体为对象进行遗传操作的，为了简化此例的说明，假定取群体规模 $n=4$ ，即群体由 4 个个体组成。初始群体的每个个体都是从遗传空间中随机选择的，假设随机选择的 4 个个体分别是 $x_1=13=01101$ ， $x_2=24=11000$ ， $x_3=8=01000$ ， $x_4=19=10011$ 。初始群体也称为进化的初始代或第一代 (first generation) 群体。

(3) 适应度函数设计和适应度计算

遗传算法在搜索求解过程中一般不需要其他外部信息，仅用设计的适应度函数来评估个体的优劣，作为对个体进行遗传操作的依据。某个个体 x_i 的适应度函数值 f_i 称为这个个体 x_i 的适应度。此例需求 $f(x)=x^2$ 的最大值的解，我们就可以设定 $f(x)=x^2$ 为个体 x 的适应度函数，由此适应度函数计算各个体的适应度，按适应度的大小来评估个体的优劣，适应度越大的个体越优良，选优汰劣，遗传进化的结果将得到 $f(x)$ 值最大的解 x 。

显然，利用 $f(x)=x^2$ 来计算个体 x_i 的适应度 f_i 时，要把个体 x_i 的基因型表示译码成为表现型表示，即将 x_i 的二进制表示转换成十进制表示，然后计算个体 x_i 的适应度 x_i^2 。

(4) 选择操作

选择操作是从当前群体中选出优良的个体，使它们有机会作为父代来繁殖下一代。判断个体优良的依据就是个体的适应度，个体的适应度越高，该个体被选择的机会就越多。

选择操作的实现方法很多，最简单易行的操作方法就是直接按适应度 f_i 选择个体，也可以把选择操作算子设计成适应度 f_i 的某种函数 $s(f_i)$ ，由 $s(f_i)$ 的值 s_i 来选择个体。在此例中，我们介绍一种常用的选择操作算子，即适应度概率选择算子。采用适应度概率选择算子来实现选择操作的步骤为：

1) 若个体 x_i 的适应度为 f_i ， $i=1, 2, \dots, n$ ，由适应度计算各个体的选择概率 $p_i = f_i / \sum_{i=1}^n f_i$ ， $i=1, 2, \dots, n$ ，且 $\sum_{i=1}^n p_i = 1$ 。

2) 根据群体 n 个个体的选择概率的大小，对个体选优汰劣。本例采用的选优汰劣的方法是：按个体选择概率 p_i 的值从大到小将 n 个个体排队，对排队序列 $LI=(x_1, x_2, \dots, x_n)$ 的队首个体 x_1 选择 2 次，对队尾个体 x_n 不选择，其他个体选择 1 次，被选择的个体经编码成为基因型表示后送入配对池 (Mating Pool) 中。

如表 7.1 所示，第一代群体的 4 个个体分别是 $x_1=24$ 、 $x_2=19$ 、 $x_3=13$ 和 $x_4=8$ ，个体适应度 $f_i = x_i^2$ 分别是 $f_1=576$ 、 $f_2=361$ 、 $f_3=169$ 和 $f_4=64$ ，选择概率 $p_i = f_i / \sum_{i=1}^n f_i$ 分别是 $p_1=0.49$ 、 $p_2=0.31$ 、 $p_3=0.14$ 和 $p_4=0.06$ 。4 个个体的排队序列是 $LI=(24, 19, 13, 8)$ 。对 x_1 选择 2 次，对 x_2 和 x_3 各选择 1 次，淘汰 x_4 ，送入配对池，则配对池 MP 中的表现型个体为 $MP=\{24, 24, 19, 13\}$ ，实际上，配对池中的个体是经编码后的基因型个体，所以，配对池为 $MP=\{11000,$

11000, 10011, 01101}。

(5) 杂交操作

杂交操作是对配对池的当前群体中的基因型个体随机配对, 然后使配成一对的个体彼此交换部分信息, 从而得到这一对个体的两个后代个体。

杂交操作可分为下述两个步骤来实现:

①随机配对。对配对池中的个体 x_i 的编号 i 随机指定一个配对的个体编号 j , 即把配对池中的个体 x_i 与 x_j 配成一对 ($1 \leq i, j \leq n, i \neq j$)。

②杂交繁殖。个体采用基因型表示时, 个体通常具有由 m 个基因座组成的一维串结构。例如, 本例中的个体的基因型表示是 5 位二进制数串, 即有 $m=5$ 。杂交繁殖是指: 对任何一对配对个体随机指定一个交叉位置 k , $1 \leq k \leq m$, 两个个体从基因座 1 至基因座 k 上的基因值对应彼此交换 (等位基因互换), 从而产生两个新的后代个体。基因座的编号是从串的低位到高位依序上升, 即串的最低位为基因座 1, 串的最高位为基因座 m 。

本例第一代群体经选择操作后, 配对池为 $MP=\{11000, 11000, 10011, 01101\}$ 。若随机配对时, 对个体编号 $i=1$ 随机指定配对个体编号 $j=4$, 对个体编号 $i=2$ 随机指定配对个体编号 $j=3$, 则把 MP 中的 $x_1=11000$ 与 $x_4=01101$ 配对, 把 $x_2=11000$ 与 $x_3=10011$ 配对。

对上述两对分别进行杂交繁殖。若对 (x_1, x_4) 随机指定交叉位置 $k=3$, 则把 x_1 和 x_4 的低 3 位的值互换, 得到两个后代个体分别为 11101 和 01000, 这两个后代个体的表现型分别为 29 和 8。若对 (x_2, x_3) 随机指定交叉位置 $k=1$, 则把 x_2 和 x_3 的低 1 位的值互换, 得到两个后代个体分别为 11001 和 10010, 这两个后代个体的表现型分别为 25 和 18。从而在配对池中得到第二代群体 $MP=\{29, 25, 18, 8\}$ 。

(6) 变异操作

变异操作是指把一个个体的基因座的基因值进行改变, 从而得到一个新的个体。对于二进制编码的个体来说, 若某位 (bit) 是“0”, 则通过变异操作后就变为“1”; 若某位是“1”, 变异操作将其改变为“0”。变异操作也是随机选择基因座进行的。一般来说, 变异概率都取得很小, 在本例中, 若取变异概率为 0.01, 那么, 需要对群体中 $20 \times 0.01=0.2$ 位进行变异, 这意味着群体中没有一位可以变异。变异操作是十分微妙的遗传操作, 它需要和杂交操作妥善地配合使用, 目的在于挖掘群体中个体的多样性, 克服有可能陷入局部最优解的弊端。

本例通过 5 次迭代处理, 由初始群体繁殖生成第六代群体, 迭代处理过程如表 7.1 所示。由此得到问题的解为 $x=31$ 。

表 7.1 遗传算法求 $f(x)=x^2$ 最大值的迭代处理过程

排队 序列 x_i	适应度 f_i	选择 概率 p_i	选择 次数	配对池 (表现型)	配对池 (基因型)	配对 个体 x_j	交叉 位置 k	后代个体 (基因型)	后代个体 (表现型)
24	576	0.49	2	24	11000	x_4	3	11101	29
19	361	0.31	1	24	11000	x_3	1	11001	25

续表

排队 序列 x_i	适应度 f_i	选择 概率 p_i	选择 次数	配对池 (表现型)	配对池 (基因型)	配对 个体 x_i	交叉 位置 k	后代个体 (基因型)	后代个体 (表现型)
13	169	0.14	1	19	10011	x_2	1	10010	18
8	64	0.06	0	13	01101	x_1	3	01000	8
29	841	0.453	2	29	11101	x_3	1	11101	29
25	625	0.337	1	29	11101	x_4	2	11110	30
18	324	0.175	1	25	11001	x_1	1	11001	25
8	64	0.035	0	18	10010	x_2	2	10001	17
30	900	0.339	2	30	11110	x_4	1	11111	31
29	841	0.317	1	30	11110	x_3	4	11101	29
25	625	0.235	1	29	11101	x_2	4	11110	30
17	289	0.109	0	25	11001	x_1	1	11000	24
31	961	0.293	2	31	11111	x_2	2	11111	31
30	900	0.275	1	31	11111	x_1	2	11111	31
29	841	0.256	1	30	11110	x_4	1	11111	31
24	576	0.176	0	29	11101	x_3	1	11100	28
31	961	0.262	2	31	11111	x_2	1	11111	31
31	961	0.262	1	31	11111	x_1	1	11111	31
31	961	0.262	1	31	11111	x_4	2	11111	31
28	784	0.214	0	31	11111	x_3	2	11111	31

在表 7.1 中, 对第一代群体 LI(1)=(24, 19, 13, 8) 经过选择、杂交和变异后, 得到第二代群体 LI(2)=(29, 25, 18, 8)。对 LI(2), 随机指定的配对为 $(x_1, x_3) = (19, 18)$ 和 $(x_2, x_4) = (25, 8)$, 对 (x_1, x_3) 随机指定的交叉位置 $k=1$, 对 (x_2, x_4) 随机指定的交叉位置 $k=2$, 生成第三代群体 LI(3)=(30, 29, 25, 17)。对 LI(3), 随机指定的配对为 $(x_1, x_4) = (30, 17)$ 和 $(x_2, x_3) = (29, 25)$, 对 (x_1, x_4) 随机指定的交叉位置 $k=1$, 对 (x_2, x_3) 随机指定的交叉位置 $k=4$, 生成第四代群体 LI(4)=(31, 30, 29, 24)。对 LI(4), 随机指定的配对为 $(x_1, x_2) = (31, 30)$ 和 $(x_3, x_4) = (29, 24)$, 对 (x_1, x_2) 随机指定的交叉位置 $k=2$, 对 (x_3, x_4) 随机指定的交叉位置 $k=1$, 生成第五代群体 LI(5)=(31, 31, 31, 28)。对 LI(5), 随机指定的配对为 $(x_1, x_2) = (31, 31)$ 和 $(x_3, x_4) = (31, 28)$, 对 (x_1, x_2) 随机指定的交叉位置 $k=1$, 对 (x_3, x_4) 随机指定的交叉位置 $k=2$, 生成第六代群体 LI(6)=(31, 31, 31, 31)。第六代群体中的所有个体之间没有差异, 因此, 迭代处理过程结束, 得到问题的解为 $x=31$ 。

7.4.2 遗传算法的应用

预测预报是人们对客观事物发展变化的一种认识与估计。预测预报对人类社会的重要性早已为人们所认识，“凡事预则立，不预则废”，说明了人们对预测的重要性的认识。预测可采用的方法很多，例如，传统的回归分析方法、基于灰色理论的灰色模型（Grey Model），以及近几年发展起来的遗传算法和人工神经网络模型等。对某个对象系统选择哪一种方法来建立预测预报模型，关键在于用此方法建立的预测模型的计算值与实际值的拟合程度。我们用一个实例来说明遗传算法在建立预测预报模型中的应用。

例 7.8 已知某港口从 1978 年~1999 年各年的货物年吞吐量（万吨）如表 7.2 所示，应用遗传算法建立该港口货物年吞吐量预测模型。

表 7.2 某港口 1978 年至 1999 年货物年吞吐量（吞吐量单位：万吨）

年度	1978	1979	1980	1981	1982	1983	1984	1985	1986	1987	1988
年吞吐量	720	542	189	714	846	1106	1616	3097	2586	2381	2235
年度	1989	1990	1991	1992	1993	1994	1995	1996	1997	1998	1999
年吞吐量	1819	1985	1854	2011	2636	3185	2816	3187	3926	4427	4550

解 应用遗传算法建立预测模型可分为以下两个步骤。

（1）建模分析

由表 7.2 给出的 1978 年~1999 年的货物年吞吐量，可作出如图 7.11 所示的实际年吞吐量曲线。由此曲线可见，港口年吞吐量的总趋势在增长，但有准周期性的振荡，因此，年吞吐量预测模型应考虑曲线总体增长趋势的描述和振荡规律的描述两部分组成。

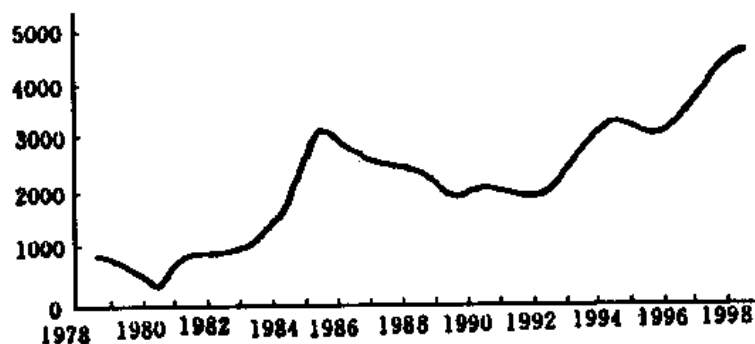


图 7.11 某港口货物实际年吞吐量曲线

1) 港口年吞吐量总体增长趋势的描述。

在一个港口的成长发展时期中，港口货物年吞吐量的变化情况可分为发生、发展、成熟三个阶段。在发生阶段，随着港口基础设施的逐步建设，货物年吞吐量增长速度较慢，并逐渐加大增长趋势；在发展阶段，随着港口建设的发展和港口经济腹地的扩展，货物年吞吐量

增长速度较快，并维持一段时间后进入成熟期；在成熟阶段，其增长速度将逐渐变慢，增长趋于饱和。港口货物的年吞吐量的这种总体发展变化规律可采用 Logistic 生长曲线描述。Logistic 生长曲线是下述非线性常微分方程：

$$\frac{dy}{dt} = a(b - y)$$

的解：

$$y = \frac{b}{1 + me^{-at}}$$

式中各参数在本例中的含义是：y 是年吞吐量，t 是时间（年），a 是年吞吐量增长速度系数，b 是港口年吞吐量的极限值，m 和 c 是与 a、b 有关的参数。

Logistic 生长曲线如图 7.12 所示，可用于反映一般处于成长发展时期中的对象系统随时间的生长情况。由于 Logistic 曲线符合港口年吞吐量随港口建设发展的变化规律，所以采用 Logistic 曲线描述港口年吞吐量的总体增长趋势是比较合适的。



图 7.12 Logistic 生成曲线

考虑到 1978 年以前的年吞吐量应对 Logistic 生成曲线的起点定位及其预测值有“抬高”的影响，故对 Logistic 曲线进行如下修改：

$$y = d + \frac{b}{1 + me^{-at}}$$

2) 港口年吞吐量振荡规律的描述。

由图 7.11 所示的实际年吞吐量曲线，可见，从 1978 年~1999 年，实际年吞吐量的变化与图 7.12 所示的 Logistic 曲线有较大差异。实际年吞吐量曲线有较大幅度的振荡，是一个多峰曲线。港口年吞吐量出现振荡多峰的原因是复杂的，但是，我们并不需要去探讨振荡的原因及其各种原因是如何影响预测模型的，只需要在预测模型中给出振荡规律的大致描述。

由图 7.11 所示的实际年吞吐量曲线的振荡具有准周期性衰减，周期性可以用正弦函数描述，幅值的衰减性可以用负指数函数描述。因此，考虑到年吞吐量曲线的振荡特征后，需要对由 Logistic 曲线表示的年吞吐量总体增长趋势乘以一个振荡修正因子，即有：

$$y = (d + \frac{b}{1 + me^{-ct}}) \times (1 - pe^{-gt} \sin[\pi(\frac{t}{h} + u)])$$

式中, p 为相对振荡幅值, h 为振荡半周期长度(年), u 为振荡的初始幅角, g 为振荡衰减系数。

上式中含有 8 个待定参数 b 、 c 、 d 、 m 、 p 、 g 、 h 和 u 。我们采用遗传算法求解这 8 个参数, 从而得出港口年吞吐量的预测模型 $y(t)$ 。

(2) 采用遗传算法求解模型的待定参数

1) 编码

把待定的 8 个参数表示成一维数组 $x = (b, c, d, m, p, g, h, u)$, 数组各元为带符号的十进制数。

2) 初始群体的随机生成

设群体规模为 n , 随机生成初始群体的 n 个个体 $x_i = (a_{i1}, a_{i2}, a_{i3}, a_{i4}, a_{i5}, a_{i6}, a_{i7}, a_{i8})$, $i=1, 2, \dots, n$ 。个体 x_i 中基因座 k 上的基因值 a_{ik} ($1 \leq k \leq 8$) 就是数组 x_i 的第 k 个元的值。

3) 适应度函数

待定参数的确定应使得预测模型的年吞吐量计算值 $y(t)$ 与年吞吐量实际值 $y^*(t)$ 的拟合最好, 即从 1978 年~1999 年共 22 年的计算值 $y(t)$ 与实际值 $y^*(t)$ 的误差累计最小。故建立适应度函数为:

$$f(b, c, d, m, p, g, h, u) = \sum_{t=1}^{22} |y(t) - y^*(t)|$$

4) 选择操作

为减小计算的复杂程度, 可直接选择适应度 f_i 最小的个体 x_i 作为优良个体, 对其复制一次, 并淘汰适应度最大的一个个体。对规模较大的群体, 优良个体被选择的次数可多于两次, 并相应淘汰多个劣质个体, 以保持群体规模 n 不变。

5) 杂交操作

对配对池 $MP = (x_1, \dots, x_i, \dots, x_j, \dots, x_n)$ 中的 n 个个体随机配对进行杂交繁殖。若随机选择 $x_i = (a_{i1}, a_{i2}, a_{i3}, a_{i4}, a_{i5}, a_{i6}, a_{i7}, a_{i8})$ 与 $x_j = (a_{j1}, a_{j2}, a_{j3}, a_{j4}, a_{j5}, a_{j6}, a_{j7}, a_{j8})$ 配对, 则杂交操作为:

$$\left. \begin{aligned} a'_{ik} &= a_{ik}(1-\beta) + \beta a_{jk} \\ a'_{jk} &= a_{jk}(1-\beta) + \beta a_{ik} \end{aligned} \right\} \quad k=1, 2, \dots, 8$$

其中, 杂交参数 β 可以是一个随机数, β 的取值范围为 $0 < \beta < 1$, a'_{ik} 是个体 x_i 与 x_j 杂交生成的后代个体 x'_i 中第 k 个基因的值, a'_{jk} 是个体 x_j 与 x_i 杂交生成的后代个体 x'_j 中第 k 个基因的值。

6) 变异操作

本例选取的变异操作的概率为 1%, 即每次对群体中的 $8n$ 个基因随机选择 $8n \times 1\%$ 个基因进行变异操作。若随机选择基因 a_{ik} 进行变异操作, 则变异后的基因为:

$$a'_{ik} = (1 - \eta)a_{ik}$$

式中, 变异参数 η 可以是一个随机数, η 的取值范围为 $0 < \eta < 1$ 。

7) 算法终止条件

遗传算法是一种迭代算法, 对生成的后代群体的 n 个个体继续进行选择、杂交和变异操作, 直至满足算法终止条件。理想的算法终止条件是连续若干代 (例如 10 代) 不再产生适应度更小的个体, 即不再能繁殖出更优良的个体。这个终止条件是比较严格的, 需要迭代计算许多次。为了减少算法运算的时空开销, 比较一般的终止条件可为: 若 $f_i \leq \varepsilon$, 则算法终止, 相应的个体 x_i 即是问题的解。 x_i 中的 8 个基因值即分别为待定的 8 个参数值。 ε 为指定的拟合精度误差。

由例 7.8 的讨论可见, 为了建立任何一个对象系统的时序预测模型, 只要能先用一个参数待定的非线性函数来大致描述该对象系统的变化规律, 无论这个非线性函数有多复杂, 都可以用一组实际值作为学习实例, 采用遗传算法来训练模型, 求解出模型的待定参数, 从而建立起这个模型。另外, 适应度函数和算法终止条件能保证建立的模型的计算值与实际值会有很好的拟合精度, 所以基于遗传算法的时序预测模型的建模方法与传统建模方法比较, 前者建立的预测模型会有较高的预测精度, 尤其是近期预测。

习题七

7.1 何谓机器学习? 简述机器学习系统应具有的主要特征。

7.2 机器学习有哪些主要的学习方法? 简述它们之间的区别。

7.3 设实例空间中的每个实体用“性别”和“职称”两个特征描述, 其中, “性别”的属性值有: 男、女, “职称”的属性值有: 高工、工程师、助工。

1) 对这个实例空间, 给出其概念空间偏序集的最大元素和最小元素, 并画出其概念空间的偏序集表示。

2) 说明消除候选者算法的步骤。

3) 应用消除候选者算法, 给出要学习的概念为“工程师”的学习步骤和学习结果。

7.4 有一批长方体形的工件, 需要根据工件的外部尺寸从中筛选可用工件。工件外部尺寸属性表为{长, 宽, 高}。若这批工件的属性值分别为:

$$\text{Value (长)} = \{a_1, a_2, a_3\}$$

$$\text{Value (宽)} = \{b_1, b_2\}^*$$

$$\text{Value (高)} = \{h_1, h_2\}$$

对这批工件分类, 用“+”表示可用, 用“-”表示不可用。若为分类决策提供学习的训练集为:

$$T = \{ \langle (a_1, b_1, h_1), + \rangle, \langle (a_1, b_2, h_2), - \rangle, \\ \langle (a_2, b_2, h_1), + \rangle, \langle (a_3, b_1, h_1), - \rangle, \dots \}$$

$$\begin{aligned} &\langle (a_3, b_2, h_1), - \rangle, \langle (a_1, b_2, h_1), + \rangle, \\ &\langle (a_3, b_2, h_2), - \rangle, \langle (a_1, b_1, h_2), - \rangle \} \end{aligned}$$

请应用 CLS 构造算法构造决策树。说明构造过程，画出构造的决策树。

7.5 应用 ID3 算法求解 7.4 题。

- 1) 说明 ID3 算法引入的启发式函数 $GI(T, A_i)$ 的含义。
- 2) 应用 ID3 算法构造 7.4 题的决策树。说明构造过程，画出构造的决策树。

7.6 设要学习的目标概念是“一个物体 (Obj1) 可以安全地放置在另一个物体 (Obj2) 上面”，即

Safe-To-Stack (Obj1, Obj2)

若由领域知识可获得把一个物体放置在另一个物体上面的下述安全准则：

$$\begin{aligned} &\neg \text{Fragile}(y) \rightarrow \text{Safe-To-Stack}(x, y) \\ &\text{Lighter}(x, y) \rightarrow \text{Safe-To-Stack}(x, y) \\ &\text{Volume}(p, v) \wedge \text{Density}(p, d) \wedge *(v, d, w) \rightarrow \text{Weight}(p, w) \\ &\text{Isa}(p, \text{table-book}) \rightarrow \text{Weight}(p, 15) \\ &\text{Weight}(p_1, w_1) \wedge \text{Weight}(p_2, w_2) \wedge \text{Smaller}(w_1, w_2) \rightarrow \text{Light}(p_1, p_2) \end{aligned}$$

训练实例为描述物体 Obj1 与 Obj2 的下述事实：

$$\begin{aligned} &\text{On}(\text{Obj1}, \text{Obj2}) \\ &\text{Isa}(\text{Obj1}, \text{book-AI}) \\ &\text{Isa}(\text{Obj2}, \text{table-book}) \\ &\text{Volume}(\text{Obj1}, 1) \\ &\text{Density}(\text{Obj1}, 0.1) \end{aligned}$$

请画出目标概念的证明树和泛化树，并由泛化树给出目标概念的可操作的一般规则。

7.7 简述遗传算法的主要特点和基本流程。

7.8 适应度函数在遗传算法中的作用是什么？试举例说明如何构造适应度函数。

7.9 应用遗传算法，编制 C 语言程序，求解例 7.7。算法终止条件为群体中的所有个体之间没有差异。并记录算法终止时的迭代次数。

7.10 应用遗传算法，编制 C 语言程序，建立例 7.8 要求的港口年吞吐量预测模型。算法终止条件为连续 10 代不再产生适应度更小的后代个体。记录算法终止时的迭代次数，并由预测模型计算连续 10 年（即 2001 年~2010 年）的预测值。

第 8 章 人工神经网络

人工神经网络（ANN, Artificial Neural Network）是反映人脑结构及功能的一种抽象数学模型，一个人工神经网络是由大量神经元节点互连而成的复杂网络，用以模拟人类进行知识的表示与存储以及利用知识进行推理的行为。

一个基于人工神经网络的智能系统是通过学习获取知识后建立的。从本质上讲，人工神经网络的学习是一种归纳学习方式，它通过对大量实例的反复学习，由内部自适应过程不断修改各神经元之间互连的权值，最终使神经网络的权值分布收敛于一个稳定的范围。神经网络的互连结构及各连接权值稳定分布就表示了经过学习获得的知识。这同基于符号的知识表示方法有很大的不同。一个已建立的人工神经网络可用于相关问题的求解，对于特定的输入模式，神经网络通过前向计算可得出一个输出模式，从而得到输入模式的一个特定解。

8.1 人工神经元与感知器

人工神经元是人工神经网络的基本单元，神经网络是由大量的神经元互连而成的。感知器（Perceptron）是由美国学者罗森勃拉特（F.Rosenblatt）于 1957 年提出的，它是一种具有单层计算单元的神经网络。最初的 Perceptron 算法相当于单个神经元，有很大的局限性，但是，它提出了自组织、自学习的思想，后来的许多神经网络模型都是在这种思想指导下建立的。感知器对能够解决的问题有一个收敛的算法，并从数学上给出了严格的证明，它在神经网络的研究中有着重要的意义，推动了人工神经网络研究的发展。

8.1.1 人工神经元模型

人工神经元是对生物神经元的简化和模拟。生物神经元由细胞体、树突和轴突三部分组成，树突是细胞的输入端，轴突是细胞的输出端。树突通过联结其他细胞体的“突触”接受周围细胞由轴突的神经末梢传出的神经冲动；轴突的端部有众多神经末梢作为神经信号的输出端子，用于传出神经冲动。生物神经元具有兴奋与抑制两种状态，当传入的神经冲动使细胞膜电位升高到阈值（约为 40mv）时，细胞进入兴奋状态，产生神经冲动，由轴突输出；若传入的神经冲动使细胞膜电位低入阈值时，则细胞进入抑制状态，没有神经冲动输出。

为了模拟生物神经细胞，可以把一个神经细胞简化为一个人工神经元，人工神经元用一个多输入、单输出的非线性节点表示，如图 8.1 所示。

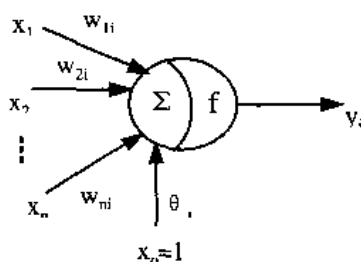


图 8.1 人工神经元

神经细胞 i 的人工神经元的输入输出关系可描述为:

$$\begin{cases} I_i = \sum_{j=1}^n w_{ji} x_j - \theta_i \\ y_i = f(I_i) \end{cases} \quad (8-1)$$

式中, x_j 是由细胞 j 传送到细胞 i 的输入量。 w_{ji} 是从细胞 j 到细胞 i 的连接权值, θ_i 是细胞 i 的阈值, f 是传递函数, y_i 是细胞 i 的输出量。

有时为了方便, 将 I_i 表示成:

$$I_i = \sum_{j=0}^n w_{ji} x_j$$

式中: $w_{0i} = -\theta_i$, $x_0 = 1$ 。

传递函数 f 可为线性函数, 或为具有任意阶导数的非线性函数。常见的传递函数有如下几种。

(1) 阶跃函数

阶跃函数的形式为:

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (8-2)$$

阶跃函数的图形如图 8.2 (a) 所示。

(2) Sigmoid 型函数

Sigmoid 型函数是函数图形如 S 形状的一类可微函数。常用的 Sigmoid 型函数有:

$$f(x) = \frac{1}{1 + e^{-x}} \quad (8-3)$$

$$f(x) = \text{th}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (8-4)$$

函数 $1/(1+\exp(-x))$ 的函数图形如图 8.2 (b) 所示, 双曲正切函数 $\text{th}(x)$ 的函数图形如图 8.2 (c) 所示。双曲正切函数 $\text{th}(x)$ 的特点是函数图形关于坐标原点对称。

(3) 高斯型函数

在径向基神经网络中, 神经元的输入输出关系用高斯函数表示为:

$$y_i = \exp\left(-\frac{1}{2\sigma_i^2} \sum_j (x_j - w_{ji})^2\right) \quad (8-5)$$

式中, σ_i^2 为标准化参数。

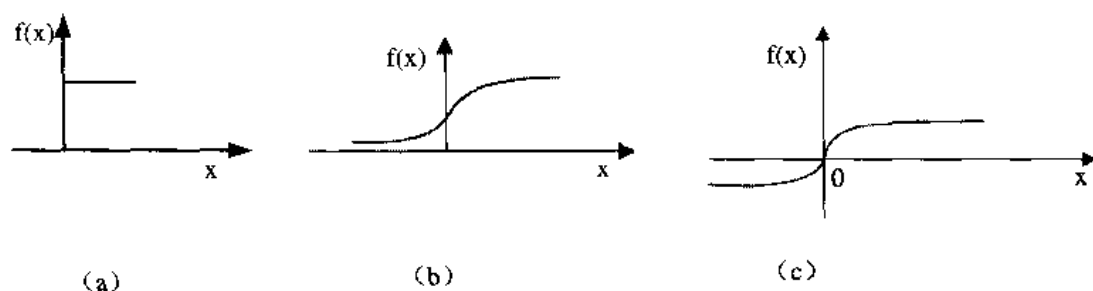


图 8.2 常用传递函数的函数图形

8.1.2 感知器及其学习算法

感知器是一种具有单层计算单元的前向神经网络模型,如图 8.3 所示,其中, n 个输入 $x_1, x_2, \dots, x_n$ 均为实数, $w_1, w_2, \dots, w_n$ 分别是 n 个输入的加权, θ 是感知器的阈值, f 是感知器的传递函数, y 是感知器的输出。

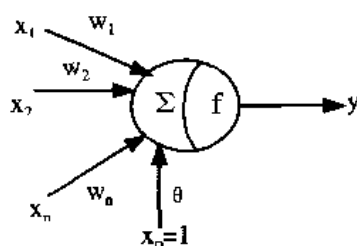


图 8.3 感知器

感知器的输出与输入的关系可描述为:

$$y = \begin{cases} 1 & \sum_{i=1}^n w_i x_i - \theta \geq 0 \\ 0 & \sum_{i=1}^n w_i x_i - \theta < 0 \end{cases} \quad (8-6)$$

实际上,感知器的传递函数 f 是阶跃函数。

感知器的学习过程为:由已知的 N 个实例 (X_k, Y_k^*) , $k=1, 2, \dots, N$, 组成一组学习样本, 其中,实例 k 的输入 X_k 可表示为一个 n 元向量 $X_k = (x_{1k}, x_{2k}, \dots, x_{nk})$, 实例 k 的期望输出为单一输出 $Y_k^* = y_k^*$ 。对实例 (X_k, Y_k^*) , 调整可表示为 n 元数组的权值分布 $W = (w_1, w_2, \dots, w_n)$ 中各连接权的值及阈值 θ 。从理论上讲,在 n 维欧氏空间 E^n 中,若点集 $\{X_k | f(X_k)=1\}$ 与点集 $\{X_k | f(X_k)=0\}$ 是线性可分的,即存在一个超平面将两个点集分开,那么感知器学习过程在有限次迭代后可收敛于一个稳定的权值分布和阈值。

如果把阈值 θ 表示成 $\theta = -w_0$, 相应地对每一个输入向量 X_k 也增加一个分量 $x_{0k}=1$, 那么, 感知器的计算输出可以表示为 $y_k=f(\sum_{i=0}^n w_i x_{ik})$ 。

感知器的学习过程就是通过对实例集 $\{(X_k, Y_k^*)\}$ 的反复学习, 不断地修改权值分布 W 中各连接权 w_i 的值的过 程, 直至感知器对各实例的计算输出 $y_k=f(\sum_{i=0}^n w_i x_{ik})$ 满足各实例的期望输出, 即有 $|y_k - y_k^*| \leq \varepsilon$, $k=1, 2, \dots, N$, ε 为指定的输出误差。此时, 称学习过程收敛于一个稳定的权值分布。修改权值分布 W 是迭代进行的, 设 t 为迭代次数, 初始化权值分布为 $W(0)$, 第 $t+1$ 次迭代将权值分布 $W(t)$ 修改为 $W(t+1)$ 。显然, 权值分布 W 的修改方法是感知器学习算法的核心。

感知器学习算法如下:

1) 初始化权值分布 $W(0)$, 对 $W(0)=(w_0(0), w_1(0), \dots, w_i(0), \dots, w_n(0))$ 中的各权值分别赋给较小的随机非零值。取第一个学习实例 (X_1, Y_1^*) , $k=1$, 初始化迭代次数 $t=0$ 。

2) 由给定的学习实例 (X_k, y_k^*) 的输入向量 $X_k=(1, x_{1k}, x_{2k}, \dots, x_{ik}, \dots, x_{nk})$, 以及权值分布 $W(t)=(w_0(t), w_1(t), \dots, w_i(t), \dots, w_n(t))$, 计算感知器的输出 $y_k(t)$ 为:

$$y_k(t) = f(\sum_{i=0}^n w_i(t) x_{ik})$$

3) 修改权值分布 $W(t)$ 为 $W(t+1)$ 。权值修改的计算为:

$$w_i(t+1) = w_i(t) + \eta (y_k^* - y_k(t)) x_{ik} \quad i=0, 1, \dots, n$$

其中, 学习率 η 为设定的修改速度控制参数, $0 < \eta \leq 1$ 。

权值修改计算公式表明: 若感知器的计算输出 $y_k(t)$ 与实例给定的期望输出 y_k^* 相等, 即 $y_k(t)=y_k^*$, 则 $w_i(t)$ 不变; 若 $y_k(t) < y_k^*$, 则增大正输入 ($x_{ik} > 0$) 的权值 $w_i(t)$, 减小负输入 ($x_{ik} < 0$) 的权值 $w_i(t)$; 若 $y_k(t) > y_k^*$, 则对权值修改的情况相反, 减小正输入的权值, 增大负输入的权值。

对权值修正的速度由 η 的取值决定, 当设定的 η 值较大时, 每次对权值的修改较大。但若 η 取值过大, 则权值的修改过程将可能围绕稳定权值振荡, 反而延长了修改过程收敛的时间。若 η 取值过小, 则权值的修改过程将缓慢地收敛于稳定权值, 同样也延长了修改过程收敛的时间。一般来说, 学习率 η 可以设定为 $0.3 \leq \eta \leq 0.9$ 。还可以采用变学习率方法来提高学习过程的收敛速度, 即在学习的初期, 学习率 η 可设定较大, 随着迭代次数 t 的增加而逐步减小学习率 η 的值。

4) $t=t+1$, 取下一个学习实例, 转步骤 2)。

N 个学习实例依次被循环使用, 直至对全部学习实例都有 $y_k=y_k^*$, 即 W 不再被修改, 说明 W 已经收敛于稳定的权值分布, 则学习过程终止。

实际上, 学习算法的终止条件可以设定宽松一些, 例如, 可设定为满足以下两个条件之一即可终止: 对全部学习实例满足 $|y_k^* - y_k| \leq \varepsilon$, 其中, ε 是初始设定的允许误差; 或者迭代次数 t 达到初始设定的最大迭代次数 $t_{\max}$ 。

需要特别指出的是, 只有当提供的学习实例集是线性可分时, 学习过程才会经有限次迭代而收敛, 才能得出稳定的权值分布 W , 因此, 需要对感知器及其学习算法进行改进, 传递函数也不采用阶跃函数, 而改用可微函数, 例如采用 Sigmoid 型函数。

8.2 人工神经网络模型

自 1943 年美国心理学家麦克洛奇 (McCulloch) 和数学家皮兹 (Pitts) 提出形式神经元的抽象数学模型 (MP 模型) 以来, 经过 50 多年的发展, 人工神经网络理论及其技术已经取得了大量的研究成果, 并在模式识别、图像处理、专家系统、组合优化、智能控制以及机器人等领域得到了广泛的应用, 提出了各种神经网络模型。这里, 我们讨论几种常用的神经网络模型。

8.2.1 神经网络的互连结构

根据神经网络中神经元之间互连的结构不同, 把神经网络可以分成以下几种类型。

(1) 不含反馈的前向网络

不含反馈的前向网络的结构形态如图 8.4 (a) 所示。网络中的神经元分层排列, 接受输入量的神经元节点组成输入层, 产生输出量的神经元节点组成输出层, 中间层亦称为隐层, 可以有若干层隐层。每一层的神经元只接受前一层神经元的输入, 输入向量经过各层的顺序变换后, 由输出层得到输出向量。

(2) 从输出层到输入层有反馈的前向网络

从输出层到输入层有反馈的前向网络简称为反馈神经网络, 其结构形态如图 8.4 (b) 所示。网络中的神经元也是分层排列, 但是输入层神经元在学习过程中接受输出层神经元或部分输出层神经元的反馈输入。

在反馈神经网络中, 由输入数据决定反馈神经网络的初始状态, 经过一系列状态转移后反馈神经网络逐渐收敛于一个稳定状态, 这个稳定状态就是反馈神经网络的最后计算结果。不含反馈的前向网络 (简称前向神经网络) 是一种强有力的学习系统, 结构简单且易于编程。从系统观点来看, 它是一种静态非线性映射, 通过大量的简单的非线性神经元的复合映射, 可获得复杂的非线性映射处理能力, 但是, 它缺乏丰富的动力学行为。从系统观点来看, 反馈神经网络是反馈动力学系统, 它比前向神经网络具有更强的计算能力。反馈神经网络的一个重要特点就是它应具有稳定状态, 稳定性是研究反馈神经网络的最重要的问题之一。

(3) 层内有相互结合的前向网络

层内有相互结合的前向网络的结构形态如图 8.4 (c) 所示。每一层的神经元除接受前一层神经元的输入之外, 也可接受同一层神经元的输入。

通过层内神经元之间的相互结合, 可以实现同层神经元之间横向的抑制或兴奋机制, 从而可以限制一层内能同时动作的神经元的个数, 或者实现把一层内的神经元分为若干组, 每

一组作为一个整体来动作。例如，可以利用横向抑制机制把同层中具有最大输出的神经元挑选出来，而抑制其他神经元处于无输出的状态。

(4) 相互结合型网络

相互结合型网络的结构形态如图 8.4 (d) 所示，这种网络中任意两个神经元之间都可能连接。在不含反馈的前向网络中，输入信号一旦通过某个神经元就将输出这个信号的变换值。但是，在相互结合型网络中，输入信号要在神经元之间反复往返传递，网络处于一种不断改变状态的动态之中。从某一初态开始，经过若干次的状态变化，网络才会到达某种稳定状态，根据网络的结构和神经元的映射特性，网络还有可能进入周期振荡或其他平衡状态，如混沌状态。

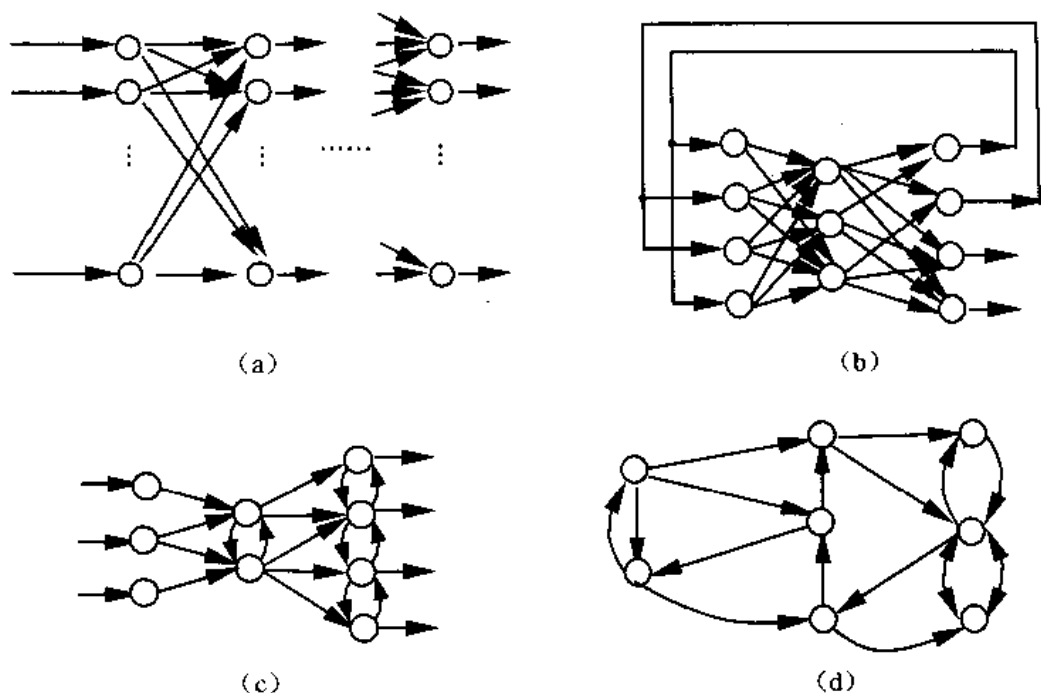


图 8.4 人工神经网络互连结构的几种类型

以上 4 种类型的神经网络，从结构形态来看，(1)、(2)、和 (3) 可看成是 (4) 的一种特殊形态，但是，不论从神经网络的计算和学习机制来看还是从网络的应用来看，这 4 种类型的神经网络是有很大的区别的。

8.2.2 前向神经网络

常见的前向神经网络有 BP 神经网络和 RBF 径向基神经网络。

1. BP 神经网络

BP 神经网络 (Backpropagation Neural Network) 是一种单向传播的多层前向神经网络，其结构如图 8.5 所示。BP 网络除有输入层和输出层之外，还有一层或多层隐层，同层节点间

无任何联结, 每个节点都是单个神经元, 神经元的传递函数通常为 Sigmoid 型函数。有时, 输入层或输出层的神经元的传递函数选取线性函数。由于同层节点间无任何耦合, 因此, 每一层的神经元只接受前一层神经元的输入, 每一层神经元的输出只影响下一层神经元的输出。

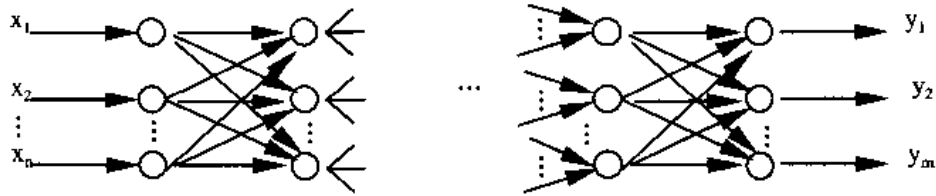


图 8.5 BP 神经网络

网络的输入数据 $X = (x_1, x_2, \dots, x_n)$ 从输入层依次经过各隐层节点, 然后到达输出层节点, 从而得到输出数据 $Y = (y_1, y_2, \dots, y_m)$ 。我们可以把 BP 神经网络看成是一个从输入到输出的高度非线性映射, 即 $f: R^n \rightarrow R^m, f(X) = Y$ 。

对于学习实例集 $\{(X_k, Y_k^*)\}$, 可以认为所有实例的输入 $X_k \in R^n$ 和实例的期望输出 $Y_k^* \in R^m$ 之间存在某一映射函数 g , 使得 $g(X_k) = Y_k^*, k=1, 2, \dots, n$ 。

现在的问题是: BP 神经网络是否存在这样的非线性映射 f 高度地逼近映射函数 g ? 如果网络的映射 f 能逼近函数 g , 那么就可以用 BP 神经网络来实现许多复杂的连续函数的映射关系, 使得许多难以通过求得连续函数解决的问题转化为用神经网络来解决。

下述的两个定理可以回答这个问题。

定理 8.1 (Kolmogorov 定理) 给定任一连续函数 $f: [0, 1]^n \rightarrow R^m$, f 可以精确地用一个三层前向神经网络实现, 此网络的第一层即输入层有 n 个神经元, 中间层有 $2n+1$ 个神经元, 第三层即输出层有 m 个神经元。

定理 8.2 给定任意 $\epsilon > 0$, 对于任意的 L_2 型连续函数 $f: [0, 1]^n \rightarrow R^m$, 存在一个三层 BP 神经网络, 它可在任意 ϵ 平方误差精度内逼近 f 。

这两个定理的证明见有关参考文献。它们说明了任一连续函数 $f: [0, 1]^n \rightarrow R^m$ 都可以用一个结构为 $n \times (2n+1) \times m$ 的三层前向神经网络来精确地逼近。

定理 8.1 和定理 8.2 不仅证明了映射网络的存在, 而且说明了映射网络的结构。但是, 在实际应用中, 有时需要使用具有多个隐层的 BP 神经网络。如何合理地选取 BP 网络的隐层数及隐层的节点数, 目前尚无准确有效的理论和方法。BP 网络输入层的节点数由学习实例的输入向量 $X_k = (x_{1k}, x_{2k}, \dots, x_{nk})$ 的输入量的个数 n 决定, 输出层的节点数由学习实例的输出向量 $Y_k^* = (y_{1k}^*, y_{2k}^*, \dots, y_{mk}^*)$ 的输出量的个数 m 决定。

2. RBF 神经网络

RBF (Radial Basis Function) 径向基神经网络由三层节点组成, 其结构如图 8.6 所示。在 RBF 神经网络中, 输入层节点仅把输入数据 $(x_1, x_2, \dots, x_n)$ 传递给隐层各节点, 隐层各节点亦

称为 RBF 节点, RBF 节点的传递函数采用高斯型函数, 输出层节点的传递函数通常采用简单的线性函数。

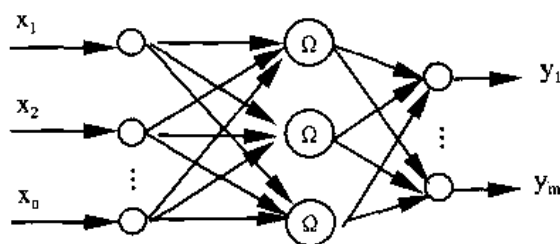


图 8.6 径向基神经网络

设 RBF 网络的输入向量为 $X=(x_1, x_2, \dots, x_n)$, 由于输入层节点只是把输入数据直接传播到隐层各节点, 因此, 隐层各节点的输出为:

$$u_i = \exp\left(-\frac{(X - c_i)^T(X - c_i)}{2\sigma_i^2}\right) \quad i=1,2,\dots,N \quad (8-7)$$

其中, u_i 为隐层节点 i 的输出, 且 $u_i \in [0,1]$, c_i 是高斯函数的中心值, σ_i 是标准化常数, N 是隐层节点数。

输出层节点的输出是隐层节点输出的线性组合, 即:

$$y_j = \sum_{i=1}^N w_{ij} u_i - \theta = W_j^T U \quad j=1,2,\dots,m \quad (8-8)$$

其中, y_j 为输出层节点 j 的输出, w_{ij} 为隐层节点 i 联结输出层节点 j 的权值, θ 为输出层节点阈值, $W_j = (w_{1j}, w_{2j}, \dots, w_{Nj}, -\theta)^T$, $U = (u_1, u_2, \dots, u_N, 1)$ 。

定理 8.3 给定任意 $\epsilon > 0$, 对任意的 L_2 型连续函数 $f: R^n \rightarrow R^m$, 存在一个径向基神经网络可在任意 ϵ 平方误差精度内逼近 f 。

定理 8.3 表明径向基神经网络同 BP 神经网络一样可逼近任意的连续非线性函数。RBF 网络与 BP 网络的区别在于: BP 网络的隐层节点的传递函数使用 Sigmoid 型函数, 其取值范围在输入空间无限大的范围内取非零值; RBF 网络的隐层节点的传递函数使用高斯型函数, 其取值范围呈现局部特征, 这是因为高斯型函数对输入数据将在局部产生响应, 也就是说, 当输入数据靠近高斯函数的中心范围时, 隐层节点将产生较大的输出, 反之, 则产生较小的输出。

8.2.3 反馈神经网络

反馈神经网络 (Feedback Neural Network) 是有反馈传播的神经网络, 在反馈神经网络系统中, 输入数据决定反馈网络系统的初始状态, 经过一系列状态转移后, 如果系统能逐渐收敛于稳定状态, 那么这个稳定状态就是反馈神经网络经计算后的输出结果。一个反馈神经网络是否能收敛于稳定状态的网络稳定性问题是反馈神经网络的最重要的问题。

美国加州工学院物理学家霍普菲尔特 (J.J.Hopfield) 于 1982 年和 1984 年提出的离散

Hopfield 神经网络和连续 Hopfield 神经网络是最典型的反馈神经网络。

1. 时间离散的 Hopfield 神经网络

离散 Hopfield 神经网络 (DHNN) 是离散时间系统, 它可用一个加权无向图表示, 图的一个节点是一个神经元, 每个节点都附有一个阈值, 联结两个节点的一个边附有一个权值, 网络的阶数相应于图中节点数。

设 N 是一个 n 阶神经网络, 则 N 由 (W, θ) 惟一确定, 其中 $W=(w_{ij})_{n \times n}$ 为 n 阶对称矩阵, w_{ij} 为联结节点 i 与节点 j 的边所附的权, $\theta=(\theta_i)_n$ 是一个向量, θ_i 为节点 i 的阈值。

DHNN 的每个节点可处于两种可能状态之一, 即 1 或 -1。以 $x_i(t)$ 表示节点 i 在 t 时刻的状态, 向量 $X(t)=(x_1(t), x_2(t), \dots, x_n(t))$ 表示神经网络 N 在 t 时刻的状态。节点 i 在状态转移中的下一状态由下述规则决定:

$$x_i(t+1) = \text{sgn}(H_i(t)) = \begin{cases} 1 & \text{若 } H_i(t) \geq 0 \\ -1 & \text{若 } H_i(t) < 0 \end{cases} \quad (8-9)$$

其中

$$H_i(t) = \sum_{j=1}^n w_{ij} x_j(t) - \theta_i$$

上式就是离散 Hopfield 神经网络的定义, DHNN 在 t 时刻的状态就是向量 $X(t) \in \{1, -1\}^n$ 。若 $w_{ii}=0, i=1, 2, \dots, n$, 则相应的 DHNN 称为无自反馈的 DHNN; 否则, 称为有自反馈的 DHNN。

若反馈神经网络在 $t=0$ 时有任意输入数据为 $X(0)=(x_1(0), x_2(0), \dots, x_n(0))$, 从初态 $X(0)$ 开始, 存在某一个有限时刻 t , 从 t 时刻之后的神经网络状态不再发生变化, 即有

$$X(t+\Delta t) = X(t) \quad \Delta t > 0$$

则称神经网络是稳定的。 t 时刻的网络状态 $X(t)$ 称为神经网络的稳定状态, 也就是神经网络计算的输出结果。

如上所述, 只有当反馈神经网络是稳定的, 才能得出稳定的计算输出结果。DHNN 的稳定性与 DHNN 的工作方式有关, DHNN 有以下两种工作方式。

(1) 串行 (异步) 工作方式

DHNN 的串行 (异步) 工作方式是指: 在任一时刻 t , DHNN 中只有某一个神经元 i (随机地或确定性选择) 的状态发生变化, 其余 $n-1$ 个神经元的状态保持不变, 即有:

$$\begin{cases} x_i(t+1) = \text{sgn}(H_i(t)) \\ x_j(t+1) = x_j(t) \quad \forall j \neq i \end{cases} \quad (8-10)$$

串行工作方式下的稳定性称为串行稳定性。关于 DHNN 的串行稳定性有下述定理。

定理 8.4 (Hopfield 定理) 一个 DHNN 可表示为 $N=(W, \theta)$, W 为一对称矩阵。若 N 工作在串行方式下, 且 W 的对角线元素非负, 即 $w_{ii} \geq 0, i=1, 2, \dots, n$, 则 N 一定收敛于稳定状态。

(2) 并行 (同步) 工作方式

DHNN 的并行 (同步) 工作方式是指: 在任一时刻 t , DHNN 中都有多个神经元或全部

神经元的状态发生变化。若在任一时刻 t , 全部神经元的状态都发生变化, 则称为全并行工作方式, 即有:

$$x_i(t+1) = \text{sgn}\left(\sum_{j=1}^n w_{ij}x_j(t) - \theta_i\right) \quad \forall i \quad (8-11)$$

并行工作方式下的稳定性称为并行稳定性。关于 DHNN 的并行稳定性有下述定理。

定理 8.5 (Goles 定理) 一个 DHNN 可表示为 $N=(W, \theta)$, W 为一对称矩阵。若 N 工作在全并行方式时, 则 N 总是收敛于稳定状态或状态空间中长度小于 2 的极限环。长度为 2 的极限环上的状态满足 $X(t-1)=X(t+1)$ 。

定理 8.6 一个 DHNN 可表示为 $N=(W, \theta)$, W 为一对称矩阵。若 W 为非负定矩阵, 则 N 具有并行稳定性。

若 W 为非负定矩阵, 那么, 对神经网络的状态变化

$$\Delta X = (\Delta x_1, \Delta x_2, \dots, \Delta x_n)$$

其中 $\Delta x_i = x_i(t+1) - x_i(t)$, 满足

$$(\Delta X)^T W (\Delta X) \geq 0 \quad (8-12)$$

2. 时间连续的 Hopfield 神经网络

连续 Hopfield 神经网络可用下述非线性微分方程描述:

$$\begin{cases} C_i \frac{dx_i}{dt} = -x_i / R_i + I_i + \sum_{j=1}^n t_{ij} y_j \\ y_j = g(x_j) \quad j=1, 2, \dots, n \end{cases} \quad (8-13)$$

上述微分方程可用一电路来表示, 如图 8.7 所示。其中, 电阻 R_i 和电容 C_i 并联, 以模拟生物神经元输出的时间常数; 跨导 t_{ij} 模拟生物神经元之间互连的突触特性; 由运算放大器来模拟生物神经元的非线性特性, 运算放大器的输出为 $y_i = g(x_i)$, x_i 为运算放大器的输入, g 为 Sigmoid 型函数。

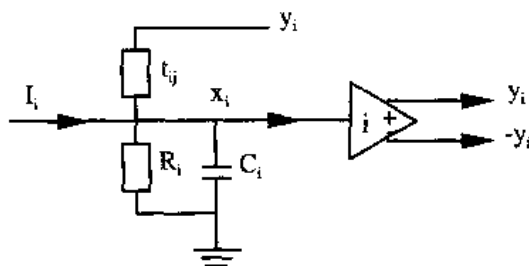


图 8.7 Hopfield 电路

连续 Hopfield 神经网络的状态由 n 个神经元的输出组成的状态向量 $Y = (y_1, y_2, \dots, y_n)$ 表示。连续 Hopfield 神经网络的稳定性可由李雅普诺夫 (Lyapunov) 能量函数来判定。定义连续 Hopfield 神经网络的 Lyapunov 能量函数 E 为:

$$E = \sum_{i=1}^n \frac{1}{R_i} \int_0^{y_i} g^{-1}(y) dy - \sum_{i=1}^n I_i y_i - \frac{1}{2} \sum_{i,j=1}^n t_{ij} y_i y_j \quad (8-14)$$

定理 8.7 对连续 Hopfield 神经网络, 若 $C_i > 0$, $t_{ij} = t_{ji}$, 则该神经网络在状态空间中的状态转移总是向能量函数 E 的值减小的方向转移, 且能量函数 E 为极小值 $E_{\min}$ 的状态就是神经网络的稳定状态。

对于理想的运算放大器, 能量函数 E 可以简化为:

$$E = -\frac{1}{2} \sum_{i,j=1}^n t_{ij} y_i y_j - \sum_{i=1}^n I_i y_i \quad (8-15)$$

定理 8.7 说明稳定的连续 Hopfield 神经网络是存在的。可以通过有限次的学习来不断修改权值 t_{ij} , $i, j=1, 2, \dots, n$, 使能量函数 E 在有限次学习中为局部最小的权值分布 $T=[t_{ij}]_{n \times n}$, $t_{ij}=0$, 即对应稳定的连续 Hopfield 神经网络。

8.3 神经网络的学习

人工神经网络卓越的非线性映射能力一是来自于网络中各神经元的非线性传递函数; 二是来自于各神经元之间的连接权分布。神经网络的连接权分布一般不能预先准确地确定, 而是通过对学习实例的反复学习来逐渐调整和修改权值分布, 使神经网络收敛于稳定状态, 从而完成学习过程。一个稳定的神经网络就是一个特定的知识表示, 可用于对相应领域问题的求解。

8.3.1 神经网络的两类学习方法

神经网络的学习方法可分为两类: 有教师学习和无教师学习。

1. 无教师学习方法

无教师学习的基本思想是: 当输入的实例模式进入神经网络后, 网络按预先设定的规则自动调整权值。

常见的无教师学习方法是 Hebb 学习规则, 它来源于心理学家赫布 (Hebb) 关于生物神经元的下述学习假设: 当两个神经元同时处于兴奋状态时, 它们之间的连接应该加强。若设 $w_{ij}(k)$ 表示神经元 i 联结神经元 j 的当前权值, I_i 和 I_j 分别表示神经元 i 和神经元 j 的激活水平, 那么, 关于神经元的 Hebb 学习规则可表示为:

$$w_{ij}(k+1) = w_{ij}(k) + I_i I_j \quad (8-16)$$

对于图 8.1 所示的人工神经元, 有:

$$\begin{cases} I_i = \sum_{j=1}^n w_{ji} x_j - \theta_i \\ y_i = f(I_i) \end{cases}$$

于是, 图 8.1 所示的人工神经元的 Hebb 学习规则可表示为:

$$w_{ij}(k+1)=w_{ij}(k)+y_i y_j \quad (8-17)$$

2. 有教师学习方法

有教师学习的基本思想是：对实例 k 的输入，由神经网络根据当前的权值分布 $W(k)$ 计算网络的输出 $Y(W,k)$ ，把网络的计算输出 $Y(W,k)$ 与实例 k 的期望输出 $Y^*(k)$ 进行比较，根据两者之间的差的某个函数的值来调整网络的权值分布，最终使差的函数值达到最小。

前述 8.1.2 节介绍的感知器及其学习算法就是一种有教师学习方法。

常见的有教师学习方法是梯度下降法，其基本思想是根据实例 k 的期望输出 $Y^*(k)$ 与网络计算输出 $Y(W,k)$ 的误差的平方最小的原则来修改权值分布。

定义误差函数 $J(W)$ 为：

$$J(W) = \frac{1}{2} (Y^*(k) - Y(W, k))^2 \quad (8-18)$$

式中， $Y^*(k)$ 是实例 k 的输出， $Y(W,k)$ 是由当前权值分布 W 对实例 k 的输入经网络计算的输出。梯度下降法就是沿着 $J(W)$ 的负梯度方向不断修正 W 的值，直至 $J(W)$ 达到最小值。

梯度下降法对权值的修改可表示为：

$$W(k+1) = W(k) + \eta(k) \left(-\frac{\partial J(w)}{\partial w} \right) \Big|_{w=W(k)} \quad (8-19)$$

式中， $\eta(k)$ 是可变学习率，用以控制权值修改的速度。在学习过程中， $\eta(k)$ 也可以不变，取成一个常量 η ， $0 < \eta < 1$ 。

8.3.2 BP 学习算法及其改进方法

1982 年，鲁梅尔哈特 (D.Rumelhart) 和麦克莱伦德 (McClelland) 以及他们的同事们成立了一个 PDP (Parallel Distributed Processing) 小组，研究并行分布信息处理方法，探索人类认知的微结构。1985 年，他们提出了 BP 神经网络及其学习算法，这个 BP 网络的学习算法称为误差反向传播算法或 BP 学习算法。

BP 学习算法是如图 8.5 所示的 BP 神经网络的学习算法。BP 学习算法是一种迭代算法，一次学习过程由输入数据的正向传播和误差的反向传播两个子过程组成。设有 N 个学习实例 (X_k, Y^*_k) , $k=1, 2, \dots, N$ ，对实例 (X_k, Y^*_k) ，在正向传播过程中，实例 k 的输入向量 $X_k=(x_{1k}, x_{2k}, \dots, x_{nk})$ 从输入层的 n 个节点输入，经隐层逐层处理，由输出层的 m 个节点的输出端得到实例 k 的网络计算输出向量 $Y_k=(y_{1k}, y_{2k}, \dots, y_{mk})$ ，比较 Y_k 和实例 k 的期望输出向量 $Y^*_k=(y^*_{1k}, y^*_{2k}, \dots, y^*_{mk})$ ，若 N 个学习实例的计算输出都达到期望的结果，则学习过程结束；否则，进入误差反向传播过程，把 Y_k 与 Y^*_k 的误差由网络输出层向输入层反向传播，在反向传播过程中，修改各层神经元的连接权值。

1. 误差反向传播计算

设 $I_k^{(l)}$ 表示实例 k 的输入向量 X_k 输入后，传播到第 l 层节点 j 的输入， $O_j^{(l)}$ 表示第 l 层节点 j 的输出， $w_{ij}^{(l-1)}$ 为第 $l-1$ 层的节点 i 连结第 l 层节点 j 的权值， $n^{(l-1)}$ 为第 $l-1$ 层的节点数， f

为节点神经元的传递函数, BP 网络的神经元传递函数一般使用可微的 Sigmoid 型函数。由 BP 网络神经元的输入输出关系, 有:

$$I_{jk}^{(l)} = \sum_{i=1}^{n^{(l-1)}} w_{ij}^{(l-1)} O_{ik}^{(l-1)} \quad (8-20)$$

$$O_{jk}^{(l)} = f(I_{jk}^{(l)}) \quad (8-21)$$

实例 k 对节点 j 的期望输出 $O_{jk}^{*(l)}$ 与节点 j 对实例 k 的网络计算输出 $O_{jk}^{(l)}$ 的误差定义为:

$$E_{jk}^{(l)} = \frac{1}{2} (O_{jk}^{*(l)} - O_{jk}^{(l)})^2 \quad (8-22)$$

若第 l 层是 BP 网络的输出层, 即节点 j 是输出节点, 则 $O_{jk}^{*(l)} = y_{jk}^*$, $O_{jk}^{(l)} = y_{jk}$, 实例 k 的输出误差为:

$$E_{jk}^{(l)} = \frac{1}{2} (y_{jk}^* - y_{jk})^2 \quad (8-23)$$

若对 N 个学习实例的任一实例 K 有输出层的 m 个输出节点的计算输出都分别满足实例 k 的 m 个期望输出, 即有 $E_{jk}^{(l)} \leq \varepsilon$, $j=1,2,\dots,m$, 则学习过程结束, ε 为指定的允许误差; 否则, 由误差反向传播过程修改权值分布 W 。

按误差的负梯度来修改权值, 即:

$$\begin{cases} w_{ij}^{(l-1)} = w_{ij}^{(l-1)} + \Delta w_{ij}^{(l-1)} \\ \Delta w_{ij}^{(l-1)} = -\eta \frac{\partial E_{jk}^{(l)}}{\partial w_{ij}^{(l-1)}} \end{cases} \quad (8-24)$$

其中, η 为学习率, $0 < \eta < 1$ 。

由 (8-22) 式、(8-21) 式和 (8-20) 式, 可有:

$$\frac{\partial E_{jk}^{(l)}}{\partial w_{ij}^{(l-1)}} = \frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} \frac{\partial O_{jk}^{(l)}}{\partial I_{jk}^{(l)}} \frac{\partial I_{jk}^{(l)}}{\partial w_{ij}^{(l-1)}} = \delta_{jk}^{(l)} \frac{\partial I_{jk}^{(l)}}{\partial w_{ij}^{(l-1)}} = \delta_{jk}^{(l)} O_{ik}^{(l-1)} \quad (8-25)$$

其中

$$\delta_{jk}^{(l)} = \frac{\partial E_{jk}^{(l)}}{\partial I_{jk}^{(l)}} = \frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} \frac{\partial O_{jk}^{(l)}}{\partial I_{jk}^{(l)}} = \frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} f'(I_{jk}^{(l)}) \quad (8-26)$$

为了得出计算 $\delta_{jk}^{(l)}$ 的表达式, 进行下述讨论。

1) 若第 l 层是输出层, 则由 (8-23) 式可得出:

$$\frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} = \frac{\partial E_{jk}^{(l)}}{\partial y_{jk}} = -(y_{jk}^* - y_{jk}) \quad (8-27)$$

由 (8-26) 式和 (8-27) 式可得下述 (8-28) 式:

$$\delta_{jk}^{(l)} = -(y_{jk}^* - y_{jk})f'(\mathbf{I}_{jk}^{(l)}) \quad (8-28)$$

由(8-24)式、(8-25)式和(8-28)式可得下述(8-29)式:

$$\Delta w_{ij}^{(l-1)} = -\eta \delta_{jk}^{(l)} O_{ik}^{(l-1)} = \eta (y_{jk}^* - y_{jk}) f'(\mathbf{I}_{jk}^{(l)}) O_{ik}^{(l-1)} \quad (8-29)$$

2) 若第 l 层不是输出层, 则根据 BP 网络的误差反向传播定义第 l 层节点 j 的误差 $E_{jk}^{(l)}$ 对节点 j 的输出 $O_{jk}^{(l)}$ 的变化率为第 $l+1$ 层的 $n^{(l+1)}$ 个节点的各节点误差对其输出的变化率之和。若 $E_{qk}^{(l+1)}$ 表示第 $l+1$ 层的节点 q 的误差, $O_{qk}^{(l+1)}$ 表示节点 q 的计算输出, 则有:

$$\frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} = \sum_{q=1}^{n^{(l+1)}} \frac{\partial E_{qk}^{(l+1)}}{\partial O_{qk}^{(l+1)}} \quad (8-30)$$

对第 $l+1$ 层的节点 q , 类似第 l 层的节点 j 的(8-20)式、(8-21)式、(8-22)式和(8-26)式, 有:

$$\mathbf{I}_{qk}^{(l+1)} = \sum_{j=1}^{n^{(l)}} w_{jq}^{(l)} O_{jk}^{(l)} \quad (8-31)$$

$$O_{qk}^{(l+1)} = f(\mathbf{I}_{qk}^{(l+1)}) \quad (8-32)$$

$$E_{qk}^{(l+1)} = \frac{1}{2} (O_{qk}^{*(l+1)} - O_{qk}^{(l+1)})^2 \quad (8-33)$$

$$\delta_{qk}^{(l+1)} = \frac{\partial E_{qk}^{(l+1)}}{\partial \mathbf{I}_{qk}^{(l+1)}} \quad (8-34)$$

因此, 可把(8-30)式表示为:

$$\frac{\partial E_{jk}^{(l)}}{\partial O_{jk}^{(l)}} = \sum_{q=1}^{n^{(l+1)}} \frac{\partial E_{qk}^{(l+1)}}{\partial O_{qk}^{(l+1)}} = \sum_{q=1}^{n^{(l+1)}} \frac{\partial E_{qk}^{(l+1)}}{\partial \mathbf{I}_{qk}^{(l+1)}} \frac{\partial \mathbf{I}_{qk}^{(l+1)}}{\partial O_{qk}^{(l)}} = \sum_{q=1}^{n^{(l+1)}} \delta_{qk}^{(l+1)} w_{jq}^{(l)} \quad (8-35)$$

由(8-26)式和(8-35)式可得:

$$\delta_{jk}^{(l)} = f'(\mathbf{I}_{jk}^{(l)}) \sum_{q=1}^{n^{(l+1)}} \delta_{qk}^{(l+1)} w_{jq}^{(l)} \quad (8-36)$$

由(8-24)式、(8-25)式和(8-36)式可得:

$$\Delta w_{ij}^{(l-1)} = -\eta \delta_{jk}^{(l)} O_{ik}^{(l-1)} \quad (8-37)$$

(8-28)式和(8-36)式给出了误差反向传播时, 计算输出层各节点和隐层各节点的 δ 的关系, 由(8-28)式计算出输出层的各节点的 δ 值后, 就可由(8-36)式反向逐层计算出各隐层的所有节点的 δ 值。由各节点的 δ 值, 采用(8-29)式或(8-37)式, 就可计算出各

节点的权值修改量 Δw ，从而对权值进行修改。

2. BP 学习算法

根据误差反向传播的计算分析，可给出下述 BP 学习算法：

1) 输入 N 个学习实例 (X_k, Y_k^*) , $k=1, 2, \dots, N$ 。

2) 建立 BP 网络结构。由学习实例输入向量 X_k 的长度 n 确定网络输入层节点数为 n ，由学习实例输出向量 Y_k^* 的长度 m 确定网络输出层节点数为 m 。确定网络层数 $L \geq 3$ 和各层节点数，第 l 层的节点数为 $n^{(l)}$ ，且 $n^{(1)}=n$, $n^{(L)}=m$ 。定义各层间连接权矩阵，第 l 层连接第 $l+1$ 层的连接权矩阵为 $W^{(l)}=[w_{ij}^{(l)}]_{n^{(l)} \times n^{(l+1)}}$, $l=1, 2, \dots, L-1$ ，初始化各连接权矩阵的元素值。

3) 输入允许误差 ε 和学习率 η ，初始化迭代计算次数 $t=1$ ，学习实例序号 $k=1$ 。

4) 取第 k 个学习实例 (X_k, Y_k^*) , $X_k=(x_{1k}, x_{2k}, \dots, x_{nk})$, $Y_k^*=(y_{1k}^*, y_{2k}^*, \dots, y_{mk}^*)$ 。

5) 由 X_k 进行正向传播计算。

计算输入层各节点的输出为：

$$O_{jk}^{(1)} = f(x_{jk}) \quad j=1, 2, \dots, n$$

逐层计算各层的各节点的输入和输出为：

$$I_{jk}^{(l)} = \sum_{i=1}^{n^{(l-1)}} w_{ij}^{(l-1)} O_{ik}^{(l-1)}$$

$$O_{jk}^{(l)} = f(I_{jk}^{(l)})$$

$$l=2, \dots, L \quad j=1, 2, \dots, n^{(l)}$$

6) 计算输出层（第 L 层）的各输出节点误差为：

$$y_{jk} = O_{jk}^{(L)}$$

$$E_{jk} = \frac{1}{2} (y_{jk}^* - y_{jk})^2 \quad j=1, 2, \dots, m$$

7) 若对 N 个学习实例的任一实例 K 值 $E_{jk} \leq \varepsilon$, $j=1, 2, \dots, m$ ，则学习过程结束；否则，进行误差反向传播修改各连接权矩阵。

8) 误差反向传播计算。

修改第 $L-1$ 层隐层至输出层（第 L 层）的连接权矩阵：

$$\delta_{jk}^{(L)} = -(y_{jk}^* - y_{jk}) f'(I_{jk}^{(L)})$$

$$\Delta w_{ij}^{(L-1)}(t) = \eta \delta_{jk}^{(L)} O_{ik}^{(L)}$$

$$w_{ij}^{(L-1)}(t+1) = w_{ij}^{(L-1)}(t) + \Delta w_{ij}^{(L-1)}(t)$$

$$j=1,2,\cdots,m \quad i=1,2,\cdots,n^{(L-1)}$$

反向逐层修改联结各隐层的联结权矩阵:

$$\delta_{jk}^{(l)} = f'(I_{jk}^{(l)}) \sum_{q=1}^{n^{(l+1)}} \delta_{qk}^{(l+1)} w_{jq}^{(l)}$$

$$\Delta w_{ij}^{(l-1)}(t) = -\eta \delta_{jk}^{(l)} O_{ik}^{(l-1)}$$

$$w_{ij}^{(l-1)}(t+1) = w_{ij}^{(l-1)}(t) + \Delta w_{ij}^{(l-1)}(t)$$

$$l=L-1,\cdots,2,1 \quad j=1,2,\cdots,n^{(l)} \quad i=1,2,\cdots,n^{(l-1)}$$

9) $k=k+1(\bmod N)$, $t=t+1$, 转步骤 4)。

BP 学习算法具有理论依据充分、推导过程严谨、物理概念清晰及通用性好等优点,使它至今仍然是 BP 网络学习的主要算法。但 BP 算法也存在一些不足之处,主要有:

- 1) BP 学习算法的收敛速度很慢,对此,已提出各种改进的方法。
- 2) BP 学习算法采用梯度下降法进行学习,这就有可能出现局部极小问题。当局部极小点产生时, BP 算法所求得解就不是全局最优解。
- 3) 网络隐层的层数及节点个数的优化选择尚无理论上的指导。
- 4) 要求所有的学习实例 (X, Y^*) 的输入 $X = (x_1, x_2, \cdots, x_n)$ 提供的特征数目 n 相同。当有新的学习实例加入时,将会影响对原有实例集的学习效果,或者说将修改网络的权值分布。

为了解决 BP 学习算法陷入局部极小的问题,通常使用一些全局最优化算法与 BP 算法相结合的方法。这里,我们主要针对 BP 学习算法的 1) 和 3) 的不足之处,讨论对 BP 学习算法的改进方法。

3. 提高 BP 学习算法收敛速度的方法

BP 学习算法的主要问题是学习过程的收敛速度比较慢,对于提供的 N 个学习实例需要反复学习,迭代计算次数 t 通常为几百次~几千次,才能使所有输出节点的误差小于指定的允许误差 ε 。为提高 BP 学习算法的收敛速度,可采用下述方法改进 BP 算法。

(1) 变学习率方法

学习率 η 的取值为 $0 < \eta < 1$, η 越大,对权值的修改越大。但是,当 BP 网络的权值分布 W 已接近稳定的权值分布时,过大的 η 将使对权值分布 W 的修改过程发生振荡,难以收敛于稳定的权值分布。若 η 较小,则每次迭代计算对权值的修改较小,修改过程将缓慢地收敛于稳定的权值分布。

变学习率方法是指:在学习过程的初期学习率取较大的值,随着学习过程的进行,逐渐减小学习率。

一种简单易行的变学习率方法可以设定学习率是迭代学习次数 t 的线性函数,使学习率随学习次数 t 的增长而线性递减。假设学习过程的最大迭代学习次数为 $t_{\max}$,第 1 次学习的学习率为 $\eta(1)$,第 $t_{\max}$ 次学习的学习率为 $\eta(t_{\max})$,每经过一次学习后,使学习率变化量为

$\Delta\eta=(\eta(1)-\eta(t_{\max}))/t_{\max}$, 那么, 学习率可表示为:

$$\eta(t) = \eta(1) - \frac{\eta(1) - \eta(t_{\max})}{t_{\max}} \cdot t \quad (8-38)$$

例如, 若估计学习过程收敛的最大迭代学习次数为 $t_{\max}=800$, 设定第 1 次学习的学习率为 $\eta(1)=0.9$, 学习率最小值为 $\eta(t_{\max})=0.1$, 则每次学习后学习率递减 $\Delta\eta=0.001$ 。

也可以在连续地对 N 个学习实例都学习一次后, 才使学习率线性递减 $\Delta\eta$ 。若将对 N 个学习实例依次学习一次称为一遍, 那么, $t_{\max}$ 次学习可分为 $t'_{\max}=\lceil t_{\max}/N \rceil$ 遍学习, 遍学习的学习遍数为 $t'=\lceil t'/N \rceil$, 那么, 学习率可表示为:

$$\eta(t') = \eta(1) - \frac{\eta(1) - \eta(t'_{\max})}{t'_{\max}} \cdot t' \quad (8-39)$$

根据 BP 网络学习过程的收敛特征来设计适当的变学习率函数, 既可加快学习速度, 又不至于引起学习过程的振荡。变学习率方法是 BP 学习算法的主要改进方法之一, 变学习率函数的设计是影响改进效果的决定因素。

(2) 附加冲量方法

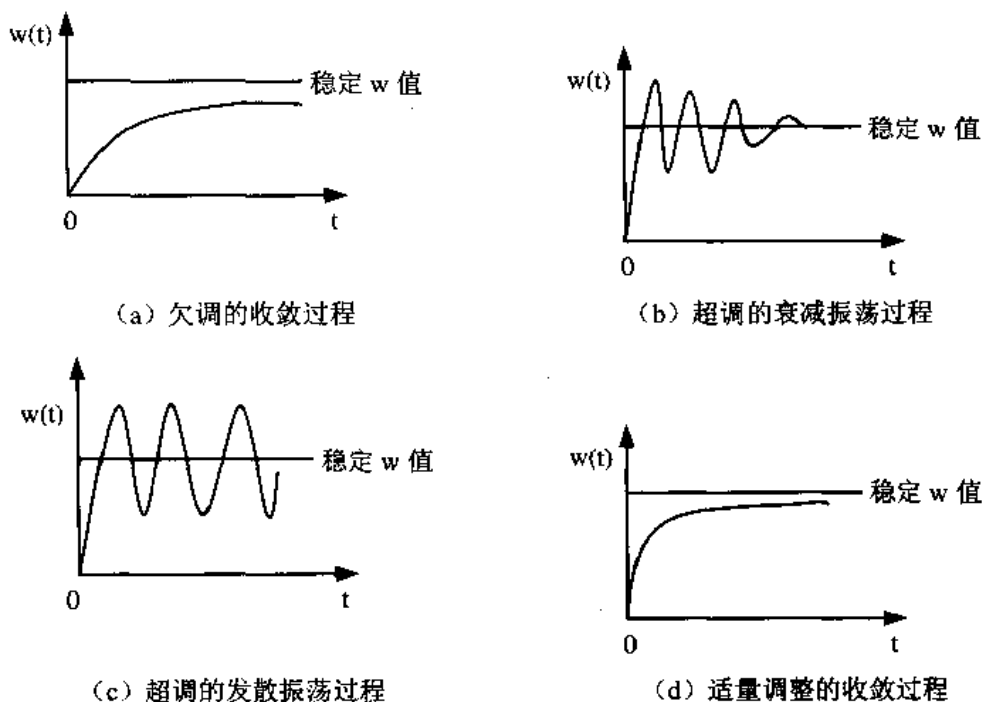
附加冲量法也称为惯性校正法。附加冲量法要求在第 t 次计算连接权的修改量 $\Delta w_{ij}^{(q-1)}(t)$ 时, 附加一项冲量, 冲量为第 $t-1$ 次计算的相应连接权的修改量 $\Delta w_{ij}^{(q-1)}(t-1)$ 的加权, 即将 (8-37) 式修改为:

$$\Delta w_{ij}^{(q-1)}(t) = -\eta \delta_j^{(q)} O_i^{(q-1)} + \beta \Delta w_{ij}^{(q-1)}(t-1) \quad (8-40)$$

其中, β 称为冲量系数, $0 < \beta < 1$ 。

附加冲量使权值的修改增大, 合适的冲量系数 β 可加快学习过程的收敛速度。但是, β 值过大, 也会引起学习过程的振荡。一般取 $0.7 < \beta < 0.9$ 。当然, 也可以使冲量系数 β 在学习过程中是可变的, 那么, 冲量系数函数的设计将直接影响学习过程的改进效果。变学习率和附加冲量都是通过学习过程中, 调整连接权修改量 Δw 来提高学习过程的收敛速度。无论采用什么方法来调整连接权修改量 Δw , 都有一个适量调整的问题。适量调整对学习过程的影响可以用图 8.8 来说明。

如果每次学习的权值调整量 Δw 较小, 即对 Δw 欠调, 则学习过程缓慢地收敛于稳定的权值, 如图 8.8 (a) 所示, 学习过程费时较多。如果每次学习的权值调整量 Δw 过大, 则学习过程是一个衰减振荡过程, 如图 8.8 (b) 所示。这是对 Δw 超调而导致学习过程振荡, 即使振荡过程是逐渐衰减的, 也需要较长的时间才会达到稳定状态。过度的超调将导致学习过程是一个发散振荡过程, 如图 8.8 (c) 所示, 那么, 学习过程不能收敛于稳定状态。理想的学习过程如图 8.8 (d) 所示, 适量调整 Δw , 使学习过程尽可能快地收敛于稳定的权值分布。因此, 需要研究 Δw 的调整方法来获得适当的 Δw 调整量。

图 8.8 调整 ΔW 对学习过程的影响

4. 动态结构 BP 网络学习算法

定理 8.1 给出三层 BP 网络的结构为：输入层为 n 个节点，隐层为 $2n+1$ 个节点，输出层为 m 个节点。在 BP 网络的实际应用中，输入层节点数 n 和输出层节点数 m 是由学习实例的输入模式和输出模式确定的。但是，如何选择隐层数和各隐层节点数，以有利于网络学习过程的收敛、网络计算的逼近程度和效率等网络性能的提高，仍然缺乏理论的支持。一般认为，较多的隐层节点有利于提高网络的收敛性和非线性映射能力。但是，较多的隐层节点显然会增加使用网络的时间与存储空间的开销。因此，希望能改进 BP 学习算法，不仅能获得一个稳定的权值分布，还希望能获得满足误差要求且隐层节点尽可能少的网络结构。为此，我们提出下述动态结构 BP 网络学习算法。

动态结构 BP 网络学习算法是在网络学习过程中，逐步递减隐层节点数，使网络收敛于 $E \leq \varepsilon$ ，且有最少的隐层节点。为此，对 8.3.2 节给出的 BP 学习算法中的步骤 7) 进行下述改进：

计算 N 个学习实例的平均输出误差为：

$$E = (\sum_{k=1}^N \sum_{j=1}^m E_{jk}) / mN$$

若 $E \leq \varepsilon$ ，说明学习过程设定的网络结构已收敛，则从所有的隐层中删去一个节点，但每个隐层的节点数最少为 $2n+1$ ，其中， n 为输入层节点数。即：

若 $n^{(l)} > 2n+1$ ，则 $n^{(l)} = n^{(l)} - 1, l=2, \dots, L-1$

转步骤 2)；

否则, 学习过程结束。

若 $E > \epsilon$, 且 $t \geq t_{\max}$, 则每个隐层增加一个隐层节点, 即:

$$n^{(l)} = n^{(l)} + 1, l = 2, \dots, L-1$$

转步骤 2) ;

若 $E > \epsilon$, 且 $t < t_{\max}$, 则转步骤 8) 进行误差反向传播修改各连接权矩阵。

8.4 人工神经网络的应用

人工神经网络是一种非逻辑、非语言、非静态和非线性的信息处理方法。虽然单个神经元的结构和功能极其简单和有限, 但是, 由大量神经元构成的神经网络系统所能实现的行为却是极其丰富多彩的。需要指出的是, 在人工智能的研究和应用领域中, 基于非线性数值计算的人工神经网络的知识表示与推理和基于符号逻辑的知识表示与推理的理论及方法虽然有

if ($x_1=0$) and ($x_2=0$) then ($y=0$)
 if ($x_1=0$) and ($x_2=1$) then ($y=1$)
 if ($x_1=1$) and ($x_2=0$) then ($y=1$)
 if ($x_1=1$) and ($x_2=1$) then ($y=0$)

这4条规则表示的“异或”逻辑用神经网络来表示,则可表示成如图8.9所示的神经网络。

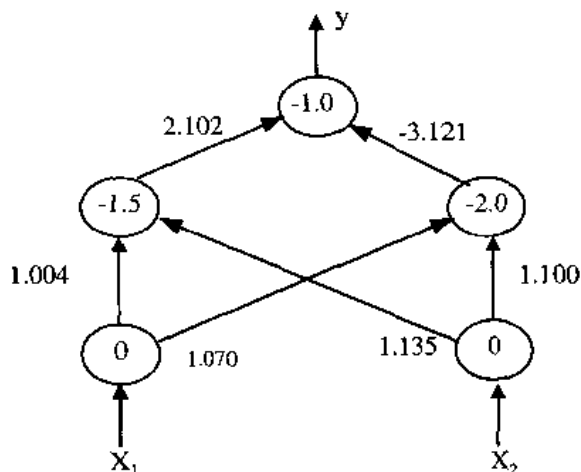


图 8.9 “异或”逻辑的神经网络表示

一个神经网络可以用一个加权有向图表示。加权有向图中的节点连接关系和权值分布可以用一个矩阵来表示,这个矩阵称为邻接权矩阵。一个有 m 个节点的神经网络的邻接权矩阵 $W=[w_{ij}]_{m \times m}$ 的定义为:

$$w_{ij} = \begin{cases} w_{ij} & \text{若节点 } n_i \text{ 有至节点 } n_j \text{ 的邻接且权值为 } w_{ij} \\ 0 & \text{若节点 } n_i \text{ 无至节点 } n_j \text{ 的邻接} \end{cases}$$

图 8.9 所示的“异或”逻辑神经网络的邻接权矩阵表示为:

$$\begin{matrix} & \begin{matrix} n_1 & n_2 & n_3 & n_4 & n_5 \end{matrix} \\ \begin{matrix} n_1 \\ n_2 \\ n_3 \\ n_4 \\ n_5 \end{matrix} & \begin{bmatrix} 0 & 0 & 1.004 & 1.070 & 0 \\ 0 & 0 & 1.135 & 1.100 & 0 \\ 0 & 0 & 0 & 0 & 2.102 \\ 0 & 0 & 0 & 0 & -3.121 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

邻接权矩阵可看成是神经网络专家系统的知识表示形式。对于单向传播的神经网络(例如 BP 神经网络),它学习后所具有的稳定的层间连接权 $W^{(1)}, \dots, W^{(L)}$ 就是它的知识表示。

基于神经网络的知识表示方法有以下优点:

- 1) 能够表示事物的复杂关系,如模糊因果关系。
- 2) 具有统一的知识表示形式,便于知识的组织。知识表示形式的通用性强。
- 3) 便于实现知识的自动获取。

4) 便于网络的知识推理。

2. 神经网络的知识自动获取

知识表示与知识获取是知识推理的基础, 只有获取的知识存于知识库中, 才能进行推理来求解问题。

神经网络是通过实例学习来实现知识自动获取的。在进行知识获取时, 要求领域专家提供学习实例及其相应的期望解, 经过网络自适应学习算法不断修改网络的权值分布, 一旦网络稳定后, 就把领域专家求解该问题的知识和经验 (通过提供的学习实例来表示) 分布到网络的互连结构及权值分布上, 从而得到推理所需要的知识库。所有说, 一个经过学习训练而达到权值分布稳定的神经网络就是神经网络专家系统的知识库。

神经网络专家系统的基本结构如图 8.10 所示。自动知识获取模块的功能主要是输入和组织存储专家提供的学习实例、确定网络结构及调用网络学习算法, 以获得知识库而完成知识获取。

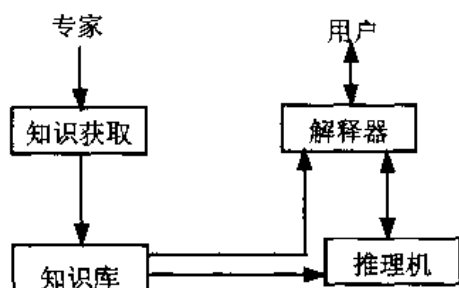


图 8.10 神经网络专家系统的基本结构

知识库可以通过知识获取模块不断更新知识。当表示新知识的新学习实例输入后, 知识获取模块可通过对新学习实例的学习, 自动获得表示包括新知识在内的更多知识与经验的新的权值分布, 从而更新了原来的知识库。

3. 神经网络的知识推理与解释

神经网络专家系统的知识推理与基于符号逻辑的专家系统的推理机制是不同的, 神经网络的推理是一种正向的非线性数值计算过程。

对于一个特定输入模式, 神经网络通过正向传播计算, 产生一个输出模式, 由各输出节点代表的多个逻辑概念被同时并行地计算出来, 因此, 神经网络的推理是一种并行推理机制。

由于神经网络各输出节点的输出是数值的, 因此, 需要一个解释器对输出模式进行解释。解释器的主要任务是将输出的数值向量转换成高层逻辑概念。对于不同的知识领域, 解释器中对输出模式的解释规则也不相同。同样, 解释器中也需要有一套解释规则将用户输入的逻辑概念转换成网络输入模式要求的数值向量。

神经网络的知识推理有以下特点:

1) 同一层神经元的计算是完全可并行的, 层间的传播是逐层串行的。但是, 同一层神经元的个数比神经网络的层数大得多, 因此, 神经网络的知识推理机制很适合实现知识的并

行推理。

2) 在基于规则的推理方法中, 若多条规则的前件均与给定的事实相匹配, 则出现可用规则的冲突问题。为了解决冲突, 需要增加冲突消解的开销, 从而降低了推理速度。神经网络的推理过程不需要搜索匹配规则, 也不存在可用规则冲突问题。基于神经网络的专家系统的推理时间开销同基于规则的专家系统的推理时间的开销比较, 前者要少得多。

3) 基于神经网络的专家系统的解释器可以有一套输入模式解释规则和一套输出模式解释规则, 分别将用户输入的符号逻辑概念转换成网络的数值输入模式和把网络的数值输出模式转换成输出的符号逻辑概念提供给用户。由于基于神经网络的专家系统的知识表示是隐式的、非局部的, 因此, 神经网络专家系统的解释器难以像基于规则的专家系统那样, 通过记录推理过程使用的规则链, 来向用户提供 why 询问和 how 询问的解释。

4. 组合神经网络专家系统

由于一个神经网络专家系统可以用一个加权有向图表示, 或者用邻接权矩阵表示, 因此, 可以方便地把同一个知识领域的几个独立的神经网络专家系统组合成一个规模更大的神经网络专家系统, 只要知道这几个神经网络之间有哪些节点有连接关系, 把它们连接起来就可以组合成一个更大的神经网络。节点之间是否有连接关系, 可根据专家知识来判断两个节点之间是否有模糊的因果关系, 至于反映因果关系的模糊程度的连接权值可以通过学习来获得。

由于能为组合神经网络专家系统提供的学习实例更多, 因此, 经过学习训练获得的知识库更可靠。相反, 对于基于规则的几个专家系统组合成一个更大的专家系统, 由于各自知识库中的规则是各自自行确定的, 因此, 把它们组合成一个知识库时, 规则的冗余程度和不一致性都较大, 各自的规则越多, 组合的知识库越不可靠。

8.4.2 基于神经网络的模糊分类器

模糊分类的基本思想是: 根据对象的若干特征值计算出这个对象属于哪一类的隶属度值, 从而确定这个对象属于隶属度值最高的那一类。

我们用下面的一个例子来说明神经网络在模糊分类中的应用。

例 8.1 根据分类对象的两个特征 $X=(x_1, x_2)$, 把对象分为 R_1 和 R_2 两类, 请建立基于神经网络的模糊分类器。提供的训练实例如表 8.1 所示, 共有 20 个训练实例, x_{1k} 和 x_{2k} 分别是实例 k 的两个特征值, y_{1k}^* 和 y_{2k}^* 分别是实例 k 属于类 R_1 和 R_2 的隶属度值, 若 $y_{1k}^*=1, y_{2k}^*=0$, 则实例 k 肯定属于 R_1 类; 若 $y_{1k}^*=0, y_{2k}^*=1$, 则实例 k 肯定属于 R_2 类。

表 8.1 模糊分类器的训练实例

K	1	2	3	4	5	6	7	8	9	10
x_{1k}	0.05	0.09	0.12	0.15	0.20	0.75	0.80	0.82	0.90	0.95
x_{2k}	0.02	0.11	0.20	0.22	0.25	0.75	0.83	0.80	0.89	0.89
y_{1k}^*	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0

续表

y_{2k}^*	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
K	11	12	13	14	15	16	17	18	19	20
x_{1k}	0.09	0.10	0.14	0.18	0.22	0.77	0.79	0.84	0.94	0.98
x_{2k}	0.04	0.10	0.21	0.24	0.28	0.78	0.81	0.82	0.93	0.99
y_{1k}^*	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
y_{2k}^*	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0

解 我们选择一个4层的BP神经网络来建立模糊分类器, BP网络的结构为 $2 \times 3 \times 3 \times 2$, 即输入层和输出层都是2个节点, 2个隐层都是3个节点, 如图8.11所示。输入层的2个节点分别输入对象的2个特征值 x_1 和 x_2 , 输入层节点的传递函数采用简单的线性函数 $f(x_j)=x_j$, $j=1,2$ 。输出层的2个节点分别输出对象分类的隶属度值 y_1 和 y_2 。隐层和输出层节点的传递函数采用 Sigmoid 函数 $f(x)=\frac{1}{1+e^{-x}}$, 由(8-20)式和(8-21)式可知, 第2、3、4层的节点 j 的输入 $I_j^{(l)}$ 和输出 $O_j^{(l)}$ 分别为:

$$I_j^{(l)} = \sum_{i=1}^{n^{(l-1)}} w_{ij}^{(l-1)} O_i^{(l-1)}$$

$$O_j^{(l)} = f(I_j^{(l)}) = \frac{1}{1 + \exp(-\sum_i w_{ij}^{(l-1)} O_i^{(l-1)})}$$

当 $l=2$ 时, 有 $n^{(l-1)}=n^{(1)}=2$, $j=1,2,3$ 。

当 $l=3$ 时, 有 $n^{(l-1)}=n^{(2)}=3$, $j=1,2,3$ 。

当 $l=4$ 时, 有 $n^{(l-1)}=n^{(3)}=3$, $j=1,2, y_1=O_1^{(4)}, y_2=O_2^{(4)}$ 。

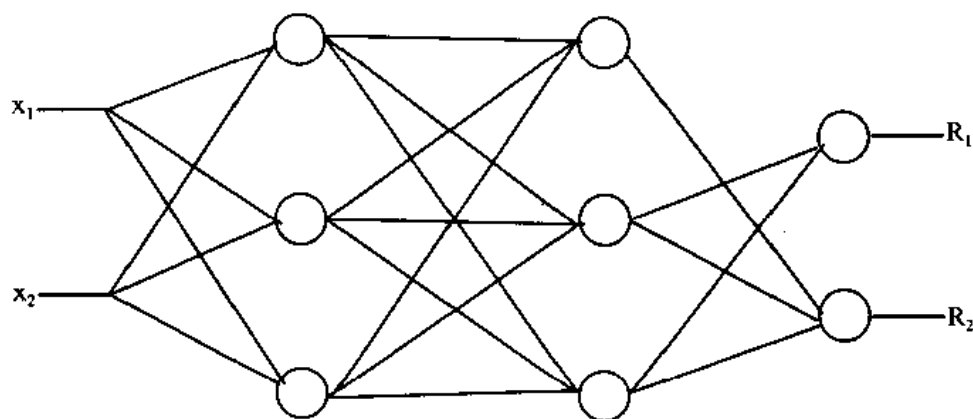


图 8.11 神经网络结构

图 8.11 所示的 BP 网络有 3 个连接权矩阵, 分别是 $W^{(1)}=[w_{ij}^{(1)}]_{2 \times 3}$, $W^{(2)}=[w_{ij}^{(2)}]_{3 \times 3}$,

$W^{(3)} = [w_{ij}^{(3)}]_{3 \times 2}$, 对连接权矩阵初始化, 设初始化随机值分别为:

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{bmatrix} = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.6 & 0.2 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} \end{bmatrix} = \begin{bmatrix} 0.10 & 0.55 & 0.35 \\ 0.20 & 0.45 & 0.35 \\ 0.25 & 0.15 & 0.60 \end{bmatrix}$$

$$W^{(3)} = \begin{bmatrix} w_{11}^{(3)} & w_{12}^{(3)} \\ w_{21}^{(3)} & w_{22}^{(3)} \\ w_{31}^{(3)} & w_{32}^{(3)} \end{bmatrix} = \begin{bmatrix} 0.30 & 0.35 \\ 0.35 & 0.25 \\ 0.45 & 0.30 \end{bmatrix}$$

取第一个对象 $k=1$ 的特征向量 $X_1=(x_1, x_2)=(0.05, 0.02)$ 进行第 1 次迭代计算。

(1) 输入正向传播计算

第 1 层 (输入层) 的输出为:

$$O_1^{(1)} = f(x_1) = x_1 = 0.05$$

$$O_2^{(1)} = f(x_2) = x_2 = 0.02$$

第 2 层的输出为:

$$\begin{aligned} O_1^{(2)} &= 1/[1 + \exp(-\sum_{i=1}^2 w_{i1}^{(1)} O_i^{(1)})] \\ &= 1/[1 + \exp(-(w_{11}^{(1)} O_1^{(1)} + w_{21}^{(1)} O_2^{(1)}))] \\ &= 1/[1 + \exp(-(0.5 \times 0.05 + 0.2 \times 0.02))] \\ &= 0.507249 \\ O_2^{(2)} &= 1/[1 + \exp(-\sum_{i=1}^2 w_{i2}^{(1)} O_i^{(1)})] \\ &= 1/[1 + \exp(-(w_{12}^{(1)} O_1^{(1)} + w_{22}^{(1)} O_2^{(1)}))] \\ &= 1/[1 + \exp(-(0.4 \times 0.05 + 0.6 \times 0.02))] \\ &= 0.507999 \\ O_3^{(2)} &= 1/[1 + \exp(-\sum_{i=1}^2 w_{i3}^{(1)} O_i^{(1)})] \\ &= 1/[1 + \exp(-(w_{13}^{(1)} O_1^{(1)} + w_{23}^{(1)} O_2^{(1)}))] \\ &= 1/[1 + \exp(-(0.1 \times 0.05 + 0.2 \times 0.02))] \\ &= 0.502250 \end{aligned}$$

第 3 层的输出为:

$$\begin{aligned} O_1^{(3)} &= 1/[1 + \exp(-\sum_{i=1}^3 w_{i1}^{(2)} O_i^{(2)})] \\ &= 1/[1 + \exp(-(w_{11}^{(2)} O_1^{(2)} + w_{21}^{(2)} O_2^{(2)} + w_{31}^{(2)} O_3^{(2)}))] \\ &= 1/[1 + \exp(-(0.10 \times 0.507249 + 0.20 \times 0.507999 + 0.25 \times 0.502250))] \end{aligned}$$

$$=0.569028$$

$$O_2^{(3)} = 1/[1 + \exp(-\sum_{i=1}^3 w_{i2}^{(2)} O_i^{(2)})]$$

$$= 1/[1 + \exp(-(w_{12}^{(2)} O_1^{(2)} + w_{22}^{(2)} O_2^{(2)} + w_{32}^{(2)} O_3^{(2)}))]$$

$$= 1/[1 + \exp(-(0.55 \times 0.507249 + 0.45 \times 0.507999 + 0.15 \times 0.502250))]$$

$$=0.641740$$

$$O_3^{(3)} = 1/[1 + \exp(-\sum_{i=1}^3 w_{i3}^{(2)} O_i^{(2)})]$$

$$= 1/[1 + \exp(-(w_{13}^{(2)} O_1^{(2)} + w_{23}^{(2)} O_2^{(2)} + w_{33}^{(2)} O_3^{(2)}))]$$

$$= 1/[1 + \exp(-(0.35 \times 0.507249 + 0.35 \times 0.507999 + 0.60 \times 0.502250))]$$

$$=0.658516$$

第4层（输出层）的输出为：

$$O_1^{(4)} = 1/[1 + \exp(-\sum_{i=1}^3 w_{i1}^{(3)} O_i^{(3)})]$$

$$= 1/[1 + \exp(-(w_{11}^{(3)} O_1^{(3)} + w_{21}^{(3)} O_2^{(3)} + w_{31}^{(3)} O_3^{(3)}))]$$

$$= 1/[1 + \exp(-(0.30 \times 0.569028 + 0.35 \times 0.641740 + 0.45 \times 0.658516))]$$

$$=0.666334$$

$$O_2^{(4)} = 1/[1 + \exp(-\sum_{i=1}^3 w_{i2}^{(3)} O_i^{(3)})]$$

$$= 1/[1 + \exp(-(w_{12}^{(3)} O_1^{(3)} + w_{22}^{(3)} O_2^{(3)} + w_{32}^{(3)} O_3^{(3)}))]$$

$$= 1/[1 + \exp(-(0.35 \times 0.569028 + 0.25 \times 0.641740 + 0.30 \times 0.658516))]$$

$$=0.635793$$

由此得出第一个对象实例的网络计算隶属度值为 $y_1 = O_1^{(4)} = 0.666334$, $y_2 = O_2^{(4)} = 0.635793$ 。

与该实例给出的隶属度值 $y_1^* = 1.0$, $y_2^* = 0.0$ 进行比较, 计算误差。误差函数定义为:

$$E_j = y_j - y_j^* \quad j=1,2$$

第一个实例的输出误差为:

$$E_1^{(4)} = y_1 - y_1^* = 0.666334 - 1.0 = -0.333666$$

$$E_2^{(4)} = y_2 - y_2^* = 0.635793 - 0.0 = 0.635793$$

设定模糊分类器的允许误差为 $\varepsilon = 0.01$, 因为 $|E_j| > \varepsilon$, $j=1,2$, 因此, 由误差反向传播来修改连接权值。

(2) 误差反向传播计算

为简化误差反向传播的计算, 我们简化传递函数 f 的导数 f' 的计算。对本例的误差反向传播修改权值的计算使用下述计算公式:

$$E_i^{(l)} = O_i^{(l)} (1 - O_i^{(l)}) \sum_{j=1}^{n^{(l+1)}} w_{ij}^{(l)} E_j^{(l+1)}$$

$$\Delta w_{ij}^{(l)} = \eta E_j^{(l+1)} O_i^{(l)}$$

$$w_{ij}^{(l)} = w_{ij}^{(l)} + \Delta w_{ij}^{(l)} = w_{ij}^{(l)} + \eta E_j^{(l+1)} O_i^{(l)}$$

对 $l=3$, $n^{(l)}=n^{(3)}=3$, 故 $i=1,2,3$; $n^{(l+1)}=n^{(4)}=2$, 故 $j=1,2$ 。

对 $l=2$, $n^{(l)}=n^{(2)}=3$, 故 $i=1,2,3$; $n^{(l+1)}=n^{(3)}=3$, 故 $j=1,2,3$ 。

对 $l=1$, $n^{(l)}=n^{(1)}=2$, 故 $i=1,2$; $n^{(l+1)}=n^{(2)}=3$, 故 $j=1,2,3$ 。

取学习率 $\eta=0.3$ 。

1) 由第4层的误差 $E_1^{(4)}$ 和 $E_2^{(4)}$, 更新第3层连接第4层的权值分布 $W^{(3)}=[w_{ij}^{(3)}]_{3 \times 2}$ 为:

$$\begin{aligned} w_{11}^{(3)} &= w_{11}^{(3)} + \eta E_1^{(4)} O_1^{(3)} \\ &= 0.30 + 0.3 \times (-0.333666) \times 0.569028 = 0.243040 \\ w_{21}^{(3)} &= w_{21}^{(3)} + \eta E_1^{(4)} O_2^{(3)} \\ &= 0.35 + 0.3 \times (-0.333666) \times 0.641740 = 0.285762 \\ w_{31}^{(3)} &= w_{31}^{(3)} + \eta E_1^{(4)} O_3^{(3)} \\ &= 0.45 + 0.3 \times (-0.333666) \times 0.658516 = 0.384083 \\ w_{12}^{(3)} &= w_{12}^{(3)} + \eta E_2^{(4)} O_1^{(3)} \\ &= 0.35 + 0.3 \times 0.635793 \times 0.569028 = 0.458535 \\ w_{22}^{(3)} &= w_{22}^{(3)} + \eta E_2^{(4)} O_2^{(3)} \\ &= 0.25 + 0.3 \times 0.635793 \times 0.641740 = 0.372404 \\ w_{32}^{(3)} &= w_{32}^{(3)} + \eta E_2^{(4)} O_3^{(3)} \\ &= 0.30 + 0.3 \times 0.635793 \times 0.658516 = 0.425604 \end{aligned}$$

2) 将误差 $E_1^{(4)}$ 和 $E_2^{(4)}$ 按修改前的权值分布 $W^{(3)}$ 反向传播至第3层, 计算第3层的 $n^{(3)}=3$ 个节点的输出误差:

$$\begin{aligned} E_1^{(3)} &= O_1^{(3)}(1 - O_1^{(3)}) \sum_{j=1}^2 w_{1j}^{(3)} E_j^{(4)} \\ &= O_1^{(3)}(1 - O_1^{(3)})(w_{11}^{(3)} E_1^{(4)} + w_{12}^{(3)} E_2^{(4)}) \\ &= 0.569028 \times (1 - 0.569028) \times [0.30 \times (-0.333666) + 0.35 \times 0.635793] \\ &= 0.030024 \\ E_2^{(3)} &= O_2^{(3)}(1 - O_2^{(3)}) \sum_{j=1}^2 w_{2j}^{(3)} E_j^{(4)} \\ &= O_2^{(3)}(1 - O_2^{(3)})(w_{21}^{(3)} E_1^{(4)} + w_{22}^{(3)} E_2^{(4)}) \\ &= 0.641740 \times (1 - 0.641740) \times [0.35 \times (-0.333666) + 0.25 \times 0.635793] \\ &= 0.009694 \\ E_3^{(3)} &= O_3^{(3)}(1 - O_3^{(3)}) \sum_{j=1}^2 w_{3j}^{(3)} E_j^{(4)} \\ &= O_3^{(3)}(1 - O_3^{(3)})(w_{31}^{(3)} E_1^{(4)} + w_{32}^{(3)} E_2^{(4)}) \\ &= 0.658516 \times (1 - 0.658516) \times [0.45 \times (-0.333666) + 0.30 \times 0.635793] \end{aligned}$$

$$=0.009127$$

由第3层的误差 $E_1^{(3)}$ 、 $E_2^{(3)}$ 和 $E_3^{(3)}$, 更新第2层连接第3层的权值分布 $W^{(2)}=[w_{ij}^{(2)}]_{3 \times 3}$ 为:

$$w_{11}^{(2)} = w_{11}^{(2)} + \eta E_1^{(3)} O_1^{(2)}$$

$$=0.10+0.3 \times 0.030024 \times 0.507249=0.104968$$

$$w_{21}^{(2)} = w_{21}^{(2)} + \eta E_1^{(3)} O_2^{(2)}$$

$$=0.20+0.3 \times 0.030024 \times 0.507999=0.204576$$

$$w_{31}^{(2)} = w_{31}^{(2)} + \eta E_1^{(3)} O_3^{(2)}$$

$$=0.25+0.3 \times 0.030024 \times 0.502250=0.254524$$

$$w_{12}^{(2)} = w_{12}^{(2)} + \eta E_2^{(3)} O_1^{(2)}$$

$$=0.55+0.3 \times 0.009694 \times 0.507249=0.551475$$

$$w_{22}^{(2)} = w_{22}^{(2)} + \eta E_2^{(3)} O_2^{(2)}$$

$$=0.45+0.3 \times 0.009694 \times 0.507999=0.451477$$

$$w_{32}^{(2)} = w_{32}^{(2)} + \eta E_2^{(3)} O_3^{(2)}$$

$$=0.15+0.3 \times 0.009694 \times 0.502250=0.151461$$

$$w_{13}^{(2)} = w_{13}^{(2)} + \eta E_3^{(3)} O_1^{(2)}$$

$$=0.35+0.3 \times 0.009127 \times 0.507249=0.351389$$

$$w_{23}^{(2)} = w_{23}^{(2)} + \eta E_3^{(3)} O_2^{(2)}$$

$$=0.35+0.3 \times 0.009127 \times 0.507999=0.351391$$

$$w_{33}^{(2)} = w_{33}^{(2)} + \eta E_3^{(3)} O_3^{(2)}$$

$$=0.60+0.3 \times 0.009127 \times 0.502250=0.601375$$

3) 将误差 $E_1^{(3)}$ 、 $E_2^{(3)}$ 和 $E_3^{(3)}$ 按修改前的权值分布 $W^{(2)}$ 反向传播至第2层, 计算第2层的 $n^{(2)}=3$ 个节点的输出误差:

$$E_1^{(2)} = O_1^{(2)}(1 - O_1^{(2)}) \sum_{j=1}^3 w_{1j}^{(2)} E_j^{(3)}$$

$$= O_1^{(2)}(1 - O_1^{(2)})(w_{11}^{(2)} E_1^{(3)} + w_{12}^{(2)} E_2^{(3)} + w_{13}^{(2)} E_3^{(3)})$$

$$=0.507249 \times (1-0.507249) \times (0.10 \times 0.030024 + 0.55 \times 0.009694 + 0.35 \times 0.009127) = 0.002882$$

$$E_2^{(2)} = O_2^{(2)}(1 - O_2^{(2)}) \sum_{j=1}^3 w_{2j}^{(2)} E_j^{(3)}$$

$$= O_2^{(2)}(1 - O_2^{(2)})(w_{21}^{(2)} E_1^{(3)} + w_{22}^{(2)} E_2^{(3)} + w_{23}^{(2)} E_3^{(3)})$$

$$=0.507999 \times (1-0.507999) \times (0.20 \times 0.030024 + 0.45 \times 0.009694 + 0.35 \times 0.009127) = 0.003390$$

$$E_3^{(2)} = O_3^{(2)}(1 - O_3^{(2)}) \sum_{j=1}^3 w_{3j}^{(2)} E_j^{(3)}$$

$$= O_3^{(2)}(1 - O_3^{(2)})(w_{31}^{(2)} E_1^{(3)} + w_{32}^{(2)} E_2^{(3)} + w_{33}^{(2)} E_3^{(3)})$$

$$=0.502250 \times (1-0.502250) \times (0.25 \times 0.030024 + 0.15 \times 0.009694 + 0.60 \times 0.009127) = 0.003390$$

$$+0.60 \times 0.009127) = 0.003609$$

由第2层的误差 $E_1^{(2)}$ 、 $E_2^{(2)}$ 和 $E_3^{(2)}$, 更新第1层连接第2层的权值分布 $W^{(1)} = [w_{ij}^{(1)}]_{2 \times 3}$ 为:

$$\begin{aligned} w_{11}^{(1)} &= w_{11}^{(1)} + \eta E_1^{(2)} O_1^{(1)} = w_{11}^{(1)} + \eta E_1^{(2)} x_1 \\ &= 0.50 + 0.3 \times 0.002882 \times 0.05 = 0.500043 \\ w_{21}^{(1)} &= w_{21}^{(1)} + \eta E_1^{(2)} O_2^{(1)} = w_{21}^{(1)} + \eta E_1^{(2)} x_2 \\ &= 0.20 + 0.3 \times 0.002882 \times 0.02 = 0.200017 \\ w_{12}^{(1)} &= w_{12}^{(1)} + \eta E_2^{(2)} O_1^{(1)} = w_{12}^{(1)} + \eta E_2^{(2)} x_1 \\ &= 0.40 + 0.3 \times 0.003390 \times 0.05 = 0.400051 \\ w_{22}^{(1)} &= w_{22}^{(1)} + \eta E_2^{(2)} O_2^{(1)} = w_{22}^{(1)} + \eta E_2^{(2)} x_2 \\ &= 0.60 + 0.3 \times 0.003390 \times 0.02 = 0.600020 \\ w_{13}^{(1)} &= w_{13}^{(1)} + \eta E_3^{(2)} O_1^{(1)} = w_{13}^{(1)} + \eta E_3^{(2)} x_1 \\ &= 0.10 + 0.3 \times 0.003609 \times 0.05 = 0.100054 \\ w_{23}^{(1)} &= w_{23}^{(1)} + \eta E_3^{(2)} O_2^{(1)} = w_{23}^{(1)} + \eta E_3^{(2)} x_2 \\ &= 0.20 + 0.3 \times 0.003609 \times 0.02 = 0.200022 \end{aligned}$$

至此, 已完成第一次学习对网络权值分布的修改。取第2个训练实例对网络进行第2次学习训练。当对20个训练实例都学习一遍之后, 若网络输出误差仍有 $|E_j| > \varepsilon$, $j=1,2$, 则再取第1个实例对网络进行第21次学习训练。反复迭代, 直至输出误差达到期望要求。这时, 满足期望要求的模糊分类器就建成了。

任一分类对象由模糊分类器进行分类的过程就是该分类对象的特征向量 $X=(x_1, x_2)$ 在分类器神经网络中的正向传播计算过程, 按正向传播计算公式逐层计算, 就可在输出层得到分类结果的输出向量 $Y=(y_1, y_2)$ 。

由模糊分类器的解释器对输出向量进行模糊解释。对象 x 的隶属度 $\mu(x)$ 的值为 $0 \leq \mu(x) \leq 1$, $\mu(x)$ 与模糊语言修饰词的简单对应关系如表8.2所示。若某一个对象 x 的分类结果向量是: $y_1=0.872312$, $y_2=0.122810$, 那么, 由解释器得出的解释是: 对象 x “几乎是”属于 R_1 类, “稍稍像是”属于 R_2 类。

表 8.2 模糊修饰词的隶属度

修饰词	隶属度	修饰词	隶属度
肯定是	1	比较像是	0.4
几乎是	0.9	有些像是	0.3
极是	0.8	有点像是	0.2
很是	0.7	稍稍像是	0.1
相当是	0.6	肯定不是	0
差不多是	0.5		

8.4.3 神经网络在预测中的应用

我们通过应用实例来说明神经网络在预测中的应用，并讨论不同结构的 BP 网络及随机初始化对预测结果的影响。

例 8.2 世界干散货海运量从 1980 年~1996 年的各年海运量如表 8.3 所示，建立基于神经网络的世界干散货海运量时序预测模型。

表 8.3 1986 年~1996 年世界干散货海运量（单位：百万吨）

年度	运量	年度	运量	年度	运量
1986	1399	1990	1588	1994	1685
1987	1467	1991	1622	1995	1789
1988	1567	1992	1611	1996	1790
1989	1595	1993	1615		

解 首先需要确定预测模型的神经网络的结构。我们可以采用 n 输入单输出的三层 BP 神经网络结构。模型建立后，一次使用可在单输出节点得到下一年度的运量。输入层节点数 n 与学习实例的输入模式的选择有关。如果选择连续 5 年的各年运量作为输入模式，则输入节点数 n=5，第 6 年的运量为实例的输出，那么，由表 8.3 给出的 1986 年~1996 年的各年运量，可以获得 6 个学习实例，第 1 个实例的输入向量为 $X_1=(1399,1467,1567,1595,1588)$ ，输出为 $y^*_1=1622$ ；第 2 个实例的输入向量为 $X_2=(1467,1567,1595,1588,1622)$ ，输出为 $y^*_2=1611$ ；…，第 6 个实例的输入向量为 $X_6=(1622,1611,1615,1685,1789)$ ，输出为 $y^*_6=1790$ 。当然，也可以选择输入模式为连续 6 年的运量，则 n=6，第 7 年的运量为实例的输出，那么，由 1986 年~1996 年的各年运量，只能获得 5 个学习实例。若选择 n=7,则只能获得 4 个学习实例。由于本例可提供的学习实例数量较少，且只有一个隐层，故设置较多的隐层节点，设置隐层节点数为 3n，所以，预测模型的 BP 网络结构为 $n \times 3n \times 1$ 。

输入节点的传递函数采用线性函数 $f(x)=x$ 。隐层节点和输出节点的传递函数采用 Sigmoid 型的双曲正切函数 $f(x)=th(x)=\frac{e^x - e^{-x}}{e^x + e^{-x}}$ 。双曲正切函数 $th(x)$ 的函数图形是关于原点对称的，当输入 $x=0$ 时，输出 $f(x)=0$ ，与预测要求一致。误差反向传播过程需要使用传递函数的导数为 $f'(x) = th'(x) = (\frac{2}{e^x + e^{-x}})^2$ 。

证明映射网络存在性的定理 8.1 和定理 8.2 说明对连续函数 $f:[0,1]^n \rightarrow R^m$ ，可用一个三层的 BP 网络精确地逼近 f。由于网络的输入量取值应有 $x_i \in [0,1]$ ， $i=1,2,\cdots,n$ ，所以，对本例学习实例的数据要进行归一化。只需要对所有的输入数据除以 10^4 ，然后对网络计算的输出结果乘以 10^4 即可。

由表 8.3 得出网络结构为 $5 \times 15 \times 1$ 的 6 个归一化后的学习实例如表 8.4 所示。

表 8.4 $5 \times 15 \times 1$ 的归一化学习实例

k	x_{1k}	x_{2k}	x_{3k}	x_{4k}	x_{5k}	y_k
1	0.1399	0.1467	0.1567	0.1595	0.1588	0.1622
2	0.1467	0.1567	0.1595	0.1588	0.1622	0.1611
3	0.1567	0.1595	0.1588	0.1622	0.1611	0.1615
4	0.1595	0.1588	0.1622	0.1611	0.1615	0.1685
5	0.1588	0.1622	0.1611	0.1615	0.1685	0.1789
6	0.1622	0.1611	0.1615	0.1685	0.1789	0.1790

网络的输入节点数为 n , 隐层节点数为 $m=3n$, 故输入层至隐层的连接权矩阵为 $W=[w_{ij}]_{n \times m}$ 。隐层各节点的阈值表示为阈值向量 $\theta=(\theta_j)_m$ 。输出层只有一个输出节点, 故隐层至输出层的连接权表示为权向量 $U=(u_j)_m$ 。输出节点的阈值为 r 。根据 BP 学习算法, 对实例 k 的学习计算过程为:

1) 计算隐层各节点输入量

$$a_{jk} = \sum_{i=1}^n w_{ij} x_{ik} - \theta_j \quad j=1,2,\dots,m$$

2) 计算隐层各节点输出量

$$b_{jk} = \text{th}(a_{jk}) \quad j=1,2,\dots,m$$

3) 计算输出节点输入量

$$c_k = \sum_{j=1}^m u_j b_{jk} - r$$

4) 计算输出节点输出量

$$y_k = \text{th}(c_k)$$

5) 计算隐层 m 个节点至输出节点的连接权的修正量

$$\delta_k = -(y_k^* - y_k) f'(c_k) = -(y_k^* - y_k) \text{th}'(c_k)$$

$$\Delta u_j = \eta \delta_k b_{jk} \quad j=1,2,\dots,m$$

6) 计算输入层 n 个节点至隐层 m 个节点的连接权的修正量

$$\delta_{jk} = f'(a_{jk}) \sum_{i=1}^m u_i \delta_k = \text{th}'(a_{jk}) \sum_{i=1}^m u_i \delta_k \quad j=1,2,\dots,m$$

$$\Delta w_{ij} = \eta \delta_{jk} x_{ik} \quad i=1,2,\dots,n \quad j=1,2,\dots,m$$

7) 修改权值分布

$$w_{ij} = w_{ij} + \Delta w_{ij} \quad i=1,2,\dots,n \quad j=1,2,\dots,m$$

$$u_j = u_j + \Delta u_j \quad j=1,2,\dots,m$$

其中, 步骤 1)~4) 是输入正向传播过程, 步骤 5)~7) 是误差反向传播过程。

对由 1986 年~1996 年运量组成的 6 个实例 (如表 8.4 所示) 学习 $t_{\max}$ 次后, 得到的 BP 网络就是一个世界干散货海运量预测模型。用 1992 年~1996 年的运量作为输入模式输入预测模型, 经过由步骤 1)~4) 组成的正向传播计算, 模型的输出值乘以 10^4 就是 1997 年运量的预测值。类似地, 用 1993 年~1997 年的运量作为输入模式就可得到 1998 年年运量的预测值。也就是说, 每次预测都是把上一年度的预测值作为本年度预测的输入模式的 x_0 的值, 按时序进行预测。

以本例为例, 对 BP 网络进行下述有关讨论。

(1) 预测模型的知识更新

如果学习实例越多, 或增加新的学习实例更新网络记忆的知识, 那么, 时序预测模型获得的知识越可靠, 预测精度越高。

本例只提供了 6 个实例, 我们可采取知识更新的方法来提高模型预测精度。具体方法是: 每次得到一个预测值后, 把它作为一个新的学习实例的期望输出 y^* , 把这个新实例增加到原来的实例集中, 重新训练网络, 用知识更新后的网络再预测下一年的预测值。当然, 每作一次预测就重新用新实例集训练网络, 然后才作下一次预测, 这样所需的时间的开销会成倍增加, 但是, 会使预测精度提高很大。

(2) 网络权值和阈值初始化对输出的影响

在网络的学习训练过程中, 连接权值和阈值初始化是随机设置的。随机设置的初始化权值和阈值对网络学习过程是有影响的。随机初始化的权值和阈值不同时, 对指定的允许误差 ϵ , 有的学习过程能在设定的最大学习次数 $t_{\max}$ 范围内收敛, 但有的学习过程在学习了 $t_{\max}$ 次之后不能达到期望的允许误差。

由表 8.4 给出的实例集, 对 $5 \times 15 \times 1$ 的 BP 网络重新独立地训练 6 次, 每次训练都对权值和阈值重新随机初始化, 得出 6 个学习实例的输出值如表 8.5 中的一列所示 (已乘以 10^4)。每一列有 6 个输出值 $y_1 \sim y_6$, 分别是实例集的 6 个实例的输出值, $\bar{y}_k$ 是实例 k 的 6 个输出值 (同一行的值) $y_k(1) \sim y_k(6)$ 的算术平均值。

表 8.5 $5 \times 15 \times 1$ 的 BP 网络输出结果

k	$y_k(1)$	$y_k(2)$	$y_k(3)$	$y_k(4)$	$y_k(5)$	$y_k(6)$	$\bar{y}_k$
1	1556.36	1552.96	1664.99	1550.46	1538.47	1581.94	1547.20
2	1610.40	1606.85	1610.55	1603.05	1599.24	1631.34	1610.24
3	1629.54	1622.75	1623.61	1640.05	1636.15	1619.04	1628.52
4	1686.96	1691.98	1700.33	1673.83	1665.13	1704.25	1687.08
5	1707.27	1706.67	1715.38	1698.63	1693.20	1728.24	1708.23
6	1770.16	1758.84	1772.07	1760.41	1752.60	1790.05	1767.36

由表 8.5 可见,当设定有限次学习次数 $t_{\max}$ 作为算法终止条件时,权值和阈值的初始化对输出值有一定的影响,表中同一行的输出值 $y_k(1) \sim y_k(6)$ 之间的差异就反映了随机初始化的影响。

(3) 网络结构对输出的影响

网络的结构不同,对网络的输出也有影响。BP 网络结构分别采用 $5 \times 15 \times 1$ 、 $6 \times 18 \times 1$ 和 $7 \times 21 \times 1$ 三种结构,对由 1986 年~1996 年运量组成的学习实例训练后的输出值如表 8.6 所示。对 $5 \times 15 \times 1$ 网络,可提供 6 个实例;对 $6 \times 18 \times 1$ 网络,由于一个实例的输入模式需要连续的 6 年运量,所以只能提供 5 个实例;对 $7 \times 21 \times 1$ 网络,只能提供 4 个实例。

表 8.6 三种不同结构的 BP 网络的输出值与误差

年度	实际运量 y_k	5×15×1 输出值		6×18×1 输出值		7×21×1 输出值	
		y_k	$y_k - \hat{y}_k$	y_k	$y_k - \hat{y}_k$	y_k	$y_k - \hat{y}_k$
1991	1622	1556.36	-65.64				
1992	1611	1610.40	-0.60	1612.92	1.92		
1993	1615	1629.54	14.54	1613.99	-1.01	1612.25	-2.75
1994	1685	1686.96	1.96	1687.10	2.10	1693.69	8.69
1995	1789	1707.27	-81.73	1789.36	0.36	1788.35	-0.65
1996	1790	1770.16	-19.84	1790.38	0.38	1794.05	4.05

由表 8.6 可见, $6 \times 18 \times 1$ 网络输出的误差平方和 $E = \frac{1}{N} \sum_{k=1}^N (y_k - \hat{y}_k)^2$ 最小,采 $6 \times 18 \times 1$ 网络结构预测的拟合精度最好, $7 \times 21 \times 1$ 网络结构的预测精度次之, $5 \times 15 \times 1$ 网络结构的预测精度最差。这里,我们忽略了网络权值与阈值随机初始化和学习实例数不同对网络输出值的影响。

采用 $6 \times 18 \times 1$ 网络结构预测 1997 年的运量为 1829.25 百万吨,同有关资料公布的 1997 年世界干散货海运量 1829 百万吨比较,预测的精度很高。

(4) 多输入多输出 BP 网络的并行推理

采用多输入单输出的 BP 网络,在使用时,对一个输入模式只能得到一个输出结果。若采用多输入多输出的 BP 网络,那么,在使用时,对一个输入模式同时得到多个输出结果,称之为神经网络的并行推理。

在本例中,可以采用 m 个输出节点的 BP 网络来建立预测模型,一次预测能同时获得连续 m 年的 m 个年运量的预测值。当然,对 m 个输出节点的 BP 网络,学习实例的输出模式 $Y_k^* = (y_{1k}^*, y_{2k}^*, \dots, y_{mk}^*)$ 由已知的连续 m 年的年运量 $y_{1k}^*, y_{2k}^*, \dots, y_{mk}^*$ 组成。

(5) 神经网络预测方法与其他方法的比较

采用传统的一元线性回归、指数曲线拟合和对数曲线拟合方法获得的 1991 年~1996 年的各年运量估计值和误差如表 8.7 所示。

表 8.7 三种传统预测方法的运量估计量与误差

年度	实际运量 y_k	一元线性回归		指数曲线拟合		对数曲线拟合	
		估计值 y_k	$y_k - \hat{y}_k$	估计值 y_k	$y_k - \hat{y}_k$	估计值 y_k	$y_k - \hat{y}_k$
1991	1622	1629.36	7.36	1625.73	3.73	1637.18	15.18
1992	1611	1666.79	55.79	1664.42	53.42	1670.03	59.03
1993	1615	1679.11	64.11	1677.36	62.36	1680.44	65.44
1994	1685	1703.79	18.79	1703.58	18.58	1700.71	15.71
1995	1789	1728.02	-60.98	1729.71	-59.29	1719.93	-69.07
1996	1790	1755.24	-34.76	1759.56	-30.44	1740.75	-49.25

比较表 8.6 与表 8.7 的对应误差, 可见 $6 \times 18 \times 1$ 和 $7 \times 21 \times 1$ 这两种 BP 网络的拟合情况是很好的, 拟合精度大大超过传统的预测方法。

8.4.4 模糊逻辑与神经网络的结合

模糊逻辑和神经网络虽然都可用于知识表示与知识推理, 但是, 两者的知识表示方式与推理方法有明显的区别。

模糊逻辑要求把使用的知识以模糊命题、模糊规则的形式显式地表示在知识库中, 模糊命题和模糊规则是模糊概念的一种逻辑表示形式。模糊知识只能通过领域专家提供或由其他途径获取, 模糊逻辑系统本身不具备学习能力。神经网络以学习实例集的形式隐含所需要的知识, 学习实例是以确定的输入输出模式表示的数值集合。神经网络具备学习能力, 经过学习后获得的知识的表示形式是网络的互连结构及其相应的邻接权矩阵。

模糊逻辑的推理是以模糊匹配和模糊集的运算方法为基础, 得出的推理结论是模糊概念之间的模糊逻辑关系。神经网络的推理是以网络的非线性数值运算为基础, 得出的推理结论也是数值的, 需要通过对输入输出模式的解释才能得出相应的逻辑结论。

虽然模糊逻辑与神经网络从概念到方法都不相同, 可以认为模糊逻辑是模仿人脑的逻辑思维, 而神经网络是模仿人脑的结构来映射输入特征与输出结论的非线性关系。但是, 模糊逻辑与神经网络都是为了表示与处理不确定性知识而引入的方法, 而且都是一种无模型的估计器。因此, 在不确定知识的处理中, 模糊逻辑与神经网络可以结合起来, 各取所长, 互相补充。

模糊逻辑与神经网络的结合方式可以多种多样, 采用什么结合方式, 应根据实际需要来确定。这里, 我们讨论两种结合应用。

1. 应用模糊逻辑获取神经网络学习实例和结论模糊化

神经网络学习实例的输入模式和输出模式都是数值形式。在许多实际应用中, 直接获取

数值化的学习实例是困难的。若允许领域专家给出基于模糊概念的学习实例,则容易得多。可以应用模糊逻辑把基于模糊概念的学习实例数值化,然后提供给网络学习。同样可把神经网络推理得出的数值化结论通过模糊化解释得出模糊逻辑结论。

我们用下述设备采购专家系统为例来说明。

例 8.3 设计一个设备采购专家系统,由专家对各供货商可提供的同型号设备进行模糊评价,从而为用户选择哪一家供货商的设备提供决策意见。

设选择供货商的某种设备的评价标准为:

$$U=\{u_1, u_2, u_3, u_4, u_5, u_6\}$$

其中: u_1 为设备质量

u_2 为设备售价

u_3 为设备交货运费

u_4 为设备备件供应情况

u_5 为设备售后服务与维护费用

u_6 为供货商交货信誉

由专家对设备按上述标准进行模糊评价,评价等级为:好(v_1)、较好(v_2)、一般(v_3)和差(v_4),组成论域:

$$V=\{v_1, v_2, v_3, v_4\}$$

根据多名专家对某供货商的某设备的模糊评价,可得出对该设备在 $U \times V$ 上的模糊关系:

$$R=[\mu_{ij}]$$

其中, $\mu_{ij} \in [0,1]$ 为多名专家对该供货商的该设备按标准 u_i 的评价等级为 v_j 的百分比。

根据不同的设备在评价标准的侧重不同,可对不同的设备制定出不同的标准权因子,6个标准权因子组成该设备的权向量:

$$A=\{a_1, a_2, a_3, a_4, a_5, a_6\}$$

由模糊变换,可得出对该供货商的该设备的模糊综合评价向量:

$$B=A \circ R=\{b_1, b_2, b_3, b_4\}$$

其中, $b_j \in [0,1]$ 的含义为综合评价该设备等级为 v_j 的百分比。

至此,利用模糊逻辑,把多名专家选择供货商 k 的设备按多条标准的模糊评价变换为数值的综合评价向量 B_k 。建立一个4输入单输出的BP网络, B_k 就是BP网络的学习实例 k 的输入向量:

$$X_k=B_k=\{b_{1k}, b_{2k}, b_{3k}, b_{4k}\}$$

学习实例 k 的期望输出值 y^*_k 是专家根据自己的经验对供货商 k 提供的设备是否采购的模糊语言修饰词经过模糊逻辑处理后的值,模糊语言修饰词的取值可参考表 8.2。

对 N 家供货商提供的同一种型号的设备的模糊评价,经上述模糊变换后得到数值化的 N 个学习实例 $(X_k, y^*_k), k=1, 2, \dots, N$, 用以训练网络,就可得对该型号设备选择供货商的专家系统。

专家系统建成后,只需根据多家用户对任何一家供货商提供的设备的综合评价意见,计

算出 BP 网络的输入向量:

$$X=\{x_1, x_2, x_3, x_4\}$$

其中 x_1, x_2, x_3, x_4 分别是用户认为该供货商提供的设备是好、较好、一般和差的百分比。由 BP 网络正向传播计算得出计算输出值 y , 就可决定是否对该供货商提供的该设备实行集中采购的决策意见。也可对 y 经模糊化解释, 得出用模糊语言修饰词表示的模糊结论。

为了可对多家供货商提供的同一型号设备选择供货商, 需要根据多家用户对这多家供货商的信任程度, 分别给出综合评价意见, 然后根据各供货商对应的网络输出值, 从中选择供货商。

由于不同的设备可以有不同的评价标准权向量 A , 甚至可以有不同的评价标准 U , 因此, 不同的设备的 BP 网络的结构和权值分布不同, 能对多种设备进行选择供货商的辅助决策专家系统由相应的多个 BP 网络组成。这正反映了专家系统对采购不同设备所需要记忆的知识不相同。

2. 应用神经网络获取模糊规则

在可信度推理和模糊推理中, 规则的表示形式为:

$$\text{if } e_1(y_1) \wedge e_2(y_2) \wedge \cdots \wedge e_n(y_n) \text{ then } h \text{ (CF, } \lambda \text{)}$$

其中 $y_1, y_2, \cdots, y_n$ 分别是规则前提中各子条件的可信度或隶属度。通过可信度推理或模糊推理可得出结论 h 及其可信度或隶属度 y 。

如果我们能够通过领域专家直接给出结论 h 的可信度或隶属度 y , 那么只需要通过规则匹配和冲突消解找到一条可用规则, 直接得出推理的结论 h 和结论的可信度或隶属度 y , 无需进行繁琐的不确定性测度的计算。但是, 如果仅通过个别领域专家就直接给出 $y_1, y_2, \cdots, y_n$ 和 y , 那么就使规则具有个别专家的主观性。我们可以把多名领域专家对同一条规则的主观意见应用神经网络处理后再表示出来, 从而避免个别专家的主观随意性。

我们用下述一个医疗诊断专家系统为例来进一步说明。

例 8.4 设计一个医疗诊断专家系统, 其中, $x_1, x_2, \cdots, x_n$ 是病人的症状, 症状的模糊程度表示为 $cf_1, cf_2, \cdots, cf_n$; $h_1, h_2, \cdots, h_m$ 为诊断的疾病名, 根据症状及其模糊程度确定患有疾病 h_j 的模糊程度为 CF_j 。

建立一个 n 输入 m 输出 BP 网络。根据多名专家对 N 个名病例进行诊断的主观意见形成 BP 网络的 N 个学习实例 (X_k, Y_k^*) , $k=1, 2, \cdots, N$ 。其中输入模式 X_k 为病人 k 的 n 种症状的模糊程度:

$$X_k=\{cf_{1k}, cf_{2k}, \cdots, cf_{nk}\}$$

输出模式 Y_k^* 为病人 k 可能患有 m 种疾病的模糊程度:

$$Y_k^*=\{CF_{1k}, CF_{2k}, \cdots, CF_{mk}\}$$

经过学习后的 BP 网络就记忆了诊断这 m 种疾病的有关 n 种症状的模糊知识。

知识库的表示与推理可以有下述两种方式:

1) 基于神经网络的方式。知识库表示就是经过学习后的 BP 网络, 推理机为 BP 网络的正向传播计算。

2) 基于模糊逻辑的方式。由学习后的 BP 网络构造出一组模糊规则, 组织到知识库中, 推理机通过模糊匹配来选择规则, 从而得出模糊结论。由此构造出的是一个高性能的模糊逻辑系统, 因为经过神经网络学习后得到的模糊规则是“集思广益”得出的, 避免了各规则由个别专家独立确定时的主观随意性; 而且能有效地避免规则的冗余及不一致等问题。

实际上, 模糊规则库的建立是模糊知识获取问题。知识获取有两种方法, 一是通过领域专家直接获取, 二是通过学习获取。神经网络的自学习能力就可以用于模糊知识的自动获取。

模糊逻辑与神经网络的结合发展很快, 把模糊逻辑用于神经网络的技术称为神经模糊技术, 称与之相应的神经网络为模糊神经网络。目前, 神经模糊技术研究的主要问题有:

- 1) 研究模糊逻辑与神经网络的对应关系。
- 2) 神经网络的输入是模糊量。
- 3) 神经网络的节点之间的连接权是模糊量。
- 4) 神经网络输入与连接权都是模糊量。
- 5) 研究模糊神经网络的学习算法。
- 6) 应用模糊逻辑研究加快神经网络学习速度的方法, 用以构造高性能的模糊逻辑系统。

可以预见, 随着神经模糊理论与技术的深入发展, 模糊逻辑与神经网络融于一体, 必将推动人工智能研究与应用进入新的发展时期。因为人们预期, 人类大脑结构和思维功能的模拟将是: 大脑神经网络结构模拟与信息模糊思维功能模拟的有机结合。

习题八

8.1 何谓人工神经网络? 人工神经网络的互连结构有哪几种基本类型? 简述这几种类型在网络结构上的基本特征。

8.2 简述感知器的学习过程, 给出感知器学习算法的流程框图。

8.3 为什么说人工神经网络是一个非线性映射系统? 如果 BP 网络中的所有节点的传递函数都为线性函数, BP 网络还是一个非线性映射系统吗? 为什么?

8.4 Hopfield 神经网络分为哪两类? 两者的区别是什么?

8.5 何谓有教师学习? 何谓无教师学习? 试分别举例说明神经网络的这两类学习方法。

8.6 BP 学习算法有哪些主要不足? 试举一例说明可采取的改进方法。

8.7 为什么说 BP 学习算法的收敛速度由权值调整量 Δw 决定? Δw 的欠调和超调对学习过程有何影响?

8.8 简述神经网络专家系统的知识表示形式和推理机制, 试举一例说明。

8.9 简述把几个基于神经网络的专家系统组合成一个规模更大的专家系统的基本方法。为什么说, 组合基于神经网络的知识库比组合基于规则的知识库的可靠性更好?

8.10 神经网络的并行推理的含义是什么? 基于规则的推理能实现这种并行推理吗? 为什么?

8.11 应用 BP 学习算法, 编制 C 语言程序, 建立例 8.1 要求的模糊分类器。BP 网络采用 $2 \times 3 \times 3 \times 2$ 的结构, 学习实例由表 8.1 给出。求出稳定权值分布矩阵和算法收敛的迭代次数。

8.12 应用 BP 学习算法, 编制 C 语言程序, 建立例 8.2 要求的时序预测模型。BP 网络采用 $5 \times 15 \times 1$ 的结构, 学习实例由表 8.4 给出。求出稳定的权值分布矩阵和算法收敛的迭代次数。并把学习实例的网络输出值与表 8.5 中的相应值进行比较, 由此说明什么问题?

8.13 应用 BP 学习算法, 编制 C 语言程序, 建立例 7.8 要求的时序预测模型。学习实例由表 7.2 给出。求稳定的权值分布矩阵和算法收敛的迭代次数。并把神经网络预测模型的预测值与遗传算法 预测模型的预测值进行预测精度比较。

8.14 模糊逻辑与神经网络的知识表示与推理方法有何主要的区别?

8.15 根据自己的体会, 试举一例说明在专家系统的设计中, 模糊逻辑与神经网络结合应用的方法。